cadence®

Genus Command Reference

Product Version 25.13
April 2026

© 2026 Cadence Design Systems, Inc.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual 4
Additional References 4
4
Reporting Problems or Errors in Manuals 5
5
Customer Support 5
5
Cadence ASK 5
Other Support Offerings 6
Supported User Interfaces 7
7
Messages 8
8
Man Pages 8
Command-Line Help 9
9
Getting Attribute Help 10
Getting the Syntax for a Command 10
Searching For Commands When You Are Unsure of the Name 10
Documentation Conventions 11
11
Text Command Syntax 11
1 12
General 12
? 12
alias 14
apropos 15
check_ccr 16
16
clear 17
clear_redline_terminal 17
17

date 17
eval_common 17
eval_cpf 18
18
exit 18
get_constant 19
19
get_db 19
get_liberty_attribute 27
27
get_license 28
get_license_list 29
get_license_names 29
get_license_version 30
get_read_files 31
get_scan_elements 32
help 32
include 34
is_attribute 35
is_collection_object 36
36
is_common_ui_mode 37
license_reset_wait 37
37
lminus 37
37
man 38
more 39
preserve_analog_pins_from_undriven_handling 39
quit 40
redirect 40
release_license 42
reset_db 43
43
resume 46
run_embedded_script 46
search_doc 47

47
set_db 47
set_multi_cpu_usage 50
set_proc_verbose 51
51
source 52
52
start_preserve_for_auto_ungroup_constraints 52
stop_preserve_for_auto_ungroup_constraints 53
stop_suspend 53
stringify 53
suppress_messages 53
suspend 54
test_super_thread_servers 55
unsuppress_msg 56
set_latch_loop_disable 57
2 58
Advanced Low Power Synthesis 58
apply_cglar 59
59
apply_power_intent 61
check_cpf 63
63
check_library 65
65
check_power_intent 67
67
check_power_structure 70
70
commit_dft_power_intent 72
72
commit_module_model 73
73
commit_power_intent 74
74
compare_crossing 75

75
copy_library_domain_attributes 76
76
cpi_insert_auto_ls 77
77
create_library_domain 78
78
define_design_partition 79
eval_1801 80
80
optimize_iso_ls 81
81
partition_design 81
read_power_intent 82
reset_power_intent 84
84
set_cglar_specification 85
write_power_intent 87
3 90
Analysis and Reporting 90
analyze_library_corners 91
91
check_design 92
check_timing_intent 96
96
compare_sdc 97
97
create_timing_bin 97
get_clock_ports 99
get_fanin 99
get_fanout 100
get_socv_reporting_nsigma_multiplier 101
101
report_analysis_views 102
report_annotated_check 102
102

report_area 103
report_auto_ungroup_hierarchies 106
report_boundary_opt 106
report_case_analysis 106
106
report_cell_delay_calculation 107
report_chains_longest_wires 108
108
report_chains_total_wirelength 109
109
report_chains_wires 110
110
report_clock_gates 111
report_clock_gating_quality 115
115
report_clock_groups 117
117
report_clock_tree_structure 118
118
report_clocks 118
report_congestion 120
report_core_wrapper_cell 121
report_cwd_check 123
report_delay_calculation 124
124
report_design_rules 124
report_dft_abstract_models 125
125
report_dft_analyzed_test_points 126
report_dft_trace_back 126
126
report_dft_violations 128
report_dont_touch 130
130
report_dp 130
report_flow 133
report_gates 134

report_hierarchy 135
report_instance 136
report_logic_levels_histogram 137
report_loop 137
report_memory 138
report_memory_cells 138
report_messages 139
report_metric 142
report_min_pulse_width 143
143
report_mode 143
143
report_module 143
143
report_multibit_inferencing 144
Reasons for Not Merging Instances 145
report_net_cap_calculation 146
report_net_delay_calculation 147
report_net_res_calculation 148
report_nets 148
report_obj 149
149
report_opcg_clock_domain_info 150
report_opcg_equivalents 151
report_ple 152
report_port 152
report_power 152
report_power_intent 155
report_power_intent_instances 157
157
report_prototype 159
report_qor 162
report_runtime 162
162
report_scan_chains 163
report_scan_compressibility 164
report_scan_compression_logic 165

165
report_scan_registers 166
report_scan_setup 168
report_sequential 172
report_signoff_timing 173
report_slew_calculation 174
report_summary 174
report_test_power 175
report_timing 177
report_timing_derate 186
report_ungroup_modules 187
187
report_units 188
188
report_utilization 189
report_yield 189
save_snapshot 190
set_activity 190
190
set_socv_reporting_nsigma_multiplier 191
191
time_info 192
report_clock_gate_decloning 193
report_instance_library 194
4 195
Chipwaredev 195
check_cwd_object 195
create_component 196
create_component_binding 197
create_component_implementation 198
create_component_library 199
create_component_operator 200
create_component_package 201
create_component_parameter 202
create_component_pin 203
create_cwd_check 205

5 208
Constraints 208
check_constraints 208
create_derived_design 209
209
define_cost_group 211
path_adjust 212
path_group 216
216
phys_enable_ocv 219
219
phys_reset_ocv 219
219
report_constraint 220
220
reset_path_adjust 226
226
reset_path_exception 227
set_path_adjust 230
230
set_path_adjust_group 232
232
set_path_specification 234
set_socv_rc_variation_factor 239
239
6 241
Customization 241
create_command_help 241
241
create_msg 242
242
define_attribute 243
243
define_msg 245
245
define_proc 247

parse_options 249
249
7 253
Design for Test 253
add_analyzed_test_points 254
add_core_wrapper_cell 257
add_hard_repair 263
add_jtag_boundary_scan 266
add_jtag_macro 269
269
add_lbist 272
272
add_lockup_element 277
277
add_los_pipeline 278
278
add_opcg 279
279
add_opcg_hold_mux 280
add_pmbist 281
add_pmbist_access_method 285
add_ptam 286
add_scan_power_gating 288
288
add_shadow_logic 290
290
add_shift_register_test_points 292
292
add_test_compression 293
293
add_test_point 295
295
add_user_test_point 298
298
add_wir_signal_bits 300
300

add_wrapper_cell_aware_clock_gating 301
analyze_atpg_testability 302
302
analyze_initmode_processing 304
analyze_scan_compressibility 306
analyze_test_points 313
check_atpg_rules 315
315
check_dft_pad_cfg 316
check_dft_rules 316
check_dft_setup 321
321
compress_scan_chains 321
321
connect_dft_hier_test_cores 330
330
connect_opcg_segments 330
330
connect_scan_chains 331
331
connect_serial_scan_chains 334
convert_to_opcg_scan 336
convert_to_scan 336
336
create_chains_database 337
337
define_dft_cfg_mode 338
define_hier_test_core_concat_order 340
340
define_hier_test_scan_mapping 340
340
define_jtag_boundary_scan_segment 341
define_jtag_instruction 342
342
define_jtag_instruction_register 345
345
define_jtag_macro 346

346
define_jtag_tap_port 349
349
define_mbist_clock 350
define_opcg_domain 352
352
define_opcg_domain_macro_parameters 354
354
define_opcg_mode 356
356
define_opcg_osc_source 357
357
define_opcg_trigger 358
358
define_pmbist_direct_access 360
360
define_pmbist_port 362
362
define_scan_abstract_segment 363
363
define_scan_chain 367
367
define_scan_fixed_segment 370
370
define_scan_floating_segment 371
371
define_scan_preserved_segment 372
define_shift_register_segment 374
374
define_test_bus_port 375
define_test_clock 378
378
define_test_signal 381
fix_dft_violations 389
389
fix_pad_cfg 391
391

fix_scan_path_inversions 392
identify_domain_crossing_pins_for_cgic_and_scan_abstracts 392
392
identify_multibit_cell_abstract_scan_segments 393
identify_shared_wrapper_cells_in_design 394
394
identify_shift_register_scan_segments 394
394
identify_test_mode_registers 395
395
map_dft_unmapped_logic 398
398
map_pmbist_cgc_to_cgic 398
map_pmbist_ffn 399
map_pmbist_ffsync 399
map_pmbist_map2mux 400
400
prepare_for_test_point_analysis 400
read_dft_abstract_model 400
read_dft_jtag_boundary_file 402
read_dft_pmbist_interface_files 403
read_pmbist_memory_view 403
reset_opcg_equivalent 405
405
reset_scan_equivalent 406
406
set_compatible_test_clocks 406
406
set_opcg_equivalent 407
407
set_scan_equivalent 408
408
update_scan_chains 409
409
write_dft_abstract_model 410
410
write_dft_atpg 412

412
write_dft_atpg_other_vendor_files 417
417
write_dft_bsdl 418
418
write_dft_compression_macro 420
write_dft_compression_test_points 425
write_dft_constraints 427
write_dft_deterministic_test_points 427
write_dft_jtag_boundary_file 429
429
write_dft_jtag_boundary_verification 430
430
write_dft_lbist_macro 432
432
write_dft_lbist_test_points 435
write_dft_lbist_testbench 437
437
write_dft_pmbist_interface_files 438
438
write_dft_pmbist_testbench 440
write_dft_rtl_model 443
443
write_dft_rtl_opcg_domain_macro 444
444
write_dft_rtl_opcg_trigger_macro 445
445
write_dft_rtl_unified_lbist_macro 445
write_memory_physical_map 446
write_modus_ctl_tp_file 446
write_modus_no_tp_file 447
447
write_scandef 447
447
8 450
Design Manipulation 450

add_tieoffs 450
bitblast_all_ports 452
bitblast_ports 454
change_link 454
connect 457
create_design 459
459
create_hinst 460
460
create_hnet 461
461
create_hport 462
create_hport_bus 463
create_inst 465
create_module 467
create_port 467
create_port_bus 468
create_primitive 470
470
dedicate_module 472
delete_obj 473
delete_unloaded_undriven 475
475
disconnect 476
flatten_complex_ports 477
477
group 478
478
remove_cdn_loop_breaker 479
rename_obj 480
reset_design 481
481
ungroup 483
uniquify 485
update_names 486
9 494

Elaboration and Synthesis 494
add_assign_buffer_options 494
add_buffer 498
create_base_cell_set 499
499
delete_buffer_tree 500
500
elaborate 501
get_remove_assign_options 505
insert_buffer 506
506
merge_to_multibit_cells 507
remove_assigns_without_opt 510
remove_clock_reconvergence 514
syn_generic 515
syn_map 518
syn_opt 521
10 524
Flowkit 524
check_flow 524
524
create_flow 526
526
create_flow_step 527
527
define_feature 529
529
delete_flow 531
531
delete_flow_config 531
531
delete_flow_step 532
532
edit_flow 533
end_flow 534
534

get_feature 534
534
get_flow_config 535
535
is_feature 537
537
is_flow 537
read_flow 538
reset_flow 539
539
run_flow 540
540
schedule_flow 542
542
set_feature 543
543
set_flow_config 544
544
set_flowkit_read_db_args 546
set_flowkit_write_db_args 546
write_flow 547
write_flow_template 548
11 550
GUI Text 550
deselect_obj 550
550
gui_clear_pv 551
551
gui_clear_sv 551
gui_configure_pi_schematic 552
552
gui_hide 552
gui_highlight_hier_instances_pv 553
553
gui_highlight_pv 553
553

gui_highlight_sv 554
554
gui_legend 556
gui_open_schematic 557
557
gui_pv_airline_add 558
gui_pv_airline_delete 559
gui_pv_airline_display 559
gui_pv_airline_raw_add 560
gui_pv_connectivity_airlines 561
gui_pv_display_collection 561
gui_pv_draw_net 561
561
gui_pv_selection 562
gui_pv_steiner 562
562
gui_raise 563
gui_redo_pv 563
gui_redraw_pv 563
563
gui_reset 563
gui_select_highlighted_pv 564
564
gui_selection 564
gui_show 564
gui_snapshot_pv 565
gui_snapshot_sv 565
gui_sv_get_instance 566
gui_sv_load 566
gui_undo_pv 566
566
gui_update 567
gui_zoom_fit_pv 567
gui_zoom_fit_sv 567
567
gui_zoom_pv 567
567

select_obj 568
568
12 569
Hierarchical Flow 569
assemble_design 569
generate_ilm 571
read_ilm 572
read_ilm_from_files 574
13 577
Incremental Synthesis 577
design_has_cache_replaced_content 577
report_incremental_synthesis 578
reset_incrsyn_preserves 579
write_incremental_synthesis_cache 580
14 581
Input and Output 581
decrypt 583
encrypt 584
export_oa_db 585
585
hdl_set_vlog_file_extension 587
587
import_oa_db 588
588
read_clock_tree_spec 589
589
read_db 590
590
read_dfm 591
read_hdl 592
read_libs 596
read_module_model 599
599
read_netlist 600
read_qrc 602

read_sdc 602
Unsupported Constraints 604
read_sdf 605
605
split_db 606
write_clp_script 607
write_db 609
write_design 612
write_do_lec 613
write_hdl 620
write_mmmc 623
write_module_model 625
625
write_name_mapping 626
write_netlist 628
628
write_sdc 631
631
write_sdf 633
write_snapshot 640
write_sv_wrapper 642
write_template 644
644
write_tempus 647
647
15 650
Joules 650
get_lib_domains 650
650
report_sdb_annotation 651
651
write_icgc_monitor 653
653
16 655
Low Power Synthesis 655
add_clock_gates_obs 655

add_clock_gates_test_connection 658
658
declone_clock_gate 659
delete_clock_gate 661
incr_read_1801 663
663
merge_clock_gate 663
read_saif 664
read_tcf 668
read_vcd 671
rebalance_clock_gating 675
share_clock_gate 677
split_clock_gate 678
state_retention 680
update_clock_gate 680
write_saif 681
write_tcf 683
write_top_power_intent 685
17 687
Machine Learning Commands 687
init_ml 687
687
write_ml_design 689
689
write_ml_timing_path 690
690
18 692
Metrics 692
create_metric_page 692
create_metric_table 693
create_metric_table_heading 695
create_snapshot 695
create_testcase 697
define_metric 698
delete_metric 700
delete_metric_header 701

701
delete_metric_page 701
delete_metric_run 702
delete_metric_table 702
delete_metric_table_heading 703
get_metric 703
get_metric_alias 704
704
get_metric_config 705
get_metric_definition 705
get_metric_header 706
706
pop_snapshot_stack 706
706
push_snapshot_stack 707
read_metric 708
read_metric_html_package 708
reset_metric_config 709
reset_metric_run_id 709
run_metric_category 709
run_parallel_commands 710
710
set_metric 712
712
set_metric_alias 713
713
set_metric_header 713
update_metric 714
update_metric_definition 715
write_metric 717
read_metric_config 718
write_metric_config 718
19 720
Multi-Mode Multi-Corner 720
all_constraint_modes 720
720

create_analysis_view 721
721
create_constraint_mode 723
723
create_delay_corner 724
create_library_set 729
729
create_net 733
733
create_opcond 734
734
create_rc_corner 736
736
create_timing_condition 740
740
get_interactive_constraint_modes 741
741
init_design 741
741
read_mmmc 743
set_analysis_view 745
set_interactive_constraint_modes 748
748
update_analysis_view 749
749
update_constraint_mode 751
751
update_delay_corner 753
update_library_set 758
758
update_opcond 760
760
update_rc_corner 762
762
update_timing_condition 765
765
update_lef_macro 767

20 768
Navigation 768
get_logical_name 768
get_object_type 768
set_top_module 769
vbasename 769
vcd 770
vdirname 772
vdirs 773
vfind 774
vls 777
vname 781
vpopd 781
vpushd 782
vpwd 784
21 785
Physical 785
check_floorplan 786
check_pbs_mim_module 787
check_placement 787
convert_polygon_to_boxes 788
create_clip 788
788
create_floorplan 790
create_group 792
create_place_blockage 795
create_place_halo 799
create_ple_model 801
create_property_alias 802
802
create_region 803
create_route_blockage 804
create_route_halo 808
create_route_rule 810
810
create_route_type 812

812
create_row 815
create_single_row 817
817
create_track 818
cut_power_domain_by_overlaps 819
def_move 820
delete_rows 821
821
duplicate_register 822
floorplan_resize_blockage 823
floorplan_resize_boundary 824
get_computed_shapes 825
get_obj_in_area 826
826
modify_power_domain_attr 827
move_blockage 829
move_boundary 830
move_instance 830
move_obj 831
831
move_port 832
place_dft 833
place_dft_sequentials 834
834
predict_floorplan 835
835
read_clip 836
836
read_congestion_map 838
838
read_def 839
read_encounter 844
read_parasitics 845
read_physical 845
read_sdp_file 848
set_cell_padding 849

specify_cell_pad 850
update_congestion_map 851
update_group 852
852
update_inst 853
853
update_instance_data 854
update_power_domain 856
write_congestion_map 858
write_def 858
write_lib_lef 861
write_parasitics 863
write_preserves 864
864
write_reports 864
write_sdp_file 866
22 868
SDC Commands 868
add_to_collection 870
870
all_clocks 870
870
all_connected 871
871
all_fanin 872
872
all_fanout 875
875
all_inputs 877
877
all_instances 878
878
all_outputs 879
879
all_registers 880
880

append_to_collection 881
881
apply_clock_gating_network_latency_table 882
compare_collections 883
883
copy_collection 884
884
create_clock 884
884
create_generated_clock 885
current_design 888
888
current_design_name 889
889
current_instance 890
890
filter_collection 891
891
foreach_in_collection 892
892
get_arcs 893
get_cells 894
894
get_clock_gating_history_dictionary_entry 897
get_clocks 897
897
get_designs 899
899
get_equivalent_cells 901
get_generated_clocks 902
902
get_leaf_cells 903
903
get_leaf_nets 904
904
get_leaf_pins 904
get_lib_arcs 905

905
get_lib_cells 907
907
get_lib_pins 909
909
get_libs 911
911
get_nets 912
912
get_object_name 913
913
get_path_groups 914
914
get_pins 915
915
get_ports 918
918
get_references 920
920
getenv 921
group_path 921
index_collection 923
923
list_property 924
924
load_of 924
read_clock_gating_network_latency_table 925
remove_case_analysis 926
remove_clock_gating_check 926
926
remove_clock_latency 927
remove_clock_transition 928
remove_disable_clock_gating_check 929
remove_disable_timing 930
remove_from_collection 931
remove_generated_clock 932
remove_ideal_net 932

remove_ideal_network 933
remove_input_delay 933
remove_output_delay 934
934
report_property 935
935
reset_annotated_check 936
936
reset_annotated_delay 938
938
reset_annotated_transition 939
939
reset_case_analysis 939
reset_clock 940
reset_clock_gating_check 941
reset_clock_gating_network_latency_table 941
reset_clock_groups 942
942
reset_clock_latency 943
reset_clock_sense 943
reset_clock_transition 944
reset_clock_uncertainty 945
945
reset_data_check 946
reset_disable_clock_gating_check 947
reset_disable_timing 948
reset_drive 948
reset_driving_cell 949
reset_generated_clock 949
reset_ideal_latency 950
reset_ideal_network 951
reset_ideal_transition 951
reset_input_delay 951
reset_load 953
reset_max_capacitance 953
reset_max_fanout 954
reset_max_time_borrow 954

954
reset_max_transition 955
reset_min_capacitance 955
reset_min_fanout 956
reset_min_pulse_width 956
reset_min_transition 957
reset_mode 957
957
reset_output_delay 958
reset_path 959
reset_path_group 960
reset_path_group_options 961
reset_resistance 961
reset_sense 962
962
reset_timing_derate 962
reset_wire_load_mode 963
reset_wire_load_model 964
reset_wire_load_selection_group 964
set_active_clocks 965
set_annotated_check 965
965
set_annotated_delay 967
967
set_annotated_transition 969
969
set_case_analysis 970
970
set_clock_exclusivity 971
set_clock_gating_check 972
set_clock_gating_network_latency_table 974
set_clock_groups 975
975
set_clock_latency 977
977
set_clock_sense 979
set_clock_skew 981

981
set_clock_transition 982
982
set_clock_uncertainty 983
set_constraint_mode 985
985
set_data_check 986
Current_Limitations 988
set_disable_clock_gating_check 988
988
set_disable_timing 989
set_dont_touch 990
set_dont_touch_network 990
990
set_dont_use 991
set_drive 991
991
set_driving_cell 993
993
set_equal 995
995
set_false_path 995
set_fanout_load 999
999
set_hierarchy_separator 1000
1000
set_ideal_latency 1001
1001
set_ideal_net 1001
set_ideal_network 1002
1002
set_ideal_transition 1002
set_input_delay 1003
1003
set_input_transition 1005
set_lib_pins 1006
1006

set_load 1007
1007
set_load_unit 1008
1008
set_logic_dc 1009
1009
set_logic_one 1009
1009
set_logic_zero 1010
1010
set_max_capacitance 1011
1011
set_max_delay 1012
set_max_dynamic_power 1015
1015
set_max_fanout 1016
1016
set_max_leakage_power 1017
1017
set_max_time_borrow 1017
set_max_time_borrow 1017
set_max_transition 1018
1018
set_min_capacitance 1020
1020
set_min_delay 1020
set_min_fanout 1022
1022
set_min_pulse_width 1023
set_min_transition 1024
1024
set_mode 1024
1024
set_module_model 1025
1025
set_multicycle_path 1026
1026

set_operating_conditions 1029
1029
set_opposite 1030
1030
set_output_delay 1031
1031
set_path_group_options 1033
1033
set_port_fanout_number 1034
1034
set_resistance 1035
1035
set_sense 1036
1036
set_size_only 1037
1037
set_time_unit 1038
1038
set_timing_derate 1039
set_unconnected 1042
1042
set_units 1043
1043
set_wire_load_mode 1044
1044
set_wire_load_model 1045
set_wire_load_selection_group 1046
1046
sizeof_collection 1048
1048
sort_collection 1049
1049
23 1051
Unified Safety Format 1051
USF File Commands 1051
apply_safety_mechanism 1051

create_esubpart 1053
create_failure_mode 1055
create_safety_mechanism 1057
create_subpart 1059
set_safety_mechanism_rules_dcls 1062
set_safety_mechanism_rules_ser 1067
set_safety_mechanism_rules_tmr 1069
USF Support Commands 1071
check_safety_mechanism 1071
read_safety_tmr 1073
read_usf 1075
reset_usf 1076
usf_version 1077
write_usf 1077
24 1079
Command Name Mapping 1079

About This Manual

This manual provides a concise reference of the commands available to the user when using the Genus software with the Stylus Common user interface.

This manual describes each command available within the Genus shell with its command options.

Additional References

The following sources are helpful references, but are not included with the product documentation:

- TclTutor, a computer aided instruction package for learning the Tcl language: http://www.msen.com/~clif/TclTutor.html.
- TCL Reference, *Tcl and the Tk Toolkit,* John K. Ousterhout, Addison-Wesley Publishing Company
- *Practical Programming in Tcl and Tk*, Brent Welch and Ken Jones
- IEEE Standard Hardware Description Language Based on the Verilog Hardware Description Language (IEEE Std.1364-1995)
- IEEE Standard Hardware Description Language Based on the Verilog Hardware Description Language (IEEE Std. 1364-2005)
- IEEE Standard for SystemVerilog--Unified Hardware Design, Specification, and Verification Language (IEEE STD 1800-2009)
- IEEE Standard VHDL Language Reference Manual (IEEE Std. 1076-1987)
- IEEE Standard VHDL Language Reference Manual (IEEE Std. 1076-1993)
- IEEE Standard VHDL Language Reference Manual (IEEE Std. 1076-2008)

For information on purchasing IEEE specifications go to http://shop.ieee.org/store/ and click on *Publications & Standards.*

Reporting Problems or Errors in Manuals

The Cadence® Doc Assistant online documentation, lets you view, search, and print Cadence product documentation. You can access Cadence Doc Assistant by typing `cda` from your Cadence tools hierarchy.

Contact Cadence Customer Support to file a CCR if you find:

- An error in the manual
- An omission of information in a manual
- A problem using the Cadence Doc Assistant documentation system

Customer Support

Cadence offers live and online support, as well as customer education and training programs.

Cadence ASK

The Cadence® ASK (Application Support & Knowledge) website offers answers to your most common technical questions. It lets you search more than 40,000 FAQs, notifications, software updates, and technical solutions documents that give you step-by-step instructions on how to solve known problems. It also gives you product-specific e-mail notifications, software updates, case tracking, up-to-date release information, full site search capabilities, software update ordering, and much more. For more information on Cadence ASK go to http://support.cadence.com

Other Support Offerings

- **Support centers**—Provide live customer support from Cadence experts who can answer many questions related to products and platforms.
- **Software downloads**—Provide you with the latest versions of Cadence products.
- **University software program support**—Provides you with the latest information to answer your technical questions.
- **Training Offerings**—Cadence offers the following training courses for Genus users:
  - Basic Static Timing Analysis
  - Advanced Synthesis with Genus Stylus Common UI
  - Fundamentals of IEEE 1801 Low-Power Specification Format
  - Genus Low-Power Synthesis Flow with IEEE 1801
  - Genus Synthesis Solution with Stylus Common UI
  - Genus Physical Synthesis Flow
  - Joules Power Calculator
  - Low-Power Synthesis Flow with Genus Stylus Common UI
  - Test Synthesis with Genus Stylus Common UI

  The Online Training is free through the **ASK** portal for the registered Cadence users. For further information on the training courses available in your region, visit Cadence Training or write to '*training_enroll@cadence.com*'.

  > The links in this section open in a new browser.

- **Digital Badge Exams—**Cadence offers **Digital Badge** exams for all the Genus courses for our customers and University students. The exams can be taken from Training Courses (cadence.com).
  Upon passing the exams, you get certificates (verified by https://info.credly.com/), which can then be shared on your LinkedIn profiles, Facebook, and Twitter. For more Information, refer to 'Become Cadence Certified'.
- **Video Library**
  Several videos are available on the support website: Genus: Video Library

For more information on the support offerings go to http://www.cadence.com/support

Supported User Interfaces

Genus supports the following user interfaces:

- **Unified User Interface.** Genus, Innovus and Tempus offer a fully unified Tcl scripting language and GUI environment. This unified user interface (also referred to as Stylus common UI) streamlines flow development and improves productivity of multi-tool users.
  When you start Genus, you will by default start with the common UI. You will see the following prompt:

  ```
  genus:root: 1>
  ```

- **Legacy User Interface.** Genus can also operate in legacy mode which supports RTL Compiler commands/attributes and scripting.
  To start Genus with legacy UI, you can
  - Start the tool with legacy UI as follows:

    ```
    %genus -legacy_ui -files script

    ....

    legacy_genus:/>
    ```

  - Switch to legacy UI if you started the tool with the default common UI.

    ```
    %genus

    genus:root: 1> set_db common_ui false

    legacy_genus:/>
    ```

    > This document provides information specific to the Stylus Common user interface.

Messages

- You can get detailed information for each message issued in your current Genus run using the `report_messages` command.

  ```
  @genus:root: number> report_messages
  ```

  The report also includes a summary of how many times each message was issued.
- You can also get specific information about a message.
  For example, to get more information about the `TUI-613` message, you can type the following command:

  ```
  @genus:root: number> vls -a TUI-613 message:TUI/TUI-613 (message) Attributes:
  base_name = TUI-613 count = 0 escaped_name = TUI/TUI-613 help = The
  user_speed_grade is only applicable to datapath subdesigns. id = 613 name =
  TUI/TUI-613 obj_type = message print_count = 0 priority = 1 screen_print_count = 0
  severity = Warning type = The attribute is not applicable to the object.
  ```

  You can also use the `help` command:

  ```
  @genus:root: number> help TUI-613 Message: name: TUI/TUI-613 severity: Warning
  type: The attribute is not applicable to the object. help: The user_speed_grade is
  only applicable to datapath subdesigns.
  ```

If you do not get the details that you need or do not understand a message, either contact Cadence Customer Support to file a CCR or email the message ID you would like improved to synthesis_pubs@cadence.com

Man Pages

In addition to the Command and Attribute References, you can also access information about the commands and attributes using the man pages in Genus.

To use man pages from UNIX shell:

1. Set your environment to view the correct directory:

   ```
   setenv MANPATH $CDN_SYNTH_ROOT/share/synth/man_common
   ```

2. Access the manpage by either of the following ways:

- Enter the name of the command or attribute that you want. For example:
  - `man check_dft_rules`
  - `man max_output_voltage`
- Specify a section number with `man` command to look for the command or attribute information in the specific section of the on-line manual.
  Commands are in section 1, attributes are in section 2, and messages are in section 3 of the on-line manual. In the absence of section number, `man` will search through sections 1, 2, 3 (in this sequence) and display the first matching manual page.
  This is useful in cases where both commands and attributes exist with the same name. For example:
  - `man 1 xyz`
    will display manhelp for `xyz` command
  - `man 2 xyz`
    will display manhelp for `xyz` attribute

    Refer to man for more information on the `man` command.

Command-Line Help

You can get quick syntax help for commands and attributes at the Genus command-line prompt. There are also enhanced search capabilities so you can more easily search for the command or attribute that you need.

The command syntax representation in this document does not necessarily match the information that you get when you type `help` *`command_name`*. In many cases, the order of the arguments is different. Furthermore, the syntax in this document includes all of the dependencies, where the help information does this only to a certain degree.

If you have any suggestions for improving the command-line help, please e-mail them to synthesis_pubs@cadence.com

Getting Attribute Help

Type the following:

```
@genus:root: number> help attribute_name
```

For example:

```
@genus:root: number> help max_transition
```

This returns the help for the `max_transition` attribute and shows on which object types the attribute can be specified.

Getting the Syntax for a Command

Type the `help` command followed by the command name.

For example:

```
@genus:root: number> help path_group
```

This returns the syntax for the `path_group` command.

Searching For Commands When You Are Unsure of the Name

You can use help to find a command or a Tcl process if you only know part of its name, even as little as one letter. You can type a single letter or sequence of letters and press `Tab` to get a list of all commands and any user-defined Tcl processes that start with that letter(s). For example:

```
@genus:root: number> ad <Tab>
```

This returns the following commands:

```
add_assign_buffer_options        add_clock_gates_obsadd_clock_gates_test_connection
add_opcg_hold_mux...
```

Documentation Conventions

To aid the readers understanding, a consistent formatting style has been used throughout this manual.

- UNIX commands are shown following the `unix>` string.
- Genus commands are shown following the @genus:root: *`number`*> string.

Text Command Syntax

The list below defines the syntax conventions used for the Genus text interface commands.

`literal`	Non-italic words indicate keywords you enter literally. These keywords represent command or option names.
`arguments and options`	Words in italics indicate user-defined arguments or information for which you must substitute a name or a value.
|	Vertical bars (OR-bars) separate possible choices for a single argument.
[ ]	Brackets indicate optional arguments. When used with OR-bars, they enclose a list of choices from which you can choose one.
{ }	Braces indicate that a choice is required from the list of arguments separated by OR-bars. Choose one from the list. `{ argument1 | argument2 | argument3 }`
`{ }`	Braces, used in Tcl commands, indicate that the braces must be typed in.
...	Three dots (...) indicate that you can repeat the previous argument. If the three dots are used with brackets (that is, [`argument`]. . .), you can specify zero or more arguments. If the three dots are used without brackets (`argument`...), you must specify at least one argument.
#	The pound sign precedes comments in command files.

1

General

- ?
- alias
- apropos
- check_ccr
- clear
- clear_redline_terminal
- date
- eval_common
- eval_cpf
- exit
- get_constant
- get_db
- get_liberty_attribute
- get_license
- get_license_list
- get_license_names
- get_license_version
- get_read_files
- get_scan_elements
- help
- include
- is_attribute
- is_collection_object
- is_common_ui_mode
- license_reset_wait
- lminus
- man
- more
- preserve_analog_pins_from_undriven_handling
- quit
- redirect
- release_license
- reset_db
- resume
- run_embedded_script
- search_doc
- set_db
- set_multi_cpu_usage
- set_proc_verbose
- source
- start_preserve_for_auto_ungroup_constraints
- stop_preserve_for_auto_ungroup_constraints
- stop_suspend
- stringify
- suppress_messages
- suspend
- test_super_thread_servers
- unsuppress_msg
- set_latch_loop_disable

?

```
?
[-all |-attribute |-category string
|-command | -message |-obj]]
[-detail | -summary]
[string] [string]
```

Description

Provides help on the specified Genus attributes, commands, objects, and messages. This is an alias for the `help` command.

> You can get help at any stage of the design.

Options and Arguments

`-all`	Prints help for all commands, attributes, objects, and messages.
`-attribute`	Limits the search to attributes only.
`-category` *string*	Filters the attribute information based on the specified category.
`-command`	Limits the search to commands only.
`-detail`	Provides full information for each of the matches.
`-message`	Limits the search to messages only.
`-obj`	Limits the search to objects only.
string string	The first string specifies the command, message ID, object type, or attribute for which you want help.
	The second string specifies a command option or attribute.
`-summary`	Provides a single line description for each of the matches.

Examples

- The following command returns summary help for the attribute `source`.

```
genus:root:> ? -att source
Attributes:
  source(specialnet):   # Specialnet creation source.
  source(pnet):         # Net creation source.
  source(direct_access_function):
                        # Source pin/port of the direct access function.
  source(programmable_direct_access_function):
                        # Source pin/port of the direct access function.

    Type 'help <obj_type> <attribute_name>' to get full description of object attribute
```

- The following command returns help for the attribute `source` that belong to the `phys` category.

```
genus:root:> ? -cat phys source
Attributes:
  source(specialnet):   # Specialnet creation source.
  source(pnet):         # Net creation source.
    Type 'help <obj_type> <attribute_name>' to get full description of object
attribute
```

- The following command returns detailed help for the attribute `source` of type `pnet`.

```
genus:root:> ? pnet source
Attribute: source (on obj_type: pnet)
  Description:            Net creation source.
  category:               phys
  default_value:
  is_computed:            false
  is_settable:            false
  skip_in_db:             false
  data_type:              string
```

- The following commands return help for the message DFT-100.

```
genus:root:> ? -message DFT-100 -detail
```

```
or
genus:root:> ? -message DFT-100
or
genus:root:> ? DFT-100
```

all return:

```
Message
  name:                   DFT/DFT-100
  severity:               Info
  type:                   Added DFT object
  help:
```

- The following command returns a list of attributes for object type `group`.

```
genus:root:> ? -obj group
Obj_type group:
  Attribute:
    base_name: (string):
                        # Object base (i.e. leaf) name.
    obj_type: (object): # Object type.
    name: (string):     # Object name used for get_db/set_db/reset_db queries.
    escaped_name: (string):
                        # Object escaped name.
    def_name: (string): # Original DEF group name.
    region: (string):   # Region associated with group.
    members: (string):  # Members associated with group.
    properties: (string):
                        # Group properties.
    user_created: (bool):
                        # Is group user created.
    design: (design):   # Parent 'design' object.
```

alias

`alias [-args` *`string`*`] alias_name command_name`

Description

Defines an alias for the specified command name.

Options and Arguments

alias_name	Specifies the alias name.
-args *string*	Specifies a list of alias names with their respective commands.
command_name	Specifies the name of the command for which you want to create an alias.
	Command name can also be specified with in " " or { }.

Examples

- The following command creates an alias "pins" for the `get_pins` command:

```
genus:root> alias pins get_pins
pins

genus:root> pins
hpin:top/inst1/in1   hpin:top/inst1/clk   hpin:top/inst1/out1
```

- The following command creates an alias "hcells" for the `get_cells -hierarchical` command:

```
genus:root> alias hcells "get_cells -hierarchical"
hcells

genus:root> hcells
hinst:top/inst1   hinst:top/inst1/inst_empty1   hinst:top/inst1/inst_empty2
hinst:top/inst1/inst_empty3
```

apropos

```
apropos [-skip_help] [-skip_commands]
[-skip_attributes] string
```

Description

Performs a case insensitive wildcard search of commands (including their options) and attributes. The command will even encompass the help text of commands and attributes. The search results will be categorized into the following different sections:

- Commands that match search text
- Commands with help text that match search text
- Commands with options (both option name and option help text) that match search text
- Attributes that match search text
- Attributes with help text and attribute objects that matches search text

Options and Arguments

`-skip_help`	Do not include help text in the search results.
`-skip_commands`	Do not include commands in the search results.
`-skip_attributes`	Do not include attributes in the search results.
string	Specifies the search string

Example

Usage example:

```
genus:root:> apropos clock_gating

Commands matching search string:rebalance_clock_gating

Commands with option text matching search string:rebalance_clock_gating

Attributes matching search string:clock_gating_integrated_cell (base_cell)clock_gating_integrated_cell (lib_cell)is_clock_gating_pin
(hpin)is_clock_gating_pin (pin)is_integrated_clock_gating (base_cell)    :

lp_insert_discrete_clock_gating_logic (root)

Attributes with help text matching search string:skip_statetable_check (root)
```

check_ccr

```
check_ccr string
```

Description

Allows you to check if a fix for the specified CCR is *integrated* in the current or previous build.

- The command returns 1 if a fix for the specified CCR was integrated in this build or a previous build.
- The command returns 0 if a fix for the specified CCR was not integrated in this build or a previous build.

> The command will only return 1 if the CCR was marked Checked_In or Validated at the time of the build.

> You can only check for master CCRs. Duplicate CCRs are not included.

Options and Arguments

string	Specifies the CCR number to be checked.

Example

The following command indicates that the specified CCR was integrated in the build.

```
genus:> check_ccr 17604641
```

clear

```
clear
```

Description

Clears the terminal screen.

clear_redline_terminal

```
clear_redline_terminal
```

Description

Clears the terminal screen.

date

```
date
```

Description

Returns the date and time. This command is equivalent to the UNIX `date` command.

Example

```
genus:> date

Wed May 27 00:15:36 -0700 2015
```

eval_common

```
eval_common script_name
```

Description

Evaluates a Tcl script in the new Common UI mode.

This command is also aliased to `eval_common_ui`.

Options and Arguments

script_name	Specifies the name of the TCL script to evaluate.

eval_cpf

`eval_cpf` *string*

Description

Evaluates a CPF command in the new Common UI mode.

Options and Arguments

string	Specifies the CPF command to evaluate.

exit

`exit [`*code*`]`

Description

Exits from the Genus shell without saving design data.

`Control-c` is a shortcut to the `exit` and `quit` commands.

> When exiting, no design data is saved, so it is important to save the design using the write_hdl command.

Options and Arguments

code	Specifies the exit code.
	The following are the built-in exit codes:
	0 – normal exit
	1 – abnormal exit
	246 – exit when no license server is available
	245 – exit when no license feature is available
	244 – exit when syntax error in script

get_constant

```
get_constant pin|port
[-input] [-output]
```

Description

Queries the design database for the state of the specified pin or the state propagated through the combinational logic cone to that pin. The value returned is 0 for logic 0, 1 for logic 1, and `no_value` for undefined.

Options and Arguments

`-input`	Returns the constant that is set on the input part of the bidirectional pins.
`-output`	Returns the constant that is set on the output part of the bidirectional pins.
pin|port	Specifies the name of the single pin to query for pin state. The pin must be an instance pin or primary I/O port.

Example

The following command queries the design database for the state of the specified pin or the state propagated through the combinational logic cone to that pin:

```
genus:> get_constant port:top/in
```

Related Information

Related command:

set_case_analysis

get_db

```
get_db[-if expression | -expr expression]
[-category string][-dbu][-foreach tcl_body][-index string][-invert][-regexp][-match_hier][-unique] [-computed] [-depth
string]string[pattern]...
```

Description

Retrieves the value of attributes or objects that were set by Genus via the `set_db` command.

You can also use this command to list

- All attributes associated with a given object type
- All objects to which the specified attribute applies.

It returns lists of objects as dual-ported object lists or collections of objects. It also returns lists of object attribute values.

There are two ways to access objects using `get_db`:

1. `get_db` *`object_type pattern`*
2. `get_db` *`object .attribute`*

The string form is normally a useful name preceded by the obj_type, so a base_cell object string form might look like base_cell:and2.

A netlist name will have the current design name included, so an instance named i1/i2 in design top, would look like inst:top/i1/i2.

The get_db command accepts collections as an input but it only returns objects in dual-ported object list form.

For example, you can use the following command to get all the driver-pins of all nets in current_design:

```
get_db current_design .nets.drivers
```

> The current_design is implicitly created by the init_design or the read_db commands.

Frequently used current_design attributes like insts or hinsts (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name.

For example, the following commands gets all instances of the current design with names that start with i1/i2*:

```
get_db current_design .insts i1/i2*
```

```
get_db insts i1/i2
```

Where, insts is a root attribute alias for current_design .insts

Some of the more commonly used aliases from the current_design attributes include: nets, pins, insts, hnets, hpins, and hinsts.

The get_db command supports auto-complete when you press the <Tab> key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:

```
get_db insts .par<Tab>
```

parent partition

```
get_db insts -if {.par<Tab>
```

parent partition

If you press the <Shift+Tab> keys together, the matching attributes are displayed along with their help string, like this:

```
get_db insts .par<Shift+Tab>
```

parent # The parent of the inst, which is either a hinst for an instance within hierarchy or the design object for an instance at the toplevel.

partition # If this inst is a physical black_box, this partition will carry the pin constraints and related data for the blackbox. Otherwise it is empty.

There are two database definition objects: obj_type and attribute. If you want to query all the details for an attribute definition on an obj_type, rather than an attribute value, use the following command:

```
get_db attribute:<obj_type>/<attribute_name> .*
```

> A Dual-Ported Object (DPO) is a Tcl_Obj in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a puts $var or return the value to the shell, it will be converted to its dual string form.

> Wildcards can be used for attribute names with `get_db` command.

Options and Arguments

`-category` *string*	Specifies a category of root attributes to return. This is intended to return all the root attributes associated with one application category. You cannot use a start argument with the -category option, so this option cannot be used for anything but root attributes. The return value is list of root attribute names. For example, the following command can be used to see all the root attributes associated with the category place: get_db -category place The output is as follows: place_design_floorplan_mode place_design_refine_place place_detail_allow_border_pin_abut ... You can view the complete list of categories with the get_db categories command as follows: get_db categories
	It can only be used with root attributes.

chain	Specifies the chain in the form [.attribute_name1 [.attribute_name2]...]. A chain always starts with a ".".
	The specified attribute names are valid for each level of the traversal in the chain.
	For example, the following command returns the names of pins of $my_insts
	get_db $my_insts .pins.name
	If the chain has a * or ? in the attribute name, it will be expanded to all matching attribute names, and the display will show the attribute name and current value for every object. This is intended for interactive usage, and no values are returned to Tcl. Values that may require significant time to compute (like timing data that requires the timing-graph) will only be displayed if -computed is given.
	For example:
	`get_db $my_insts .b*`
	`Object: inst:top/i1`
	`base_cell: base_cell:BUFX2`
	`bbox: {34.32 10.08 38.94 15.12}`
	`Object: inst:top/i2`

`-computed`	Forces get_db to output the values of attributes that may require significant time to compute (like timing data that requires the timing-graph) when browsing many attributes at the same time. This is intended for interactive usage when using a pattern match with * or ? for an attribute name. If only a single attribute name is given, then the value is always returned.
	For example:
	`get_db $my_pins .slack_max*`
	`Object: pin:top/inst$odd/Y`
	`slack_max: NC`
	`slack_max_edge: NC`
	get_db $my_pins .slack_max* -computed #if timing-graph not available yet, it will be computed
	`Object: pin:top/inst$odd/Y`
	`slack_max: 550.12`
	`slack_max_edge: rise`
	`get_db $my_pins .slack_max #single attribute names always return value and force computation if needed`
	`550.12 ...`

`-dbu`	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be returned as an int in database units. Default: If not given, the value returned is a double in microns (or microns**2). For example: - In the following command the return type is a double in microns. get_db $my_inst .location {1.0 1.0} - In the following command the return type is an int in database units. get_db -dbu $my_inst .location {1000 1000}
`-depth` *`string`*	Specifies the hierarchy depth range. It expands each design or hinst object in the input list into a list of design or hinst objects by descending the hinst hierarchy from min to max depth. The default value of min is 0, if not specified. 0 here signifies the level of the starting object. You can specify two values or one in the following format: - {min max} - {max} -- the minimum value is assumed to be 0. This option applies only to the design and hinst objects. It is a recursive descent option, implemented as a pre-processor expansion of the input list of design or hinst objects into design and hinsts objects per the depth min/max level. The expanded list is then processed as any other input list. The -depth expansion happens before any other options or filtering. For example: get_db hinst:top/h1 -depth {0 1} Returns hinst:top/h1, and the first level of hinsts below it.

`-expr` *expression*	Specifies a Tcl style expression. == doesn't require a quoted string =~ allows simple patterns Operators like >, <, <=, >=, &&, ||, and the unary negate operator ! are supported. get_db insts -expr {$obj(.base_cell.name) eq AND2} #all insts that are AND2 This parameter can be used with a chain and pattern like this: get_db $my_insts .pins i1/* -expr {$obj(.base_name) eq A} Here, the i1/* pattern is checked first for pin names that match i1/*, then the pin .base_name is checked if it is equals A. -expr cannot be used at the same time with -if.
`-foreach` *tcl_body*	Specifies a list of Tcl commands to process for each object. The return value is the number of objects that are processed and not the list of objects. This parameter avoids the overhead of creating long lists of objects. The Tcl array $obj() is populated with the attributes of each object when it is visited, and $obj(.) is the object itself. The Tcl commands break and continue also work as expected in a foreach loop (break stops the traversal of any more objects and get_db returns). Unlike inside -if, the tcl_body is strict Tcl. The .attribute names are not substituted, you must use $obj() to access the attribute values instead. If you have a pattern or -if, or -expr usage, then -foreach only processes each $object that matches the pattern, -if, or -expr expression. For example, the following command prints the names of every inst that starts with "abc" and is an AND2 gate: get_db insts abc* -if {.base_cell.name == AND2} -foreach {puts $obj(.name)} it returns the total number of insts that are printed.

`-if` *`expression`*	Specifies a boolean expression, with extensions to make pattern matching simpler. - == and != do not require a quoted string, and they support pattern matching with * and ?. - Other operators like >, <, <=, >=, &&, ||, and the unary negate operator ! are supported. - Parentheses ( ) can be used to group complex expressions. Note: Unlike the <pattern> argument, inside the -if expression, you cannot use a string pattern to implicitly match the .name attribute of an object. You must use the .name field directly. For example, the following command will not match anything because .base_cell will become something like base_cell:AND2: get_db insts -if {.base_cell == buf*} #matches nothing use either of these instead: get_db insts -if {.base_cell.name == buf*} #match the .name attribute get_db insts -if {.base_cell == base_cell:buf*} #match the DPO name directly If you use an attribute that is an object list (like .pins), every object in the list is tested in an implicit OR, so if any of them pass, the expression is matched. - The -index option MUST be valid for ALL attributes inside the -if statement otherwise an error occurs For example: - For example, the following command finds all instances with base_cell names that matches buf*: get_db insts -if {.base_cell.name == buf*} - The following command returns all insts that have one or more pins that match CLK*. Only one copy of the matching inst object is returned, even if multiple pin names match CLK*. get_db insts -if {.pins.base_name == CLK*} - The following command gets all pins where delay_max_rise is greater than 5 and slew_max_fall is greater than 8. This is a valid command as both delay* and slew* are attributes indexed by view (view is a legal abbreviation of analysis_view). get_db pins -if {.delay_max_rise > 5 || .slew_max_fall < 8 } -index {view v1}
`-index` *`string`*	Some attributes represent an array of values. For example, a pin .slack_max_rise attribute has a different value for each analysis_view and each clock that can affect that pin. With no index, a default method defined for that attribute is used (for example, the maximum, or minimum of all values in the array). For slack_max_rise with no -index value, the value returned is the worst-case slack across all analysis_views. If you want to return a specific value, you must include the index value. The index values are object pointers, or a pair of obj_type name and object name (e.g. "analysis_view view1"). The word view is accepted as an abbreviation for analysis_view. For example: get_db $my_pins .slack_max_rise -index {view func_wc_cworst} will return the worst case .slack_max_rise for the analysis_view func_wc_cworst.
`-invert`	Inverts the *`pattern`* and `-if` *`expression`* filter to return the objects or attributes that do not match.

`-match_hier`	Specifies that the * in the *pattern* and *expression* expands to the next hierarchical divider character (/) instead of a complete string match. For example, the following command matches i1, i2, but not i1/i2: get_db insts i1* -match_hier
pattern	Specifies string pattern(s), where: * matches 0 or more of any characters ? matches exactly one characterMore complex patterns require -regexp or -if expression. For example: - The following command finds all instance names starting with A: get_db insts .name A* A0 A1/A2 ... Note: The pattern matching can also work directly on a dual-ported object. In that case, it will match the '.name' attribute of the object, but return the object rather than the .name string. This matching works only for objects that have a .name attribute defined. - The following command finds all instance objects (not names) with a name starting with A: get_db insts A* inst:top/A0 inst:top/A1/A2 ... Note: For the special case where a pattern string starts with '.' you need to use an extra '.' (an empty attribute name) so that '.text' is treated as a pattern and not as an attribute name. - The following command finds all nets whose names equal to .odd_name get_db nets . .odd_name - The following command finds all instance names starting with A or B: get_db insts A* B*
`-regexp`	Specifies that the simple pattern used in *pattern* and *expression* options is a regexp instead of the simple * and ? style. It does not change the `-if` pattern matching.
string	Specifies an object list.
	It can be an object list, collection, root (/), or root attribute. Collections are in the form of nets, insts and so on.
`-unique`	Specifies that the return list will not have duplicate entries.

Examples

- The following example retrieves the current value of the `library` attribute on the root directory:

```
genus:> get_db / .library
tutorial.lbr
```

- The following example assumes you are already at the root of the design hierarchy, so the object specification is omitted:

```
genus:> get_db library
tutorial.lbr
```

- The following example stores the attribute value in a variable:

```
genus:> set old_library [get_db library]
tutorial.lbr
```

- The following example returns the area of a library cell. `get_db` is nestable.:

```
genus:> get_db [get_db lib_cells *DAL*] .area
1.5
```

- The following example lists all root-level attributes starting with `lp`. It lists the attributes along with their values:

```
genus:> get_db / .lp*
```

- The following example lists how to use the chain. This example lists all the preserved nets for pins of the instance INST1:

```
genus:> get_db inst:top/mid1/leaf1/INST1 .pins.net.preserve
```

- The following example lists all attributes for root. It lists all attributes under root:

```
genus:> get_db obj_type:root .attributes
```

- To get a list of attributes based on a pattern (starts with `a`), use the following. It will not give the values corresponding to the attributes:

```
genus:> get_db obj_type:root .attributes */a*
```

- The following example shows matching all pins that lie in hierarchy a* only and not any further.

```
genus:> get_db pins a/* -sdc_match
```

- The following example lists all the nets containing a pattern "pso" which are not preserved:

```
genus:> get_db nets *pso* -if {.preserve == false}
```

- The following example shows the usage of `foreach` in `get_db`:

```
genus:> get_db insts *HE13*_2* -foreach {puts "inst $object has following pins : [get_db $object .pins]"}
inst inst:top/iA/HE13_62_2 has following pins : pin:top/iA/HE13_62_2/SLEEP pin:top/iA/HE13_62_2/SLEEPOUT
inst inst:top/iA/HE13_83_20 has following pins : pin:top/iA/HE13_83_20/SLEEP pin:top/iA/HE13_83_20/SLEEPOUT
inst inst:top/iA/HE13_85_21 has following pins : pin:top/iA/HE13_85_21/SLEEP pin:top/iA/HE13_85_21/SLEEPOUT
3
```

- The following command returns list of hpins at depth 3 and 4 that have `clk` in the name:

```
genus:> get_db hinst:h1 .local_hpins -depth {3 4} -if {.name == *clk }

hpin:top/h1/h2/h1/sys_clk

hpin:top/h1/h2/h2/sys_clk
hpin:top/h1/h2/h2/bist_clk

hpin:top/h1/h2/h1/h1/sys_clk

hpin:top/h1/h2/h1/h2/sys_clk
```

- The following examples show the difference between the vls `-attribute` and `get_db` commands.
  - Using the `vls -attribute` command:

```
genus:designs> vls -attribute
```

```
Total: 9 items
   base_name = /
      cmd_file = genus.cmd4
      command_log = genus.cmd4
      current_design = design:alu
      db_units = 1000
      elapsed_runtime = 81613 seconds
      escaped_name =

....

....
```

- Using the `get_db wireload` command:

  `genus:designs> get_db design:async_set_reset_flop_n .wireload`
  returns the following wireload model:

  `wireload:default_emulate_libset_max/slow/sartre18_Conservative`
  The `vls -attribute` command lists all user modified attributes and their values. The `get_db` command lists only the value of the specified attribute. The `get_db` command is especially useful in scripts where its returned values can be used as arguments to other commands.

Related Information

Affected by this command:

set_db
reset_db

Related command:

vls

get_liberty_attribute

`get_liberty_attribute` *attribute_name*
{*lib_arc* |*lib_cell* | *lib_pin* | *pg_lib_pin* | *library*| *operating_condition* | *wireload*}...

Options and Arguments

attribute_name	Specifies the name of the Liberty attribute whose value you want to retrieve.
{*lib_arc* | *lib_cell* | *lib_pin* | *pg_lib_pin* | *library* | *operating_condition* | *wireload*}	
	Specifies the name of the object for which you want to retrieve a Liberty attribute value.

Example

The following command retrieves the `default_operating_conditions` attribute of the `tutorial` library.

```
genus: number> get_liberty_attribute default_operating_conditions tutorialtypical_case
```

get_license

`get_license` *`license`* `[-version` *`float`*`] [-wait]`

Description

Checks-out an additional product license. Licenses can only be checked-out one at a time.

If the license is available, it is checked out and `1` is returned. If the license is not available, a warning message is issued and `0` is returned. To check-in a license, use the `license_checkin` command or quit Genus.

Options and Arguments

license	Specifies the license to be checked out. The licenses that can be checked out are:
	`Genus_Physical_Opt`
	`Genus_CPU_Opt`
	`Genus_Low_Power_Opt`
	`Genus_Synthesis`
	`Genus_Trial_CTS_Opt`
	`GEN_ENG100`
	`Cerebrus_Genus_Opt`
	Cerebrus_Replay
	`ISLAND`
	`Innovus_GigaPlace_XL_Opt`
	`Innovus_ML_Impl_Opt`
	`Innovus_GigaPlace_GXL_Opt`
	Innovus_Automotive_Opt
	Midas_User
	Midas_Verif
`-version` *float*	Allows you to specify a specific version of the license. By default, the command uses the hardcoded primary or alternate version of the license that you want to check out.
`-wait`	Waits until a license becomes available. The wait timeout is controlled by the value specified for the `-wait` option when you started Genus. The default wait timeout is 600s.
	This option does not require the `-queue` option to be specified when starting Genus.

Example

The following example checks-out the `Genus_Synthesis` license:

```
genus:> get_license Genus_Low_Power_OptChecking out license: Genus_Low_Power_Opt1
```

get_license_list

`get_license_list`

Description

Returns a Tcl list of additional licenses that are currently checked-out. The license used to launch Genus is not included in this list.

get_license_names

```
get_license_names license [-version float]
```

Description

Checks if the specified license feature exists. The command returns "1" if the specified license feature is available, and "0" if it is not available.

Use this command in scripts to check a license before checking it out.

Options and Arguments

license	Specifies the license to be checked. The licenses that can be checked are:
	`Genus_Physical_Opt`
	`Genus_CPU_Opt`
	`Genus_Low_Power_Opt`
	`Genus_Synthesis`
	`Genus_Trial_CTS_Opt`
	`GEN_ENG100`
	`Cerebrus_Genus_Opt`
	`Cerebrus_Replay`
	`ISLAND`
	`Innovus_GigaPlace_XL_Opt`
	`Innovus_ML_Impl_Opt`
	`Innovus_GigaPlace_GXL_Opt`
	`Innovus_Automotive_Opt`
	`Midas_User`
	`Midas_Verif`
	Following licenses are startup licenses only, and if specified they would only consume a license for no benefit:
	`Virtuoso_Digital_Implem`
	`Virtuoso_Digital_Implem_XL`
	`Vdixl_Capacity_Opt`
`-version` *float*	Specifies the license version.

Example

The following example checks if the `Genus_Synthesis` product license exist:

```
genus:> get_license_names Genus_Synthesis1
```

The returned value"1" indicates that the license is available.

get_license_version

`get_license_version` *license*

Description

Returns the actual version of the license checked out.

Options and Arguments

license	Specifies the license name for which to check the version. You can specify any of the following:
	Genus_Physical_Opt
	Genus_CPU_Opt
	Genus_Low_Power_Opt
	Genus_Synthesis

Example

The following example checks if the Genus_Synthesis product license exist:

```
genus: number> get_license_version Genus_Synthesis18.1
```

get_read_files

```
get_read_files [-quiet]
[-command command | -clear command]
```

Description

Returns information on the files that have been read in.

Options and Arguments

-clear *command*	Removes the entries stored for the specified command.
-command	Specifies the command for which you want the information.
	By default, the command returns the information for all commands that read in files
-quiet	Suppresses the status message.

Example

```
genus:> get_read_files{source ./simple_ple.g} {read_netlist dtmf_chip.v} {read_sdc dtmf_tight.sdc} {read_def fplan_mp.def}

genus:> get_read_files -command read_def         fplan_mp.def

genus:> get_read_files -clear read_sdcClearing recorded files for command 'read_sdc'.

genus:> get_read_files                                  {source ./simple_ple.g} {read_netlist dtmf_chip.v} {read_def fplan_mp.def}
```

```
genus:> get_read_files -clear source -quietgenus:>
```

get_scan_elements

```
get_scan_elements
 [-flops]
 [-segments]
 [-instances hinst...]
 [design]
```

Description

Returns the scan elements under any given hierarchy.

Options and Arguments

`-flops`	Indicates that flops need to be returned.
`-segments`	Indicates that scan segments need to be returned.
`-instances hinst`	Specifies the list of hierarchical instances under which scan elements are required.
design	Specifies the name of the top-level design.

help

```
help
[-all |-attribute |-category string
|-command | -message |-obj]]
[-detail | -summary]
[string] [string]
```

Description

Provides help on the specified Genus attributes, commands, objects, and messages.

> You can get help at any stage of the design.

Options and Arguments

`-all`	Prints help for all commands, attributes, objects, and messages.
`-attribute`	Limits the search to attributes only.
`-category` *string*	Filters the attribute information based on the specified category.
`-command`	Limits the search to commands only.
`-detail`	Provides full information for each of the matches.
`-message`	Limits the search to messages only.
`-obj`	Limits the search to objects only.
string string	The first string specifies the command, message ID, object type, or attribute for which you want help.
	The second string specifies a command option or attribute.
`-summary`	Provides a single line description for each of the matches.

Examples

- The following command returns summary help for the attribute `source`.

```
genus:root:> help -att source
Attributes:
  source(specialnet):   # Specialnet creation source.
  source(pnet):         # Net creation source.
  source(direct_access_function):
                        # Source pin/port of the direct access function.
  source(programmable_direct_access_function):
                        # Source pin/port of the direct access function.

    Type 'help <obj_type> <attribute_name>' to get full description of object attribute
```

- The following command returns help for the attribute `source` that belong to the `phys` category.

```
genus:root:> help -cat phys source
Attributes:
  source(specialnet):   # Specialnet creation source.
  source(pnet):         # Net creation source.
    Type 'help <obj_type> <attribute_name>' to get full description of object
attribute
```

- The following command returns detailed help for the attribute `source` of type `pnet`.

```
genus:root:> help pnet source
Attribute: source (on obj_type: pnet)
  Description:            Net creation source.
  category:               phys
  default_value:
  is_computed:            false
  is_settable:            false
  skip_in_db:             false
  data_type:              string
```

- The following commands return help for the message DFT-100.

```
genus:root:> help -message DFT-100 -detail
```

```
or
genus:root:> help -message DFT-100
or
genus:root:> help DFT-100
```

all return:

```
Message
  name:                   DFT/DFT-100
  severity:               Info
  type:                   Added DFT object
  help:
```

- The following command returns a list of attributes for object type `group`.

```
genus:root:> help -obj group
Obj_type group:
  Attribute:
    base_name: (string):
                        # Object base (i.e. leaf) name.
    obj_type: (object): # Object type.
    name: (string):     # Object name used for get_db/set_db/reset_db queries.
    escaped_name: (string):
                        # Object escaped name.
    def_name: (string): # Original DEF group name.
    region: (string):   # Region associated with group.
    members: (string):  # Members associated with group.
    properties: (string):
                        # Group properties.
    user_created: (bool):
                        # Is group user created.
    design: (design):   # Parent 'design' object.
```

include

```
include
      file
```

Description

Executes scripts containing Genus or Tcl commands in the order they are listed in the `include` command.

When Genus begins executing, a number of scripts are automatically included.

The use of script files allows automation and modularization of the design flow by placing commonly used commands and sequences of commands into their own scripts. For externally defined components like memories, the vendor can supply a script to create and set up the memory.

This command is identical to the `source` command.

If an error occurs during execution of one of the scripts, execution of that script (and all scripts following it) is stopped.

Options and Arguments

file	Specifies the name of the script file to include.

Examples

- The following example includes one script file:

  ```
  genus:> include constraint.g
  ```

- The following example includes multiple scripts at once:

  ```
  genus:> include constraint.g; include synth.g
  ```

Related Information

Affected by these attributes:

init_hdl_search_path
init_lib_search_path
script_search_path

is_attribute

`is_attribute` *attribute* `-obj_type` *object_type*

Description

Checks whether the specified attribute exists or not.

A `1` is returned when the attribute exists, a `0` when the attribute does not exist.

Options and Arguments

attribute	Specifies the attribute name to be checked.
`-obj_type` *object_type*	
	Specifies the object type for which the attribute must be checked. All valid object types can be found by typing `get_db object_types .name` at the prompt.
	The command help will show a list of all valid object types.

Examples

- The following command checks whether `osc_source` exists for instances:

```
genus: xx> is_attribute osc_source -obj_type inst
0
```

- The following is_attribute command checks whether `lp_clock_gating_min_flops` exists for design:

```
genus: xx> is_attribute lp_clock_gating_min_flops -obj_type design

1
```

Related Information

Related command:

get_db

is_collection_object

```
is_collection_object string
```

Description

Checks whether an object is a collection object or not. The value 1 is returned when the object is a collection and 0 when the object is not a collection.

Options and Arguments

collection_object	Specifies the object to be checked.

Examples

- The following `is_collection_object` command checks whether `get_cells` returns a collection object or not.

```
genus:> is_collection_object [get_cells]

1
```

- The following `is_collection_object` command checks whether the tcl variable *all_instances* is a collection object or not.

```
genus:> is_collection_object $all_instances

1
```

Related Information

Related Commands:

sizeof_collection

foreach_in_collection

is_common_ui_mode

```
is_common_ui_mode
```

Description

Checks if the common-UI mode is enabled or not.

Checks if the Stylus Common UI mode is enabled or not. This command returns "1" or "0" depending on whether it is executed in Stylus Common UI mode or not (respectively).

Examples

- The following example shows execution of `is_common_ui_mode` command on the Genus prompt:

  ```
  genus: xx> is_common_ui_mode
  1
  ```

  Return value 1 indicates that the common-UI mode is enabled.

  ```
  legacy_genus:/> is_common_ui_mode
  0
  ```

  Return value 0 indicates that the Stylus Common UI mode is not enabled. Genus prompt is also in legacy UI mode.
- The following example checks UI mode of current Genus prompt and executes required code in respective UI:

  ```
  if {[is_common_ui_mode]} {
   set_db syn_generic_effort medium
  } else {
   set_attr syn_generic_effort medium
  }
  ```

license_reset_wait

```
license_reset_wait
```

Resets the license wait option if used during startup.

lminus

```
lminus [-exact] list1 list2
```

Description

Removes one or more elements specified in the second list (list2) from the first specified list (list1). If none of the elements are found, genus will return original (list1).

Options and Arguments

`-exact`	Searches for an exact pattern match.
	By default, searches to expand wildcards.
list1	Specifies the list from which elements must be removed.
list2	Specifies the list of elements to remove from original list (list1).

Examples

- The following command subtracts the elements from list `l2` from list `l1`.

  ```
  genus: xx> set l1 {a a* ab2 b1 b2 c} genus: xx> set l2 {a c} genus: xx> lminus $l1 $l2 a* ab2 b1 b2
  ```

- The following command subtracts a list consisting of `a*` and `c` from the list of elements of `l1`.

  ```
  genus: xx> lminus $l1 {a* c} b1 b2
  ```

- The following command uses an exact search.

  ```
  genus: xx> lminus -exact $l1 {a* c} a ab2 b1 b2
  ```

- The following command return same list in case no element found:

  ```
  genus: xx> lminus $l1 {f} a a* ab2 b1 b2 c
  ```

man

```
man [section] { command_name | attribute_name | message_ID}...
```

Description

This command formats and displays the on-line manual pages for a command, attribute, or message ID. In order to access on-line manual pages outside Genus, you must first set your `MANPATH` environment variable to the following path:

```
$CDN_SYNTH_ROOT/share/synth/man_common
```

After setting the `MANPATH` variable, you can obtain manpages for commands, attributes and messages, from within Genus or within the Unix shell.

Options and Arguments

`section`	If you specify a section number, man only looks in that section of the on-line manual.
	Commands are in section 1. Attributes are in section 2. Messages are in section 3.
	In the absence of section number, man will display the first matching manual page found looking at section 1, 2, 3 in that order.
{*command_name* | *attribute_name* | *message_ID*}	
	Name(s) of the command, attribute, or message ID.

Examples

- The following examples display `syn_generic` command on-line manual pages:

  ```
  man syn_generic
  ```

  ```
  man 1 syn_generic
  ```

- The following examples display on-line manual pages for message TIM-101:

  ```
  man TIM-101
  ```

  ```
  man 3 TIM-101
  ```

- The following examples display command on-line manual pages for retime attribute:

  ```
  man 2 retime
  ```

more

`more` *command*

Description

Displays the output of the specified command one screen at a time.This command is similar to the UNIX `more` command.

If the output spans several screens, the percentage of characters displayed so far is also shown at the bottom of the screen.

- To scroll down the display one line at a time, press *Return.*
- To display the next screen, press the SPACE bar.
- To stop the display, press *q.*

Options and Arguments

command	Specifies the command whose output you want to display with the `more` command.

preserve_analog_pins_from_undriven_handling

```
preserve_analog_pins_from_undriven_handling [> file]
```

Description

Identifies the analog pins or driving end of nets on such pins, and makes sure that the undriven handling is skipped for them. This command is used after `elaborate` and before `syn_gen`.

Related Information

Related command:

elaborate

quit

`quit` *code*

Description

Exits from the Genus shell without saving design data.
Control-C is a shortcut to the `exit` and `quit` commands.

> When exiting, no design data is saved, so it is important to save the design using the write_hdl and write_db commands.

Options and Arguments

code	Specifies the exit code.
	The following are the built-in exit codes:
	0 – normal exit
	1 – abnormal exit
	246 – exit when no license server is available
	245 – exit when no license feature is available
	244 – exit when syntax error in script

redirect

`redirect [-append] [-tee] [-stderr]`
`[-variable] [-msg]` *target command*

Description

Redirect command performs the same function as traditional unix redirection operators (> & >>) and redirects the output to a file (by default) or variable by using `-variable` option. You can also directly use redirect characters/operators without explicit use of the redirect command. Any Tcl return value from command is returned by redirect. If redirect is used at the prompt, any echoing of the Tcl return at the prompt is not redirected. You can also use the redirect command to write the contents directly to a gzip compressed file by ending the file name with .gz.

Messages generated by the command whose output you are redirecting, can be mixed with the command output. To limit the number of messages included, you can change the value of the information_level root attribute.

It is also possible to use redirect characters similar to Unix usage, for Genus commands, but not for built-in Tcl commands. This usage works for any Innovus command that does not explicitly register > or >> as options (if > or >> are visible in the man page or -help output for the command, it is explicitly registered).

Usage of redirect characters and how to use & to indicate a Tcl var name are shown below:

> *name* - redirect stdout to the file *name*

>> *name* - redirect stdout and append to the file *name*

> &*name* - redirect stdout to the Tcl var *name*

>> &*name* - redirect stdout and append to the Tcl var *name*

Options and Arguments

`-append`	Appends stdout/stderr output to the specified file or Tcl variable.
`-msg`	Prevents messages from being suppressed.
`-stderr`	Redirects stderr also to the specified file or Tcl variable.
`-tee`	Redirects output to both, the file or variable and the standard output (`stdout`).
`-variable`	Redirects the output to a Tcl variable.
command	Specifies the command or Tcl code to execute as a quoted string or curly braces.
target	Specifies the name of the file or variable to which to write the output.

Examples

- The following example sends the output generated by the `report_gates` command to a file called `gates.rep`:

  ```
  redirect gates.rep report_gates
  ```

- The following command redirects the output to a new file timing.txt

  ```
  redirect timing.txt "time_design -post_route"
  ```

- The following command appends the output to the file test.out.gz in compressed gzip format

  ```
  redirect test.out.gz "source test.tcl" -append
  ```

- The following command redirects and appends to the file a.log

  ```
  report_area >> a.log
  ```

- The following command redirects to the Tcl var a

  ```
  report_area > &a
  ```

- The following command redirects and appends to the Tcl var a

  ```
  report_area >> &a
  ```

- The following command allows redirecting output of multiple commands with single redirect command to a file or variable:

  ```
  redirect out.txt {
         ==> puts "Hello"
         ==> echo "World" }

  exec cat out.txt
         Hello
         World
  ```

- Redirect command allow to tee output to previous output target:

  ```
  redirect -tee out.txt {
        ==> echo "Hello"
        ==> redirect out.txt -append {
        ==> echo "World"
        } }

  exec cat out.txt
         Hello
         World
  ```

release_license

`release_license` *`license`*

Description

Checks in a product license that was previously checked out.

Options and Arguments

license	Specifies the license to be checked in. The licenses that can be checked in are:
	• Genus_CPU_Opt
	• Genus_Synthesis
	• Innovus_GigaPlace_XL_Opt
	• Innovus_GigaPlace_GXL_Opt

Example

The following example checks in the Genus_Synthesis license:

```
genus:> release_license Genus_Synthesis
```

reset_db

```
reset_db [-category string] [-index string] [-match string] string [chain] [-quiet] [-verbose]
```

Description

Resets attribute value to a predefined default value. It returns a single value when resetting a single attribute, a list of values when resetting same attribute on different object instances, or a human readable table for multiple attributes in the following format:

```
Attribute_name value
```

For example:

```
place_xxx true
```

```
add_filler true
```

> The -match and -category parameters of the reset_db command can be used to reset root attributes only. Both user-defined and system root attributes can be reset using –match and/or –category options.

The attribute value can be reset if the attributes default_value is specified.

Options and Arguments

-category *string*	Specifies root attribute category based on predefined list of categories.

	For example, the following command resets all the root attributes to the default value in the `ccopt` category:
	`reset_db -category ccopt`
	You can find all the categories by:
	`get_db categories`

	Note: This applies only to the root level attributes and is ignored for the lower levels.
chain	Specifies an optional list of object names in the form of:
	[.*attribute_name1* [.*attribute_name2*]...]
	The specified attribute names are expected to be valid for each level of the traversal in the *chain*.
`-index` *string*	Specifies a list of indices in a form of pairs of `index_name` *value* or a dual port object name.
`-match` *string*	Specifies a simple string pattern with or without wildcards (* or ?).
	It is applicable only to root attributes.
`-quiet`	Suppresses information messages.
	Warning and error messages will be shown.
	By default `set_db` is quiet. If the `set_db_verbose` root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the -verbose parameter's description for an example). -quiet forces set_db to be quiet even if set_db_verbose is true. This is intended for internal scripts that want to be certain that the verbose output is off.
string	Specifies as object list. Can be any object list (or collection) or shorthand.
	It can be a collection, such as, nets, insts.
`-verbose`	Returns output with maximum verbosity.
	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected.

	For example:
	`set_db -verbose [get_db hinsts D*] .dont_touch false Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false`
	`Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false2 false`

	The `set_db_verbose` root attribute can also be set to `true`, to force verbose output for all `set_db` and `reset_db` usage without requiring `-verbose`.

Examples

- The following example specifies that all retiming attributes should be returned to their default values:

```
genus: xx> reset_db retime* *
```

- The following example specifies that all attributes on all sequential instances be returned to their default values:

```
genus: xx> reset_db * [get_db insts *seq/*]
```

- The following `reset_db` command resets the value of instance attribute:

```
define_attribute  test_reset_inst -obj_type inst -data_type int -category \
        reset -help_string "test reset function" -default 1
1

set_db insts .test_reset_inst 10
10 10 10 10 10 #Four instances in the design

reset_db insts .test_reset_inst
1 1 1 1 1 #All 4 attributes are reset to default values
```

- The following `reset_db` command resets the root attributes using the -match option:

```
define_attribute test_reset1 -obj_type root -data_type int  -category  \
      reset  -help_string  "test  reset function" -default 1

define_attribute  test_reset2  -obj_type  root  -data_type  int -category \
      reset -help_string "test reset function" -default 1

define_attribute test_reset3 -obj_type root -data_type int  -category  \
      reset  -help_string  "test  reset function" -default 1

set_db test_reset1 10
10

set_db test_reset2 10
10

set_db test_reset3 10
10
```

  When you specify:

```
reset_db -match test_r*
```

  The attributes are reset to their default values and a human readable table with new values is displayed.

```
test_reset1 1

test_reset2 1

test_reset3 1
```

- The following command resets same set of attributes using the -`category` parameter:

```
reset_db -category reset
```

  The attributes are reset to their default values and a human readable table with new values is displayed:

```
test_reset1 1

test_reset2 1

test_reset3 1
```

- The following `reset_db` command resets all the attributes with -`match` command to default values:

```
reset_db -match syn_*
Setting attribute of root '/': 'syn_global_effort' = none
Setting attribute of root '/': 'syn_opt_effort' = high
Setting attribute of root '/': 'syn_map_effort' = high
Setting attribute of root '/': 'syn_generic_effort' = medium
1 none high high medium
```

- The following command simply resets the single attribute to default value:

```
reset_db lp_insert_clock_gating
Setting attribute of root '/': 'lp_insert_clock_gating' = false 1 false
```

Related Information

Related commands:

get_db
set_db

resume

```
resume
```

Restarts the current Tcl process and brings back the regular `genus:>` prompt. This command only works in conjunction with the `suspend` command. See the `suspend` command to see how these commands work together to stop and restart a Tcl process.

Related Information

Related commands:

stop_suspend
suspend

run_embedded_script

```
run_embedded_script
[-design string] [-module string]
```

Description

Executes embedded scripts found in a specified design or subdesign. To execute the scripts on all top-designs and their subdesign, use the `run_embedded_script` command without any arguments. Use this command after the elaborate command.

The embedded script of a design or subdesign is stored in the embedded_script attribute of that design or subdesign.

Options and Arguments

`-design` *string*	Specifies the top level design for which the embedded script needs to be executed. Using this option, the command executes scripts for the specified design and every module in it.
`-module` *string*	Specifies the full path of the subdesign for which the embedded script needs to be executed. Using this option, the command executes the script only for the specified subdesign.

Related Information

Related command:

elaborate

Related attributes:

hdl_auto_exec_sdc_scripts

search_doc

`search_doc` *string*

Searches the specified string(s) in the installed documentation hierarchy and on Cadence Customer Website (COS), and displays the results in Cadence Doc Assistant browser.

Options and Arguments

string	Specifies the string that needs to be searched in the installed Genus Documentation set.

set_db

```
set_db [-dbu]
[-index string]
string [string] value...
[-quiet] [-verbose]
```

Description

Changes the specified attribute value for a database object. The `set_db` command takes a dual-ported object, dual-ported object list, or a collection as input, and then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema. This command does not accept wildcards in the database object argument.

- The string form is normally a useful name preceded by the `obj_type` so a `base_cell` object string form might look like `base_cell:and2`.
- A netlist name will have the current design name included, so an instance named `i1/i2` in design `top`, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex value like wire:`0x22222`.

Two ways to modify values of objects using `set_db`:

1. `set_db` *`object(s) .chain value`*

2. `set_db` *`object(s) .attribute value`*

Attributes are placed on directory objects to control the way they are processed by Genus. They can also be used to control the synthesis process, the style of the generated code, and numerous other things. A complete list of all attributes is contained in the *Genus Attribute Reference*.

> Not all attributes can be set. Attempting to set a *read-only* attribute returns an error. The *Genus Attribute Reference* indicates whether an attribute is read-write or read-only.

The `set_db` command does not create or delete objects.

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (For example, layers and libraries). The root object is the default starting point if there is no input list or collection given.

You can set any root attribute in any of the following ways:

- `genus:> set_db / .flow_mail_to` *`name`*
  where, '/' is the notation for the root object, or,
- `genus:> set_db flow_mail_to` *`name`*
  This implicitly points to the root object.

Frequently used `current_design` attributes like insts or hinsts (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name.

For example, the following two commands set all insts `.place_status` to fixed.

```
genus:> set_db current_design .insts.place_status fixed
```

```
genus:> set_db insts .place_status fixed
```

Some of the more commonly used aliases from the `current_design` attributes include: `insts`, `hinsts`, `nets`, `hnets`, `ports`, `hpins`, `pins`.

The `set_db` command supports auto-complete when you press the *`Tab`* key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For information on how to access DB objects, see the `get_db` command.

Options and Arguments

chain	Specifies an optional list of object names.
	It is in the form [.*attribute_name1* [.*attribute_name2*]...]. The specified attribute names are expected to be valid for each level of the traversal in the *chain*. All valid attributes auto-complete on `tab` press.
`-dbu`	Indicates that attribute values (coord, pt, rect, polygon, area,...) and they should be in database units.
	Default: A double in microns.
`-index` *string*	Specifies a list of indices in a form of pairs of {`index_name value`} or a dual port object name. It is applicable to attributes that represent an array of values.
	Some attributes represent an array of values. For example, a `pins.slack_max_rise` attribute has a different value for each `analysis_view`.
	With no index, all the values defined for that attribute are modified by the `set_db` command.
	If you want to change a specific value, you must include the index value.
	The index values are pairs of `obj_type` names (for example, `analysis_view`, `clock`, `skew_group`) followed by the name.
`-quiet`	Suppresses information messages. By default `set_db` is quiet. If the `set_db_verbose` root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the -`verbose` parameter's description for an example). -`quiet` forces `set_db` to be quiet even if `set_db_verbose` is true. This is intended for internal scripts that want to be certain that the verbose output is off.
	Warning and error messages will be shown.
string	Input list of dual-ported objects or a collection. Wildcards are not accepted.
value	Specifies the new value for the specified object attribute elements. The value type depends on the object attribute type that is being modified.
`-verbose`	Returns output with maximum verbosity.
	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected.

	For example:
	`set_db -verbose [get_db hinsts D*] .dont_touch false`
	`Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false`
	`Setting a0ttribute of hinst 'DECODE_INST': 'dont_touch' = false`
	
	`2 false`

	The `set_db_verbose` root attribute can also be set to true, to force verbose output for all `set_db` and `reset_db` usage without requiring -verbose.

Examples

- The following example sets the information_level attribute, which controls the verbosity of the tools, to the value of `5` and assumes the current directory for the path:

  ```
  genus:> set_db information_level 5
  ```

- The following locks the technology library search path to `/home/Test/foo` by locking the `lib_search_path` attribute. For the rest of the session, the `lib_search_path` attribute becomes read-only.:

  ```
  genus:> set_db -lock lib_search_path /home/Test/foo
  ```

- The following example shows how to set preserve attribute for instance a/b

  ```
  genus:> set_db inst:a/b .preserve true
  ```

- The following example shows how to set preserve attribute for all instances

  ```
  genus:> set_db [get_db insts] .preserve true
  ```

  > Wildcards cannot be explicitly added to the object-list but can be added as a TCL-list.

  > Wildcards can be used for attribute names with `get_db` command, but not with `set_db` command.

- The following example shows how to set `max_fanout` attribute for all input ports

  ```
  genus:> set_db [get_ports -filter direction==in] .max_fanout 10
  ```

- The following example shows how to set preserve attribute for the nets of the instance a/b

  ```
  genus: number> set_db inst:a/b .pins.net.preserve true
  ```

- The following command sets an instance attribute (user-defined) value to 5 only if its current value is 0:

  ```
  genus: number> set_db [get_db insts -if {.myattr == 0}] .myattr 5
  ```

Related Information

Affects these commands:

vls

Related command:

get_db
reset_db

set_multi_cpu_usage

```
set_multi_cpu_usage
[-local_cpu integer] [-verbose]
```

Description

Specifies the maximum number of active CPUs allowed to be used on one server.

Options and Arguments

`-local_cpu` *integer*	
	Specifies the number of available local CPUs. *Default*: 8
`-verbose`	Displays all messages when changing the settings.

Related Information

set_proc_verbose

```
set_proc_verbose
{procedure_name [-quiet] |[-report]}
```

Description

Controls the verbosity of a procedure or reports all verbose procedures.

Options and Arguments

procedure_name	Specifies the name of the procedure to be made verbose.
`-quiet`	Changes the procedure back to not being verbose.
`-report`	Reports procedures that are verbose.

Examples

- The following commands sets `myprocedure` as verbose:

  ```
  set_proc_verbose myprocedure
  myprocedure
  ```

- The following command reports `myprocedure` as a verbose procedure

  ```
  set_proc_verbose -report
  myprocedure
  ```

source

```
source [-quiet] [-verbose] [-rsrc] [-check] [-rsrcid] [-echo] file
```

Description

Reads a file and executes the commands in the file (scripts).

> The default behavior is different between Stylus UI and Legacy UI because the default value of `source_verbose` attribute is true in Stylus UI, while it is false in Legacy UI.

In Stylus UI, the default behavior is same as having -`verbose` option.

Options and Arguments

`-echo`	Echoes each line in the script.
file	Specifies the file to be sourced.
-check	Specifies the file to be check
`-quiet`	Prevents line-by line information printing.
`-rsrc`	Genus does *not* support this option.
`-rsrcid`	Genus does *not* support this option.
`-verbose`	Prints line-by line information.

Example

The following example run the script in `sample.tcl`:

```
genus:root:> source sample.tcl
```

Related Information

Related attribute:	source_verbose

start_preserve_for_auto_ungroup_constraints

```
start_preserve_for_auto_ungroup_constraints
```

Description

Starts to tag the preserve setting `ungroup_ok_constraint` on the hierarchical instance having constraints.

stop_preserve_for_auto_ungroup_constraints

`stop_preserve_for_auto_ungroup_constraints`

Description

Stops tagging the preserve setting `ungroup_ok_constraint` on the hierarchical instance having constraints.

stop_suspend

`stop_suspend`

Description

Terminates the script execution when you have stopped the current Tcl process with the `suspend` command (which brings up the `genus-suspend@root:>` prompt) and returns from the suspend command loop (brings back the regular `genus:>` prompt).

This allows you to have access to the GUI to analyze any issues reported so far.

Related Information

Related commands:

resume
suspend

stringify

`stringify` *`string`*

Description

Converts a Tcl object into a string.

Options and Arguments

`string`	Specifies the object to be converted.

suppress_messages

`suppress_messages [-number` *`integer`*`] [-quiet]` *`message_id...`*

Description

Suppresses printing of the specified messages in the log file.

Options and Arguments

message_id	Specifies the list of message IDs to be suppressed. The list can be in the form of Tcl list or space separated collection of multiple message IDs. Wild cards are accepted.
-number *integer*	Prints the specified message only *integer* number of times in current session. The default value is 0.
-quiet	Suppresses those messages that indicate which attributes are being affected for the messages that are being suppressed.

Examples

- The following example suppresses the VLOG-1 and VLOG-2 messages – tcl list of messages provided as argument:

```
genus: xx> suppress_messages { VLOGPT-1 VLOGPT-2 }

Setting attribute of message 'VLOGPT-1': 'max_print' = 0
Setting attribute of message 'VLOGPT-2': 'max_print' = 0
```

- The following example suppresses the VLOG-1 and VLOG-2 messages – space separated list of messages provided as argument:

```
genus: xx> suppress_messages VLOGPT-1 VLOGPT-2

Setting attribute of message 'VLOGPT-1': 'max_print' = 0
Setting attribute of message 'VLOGPT-2': 'max_print' = 0
```

- The following example suppresses the printing of WDO-209 and all VRO messages to a maximum of 2 times in current session – wildcard usage:

```
genus: xx> suppress_messages  WDO-209 VRO-* -number 2

Setting attribute of message 'WDO-209': 'max_print' = 2
Setting attribute of message 'VRO-10': 'max_print' = 2
Setting attribute of message 'VRO-19': 'max_print' = 2
Setting attribute of message 'VRO-20': 'max_print' = 2
Setting attribute of message 'VRO-21': 'max_print' = 2
Setting attribute of message 'VRO-22': 'max_print' = 2
```

Related Information

Related command:

unsuppress_msg

suspend

```
suspend
```

Description

Suspends your script and returns to the Genus prompt.

Stops the current Tcl process and brings up the `genus-suspend@root:>` prompt. Any commands or attributes that are issued during this time will not have access to the temporary variables from the suspended Tcl process. However, global variables and Tcl processes can still be accessed.

The `suspend` command gives you flexibility in managing and debugging your scripts. You can insert multiple `suspend` commands in a script. The `suspend` command can also be used in nested scripts. When the script reaches a point where you have inserted the `suspend` command, it stops and restores access to the Genus prompt. You can then type any command required for debugging.

For example,

```
read_hdl -f $RTL_FILE_LISTsuspendelaborate $TOP_MODULE_NAME
```

Whenever you want to resume your script, just type `resume` at the Genus prompt. If you do not want to resume your script and wants to come out of `suspend` mode, just type `stop_suspend`.

> Any commands that you enter after issuing the `suspend` command but before the `resume` command will not be included in Tcl's command history. However, these commands will be included in Genus's command editor history.

Related Information

Related commands:

resume
stop_suspend

test_super_thread_servers

```
test_super_thread_servers
```

Returns status information on the machines used for super-threading.

Examples

```
genus:> set_db super_thread_servers {rcl117 rcl118}
Setting attribute of root '/': 'super_thread_servers' = rcl117 rcl118
genus:> test_super_thread_servers
Info : Attempting to launch a super-threading server. [ST-120]
: Attempting to Launch server 1 of 2.
: The tool is entering super-threading mode and is launching a CPU server process. This is enabled by the root
attribute 'super_thread_servers' or
'auto_super_thread'.
: The server is rsh process '23453' to host 'rcl117'.
Info : Attempting to launch a super-threading server. [ST-120]
: Attempting to Launch server 2 of 2.
: The server is rsh process '23459' to host 'rcl118'.
Info : Requested number of super-threading servers are available. [ST-124]
: The number of background servers requested are available and are working fine.
```

unsuppress_msg

`unsuppress_msg` *`message_id...`*

Description

Allows a previously suppressed message to be printed. The list can be in the form of TCL list or space separated collection of multiple message IDs. Wild cards are accepted.

Options and Arguments

`message_id`	Specifies a ist of message IDs to unsuppress.
	The list can be a TCL list or space-separated collection of message IDs. Wilcards are accepted.

Examples

- The following example suppresses the `VLOGPT-1` and `VLOGPT-2` messages and then allows them to be printed again:

```
genus:> suppress_messages { VLOGPT-1 VLOGPT-2 }
Setting attribute of message 'VLOGPT-1': 'max_print' = 0
Setting attribute of message 'VLOGPT-2': 'max_print' = 0
```

- The following example provides a TCL list of messages as argument:

```
genus:> unsuppress_msg { VLOGPT-1 VLOGPT-2 }
Setting attribute of message 'VLOGPT-1': 'max_print' = inf
Setting attribute of message 'VLOGPT-2': 'max_print' = inf
```

- The following example provides a space-separated list of messages as argument:

```
genus:> unsuppress_msg VLOGPT-1 VLOGPT-2
Setting attribute of message 'VLOGPT-1': 'max_print' = infinity
Setting attribute of message 'VLOGPT-2': 'max_print' = infinity
```

- The following example shows wildcard usage:

```
genus:> unsuppress_msg MESG*
Setting attribute of message 'MESG-1': 'max_print' = inf
Setting attribute of message 'MESG-2': 'max_print' = inf
Setting attribute of message 'MESG-3': 'max_print' = inf
Setting attribute of message 'MESG-4': 'max_print' = inf
Setting attribute of message 'MESG-5': 'max_print' = inf
Setting attribute of message 'MESG-6': 'max_print' = inf
Setting attribute of message 'MESG-7': 'max_print' = inf
Setting attribute of message 'MESG-8': 'max_print' = inf
Setting attribute of message 'MESG-9': 'max_print' = inf
Setting attribute of message 'MESG-10': 'max_print' = inf
Setting attribute of message 'MESG-11': 'max_print' = inf
```

Related Information

Related command:

suppress_messages

set_latch_loop_disable

```
set_latch_loop_disable
[-from string]
[-to string]
[inst]..
```

Description

Specifies the latch breaking point where D pin will be considered as endpoint.

Options and Arguments

[-from *string*]	
	Specifies the from pin name of cell.
[-to *string*]	
	Specifies the to pin name of cell.
[*inst*]..	
	Specifies the list of instance pins, instances, or clocks on which the borrow value is applied. The argument can be a collection.

Example

```
set_latch_loop_disable -from D -to Q [get_cells {ll1}]
```

2

Advanced Low Power Synthesis

- apply_cglar
- apply_power_intent
- check_cpf
- check_library
- check_power_intent
- check_power_structure
- commit_dft_power_intent
- commit_module_model
- commit_power_intent
- compare_crossing
- copy_library_domain_attributes
- cpi_insert_auto_ls
- create_library_domain
- define_design_partition
- eval_1801
- optimize_iso_ls
- partition_design
- read_power_intent
- reset_power_intent
- set_cglar_specification
- write_power_intent

The following command is common with Analysis and Reporting commands:

- report_power_intent
- report_power

apply_cglar

```
apply_cglar
[-elab_db string] [-elab_tcl string] [-mapped_db string]
[-mapped_tcl string] [-netlist string] [-top_design string]
[-sdc string] [-def string] [-spef string] [-lib_proc string]
[-read_stimulus_options string]  [-compute_cglar_options string]
[-report_cglar_options string] [-pre_script string]
[-post_script string] [-do_not_implement_cglar]
[-no_db] [-power_data] [-verbose] [-work string]
[-unit string] [-format string] [-cg_slack_delta_mode integer]
```

Description

Performs XOR gating analysis for the design. Use the options to control the implementation and power reporting.

Options and Arguments

`-compute_cglar_options` *string*	
	Specifies the `compute_cglar` options.
`-def` *string*	Specifies the DEF file with the netlist.
`-spef` *string*	Specifies the SPEF file with the netlist.
`-do_not_implement_cglar`	
	Specifies not to implement this cglar back in Genus session.
`-elab_db` *string*	Specifies elaborated DB with Joules default naming rules for rtlstim2gate flow.
`-elab_tcl` *string*	Specifies the Tcl applicable with elaborated DB.
`-format` *string*	Specifies the power format.

	This option is supported only with `power_data` option.
`-lib_proc` *string*	Specifies TCL proc used for library settings.
`-mapped_db` *string*	Specifies Genus mapped DB.
`-mapped_tcl` *string*	Specifies the Tcl applicable with target DB.
`-netlist` *string*	Specifies the third party netlist.
`-no_db`	Specifies to write netlist based flow.
`-post_script` *string*	Specifies the TCL file to be sourced after implementation of cglar.
`-power_data`	Specifies to enable power reporting before and after XOR gating.
`-pre_script` *string*	Specifies the TCL file to be sourced before `read_stim`.
`-read_stimulus_options` *string*	
	Specifies the `read_stimulus` options.
`-report_cglar_options` *string*	
	Specifies the `report_cglar` options.
`-sdc` *string*	Specifies the SDC file with the netlist.
`-top_design` *string*	Specifies the top design name in the netlist.
`-unit` *string*	Specifies the power unit {`W| mW | uW | nW | pW`}
	`Default`: W
	This option is supported only with `power_data` option.

`-verbose`	Specifies to apply XOR gating in verbose mode.
`-work` *string*	Specifies Joules working directory. *Default*:genus2joules_work
`-cg_slack_delta_mode` *integer*	
	The slack delta (ps) added for clock gates slack margin/threshold.

Example

```
apply_cglar -read_stim_options "-file ACTIVITY/top.vcd -resim_cg_enables " \
    -compute_cglar_options "-da_threshold 0.3  -min_slack_threshold -50 \
     -process_gated_flops" \-report_cglar_options "-min_slack_margin 100 -add_xor_options \
    {-skip_async_check -max_distance 100} \-generate_script bank -out scrub.rpt" -power_data -
pre_script pre.tcl -work workdir
```

Related Information

For options `-compute_cglar_options` and `-report_cglar_options`, refer to `compute_cglar` and `report_cglar` commands in Joules.

Related commands:

syn_map
set_cglar_specification

apply_power_intent

```
apply_power_intent
[-design design]
[-module module]
[-summary]
```

Description

Applies the power intent that was previously read in from power intent file(s).

You should not execute any command (such as `update_names` or `ungroup`) that can cause changes in the name of the design objects before you apply the power intent file(s). Otherwise, the `apply_power_intent` command may not find some design objects.

Options and Arguments

`-design` *design*	Specifies the design to which the power intent applies.
	This option is only required when multiple designs are loaded in the session. If specified, it must be used with the `-module` option.
`-module` *module*	Specifies the top module for which to apply the power intent.
	This option is required when the power intent file was read in before the design is read in, and when the elaborated design name differs from the top module name in the RTL. If specified, it must be used with the `-design` option.
`-summary`	Prints a summary of the power intent. Prints the number of power domains, isolation rules, level shifter rules, state retention rules, and power modes, as well as a list of partially supported commands.

Related Information

Related commands:

commit_power_intent
read_power_intent
write_power_intent

check_cpf

```
check_cpf  [-design design]
[-all | -isolation | -level_shifter | -retention]
[-lp_only] [-detail]
[-pre_read script] [-pre_exit script]
[-run_dir directory] [-debug] [-tcl]
[-continue_on_error] [-license string] [-generated] [> file]
```

Description

Checks the validity of the CPF rules against the RTL of the design. This enables designers to capture any violations of the low power intent of the design early in the design cycle.

- If no low power rule check errors are detected, the command returns `1`.
- If rule check errors are detected, the command returns `0`. In this case, you need to make the necessary changes to your CPF file before proceeding further with synthesis.

To run this command you need to have access to Conformal® Low Power.

Options and Arguments

`-all`	Reports all problems with the CPF file.
	By default, all problems will be reported.
`-continue_on_error`	Allows the tool to continue when low power rule check errors are encountered.
`-debug`	Creates a dofile and other required files for Conformal Low Power allowing you to debug any errors further in Conformal Low Power.
`-design` *design*	Specifies the name of the design in case of multiple top modules.
`-detail`	Provides a detailed report.
file	Redirects the report to the specified file.

`-generated`	Checks the validity of the CPF rules against the current state of the design.
	By default, the tool checks the validity of the CPF rules against the RTL of the design.
`-isolation`	Reports only problems with the isolation rules.
`-level_shifter`	Reports only problems with the level-shifter rules.
`-license` *string*	Specifies the Conformal license to be used for this command.
`-lp_only`	Reports only the low power rule check errors and warnings.
	By default, non low-power related issues, such as structural issues are reported as well.
`-pre_exit` *string*	Specifies the name of the dofile (script) that must be sourced before the CLP `exit` command.
`-pre_read` *string*	Specifies the name of dofile (script) that must be sourced before the libraries and the design are read.
`-retention`	Reports only problems with the state retention rules.
`-run_dir` *directory*	
	Specifies the directory in which the required files for Encounter Conformal Low Power must be stored.
	Default: `.clp`
`-tcl`	Specifies to generate the dofile as a Tcl script.

Related Information

Affected by this attribute:

wclp_lib_statetable

check_library

```
check_library
[-isolation_cell] [-level_shifter_cell]
[-retention_cell]
[-library_domain library_domain_list]
[-lib_cell lib_cell_list]
[> file]
```

Description

Allows you to check specific information in the loaded libraries with regards to level shifters, isolation cells, and state retention cells. The report also lists the unusable cells. The information returned can be fine tuned by combining several options.

Without any options specified, this command lists the number of level shifters, isolation cells, and state retention cells available in each of the library domains. If no library domains exist, the report lists the library names instead.

Options and Arguments

file	Specifies the name of the file to which to write out the library information.
-isolation_cell	Returns two lists of library cells:
	• A list of pure isolation cells with their isolation type
	• A list of combo cells for each cell
	◦ The isolation type
	◦ The voltage ranges that they support

	◦ Whether the combo cell can be used between a lower and higher voltage, or vice versa
	◦ The valid location for the cell
`-level_shifter_cell`	
	Returns a list of level shifter cells found in each of the library domains and specifies for each cell
	• The supported input and output range
	• Whether the level shifter can be used between a lower and higher voltage, or vice versa
	• The valid location for the cell
`-library_domain` *`library_domain_list`*	
	List the number of level shifters, isolation cells, and state retention cells available in each of the specified library domains.
`-lib_cell` *`lib_cell_list`*	
	If not combined with any other option, indicates for each of the specified library cells to which library domain it belongs, whether it is a level shifter, isolation cell, retention cell, always on cell, and the function of the cell.
`-retention_cell`	Returns for each library domain the following information:
	• A list of sequential elements that have no state-retention equivalent
	• A list of state-retention cells available in that domain

Related Information

Related commands:

read_power_intent

check_power_intent

```
check_power_intent [-design design]
[-always_on] [-retention]
[-isolation] [-level_shifter] [-show_full_names]
[-detail]
```

Description

Checks specific power intent issues. After reading the CPF/UPF file, check the validity of the CPF/UPF rules against the RTL of your design. This enables you to capture any violations of the low power intent of the design early in the design cycle. This verifies whether the low power cells in the power domain conform to the rules and specifications of the power intent. Specifically, Genus will flag if there are any missing isolation, level shifter, or state retention cells or if the low power cells are not connected appropriately.

If rule check errors are detected, you need to make the necessary changes to your CPF/UPF file before proceeding further with synthesis.Such CPF/UPF quality checking ensures that the specified power intent is correct and complete.

Options and Arguments

`-always_on`	Reports always-on cell-related issues.
`-design` *design*	Specifies the name of the design in case of multiple top modules.
`-detail`	Show detailed information.
`-isolation`	Reports isolation-related issues.
`-level_shifter`	Reports level-shifter cell-related issues.
`-retention`	Reports state retention cell-related issues.
`-show_full_names`	Specifies to print full names in the report.

Examples

- The following command reports problems with the level shifter rules:

```
genus:root: XX> check_power_intent -level_shifter

Summary
================================================
Number of pins missing level shifter cell : 860
================================================
Total number of issues found : 860
```

- The following command reports there are no issues with the isolation cells rules:

```
genus:root: XX> check_power_intent -isolation

Total number of issues found : 0
```

- The following command reports the problems with the level shifter rules in detail:

```
genus:root: XX> check_power_intent -detail -level_shifter

Summary
================================================
Number of pins missing level shifter cell : 5
================================================
Total number of issues found : 5

============================================================
Details of pins missing Level Shifter cell : (Total: 5)
============================================================
------------------------------------------------------------------------------

Source  : Icore/Iacc_top/dtb0_spare[0] (SS_PD_PADS)
Sink    : Icore/Imain_dig/Iatbdtb_ctl_hv/Icore/mux_dtb2_inbus_58_7/g1/data48
          (Icore/Imain_dig/SS_PD_HV)(1.53V)
Pins    : Icore/Imain_dig/dtb0_spare[2]
Strategy: PD_HV.LS_LV2HV
------------------------------------------------------------------------------

Source  : Icore/Iacc_top/dtb0_spare[1] (SS_PD_PADS)
Sink    : Icore/Imain_dig/Iatbdtb_ctl_hv/Icore/mux_dtb2_inbus_58_7/g1/data49
         (Icore/Imain_dig/SS_PD_HV)(1.53V)
```

```
Pins    : Icore/Imain_dig/dtb0_spare[3]
Strategy: PD_HV.LS_LV2HV
-------------------------------------------------------------------------------

Source  : Icore/Iacc_top/dtb1_spare[0] (SS_PD_PADS)
Sink    : Icore/Imain_dig/Iatbdtb_ctl_hv/Icore/mux_dtb1_inbus_55_7/g1/data48
          (Icore/Imain_dig/SS_PD_HV)(1.53V)
Pins    : Icore/Imain_dig/dtb1_spare[2]
Strategy: PD_HV.LS_LV2HV
-------------------------------------------------------------------------------

Source  : Icore/Iacc_top/dtb1_spare[1] (SS_PD_PADS)
Sink    : Icore/Imain_dig/Iatbdtb_ctl_hv/Icore/mux_dtb1_inbus_55_7/g1/data49
          (Icore/Imain_dig/SS_PD_HV)(1.53V)
Pins    : Icore/Imain_dig/dtb1_spare[3]
Strategy: PD_HV.LS_LV2HV
-------------------------------------------------------------------------------

Source  : Icore/Iacc_top/dtb1_spare[2] (SS_PD_PADS)
Sink    : Icore/Imain_dig/Iatbdtb_ctl_hv/Icore/mux_dtb1_inbus_55_7/g1/data50
           (Icore/Imain_dig/SS_PD_HV)(1.53V)
Pins    : Icore/Imain_dig/dtb1_spare[4]
Strategy: PD_HV.LS_LV2HV
-------------------------------------------------------------------------------
```

Related Information

Related commands:

check_power_structure

check_power_structure

```
check_power_structure [-design design]
[-isolation] [-level_shifter] [-retention]
[-all] [-lp_only] [-nwell_analysis]
{-pre_synthesis | -post_synthesis | -post_pg}
[-pre_read script] [-pre_exit script] [-detail]
[-run_dir directory] [-debug] [-tcl_mode]
[-continue_on_error] [-license string] [> file]
```

Description

Verifies whether the low power cells in the design conform to the rules and specifications in the loaded CPF file. Specifically, Genus will flag if there are any missing isolation, level shifter, or state retention cells or if the low power cells are not connected appropriately.

To run this command you need to have access to Conformal ® Low Power.

Options and Arguments

`-all`	Reports all violations.
`-continue_on_error`	Allows the tool to continue when low power rule check errors are encountered. This is not a default option.
`-detail`	Generates a detailed report.
`-debug`	Creates a dofile and other required files for Encounter Conformal Low Power allowing you to debug any errors further in Conformal Low Power.
`-design` *design*	Specifies the name of the design in case of multiple top modules.
file	Redirects the report to the specified file.
`-isolation`	Reports only isolation related violations.

	Has priority over `-all` option.
`-level_shifter`	Reports only level-shifter related violations.
	Has priority over `-all` option.
`-license` *string*	Specifies the Conformal license to be used for this command.
`-lp_only`	Reports only the low power rule check errors and warnings.
	This is now the default.
`-nwell_analysis`	Checks the design for shared and insulated nwell violations.
`-post_pg`	Checks the design for power and ground connections after mapping.
`-post_synthesis`	Runs Conformal Low Power, after synthesis, on low power cells.
`-pre_exit` *string*	Specifies the name of the dofile (script) that must be sourced before the `exit` command.
`-pre_read` *string*	Specifies the name of dofile (script) that must be sourced before the libraries and the design are read.
`-pre_synthesis`	Runs Conformal Low Power, before synthesis, to check the existing low power cells.
`-retention`	Reports only state retention related violations.
	Has priority over `-all` option.
`-run_dir` *directory*	

	Specifies the directory in which the required files for Encounter Conformal Low Power must be stored.
	Default: .
`-tcl_mode`	Specifies to generate the dofile as a Tcl script.

Related Information

Related command:

commit_power_intent

Affected by this attribute:

wclp_lib_statetable

commit_dft_power_intent

```
commit_dft_power_intent
                  [-dont_clone_dft_created_pins]
                  [-dont_update_iso_rules]
                  [-dont_update_retention_rules]
                  [-dont_run_commit_power_intent]
                  [-dont_run_optimize_iso_ls]
                  [design]
```

Description

Finalizes the power intent of the design after DFT logic insertion. For example, it ensures that each DFT-created hierarchical pin only drives a single power domain and that appropriate no-isolation and no-retention rules are defined for the DFT logic.

Options and Arguments

`-dont_clone_dft_created_pins`	
	Specifies not to clone the DFT created pins.
`-dont_update_iso_rules`	
	Specifies not to update the isolation rules on the DFT created pins.
`-dont_update_retention_rules`	
	Specifies not to update the retention rules on the DFT created instances.
`-dont_run_commit_power_intent`	
	Specifies not to commit power intent.
`-dont_run_optimize_iso_ls`	
	Specifies not to run optimized ISO level shifter.
`design`	Specifies the design to which DFT power intent command applies.

Related Information

Related commands:

commit_power_intent
read_power_intent
write_power_intent

commit_module_model

`commit_module_model`

Description

This command is used after the `set_module_model` command to load and commit the module models.

Related Information

Related commands:

set_module_model
read_module_model
write_module_model
assemble_design

commit_power_intent

```
commit_power_intent
[-design design]
```

Description

Inserts level-shifter logic and isolation logic as requested based on the rules specified in previously read in power intent file(s).

Options and Arguments

`-design` *design*	Specifies the design to which the power intent applies.
	This option is only required when multiple designs are loaded in the session.

Related Information

Related commands:

check_cpf
read_power_intent
write_power_intent

compare_crossing

```
compare_crossing
-from power_domain -to power_domain [-ls]
```

Description

Compares primary power supply of specified domains.

Options and Arguments

`-from` *power_domain*	Specifies the source power domain.
`-to` *power_domain*	Specifies the sink power domain.
`-ls`	Specifies to compare supplies for possible shift in voltages.

Related Information

Related commands:

read_power_intent
check_power_intent

copy_library_domain_attributes

```
copy_library_domain_attributes
-source source_library_domain -target target_library_domain
```

Description

This command copies the specified library domains attributes from source library domain to target library domain. Attributes which are copied mainly belong to `library`, `lib_cell`, `lib_pin`, `lib_arcs` object types.

All non-default attributes are copied using `copy_library_domain_attributes` command.

> There is no limitation on the number of attributes whose information is copied.

Options and Arguments

`-source` *source_library_domain*	
	Specifies the name of the source library domains whose attributes data are to be copied.
`-target` *target_library_domain*	
	Specifies the name of the target library domains where attributes data are to be copied.

Example

- The following example copies attribute from dom_1 library domain to dom_2 library domain:

  ```
  genus:root:> copy_library_domain_attributes -source dom_1 -target dom_2
  ```

Related Information

Related attributes:	library
	lib_cell
	lib_arcs
	lib_pins

cpi_insert_auto_ls

```
cpi_insert_auto_ls  -to_combo
[-design design] [-verbose]
```

Description

Inserts level shifters based on power states on crossings where level shifter rule is not given.

Options and Arguments

`-design` *design*	Specifies the design to which the power intent applies.
	This option is only required when multiple designs are loaded in the session.
`-to_combo`	Allows converting existing isolation cells to combinational isolation cells for level shifting.
`-verbose`	Enables verbose output

Related Information

Related commands:

commit_power_intent
read_power_intent
write_power_intent

create_library_domain

```
create_library_domain domain_list
```

Description

Creates the specified library domains. To use dedicated libraries with portions of the design, you must use this command before you read in any libraries for the specified library domains. The command returns the directory path to the library domains that it creates.

You can find the objects created by the `create_library_domain` command in:

```
/libraries/library_domains
```

```
root:.libraries.library_domains:...
```

> There is no limitation on the number of library domains you can create.

Options and Arguments

domain_list	Specifies the names of the library domains to be created. Specify the library domains as a Tcl list.

Examples

- The following example creates three library domains:

```
genus:root: XX> create_library_domain {dom_1 dom_2 dom_3}

library_domain:dom_1 library_domain:dom_2 library_domain:dom_3
```

- The following example creates two library domains(dom1, dom2) and assigns different libraries(lib1.lib, lib2.lib) for each domain:

```
create_library_domain dom1
library_domain:dom1

create_library_domain dom2
library_domain:dom2

set_db [get_db library_domains dom1] .library lib1.lib
Setting attribute of library_domain 'dom1': 'library' = lib1.lib
1 lib1.lib

set_db [get_db library_domains dom1] .library lib2.lib
Setting attribute of library_domain 'dom2': 'library' = lib2.lib
1 lib2.lib
```

Related Information

Create Library Domains in *Genus Synthesis Flows Guide.*

Related attribute:

library

define_design_partition

```
define_design_partition
[-instance hinst][-floor_plan string]
```

Description

Specifies the hierarchical instance to be partitioned and its bounding information. The specified instance should be a power domain boundary instance.

Options and Arguments

`-instance` *`hinst`*	Specifies the top-level hierarchical instance for partitioning.
`-floor_plan` *`string`*	Specifies the bounding boxes of the partition in µm.

eval_1801

```
eval_1801
upf_command... [-design design]
```

Description

Specifies an 1801 (UPF) command to be updated incrementally.

Options and Arguments

`-design` *`design`*	Specifies the design for which the command needs to be updated.
`upf_command`	Specifies the UPF command to be updated.

Example

```
eval_1801 set_isolation ISO_Test -domain PD_Test -applies_to inputs ...
```

optimize_iso_ls

```
optimize_iso_ls
[-design design]
```

Description

Removes any redundant level-shifter cells and isolation cells together with the connected supply nets.

Options and Arguments

`-design` *design*	Specifies the design from which to remove the redundant isolation cells and level-shifter cells.
	This option is only required when multiple designs are loaded in the session.

Related Information

Related command:

commit_power_intent

partition_design

```
partition_design  [-base_name string] [-scan_def] [-sdc] [-spef] [-verbose]
```

Description

Partitions the design using Innovus and writes block level power_intent.

Options and Arguments

-base_name *string*	Specifies the path and base filename for output data Default: genus_invs_pi_part/genus
`-scan_def`	Specifies to add an option in the setup to write the scan information
`-sdc`	Specifies to add an option in the setup to write_sdc.
`-spef`	Specifies to write the spef file
`-verbose`	Enables verbose output

read_power_intent

```
read_power_intent
 [-module module] [-1801 | -cpf] [-version string] [-verbose]
 file [file]...
```

Description

Reads the power intent for the module from the specified file(s).In general, the `read_power_intent` command does not change the netlist. With this command, the basic power attributes are set in the database.

In case CPF is specified using the -`cpf` option, additional checks are performed to ensure that the timing intent is specified in the MMMC format. Hence, the timing intent should be specified using the `read_mmmc` command. If timing intent is specified in the CPF file, it will be ignored and execution will be stopped with an error.

Options and Arguments

`[-1801 | -cpf]`	Specifies in which format the power intent files are written: the IEEE 1801(UPF) or the CPF format.
	Currently only the IEEE 1801-2009 version of the UPF standard is supported.
	Default: `-1801`
file	Specifies the name of the power intent file. The file can have any name, suffix, or length.
`-module` *module*	Specifies the top module to which the power intent is applied.
	This option is required in the following cases:
	• The file is read in before the design is read in.
	• The file is read after the design is loaded, but multiple designs were loaded in the session.
	By default, the tool assumes that the power intent applies to the (single) design that was read in.
`-version` *string*	Specifies the version of the power intent file. in UPF format
	Valid values are: `1.0`, `2.0`, `2.1`., `3.0` and `3.1`
	This option is only used if the UPF file is missing the `upf_version` command.
`-verbose`	Prints details along with verbosity.

Related Information

See the following section in *Genus Low Power Guide*: • Using CPF for Multiple Supply Voltage (MSV) Designs • Using CPF for Designs Using Power Shutoff (PSO) Methodology • Using UPF for Designs Using MSV and PSO	
Related attributes:	init_power_intent_files
Related commands:	check_cpf
	commit_power_intent
	write_power_intent

reset_power_intent

```
reset_power_intent
[-design design]
```

Description

Clears the power intent from the design.

> This command will not undo any netlist modifications made with the previous power intent.

Options and Arguments

`-design` *design*	Specifies the design from which to remove the power intent constraints.
	This option is only required when multiple designs are loaded in the session.

Related Information

Related commands:

check_cpf
read_power_intent

set_cglar_specification

```
set_cglar_specification -root  design
 [-elab_db  string] [-elab_tcl  string]
 [-reg_list string] [-locations_list string]
 [-d_slack_list string] [-cg_en_slack_list  string]
 [-apply_cglar_options string] [-compute_cglar_options string]
 [-report_cglar_options string] [-read_stimulus_options string] [-allow_mismatch_factor float]
 [-original_name] [-reset]  [-detailed_help] [-exclude_flops string]
```

Description

Performs auto cglar implementation during `syn_map` before MBCI.

Use the options to control the cglar implementation.

Options and Arguments

`-allow_mismatch_factor` *float*	
	Allows mismatch of flops from the list in the range of 0 to 1. *Default* is 0.2
`-apply_cglar_options` *string*	
	Specifies the custom `apply_cglar` command options.
`-cg_en_slack_list` *string*	
	Specifies the custom slack information at clock enable pin.
`-compute_cglar_options` *string*	

	Specifies the custom `-compute_cglar_options` option value of `apply_cglar` command.
`-d_slack_list` *string*	
	Specifies the custom slack information at data pin.
`-detailed_help`	Specifies to extract help on `compute_cglar` and `report_cglar` from Joules
`-elab_db` *string*	Specifies the path to elaborate database (db) file.
`-elab_tcl` *string*	Specifies the path to elaborate database Tcl file.
`-exclude_flops` *string*	
	Specifies a list of flops to be excluded.
`-locations_list` *string*	
	Specifies the custom location information.
`-original_name`	Specifies to use only `original_name` data.
`-reg_list` *string*	Specifies the list of flops.
`-read_stimulus_options` *string*	
	Specifies the custom `-read_stim_options` option value of `apply_cglar` command.
`-report_cglar_options` *string*	
	Specifies the custom `-report_cglar_options` option value of `apply_cglar` command.
`-reset`	Specifies to reset data.
`-root` *design*	Specifies the top design.

Example

```
set_cglar_specification -root [get_root] \
    -locations_list  {{{{int_diff[0]}}  1  1} {{{int_diff[1]}} 1 1} \
    {{{int_diff[2]}} 1 1} {{{int_sum[0]}} 1 1} {{{int_sum[1]}} 1 1}  \
    {{{int_sum[2]}} 1 1}} -d_slack_list {{{{int1[2]}} 100} \
    {{{int1[1]}} 100} {{{int1[0]}} 100} {{{int2[1]}} 100} \
    {{{int2[0]}} 100} {{{int2[2]}} 100}} \
-cg_en_slack_list {{{{regTmp1[0]}} 50} {{{regTmp1[1]}} 50}\
    {{{regTmp1[2]}} 50} {{{regTmp2[0]}} 50} {{{regTmp2[1]}} 50} \
    {{{regTmp2[2]}} 50}} \
-compute_cglar_options { -max_distance 50 -da_threshold 0.6 \
    -min_slack_threshold 0 -min_cg_en_slack_threshold 0 \
    -original_names_in_custom_lists} \
-report_cglar_options  {-ps_threshold  -1 -ps_absolute -1 \
    -cols {design_slack orig_slack user_slack design_location \
    user_location orig_cgen_slack design_cgen_slack user_cgen_slack} \-bit_blast -out
report_cglar.rpt -generate_script bank}
```

Related Information

Related commands:

apply_cglar
syn_map

write_power_intent

```
write_power_intent[-1801| -cpf] [-force]
[-base_name string] [-design design]
[-overwrite] [-to_macro] [-partition_assemble]
```

Description

Design objects that are specified in the power intent can be renamed during synthesis, for example during hierarchy ungrouping or user applied naming rules. This command writes out an updated power intent file reflecting the changes in either the IEEE 1801-2009 standard or in CPF format. The command returns the path to the file with the power intent information.

The updated power intent can be formally verified against the original in the Conformal Low Power tool using the compare power intent command.

This command has currently some limitations:

- All unsupported commands and commands that are not applicable to synthesis are written out as they were entered.
- All unsupported options of supported commands are skipped.
- If there are any errors or warnings given during `read_power_intent`, the power intent written out may not be same as the power intent read.
- If wildcards are used in net or port names specified with a command option, the result may not be as expected.

For more information on the command and command option support, refer to Appendix B: UPF (IEEE-1801) Support in *Genus Low Power Guide*.

Options and Arguments

`[-1801 | -cpf]`	Specifies in which format the power intent file must be written: the IEEE 1801-2009 standard, or the CPF format
	Default: `-1801`
`-base_name` *`string`*	Specifies the path and basename for the generated file.
`-design` *`design`*	Specifies the design for which you want to write out the power intent.
	If you omit this option, the constraints will be reapplied to the current design.
`-force`	Writes out UPF 1.0 in IEEE 1801 format.
	By default, when the input is UPF 1.0, the `write_power_intent` command does not write anything.
`-overwrite`	Allows to overwrite any existing files.

`-partition_assemble`	
	Generates power intent file for partitioned flow.
	This option is only available as a limited access feature.
`-to_macro`	Creates a CPF macro for the design.
	You can only create a CPF macro for the top-level design after you have synthesized the design and committed the CPF file.
	You must define the isolation and level-shifter rules for the *input* ports using the `-to` *`domain`* and `-pins` *`pins`* options.
	You must define the isolation and level-shifter rules for the *output* ports using the `-from` *`domain`* and `-pins` *`pins`* options.
	In addition, you should have checked the quality of your original CPF file after reading the CPF file, performed a low power equivalence check after committing the CPF file, and the results should be clean in both cases.

Related Information

See the following chapters in *Genus Low Power Guide*

- Using CPF for Multiple Supply Voltage (MSV) Designs
- Using CPF for Designs Using Power Shutoff (PSO) Methodology
- Using CPF for Designs Using Dynamic Voltage Frequency Scaling
- Using UPF for Designs Using MSV and PSO

Related commands:	check_cpf
	commit_power_intent
	read_power_intent

3

Analysis and Reporting

- analyze_library_corners
- check_design
- check_timing_intent
- compare_sdc
- create_timing_bin
- get_clock_ports
- get_fanin
- get_fanout
- get_socv_reporting_nsigma_multiplier
- report_analysis_views
- report_annotated_check
- report_area
- report_auto_ungroup_hierarchies
- report_boundary_opt
- report_case_analysis
- report_cell_delay_calculation
- report_chains_longest_wires
- report_chains_total_wirelength
- report_chains_wires
- report_clock_gates
- report_clock_gating_quality
- report_clock_groups
- report_clock_tree_structure
- report_clocks
- report_congestion
- report_core_wrapper_cell
- report_cwd_check
- report_delay_calculation
- report_design_rules
- report_dft_abstract_models
- report_dft_analyzed_test_points
- report_dft_trace_back
- report_dft_violations
- report_dont_touch
- report_dp
- report_flow
- report_gates
- report_hierarchy
- report_instance
- report_logic_levels_histogram
- report_loop
- report_memory
- report_memory_cells
- report_messages
- report_metric
- report_min_pulse_width
- report_mode
- report_module
- report_multibit_inferencing
- report_net_cap_calculation
- report_net_delay_calculation
- report_net_res_calculation
- report_nets
- report_obj
- report_opcg_clock_domain_info
- report_opcg_equivalents
- report_ple
- report_port
- report_power
- report_power_intent
- report_power_intent_instances
- report_prototype
- report_qor
- report_runtime
- report_scan_chains
- report_scan_compressibility
- report_scan_compression_logic
- report_scan_registers
- report_scan_setup
- report_sequential
- report_signoff_timing
- report_slew_calculation
- report_summary
- report_test_power
- report_timing
- report_timing_derate
- report_ungroup_modules
- report_units
- report_utilization
- report_yield
- save_snapshot
- set_activity
- set_socv_reporting_nsigma_multiplier
- time_info
- report_clock_gate_decloning
- report_instance_library

The following command is common with Joules commands:

- report_power
- report_flow

analyze_library_corners

```
analyze_library_corners
{-libraries list | -cpf file}
[-buffer_libcell lib_cell]
[-fanout integer] [-fanin integer]
 [> file]
```

Description

Reads in the specified multi-corner libraries and determines the slowest corner. Multi-corner libraries have the same libcells but are each characterized for a specific set of operating conditions resulting in different delay and slew values.

For each libcell the tool takes into account a given load at the input pins (specified through the number of buffers at the input) and a given load at the output pins (specified through the number of buffers at the output). Given that configuration, the tool calculates all timing arcs for the libcells for each corner and reports the average delay per corner. In addition, it reports the list of libcells whose delay exceeds the delay of the corresponding cell in the slowest library.

> This command should be the only command run in the synthesis session.

Options and Arguments

`-buffer_libcell` *cell*	
	Specifies the libcell to be used as buffer.
`-cpf` *file*	Specifies the name of the CPF file that has the libraries for the multi corners.
`-fanin` *integer*	Specifies the number of buffers to consider in the fanin of the input pins of each libcell.
	Default: `2`
`-fanout` *integer*	Specifies the number of buffers to consider in the fanout of the output pins of each libcell
	Default: `10`
`-libraries` *list*	Specifies the list of multi-corner libraries.

Example

The following example analyzes libraries typical.lib and slow.lib:

```
analyze_library_corner -libraries "typical.lib slow.lib"
```

Output:

Library Filename Average delay

---

library:default_emulate_libset_max/typical 192
library:default_emulate_libset_max/slow 148

The slowest library: library:default_emulate_libset_max/typical

The average delay for this lib: 192

Libcell Library Delay Slowest Library Delay

TLATSRX4 library:default_emulate_libset_max/slow 195 library:default_emulate_libset_max/typical 185

OAI222X1 library:default_emulate_libset_max/slow 208 library:default_emulate_libset_max/typical 194

NAND3BX4 library:default_emulate_libset_max/slow 86 library:default_emulate_libset_max/typical 81

---

check_design

```
check_design [-lib_lef_consistency]
[-undriven [-threshold_fanout integer]]
[-multiple_driver] [-unloaded] [-unloaded_comb]
[-unresolved] [-cross_hier] [-collection] [-assigns] [-combo_loops]
[-constant [-threshold_fanout] [-through_tie_cell]]
[-preserved] [-logical_only] [-physical_only]
[-only_user_hierarchy] [-vname] [-feedthrough]
[-report_scan_pins | -skip_scan_pins] [-skip_async_pins]
[-long_module_name] [-status] [-all] [design] [> file]
```

Description

Provides information on undriven and multi-driven ports and pins, unloaded sequential elements and ports, unresolved references, constants connected ports and pins, any assign statements and preserved instances in the given design. In addition, the command can report any cells that are not in both .lib and the physical libraries (LEF files). By default, if you do not specify an option, the `check_design` command reports a summary table with this information.

Options and Arguments

`-all`	Reports all information for the design with a summary at the end.
`-assigns`	Reports assign statements in the design.
`-combo_loops`	Returns 1 if a combinational cycle is detected.
`-constant`	Reports ports and pins in the design that are connected to a constant.
`-collection`	Reports all information as tcl_list.
design	Specifies the name of the design to write the report.
	This option is required when multiple designs are loaded.
`-feedthrough`	Reports feed-through modules in the design.
`-cross_hier`	This option is used in undriven, multi-driven, constant driven, and unloaded driver computation using cross hierarchy checks.
file	Specifies the name of the file to write the report.
`-lib_lef_consistency`	
	Reports the cells that are present in .lib but not in the LEF file(s) and vice versa.
`-logical_only`	Reports all instances of logical-only libcells, that is, library cells that are only available in a .lib library.
`-long_module_name`	Reports modules whose name exceeds 1.5K.
`-multiple_driver`	Reports ports and pins in the design that are multi-driven.
`-only_user_hierarchy`	
	Skips the tool generated internal hierarchies (such as Mux and Adders) when performing checks and only reports checks on user-created hierarchies.
`-physical_only`	Reports all instances of physical-only libcells, that is, library cells that are only available in a LEF library.
`-preserved`	Reports all hierarchical and leaf instances in the design for which the `preserve` attribute is set to `true`.

`-report_scan_pins`	Includes the scan (DFT-related) pins in the checks and reports.
`-skip_async_pins`	Excludes the asynchronous pins of flops and latches when reporting on undriven or unloaded pins.
`-skip_scan_pins`	Excludes the scan (DFT-related) pins from the checks and reports.
	By default, the scan pins are included in all checks and reports.
`-status`	Returns 1 if any instances were reported, otherwise returns 0.
`-threshold_fanout`	Filters undriven or constant pins and ports with a fanout below the specified threshold.
	This option must be specified with the `-constant` option.
`-through_tie_cell`	Reports pins and ports connected to a constant through tie high/low cells.
	This option must be specified with the `-constant` option.
`-undriven`	Reports ports and pins in the design that are undriven.
`-unloaded`	Reports ports and sequential elements in the design that are unloaded.
`-unloaded_comb`	Reports the combinational elements in the design that are unloaded.
`-unresolved`	Reports unresolved references and empty modules in the design.
`-vname`	Uses the Verilog names instead of Genus design hierarchy path names in the report.

Examples

- The following example shows a sample report given when the command is executed without any options.

```
genus:root:> check_design
  Checking the design.

        Check Design Report
        -------------------

 Summary
 -------
                Name                 Total
-------------------------------------------
Unresolved References                    0
Empty Modules                            0
Unloaded Port(s)                         0
Unloaded Sequential Pin(s)               0
Assigns                                  0
Undriven Port(s)                         0
Undriven Leaf Pin(s)                     0
Undriven hierarchical pin(s)             1
Multidriven Port(s)                      0
Multidriven Leaf Pin(s)                  0
Multidriven hierarchical Pin(s)          0
Multidriven unloaded net(s)              0
Constant Port(s)                         0
Constant Leaf Pin(s)                     0
```

```
Constant hierarchical Pin(s)             0
Preserved leaf instance(s)               5
Preserved hierarchical instance(s)       1
Libcells with no LEF cell              601
Physical (LEF) cells with no libcell   846

  Done Checking the design.
```

- The following example reports the preserved instances in your design.

```
genus:root:> check_design -preserved
  Checking the design.

         Check Design Report
         -------------------

  Preserved instance(s)
  ---------------
design 'test1' has the following preserved combinational instance(s)
inst:test1/and1
inst:test1/and2
inst:test1/and3
inst:test1/U1/and4
Total number of preserved combinational instances in design 'test1' : 4
design 'test1' has the following preserved sequential instance(s)
inst:test1/dff1
Total number of preserved sequential instances in design 'test1' : 1
design 'test1' has the following preserved hierarchical instance(s)
inst:test1/U1
Total number of preserved hierarchical instances in design 'test1' : 1

  Done Checking the design.
```

- The following command reports the cells which are only in .lib or in the physical library (LEF files).

```
genus:root:> check_design -lib_lef_consistency
  Checking the design.

         Check Design Report
         -------------------

Libcells with no corresponding LEF
----------------------------------
lib_cell:mylib.slow/ACCSHCINX2
...
lib_cell:mylib.slow/p_SMDFFHQX8

Total number of cell(s) with only library (.lib) info : 601

LEF cells with no corresponding libcell
---------------------------------------
lib_cell:physical_cells/AN2D0
...
lib_cell:physical_cells/XOR4D4
Total number cell(s) with only physical (LEF) Info : 846

  Done Checking the design.
```

- The following command reports all undriven pins including the DFT-related pins (highlighted in the report below).

```
genus:root:> check_design -undriven
  Checking the design.
```

```
         Check Design Report
        --------------------

 Undriven Port(s)/Pin(s)
 -------------------------
No undriven combinational pin in design 'test1'

The following sequential pin(s) in design 'test1' are undriven
pin:test1/dff1/SE
pin:test1/dff1/SI
Total number of sequential undriven pins in design 'test1' : 2

The following hierarchical pin(s) in design 'test1' are undriven
hpin:test1/U1/A       (fanout : 0)
Total number of hierarchical undriven pins in design 'test1' : 1

No undriven port in 'test1'

  Done Checking the design.
```

- The following command reports all undriven pins *excluding* the DFT-related pins.

```
genus:root:> check_design -undriven -skip_scan_pins
  Checking the design.

         Check Design Report
        --------------------

 Undriven Port(s)/Pin(s)
 -------------------------
No undriven combinational pin in 'test1'

No undriven sequential pin in 'test1'

The following hierarchical pin(s) in design 'test1' are undriven
hpin:test1/U1/A       (fanout : 0)
Total number of hierarchical undriven pins in design 'test1' : 1

No undriven port in 'test1'

  Done Checking the design.
```

- The following command reports physical-only instances.

```
genus:root:> check_design -phys
  Checking the design.

         Check Design Report
         --------------------

  Physical only instances(s)
  ----------------
design 'rct' has the following physical only instance(s)
inst:rct/ram_bank0
isnt:rct/ram_bank1
inst:rct/ram_sort

  Done Checking the design.
```

check_timing_intent

```
check_timing_intent
[-view analysis_view]
[-verbose]
```

Prints a timing lint report.

Description

`check_timing_intent` checks the SDC quality and timing attributes placed on the current design and accordingly issues warnings as needed. These warnings do not indicate design has problems, instead indicate that the SDC has potential errors.

> It is strongly recommended to execute this command after `read_sdc` command, and fix these warnings before downstream run. You need to double check each warning to confirm whether to ignore or to fix it.

By default, this command reports verbose information for each warning.

The checks included are:

- Unconnected or logic driven clocks
- Sequential data pins driven by a clock signal
- Sequential clock pins without clock waveform
- Sequential clock pins with multiple clock waveforms
- Generated clocks without clock waveform
- Generated clocks with incompatible options
- Generated clocks with multi-master clock
- Paths constrained with different clocks
- Loop-breaking cells for combinational feedback
- Nets with multiple drivers
- Timing exceptions with no effect
- Suspicious multi_cycle exceptions
- Pins/ports with conflicting case constants
- Inputs without clocked external delays
- Outputs without clocked external delays
- Inputs without external driver/transition
- Outputs without external load
- Exceptions with invalid timing start or endpoints

Options and Arguments

-verbose	Prints a verbose report.
`-view analysis_view`	
	Specifies that the report should only apply to the specified *analysis_view*.

compare_sdc

```
compare_sdc [-design string] [-rtl | -netlist file]
-golden_sdc files [-revised_sdc files]
[-file file] [-detail] [> file]
```

Description

Compares two (or two sets of) SDC files for a design and generates a report containing differences.

To run this command you need to have access to the Encounter ® Conformal ® Constraint Designer (CCD) software.

Options and Arguments

`-design string`	Specifies the top-level design in Genus.		
`-detail`	Requests a detailed comparison report.		
file	Specifies the file to which the report must be written.		
`-file file`	Specifies the name of the CCD logfile. You must specify the UNIX path to the file.		
		`-netlist file`	Specifies the UNIX path to the netlist. By default, the tool uses the RTL.
			If this option is not specified, the tool internally generates an SDC file for the current state of the design and uses this file for the comparison.
`-rtl`	Specifies to use the RTL as input.		

create_timing_bin

```
create_timing_bin -name string
[-parent string]
[-num_paths integer] [-worst integer]
[-slack_limit delay]
[-from {inst|hinst|external_delay|clock|port|pin|hpin}...]
[-through {inst|hinst|port|pin|hpin}..]
[-to {inst|hinst|external_delay|clock|port|pin|hpin}...]
[-exceptions exception...] [-cost_group cost_group...]
[-paths string][-view analysis_view]
[-not_through {inst|hinst|external_delay|clock|port|pin|hpin}..]
```

Description

Allows you to select and save timing paths and their associated timing information, into a timing bin. Timing bins enable you to subsequently examine and analyze the timing paths without re-invoking the timer.

The data in a timing bin is a static snapshot created when the `create_timing_bin` command was invoked. If the design or constraints change, you can create new timing bin(s) and delete the old one(s).

Clock ports can be identified by the `is_clock` attribute present on the port.

Options and Arguments

`-cost_group` *cost_group*	
	Selects only paths for the specified cost groups.
`-exceptions`	Selects only paths to which one of the specified exceptions applies.
	Timing exceptions are specified through one of the following Genus commands (or their SDC equivalent): `path_adjust` or `path_group`.
	This option can be combined with `-from`, `-through`, `-to` options to further restrict the path selection.
`-from {`*pin*`|`*hpin*`|`*port*`|`*clock*`|`*external_delay*`|`*inst*`|`*hinst*`}`	
	Specifies a Tcl list of start points for the paths. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which specified external delay timing exceptions apply.
`-name` *string*	Specifies the name of the timing bin to be created.
`-not_through {`*pin*`|`*hpin*`|`*external_delay*`|`*clock*`|`*port*`|`*inst*`|`*hinst*`}`	
	Specifies a Tcl list of points that a path must not traverse. Points to avoid can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances, external delays or clocks.
`-num_paths` *integer*	Specifies the maximum number of paths to include in the timing bin.
`-parent` *string*	Specifies the name of the timing bin from which you want to derive this bin.
`-paths` *string*	Includes the selected timing restricted paths. Create the string argument using the set_path_specification command.
`-slack_limit` *delay*	Includes only paths with a slack less than the specified value (in picoseconds).
`-through {`*pin*`|`*hpin*`|`*port*`|`*inst*`|`*hinst*`}`	
	Specifies a Tcl list of points that a path must traverse. Points to traverse can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances.
	You can repeat the `-through` option to require that a path first must traverse one of the objects in the first set, then pass through one of the objects in the second set, and so on.
`-to {`*pin*`|`*hpin*`|`*port*`|`*clock*`|`*external_delay*`|`*inst*`|`*hinst*`}`	
	Specifies a Tcl list of end points for the paths. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which specified external delay timing exceptions apply.
`-view` *analysis_view*	
	Specifies an *analysis_view*, the paths should be restricted to.
`-worst` *integer*	Specifies the number of worst paths to each endpoint to include.

Related Information

Creates these attributes	Timing Bin Attributes
	Timing Path Attributes

get_clock_ports

```
get_clock_ports [design]
```

Description

Returns input ports of your design that are clock inputs.

> Only input ports at the top level are listed. Gated clocks and clock pins that are present in the hierarchical design internally (typical PLL outputs) will not be identified.

> This command is useful when you are working with an unfamiliar design.

Options and Arguments

design	Specifies the design for which you want to list the clock input ports.

get_fanin

```
get_fanin {pin|hpin|pg_pin|port|hport|net|hnet}...
[-max_logic_depth integer] [-min_logic_depth integer]
[-max_pin_depth integer] [-min_pin_depth integer]
[-structural [-stop_at_seqs] [-trace_through_combinational]
  [-floating_hport] [-ignore_broken_timing_paths] ]
[-startpoints] [-gui]
```

Description

Returns pins and ports in the fanin cone for the specified objects (pins, ports, or nets) starting from the nearest timing startpoints based on the timing arcs of the libcells of the instances. Use the `-structural` option to do a connectivity trace (ignoring timing arcs and functions) with every output of an instance having a "structural connection" to every input and vice-versa.

Options and Arguments

`-floating_hport`	Reports only floating (unconnected) hports found in the connectivity-based structural trace. You can only specify this option with the `-structural` option.
	By default, only non-floating subports and constants are returned.
`-gui`	Highlights the instances along the fanin cone in the Physical Viewer. The instance at the start of the fanin cone is marked in yellow, the other instances are marked in red.
	> The GUI should be enabled to have any effect.
`-ignore_broken_timing_paths`	
	Ignores broken timing paths during a connectivity-based structural trace. You can only specify this option with the `-structural` option.
`-max_logic_depth integer`	
	Specifies the maximum number of logic levels to go back to report the fanin cone information.
	Default: Infinity
`-max_pin_depth integer`	

	Specifies the maximum number of pin levels to go back to report the fanin cone information.
	Default: Infinity
`-min_logic_depth` *integer*	
	Specifies the minimum number of logic levels to go back to report the fanin cone information.
	Default: `0`
`-min_pin_depth` *integer*	
	Specifies the minimum number of pin levels to go back to report the fanin cone information.
	Default: `0`
{*pin* | *hpin* | *pg_pin* | *port* | *hport* | *net* | *hnet*}	
	Specifies the name of a pin, hpin, pg_pin, port, hport, net, or hnet for which you want the fanin cone information.
`-startpoints`	Returns only timing start points in the fanin cone.
`-stop_at_seqs`	Stops the structural trace at sequential elements.
`-structural`	Specifies to perform a connectivity-based structural trace instead of the default timing trace based on timing arcs.
	If there are missing timing arcs, for example, when using the SDC `set_case_analysis` command, the traces may report different results.
	Use this option with care as it can increase runtime considerably.
`-trace_through_combinational`	
	Specifies to trace through combinational timing model libcells. You can only specify this option with the `-structural` option.

get_fanout

```
get_fanout {pin|hpin|pg_pin|port|hport|net|hnet}...
[-min_logic_depth integer]
[-max_logic_depth integer]
[-min_pin_depth integer]
[-max_pin_depth integer]
[-structural [-stop_at_seqs] [-trace_through_combinational]
  [-floating_hport] [-ignore_broken_timing_paths] ]
[-endpoints] [-gui]
```

Description

Returns pins and ports in the fanout cone for the specified objects (pins, ports, or nets) stopping at the nearest timing endpoints based on the timing arcs of the libcells of the instances. Use the `-structural` option to do a connectivity trace (ignoring timing arcs and functions) with every output of an instance having a "structural connection" to every input and vice-versa.

Options and Arguments

`-endpoints`	Returns only timing end points in the fanout cone.
`-floating_hport`	Reports only floating (unconnected) subports found in the connectivity-based structural trace. You can only specify this option with the `-structural` option.
	By default only non-floating subports and constants are returned.

`-gui`	Highlights the instances along the fanout cone in the Physical Viewer. The instance at the start of the fanout cone is marked in yellow, the other instances are marked in red.
	The GUI should be enabled to have any effect.
`-ignore_broken_timing_paths`	
	Ignores broken timing paths during a connectivity-based structural trace.
`-max_logic_depth` *integer*	
	Specifies the maximum number of logic levels to go back to report the fanout cone information.
	Default: Infinity
`-max_pin_depth` *integer*	
	Specifies the maximum number of pin levels to go back to report the fanout cone information.
	Default: Infinity
`-min_logic_depth` *integer*	
	Specifies the minimum number of logic levels to go back to report the fanout cone information.
	Default: `0`
`-min_pin_depth` *integer*	
	Specifies the minimum number of pin levels to go back to report the fanout cone information.
	Default: `0`
{*pin* | *hpin* | *pg_pin* | *port* | *hport* | *hnet* | *net*}	
	Specifies the name of a pin, hpin, pg_pin, port, hport, or hnet for which you want the fanout cone information.
`-stop_at_seqs`	Stops the structural trace at sequential elements. You can only specify this option with the `-structural` option.
`-structural`	Specifies to perform a connectivity-based structural trace instead of the default timing trace based on timing arcs.
	If there are missing timing arcs, for example, when using the SDC `set_case_analysis` command, the traces may report different results.
	Use this option with care as it can increase runtime considerably.
`-trace_through_combinational`	
	Specifies to trace through combinational timing model libcells. You can only specify this option with the `-structural` option.

get_socv_reporting_nsigma_multiplier

`get_socv_reporting_nsigma_multiplier`

Description

Retrieves the sigma multiplier factors for all the analysis views.

report_analysis_views

```
report_analysis_views [-type {all | setup | hold | active} ]
```

Description

Generates a report for the current multi-mode multi-corner configuration. By default, the `report_analysis_views` command generates a report for all the defined views in the design. Use the `-type` parameter to generate a report on a specific type of analysis view.

Use this command after creating analysis views using the `create_analysis_view` command and setting up active setup and hold views using the `set_analysis_view` command, for the design.

Options and Arguments

`-type {all | setup | hold | active}`	
	Specifies the type of report to generate. It can have any of the following values:
	`active`: Generates a report for all active setup and hold views in the design.
	`all`: Generates a report for all the defined views in the design. These include even the inactive views. This is the default.
	`hold`: Generates a report for all the active hold views in the design.
	`setup`: Generates a report for all the active setup views in the design.
	Default: all

Related Information

Related commands:

create_analysis_view
set_analysis_view

report_annotated_check

```
report_annotated_check [-check_type string] [-list_annotated] [-list_non_annotated] [-view analysis_view] [inst|hinst+] [> file]
```

Description

Reports coverage of annotated timing check. If you specify this command without any parameters, reports are generated annotated checks for the all instances in the design.

Options and Arguments

`-check_type string`	
	Allows you to specify the check type. The valid check types are setup, pulse_width, and recovery.
`-list_annotated`	Lists the detailed annotation information for the setup, recovery, and pulse-width timing checks.

`-list_non_annotated`	
	Lists the timing checks that do not have annotated delay information. The software checks for the setup, recovery, and pulse-width timing checks.
-view *analysis_view*	
	Reports coverage of annotated timing checks for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. The default behavior is to report annotated timing checks for the default analysis view.
inst|hinst	Specifies the instances to report.

Examples

- The following example shows how to report the annotated checks on all instances in the design.

  ```
  genus@root: report_annotated_check
  ```

- The following example shows how to report the annotated check on a instance.

  ```
  genus@root: report_annotated_check [instance_name]
  ```

Related Information

Related command	set_annotated_check

report_area

```
report_area
[-depth integer] [-hinst inst|hinst] [-min_count integer] [-min_area float] [-physical] [-gates] [-normalize_with_gate lib_cell] [-show_module_names] [-
show_full_names] [-show_leaf_cells] [-skip_modules_logical_hier_false] [-summary] [-detail] [-table_style string] [-show_msv_cells] [design]... [> file]
```

Description

Reports the following information:

- The total count of cells mapped against the hierarchical blocks in the current design
- The combined cell area in each of the blocks and the top level design (hierarchical breakup)
  The `Cell Area` numbers are based on the cell implementations taken from the technology library after mapping. However, in PLE mode, the numbers are based on the information in the LEF libraries. It might be 0 if the information is missing in the LEF libraries.
- The `Net Area` refers to the estimated post-route net area and is only meaningful if you read in the LEF libraries. Net area is based on the minimum wire widths defined in the LEF and capacitance table files and the area of the design blocks.
- The wireload model adopted for each of the blocks

The units used in the report depend on the units used in the library.

Options and Arguments

`-depth` *integer*	Specifies the number of levels of recursion. Default value is 100.
design	Specifies the design for which you want to generate a report. You can also `cd` into the particular design directory and generate the report.
`-detail`	Reports detailed area information. Reports area for different categories, such as, combinational cells, sequential cells, buffers, inverters, clock-gating instances, and so on.
`-table_style` *string*	

	Prints the cell info vertically or horizontally. This option is only supported with `-detail` option and by default cell info is printed horizontally. With `-table_style`, if we provide vertical (string option) then it reports vertically.
`-show_msv_cells`	Prints multiple supply voltage cells categorization.
`file`	Specifies the name of the file to which to write the report.
`-gates`	Prints the area of all combinational cells, sequential cells, and hierarchical instances.
`-hinst` *`inst|hinst`*	Reports the area of the leaf instances in the specified hierarchical instance. This option must be specified together with the `-depth` option.
`-min_count` *`integer`*	Specifies the minimum instance count per block.The default value is 1.
`-min_area` *`float`*	Specifies the minimum area per block.
`-normalize_with_gate` *`lib_cell`*	
	Normalizes all area numbers in the report with the area of the specified library cell.
-show_full_names	Prints the hierarchical name.
`-physical`	Specifies to use the LEF cell width and height values to calculate the area.
`-show_leaf_cells`	Prints the leaf instances at the design level in the report.
-show_module_names	Prints the module name.
`-skip_modules_logical_hier_false`	
	Skips reporting those modules for which the `logical_hier` attribute has the `false` value.
`-summary`	Prints the area summary for the design.

Examples

- The following example generates the area report for the current design.

```
genus@root: number> report_area
================================================================
  Generated by:          Genus Synthesis Solution version
 Generated on: Current date Current time
 Module: mult_bit_muxed_add Technology library: tutorial Operating conditions: _nominal_ (balanced_tree) Wireload mode: enclosed Area
mode: timing library ================================================================ ************* Area ************* Instance
Cells Cell Area Net Area Wireload ------------------------------------------------------------------ mult_bit_muxed_add 6 69 0
suggested_10K (S) ma1 3 35 0 suggested_10K (S) ma0 3 35 0 suggested_10K (S) (S) = wireload was automatically selected
```

- The following command prints the area summary.

```
genus@root: number> report_area -summary
============================================================
...
============================================================

     Instance       Cells  Cell Area  Net Area     Wireload
-----------------------------------------------------------------------
mult_bit_muxed_add      6         69         0  suggested_10K (S)
```

- The following command reports the area for the leaf instances of instance `ma1`.

```
genus@root: number> report_area -hinst ma1 -depth 2
============================================================
...
============================================================

Instance  Cells  Cell Area  Net Area     Wireload
-----------------------------------------------------------
  ma1         3         35         0  suggested_10K (S)
    g53       0         12         0  suggested_10K (S)
    g52       0         12         0  suggested_10K (S)
    g51       0         10         0  suggested_10K (S)

 (S) = wireload was automatically selected
```

- The following example shows the effect of normalizing the report. The first report is a regular report, while the 2nd report shows the normalized report.

```
genus@root: number> report_area
============================================================
...
============================================================

    Instance              Cells  Cell Area  Net Area   Total Area      Wireload


-------------------------------------------------------------------------------------------counter                      340       8759         1         8760
wireload  monitor_power              234       5864         1         5864      wireload    mux_nst_120_12             6       1116         0         1116
wireload    mux_nst_adder_178_12       6        956         0          956      wireload    ctl_cst_120_12            28        345         0          345
wireload...


genus@root: number> report_area  -normalize_with_gate default_emulate_libset_max/typical/NAND2BX2


Info    : Area report is normalized with area of the specified reference cell.[RPT-70]
    : Reference cell used for area normalization is lib_cell:default_emulate_libset_max/typical/NAND2BX2.


============================================================...================================================================
Instance                    Cells  Normalized Cell Area  Normalized Net Area  Normalized Total Area Wireload----------------------------------------------------
-----------------------------------------------------------
```

```
counter                   340                  860                       0                  860 wireload   monitor_power           234
576                         0                  576 wireload    mux_nst_120_12          6                  110                         0
110 wireload    mux_nst_adder_178_12    6                   94                         0                   94 wireload    ctl_cst_120_12         28
34                          0                   34 wireload
```

Related Information

Affected by these commands:	syn_generic
	syn_map
Affected by these attributes	logical_hier
	shrink_factor

report_auto_ungroup_hierarchies

```
report_auto_ungroup_hierarchies
[design]
[> file]
```

Description

Reports the hierarchies which are skipped ungrouping when '`opt_skip_ungroup_with_pin_or_leaf_exception`' is set to true.

Options and Arguments

design	Specifies the design for which you want to generate a report.

report_boundary_opt

```
report_boundary_opt [inst|hinst]...
```

Description

Reports a summary of the boundary optimization done on the design.

This is a summary of boundary changes on the hierarchical pins.

Options and Arguments

inst|hinst	Reports the boundary optimization done on the specified hierarchical instance.

report_case_analysis

```
report_case_analysis design
```

Description

Reports ports and pins that have a set_case_analysis constraint.

This command generates a User Case Analysis table, which reports the ports and pins that have a `set_case_analysis` constraint. These ports and pins are

constant only for timing analysis.

You can optionally generate a Logic Constant Value table, which reports all the ports and pins that have a constant status as part of the design. Information from this table includes ports or pins with `set_logic constraints` (`set_logic_zero` and `set_logic_one`), netlist explicit constants, and output pins with constant output functions.

Options and Arguments

design	Specifies the design for which to generate the report.

Example

```
report_case_analysis design1

------------------------------Report case_analysis ------------------------------Pin-Name Case --------------TE         0 MUL1/se    1
```

Related Information

Related commands:	set_case_analysis

report_cell_delay_calculation

```
report_cell_delay_calculation
-from_pin {pin|hpin}  -to_pin {pin|hpin}
[-from_rise] [-from_fall]
[-to_rise] [-to_fall]
[-view analysis_view] [> file]
```

Description

Reports how the cell delay is calculated from the look up table in the loaded technology library. Specify the cell by choosing its the driving and loading pins.The formula for calculating the delay is provided at the bottom of the report.

Options and Arguments

file	Redirects the report to the specified file.
-from_pin {*pin*|*hpin*}	
	Specifies the driving pin.
-from_fall	Uses the fall delay calculation from the driving pin.
-from_rise	Uses the rise delay calculation from the driving pin.
-to_fall	Uses the fall delay calculation of the loading pin.
-to_pin {*pin*|*hpin*}	Specifies the loading pin.
-to_rise	Uses the rise delay calculation of the loading pin.
-view *analysis_view*	
	Specifies that the report should only apply to the specified *analysis_view*.

report_chains_longest_wires

```
report_chains_longest_wires
[-num_wires integer]
[-section_type {floating|ordered|both}] [-gui]
```

Description

Reports the longest scan path connections across all the scan chains. Distances are reported in microns.

This command can be used on a placed design that has connected scan chains.

Options and Arguments

`-gui`	Highlights the airlines of the reported connections in the GUI.
`-num_wires integer`	Specifies the number of long wires to report.
	Default: `10`
`-section_type {floating|ordered|both}`	
	Reports wires that are reorderable (floating) or fixed (ordered)
	Default: `both`

Example

- The following example reports ten wires with a total distance of 5523.060.

```
genus:root:> report_chains_longest_wires -gui

Listing 10 longest scan wires within both floating and ordered sections
Distance   Location        Chain       Segment     Scan
657.900  {300.96 35.28}  AutoChain_1  AutoChain_1  MAR_REG_STORAGE_Q_N52_reg[12]         {485.1 509.04}  AutoChain_1  AutoChain_1
THE_REG_FILE_REG.REGISTER_FILE_reg[932]611.220  {308.22 120.96} AutoChain_1  AutoChain_1  C_REG_STORAGE_Q_N57_reg[7]         {69.96 493.92}  AutoChain_1
AutoChain_1  THE_REG_FILE_REG.REGISTER_FILE_reg[828]587.220  {510.18 524.16} AutoChain_1  AutoChain_1  THE_REG_FILE_REG.REGISTER_FILE_reg[865]         {300.96
146.16} AutoChain_1  AutoChain_1  A_REG_STORAGE_Q_N46_reg[18]569.340  {467.94 504.0}  AutoChain_1  AutoChain_1  THE_REG_FILE_REG.REGISTER_FILE_reg[0]
{402.6 0.0}     AutoChain_1  AutoChain_1  INSTR_REG_STORAGE_Q_N38_reg[26]566.520  {522.72 25.2}   AutoChain_5  AutoChain_5  DFT_lockup_latch_neg_edge185345
{339.24 408.24} AutoChain_5  AutoChain_5  THE_REG_FILE_REG.REGISTER_FILE_reg[59]566.100  {529.32 277.2}  AutoChain_5  AutoChain_5
THE_REG_FILE_REG.REGISTER_FILE_reg[300]         {8.58 322.56}   AutoChain_5  AutoChain_5  THE_REG_FILE_REG.REGISTER_FILE_reg[328]511.140  {42.9 15.12}
AutoChain_1  AutoChain_1  PC_REG_STORED_VALUE_reg[8]         {518.76 50.4}   AutoChain_1  AutoChain_1  DFT_lockup_latch_neg_edge490.920  {189.42 478.8}
AutoChain_1  AutoChain_1  THE_REG_FILE_REG.REGISTER_FILE_reg[710]         {428.34 226.8}  AutoChain_1  AutoChain_1
THE_REG_FILE_REG.REGISTER_FILE_reg[268]489.300  {22.44 322.56}  AutoChain_5  AutoChain_5  THE_REG_FILE_REG.REGISTER_FILE_reg[463]         {461.34 372.96}
AutoChain_5  AutoChain_5  THE_REG_FILE_REG.REGISTER_FILE_reg[515]473.400  {518.76 50.4}   AutoChain_1  AutoChain_1  DFT_lockup_latch_neg_edge         {498.96
504.0}  AutoChain_1  AutoChain_1  THE_REG_FILE_REG.REGISTER_FILE_reg[961]
```

Related Information

Affected by this command:	create_chains_database

report_chains_total_wirelength

```
report_chains_total_wirelength
[-sort {floating|ordered|chains|total}] [-gui]
```

Description

Divides the wirelength for each scan chain into two categories:

- The *ordered* wirelength includes wires between components that are not reorderable.
- The *floating* wirelength lists the wirelength between components that are reorderable.

Distances are reported in microns.

This command can be used on a placed design that has connected scan chains.

Options and Arguments

`-gui`	Highlights the chains in the GUI.
`-sort {floating|ordered|chains|total}`	
	Specifies which column to use for the sorting.
	Default: `total`

Example

- The following example shows the report sorted by chain names.

```
report_chains_total_wirelength -sort chains

Chain                          Floating Length   Ordered Length   Total Length

AutoChain_1:                       9188.280        0.000           9188.280
AutoChain_5:                       7423.620        0.000           7423.620
AutoChain_8:                       6788.400        0.000           6788.400
AutoChain_3:                       6695.220        0.000           6695.220
AutoChain_2:                       6198.660        0.000           6198.660
AutoChain_4:                       5983.020        0.000           5983.020
AutoChain_7:                       5824.440        0.000           5824.440
AutoChain_6:                       5473.200        0.000           5473.200
Total:                            53574.840        0.000          53574.840
```

Related Information

Affected by this command:

create_chains_database

Related commands:

create_chains_database
report_chains_longest_wires
report_chains_wires

report_chains_wires

```
report_chains_wires
[-chain actual_scan_chain]
[-color string] [-scandef_chain_name name]
```

Description

Lists the Manhattan distance between every scan element in the specified chain. In addition, the chain can be highlighted in the GUI using airlines between the instances. Distances are reported in microns.

This command can be used on a placed design that has connected scan chains.

Options and Arguments

`-chain actual_scan_chain`	
	Specifies the scan chain for which to report the wire-lengths.
`-color string`	Specifies the color to use to highlight the specified chain.
`-scandef_chain_name name`	
	Specifies the chain name in the scanDEF file.

Example

The following command shows the details for chain `AutoChain_1`.

```
report_chains_wireS -chain AutoChain_1


Wirelength for chain: AutoChain_1
SI pin: DFT_sdi_1 Location: no_value

Dist To   Type      Parent         Location        Element0.000    floating  AutoChain_1     no_value        COMPACTOR/msi[7]0.000    floating  AutoChain_1
no_value        COMPACTOR/mso[7]0.000    floating  AutoChain_1    {360.36 161.28}  DFT_lockup_latch_neg_edge18549786.400   floating  AutoChain_1    {344.52
90.72}   PC_REG_STORED_VALUE_reg[23]281.940  floating  AutoChain_1    {143.22 10.08}   PC_REG_STORED_VALUE_reg[18]19.320   floating  AutoChain_1    {152.46 0.0}
PC_REG_STORED_VALUE_reg[22]29.040   floating  AutoChain_1    {123.42 0.0}     PC_REG_STORED_VALUE_reg[29]24.420   floating  AutoChain_1    {99.0 0.0}
PC_REG_STORED_VALUE_reg[25]26.340   floating  AutoChain_1    {87.78 15.12}    PC_REG_STORED_VALUE_reg[15]6.360    floating  AutoChain_1    {89.1 20.16}
PC_REG_STORED_VALUE_reg[14]........5.040    floating  AutoChain_1    {146.52 372.96} THE_REG_FILE_REG.REGISTER_FILE_reg[894]12.060   floating  AutoChain_1
{148.5 362.88}  THE_REG_FILE_REG.REGISTER_FILE_reg[985]146.340  floating  AutoChain_1    {249.48 317.52} THE_REG_FILE_REG.REGISTER_FILE_reg[54]0.000
Scan Out Pin    no_value        DFT_sdo_1
Total floating: 9188.280Total ordered: 0.000
Total: 9188.280
```

The report lists the scan elements in order from the chain's scan data input to the scan data output. The Manhattan distance is listed from the previous element to the current element. Each connection is categorized as floating or ordered and the location of the scan element is also given.

> The location used for each scan element is the lower left corner of that instance. However, when performing physical-based scan connection, the locations of the pins are used to determine the order of the scan elements.

Related Information

Affected by this command:	create_chains_database
Related commands:	report_chains_longest_wires

	report_chains_total_wirelength

report_clock_gates

```
report_clock_gates  [-sinks_histogram]  [-fanout_summary] [-tree_view]
 [-tree_view_include_one_sink] [-tree_view_include_all_sinks]
 [-include_activity_info]  [-get_sinks {all gated ungated gated_by_synthesis gated_only_by_user not_gated_by_synthesis}] [-detail]  [-design] [-cgs] [-
include_timing] [-get_clock_gates_by_level] [-metrics] [> file]
```

Description

Generates a report on the clock gates in the design.

If you specify this command without any options, the command prints a summary report of the clock gating inserted in the design that includes the number of Genus and non-Genus clock-gating instances, the number of Genus gated and user gated flip-flops, the number of gated and ungated flip-flops, the total number of flip-flops in the design, and the number of CGs in different levels.

Options and Arguments

`-sinks_histogram`	Generates a histogram table showing distribution of CGs across sink bit count ranges.
`-fanout_summary`	Generates a table showing distribution of fanout (sinks and CGs) across CG levels, and list CGs violating fanout constraints.
`-tree_view`	Generates a tree-view of the CGs.
`-tree_view_include_one_sink`	
	Specifies the tree-view that shows one sink for each CG. This is dependent on the `-tree_view` option.
`-tree_view_include_all_sinks`	
	Specifies the tree-view that shows all the sinks for each CG. This is dependent on the `-tree_view` option.
`-include_activity_info`	Shows the average toggle savings in summary output.
`-get_sinks {all | gated | ungated | gated_by_synthesis | gated_only_by_user | not_gated_by_synthesis}`	
	Returns a Tcl list of sinks. A sink is included in the returned list if it matches any of the specified categories.
`-detail`	Shows the detailed information about each CG and its fanout.
`-design`	Generates a report for the specified design.
`-cgs`	Generates a report for the specified clock gates only.
`-include_timing`	Includes a table showing setup slacks (minimum and total) for each CG level in the summary output.
`-get_clock_gates_dict_by_level`	
	Returns a Tcl list of pairs. Each pair consists of a level, and a list of CGs found at that level.
file	Specifies the filename for the output.

Examples

The following examples show sample usage for the command.

- The following command prints a summary report of the clock gating inserted in the design. `report_clock_gates`
  The command reports a summary report of the clock gating inserted in the design that includes the number of Genus and non-Genus clock-gating instances, the number of Genus gated and user gated flip-flops, the number of gated and ungated flip-flops, the total number of flip-flops in the design, and the number of CGs in different levels.
- The following command reports a histogram table showing distribution of CGs across sink bit count ranges.

  ```
  @genus:root: number> report_clock_gates -sinks_histogram
  ```

```
        ========================================================
        ========================================================

        Clock gate sink histogram
        -------------------------
        |-------------------------------------------------------------|
        | Direct sink bits | Clock gates | Total sinks | Total bits |
        |------------------+-------------+-------------+------------|
        | 0 to 2           |           1 |           0 |          0 |
        | 3 to 4           |           1 |           4 |          4 |
        | 5 to 8           |           1 |           6 |          6 |
        | 9 to 16          |           2 |          13 |         22 |
        | 17 to 32         |           1 |          15 |         30 |
        |-------------------------------------------------------------|
```

- The following command reports a table showing distribution of fanout (sinks and CGs) across CG levels.

```
@genus:root: number> report_clock_gates -fanout_summary
        ========================================================
        ========================================================

        Clock gate fanout summary
        -------------------------
        L2U:
        |-----------------------------------------------------------------------------------------|
        |        | 0 bits | 1 bits | 2 bits | 3 bits | 4 bits | 5 bits | 6 bits | 7 bits | 8+ bits |
        |--------+--------+--------+--------+--------+--------+--------+--------+--------+---------|
        |  0 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  1 CG  | 1      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  2 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  3 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  4 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  5 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  6 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  7 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        | 8+ CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |-----------------------------------------------------------------------------------------|
        L1U:
        |-----------------------------------------------------------------------------------------|
        |        | 0 bits | 1 bits | 2 bits | 3 bits | 4 bits | 5 bits | 6 bits | 7 bits | 8+ bits |
        |--------+--------+--------+--------+--------+--------+--------+--------+--------+---------|
        |  0 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  1 CG  | -      | -      | -      | -      | -      | -      | -      | -      | 1       |
        |  2 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  3 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  4 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  5 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  6 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |  7 CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        | 8+ CGs | -      | -      | -      | -      | -      | -      | -      | -      | -       |
        |-----------------------------------------------------------------------------------------|
        L0G:
        |-----------------------------------------------------------------------------------------|
        |        | 0 bits | 1 bits | 2 bits | 3 bits | 4 bits | 5 bits | 6 bits | 7 bits | 8+ bits |
        |--------+--------+--------+--------+--------+--------+--------+--------+--------+---------|
        |  0 CGs | -      | -      | -      | -      |      1 | -      |      1 | -      |       2 |
        |-----------------------------------------------------------------------------------------|

        Genus clock gates violating lp_clock_gating_min_flops
        -----------------------------------------------------
```

```
0 violations found.

Genus clock gates violating lp_clock_gate_multi_level_min_fanout
-----------------------------------------------------------------
0 violations found.
```

- The following command reports a tree-view of the CGs.

```
@genus:root: number> report_clock_gates -tree_view

=========================================================
=========================================================

Clock gate tree view
--------------------
|-----------------------------------------------------------------|
| Clock gate     | Level | Direct             | Transitive         |
|                |       | sinks | bits | CGs | sinks | bits | CGs |
|----------------+-------+-------+------+-----+-------+------+-----|
| cg_user_top/cg | L2U   |     0 |    0 |   1 |    18 |   42 |   2 |
| -cg_user/cg    | L1U   |     3 |   12 |   1 |    18 |   42 |   1 |
|   -cg_added1   | L0G   |    15 |   30 |   0 |    15 |   30 |   0 |
|-----------------------------------------------------------------|
|     cg_added2  | L0G   |    10 |   10 |   0 |    10 |   10 |   0 |
|-----------------------------------------------------------------|
|     cg_added3  | L0G   |     6 |    6 |   0 |     6 |    6 |   0 |
|-----------------------------------------------------------------|
|     cg_added4  | L0G   |     4 |    4 |   0 |     4 |    4 |   0 |
|-----------------------------------------------------------------|
```

- The following command reports the tree-view that shows one sink for each CG.

```
@genus:root: number> report_clock_gates -tree_view -tree_view_include_one_sink

=============================================================================

=============================================================================

Clock gate tree view
--------------------
|-------------------------------------------------------------------------------------------------------|
| Clock gate                                                                 | Level | Direct             | Transitive         |
| (and one sink per CG)                                                      |       | sinks | bits | CGs | sinks | bits | CGs |
|----------------------------------------------------------------------------+-------+-------+------+-----+-------+------+-----|
| cg_user_top/cg                                                             | L2U   | 0     | 0    | 1   | 18    | 42   | 2   |
| -cg_user/cg                                                                | L1U   | 3     | 12   | 1   | 18    | 42   | 1   |
| |-CDN_MBIT_t1_q_reg[0]_MB_t1_q_reg[1]_MB_t1_q_reg[2]_MB_t1_q_reg[3]/CP | -     | -     | -    | - | -     | -    | -   |
|   -cg_added1                                                               | L0G   | 15    | 30   | 0   | 15    | 30   | 0   |
|     -added_0/CP                                                            | -     | -     | -    | -   | -     | -    | -   |
|-------------------------------------------------------------------------------------------------------|
|     cg_added2                                                              | L0G   | 10    | 10   | 0   | 10    | 10   | 0   |
|     -g1i/added2_0/CK                                                       | -     | -     | -    | -   | -     | -    | -   |
|-------------------------------------------------------------------------------------------------------|
|     cg_added3                                                              | L0G   | 6     | 6    | 0   | 6     | 6    | 0   |
|     -g1i/added2_10/CK                                                      | -     | -     | -    | -   | -     | -    | -   |
|-------------------------------------------------------------------------------------------------------|
|     cg_added4                                                              | L0G   | 4     | 4    | 0   | 4     | 4    | 0   |
|     -g1i/added2_16/CK                                                      | -     | -     | -    | -   | -     | -    | -   |
|-------------------------------------------------------------------------------------------------------|
```

- The following command reports the average toggle savings in summary output.

```
@genus:root: number> report_clock_gates -include_power_info
Summary
-------
```

```
|-------------------------------------------------------------------------------------|
|                               | Count        | Bits         | Average Toggle Saving* |
|-------------------------------+--------------+--------------+------------------------|
| Total clock gates             | 6  (100.00%) | -            | -                      |
| Total sinks                   | 38 (100.00%) | 62 (100.00%) | -                      |
|-------------------------------------------------------------------------------------|
| Genus clock gates             | 4  (66.67%)  | -            | -9.38%                 |
| User clock gates              | 2  (33.33%)  | -            | 25.00%                 |
|-------------------------------------------------------------------------------------|
| Sinks gated by Genus CGs      | 35 (92.11%)  | 50 (80.65%)  | 15.18%                 |
| Sinks gated only by user CGs | 3  (7.89%)   | 12 (19.35%)  | 50.00%                 |
|-------------------------------------------------------------------------------------|
| Total gated sinks             | 38 (100.00%) | 62 (100.00%) | -                      |
| Total ungated sinks           | 0  (0.00%)   | 0  (0.00%)   | -                      |
|-------------------------------------------------------------------------------------|

Level Summary
-------------
|-----------------------------------------------------------------|
|                          | Count       | Average Toggle Saving* |
|--------------------------+-------------+------------------------|
| Level 2 user CGs (L2U)   | 1 (16.67%)  | 50.00%                 |
| Level 1 user CGs (L1U)   | 1 (16.67%)  | 0.00%                  |
| Level 0 Genus CGs (L0G) | 4 (66.67%)  | -9.38%                 |
|-----------------------------------------------------------------|
| Total                    | 6 (100.00%) | 2.08%                  |
|-----------------------------------------------------------------|

Multibit Sink Summary
---------------------
|-------------------------------------------------------------------------------|
| Width | Count | Bits | Gated by Genus CGs | Gated only by user CGs | Ungated   |
|-------+-------+------+--------------------+------------------------+-----------|
| 4-bit |     3 |   12 | 0  (0.00%)         | 3 (100.00%)            | 0 (0.00%) |
| 2-bit |    15 |   30 | 15 (100.00%)       | 0 (0.00%)              | 0 (0.00%) |
| 1-bit |    20 |   20 | 20 (100.00%)       | 0 (0.00%)              | 0 (0.00%) |
|-------------------------------------------------------------------------------|
| Total |    38 |   62 | 35                 | 3                      | 0         |
|-------------------------------------------------------------------------------|
```

- The toggle saving for a clock gate is calculated as the percentage reduction in the toggle rate at the clock gate's clock input pin compared to the toggle rate at the clock gates clock output pin. The toggle saving for a sink is calculated similarly, as a percentage reduction in the toggle rate of the ungated clock source compared to the toggle rate of the sink's clock input pin.
- The following command returns a Tcl list of Genus gated sinks.

  ```
  report_clock_gates -get_sinks gated_by_genus
  ```

- The following command updates CG metrics.

  ```
  report_clock_gates -metrics
  ```

Related Information

Clock Gating Cell Specification in the *Genus Library Guide*.	
Affected by these commands:	syn_generic
	syn_map
	syn_opt
Affected by these attributes:	lp_clock_gating_add_obs_port
	lp_clock_gating_cell
	lp_clock_gating_control_point
	lp_clock_gating_exclude
	lp_clock_gating_style

report_clock_gating_quality

```
report_clock_gating_quality
[-feedback_depth_limit integer]
[-feedback_hierarchical]
[-per_sink_data_filename string]
[-per_generic_flop_enable_driver_data_filename string]
[-metrics]
[> filename]
```

Description

Generates a quality report on the clock gates in the design.

If you specify this command without any options, the command prints a quality report of the clock gating inserted in the design that includes total number of sinks, the number of generic instance sena and srl pin constants, feedback analysis, and feedback and sena analysis.

Options and Arguments

`-feedback_depth_limit integer`	
	Defines the feedback check maximum depth.
`-feedback_hierarchical`	
	Enables cross-hierarchical feedback check.
`-per_sink_data_file string`	
	Enables the detailed per sink data file output and sets the filename.
`-per_generic_flop_enable_driver_data_filename string`	
	Enables the detailed per sena/srl driver output and sets the filename.
`-metrics`	Sets metrics of CG quality.
`filename`	Specifies the name of the file to which to the report is written.

Examples

The following examples show sample usage for the command:

- The following command prints a quality report of the clock gating inserted in the design:
  `report_clock_gating_quality`
  This command prints a quality report of the clock gating inserted in the design that includes total number of sinks, the number of generic instance sena and srl pin constants, feedback analysis and feedback and sena analysis.

```
@genus:root: number> report_clock_gating_quality
         0/3 (0%) 1678187008422
         1/3 (33%) 1678187008436
         2/3 (66%) 1678187008479
         3/3 (100%) 1678187008500

       Total number of sinks: 3
            Excluded from clock gating: 0
            Permitted for clock gating: 3
            Clock gated by Genus:  0

       Generic instance sena & srl pin constants:
            non-const: 3
            const1: 0
            const0: 0
            mapped instance: 0

       Generic instance sena or srl shared with CG: 0

       Feedback analysis:
            sinks with structural feedback: 2
            sinks without structural feedback: 1
            bits with structural feedback: 6
            bits without structural feedback: 3

       Feedback and sena analysis:
            sinks with structural feedback or nonconst sena: 3
            sinks without structural feedback and constant sena: 0
            bits with structural feedback or nonconst sena: 9
            bits without structural feedback and constant sena: 0

       Nets driving >= 3 sena/srl pins:
            number of nets: 0
            max fanout count: 0
            min fanout count: 0
            average fanout count: 0
```

- The following command generates a quality report with a defined maximum depth:
  `report_clock_gating_quality feedback_depth_limit 8`
- The following command enables cross-hierarchical feedback check:
  `report_clock_gating_quality -feedback_hierarchical`
- The following command enables the detailed per sink data file output and sets the filename:
  `@genus:root: number> report_clock_gating_quality -per_sink_data_file file1` Per sink data written to file 'file1'
- The following command enables the detailed per sena/srl driver output and sets the filename:
  `@genus:root: number> report_clock_gating_quality -per_senasrl_driver_data_file file2 Per sena/srl driver data written to file 'file2'`
- The following command reports CG quality metrics:
  `report_clock_gatng_quality -metrics`

Related Information

Affected by these commands:	syn_generic
	syn_map
	syn_opt
Affected by these attributes:	lp_clock_gating_add_obs_port
	lp_clock_gating_cell
	lp_clock_gating_control_point
	lp_clock_gating_exclude
	lp_clock_gating_style

report_clock_groups

```
report_clock_groups
[-design design] [-group string] [>file]
```

Description

Generates a clock group report of the current design.

Options and Arguments

`-design design`	Specifies the design for which to create the report.
	If no design is specified, the report is given for the current design.
	This option is required when multiple designs are loaded.
file	Specifies the name of the file to which to write the report.
`-group string`	Specifies the group name for which to generate the report.

Example

- The following command displays the default report:

```
genus:root:> report_clock_groups

Group Name   Description       Exception            From Clocks   To Clocks
-------------------------------------------------------------------------

cg_group1    log_ex        set_clock_groups_exception     clock1     clock2 clock3
                               set_clock_groups_exception_1   clock2 clock3    clock1
```

Related Information

Related commands:

create_clock
create_generated_clock

set_clock_groups

report_clock_tree_structure

```
report_clock_tree_structure
[-clock_trees list]...
[-expand_below_logic] [-show_sinks] [-out_file file]
[-expand_generated_clock_trees  {independently | inline | independently_and_inline}]
```

Description

Generates a clock tree structure report of the current design.

Options and Arguments

`-clock_trees list`	Specifies a Tcl list of clocks or clock sources to be included in the report. By default all clocks are analyzed.
`-expand_generated_clock_trees {independently | inline | independently_and_inline}`	
	Specifies how generated clock trees should be displayed.
`-expand_below_logic`	Sub-trees below multi-input logic will be fully displayed.
`-out_file file`	Specifies the name of the file to which to write the report.
`-show_sinks`	Display timing path sinks in the clock tree report.

Example

The following example show sample usage for the command:

```
report_clock_tree_structure

report_clock_tree_structure -show_sinks -clock_trees {clk1 clk2}

report_clock_tree_structure -expand_generated_clock_trees inline

report_clock_tree_structure -expand_below_logic

report_clock_tree_structure -out_file clks.rpt
```

Related Information

Related command:

report_clocks

report_clocks

```
report_clocks
[-ideal] [-generated] [-view analysis_view]
[-uncertainty_table] [clock]... [> file]
```

Description

Generates a report on the clocks of the current design. Reports information on clocks created with the `create_clock` and `create_generated_clock` commands. The report includes information about all clock waveforms, clock arrival points, and clock uncertainties.

Use the `-generated` option to report generated clock information, and use the `-ideal` option to report an ideal clock - clock relationship.

Options and Arguments

clock	Specifies the name of the clock for which you want to generate the report. If a list is given, only information about those clocks is reported. You can use wildcards in the names.
	Default: All clocks, except generated clocks, are reported. Use the `-generated` option to report generated clocks.
file	Writes the report to the specified file name. If the file already exists, the software overwrites it.
	To write a compressed report, add the .gz extension to the file name.
	Default: Report is displayed on standard output without being saved.
`-generated`	Adds generated clock information to the description, uncertainty, and the relationship table.
`-ideal`	Reports a clock description with the ideal clock - clock relationship, i.e. uncertainty and latency are not included on this report.
`-view` *analysis_view*	
	Generates a clock report for the specified analysis view only: *analysis_view*.
	You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`-uncertainty_table`	Reports clock uncertainty information.

Example

- The following command displays the default report:

```
genus:root:> report_clocks

Clock Description
-----------------
Clock                              Clock      Source       No of
Name    Period   Rise   Fall       Domain     Pin/Port    Registers
-------------------------------------------------------------------
clk1    4000.0   0.0    2000.0      domain_1   clk          75
I1      2000.0   0.0    1000.0      domain_1   I1           10

Clock Network Latency / Setup Uncertainty
-----------------------------------------
        Network   Network     Source       Source      Setup        Setup
Clock   Latency   Latency     Latency      Latency   Uncertainty   Uncertainty
Name    Rise      Fall        Rise         Fall        Rise          Fall
-------------------------------------------------------------------------------
clk1     20.0      20.0        40.0         40.0        200.0         200.0
I1       20.0      20.0        40.0         40.0        0.0           0.0

Clock Relationship (with uncertainty & latency)
-----------------------------------------------
From      To     R->R     R->F    F->R    F->F
-----------------------------------------------
clk1     clk1   3800.0   1800.0  1800.0  3800.0
clk1      I1    2000.0   1000.0  2000.0  1000.0
I1       clk1   1800.0   1800.0   800.0   800.0
I1        I1    2000.0   1000.0  1000.0  2000.0
```

report_congestion

```
report_congestion [ >file]
```

Description

Reports the percentage of gcells with routing overflow, the total overflow of the design as well as the maximum overflow.

Options and Arguments

file	Specifies the name of the file to which to write the report.

Example

- The following command shows the congestion summary for design test.

```
genus:root:> report_congestion
============================================================
Generated by:         Genus(TM) Synthesis Solution Version
Generated on:         Date
Module:               test
Technology libraries: lib1
                      lib2
Operating conditions: slow
Interconnect mode:    placement
Area mode:            physical library
============================================================

H overflow            : 1.20%
V overflow            : 0.52%
Total number of GCELLS: 579
```

Related Information

Affected by these commands:	syn_generic -physical
	syn_opt

report_core_wrapper_cell

```
report_core_wrapper_cell
[-wrapped | -unwrapped]
[-location {pin|hpin|port|hport}...]
[-report_flops] [-summary]
[-max_list_objects integer]
[design|instance] [>file]
```

Description

Reports the wrapper cells inserted in the design.

Options and Arguments

{*design*|*instance*}	Specifies the design or instance for which you want to generate a report.
	By default a report is created for all designs currently loaded in memory.
file	Specifies the name of the file to which to write the report.
-location {*pin*|*hpin*|*port*|*hport*}	
	Limits the reported wrapper cells to the specified pin(s), port(s) or subport(s).
-max_list_objects *integer*	
	Specifies the maximum number of wrapper segments or flops to be reported on any pin, port or subport.
	Default: 9999
-report_flops	Reports the flops associated with the wrapper segments.
-summary	Reports the summary only.
-unwrapped	Lists only those ports or pins that do not have a wrapper cell associated with them.

`-wrapped`	Lists only those ports or pins that have an associated wrapper cell.

Example

- The following examples illustrate different report options on the same design.

```
genus@root: number> report_core_wrapper_cell test


Reporting 13 ports/pins Port/Pin                        type                usage                 wrapper_objects ========                        ====
=====                  =============== WEXT                             wrapper_control        -                     - WIG
wrapper_control         -                     - WINT                          wrapper_control        -                     - WOG
wrapper_control         -                     - WSEN_in                       wrapper_control        -                     - clk
wrapper_control         -                     - WSEN_out                      shift_enable           -                     - se
shift_enable            -                     - in1                           shared               input                 wrap_1
wrap_4 in1                              shared               output                wrap_3 in2                              shared               input
wrap_1                                                                         wrap_4 in2                               shared               output
wrap_3 in3                              shared               input                 wrap_2 out1                             shared               input
wrap_1                                                                         wrap_4 out2                              shared               input
wrap_1                                                                         wrap_4
Wrapper insertion summary report
================================
Number of Dedicated wrapper cells :0     input bounding :0    output bounding :0Number of Shared wrapper cells    :4     input bounding :3    output bounding
:1----------------------------------------------------------------------------Total no. of wrapper cells        :4     input bounding :3    output bounding
:1


genus@root: number> report_core_wrapper_cell test -wrapped


Reporting 13 ports/pins Port/Pin                        type                usage                 wrapper_objects ========                        ====
=====                  =============== in1                              shared               input                 wrap_1
wrap_4 in1                              shared               output                wrap_3 in2                              shared               input
wrap_1                                                                         wrap_4 in2                               shared               output
wrap_3 in3                              shared               input                 wrap_2 out1                             shared               input
wrap_1                                                                         wrap_4 out2                              shared               input
wrap_1                                                                         wrap_4
Wrapper insertion summary report
================================
Number of Dedicated wrapper cells :0     input bounding :0    output bounding :0Number of Shared wrapper cells    :4     input bounding :3    output bounding
:1----------------------------------------------------------------------------Total no. of wrapper cells        :4     input bounding :3    output bounding
:1


genus@root: number> report_core_wrapper_cell -report_flops


Reporting 13 ports/pins


 Port/Pin        type              Usage                wrapper_objects               wrapper_flops-------------------------------------------------------------
----------------------------------------   WEXT         wrapper_control        -                     -          WIG          wrapper_control        -
-          WINT         wrapper_control        -                     -          WOG          wrapper_control        -                     -          WSEN_in
wrapper_control        -                     -          clk          wrapper_control        -                     -          WSEN_out     shift_enable
-                      -          se           shift_enable           -                     -          in1          shared               input
wrap_1                          tmp5_reg in1          shared               input                 wrap_4                          tmp4_reg in1          shared
output                 wrap_3                         tmp3_reg in2          shared               input                  wrap_1                          tmp5_reg in2
shared                input                  wrap_4                         tmp4_reg in2          shared               output                 wrap_3
tmp3_reg in3          shared               input                 wrap_2                         tmp2_reg out1         shared               input
wrap_1                          tmp5_reg out1         shared               input                 wrap_4                          tmp4_reg out2         shared
input                  wrap_1                         tmp5_reg out2         shared               input                  wrap_4                          tmp4_reg
Wrapper insertion summary report
================================
Number of Dedicated wrapper cells :0     input bounding :0    output bounding :0   Number of Shared wrapper cells    :4     input bounding :3    output bounding
:1   ----------------------------------------------------------------------------Total no. of wrapper cells        :4     input bounding :3    output
bounding :1


genus@root: number> report_core_wrapper_cell test -summary
```

```
Wrapper insertion summary report==============================Number of Dedicated wrapper cells :0     input bounding :0    output bounding :0Number of Shared
wrapper cells    :4     input bounding :3    output bounding :1----------------------------------------------------------------------------------Total no. of
wrapper cells        :4     input bounding :3    output bounding :1
```

Related Information

Affected by this command:	add_core_wrapper_cell

report_cwd_check

```
report_cwd_check
[-checkpoint string] [-check_name string]
[-max_width string] [> file]
```

Description

Reports the registered checking rules. With each checking rule, it lists the:

- Name of the checking rule
- Checkpoint(s) the rule is associated with
- Check proc(s) attached to the associated checkpoint(s)
- Effort level of the rule
- Severity level of the rule
- Description string of the rule

> The `-check_name` and `-checkpoint` options cannot be used simultaneously.

Options and Arguments

`-check_name` *string*	Specifies, by name, a set of checking rules to report. Asterisk(*) can be used to see all names for check_name.
`-checkpoint` *string*	This switch specifies a set of checkpoints to report. Asterisk(*) can be used to see all names for checkpoint.
`-max_width` *string*	Limits the width of the columns in the table produced by this command. Limiting the width of a column to zero means removing that column from the table.
	This option takes a Tcl list of Tcl lists. Each sub-list represents a column in the table produced by this command. Each sub-list should have two elements: the first being name of the column (as seen in the report) and the second being an integer representing the maximum number of characters allowed for this column in the table.
	Default value is `{{check_name 40} {checkpoint 30} {check_proc 30} {effort 6}{severity 8} {description 30}}`
file	Specifies the filename to store the output of the command.

Examples

- The following example reports all checking rules that have been registered:

  ```
  report_cwd_check -checkname {*}
  ```

- This reports info about all checkpoints:

  ```
  report_cwd_check -checkpoint {*}
  ```

- The following example reports all checking rules whose `checkname` contains string "pin":

  ```
  report_cwd_check -checkname {*pin*}
  ```

- The following example reports information about the `arch_pin_order` and `arch_parameter_order` checking rules:

  ```
  report_cwd_check -checkname {arch_pin_order arch_parameter_order}
  ```

report_delay_calculation

```
report_delay_calculation
-from {pin|hpin}  -to {pin|hpin}
[-mode mode] [-view analysis_view] [> file]
```

Description

Reports how the delay is calculated from the look up table in the loaded technology library. The formula for calculating the delay is provided at the bottom of the report.

Options and Arguments

file	Redirects the report to the specified file.
-from {*pin*|*hpin*}	Specifies the driving pin.
-mode *mode*	Specifies the mode.
-to {*pin*|*hpin*}	Specifies the loading pin.
-view *analysis_view*	
	Specifies that the report should only apply to the specified *analysis_view*.

report_design_rules

```
report_design_rules [design]...
[-include_max_lut_check] [> file]
```

Description

Reports any design rule violations that are present in the specified design objects.

Options and Arguments

design	Specifies the design for which you want to generate a report.

	By default, a report is created for all designs currently loaded in memory.
file	Specifies the name of the file to which to write the report.
-include_max_lut_check	
	Reports the following additional information when frequency-based maximum capacitance calculation is enabled (using the use_max_cap_lut attribute) during design rule fixing and verification:
	• The instances that are operating at a frequency outside of the cell's max_cap lookup table range
	• The set and count of instantiated technology cells that did not have a max_cap lookup table

Related Information

Affected by these commands:	syn_map
	syn_opt
Affected by these attributes:	ignore_library_max_fanout
	max_capacitance
	max_fanout
	max_transition

report_dft_abstract_models

```
report_dft_abstract_models
[-ctl] [design] file
```

Description

Reports the scan abstract models read in using the read_dft_abstract_models command. The scan abstract model defines the scan chain architecture of the subdesign and is used as scan chain segments in the configuration of the top-level scan chains of the current design.

Options and Arguments

-ctl	Reports only scan abstract models that were read in from a Core Test Language (CTL) format (IEEE format P1450.6) file.
design	Specifies the design from which to report the abstract segment modules.
file	Specifies the name of the file to which to write the report.

Related Information

Related commands:	read_dft_abstract_model
	write_dft_abstract_model

report_dft_analyzed_test_points

```
report_dft_analyzed_test_points
[-testpoint_type {observe|control}]
[-partition_type {observe|async_0|async_1|double_control_0|double_control_1}]
[-highlight_in_gui]
[-colors string]
[-summary]
[-clear_GUI_first][-uninserted]
[-verbose]
[design]
```

Description

Reports the analyzed test point statistics and plots the test point nodes in the physical GUI.

Options and Arguments

`-testpoint_type`	Specifies the type of test point to report. `observe|control`
`-partition_type`	Specifies one of the following type of test points to report. `observe|async_0|async_1|double_control_0|double_control_1` When this option is specified, the above option `-testpoint_type` is ignored.
`-colors`	Specifies the list of colors to be used in the GUI. The default is `{red blue brown cyan green magenta orange pink purple yellow white}`.
design	Specifies the name of the top-level design to report test points on. If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-highlight_in_gui`	Specifies to direct airlines from test point nodes to the test point flops in the physical view GUI.
`-verbose`	Specifies to print test point details.
`-summary`	Specifies to only list the summary for each (test point) TP type.
`-clear_GUI_first`	Specifies to clear the physical view GUI.
`-uninserted`	Reports uninserted test points only.

Example

The following command prints a table and summary.

```
genus:/> report_dft_analyzed_test_points -highlight_in_gui
```

Related Information

Affected by this command:	add_analyzed_test_points

report_dft_trace_back

```
report_dft_trace_back
[-mode integer] [-polarity]
[-continue] [-print] [-list]
{pin|hpin|port|hport}
```

Description

Returns the pin or port found by tracing back one level from the specified pin or port based on the requested mode. If a constant is encountered, the command returns 0 or 1.

This command uses the default DFT mode to perform constant propagation, but alternative modes can be selected by running `check_dft_rules -dft_configuration_mode` *`new_configuration_mode`* beforehand. Also, functional tracing can usually be done by using mode 0.

Options and Arguments

`-continue`	Specifies to continue the trace back until a primary input, complex gate, or sequential gate is reached. Additionally, the trace will terminate if a logic constant is returned for the trace back pin.
`-list`	Prints the status and pin at every trace back.
`-mode` *`integer`*	Specifies the mode for tracing back.
	• 0 does not perform constant propagation. This is usually identical or close to a functional mode trace back.
	• 1 performs tied-constant propagation
	• 2 performs tied-constant and test-mode propagation
	• 3 performs tied-constant, test-mode and shift-enable propagation. This is usually identical to the default scan shift mode.
	Default: `3`
{*`pin`*|*`hpin`*|*`port`*|*`hport`*}	
	Specifies the pin or port from which to start the trace back.
`-polarity`	Specifies whether to report if the polarity changed through the trace.
	A returned value of 0 indicates no inversion.
	A returned value of 1 indicates an inversion.
`-print`	Prints the pin and polarity at every trace back.

Examples

- Following command traces back without performing constant propagation.

```
report_dft_trace_back -mode 0 pin:top/g121/CME
port:top/cme
```

- Following command traces back using the non-constant propagation mode and continues tracing back and printing pins until the trace back stops. Notice that this is identical to the functional path

```
report_dft_trace_back -print -continue -mode 0 pin:top/tmp_reg[3][15]/D
Trace status: continue
fopt12495/Y (non-inverted)
Trace status: continue
fopt12495/A (inverted)
Trace status: continue
fopt12496/Y (inverted)
Trace status: continue
fopt12496/A (inverted)
Trace status: continue
fopt12498/Y (inverted)
Trace status: continue
fopt12498/A (inverted)
Trace status: continue
fopt12499/Y (inverted)
Trace status: continue
fopt12499/A (non-inverted)
Trace status: continue
tmp_reg[2][15]/Q (non-inverted)
Trace status: sequential_gate
{pin:top/tmp_reg[2][15]/Q}
```

- Following command traces back using the default mode. In this case a constant logic 1 value is reported.

```
genus@root: number> dft_trace_back pin:top/g121/CME
1
```

- Following command traces back using the default mode and requests to report whether there was a change in polarity. In this case, a constant logic 1 with no change in logic polarity is reported.

```
genus@root: number> dft_trace_back pin:top/g121/CME -polarity
1 0
```

report_dft_violations

```
report_dft_violations [-async] [-clock]
[-abstract] [-shift_register] [-tristate]
[-race] [-xsource] [-xsource_by_instance]
[-violation_id id] [-verbose]
[design] [> file]
```

Description

Reports for each violation the object name, the type of violation, the description of the violation, how to find the root pin or port of the problem, the source file and the line number where to find the problem, the number of registers affected, and the instance names of the affected registers. The report is sorted by violation type. Run the DFT rule checker (`check_dft_rules` command) to get meaningful information. This command extracts the list of all violations with the corresponding violation points as the `check_dft_rules` command is not verbose by default.

Without any options specified, the command reports on all types of violations.

Options and Arguments

`-abstract`	Reports all abstract segment test mode violations.
`-async`	Reports all async set and async reset violations.
`-clock`	Reports all clock violations.
design	Specifies the design for which you want to generate a report.
	By default a report is created for all designs currently loaded in memory.
file	Specifies the name of the file to which to write the report.
`-race`	Specifies to report potential race condition violations.
`-shift_register`	Reports all shift register segment violations.
`-tristate`	Reports tristate design rules checking violations.
`-verbose`	Lists all the violation elements.
`-violation_id` *string*	
	Reports a single violation based on ID.
`-xsource`	Reports X-Source violations.
`-xsource_by_instance`	
	Reports X-Source violations by instance.

Example

The following example shows that the design has four violations, but only detailed information on the clock violations is requested.

```
report_dft_violations -clockTotal 4 violations (muxed_scan)
Clock Rule Violations
=====================
Reporting 2 clock violations

Violation #0:
  Object name: vid_0_clock  Type: clock violation  Description: [CLOCK-05] internal or gated clock signal  Source: g1/z (test10.v:12)  Number of registers
affected: 4  Affected registers:    out1_reg[0]    out1_reg[1]    out1_reg[2]    out1_reg[3] Violation #1:  Object name: vid_1_clock  Type: clock violation
Description: [CLOCK-06] clock signal driven by a sequential element  Source: divClk_reg/q (test10.v:14)  Number of registers affected: 4  Affected
registers:    out2_reg[0]    out2_reg[1]    out2_reg[2]    out2_reg[3]
Violation Rule Summary Report
=============================
[CLOCK-05] internal or gated clock signal : 1
[CLOCK-06] clock signal driven by a sequential element : 1
```

Related Information

Affected by this command:	check_dft_rules
	fix_dft_violations
Related attributes:	Violations Attributes

report_dont_touch

```
report_dont_touch
[object...] [>file]
```

Description

Generates a report for each netlist object to understand the source and reason for dont_touch.

Options and Arguments

object	Specifies the design or object for which to create the report.
file	Specifies the name of the file to which to write the report.

Related Information

Related Attributes:	dont_touch_reason

report_dp

```
report_dp [-full_path]
[-no_header] [-instance string] [-no_area_statistics]
[-mux] [-all] [-max_width string] [-all_stages]
[-print_instantiated] [-print_inferred]
[-sort keys] [design] [-ungrouped] [> file]
```

Description

Reports datapath operators that were inferred from the design. This command is available after elaboration. You must set the `hdl_track_filename_row_col` attribute to `true` to enable filename, column, and line number tracking in the datapath report; otherwise these headings will be hidden.

Options and Arguments

`-all`	Reports all datapath operators present in the design including muxes.
`-all_stages`	Reports rtlopto_group_info for all the stages.
	The mux operators are different from the MUX library cells that are picked by the mapper or are available in the technology library.
design	Specifies a particular design on which to report_dp operators. By default, Genus reports on the current design.
file	Specifies the name of the file to which to write the report.
`-full_path`	Reports the full UNIX path name of the filename, including the filename. By default, Genus only reports the design name. See Examples for more information.
`-max_width` *string*	Specifies the maximum width of individual column names. By default, the maximum width for a column is 20. If a name is more than 20, it will wrap to the next line.

	The valid column names are *Operator*, *Signedness*, *Inputs, Outputs*, *Cell Area*, *Line*, *Col*, and *Filename.*
`-mux`	Reports muxes present in the design. Muxes are not reported by default.
	Using the `-mux` option only displays the muxes in the design and suppresses the other datapath operators. To view both, use the `-all` option.
`-no_area_statistics`	Suppresses the table that only shows the total area and percentage information. The area and the percentage of the total area consumed by the datapath operators in the design are only available after issuing the `syn_map` command.
`-no_header`	Suppresses the header information.
`-print_inferred`	Reports only the inferred datapath components in the design.
`-print_instantiated`	Reports only the instantiated datapath components in the design.
`-sort` *keys*	Indicates how to sort the report. You can sort on the following keys:
	• `architectures` sorts the architectures in alphabetical order
	• `area` sorts by descending area
	• `inputs` sorts based on the number*width (number of bits) of the input signals—components with higher number of bits are printed first
	• `instances` sorts the instances in alphabetical order
	• `outputs` sorts based on the number*width (number of bits) of the output signals—components with higher number of bits are printed first
	• `operators` sorts by operator
	• `slack` sorts by ascending slack
	• `modules` sorts the modules in alphabetical order
	By default, the report does not contain the slack numbers.
	You can sort on several keys.
`-instance` *string*	Specifies the instance name.
`-ungrouped`	Specifies to report ungrouped datapath operators.

Examples

- The following example generates a report of the datapath components for the `rpdp1_basic` design.

```
genus@root: number> report_dp rpdp1_basic=============================================================...  Module:                 rpdp1_basic  Technology
library:     tutorial 1.0...============================================================Instantiated datapath components
              Operator Signedness  Inputs Outputs CellArea Line Col
Filename=================================================================================rpdp1_basic d1  u2  module:CW__CW_multadd__builtin_wA8_wB8_wC8_wZ16
CW/CW_multadd/builtin                 n/a   n/a        8x8x8x1 16       1239.75   38  52
impl_inf.v++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++   mul_1_19    very_fast/non_booth                 *    signed     9x9
16        912.75++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++   add_1_24    very_fast     +    signed     16x9    16
322.50=================================================================================
Inferred components

              Operator Signedness  Inputs Outputs CellArea Line Col
Filename=================================================================================rpdp1_basic mul_18_28 module:mult_unsigned  very_fast/non_booth
*    unsigned   16x8    16       1044.00   18  28 impl_inf.v=======================================================================================
```

```
     Type       CellArea Percentage-------------------------------------datapath modules  2283.75      20.55external muxes       0.00       0.00others
8829.00      79.45-------------------------------------total           11112.75     100.00
```

- By default, when using the `report_dp` command on a mapped netlist containing datapath operators, you will get the area statistics of the design, as shown in the following example.

  ```
  ------------------------------------
  datapath modules 4938.00     100.00
  mux modules         0.00       0.00
  others              0.00       0.00
  ------------------------------------
  total            4938.00     100.00
  ```

  This information is useful in determining the percentage of the design that contains datapath operators. If you do not want to report this information, then use the `-no_area_statistics` option.

  By default, the area report is suppressed for a netlist that contains only generic cells (no library cells).

- The following command provides a 30-character width to the *filename* column and provides a 0-character width to the *area* column.

  ```
  report_dp -max_width {{filename 30} {area 0}}
  ```

- The following command generates a report sorted by area.

  ```
  genus@root: number> report_dp -sort area

  ============================================================
  ...
  ============================================================



  Inferred components

              Operator Signedness Inputs Outputs CellArea Line Col Filename
  ================================================================================
  lt_leq
   add_14_26
   module:add_signed
    very_fast     +    signed     4x4    4          31.50   14  26 lt_leq.v
  ================================================================================
  lt_leq
   lt_13_25
   module:lt_signed
    very_fast     <    signed     4x4    1          26.25   13  25 lt_leq.v
  ================================================================================
  lt_leq
   lt_16_25
   module:lt_unsigned
    very_fast     <    unsigned   4x4    1          26.25   16  25 lt_leq.v
  ================================================================================
  lt_leq
   le_17_26
   module:leq_unsigned
    very_fast    <=    unsigned   4x4    1          25.50   17  26 lt_leq.v
  ================================================================================

        Type       CellArea Percentage
  ------------------------------------
  datapath modules   109.50      75.26
  external muxes       0.00       0.00
  others              36.00      24.74
  ------------------------------------
  total              145.50     100.00
  ```

- The following command sorts the report first by slack, then by area.

```
genus@root: number> report_dp -sort {slack area}
============================================================
...
============================================================


Inferred components

            Operator Signedness Inputs Outputs CellArea Line Col Filename  Slack
================================================================================
lt_leq
 lt_13_25
 module:lt_signed
  very_fast     <    signed     4x4    1          26.25   13  25 lt_leq.v -145.2
================================================================================
lt_leq
 lt_16_25
 module:lt_unsigned
  very_fast     <    unsigned   4x4    1          26.25   16  25 lt_leq.v -145.2
================================================================================
lt_leq
 le_17_26
 module:leq_unsigned
  very_fast    <=    unsigned   4x4    1          25.50   17  26 lt_leq.v -53.3
================================================================================
lt_leq
 add_14_26
 module:add_signed
  very_fast     +    signed     4x4    4          31.50   14  26 lt_leq.v 49.0
================================================================================

      Type       CellArea Percentage
---------------------------------------
datapath modules   109.50      75.26
external muxes       0.00       0.00
others              36.00      24.74
---------------------------------------
total              145.50     100.00
```

Related Information

Affected by this attribute:	hdl_track_filename_row_col

report_flow

`report_flow`

Description

Generates a report with the following information:

- The flow type, section list, and generation date of the templates
- The flow step order
- One line for every item in `flow_history`, indicating the success of that item and the configuration of optional flow features, and any error message

There is some overlap between `report_flow` and the summary printed at the end of `run_flow` commands. The difference is that `report_flow` reports information about the entire flow history (since the last `run_flow –reset`, if any), whereas the summary at the end of `run_flow` only includes data about the steps run by that command.

report_gates

```
report_gates [-inst_info] [-library_domain library_domain_list]
 [-power [-view view]] [-leakage_power] [-net_power]
 [-yield] [-hinst {inst|hinst}][design][-sort string][> file]
```

Description

Reports the technology library cells that were implemented (and identifies their originating libraries), the area of the cell instances, and the break up of the instances into timing models, sequential cells, integrated clock-gating cells, inverters, buffers, and logic gate cells. Optionally power information can be reported.

The result is based on the `cell_area` calculated from LIB information. Use `set_db use_area_from_lef true` (default: `false`), if you want to get correlation between reports generated with `report_gates` command and `report_qor` or `report_utilization` commands.

> Timing models can refer to memory cells, IPs, and so on.

Options and Arguments

design	Specifies the block for which you want to generate a report. You can also `cd` into the particular design directory and generate the report.
file	Specifies the name of the file to which to write the report.
`-hinst {`*inst*`|`*hinst*`}`	
	Restricts the reported information to the specified hierarchical instance.
`-inst_info`	Adds cells information by types.
`-library_domain` *library_domain_list*	
	Restricts the reported information to the specified library domains.
	This option only applies when using CPF.
`-leakage_power`	Adds leakage power information only.
`-net_power`	Adds net power information only.
`-power`	Adds leakage power and internal power information.
`-sort`	Sorts the report. (name | insts | area | leakage_power | internal_power)
`-view` *view*	Reports the power for the gates for the specified analysis view.
`-yield`	Reports the yield cost and yield percentage values for each library cell.

Example

The following example reports information such as the type of cells that were used, number of instances, area, and originating library of the current design.

```
    genus@root: number> report_gates================================================================...  Module:                alu  Technology library:    tutorial
    1.0  Operating conditions:  typical_case (balanced_tree)  Wireload mode:         enclosed=============================================================   Gate
    Instances   Area     Library   ---------------------------------------flopdrs          9   72.000    tutorial inv1           35  105.000    tutorial nand2
    112  112.000    tutorial nor2           37   55.500    tutorial xor2           17   34.000    tutorial ---------------------------------------total
    210  378.500


       Type    Instances   Area   Area % ---------------------------------sequential         9   72.000   19.0 inverter          35  105.000   27.7 logic
    166  201.500   53.2 ---------------------------------total            210  378.500  100.0
```

Related Information

Affected by these commands:	syn_generic
	syn_map
	syn_opt

report_hierarchy

```
report_hierarchy [-module module] [-only_user_hierarchy] [design]
 [> file]
```

Description

Generates a report based on the design hierarchy starting from the top level module down to the leaf module. The report will be in the following format:

```
level instance (module ) <status>
status : preserve_<value> -- indicating preserve hierachy or inherited_preserve        value       : blackbox --  indicating  unresolved instance
```

The `hdl_track_filename_row_col` attribute needs to be set to `true` before elaboration in order to successfully use this command.

Options and Arguments

file	Specifies the name of the file to which to write the report.
design	Specifies the design to report when there are multiple designs.
`-module` *module*	Specifies the module for which a hierarchical report is required.
`-only_user_hierarchy`	Reports the user defined hierarchies.

Example

- The following example reports the hierarchy of design `m1`.

```
=============================================================================
Hierarchy Report Format :

       level instance ( module ) <status>
       status :    preserve_<value> -- indicating preserve hierachy or
          inherited_preserve value
              :    blackbox --  indicating  unresolved instance
=============================================================================
        0 m1
         1 m2 ( m2 )
            2 m3 ( m3 )
               3 m3_0 ( m3_0 )
                  4 m3_0_0 ( m3_0_0 )
               3 m4 ( m4 )
                  4 m5 ( m5 )
                     5 m5_bbox ( m5_bbox ) blackbox
                  4 m6 ( m6 )
            2 m2myclk ( m2myclk )
```

Related Information

Affected by this attribute:	hdl_track_filename_row_col

report_instance

```
report_instance[-timing] [-power] [-detail]
{inst|hinst}... [> file]
```

Description

Generates a report on the instances of the current design. By default, the report gives timing related information on the instances. Get power related information using the `-power` option.

Options and Arguments

file	Specifies the name of the file to which to write the report.
-detail	Reports information such as pin direction, propagated constants, propagated pins, and disabled timing arcs. Disabled timing arc info is only available for mapped designs.
inst|hinst	Specifies the instance for which to generate the report.
-power	Reports instance leakage, internal power, net power and the computed probability, toggle rate, and net power on the nets of the instance, and the activity and power for each of the arcs.
	In case the switching activities are user-asserted, the values are appended with an asterisk (*).
	The `lp_power_unit` attribute does not affect the units of the Arc Energy column.
-timing	Reports timing information, such as slew-in, load, slew-out, delay, and slack.

Example

- The following example reports timing information for the `n0000D3666` instance.

```
genus@root: number> report_instance -timing n0000D3666

  ...
  Module:               dpldalgn
  ...
============================================================
         Instance Timing Info
         --------------------
 Instance n0000D3666 of libcell nor2

 Arc  Arc  Slew        Slew
From  To    In   Load  Out   Delay   Slack
-------------------------------------------
 A r  Y f  46.2  20.7  57.2  136.0  -1022.4
 A f  Y r  46.2  20.7  57.2  136.0  -1022.4
 B r  Y f  16.5  20.7  57.2  134.0   -900.2
 B f  Y r  16.5  20.7  57.2  134.0   -900.2
```

report_logic_levels_histogram

```
report_logic_levels_histogram [-bars integer]
[-threshold integer] [-slack_limit float]
[-skip_buffer] [-skip_inverter]
[-cost_group] [-std_cells] [-unit_delay_setup]
[-view analysis_view] [-per_cost_group] [> file]
```

Description

Prints the number of timing endpoints in the design in a histogram format.

The timing paths are grouped according to the logical depth number of the paths.

Options and Arguments

`-bars integer`	Specifies the logic level distribution histogram number.
	Default: `5`
`-cost_group`	Specifies the cost group to generate the report for.
`-details`	Reports all endpoints and logic levels.
file	Redirects the report to the specified file.
`-per_cost_group`	Specifies to report the logic level histogram for each cost group.
`-skip_buffer`	Excludes buffers from the logic level count.
`-skip_inverter`	Excludes inverters from the logic level count.
`-slack_limit float`	Limits the report to paths with less than the specified slack.
`-std_cells`	Specifies to create logic level depth histogram across standard cells.
`-threshold integer`	Specifies the threshold for the number of logic levels to report on. Only logic levels equal to or greater than the specified number are reported.
`-unit_delay_setup`	Specifies to report logic level histogram using `unit_delay`
`-view analysis_view`	
	Specifies to report the histogram for the specified analysis view.

Related Information

Related command:	report_timing

report_loop

```
report_loop [-sdcfile string]
[-version string] [design] [> file]
```

Description

Reports the loop breaker buffers that were added by the timing engine to break the combinational loops during timing analysis.

The report lists for every loop breaker the instance name, and the driver and the load of the loop breaker.

Options and Arguments

design	Specifies the design for which you want the report.
file	Redirects the report to the specified file.
-sdcfile *string*	Creates an SDC file with the appropriate `set_disable_timing` and `set_false_path` settings.
-version *string*	Specifies a particular SDC version for the SDC file created with the `-sdcfile` option. The available versions are: `1.1, 1.3, 1.4, 1.5` or `1.7`.
	Default: `1.7`.

Example

The following report shows the loop breakers inserted in design `loop`.

```
genus@root: number> report_loop ============================================================  Generated by:          Genus Synthesis Solution version
Generated on:          date  Module:                loop  Technology library:     tutorial 1.1  Operating conditions:   typical_case (balanced_tree)  Wireload
mode:          enclosed  Area mode:              timing library=================================================================
   CDN Loop breaker      Driver   Load ---------------------------------------inst1/cdn_loop_breaker inst1/i1/Y i0/B
```

Related Information

Related command:	remove_cdn_loop_breaker

report_memory

```
report_memory [> file]
```

Description

Reports the memory resource used by the compiler in the computing platform.

Options and Arguments

file	Specifies the name of the file to which to write the report.

report_memory_cells

```
report_memory_cells [library] [> file]
```

Description

Reports the memory cells in the library.

Options and Arguments

file	Specifies the name of the file to which to write the report.

library	Specifies the name of the library for which to report the memory cells.
	If no library is specified, the report applies to all libraries that are loaded.

Example

The following command lists the memory cells in the RF32X32_lib library.

```
================================================================================# Generated by: Genus Synthesis Solution version# Generated on:
date==========================================================================            Library : RF32X32_lib         Memory Cell: RF32X32         Memory
Type: ram         Memory address width: 5         Memory word width: 32----------------------------------------------------------------------------------------------------
--------------------   PIN/BUS       DIRECTION IS_BUS             BUS_TYPE CAPACITANCE RELATED_PIN     TIMING_TYPE    TIMING_SENSE-----------------------------
-------------------------------------------------------------------------------                                                                                 Q
output 31 to 0    YES         RF32X32_DATA                    CLK     rising_edge       non_unate
D   input 31 to 0    YES         RF32X32_DATA       0.002         CLK     hold_rising
CLK    setup_rising
CEN    input 0 to 0     NO                            0.002         CLK     hold_rising
CLK    setup_rising                                                                    WEN    input 0 to 0     NO                            0.010
CLK     hold_rising                                                                      CLK    setup_rising
A    input 4 to 0    YES      RF32X32_ADDRESS       0.008         CLK     hold_rising
CLK    setup_rising----------------------------------------------------------------------------------------------------------------------     CLOCK       DIRECTION
CAPACITANCE MAX_TRANSITION MIN_PULSE_WIDTH_HIGH  MIN_PULSE_WIDTH_LOW  MIN_PERIOD-----------------------------------------------------------------------------------------
--------------------------
       CLK    input 0 to 0       0.038          1.000               0.055               0.130      0.867-----------------------------------------------------
-----------------------------------------------------
```

report_messages

```
report_messages
 [-all] [-include_suppressed] [-errors]
 [-warnings] [-info] [-message_list object...] [> file]
```

Description

Summarizes the information, warning and error messages that have been issued by Genus in the current run since the last report. The report contains the number of times the message has been issued, the severity of the message, the identification number, and the message text.

The `-all` option does not report those messages that were suppressed through the `suppress_messages` command. Specify the `-include_suppressed` option to report such messages.

By adding report_messages to your Tcl prompt, Genus can summarize all messages issued during the last command right before prompting you for more input. This is useful after long commands (such as `elaborate`) that can generate many messages.

Info type messages print is driven by an attribute: information_level. The higher number (between 1 and 11, default 1) the more information messages are printed out. A high number will make the log file bigger but it will contain more information useful to analyze and debug.

Like many objects in the database, messages have associated attributes. Some attributes can be used to drive the way the messages are printed out, save in the logs etc.

Options and Arguments

`-all`	Returns a list of all issued message IDs as a Tcl result. The result does not include suppressed message IDs.
`-errors`	Returns a list of issued error message IDs as a Tcl result.
file	Specifies the name of the file to which to write the report.
`-include_suppressed`	Returns a list of suppressed message IDs as a Tcl result. These messages were suppressed through the `suppress_messages` command.

`-info`	Returns a list of issued information message IDs as a Tcl result.
`-message_list` *object*	
	Returns a list of issued message IDs as specified in the list.
`-warnings`	Returns a list of issued warning message IDs as a Tcl result.

Examples

- The following command prints a summary table and returns no Tcl value:

```
report_messages
```

- The following command reports all issued message IDs:

```
report_messages -all
```

- The following command reports all issued message IDs:

```
report_messages -suppressed
```

- The following command prints a summary table and returns no Tcl value:

```
report_messages

==================
  Message Summary
==================

--------------------------------------------------------------------------------
|   Id    | Sev    |Count |                  Message Text                    |
--------------------------------------------------------------------------------
| LBR-109 |  Info   | 1   | Set default library domain.                      |
| LBR-412 |  Info   | 1   | Created nominal operating condition.             |
|         |         |     | The nominal operating condition represents either|
|         |         |     | the nominal PVT values if specified in the library|
|         |         |     | source, or the default PVT values (1.0, 1.0, 1.0).|
--------------------------------------------------------------------------------

report_messages
```

> The second call in a row reports no message as the cache is reset after each call to the command

- An example with the -all switch

```
report_messages -all

==================
Message Summary
==================

--------------------------------------------------------------------------------
| Id    |Sev  |Count |               Message Text                            |
--------------------------------------------------------------------------------
|LBR-109|Info |  1   |Set default library domain.                            |
|LBR-412|Info |  1   |Created nominal operating condition.                   |
|       |     |      |The nominal operating condition represents either the|
|       |     |      |nominal PVT values if specified in the library         |
|       |     |      |source, or the default PVT values (1.0, 1.0, 1.0).     |
--------------------------------------------------------------------------------
```

> Here you can see all the messages from the beginning of the session, because of the `-all` switch.

- In this example, all the messages, including the suppressed by the `suppress_message`, (or `set_db [get_db <messageID> .max_print 0]`) are printed out:

```
 report_messages -all -include_suppressed

=================
  Message Summary
=================

-------------------------------------------------------------------------------
|   Id    |  Sev   |Count|                  Message Text                   |
-------------------------------------------------------------------------------
| INVS-8  |  Info  | 1   |Innovus executable found.                        |
|         |        |     |The specified Innovus executable will be used for |
|         |        |     |the Innovus batch jobs.                          |
| LBR-109 |  Info  | 1   |Set default library domain.                      |
| LBR-155 |  Info  | 744 |Mismatch in unateness between 'timing_sense'       |
|         |        |     |attribute and the function.                      |
|         |        |     |The 'timing_sense' attribute will be respected.   |
| LBR-161 |  Info  | 1   |Setting the maximum print count of this message to|
|         |        |     |10 if information_level is less than 9.          |
| LBR-162 |  Info  | 124 |Both 'pos_unate' and 'neg_unate' timing_sense arcs|
|         |        |     |have been processed.                             |
|         |        |     |Setting the 'timing_sense' to non_unate.         |
| LBR-170 |  Info  |  48 |Ignoring specified timing sense.                 |
|         |        |     |Timing sense should never be set with 'rising_edge'|
|         |        |     |or 'falling_edge' timing type.                   |
| LBR-41  |  Info  | 1   |An output library pin lacks a function attribute. |
|         |        |     |If the remainder of this library cell's semantic  |
|         |        |     |checks are successful, it will be considered as a |
|         |        |     |timing-model (because one of its outputs does not |
|         |        |     |have a valid function.                           |
| LBR-412 |  Info  | 1   |Created nominal operating condition.             |
|         |        |     |The nominal operating condition represents either |
|         |        |     |the nominal PVT values if specified in the library|
|         |        |     |source, or the default PVT values (1.0, 1.0, 1.0).|
| LBR-436 |  Info  |2184 |Could not find an attribute in the library.      |
| LBR-516 |  Info  | 1   |Missing library level attribute.                 |
| LBR-518 |  Info  | 1   |Missing a function attribute in the output pin    |
|         |        |     |definition.                                      |
| TUI-31  | Warning| 2   |Obsolete command.                                |
|         |        |     |This command is no longer supported.             |
-------------------------------------------------------------------------------
```

- The following examples shows all warning messages include the suppressed messages:

```
report_messages -all -warning -include_suppressed

=================
  Message Summary
=================

-------------------------------------------------------------------------------
|   Id    |  Sev   |Count|                  Message Text                   |
-------------------------------------------------------------------------------
| TUI-31  | Warning| 2   |Obsolete command.                                |
|         |        |     |This command is no longer supported.             |
-------------------------------------------------------------------------------
```

Related Information

Related command:	define_msg

Related attributes	count
	max_print
	print_count
	screen_max_print
	screen_print_count

report_metric

```
report_metric
[-out_file file]
[-format enum {html|json|excel|text|vivid}]
[-id string][-names string][pattern]
```

Description

User defined proc.

Reports the statistics information (QoR metrics) that was recorded at various stages or that was read in, or compares the metrics of two specified runs.

Options and Arguments

`-format enum {html|json|excel|text|vivid}`	
	Specifies the format type of the report.
	Use `write_metric -format json` for json based metric report.
`-id string`	Specifies the ID of the metrics for which you want to see the report.
`-names string`	Specifies the stage(s) of the run that you want to list in the report. Note: Wildcards are supported.
`-out_file file`	Specifies the file name to which to write the report.
pattern	Specifies the pattern to match.

Example

The following example shows how to use the `report_metric` command to collect data in the metrics file:

```
push_snapshot_stacksyn_opt -incrpop_snapshot_stackcreate_snapshot -categories all -label syn_opt_incrreport_metric -format html -out_file syn_opt.html
```

Related Information

The "Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:	create_snapshot
	push_snapshot_stack
	pop_snapshot_stack

report_min_pulse_width

```
report_min_pulse_width [-view analysis_view]
{pin|port}... >file
```

Description

Prints the list of pins and ports in the design where the minimum clock pulse width is violated.

Options and Arguments

file	Redirects the report to the specified file.
{*pin*|*port*}	Pin or port where the minimum clock pulse width is violated.
-view *analysis_view*	
	Specifies to report only for the specified analysis view.

Related Information

Related commands:	create_clock
	set_min_pulse_width

report_mode

```
report_mode object
```

Description

Prints active and inactive modes for the specified instance.

Options and Arguments

object	Specifies the instance for which to print the report

report_module

```
report_module
[-depth integer] [-min_count integer]
[-min_area float] [-summary] [module]...
```

Description

Prints a report for the specified modules.

Options and Arguments

`-depth` *integer*	Specifies the number of levels of recursion.
`-min_count` *integer*	
	Specifies the minimum instance count per module.
`-min_area` *float*	Specifies the minimum area per block.
module	Specifies the module to report on.
`-summary`	Specifies to report the area summary.

report_multibit_inferencing

```
report_multibit_inferencing
[-combinational] [-sequential] [-power]
[[-hinst instance] [design]] | -lib]
[-not_merged_summary] [-not_merged_instance_list][-reason_not_merged string] [-tcl] [-find_libcell lib_cell]
[-mapping] [-no_header] [-iso_ls]  [-exclude_synchronizer]  > file
```

Description

Reports detailed information on the combinational and sequential multibit cells that are used in the design or available in the libraries. Use the options to filter the information. When no options are specified, the report returns the information for all multibit cells used in the design.

Options and Arguments

`-combinational`	Limits the report to combinational multibit cells.
	For each multibit libcell the following is reported: bitwidth, the number of instances in the design mapped to this libcell, total area of these instances, the multibit conversion ratio (number of single bit instances merged to this multibit cell to the total number of single bit instances), and the library.
design	Specifies the design for which to create the report.
	If no design is specified, the report is given for the current design.
	This option is required when multiple designs are loaded.
	You cannot specify this option together with the `-lib` option.
`-exclude_synchronizer`	
	Specifies to report multibit coverage excluding synchronizer.
`-find_libcell` *lib_cell*	
	Specifies to provides information about the specified *lib_cell*.
file	Redirects the report to the specified file.
`-hinst` *instance*	
	Prints the report for the specified hierarchical instance.
`-iso_ls`	Limits the report to iso/ls multibit cells.
`-lib`	Reports the multibit cells available in the libraries.
	For each libcell, the following is reported: value of the avoid attribute, usability of the multibit cell, bitwidth, cell area, cell leakage power, library, library domain.

	`Multibit_Usable` shows whether a libcell is valid to use during multibit merging or not.
	This option can be used after the libraries are loaded and before the design is loaded.
	You cannot specify this option together with the `-hinst` option or when you specify the design.
`-mapping`	Reports the mapping between any multibit instance and the corresponding original single-bit instances.
`-no_header`	Prints the report without a header.
`-not_merged_instance_list`	
	Prints all single bit sequential instances that could not be merged into multibit instances.
`-not_merged_summary`	Prints for each of the different reasons the number of instances that were not merged into a multibit instance.
`-power`	Reports leakage power information for the multibit cells in the design.
`-reason_not_merged` *string*	
	Reports the instances that could not be merged for the specified reason. Specify any of the reasons listed in **Reasons for Not Merging Instances** below, or specify "all" to report all instances with the reason why they could not be merged.
`-sequential`	Limits the report to sequential multibit cells.
	For each multibit libcell the following is reported: bitwidth, the number of instances in the design mapped to this libcell, total area of these instances, the multibit conversion ratio (number of single bit instances merged to this multibit cell to the total number of single bit instances), and the library.
`-tcl`	Generates a `create_multibit.tcl` file which contains a list of commands to merge single bit flops individually.

Reasons for Not Merging Instances

INST_DFT_PRESERVE

These instances are part of a scan chain or scan segments.

By default, instances that are part of a scan chain are merged (scan chain merging). In some cases, certain limitations still prevent merging within a scan chain.

INST_MISMATCH_EXCEPTION

Only instances with the same timing exceptions can be merged into multibit instances. These instances have a mismatch in one or more of the exceptions, which excludes them from multibit merging.

INST_MISMATCH_SPRG_CONSTRAINTS

Only instances with the same SRPG constraints can be merged into multibit instances. These instances have a mismatch in one or more of the constraints which excludes them from multibit merging.

INST_MULTIPLE_CONNECTED_OUTS

These instances have an output pin that is neither a q, qb, or a scan-data output pin and the pin is driving some load.

The tool only recognizes q, qb, or scan-data output pins for multibit cell inferencing (MBCI). These instances have output pins that are not recognizable which excludes them from multibit merging.

INST_NET_PRESERVE

A net connected to the instances is preserved. By default, multibit merging is allowed on instances with preserved nets. In this case, the `multibit_preserved_net_check` was set to `true` to prevent multibit merging when any of the nets connected to the instances is marked preserved.

INST_ NO_CLUSTER

During physical multibit merging, instances that are closer are merged into multibit instance clusters. These instances do not belong to any cluster and were rejected for multibit merging.

INST_NO_VALID_REPLACEMENT

No multibit libcell was found in the library that is compatible with the single bit instances.

For example, the single bit instances are scan instances where as all multibit instances in the library are non-scan.

INST_ONLY_ONE_BIT

The module has either only one single bit instance which cannot be converted into a multibit instance since two instances are needed for creating multibit, or this is the only instance that remains after creating multibit instances from the single bit instances in this module.

INST_OUTPUT_PIN_BUNDLED / INST_OUTPUT_PIN_BUSSED

Single bit instances with bundled or bussed pins are not considered for multibit merging.

INST_PIN_CASE_CONSTANT

If any instance pin is forced to assume a constant value for timing analysis purposes, that instance is not considered for multibit merging.

INST_PRESERVE

The instance or any of its pins is preserved. Preserved instances are not merged into multibit instances.

INST_SCAN_OUT_NON_LOOP

If a scan flip-flop is not part of a scan chain, the scan-data output of the instance should be connected in loopback mode (scan-data output pin of the scan flop is connected to its own scan-data input pin). In this case, the scan-data output of the instance is not connected in loopback mode, therefore the instance is not considered for multibit merging.

INST_SPLIT

This single bit instance was created by splitting a multibit instance for timing purposes. Merging it again into a multibit instance would not benefit the qor.

INST_TIMING_EXCEPTION

The instances have a timing exception. By default, these instances are considered for multibit merging. In this case, the `multibit_auto_exclude_registers_with_exceptions` attribute was set to `true` to prevent multibit merging of sequential instances with exceptions.

INST_UNRECOGNISED_CONTROL_INPUT

The instances have a pin that is not recognized by the synthesis tool. Instances with pins that are not recognizable are not considered for multibit merging.

INST_WORSE_QOR

Adding this single bit instance to a multibit instance would worsen the qor.

report_net_cap_calculation

`report_net_cap_calculation` *`hnet`*
`[-fall] [-rise] [>` *`file`*`]`

Description

Reports the capacitance values for the different pins or ports that are connected to the specified net.

- For a pin, the capacitance value is the capacitance of the libcell pin (obtained from the `.lib`).
- For a port, the capacitance value is any capacitance annotated on the port (sum of the `external_pin_cap` and `external_wire_cap` attributes).
- The net capacitance is computed from the wire-load model available in the library. This is based on the `wireload_mode` attribute (`top`, `segmented`, or `enclosed`).
- The final capacitance is the sum of the net capacitance and the pin and port capacitance values to which the net is connected.

The columns "Terms from .lib" and "Cap from .lib" in the report will be populated if the wireload model in the `.lib` appears as a table. Otherwise, it will be empty.

This command is not supported on those nets that have the `physical_cap` attribute set on them. Also, when you are in PLE mode, only the final capacitance is shown (no computation).

Options and Arguments

`-fall`	Reports the fall capacitance of the net.
`file`	Redirects the report to the specified file.
`hnet`	Specifies the net name for which the report should be generated.
`-rise`	Reports the rise capacitance of the net.

Related Information

Related command:	report_net_res_calculation

report_net_delay_calculation

```
report_net_delay_calculation
[-driver_pin {port|hpin|pin}...]
[-load_pin {port|pin}...]
[-rise_rise] [-fall_fall] [> file]
```

Description

Reports the net delay, in picoseconds, between the specified driver and load pins. Both the driver and load pins should be on the same net. The delay computed would depend upon the tree type used (`tree_type` attribute): best case, worst case, or balanced tree.

> This command assumes that the `interconnect_mode` attribute is set to `wireload`.

Options and Arguments

`-driver_pin {`*`port`*`|`*`hpin`*`|`*`pin`*`}`	
	Specifies the starting port or pin on which to obtain the net delay.
`-fall_fall`	Restricts report of net delay to fall to fall arcs
`file`	Redirects the report to the specified file.
`-load_pin {`*`port`*`|`*`pin`*`}`	
	Specifies the ending port or pin on which to obtain the net delay.
`-rise_rise`	Restricts report of net delay to rise to rise arcs.

Related Information

Affected by this attribute:	tree_type

report_net_res_calculation

```
report_net_res_calculation hnet [> file]
```

Description

Reports the total wire resistance of the specified net. The total wire resistance is the sum of the individual net segment resistance (obtained from the wire-load model) and the external wire resistance (annotated on any port and obtained from the `external_wire_res` attribute) that is connected to the net.

- For a port, the resistance value is any resistance annotated on the port (the `external_wire_cap` attributes).
- The net resistance is computed from the wire-load model available in the library. This is based on the `wireload_mode` attribute (`top`, `segmented`, or `enclosed`).
- The final resistance is the sum of the net resistance and the pin and port resistance values to which the net is connected.

The columns "Terms from .lib" and "Cap from .lib" in the report will be populated if the wire-load model in the `.lib` appears as a table. Otherwise, it will be empty.

This command is not supported on those nets that have the `physical_res` attribute set on them. Also, when you are in PLE mode, only the final resistance is shown (no computation).

Options and Arguments

file	Redirects the report to the specified file.
hnet	Specifies the net name for which the report should be generated.

Related Information

Related command:	report_net_cap_calculation

report_nets

```
report_nets [-hierarchical] [-pin {pin|hpin}...]
[-min_fanout integer] [-max_fanout integer]
[-sort string] [-cap_worst integer] [-flat]
[inst|hinst|hnet]... [> file]
```

Description

Generates a report on the nets of the current design. The report gives information for the top-level nets in the design. You can specify pin names, nets, instances, maximum and minimum fanout threshold values, nets, and instances. Control the data printed out using the `-min_fanout` and `-max_fanout` options for nets that have fanout between these values.Nets that are followed by the "`@`" symbol in parenthesis indicate SPEF annotation.

Options and Arguments

`-cap_worst` *integer*	Specifies the number of worst capacitance nets that are to be reported.
file	Specifies the name of the file to which to write the report.
`-flat`	Reports flat net information.
`-hierarchical`	Reports all the nets in the design hierarchy.
`-max_fanout` *integer*	
	Specifies an *integer* value and reports nets whose fanouts are below the given threshold value.

-min_fanout *integer*	
	Specifies an *integer* value and reports nets whose fanouts are above the given threshold value.
inst|*hinst*|*hnet*	Reports information on the specified nets or nets belonging to the instance.
-pin {*pin*|*hpin*}	Specifies a list of pin names and reports the nets connected to the pins.
-sort *string*	Specifies the field name on which to sort. Valid field names are `load`, `resistance`, or `capacitance`.

report_obj

```
report_obj
[string1] [string2]
[-all | [-settable] [-computed]
[-categories list_of_categories]
[-tcl | -verbose] [-obsolete]
```

Description

Summarizes all attributes associated with the object being reported. The default output shows the non-default root attributes which were modified either by user or in the run script, Extra customization can be added using filters to report specific attributes of interest.

The default output reports those attributes that are set to non-default values and are not computed. This makes it easy to see any changes that were made in the run scripts.

The default output reports those attributes that are set to non-default values and are not computed. This makes it easy to see any changes that were made in the run scripts.

You can also add more attributes or filter out some attributes using various options below.

Options and Arguments

-all	Reports all non-computed attributes and their current value, even if the current value matches the default value.
-categories *list_of_categories*	
	Reports only those attributes that are in the given categories. Wildcards * and ? are allowed. Attributes that do not have a category are assigned the reserved category name NULL. The report puts all the attributes in a category together.
-computed	Includes attributes that are computed (for example `is_computed == true`).
	By default, the computed attributes are not included. If the data is not computed, this parameter triggers building the timing graph, and so on, that may take significant run time. This parameter cannot be used with -`all`.
-settable	Restricts the report to all attributes marked as `is_settable` that are not set to the default value. This parameter cannot be used with `-all`.
string1	Reports attributes for the specified objects.
	Default: `root`
string2	Returns only those attributes whose names match the specified pattern. You can use the * and ? wild characters.
	Default: `*`
-tcl	Returns the objects and their values in a Tcl-style list, that is, a list of keyword-value pairs.
-verbose	Returns the objects data type and default value.
	You cannot use this option with the `-tcl` option.

`-obsolete`	Reports all attributes marked "is_obsolete" in the data model and are non-default.

Example

- Example report

```
genus:design:alu number> report_obj

******************************************************************************** Object:
root*********************************************************************************Attribute Name                                                                  Current
Value ------------------------------------------------------------------------------base_name                                                                    /
elapsed_runtime                                                845 escaped_name                                                {} information_level
9 load_average                                                   0.0 memory_usage                                              409.48 name
{} obj_type                                                       root peak_memory                                             427.39
peak_physical_memory_usage                                     153.2 physical_memory_usage                                     153.2 platform_wordsize
64 program_name                                                   Genus(TM) Synthesis Solution program_version
16.20-p003_1 startup_license                                               Genus_Synthesis tinfo_tstamp_file
.rs_ria.tstamp library                                                       tutorial.lib current_design
design:alu db_units                                                       no_value phys_assume_met_fill                                                  0.0
```

- The following command displays the root attributes for the timing category that are not computed, and do not match the default value with their current values:

  ```
  report_obj -categories timing -all
  ```

- The following command displays the root attributes that do not match their default values, and are settable:

  ```
  report_obj -settable
  ```

- The following command displays all the root attributes that start with place*:

  ```
  report_obj / place* -all
  ```

- The following command displays the attributes that do not match the default value for the net objects in the list:

  ```
  report_obj [get_db nets *clk]
  ```

report_opcg_clock_domain_info

```
report_opcg_clock_domain_info
[-end_point_limit integer]
[-system_clock_violations]
[-report_internal] [-verbose]
[design] [> file]
```

Description

Reports the DFT clock domain information.

This command should be run after `connect_scan_chains` command.

Options and Arguments

design	Specifies the design for which you want to report the DFT clock domain information.
`-end_point_limit` *integer*	
	Prevents reporting endpoints if the number is less than the specified limit.
`-report_internal`	Reports the number of endpoints that receive data from the same clock domain
`-system_clock_violations`	

	Reports only system clock violations.
`-verbose`	Lists all endpoints in the report.

Example

Sample output:

```
report_opcg_clock_domain_info

Test clock domain report:Clock domain 1: opcg_load_clkTest clock 1: opcg_load_clk (root pin: opcg_load_clk)       Total endpoints: 0       System mode clock
control: 0 clock pins.              Total 0 endpoints exempted from system clock control check.       Domain crossing capture report: total 0 endpoints and
0 domains.       Domain crossing lauching report: total 0 endpoints and 0 domains.

Clock domain 2: scanclkTest clock 1: scanclk (root pin: scanclk)       Total endpoints: 82       System mode clock control: 1 clock pins.              Total
0 endpoints exempted from system clock control check.              82 endpoints are controlled by system clock scanclk.       Domain crossing capture
report: total 0 endpoints and 0 domains.       Domain crossing lauching report: total 0 endpoints and 0 domains.Test clock 2: DFT_test_clock_OD1 (root pin:
scanclk)       Total endpoints: 5       System mode clock control: 1 clock pins.              Total 0 endpoints exempted from system clock control check.
5 endpoints are controlled by system clock              DFT_OD1/i_opcg_clkout_mux/opcg_clk_mux_OD1_p2_d7_b1_opcg_              clkout_mux/mux_1_19_g1/Y.
Domain crossing capture report: total 5 endpoints and 1 domains.       5 endpoints receive data from domain 'UNKNOWN'.       Domain crossing lauching
report: total 0 endpoints and 0 domains.
```

Related Information

Related command:	add_opcg

report_opcg_equivalents

```
report_opcg_equivalents
[-pin_value] [design] [> file]
```

Description

Reports the `OPCG`-equivalence mappings specified using the `set_opcg_equivalent` command(s).

Options and Arguments

design	Specifies the design for which you want to report the `OPCG`-equivalency mappings.
file	Specifies the name of the file to which to write the report.
`-pin_value`	Prints the pin constant values.

Related Information

Affected by these commands:	reset_opcg_equivalent
	set_opcg_equivalent
Related commands:	convert_to_opcg_scan

report_ple

```
report_ple [design]... [> file]
```

Description

Returns the physical layout estimation information for the specified design. The command reports information like aspect ratio, shrink factor, site size, layer names, direction of layers (**H**orizontal, **V**ertical or **U**ndefined), layer utilization, capacitance, resistance, area, and the source used to extract the physical information.

Options and Arguments

design	Specifies the design name on which to report. If no design is specified, the current design is loaded.
file	Specifies the name of the file to which to write the report.

report_port

```
report_port [-delay] [-driver] [-load] port... [> file]
```

Description

Generates reports on the ports of the current design. By default, the report gives information on port direction, external delays, exception objects and their types, driver, slew, fanout load, pin capacitance and wire capacitance for the ports. You can also specify the port names on which the report is to be generated and control the data printed using the `-delay`, `-driver` and `-load` options.

Options and Arguments

`-delay`	Reports external delay information (rise and fall delay and the external delay object)
`-driver`	Reports the external driver name and the slew (rise and fall) values of the ports.
file	Specifies the name of the file to which to write the report.
`-load`	Reports the external fanout load and the pin and wire capacitance values of the ports.
port	Specifies the port for which to generate the report.

report_power

```
report_power
[-stims {string}...] [-frames {string}...] [-inst {hinst}...]
[-module {module}...] [-clock_domain {clock_domain}...]
[-power_domain {power_domain}...] [-collate {frames|hier|domain|all|none}]
[-power_mode {power_mode}] [-power_rail {power_rail}...] [-by_category]
[-by_hierarchy] [-by_leaf_instance] [-by_rail] [-by_func_type] [-by_libcell]
[-by_tiles {string}] [-unit W|mW|uW|nW] [-format {format}] [-header]
[-view {analysis_view}] [-skip_port_switching_power] [> file]
```

Description

Reports the power consumed. The information returned depends on your current *position* in the design hierarchy and on the specified objects. If no objects are specified, the report is given for the design or instance at the current *position* in the design hierarchy.

Nets connected to primary inputs and outputs are only reported at the top-level of an instance-based power report.

Options and Arguments

`-by_category`	Specifies to report the power by category. The following is the list of sub-options:	
	`-cols`	Specifies the columns to report. Default columns are: `leakage | internal | switching`
	`-category`	Reports power by the selected category. Category can be any of the following: {`memory | register | latch | logic | bbox | clock | clock_comb | clock_seq | pad | pm`} `pm` stands for power management cells (such as level shifter, retention, isolation cells).
	`-reassign_power`	Specifies to redistribute power of leaf instances specified using suboptions below from <*`from_cat`*> to <*`to_cat`*>. You can specify multiple `-reassign_power` options in the same command.
	`-sort_by`	Specifies to sort the output based on the selected parameter.
`-by_func_type`	Specifies to report the power by function type.	
	`-cell`	Leaf instances mapped to cell matching the specified pattern.
	`-cols`	Specify the columns to report. Default columns reported are: `leakage internal switching total type`
	`-lib`	Leaf instances mapped to cell in library matching the specified pattern.
	`-name`	Used to match the cell names (using glob expression), case insensitive match, and inverting the match. If `-nocase` is specified with this option, case sensitive matching is done. If `-invert` option is specified, it filters out names that match the pattern and return those that do not.
	`-rtl_group`	Each RTL group above is mapped to a set of RTL types.
	`-rtl_type`	Select the RTL type for reporting.
	`-sort_by`	Sort the report based on the specified parameter. If `-increasing` option is specified, the output is sorted in increasing order. If not specified, the data is sorted in decreasing order.
`-by_hierarchy`	Specifies to report the power by hierarchy. The hierarchy name is always displayed as the rightmost column in the report.	
	`-cols`	Columns to report. Default columns are: `cells pct_cells leakage internal switching total level hier`
	`-indent_inst`	Indentation to report for the instance. By default, full inst path is reported.
	`-levels`	Number of levels to report. Default is 2.
	`-min_leaf_cnt`	Skip hierarchies with leaf cells less than the specified value.
	`-sort_by`	Sort the report based on the specified parameter.
`-by_leaf_instance`	Specifies to report the power by leaf instance. This is the equivalent of the Genus legacy command: `report power -flat`	
	`-cell`	Leaf instances mapped to cell matching the specified pattern.
	`-cols`	Columns to report. Available options: `leakage internal switching total leaf` *Default*: `total`
	`-leaf`	Specifies to treat either a macro (for example, adder, multiplier) or gate (library element) as the leaf instance.

	`-lib`	Leaf instances mapped to cell in library matching the specified pattern.
	`-name`	Used to match the leaf instance by name, case insensitive match, and inverting the match. If `-nocase` is specified with this option, case sensitive matching is done. If `-invert` option is specified, it filters out names that match the pattern and return those that do not.
	`-rtl_group`	Used to select the type of leaf instance for reporting. Each RTL group above is mapped to a set of RTL types.
	`-rtl_type`	Used to select the type of leaf instance for reporting.
	`-sort_by`	If multiple tables are displayed, this option specifies to sort the order in which the tables are displayed by the specified category (register, memory, logic, clock, etc.) and optionally power type (leakage, internal, switching). By default, the tables are unsorted. If `-increasing` option is specified, the output is sorted in increasing order. If not specified, the data is sorted in decreasing order.
`-by_libcell`	Specifies to report the power by libcell.	
	`-rtl_group`	Used to select the type of leaf instance for reporting. Each RTL group above is mapped to a set of RTL types.
	`-rtl_type`	Used to select the type of leaf instance for reporting.
`-by_rail`	Reports power dissipation for each rail.	
`-by_tiles` *string*	Specifies to report the power by tiles, that is, <number of rows> X <number of columns>. *string* is of the following format: <*rows*>X<*cols*> [`-style {transient | simple}`] [`-cols {total | leakage | internal | switching}`].	
	`-style`	Specifies the style
		`simple | transient`
		Default: `transient`
	`-cols`	Specify the columns to report. Available options: `leakage internal switching total` *Default*: `total`
`-clock_domain` *clock_domain*		
	Specifies the name of the clock domain for which you want to report the power.	
`-collate {frames | hier | domain | all | none}`		
	Specifies to collate the power reports across frames, or hierarchical instances, or clock/power domains, or all of these.	
	Default: `frames`	
file	Specifies the name of the file to which to write the report.	
`-format` format	Specifies the format of numerics in the report.	
`-frames` string	Report power for specified list of frames. By default, power for all frames will be reported. *Default*: `frame#0 of all stimuli`	
`-header`	Generates the report with a header.	
`-inst` *hinst*	Specifies a list of hierarchical and/or leaf instances for power reporting.	
`-module` *module*	Specifies the design or module for which you want the power to be reported. Specify the path name to the design.	
	If specified, all instances of the specified module are selected for power reporting.	
	`-levels` *string*	Specifies the number of levels to be reported. Power is reported for all hierarchical instances found upto the specified level where level for root is 0. Default value is 0 (root). If `-levels all` is specified, all levels will be reported.
`-power_domain` *power_domain*		

	Reports power for the specified power domain(s).
`-power_mode` *power_mode*	
	Reports power for the specified power mode.
	The specified power mode must correspond to one of the power mode names defined with a `create_power_mode` command in the CPF file that was read in.
	If you have a multi-mode design and you omit this option, the report will apply to the current state of the design.
`-power_rail` *power_rail*	
	Reports power for the specified power rail(s).
`skip_port_switching_power`	
	Skips port switching power while reporting.
`-stims` *string*	Report power for specified list of stimuli.
	If not specified, power for all stimuli is reported.
`-unit W|mW|uW|nW|pW`	
	Specifies the power unit to be used for reporting. Default value is value of attribute `lp_power_unit`.
`-view` *analysis_view*	
	Specifies to report the power for the specified analysis view.

Related Information

Affected by these commands:	syn_generic
	syn_map
	syn_opt
Affected by these attributes:	cell_leakage_power
	leakage_power_scale_in_nW
	lp_asserted_probability
	lp_asserted_toggle_rate
	lp_power_unit
Related attributes	lp_clock_tree_buffers
	lp_clock_tree_leaf_max_fanout
	lp_computed_probability
	lp_computed_toggle_rate
	lp_leakage_power

report_power_intent

```
report_power_intent
[-design design]
[-isolation_rule_only | -level_shifter_rule_only
|-power_domain_only | -state_retention_rule_only]
[-domain domain] [-show_full_names][-power_states] [-supply string][-summary] [-detail] [> file]
```

Description

Prints out the power intent included in the power intent file that was read in.

This command is aliased to `report_low_power_intent`.

Options and Arguments

`-design` *design*	Specifies the design for which to create the report.
	If no design is specified, the report is given for the current design.
	This option is required when multiple designs are loaded.
`-detail`	Requests a detailed report.
`-domain` *domain*	Reports all the rules defined for the specified domain.
file	Specifies the name of the file to which to write the report.
`-isolation_rule_only`	
	Reports the isolation rules for the power domain they were defined.
`-level_shifter_only`	
	Reports the level shifter rules for the power domain they were defined.
`-power_domain_only`	
	Reports only the power domains.
`-power_states`	Prints for each power state a power state table.
`-show_full_names`	Specifies to print full names in the report.
`-state_retention_rule_only`	
	Reports the state retention rules for the power domain they were defined.
`-summary`	Reports the number of power domains, level shifter rules, isolation rules, state retention rules, and power modes.
`-supply` *string*	Prints the power state table for the specified supply. You can specify a supply set, a power pin, port, or net, or a ground pin, port, or net.

Examples

- The following command reports only Isolation rule defined in the power intent file:

```
report_power_intent -isolation_rule_only

Power Intent
============================
Format : CPF
File   : CPF/msv_pso.DTMF.cpf
============================
============================
Isolation Rules : Isorule
============================
```

- The following command reports only Level Shifter Rules defined in the power intent file:

```
report_power_intent -level_shifter_rule_only

Power Intent
============================
Format : CPF
File   : CPF/msv_pso.DTMF.cpf
============================

================================
Level Shifter Rules : LSruleH2L
                    : LSruleH2Lao
                    : LSruleL2H
================================
```

Related Information

Affected by these commands:	commit_power_intent
	read_power_intent

report_power_intent_instances

```
report_power_intent_instances [-design design]
[-instance instance] [-lp_instance instance]
[-isolation_only | -level_shifter_only |-combo_only
|-state_retention_only] [-show_full_names]
[-rule {isolation_rule | level_shifter_rule
 | state_retention_rule}]
[-summary] [-detail] [> file]
```

Description

Reports information about the low power cells inserted in the design.

> This command also reports pre-inserted low power cells.

Options and Arguments

`-combo_only`	Reports isolation-level shifter combo cells only.
`-design` *design*	Specifies the design for which to create the report.
	If no design is specified, the report is given for the current design.
	This option is required when multiple designs are loaded.
`-detail`	Requests a detailed level shifter report, including pins on which the low power cells were inserted.
file	Specifies the name of the file to which to write the report.
`-instance` *instance*	Restricts the reported information to the specified hierarchical instance.
`-isolation_only`	Reports isolation cells only
`-level_shifter_only`	Reports level shifters only.
`-lp_instance` *instance*	
	Reports detailed information for the specified low power cell instance.
`-rule {`*isolation_rule*`|`*level_shifter_rule*`|`*state_retention_rule*`}`	
	Reports the low power cells inserted for the specified rule.
`-show_full_names`	Specifies to print full names in the report.
`-state_retention_only`	
	Reports state retention cells only.
`-summary`	Reports the number of level shifters, isolation cells, combo cells, and state retention cells inserted in the design.

Examples

- The following command returns all low power cells inserted in the design:

```
genus:root:> report_power_intent_instances

============================================================
Generated by:           Genus(TM) Synthesis Solution
Generated on:           date
Module: dtmf_recvr_core ============================================================

Level Shifter Report
-------------------------------------------------
|         Design/Instance        |  Rule    |Count |
-------------------------------------------------
| dtmf_recvr_core                |LSruleL2H |   57 |
| dtmf_recvr_core/TDSP_CORE_INST |LSruleH2L |   53 |
-------------------------------------------------
 Total Level Shifters :                        110
=================================================
```

- The following command generates detailed report for level shifters which includes, for each level shifter rule, how many level shifters are inserted along with the pin and instance details.

```
genus:root:> report_power_intent_instances -level_shifter_only -detail

============================================================
Generated by:           Genus(TM) Synthesis Solution
Generated on:           date
Module: dtmf_recvr_core ============================================================

Level Shifter Report
-------------------------------------------------------------------------
|   RULE    |              PIN                  |       INSTANCE      |
-------------------------------------------------------------------------
| LSruleH2L |TDSP_CORE_INST/bio                 |CPF_LS_bio           |
|           |TDSP_CORE_INST/bus_grant           |CPF_LS_bus_grant     |
|           |TDSP_CORE_INST/clk                 |CPF_LS_clk           |
-------------------------------------------------------------------------
|           |TDSP_CORE_INST/t_data_in[9]        |CPF_LS_t_data_in_9   |
| LSruleL2H |TDSP_CORE_INST/address[0]          |CPF_LS_address_0     |
|           |TDSP_CORE_INST/address[1]          |CPF_LS_address_1     |
|           |TDSP_CORE_INST/address[2]          |CPF_LS_address_2     |
|-----------------------------------------------------------------------
|           |TDSP_CORE_INST/write               |CPF_LS_write         |
-------------------------------------------------------------------------

| Total Level Shifters :                                        110
=========================================================================
```

Related Information

Affected by these commands:	commit_power_intent

report_prototype

```
report_prototype [-hdl] [-feasible_targets [-num integer]] [> file]
```

Description

Reports information about the prototype synthesis flow.

> Limited access feature

Options and Arguments

file	Specifies the file to which the report must be written.
-feasible_targets	Prints the endpoints whose timing constraints are relaxed with path adjust exceptions.
	The results are sorted first according to the path adjust value, then according to the path adjust exception name, then according to the endpoint names.
-hdl	Prints a summary of incomplete RTL scenarios found during elaboration.
-num *integer*	Specifies the number of endpoints to be printed according to the decreasing path adjust value.
	If this option is not specified, all paths with path_adjust values are printed.

Examples

- The following command prints a list of endpoints whose timing constraints were relaxed.

```
genus@root: number> report_prototype -feasible_targets

============================================================
  ...
============================================================

============================================================
No.  Endpoint             Path Adjust       Path Adjust
                           Value(ps)        Exception
============================================================

1   top/piano/u_reg/d         1500         proto_ft_adj_1
2   top/piano/p_reg/d         1500         proto_ft_adj_2
3   top/piano/p_reg/d         1500         proto_ft_adj_2
4   top/q_reg1/d               450         proto_ft_zipped_1
5   top/q_reg2/d               450         proto_ft_zipped_2
============================================================
```

- The following command prints a list of incomplete RTL scenarios.

```
genus@root: number> report_prototype -hdl

============================================================
  ...
============================================================

INCOMPLETE HDL SCENARIO FOR DESIGN: design:top

PORTS SEEN IN INSTANCES BUT MISSING IN MODULE DEFINITION
============================================================

PORT NAME:     in2
INSTANCE NAME: u1
FILE NAME:     missing_port.v
MODULE NAME:   mid

INCOMPLETE HDL SCENARIO FOR module: module:top/bot

PORTS SEEN IN INSTANCES BUT MISSING IN MODULE DEFINITION
============================================================

PORT NAME:     in3
INSTANCE NAME: I1
FILE NAME:     missing_port.v
MODULE NAME:   mid
```

Related Information

Affected by these commands:	elaborate
	syn_map
Related attributes:	proto_feasible_target
	proto_hdl

report_qor

```
report_qor
[-levels_of_logic] [-no_power | -power]
[-exclude_constant_nets] [-ndr_min_layer]
[-view analysis_view] [design]... [> file]
```

Description

Reports the critical path slack, total negative slack (TNS), number of gates on the critical path, and number of violating paths for each cost group. It also gives the instance count, total area (net and cell area), cell area, runtime, and host name information.

> Power information is only reported by default if you set at least one of these power constraint attributes: `max_leakage_power` or `max_dynamic_power`.

If you perform physical synthesis and start with a floorplan, the report also contains the floorplan utilization in %.

> In case of a multi-mode design, the TNS column is not printed.

Options and Arguments

design	Specifies the design name on which to report. If no design is specified, the report is given for the current design.
-exclude_constant_nets	
	Excludes constant nets from the fanout statistics computation. This can affect the `Max Fanout`, `Min Fanout`, `Average Fanout` and the `Terms to net ratio` numbers in the report.
file	Specifies the file to which the report must be written.
-levels_of_logic	Prints the number of gates on the critical path per cost group.
-ndr_min_layer	Reports the number of non-default routes and nets assigned to higher metal layers by Innovus.
-no_power	Prevents power computation for this report.
-power	Forces to compute and report the power results when the power constraints were not set.
-view *analysis_view*	
	Specifies that the report should only apply to the specified *analysis_view*.

Related Information

Affected by these attributes:	report_ndr_min_layer_count

report_runtime

```
report_runtime
[string] [> file]
```

Description

Reports the runtime and memory used up to this stage (time that the information was requested).

Options and Arguments

string	Specifies a user-defined string which allows you to identify at which stage in the design the information was requested.
	Default: `undefined`.
file	Redirects the command output to the specified file.

report_scan_chains

```
report_scan_chains [design] [-chain actual_scan_chain]...
[-dft_cfg_mode dft_config_mode_name] [-numerical_order]
[-opcg_side_scan] [-partition string]
[-summary] [-power_domain] [> file]
```

Description

Reports for every chain in the design the following elements:

- The scan data input port and its hookup pin
- The scan data output port and its hookup pin
- The shift enable port and its hookup pin
- The DFT clock domain and the DFT clock domain edge of the chain (for muxed scan only)
- The length of the chain
- The elements in the chain
  In case of the muxed scan style, the report also lists for each element the test clock and test clock edge determined by the `check_dft_rules` command for that element.
- Any user-specified segments with their elements
  In case of the muxed scan style, this includes data lockup elements, and the location of the data lockup element in the chain.
- The bit position and length of any user-specified segment in the chain
- The head and tail test clock and test clock edge for any abstract segment in the chain

Options and Arguments

`-chain` *actual_scan_chain...*	
	Reports only the listed scan chains
design	Specifies the design for which you want to generate a report.
	If you have multiple top designs, you must either specify the design name, or change to the directory in the design hierarchy that contains the design for which you want the report.
`-dft_cfg_mode` *dft_configuration_mode_name*	
	Reports the scan chains related to the specified scan mode.
file	Specifies the name of the file to which to write the report.
`-numerical_order`	Reports scan chains in numerical order.
`-opcg_side_scan`	Reports only the side scan chains that contain the domain macro segments.
`-partition` *string*	Specifies the partition to process. If not specified, design is used.

`-power_domain`	Reports the power domain of scan chain elements.
`-summary`	Condenses the scan chain report to include only chain name, scan-data pins, shift enable, clock domain and length information.

Related Information

Affected by this command:	connect_scan_chains
	compress_scan_chains
Related attributes:	Actual Scan Chain Attributes
	Actual Scan Segment Attributes

report_scan_compressibility

`report_scan_compressibility -directory` *`atpg_directory`* `[>` *`file`*`]`

Description

Reports the scan compressibility of a design. This command is used to report the results of the `analyze_scan_compressibility` command which determines an optimal Compression architecture for your design by performing multiple compression analysis using Modus.The compression analysis can be run on the scan mapped design prior to adding the Test compression logic.

Options and Arguments

`-directory` *`atpg_directory`*	
	Specifies the directory containing the ATPG compression runs created by the `analyze_scan_compressibility` command.
	Default: *`current_working_directory`*`/asc`
`file`	Specifies the name of the file to which the report must be written.

Example

The following command reports the compressibility analysis resulting from the ATPG directory `asc0`.

```
    genus@root: number> report_scan_compressibility -directory asc0


Analyzing from work directory 'asco' .....

----------------------------------------Results of analyze_scan_compressibility----------------------------------------       Design       - test
Decompressor - broadcast       Compressor   - misr       mask         - widel
Achieved compression table with fullscan topup vectors
-----------------------------------------------------
IC    TATR   TDVR    ATCov.   CL       Pat-comp  Pat-fs    Cycles        Runtime    Gatecount    Area-------------------------------------------------------------
-----------------------------------------fs    1.0    1.0     100.00%  63       -         3         336           00:00:00   -            -   2     0.3    0.5
99.91%   32       9         5         1258          00:00:01   437          24774.3999999
Achieved compression table without fullscan topup vectors
-----------------------------------------------------
IC    TATR   TDVR    ATCov.   CL       Pat-comp  Pat-fs    Cycles-----------------------------------------------------------------fs    1.0    1.0     100.00%
63                3         336       2     0.4    1.2     98.76%   32       9         -         838
Total atpg runtime for exp. 00:00:01 hrs.

IC        - Inserted compressionTATR      - Test application time reductionTDVR      - Test data volume reductionCov.      - Atpg coverageCL.       - Channel
```

```
LengthPat-comp  - No. of compression test patternsPat-fs    - No. of fullscan test patternsRuntime   - Atpg runtimefs        - fullscan run
```

Related Information

Affected by this command:	analyze_scan_compressibility

report_scan_compression_logic

```
report_scan_compression_logic
[-compactor hinst] [-testic module]
[-highlight_in_gui] [-colors string]
[-report_unplaced] [-report_tracing {none|summary|detail|debug}]
```

Description

Reports the area and wirelength overhead of the scan compression logic.

> The wirelength overhead is only reported if the design is placed.

Options and Arguments

`-colors` *string*	Specifies a list of category and color pairs.
	Category can be any of: `SI`, `HEAD`, `TAIL`, `MASK`, `FULLSCAN`, `CODEC`, or `SO`.
	Default: `"HEAD lightblue TAIL lightgreen MASK blue FULLSCAN yellow CODEC red"`
`-compactor` *hinst*	Specifies the name of the scan compression logic macro.
	Default: `COMPACTOR`
`-hightlight_in_gui`	
	Highlights the scan compression logic wires in the GUI.
`-report_tracing {none|summary|detail|debug}`	
	Reports details on compression logic tracing.
	Default: `none`
`-report_unplaced`	Reports the unplaced pins and ports.
`-testic` *module*	Specifies the name of the test interconnect macro if present.
	Default: `TestIC`

Example

```
genus:root:> report_scan_compression_logic -compactor COMPACTOR

Reporting statistics for scan compression macro: hinst:DLX_CORE/COMPACTOR ...

                Area (um^2)     #Insts      #Bufs      #Nets Wirelength (um)SI to CoDec          125.856         47          0          8
974.936Channel Heads        636.120        128          0         64         169.622Channel Tails         87.552         64          0         64
```

```
1706.644CoDec                757.872        294          2        471       10314.399Mask                1140.570        308          0        247
5603.287Fullscan Muxing        0.684          1          1          2        1258.399CoDec to SO           13.680          8          0          8
85.400

All                 2762.334        850          3        864       20112.687
```

report_scan_registers

```
report_scan_registers [-pass_tdrc] [-fail_tdrc]
[-lockup] [-latch] [-dont_scan] [-other]
[-abstract_dont_scan] [-shift_register] [-multi-bit]
[-test_clock test_clock [-test_clock_edge test_clock_edge]] [design] [-not_in_scan_chain] [> file]
```

Description

Reports the DFT status of all flip-flop instances in the design. Use this command after running `check_dft_rules`. More specifically, the command reports

- Registers which pass the DFT rule checker
  For each flip-flop, it reports the hierarchical instance name along with their DFT test clock domain and active edge.
- Registers which fail the DFT rule checker
  For each flip-flop, it reports the hierarchical instance name along with the violation type (clock, or asynchronous set/reset) and the violation Id number.
  For an abstract segment that fails the DFT rule checker, it reports the name of the abstract segment, and the number of flip-flops in the segment.
- Registers that are preserved or marked `dont_scan`

  Mapped flip-flops can be listed in this category if

  - The flip-flop is marked preserved and it is mapped to a non-scan flop
    However, if a flip-flop is marked preserved and is already mapped to scan for DFT purposes, it will be listed as either passing or failing the DFT rule checker—based on the DFT rule checker analysis—and it will not be listed in this category.
  - The flip-flop is mapped to a scan flop for non-DFT purposes
- Registers that are marked Abstract Segment dont-scan.
- Registers that are part of shift register segments
- Registers that are identified as lockup elements
- Registers that are level-sensitive elements
- Registers that are implemented as multibit
- Registers not part of any of the other categories

Without any options specified, the command reports on all categories of registers.

The output of the `report_scan_registers` command is independent of the selected scan style.

Options and Arguments

design	Specifies the design for which you want to generate a report. By default a report is created for all designs currently loaded in memory.
-dont_scan	Reports all edge-triggered registers that are not to be mapped to scan flops, or connected into the top-level chains.
-abstract_dont_scan	
	Reports registers marked dont_scan because of abstract segment defined on it or on the parent module.
-fail_tdrc	Reports all edge-triggered registers that fail the DFT rule checks.
file	Specifies the name of the file to which to write the report.
-latch	Reports all registers of type latch in the design.

	Latch instances which are instantiated as lockup elements in a preserved segment are reported with the `-lockup` option.
`-lockup`	Reports data lockup elements of type latch or flop that are either instantiated in a preserved segment, or added to the design during scan chain configuration.
`-other`	Reports all registers that are not reported through any of the other options, such as clock-gating registers.
`-multi_bit`	Reports all registers that are implemented as multibit elements.
`-pass_tdrc`	Reports all edge-triggered registers that pass the DFT rule checks.
`-not_in_scan_chain`	
	Reports registers that pass dft rule checks and are not a part of scan chains.
`-shift_register`	Reports all registers that are part of shift register segments.
`-test_clock` *test_clock*	
	Returns the number of registers controlled by the specified test clock.
`-test_clock_edge` *test_clock_edge*	
	Returns the number of registers controlled by the specified test clock edge.
	This option must be specified with the `-test_clock` option.

Examples

- This example shows the report for a design without any user-defined test clocks.

```
report_scan_registers

Reporting registers that pass DFT rules

u_core/out_reg_0 PASS; Test clock: clk/rise
...
u_core/out_reg_7 PASS; Test clock: clk/rise
Reporting registers that fail DFT rules
Reporting registers that are preserved or marked dont-scan
Reporting registers that are marked Abstract Segment Dont Scan
Reporting registers that are part of shift register segments
Reporting registers that are identified as lockup elements
Reporting registers that are level-sensitive elements
Reporting misc. non-scan registers

Summary:
Total registers that pass DFT rules: 8
Total registers that fail DFT rules: 0
Total registers that are marked preserved or dont-scan: 0
Total registers that are marked Abstract Segment dont-scan: 0
Total registers that are part of shift register segments: 0
Total registers that are lockup elements: 0
Total registers that are level-sensitive: 0
Total registers that are misc. non-scan: 0
Note: For each flip-flop that passes the DFT rules, the command reports the hierarchical
instance name along with their DFT test-clock domain and active edge.
```

- This example shows the report for the design without any user-defined test clocks, using the clocked LSSD scan style.

```
report_scan__registers
```

```
Reporting registers that pass DFT rules

u_core/out_reg_0 PASS; Test clock: clk/rise;
...
u_core/out_reg_7 PASS; Test clock: clk/rise;
Reporting registers that fail DFT rules
Reporting registers that are preserved or marked dont-scan
Reporting registers that are marked Abstract Segment Dont Scan
Reporting registers that are part of shift register segments
Reporting registers that are identified as lockup elements
Reporting registers that are level-sensitive elements
Reporting misc. non-scan registers

Summary:
Total registers that pass DFT rules: 8
Total registers that fail DFT rules: 0
Total registers that are marked preserved or dont-scan: 0
Total registers that are marked Abstract Segment dont-scan: 0
Total registers that are part of shift register segments: 0
Total registers that are lockup elements: 0
Total registers that are level-sensitive: 0
Total registers that are misc. non-scan: 0
```

- Reporting failing registers only

```
report_scan_registers -fail_tdrc

Reporting registers that fail DFT rules

Segment abstract1 FAIL; (length 5)
Summary:
Total registers that fail DFT rules: 5
```

- Reporting passing registers only.

```
report_scan_registers -pass_tdrc

Reporting registers that pass DFT rules

out1_reg_0 PASS; Test clock: clk1/rise
...
out2_reg_6 PASS; Test clock: clk2/rise
out2_reg_7 PASS; Test clock: clk2/rise
Segment abstract2 PASS; Test clock: clk2/rise -> clk2/rise (length 3)

Summary:
Total registers that pass DFT rules: 15
```

Related Information

Affected by this command:	check_dft_rules
Related attributes:	dft_dont_scan
	dft_scan_style
	dft_status
	dft_violation
	preserve
	Scan Segment Attributes

report_scan_setup

```
report_scan_setup [design] [> file]
```

Description

Reports DFT setup information for the design including both user-defined and tool-inferred information.

You must load and elaborate a design before you can use this command. The contents of this report further depends on the stage of the design that you are at.

Use this command at the end of the design process to document the DFT setup and resulting configuration of the design.

Options and Arguments

design	Specifies the design for which you want to generate a report.
	By default a report is created for all designs currently loaded in memory.
file	Specifies the name of the file to which to write the report.

Examples

- The following example shows the report after you have elaborated the design. Because you have not specified any setup information yet, the information shown is based on the default settings for DFT.

```
genus@root: number> report_scan_setup

Design Name
===========
   top

Scan Style
==========
    muxed_scan
DFT rule check status is not available. Need to (re)run check_dft_rules

Global Constraints
==================
    Minimum number of scan chains: no_value
    Maximum length of scan chains: no_value
    Lock-up element type: level_sensitive
    Mix clock edges in scan chain: false
    Prefix for unnamed scan objects: DFT_

Test signal objects
===================

Test clock objects
==================
Reporting all test clocks as TDRC status is not available

DFT controllable objects
========================

DFT don’t scan objects
======================

DFT abstract don’t scan objects
===============================

DFT scan segment constraints
============================

DFT scan chain constraints
```

```
=========================

DFT actual scan chains
======================
```

- The following example shows the report after the scan configuration engine was run.

```
genus@root: number:design:test> report_scan_setup

Design Name
===========
   top

Scan Style
==========
    muxed_scan
Design has a valid DFT rule check status

Global Constraints
==================
    Minimum no of Scan chains: no_value
    Maximum length of scan chains: no_value
    Lock-up element type: level_sensitive
    Mix clock edges in scan chain: true
    Prefix to name user defined scan chains: DFT_

Test signal objects
===================
    shift_enable:
          object name: SE
          pin name: SE
          hookup_pin: SE
          hookup_polarity: non_inverted
          active: high
          ideal: true
          user defined: true


Test clock objects
==================
    test_clock:
          object name: clk
          user defined: false
          source: clk
          root source: clk
          root source polarity: non_inverting
          hookup_pin: clk
          period: 50000.0


DFT controllable objects
========================

DFT don’t scan objects
======================

DFT abstract don’t scan objects
===============================

DFT scan segment constraints
```

```
============================
    Segment:
          object name: segHead
          scan-in:
          scan-out:
          shift-enable: internal
          connected_shift_enable: false
          length: 2
          type: floating

    Segment:
         object name: segTail
         ...

    Segment:
          object name: segBody
          scan-in:
          scan-out:
          shift-enable: internal
          connected_shift_enable: false
          length: 2
          type: fixed


DFT scan chain constraints
==========================
    User Chain:
          object name: DFT_chain_1
          scan-in: in[0]
          scan-in hookup_pin: in[0]
          scan-out: out[1]
          scan-out hookup_pin: out[1]
          shared out: true
          shift_enable object name:
          max length: no_value
          head segment: segHead
          tail segment: segTail
          complete: false


DFT actual scan chains
======================
    Actual Chain:
          object name: DFT_chain_1
          scan-in: in[0]
          scan-in hookup_pin: in[0]
          scan-out: out[1]
          scan-out hookup_pin: out[1]
          shared out: true
          shift_enable: SE
          length: 8
          segment objects: segHead segBody segTail
          analyzed: false
          test_clock domain: clk
          test_clock edge: any
```

Related Information

Affected by these constraints:	define_test_signal -function <shift_enable | test_mode>
	define_test_clock
Affected by this command:	check_dft_rules
	connect_scan_chains
Related attributes:	dft_controllable
	dft_dont_scan
	dft_lockup_element_type
	dft_max_length_of_scan_chains
	dft_min_number_of_scan_chains
	dft_mix_clock_edges_in_scan_chains
	dft_prefix
	dft_scan_style
	preserve
	Actual Scan Chain Attributes
	Actual Scan Segment Attributes
	Scan Segment Attributes
	Scan Chain Attributes
	Test Clock Attributes
	Test Signal Attributes

report_sequential

```
report_sequential
[-insts instance...] [-hier] [-full] [-deleted ]
[module | design |hinst |inst] [-hinst {hinstance|instance}] [-mapped] [-optimized] [> file]
```

Description

Generates a report on the sequential elements of the current design. The report provides the sequential element name, row, column, filename information, and the sequential element type (flip-flop (async set/rest, sync set/reset, sync enable), latch, or timing model).

> Set the hdl_track_filename_row_col attribute to `true` before using the `elaborate` command to track the filename, row and column information.

Options and Arguments

file	Specifies the name of the file to which to write the report.
-deleted	Reports the sequential elements that were removed during optimization. For example, sequential elements can be removed during constant propagation, redundancy removal, when merged with other sequential instances, or when they are not driving any primary outputs.

	If the `delete_unloaded_seqs` attribute is set to `false`, the sequential elements that were optimized but did not get deleted will also be reported. The reason for these flops is reported with an asterisk (*). In this case, the flop's output pin will be dangling, not driving any loads. The loads driven by the flop before optimization, will be driven by a constant, or by the output pins of the merged flops pin.
	This option cannot be combined with any other option.
`-hier`	Reports all the flops in the design.
`-full`	Reports all the sequential elements in the design.
`-hinst {`*`hinstance`*`|`*`instance`*`}`	
	Specifies the hierarchical instances name to report sequential elements.
`-insts` *`instance`*	Specifies the instance name to report sequential elements.
`-mapped`	Specifies to report all messages raised during mapping on all or given instances.
`module` `|` *`design`* `|` *`hinst`* `|` *`inst`*	Specifies the design, module, or the hierarchical instance name to report flops.
`-optimized`	Specifies to report all the optimized but not deleted sequential elements from the design.

report_signoff_timing

```
report_signoff_timing
[-sdc sdc_files] [-netlist path] [-libraries lib_list]
[-include file | -report_timing_command command]
[> file]
```

Description

Generates an Encounter Timing System timing report.

To run this command you need to have access to the Encounter ® Timing System software.

Options and Arguments

`file`	Specifies the name of the file to which the report must be written.
	Default: `vtim_ets_timing_rpt`
`-include` *`file`*	Specifies the file containing the Encounter Timing System commands to be executed.
	These commands will be read in after the libraries, netlist and SDC constraints have been read into the Encounter Timing System tool.
`-libraries` *`lib_list`*	Specifies the list of libraries to be read by Encounter Timing System.
	If no libraries are specified, the same libraries that were specified for synthesis are used.
`-netlist` *`path`*	Specifies the path to the netlist.
	If no netlist is specified, the netlist is generated with the `write_hdl` command in the `.vtim_ets` directory.

-report_timing_command *command*	
	Specifies a string that contains a single Encounter Timing System report command.
-sdc *sdc_files*	Specifies the SDC file(s) for the design.
	If no SDC file(s) are specified, the SDC files are generated using the `write_sdc` command in the `.vtim_ets` directory.

report_slew_calculation

```
report_slew_calculation  {pin|hpin}
[-rise | -fall]
[-view analysis_view] [> file]
```

Description

Reports how the slew of a cell driver pin is calculated from the look up table in the loaded technology library. The formula for calculating the delay is provided at the bottom of the report.

Options and Arguments

file	Redirects the report to the specified file.
[-fall | -rise]	Uses the falling or rising slew calculation on the driver pin.
	By default, all possible arcs are reported.
{*pin*|*hpin*}	Specifies the cell driver pin.
-view *analysis_view*	
	Specifies that the report should only apply to the specified *analysis_view*.

report_summary

```
report_summary [-directory path]
```

Description

Generates a summary table which includes various QoS numbers for various stages in the Genus flow.

> You must have run the `write_reports` or `write_snapshot` command before you use this command.

Options and Arguments

-directory *path*	Specifies the path to the directory where the output data of the this command must be stored.

Related Information

Affected by these commands:	syn_generic
	syn_map
	write_reports
Affected by this attribute:	qos_report_power

report_test_power

```
report_test_power
[[-clock float][-flop_toggle_percentage float]
| -atpg  -library string  [-clock float]
  [-directory path] [-capture | -scan_shift]
  [-lower_bound | -upper_bound]
  [-compression_mode mode]
| -tcf file] [-view analysis_view]
[-power_mode {power_mode}] [> file]
```

Description

Reports the estimated average power consumption of the design during test. Additionally, when using the `-atpg` option, the command reports the average switching activities for the scan shift or capture mode. The test power will be estimated using the power tables in the synthesis library (.lib).

This command should be run after the design is mapped to technology cells and scan chains are connected. Test power can only be estimated in a scan-mapped design with functioning scan chains. If the netlist inserted into the synthesis session has existing scan chains, scan chains must first be analyzed using the `define_scan_chain -analyze` before executing this command.

Options and Arguments

`-atpg`	Invokes Modus to compute the toggle activity from the test patterns.
	To use this option, you must first build the scan chains using the `connect_scan_chains` command. Or if you start from a netlist with existing scan chains, you must first analyze the scan chains using the `define_scan_chain -analyze` command before you can use this option.
`-capture`	Estimates the average power consumed during the capture mode of the test patterns.
	If neither the `-scan_shift` or `-capture` option are specified, the default is `-scan_shift`.
`-clock float`	Specifies the frequency in MHz for the specified test mode (capture or scan).
	Default: frequency of the test clock with highest frequency.
`-compression_mode {COMPRESSION | COMPRESSION_DECOMP | OPMISRPLUS | OPMISRPLUS_DECOMP}`	
	Specifies which compression test mode to select for power analysis.
	If you omit this option, the default test mode that will be used is `FULLSCAN`.
`-directory string`	Specifies the work directory to which the script files must be written. If the script files exist, the command can overwrite them.
	Default: *`current_working_directory`*`/test_power`
`file`	Specifies the file to which to redirect the report.
`-flop_toggle_percentage float`	
	Specifies the percentage of the flops that are changing state in each test clock cycle.
	Default: 50% of the activity level in the design.
`-library string`	Specifies the list of Verilog structural library files. Specify the list in a quoted string.
	This option is only required when you invoke this command on a mapped netlist.

`-lower_bound`	Runs ATPG with repeat fill to provide a lower bound on the estimated power.
	If you specified neither the `-lower_bound` nor the `-upper_bound` option, the `-upper_bound` option will be used by default.
	This option must be specified with the `-atpg` option.
`-power_mode {power_mode}`	
	Specifies the mode for which the test power must be reported.
	This option is required when the design has multiple power modes.
`-scan_shift`	Reports the average power consumed in scan shift mode.
	If neither the `-scan_shift` or `-capture` option are specified, the default is `-scan_shift`.
`-tcf file`	Specifies the TCF file from which to read the switching activities.
`-upper_bound`	Runs ATPG with random fill to provide an upper bound on the estimated power.
	If you specified neither the `-lower_bound` nor the `-upper_bound` option, the `-upper_bound` option will be used by default.
	This option must be specified with the `-atpg` option.
`-view analysis_view`	
	Reports the power for the given analysis view.

Related Information

Affected by this constraint:	define_test_clock

report_timing

```
report_timing
[-fields column_list] [-output_format {text|gtd}]
[-path_type {full|summary|full_clock|endpoint}]  [-path_exceptions]
[-split_delay | -stage_delay] [-hpins] [-nets]
[-physical] [-user_derate]
[{-from | -from_fall | -from_rise}  {inst|hsinst|external_delay|clock|port|pin}...]
[-through {inst|hsinst|port|pin}...]...
[-through_fall {inst|hsinst|port|pin}...]...
[-through_rise {inst|hsinst|port|pin}...]...
[{-to|-to_fall|-to_rise}
  {inst|hsinst|external_delay|clock|port|pin}...]
[{-capture_clock_pins | -capture_clock_pins_fall_clock
| -capture_clock_pins_rise_clock
| -capture_clock_pins_fall_pin
| -capture_clock_pins_rise_pin}  {pin|hpin}...]
[-max_paths integer | -nworst integer]
[[-max_slack integer] [-min_slack integer]
| -unconstrained ]
[-logic_levels integer] [-logic_levels_tcl_list]
[-skip_buf] [-skip_inv]
[-exception_data] [-exceptions exceptions]
[-group group_list]
[-domain clock_domain] [-views analysis_view]
[-paths string] [-timing_path path] [-timing_bin bin]
[-collection] [-gui] [-incr_derate] [-slew_derate]
[-user_mean_derate] [-user_sigma_derate]
[-not_through {inst|hinst|port|pin|hpin}..]
[-not_through_rise {inst|hinst|port|pin|hpin}..]
[-not_through_fall {inst|hinst|port|pin|hpin}..]
[> file]
```

Description

Generates a timing report of the current design. By default, the report gives a detailed view of the critical path of the current design. If the current session has multiple designs, use the `vcd` command to navigate to the desired design to generate the report. You can also generate a report on possible timing constraint problems (timing lint) or view slack at endpoints.

> All values are always expressed in picoseconds. This unit cannot be changed.

> When Genus detects a combinational feedback loop, it inserts a buffer from the technology library as a loop breaker instance before it performs timing analysis. To add the loop breaker, Genus might first need to uniquify a hierarchical instance which can result in a change in the netlist. As the module and instance name can change, this can affect your scripts and your database search.

Where applicable, special footnotes are used as shown in the table below to enhance the usability of the report.

Footnote	Meaning
(*)	Zero Slack Borrow Limit = Maximum amount that can be borrowed without violating timing on the lending side
(@)	Annotated capacitance
(#)	NDR net
(a)	Net has asynchronous load pins which are being considered ideal
(b)	Timing paths are broken
(B)	Net has high fanout (fanout >= 1000)
(C)	Cell belongs to congested region (applies only to physical flow)

(e)	Net has extra wire length
(H)	Estimated wire delay based on physical-aware mapping using virtual buffering as needed.
(i)	Net is ideal
(m)	Attribute `cell_delay_multiplier` is modified for this library cell
(P)	Instance is preserved
(p)	Instance is preserved but may be resized
(u)	Net has unmapped pin(s)
(V)	Net with virtual buffer(s)

Options and Arguments

`-capture_clock_pins {`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of sequential clock pins that capture data. This option can be useful with complex sequential cells such as RAMs that
`-capture_clock_pins_fall_clock {`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the falling edge of the ideal clock waveform. This option can be useful wit
`-capture_clock_pins_fall_pin {`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the fall transitions of the specified sequential clock pin. This option can be
`-capture_clock_pins_rise_clock {`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the rising edge of the ideal clock waveform.
`-capture_clock_pins_rise_pin {`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the rise transitions of the specified sequential clock pin.
`-collection`	`Specifies to return a collection of timing paths.`
`-domain` *`clock_domain`*	
	Restricts the paths to the specified clock domain.
`-exceptions` *`exceptions`*	
	Reports only paths to which one of the specified exceptions applies.
`-exception_data`	`Displays the exception cross-linking information.`
`-fields` *`column_list`*	
	Formats the report according to the *`column_list`*. The *`column_list`* specifies which columns to display in the timing report and the order i
	`aocv_derate`—aocv derating information
	`arc`—timing arc for which the timing information was applied
	`arrival`—arrival time at a leaf cell
	`arrival_mean`—mean arrival time at a leaf cell

	`arrival_sigma`—sigma arrival time at a leaf cell
	`cell`—cell type, exception type
	`delay`—delay through a leaf cell
	`delay_mean`—mean delay through a leaf cell
	`delay_sigma`—sigma delay through a leaf cell
	`edge`—rising or falling edge
	`endpoint_info`— endpoint information
	`fanout`—fanout of leaf cells
	`flags`—flags associated with the column timing point
	`frc`—file row-column information
	`incr_derate`—incr derating information
	`slew_derate`—slew derating information
	`instance_location`—instance location
	`lib_set`—library set of the cell
	`load`—load of the leaf cells
	`module`—name of the subdesign
	`pin_location`—pin location
	`power_domain`—power_domain associated with the object
	`socv_derate`—socv spatial derating information
	`stage_count`—aocv stage depth
	`timing_point`—names of the pins
	`total_derate`—total derating information
	`transition`—slew of the leaf cell outputs
	`transition_mean`—mean slew of the leaf cell outputs
	`transition_sigma`—sigma slew of the leaf cell outputs
	`user_derate`—derating information
	`user_mean_derate`—mean derating information
	`user_sigma_derate`—sigma derating information
	`voltage`—voltage information
	`wire_length`—wire length
file	Specifies the name of the file to which to write the report.
`-from {`*inst*`|`*hinst*`|`*external_delay*`|`*clock*`|`*port*`|`*pin*`|`*hpin*`}`	
	Specifies a Tcl list of start points for the paths. The start points can be input ports of your design, clock pins of fl
`-from_fall {`*inst*`|`*hinst*`|`*external_delay*`|`*clock*`|`*port*`|`*pin*`|`*hpin*`}`	
	Specifies a Tcl list of start points with fall transition at the start point. The start points can be input ports of you
`-from_rise {`*inst*`|`*hinst*`|`*external_delay*`|`*clock*`|`*port*`|`*pin*`|`*hpin*`}`	

	Specifies a Tcl list of start points with a rise transition at the start point. The start points can be input ports of y
-group *group_list*	Reports paths for the specified path group or cost group.
-gui	Allows you to create a detailed timing report in the GUI without having to use the menu commands. By using this option f
	This option has only effect when you run the tool in GUI mode.
-hpins	Displays the hierarchical pins in the data path.
-incr_derate	Controls the display of an additional column for the incr_derate values in the timing report.
-logic_levels *integer*	
	Specifies the number of paths to report in the output.
-logic_levels_tcl_list	
	Returns the start and end points, and the number of levels of logic for the number of top paths reported in Tcl format.
-max_paths *integer*	Specifies the maximum number of paths to report.
	Default: the value of -nworst
-max_slack *integer*	Reports only paths with a slack smaller than the specified number. The slack is reported in picoseconds.
	This option cannot be combined with the -unconstrained option.
-min_slack *integer*	Reports only paths with a slack greater than the specified number. The slack is reported in picoseconds.
	This option cannot be combined with the -unconstrained option.
-nets	Adds a row to the "Timing Point" field column to display the net names.
-not_through {*pin*|*hpin*|*port*|*inst*|*hinst*}	
	Specifies a Tcl list of paths that do not pass through these pins or instances. Objects can be ports, hierarchical pins, pins on a sequential
-not_through_fall {*pin*|*hpin*|*port*|*inst*|*hinst*}	
	Specifies paths that do not contain a fall transition in one of these objects. Objects can be ports, hierarchical pins, pins on a sequential o
-not_through_rise {*pin*|*hpin*|*port*|*inst*|*hinst*}	
	Specifies paths that do not contain a rise transition in one of these objects. Objects can be ports, hierarchical pins, pins on a sequential o
-nworst *integer*	Specifies the maximum number of paths to report to each endpoint.
	Default: 1
-output_format {text|gtd}	
	Specifies whether to create the report in machine-readable format or in text format.
	gtd – Generates the report in machine readable format. This is a file format used for representing timing information that can be understood by slack, failing paths, components contributing the most delay in each path, the total number of violations, and more in the design.
	Default: text
-path_type {full|summary|full_clock|endpoint]	
	Specifies the format of the report by path type:
	endpoint—Generates an end point report for each path consisting of an endpoint, cause, slack, arrival time, required time, and phase.
	full—Generates a report that displays the full path with accompanying required time and slack calculation.

	`full_clock`—Reports the full clock path (Other End Path) in addition to the full data path (Timing Path) if the path reported ends at a timir
	`summary`—Generates a summary report for each path consisting of a start point, endpoint, cause, slack, arrival time, required time, and ph
	`Default: full`
`-path_exceptions` *`string`*	Specifies the supported path exceptions. all | applied | ignored
`-paths` *`string`*	Reports only the specified timing restricted paths. Create the string argument using the set_path_specification command.
`-physical`	Reports physical information, like the x, y location.
`-skip_buf`	Excludes buffers from the logic level count.
`-skip_inv`	Excludes inverters from the logic level count.
-slew_derate	Controls the display of an additional column for the `slew_derate` values in the timing report.
`-split_delay`	Displays both drivers and loads in the data path.
`-stage_delay`	Displays only the drivers in the data path.
`-through {`*`inst`*`|`*`hinst`*`|`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of a sequence of points that a path must traverse. Points to traverse can be ports, hierarchical pins, pins on a sequenti
	You can repeat the -`through_rise` option to require that a path first must traverse one of the objects in the first set, then pass through one
`-through_fall {`*`inst`*`|`*`hinst`*`|`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of a sequence of points that have a fall transition and that a path must traverse. Points to traverse can be ports, hierarc
	You can repeat the -`through_fall` option to require that a path first must traverse one of the objects in the first set, then pass through one
`-through_rise {`*`inst`*`|`*`hinst`*`|`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of a sequence of points that have a rise transition and that a path must traverse. Points to traverse can be ports, hierar
	You can repeat the -`through_rise` option to require that a path first must traverse one of the objects in the first set, then pass through one
`-timing_bin` *`bin`*	Reports the timing for all paths in the specified timing bin.
	The timing bin must have been previously created with the create_timing_bin command.
	This option can only be used in conjunction with the -max_paths and -gui_phys options.
`-timing_path` *`path`*	`Reports the timing for the specified timing path.`
	`This timing path must be part of a timing bin.`
`-to {`*`inst`*`|`*`hinst`*`|`*`external_delay`*`|`*`clock`*`|`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of end points for the paths. The end points can be output ports of your design, input pins of flip-flops, clock objects, or
`-to_fall {`*`inst`*`|`*`hinst`*`|`*`external_delay`*`|`*`clock`*`|`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of end points with a fall transition at the end point. The end points can be output ports of your design, input pins of flip-
`-to_rise {`*`inst`*`|`*`hinst`*`|`*`external_delay`*`|`*`clock`*`|`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies a Tcl list of end points with a rise transition at the end point. The end points can be output ports of your design, input pins of flip
`-unconstrained`	Reports only the unconstrained paths (paths with no slack). Each signal arriving at the path end node which does not have a matching re
`-user_derate`	Controls the display of an additional column for the user derating values in the timing report. A derate of 1.0 is not reported as 1.0, but as "-".
`-user_mean_derate`	Controls the display of an additional column for the user mean derating values in the timing report.

`-user_sigma_derate`	Controls the display of an additional column for the user sigma derating values in the timing report.
`-views` *analysis_view*	
	Specifies the *analysis_view* that the report should be restricted to.

Examples

- The following command reports only the worst path being launched by clock `clk1`.

```
genus:root:> report_timing -paths [specify_paths -from clk1]
```

- The following example reports the slack at the four most-constrained endpoints.

```
genus:root: alu> report_timing -endpoints -max_paths 4
============================================================
  ...
============================================================

 Slack      Endpoint    Cost Group
---------------------------------------
  +30ps aluout_reg[7]/D I2C_cg
 +289ps aluout_reg[6]/D I2C_cg
 +292ps aluout_reg[5]/D I2C_cg
 +536ps aluout_reg[4]/D I2C_cg
```

- The following example reports the path from input port `accum[1]` that has the least slack.

```
genus:root:> report_timing -from [get_db ports *accum[1]]
============================================================
  ...
============================================================

    Pin               Type      Fanout  Load  Slew   Delay   Arrival
                                        (fF)  (ps)    (ps)    (ps)
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(clock clock)         capture                                3500 R
                      adjustments                    -500    3000
------------------------------------------------------------------------
Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :      30ps
Start-point  : accum[1]
End-point    : aluout_reg[7]/D
```

- The following example generates a report for the worst path to each of the four most-constrained endpoints. The extraction of the report shows four different endpoints.

```
genus:root:design:alu n> report_timing -max_paths 4
============================================================
  ...
============================================================

path   1:

      Pin            Type       Fanout Load Slew Delay Arrival
                                       (fF) (ps)  (ps)    (ps)
------------------------------------------------------------------
...
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(clock clock)      capture                                  3500 R
                   adjustments                    -500      3000
```

```
------------------------------------------------------------
Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :      30ps
Start-point  : accum[1]
End-point    : aluout_reg[7]/D

path   2:

      Pin             Type       Fanout Load Slew Delay Arrival
                                        (fF) (ps)  (ps)   (ps)
------------------------------------------------------------
...
------------------------------------------------------------
Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :     289ps
Start-point  : accum[1]
End-point    : aluout_reg[6]/D


path   3:

      Pin             Type       Fanout Load Slew Delay Arrival
                                        (fF) (ps)  (ps)   (ps)
------------------------------------------------------------
...
------------------------------------------------------------
Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :     292ps
Start-point  : accum[1]
End-point    : aluout_reg[5]/D

path   4:

------------------------------------------------------------
...
------------------------------------------------------------
Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :     536ps
Start-point  : accum[1]
End-point    : aluout_reg[4]/D
```

- The following example also generates a report for the four worst paths in the current design, but compared to the previous example, all worst paths now have the same endpoint.

```
genus:root:design:alu n> report_timing -max_paths 4 -worst 4
============================================================
  ...
============================================================
 path   1:

      Pin             Type       Fanout Load Slew Delay Arrival
                                        (fF) (ps)  (ps)   (ps)
------------------------------------------------------------
...
------------------------------------------------------------

Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
```

```
Timing slack :      30ps
Start-point  : accum[1]
End-point    : aluout_reg[7]/D

path   2:

      Pin            Type       Fanout Load Slew Delay Arrival
                                       (fF) (ps)  (ps)    (ps)
-------------------------------------------------------------------
...
-------------------------------------------------------------------

Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :      30ps
Start-point  : data[1]
End-point    : aluout_reg[7]/D


path   3:

      Pin            Type       Fanout Load Slew Delay Arrival
                                       (fF) (ps)  (ps)    (ps)
-------------------------------------------------------------------
...
-------------------------------------------------------------------

Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :      30ps
Start-point  : accum[1]
End-point    : aluout_reg[7]/D

path   4:

      Pin            Type       Fanout Load Slew Delay Arrival
                                       (fF) (ps)  (ps)    (ps)
-------------------------------------------------------------------
...
-------------------------------------------------------------------
Exception    : 'path_adjusts/adj_1' path adjust    -500ps
Cost Group   : 'I2C_cg' (path_group 'I2C_pg')
Timing slack :      30ps
Start-point  : data[1]
End-point    : aluout_reg[7]/D
```

- The following command reports the path with the least slack that uses a `multi_cycle` exception named `mc_2`.

  ```
  genus:root:> report_timing -exceptions [get_db exceptions *mc_2]
  ```

- The following example reports only a condensed version of the timing report.

  ```
  genus:root:> report_timing -summary

  ============================================================
    Generated by:          Genus Synthesis Solution version
   .... ============================================================ Timing slack : 30ps Start-point : accum[1] End-point :
  aluout_reg[7]/D
  ```

- The following example illustrate how to use the `-gtd` option.

  ```
  genus:root:> report_timing -from ..... -to ..... -gtd > viol.mtarpt
  ```

  Open `viol.mtarpt` in the global timing debug viewer in Innovus.

- The following example shows the timing report when the `-user_derate` option is specified. In this example, the `cell_delay_multiplier` of cell `ffopd` was

modified to `3.0`.

```
genus:root:> report_timing -user_derate

Warning : Possible timing problems have been detected in this design. [TIM-11]
: The design is 'top'.
============================================================
  ....
============================================================
   Pin          Type         Fanout Load Slew  User  Delay Arrival
                                    (fF) (ps) Derate  (ps)   (ps)
-------------------------------------------------------------------------
(clock clk)   launch                                          0 R
inst1/CK                                  0                   0 R
inst1/Q       fflopd     (m)      1 20.4   16  3.000  +443     443 F
inst2/A                                                 +0     443
inst2/Y       inv1               1 25.4   17  1.000   +85     528 R
inst6/A                                                 +0     528
inst6/Y       buf1               1 20.4   16  1.000  +134     662 R
inst3/D       fflopd_ckn                    1.000    +0     662
inst3/CKN     setup                      0         +100     762 F
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(clock clk)   capture                                        50 F
-------------------------------------------------------------------------
Cost Group   : 'clk' (path_group 'clk')
Timing slack :    -712ps (TIMING VIOLATION)
Start-point  : inst1/CK
End-point    : inst3/D

(m) : Attribute cell_delay_multiplier is modified for this library cell.
```

- The following example reports the physical information (if the design has been placed):

```
genus:root:> report_timing -physical

============================================================
  ....
============================================================
      Pin               Type     Fanout Load Slew Delay Arrival    Location
                                        (fF) (ps)  (ps)    (ps)        (x,      y)
-------------------------------------------------------------------------------
(clock clock1)         launch                                0 R
wr_addr_reg[0]/CK                                 0            0 R
wr_addr_reg[0]/Q  (@)  SDFFRHQX1      2  6.0  118  +311     311 R  (107640, 51660)
g154/A                                      118    +0     311
g154/Y            (@)  CLKMX2X2       3 10.9  105  +206     517 R  (102580, 51660)
g146/B                                      105    +0     517
g146/CO           (@)  ADDHXL         1  4.6  167  +189     706 R  (101660, 59040)
g145/A                                      167    +0     706
...
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(clock clock1)         capture                          10000 R
-------------------------------------------------------------------------------
Timing slack :    7997ps
Start-point  : wr_addr_reg[0]/CK
End-point    : wr_addr_reg[7]/D

(@) : Annotated capacitance.
```

- The following report shows the timing report with the **-encounter** option.

```
genus:root:> report_timing -encounter

============================================================
```

```
  ....
============================================================
Path 1: MET Setup Check with Pin ff2/CK
Endpoint:   ff2/D (v) checked with leading edge of 'CK'
Beginpoint: ff1/Q (^) triggered by leading edge of 'CK'
Other End Arrival Time          12000
- Setup                           304
- Uncertainty                    1000
= Required Time                 10696
- Arrival Time                   2373
= Slack Time                     8323
    Clock Rise Edge                          0
    + Clock Network Latency (Ideal)       2000
    = Beginpoint Arrival Time             2000
------------------------------------------------------------------------
         Pin      Edge   Cell    Fanout Load Slew Delay  Arrival Required
                                        (fF)  (ps)  (ps) Time(ps) Time(ps)
------------------------------------------------------------------------
       ff1/CK     ^                                0          2000    10323
       ff1/Q      ^     DFFHQX1       1  7.1  160  +310      2310    10633
       b2/A       ^                                +0      2310    10633
       b2/Y       v     INVX1         1  5.7   88   +63      2373    10696
       ff2/D  <<< v     DFFHQX1                    +0      2373    10696
------------------------------------------------------------------------
Cost Group   : 'CK' (path_group 'CK')
```

- The following example reports a specific path in the sub-bin `1ns_slack_2ns` in parent bin `worst1000_c2`.

  ```
  enus:root:> report_timing -timing_path worst1000_c2c/1ns_slack_2ns/p_27
  ```

- The following example illustrates how to report and analyze a parent bin using linked physical and timing viewers.

  ```
  genus:root:> report_timing -timing_bin worst1000_c2c -max_paths 100 -gui_phys
  ```

  Parent bin `worst1000_c2c` will be raised in a timing viewer. A physical viewer will also be raised and will be linked to the timing_gui.

- The following command reports the path contained in sub bin `1ns_slack_2ns` of parent bin `worst1000_2c`.

  ```
  genus:root:> report_timing -timing_path worst1000_c2c/1ns_slack_2ns/p_27
  ```

- The following command shows how the same timing view can be linked to a specific physical gui.

  ```
  genus:root:> report_timing -timing_path worst1000_c2c/1ns_slack_2ns/p_27 -gui -id 2
  ```

Related Information

Affected by these commands:	set_path_specification
	syn_generic
	syn_map
	syn_opt
Affected by this attribute:	cell_delay_multiplier

report_timing_derate

```
report_timing_derate
[-delay_corner delay_corner]
{lib_cell | inst | hinst  | design}...
```

Description

Reports delay scaling or derating factors for early and late clock and data paths in the current design. If you specify this command without any parameters, reports are generated for global derating factors for each instance, net, or cell for which derating is asserted.

Options and Arguments

`-delay_corner` *delay_corner*	
	Reports the derating factors from the specified delay corner only.
	Default: Reports derating factors for all delay calculation corners.
design	Specifies the design from which the derating factors must be reported.
inst|hinst	Specifies the instance(s) from which the derating factors must be reported.
lib_cell	Specifies the library cells from which the derating factors must be reported.

Example

- The following example shows how to report the timing derate on whole design:

  ```
  genus:root:> report_timing_derate
  ```

- The following example shows how to report the timing derate on a library cell:

  ```
  genus:root:> report_timing_derate [vfind -lib_cell TCHDFFX5]
  ```

Related Information

Related commands:	set_timing_derate
	report_timing

report_ungroup_modules

```
report_ungroup_modules
[-size integer] [object+] [-type string] [ > file]
```

Description

Reports the maximum number of ungrouped names used in the design.

Options and Arguments

size	Specifies the maximum number of ungrouped names used in the design.
object	Specifies the design for which you want to generate a report.
`-type` *string*	Specifies the type of ungroup {all, auto, user}.
file	Redirects the report to the specified file.

Example

- The following example reports ungrouped modules when design and size is not provided. The below table lists all the ungrouped modules for top module.

```
@genus@root: number>  report_ungroup_modules
      ============================================================
        Module:                 top_alu
        Technology library:     tutorial 1.0
        Operating conditions:   typical_case (balanced_tree)
        Wireload mode:          enclosed
        Area mode:              timing library
      ============================================================

               Orignal                  Replaced
      ------------------------------------------------------
      alu/add_23_29                       add_23_29_g219
                                          add_23_29_g218
                                          add_23_29_g217
                                          add_23_29_g216
                                          add_23_29_g215
                                          add_23_29_g214
```

- The following example reports ungrouped modules when design is not provided and -size is provided as 2, that's why we see only two ungroup modules for each top module.:

```
@root: number>  report_ungroup_modules -size 2
      ============================================================
        Module:                 top_alu
        Technology library:     tutorial 1.0
        Operating conditions:   typical_case (balanced_tree)
        Wireload mode:          enclosed
        Area mode:              timing library
      ============================================================

               Orignal                  Replaced

      ------------------------------------------------------
      alu/add_23_29                       add_23_29_g219
                                          add_23_29_g218
      alu/add_24_29                      add_24_29_g1045
                                         add_24_29_g1044
```

report_units

```
report_units
[object] [ > file]
```

Description

Reports the units used in the libraries.

You must have loaded the libraries before you can use this command.

Options and Arguments

object	Specifies the library for which you want to generate a report.
file	Redirects the report to the specified file.

Related Information

Design For Manufacturing Flow

Affects these commands:	all report commands
Affected by this attribute:	lp_power_unit

report_utilization

```
report_utilization [>file]
```

Description

Reports the floorplan utilization for the design. The result is based on the `cell_area` calculated from LEF information

Options and Arguments

file	Redirects the report to the specified file.

report_yield

```
report_yield
[-depth integer] [-min_count integer]
[-min_area float] [-summary] [object...] [> file]
```

Description

Reports the yield cost and yield percentage for each instance. This command is used in the design for manufacturing (DFM) flow.

Options and Arguments

`-depth` *integer*	Specifies the number of levels of recursion.
file	Redirects the report to the specified file.
`-min_count` *integer*	
	Specifies the minimum instance count per block.
`-min_area` *float*	Specifies the minimum area per block
object	Design to report on.
`-summary`	Specifies to report the area summary.

Related Information

Affected by this command:	read_dfm
Related command:	report_gates -yield
Affected by this attribute:	optimize_yield

save_snapshot

```
save_snapshot
[-to_file file]
```

Description

Writes out the statistics information (QoR metrics) recorded at various stages to the specified database file.

When you specify an existing file, the tool will overwrite the existing data.

Options and Arguments

`-to_file file`	Specifies the file to which to write the statistics information.

Related Information

Related command:	read_metric

set_activity

```
set_activity [-pin] pin-path \
-activity_type user|default (default: user)
-duty duty (duty cycle is a fraction from 0 to 1)
-freq freq (activity_type user :frequency e.g. 150.0e+6
activity_type default: absolute freq or pct of specified -clock option)
-pin_types {primary_input|seq_out|flop_out|latch_out|memory_out|icgc_out|icgc_enable|bbox_out|comb_out|all}+ (default = all pins)
[-clock related|clock_pin_or_net] (default = "related" clock)
[-reset] (Resets all the lp_asserted* activities) [-silent]
```

Description

Sets the default activity on the pins.

Options and Arguments

-pin	Specifies the input pin on which the activity needs to be set.
-activity_type	Specifies the activity type is 'user' or 'default'. Activity type user applies the activity on pin. Activity type default applies the activity on different `pin_types`. The activity type default is applied on the pins which don't have previously asserted activities Default: `user`
-duty	Specifies the duty cycle. This is a fraction between 0 and 1.
-freq	Specifies the frequency value for the pin. `activity_type user`: frequency. For example, 150.0e+6. `activity_type default`: absolute freq or pct of specified `-clock` option.

`-pin_types`	Specifies that the given activity will be set on all the pins matching the type. The value can be one or more of the following.
	`-primary_input`
	`-seq_out`
	`-flop_out`
	`-latch_out`
	`-memory_out`
	`-icgc_out`
	`-icgc_enable`
	`-bbox_out`
	`-comb_out`
	`-all (Default)`
	This option cannot be used concurrently with `-pin`.

`-clock`	Specifies that the frequency for the pin will be a percentage of the given clock's frequency.
	Possible options are:
	`-related (Default)`
	`-clock_pin_or_net`

`-reset`	Resets the asserted activities.
`-silent`	Stops printing any info logs.

Examples

- `set_activity /cpu_10bit/clk -duty 0.40 -freq 200e6`
- `set_activity -activity_type default -pin_types primary_input -duty 0.3 -freq 2e7`
- `set_activity -activity_type default -pin_types flop_out -duty 0.5 -freq 0.3 -clock related`

`Return value: 0 = success, 1 = failure`

set_socv_reporting_nsigma_multiplier

```
set_socv_reporting_nsigma_multiplier
[-hold float]
[-setup float]
[-transition float]
[-views string]
```

Description

Applies the sigma multiplier factor to the specified list of analysis views.

Options and Arguments

`-hold` *float*	Specifies the sigma multiplier for hold mode.
`-setup` *float*	Specifies the sigma multiplier for setup mode.
`-transition` *float*	Specifies the sigma multiplier for transition.
`-views` *string*	Applies the sigma multiplier factor to the specified analysis view

time_info

```
time_info [-report | -current]
[-table tableID] [-stamp]
[-save tstampfile] [-load tstampfile]
[stage][> file]
```

Description

Reports and tracks runtime and memory performance.

> This command provides a shortcut syntax for -report and -stamp command options. For example, time_info STAGE_6 is the shortcut for time_info -stamp STAGE_6 -report.

Options and Arguments

-current	Specifies to report upto the current stage.
file	Specifies the name of the file to which to write the report.
-load *tstampfile*	Specifies the name of the ".tstamp" file from where to load runtime metrics.
-report	Specifies to report all stages upto this point.
-save *tstampfile*	Specifies the name of the ".tstamp" file to save the runtime metrics.
stage	Specifies the name of the stage.
-stamp	Specifies to create a timestamp.
-table tableID	Specifies the metrics table ID to support multiple tables.
	Default: 'default'

Example

```
genus:root:> time_info "write data"

stamp 'write data' being created for table 'default'


Total Time (Wall) |  Stage Time (Wall)| % (Wall)     |  Date - Time | Memory | Stage

------------------+-------------------+----------+--------------+--------+----------------

00:00:04(00:00:00)| 00:00:00(00:00:00)| 0.0(0.0)|12:00:11(Dec11)|106.6 MB| init

------------------+-------------------+----------+--------------+---------+---------------

00:00:06(00:00:03)| 00:00:02(00:00:03)| 2.4(0.1)|12:00:14(Dec11)|187.0 MB|initial settings

------------------+-------------------+----------+--------------+--------+----------------

00:00:09(00:00:24)| 00:00:03(00:00:21)| 3.6(0.5)|12:00:35(Dec11)|313.1 MB| load lib/OA/QRCtech

------------------+-------------------+----------+--------------+--------+---------------
```

```
00:00:10(00:00:25)| 00:00:01(00:00:01)| 1.2(0.0)|12:00:36(Dec11)|313.1 MB| load HDL, elaborate

------------------+------------------+---------+---------------+--------+----------------

00:00:10(00:00:26)| 00:00:00(00:00:01)| 0.0(0.0)|12:00:37(Dec11)|313.1 MB| settings

------------------+------------------+---------+---------------+--------+----------------

00:00:45(00:01:01)| 00:00:35(00:00:35)|41.7(0.9)|12:01:12(Dec11)|399.9 MB| syn_gen

------------------+------------------+---------+---------------+--------+----------------

00:00:56(00:01:12)| 00:00:11(00:00:11)|13.3(0.3)|12:01:23(Dec11)|344.6 MB| syn_map

------------------+------------------+---------+---------------+--------+----------------

00:00:56(00:01:12)| 00:00:00(00:00:00)| 0.0(0.0)|12:01:23(Dec11)|344.6 MB| connect_scan_chain

------------------+------------------+---------+---------------+--------+----------------

00:01:25(00:02:16)| 00:00:28(00:01:04)|34.3(1.6)|12:02:27(Dec11)|583.9 MB| syn_opt

------------------+------------------+---------+---------------+--------+----------------

00:01:26(00:02:46)| 00:00:01(00:00:30)| 1.2(0.8)|12:02:57(Dec11)|588.5 MB| write data

------------------+------------------+---------+---------------+--------+----------------

genus:root:> time_info -current

=======================================================

CPU Runtime:    00:01:29

Wall Runtime:   01:06:37

Memory Usage:   603.6 MBytes

Current Time:   13:06:48 (Dec11)

=======================================================
```

report_clock_gate_decloning

```
report_clock_gate_decloning
```

```
[-design object]
[-clock_gate_type enum {all synthesis user}]
[-get_dict]
[-get_dict_key_types]
```

Description

Reports data relevant to clock gate decloning opportunities.

Options and Arguments

`-design object`
Generates a report for the specified design.
`-clock_gate_type enum {all synthesis user}`
Limits the report to the specified categories of clock gates (enum all synthesis user)
`-get_dict`
Returns report output as Tcl dict.
`-get_dict_key_types`
Returns a dict of key constraint types in the Tcl output.

Related Information

Clock Gating Cell Specification in the *Genus Library Guide*.

Related Commands
declone_clock_gate

report_instance_library

```
report_instance_library
[-inst inst+]
[-view analysis_view]
[> file]
```

Description

Generates a report of the instances in the library.

Options and Arguments

`-inst inst+`	Prints the libraries used for scaling for a specified instance.
`-view analysis_view`	Specifies the analysis view for reporting

4

Chipwaredev

- check_cwd_object
- create_component
- create_component_binding
- create_component_implementation
- create_component_library
- create_component_operator
- create_component_package
- create_component_parameter
- create_component_pin
- create_cwd_check

The following command is common with the Analysis and Reporting commands:

- report_cwd_check

check_cwd_object

```
check_cwd_object [-effort string]
[-skip {hdl_pin | hdl_parameter | hdl_bind
  | hdl_implementation | hdl_architecture | hdl_operator
  | hdl_package| hdl_component | hdl_lib}... ]
[-summary] [-verbose | -quiet ]
[[ hdl_pin | hdl_parameter | hdl_bind | hdl_implementation
  | hdl_architecture | hdl_operator | hdl_package
  | hdl_component | hdl_lib]...]
[> file]
```

Description

Exercises checking rules and summarizes the outcome in various degrees of verboseness. The `check_cwd_object` command can run on one or more of any of the following `hdl_objects`:

- `hdl_architecture`
- `hdl_bind`
- `hdl_component`
- `hdl_implementation`
- `hdl_lib`
- `hdl_operator`
- `hdl_package`
- `hdl_parameter`
- `hdl_pin`

There are various "check points" associated with these objects. A check point is a specific point in the flow in which the CWD objects are processed. There are various checking rules associated with CWD check points.

By default, `check_cwd_object` checks all `hdl_objects` underneath the specified set of `hdl_objects`. That is, it traverses the directory tree hierarchically and exercises all checks of all `hdl_objects` it traverses.

Options and Arguments

`-effort` *string*	Specifies the effort level. There are two effort levels: `low` and `medium`. The default effort level is `low`.
	`Low` — CWD linter runs checking rules that are registered as `low` effort. That is, it runs those checking rules that do not require reading any HDL code (of synthesis models).
	`Medium` — CWD linter runs checking rules that are registered as `low` and `medium` effort. That is, it runs those checking rules that may require parsing HDL code (of synthesis models) but do not require elaborating it.
	With each `medium` effort level check, the CWD linter automatically loads the HDL code before performing the check. For example, a check at this effort level may look at the `hdl_architecture` of an `hdl_implementation` and examine ordering of pins and parameters.
file	Specifies the filename to store the output of the command.
hdl_pin|*hdl_parameter*|*hdl_bind*|*hdl_implementation*| *hdl_architecture*|*hdl_operator*|*hdl_package*|*hdl_component*| *hdl_lib*	
	Specifies the `hdl_objects` to check.
`-quiet`	Only reports error and warning messages, if any. This is the recommended verbosity level when CWD linting is part of a routine process without any error expectations.
`-skip` {*hdl_pin*|*hdl_parameter*|*hdl_bind*|*hdl_implementation*| *hdl_architecture*|*hdl_operator*|*hdl_package*|*hdl_component*| *hdl_lib*}	
	Specify one or more `hdl_objects` to skip.
`-summary`	First reports error or warning messages, if any, and then produces a summary table of the pass/fail count of each checking rules exercised. This level of detail is the default verbosity level.
`-verbose`	Produces a detailed report. In addition to the information produced by the `-summary` option, it also reports the pass/fail status of each check process exercised on each `hdl_object`.

Related Information

Checking Rules in *Genus ChipWare Developer*.

Chipware Developer Linter in *Genus ChipWare Developer*.

create_component

```
create_component -name component_name
[hdl_lib | components]
```

Description

Creates a ChipWare component.

Options and Arguments

`-name` *component_name*
Specifies the name of the component that will be created.
hdl_lib | *components*
Specifies the path of the library that holds this component.

Example

The following example creates a ChipWare component 'counter' in library 'mylib':

```
create_component_library -name mylib

create_component -name counter mylib
```

Related Information

See the following sections in *Genus ChipWare Developer*

- CWD Component
- ChipWare Registration
- Related commands:

create_component_binding
create_component_implementation
create_component_library
create_component_operator
create_component_package
create_component_parameter
create_component_pin

create_component_binding

```
create_component_binding binding_name
-operator operator_name
[hdl_component | bindings]
```

Description

Creates a binding between a synthetic operator and a ChipWare component. A synthetic component is also known as a ChipWare component.

Run-time can be reduced if you change directory (`vcd`) to the to the *component name* directory before issuing the command instead of specifying the entire component path name.

Options and Arguments

binding_name	Specifies the name of the binding that will be created.
hdl_component | *bindings*	
	Specifies the path name of the component that holds this binding.
-operator	Specifies the synthetic operator that will be bound by this binding.

Example

The following example creates a binding named b1 between the MULT_TC_OP operator and the my_CW_mult component:

```
create_component_binding b1 -operator MULT_TC_OP \/hdl_libraries/my_CW/components/my_CW_mult
```

Related Information

- CWD Component in *Genus ChipWare Developer*

Related commands:

create_component
create_component_implementation
create_component_library
create_component_operator
create_component_package
create_component_parameter
create_component_pin

create_component_implementation

```
create_component_implementation -name implementation_name
[-language string] [-config string]
[hdl_component | implementations]
```

Description

Creates an implementation for a ChipWare component. All components can have one or more implementations (or architectures) associated with them. This applies to both user and tool defended components. All implementations created with this command have a default priority of 1. The language option needs to be specified.

Options and Arguments

`-config` *string*	Specifies the name of the configuration to be used.
	Use this option when the VHDL entity was specified with a configuration to create the component.
hdl_component | *implementations*	
	Specifies the path name of the component that owns this implementation.
`-language` *string*	Specifies the language of the HDL files.
	Supported values are {`v1995` | `sv` | `v2001` | `vhdl87` | `vhdl93`}
`-name` *implementation_name*	
	Specifies the name of the implementation that will be created.

Example

The following example creates the `my_booth` implementation with `vhdl1993` on MY_MULT component.

```
create_component_implementation -name my_booth -language vhdl93 /hdl_libraries/MY_CW/components/MY_MULT
```

Related Information

See the following sections in *Genus ChipWare Developer*

- CWD Component
- CWD Implementation
- Chipware Installation and Registration
- Related Commands

create_component
create_component_binding
create_component_library
create_component_operator
create_component_package
create_component_parameter
create_component_pin

- Affects this attribute:

priority

create_component_library

`create_component_library -name` *library_name*

Description

Creates a user defined HDL library. An HDL library can be a library of ChipWare components, a library of synthetic operators, or a VHDL library.

Options and Arguments

`-name` *library_name*	
	Specifies the name of the library that will be created.

Related Information

See the following sections in *Genus ChipWare Developer*

- CWD Component
- HDL Library
- ChipWare Installation and Registration
- Related Commands

create_component
create_component_binding
create_component_implementation
create_component_operator
create_component_package
create_component_parameter
create_component_pin

create_component_operator

```
create_component_operator operator_name
[-type {signed|unsigned}]
```

Description

Creates a synthetic operator. The operator type is unsigned. Genus has a set of tool-defined synthetic operators, but also provides the capability to add user-defined synthetic operators. All synthetic operators, whether they are tool or user-defined, are located under the operators directory(`/hdl_libraries/synthetic/operators/`).

Options and Arguments

operator_name	Specifies the name of the synthetic operator that will be created.
-type	Specifies the type of the operator.
	signed - Specifies the created operator to be a signed operator.
	unsigned - Specifies the created operator to be an unsigned operator.
	The operator type must be specified.

Example

The following example creates the MY_OP operator with unsigned type:

```
create_component_operator MY_OP -unsigned
```

Related Information

- Synthetic Operator in *Genus ChipWare Developer*
- Related Commands

create_component
create_component_binding
create_component_implementation
create_component_library
create_component_package
create_component_parameter
create_component_pin

create_component_package

```
create_component_package pkg_name
-path path_to_pkg
[hdl_lib | packages]
```

Description

Registers a VHDL package in the ChipWare Developer framework. Packages that are not registered are deleted after elaboration. However, registered packages are never deleted and their information can be further considered during synthesis as opposed to just during elaboration.

Registered packages are in the same location within Genus as non-registered packages: /hdl_libraries/library_name/packages/

Options and Arguments

hdl_lib | *packages*	
	Specifies the path name of the library that holds this package.
`-path` *path_to_pkg*	Specifies the UNIX path name of the package to register.
pkg_name	Specifies the name of the package that will be created.

Example

The following example creates the sample_std package for the sample_lib library:

```
create_component_package sample_std -path /sample/vhdl/sample_std.vhd \/hdl_libraries/sample_lib/packages
```

Related Information

Related commands:

create_component
create_component_binding
create_component_implementation
create_component_library
create_component_operator
create_component_parameter
create_component_pin

create_component_parameter

```
create_component_parameter -name parameter_name
[-hdl_hidden]
[hdl_component | parameters]
```

Description

Creates a parameter for a ChipWare component. The created parameter will be a `hdl_parameter` object type and located under *../component_name*`/parameters`. The default `hdl_parameter` attribute value for parameters created with this command will be `true`. However, if the `-hdl_hidden` option is specified, the default value becomes `false`.

Options and Arguments

hdl_component | *parameters*	
	Specifies the path name of the component that holds this parameter.
-hdl_hidden	Specifies that the parameter cannot be accessed from the HDL. The value of the `hdl_parameter` attribute for this parameter becomes `false` with this option.
-name *parameter_name*	
	Specifies the name of the parameter that will be created.

Related Information

See the following sections in *Genus ChipWare Developer*

- CWD Component
- ChipWare Installation and Registration
- Affects this attribute: hdl_parameter
- Related commands:

create_component
create_component_binding
create_component_implementation
create_component_library
create_component_operator
create_component_package
create_component_pin

create_component_pin

```
create_component_pin -name pin_name
{-input | -output | -inout}
[hdl_pins | hdl_operator | hdl_component]
```

Description

Creates a pin for either a ChipWare component or a synthetic operator. You must specify a pin direction.

Options and Arguments

hdl_pins | *hdl_operator* | *hdl_component*	
	Specifies the path name of the component or synthetic operator that holds the created pin.
`-inout`	Specifies that the created pin will be an bidirectional pin.
`-input`	Specifies that the created pin will be an input pin.
`-name` *pin_name*	Specifies the name of the pin that will be created.
`-output`	Specifies that the created pin will be an output pin.

Example

The following example creates an input pin A and an output pin Z:

```
create_component_pin -name A -input

create_component_pin -name Z -output
```

The bit-widths and other attributes for the pins should be set separately.

Related Information

See the following sections in *Genus ChipWare Developer*

- CWD Component
- ChipWare Installation and Registration
- Synthetic Operator
- Related commands:

create_component
create_component_binding
create_component_implementation
create_component_library
create_component_operator
create_component_package
create_component_parameter

create_cwd_check

```
create_cwd_check
-check_name string
-severity integer
-description string
-check_list string
[-effort {low|medium}] [-force] [> file]
```

Description

Registers user-defined checking rules for the CWD linter.

Registered rules can be reported using the `report_cwd_check` command.

Options and Arguments

`-check_list` *string*	Specifies, as a Tcl list of Tcl lists, checkpoint and Tcl proc pairs. The specification therefore would be in the form:
	`-check_list {{point_1 proc_1} {point_2 proc_2}}`
	Every sub-list has two elements. The first element is the name of a check point, defined by Genus. The second element is the name of a Tcl proc, defined by the user.
	Each sub-list specifies a check proc that is to be called at a certain check point. This check proc will be executed every time the flow reaches this check point.
	This Tcl list has one or more sub-lists. One checking rule can be associated with one or more check points.
	The check procs may or may not be allowed to parse or elaborate the HDL code of the synthesis model, depending on the effort level of this checking rule. The check procs may print out error, warning, or info messages. Each check proc should return a string whose value is either `PASS` or `FAIL`.
`-check_name` *string*	Specifies an unique name for a new checking rule.
`-description` *string*	
	Specifies a character string that concisely describes what this checking rule examines.
`-effort` {`low` | `medium`}	
	Specifies the effort level.
	Default: `low`
	`low` — The check is not allowed to parse the HDL code of the synthesis models.
	It can check correctness, consistency, or completeness of the CWD registration, including availability of UNIX files referred to by the `location` and `sim_model` attributes.
	`medium` — The check can read, load, and parse the HDL code of the synthesis models, but it cannot elaborate or synthesize the HDL code. When exercising such a checking rule, the HDL code is automatically loaded before checking is performed.
file	Specifies the filename to store the output of the command.
`-force`	Removes the existing checkname and adds the current specification. Alternatively, the same checkname will now correspond to the new definition.
`-severity` *integer*	Specifies the severity level of the message produced by this checking rule. The possible values (`0` through `10`) are the same as those for the `information_level` attribute.
	`-severity 0`: It is an error message to violate this rule
	`-severity 1`: It is a warning message to violate this rule
	`-severity 2`: It is a level 2 info message to violate this rule
	Severity levels 3 through 10 are all info messages at their respective levels.

Example

A simple usage example:

```
create_cwd_check -check_name pin_bit_width_1 -severity 0 -effort low \          -description "Check whether the bit_attr attribute
is set to null"\          -checklist {{AT_COMP_PIN ::cwd::pin_bit_width::check_proc}}


report_cwd_check -check_name pin_bit_width_1-------------------------------------------------------------------------------------
-Check_name         Checkpoint          Check_proc         Effort Severity Description-----------------------------------------
---------------------------------------pin_bit_width_1 AT_COMP_PIN ::cwd::pin_bit_width::check_proc  low   0   Check whether
the bit_attr                                                                    attribute is set
to null-------------------------------------------------------------------------------------
```

Related Information

Checking Rules in *Genus ChipWare Developer*

5

Constraints

- check_constraints
- create_derived_design
- define_cost_group
- path_adjust
- path_group
- phys_enable_ocv
- phys_reset_ocv
- report_constraint
- reset_path_adjust
- reset_path_exception
- set_path_adjust
- set_path_adjust_group
- set_path_specification
- set_socv_rc_variation_factor

check_constraints

```
check_constraints [-rtl] [-netlist string]
[-init_sequence_file string] [-sdc string]
[-file string] [> file]
```

Description

Validates the design constraints specified in the SDC files against the RTL or netlist by invoking Conformal Constraint Designer (CCD) in the background. The command can be used any time after elaborating the design. If you do not specify either the `-rtl`, `-netlist`, or `-sdc` options, Genus will validate the internally generated constraints against the design at its current state.

Use this command in the False Path and Multi-cycle Path validation flows.

> The CCD executable must be visible from the system environment variables VERPLEX_HOME_CCD/PATH, or it can be specified with the root attribute `ccd_executable`.

Options and Arguments

file	Specifies the name of the file to write the report.
-init_sequence_file *string*	
	Specifies the UNIX path to the initialization sequence file for MCP validation.
-file *string*	Creates a separate CCD logfile. You must specify the UNIX path to the file.
-netlist *string*	Specifies the UNIX path to the netlist. This option also indicates that the validation should happen against the netlist (as opposed to the RTL).
-rtl	Indicates that the validation should happen against the RTL (as opposed to the netlist).
-sdc *string*	Specifies the UNIX path to the SDC files.

create_derived_design

```
create_derived_design [-name string] {inst|hinst}
[-sdc_only]
```

Description

Creates a new design from the specified instance and creates timing constraints for all modes for this design based on the timing constraints that the instance had in the original design.

This command does not perform time-budgeting. The slack at each pin in the new design matches the slack at the corresponding pin in the original design.

By default, the `create_derived_design` command uses Genus's more powerful constraint language that produces a more accurate timing view, but is not understood by other tools. Use the `-sdc_only` option to specify that Genus only apply constraints to the new design that can be expressed in SDC.

If you use the `create_derived_design` command to generate constraints for a module then try to write them out. Often the constraints cannot be expressed in SDC and you will get the following error message:

```
Error   : The design contains constraints which have no SDC equivalent.[SDC-19]

        : The design is design:Madd_addinc.

        : If the design constraints were created using the 'create_derived_design' command, use the
'-sdc_only' option so that only constraints that can be expressed in SDC are generated. By default
the 'create_derived_design'command uses the more powerful Genus constaints, which cannot always be
converted to SDC.
```

Options and Arguments

inst| *hinst*	Specifies the name of the instance whose environment (constraints) you want to derive.
`-name` *string*	Specifies the name of the target design.
`-sdc_only`	Specifies that the tool only apply constraints to the new design that can be expressed in SDC.
	Using this option makes the new constraints less accurate, but makes it possible to use the constraints in flows between different tools that support SDC.
	By default, the `createse_derived_design` command uses Genus's more powerful constraint language that produces a more accurate timing view, but is not understood by other tools.

Related Information

Synthesizing Submodules section in Synthesizing your Design in the Genus User Guide.	
Affected by this command:	path_adjust
Affects these commands:	report_timing
	syn_generic
	syn_map
	syn_opt
Sets this attribute:	precluded_path_adjusts

define_cost_group

```
define_cost_group -name string
[-weight integer] [-design design]
```

Description

Defines a cost group. The command returns the directory path to the object that it creates.

Options and Arguments

`-design` *design*	Specifies the name of the design for which you want to define the cost group.
`-name` *string*	Specifies the name of the cost group.
	Default: `grp_x`
`-weight` *integer*	Specifies the weight of the cost group.
	The weight factor is only taken into account during incremental optimization. When two paths have identical cost factors, the tool will pick the one with the highest weight.
	Default: `1`

Related Information

Affects these commands:

path_group
report_timing

Sets this attribute:

weight

path_adjust

```
path_adjust -delay integer
[{ -from {inst|hinst|external_delay|clock|port|pin|hpin}...
  |-through {inst|hinst|port|pin|hpin}...[-through...]...
  |-to {inst|hinst|external_delay|clock|port|pin|hpin}...
  |-paths string } ]
[-setup] [-hold]
[-view analysis_view] [-name string]
```

Description

Modifies path constraints. Paths can be selected using the `-from`, `-through`, `-to`, or `-paths` options. You must provide at least one of these four options. The `-paths` option cannot be used in conjunction with any of the other three path selection options.

These path constraints could have been

- Computed by the timing engine using the launching and capturing waveforms

The command returns the directory path to the object that it creates.

It is possible to have multiple `path_adjust` constraints on a path.

Options and Arguments

`-delay` *integer*	Specifies the delay constraint value, in picoseconds, by which the path has to be adjusted. A positive adjustment relaxes the clock constraint and a negative adjustment tightens it.
`-from` {*inst*| *hinst*|*external_delay* | *clock* | *port* | *pin*| *hpin*}	
	Specifies a Tcl list of start points for the paths. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which the specified external delay timing exception applies.
	Only paths that start at one of the ports or pins, or paths that are launched by one of the clock objects will have the timing exception applied to them.
`-hold`	Specifies that exception is for hold timing analysis only.
	By default, exception applies for both setup and hold timing analysis.
`-name` *string*	Associates a name with the specified timing exception. If another timing exception already exists with that name, the existing timing exception is replaced with the new one.
	The name may be useful for later finding the exception (with the vfind command) and for recognizing the exception in reports.
	Default: `adj_n`

`-paths` *string*	Specifies the paths to which the exception should be applied. The string argument should be created using the set_path_specification command. The `-paths` option cannot be used in conjunction with any of the other three path selection options (`-from`, `-through`, `-to`).
`-setup`	Specifies that exception is for setup timing analysis only.
	By default, exception applies for both setup and hold timing analysis.
`-through` {*inst*| *hinst* | *port* | *pin*| *hpin*}	
	Specifies a Tcl list of a sequence of points that a path must traverse. Points to traverse can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances.
	You can repeat the `-through` option to require that a path first must traverse one of the objects in the first set, then pass through one of the objects in the second set, and so on.
`-to` {*inst*| *hinst* | *external_delay* | *clock* | *port* | *pin*| *hpin*}	
	Specifies a Tcl list of end points for the paths. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which the specified external delay timing exception applies.
	Only paths that end at one of the ports or pins, or paths that are captured by one of the clock objects, have the exception applied to them.
`-view` *analysis_view*	Specifies the *analysis_view*, the paths should be restricted to.

Example

The following example is equivalent and apply to paths starting from (clock) pin `clk1`:

```
path_adjust -from clk1

path_adjust -paths [set_path_specification -from clk1]
```

Related Information

Affects these commands:	report_timing
	report_qor
	set_path_specification
	syn_generic
	syn_map
	syn_opt
	write_design
	write_sdc
Related commands:	path_group
	set_path_specification
Sets these attributes:	exception Attributes

path_group

```
path_group
{ -from {inst|hinst|external_delay|clock|port|pin|hpin}...
 |-through {inst|hinst|port|pin|hpin}... [-through...]...
 |-to {inst|hinst|external_delay|clock|port|pin|hpin}...
 |-paths string} ...
[-view analysis_view] [-group string] [-setup] [-hold]
[-name string]
[-not_through {inst|hinst|external_delay|clock|port|pin|hpin}..]
```

Description

Assigns paths that meet the path selection criteria to a cost group. Paths can be selected using the `-from`, `-through`, `-to`, or `-paths` options. You must provide at least one of these four options. The `-paths` option cannot be used in conjunction with any of the other three path selection options.

Options and Arguments

`-from {`*`inst`* `|` *`hinst`* `|`*`external_delay`* `|` *`clock`* `|` *`port`* `|` *`pin`* `|` *`hpin`*`}`	
	Specifies a Tcl list of start points for the paths. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports.
`-group` *`string`*	Specifies the name of the cost group (defined with `define_cost_group`) to which the path is added.
`-hold`	Specifies that exception is for hold timing analysis only.
	By default, exception applies for both setup and hold timing analysis.
`-name` *`string`*	Associates a name with the specified timing exception.
`-not_through {`*`pin`*`|`*`hpin`*`|`*`external_delay`*`|`*`clock`*`|`*`port`*`|`*`inst`*`|`*`hinst`*`}`	
	Specifies a Tcl list of points that a path group not include. Points to avoid can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances, external delays or clocks.
`-paths` *`string`*	Specifies the paths to which the exception should be applied. The string argument should be created using the set_path_specification command. The `-paths` option cannot be used in conjunction with any of the other three path selection options (`-from`, `-through`, `-to`).
`-setup`	Specifies that exception is for setup timing analysis only.
	By default, exception applies for both setup and hold timing analysis.
`-through {`*`inst`* `|` *`hinst`* `|` *`port`* `|` *`pin`* `|` *`hpin`*`}`	

	Specifies a Tcl list of a sequence of points that a path must traverse. Points to traverse can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances.
	You can repeat the `-through` option to require that a path first must traverse one of the objects in the first set, then pass through one of the objects in the second set, and so on.
`-to {`*`inst`* `|` *`hinst`* `|` *`external_delay`*`|` *`clock`*`|` *`port`*`|` *`pin`* `|` *`hpin`*`}`	
	Specifies a Tcl list of end points for the paths. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports.
`-view` *`analysis_view`*	
	Specifies an *`analysis_view`*, the paths should be restricted to.

Examples

- The following example assigns the paths from all inputs to all outputs to the group called I2O, which was previously defined by a `define_cost_group` command:

  ```
  path_group -from port:design_name/in* -to port:design_name/out* -group I2O
  ```

- The following examples are equivalent and apply to paths starting from (clock) pin `clk1`:

  ```
  path_group -from clk1
  ```

  ```
  path_group -paths [set_path_specification -from clk1]
  ```

Related Information

Affects these commands:	report_timing
	report_qor
	set_path_specification
	syn_generic
	syn_map
	write_design
	write_sdc
Related commands:	path_adjust
	set_path_specification
Sets these attributes:	exception Attributes

phys_enable_ocv

```
phys_enable_ocv
[-socv_to_aocv|-native_socv|-native_aocv]
-design design
```

Description

Controls enabling of the Advanced On-Chip Variation (AOCV) and Statistical On-Chip Variation (SOCV) flows.

Options and Arguments

`-design` *design*	Specifies the name of the design for which to enable the specified flow.
`-native_aocv`	Enables the native AOCV flow.
`-native_socv`	Enables the native SOCV flow.
`-socv_to_aocv`	Enables the SOCV to AOCV flow.

Related Information

Related command:

phys_reset_ocv

phys_reset_ocv

```
phys_reset_ocv
-design design
```

Description

Resets the Advanced On-Chip Variation (AOCV) and Statistical On-Chip Variation (SOCV) flows.

Options and Arguments

`-design` *design*	Specifies the name of the design for which to reset the flow.

Related Information

Related command:

phys_enable_ocv

report_constraint

```
report_constraint [-check_type {clock_period|pulse_width}]
[-drv_violation_type {max_capacitance | max_transition | max_fanout | min_capacitance |
min_transition | min_fanout}] [-view analysis_view+]
[-all_violators] [-no_wrap] [{ > | >>} file]
```

Description

Reports constraint information of current design. The constraints information includes information such as whether or not constraint is violated, by how much amount, and the worst violator object.

The following timing checks are reported with the `report_constraint` command:

- DRV violations
- Clock and asynchronous checks including `pulse_width` and `min_period`

The `report_constraint` command supports DRV constraints, that is, `set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`, applied on library pins.

> By default, the `report_constraint` command reports only the mean slews.

Options and Arguments

`-all_violators`	Specifies to display all the violating constraints. Specifies to report negative slack values for timing checks. The supported options that can be given with `check_type` and `drv_violation_type` are: ○ `clock_period` ○ `pulse_width` ○ `max_capacitance` ○ `min_capacitance` ○ `max_transition` ○ `min_transition` ○ `max_fanout` ○ `min_fanout`
	Use the `-all_violators` parameter only for reporting specific check types to generate report for all violators of that particular check type only.
`-check_type` *`string`*	Specifies the type of constraint check (setup or hold) to be performed.
	The supported check types are: • `pulse_width` — Reports only the paths that end at the `pulse_width` timing check. If the `-all_violators` parameter is not specified, then the `pulse_width` option reports only one entry of the most critical width wise check. The DRV checks also report only the most critical entry. To report all the violated `pulse_width` checks, you need to specify `report_constraint -check_type pulse_width -all_violators` command (which is consistent with other check types of the `report_constraint` command). • `clock_period` — Reports only the paths that end at the `clock_period` timing check.
`-drv_violation_type` *`string`*	
	Specifies the type of DRV violation check to be performed.

	The supported DRV check types are: • `max_capacitance` — Reports pins having the worst `max_capacitance` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual capacitance is greater than the `max_capacitance` value specified in library and timing constraints file. • `min_capacitance` — Reports pins having the worst `min_capacitance` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual capacitance is less than the `min_capacitance` value specified in the library and timing constraints file. • `max_transition` — Reports pins having the worst `max_transition` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual transition is greater than the `max_transition` value specified in the library and timing constraint file. • `min_transition` — Reports pins having the worst `min_transition` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual transition is less than the `min_transition` value specified in the library and timing constraints file. • `max_fanout` — Reports pins having the worst `max_fanout` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual fanout is greater than the `max_fanout` value specified in the library and timing constraints file. • `min_fanout` — Reports pins having the worst `min_fanout` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual fanout is less than the `min_fanout` value specified in the library and timing constraints file.
`{> | >>} file`	Specifies the name of the file to write the report.
	When specified with >, the command writes the report to the specified file name. If the file already exists, the software overwrites it.

	When specified with >>, the command writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
	Default: Report is displayed on standard output without being saved.
	The filename parameter must be the last argument in the list.
-no_wrap	Specifies to skip wrapping the report output.
-view *analysis_view*	
	Specifies the view for which reporting should be done.

Examples

- The following example reports all the pins having the `max_transition` constraint violated:

```
genus:root: top> report_constraint -drv_violation_type max_transition -
all_violators

============================================================
check type : max_transition for mode default
------------------------------------------------------------------------------
|     Pin      |Required Slew (ps)|Actual Slew (ps)|Slack Slew (ps)| Mode  |
------------------------------------------------------------------------------
| pin:top/g1/A |              280 |             700 |           -420 |default |
| pin:top/g3/A |              280 |             700 |           -420 |default |
| pin:top/g2/A |              280 |             700 |           -420 |default |
| port:top/in1 |              500 |             700 |           -200 |default |
| port:top/clk |              500 |             700 |           -200 |default |
| port:top/rst |              500 |             700 |           -200 |default |
| port:top/vdd |              500 |             700 |           -200 |default |
| port:top/vss |              500 |             700 |           -200 |default |
------------------------------------------------------------------------------
```

- The following example reports the pin having the worst `max_transition` constraint.

```
genus:root: top> report_constraint -drv_violation_type max_transition

============================================================

check type : max_transition for mode default

-------------------------------------------------------------------------------
|     Pin       |Required Slew (ps) |Actual Slew (ps) |Slack Slew (ps) | Mode   |
-------------------------------------------------------------------------------
| pin:top/g1/A |               280 |             700 |           -420 |default|
-------------------------------------------------------------------------------
```

- The following example reports the path with the most critical `pulse_width` check

```
genus:root: top> report_constraint -check_type pulse_width
============================================================

check type : pulse_width

-------------------------------------------------------------------------------
|         Endpoint          |Required Pulse Width |Clock Pulse Width |Slack |
-------------------------------------------------------------------------------
| pin:top/u_sub/out1_reg/CK |                  73 |             2000 | 1927 |
-------------------------------------------------------------------------------
```

Related Information

Affected by these commands:

syn_map
syn_opt

Affected by these attributes:

max_capacitance
max_fanout
max_transition
min_capacitance

min_fanout
min_transition

reset_path_adjust

```
reset_path_adjust
[-view analysis_view] -path_adjust_group string
```

Description

Resets the path adjust constraints that were set to a `path_adjust` group.

Options and Arguments

`-path_adjust_group` *string*	
	Specifies the group of path to reset. The group was created by `set_path_adjust_group` command.
`-view` *analysis_view*	Specifies the *analysis_view*, the paths should be restricted to.

Example

The following command reset path adjust constraints for group `r2r`:

```
reset_path_adjust -path_adjust_group r2r
```

Related Information

Related commands:

path_group

path_adjust
set_path_adjust
set_path_adjust_group

reset_path_exception

```
reset_path_exception
     [-rise]
     [-fall]
     [-from clock|port|inst|hinst|pin|hpin|port_bus+]
     [-rise_from clock|port|inst|hinst|pin|hpin|port_bus+]
     [-fall_from clock|port|inst|hinst|pin|hpin|port_bus+]
     [-to clock|port|inst|hinst|pin|hpin|port_bus+]
     [-rise_to clock|port|inst|hinst|pin|hpin|port_bus+]
     [-fall_to clock|port|inst|hinst|pin|hpin|port_bus+]
     [-through port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus+]+
     [-rise_through port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus+]+
     [-fall_through port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus+]+
     [-setup]
     [-hold]
     [-all]
     [-type string]
```

Description

Removes any previously set path exceptions (or path exception of the specified type) for the given paths. To remove all path exceptions, use the `-all` option with the `reset_path_exception` command.

By default, all path exception assertions are reset.

Options and Arguments

`-rise`	Specifies the rise value

`-fall`	Specifies the fall value
`-from clock|port|inst|hinst|pin|hpin|port_bus`	Specifies pins at the start of the path exceptions that are to be reset.
`-rise_from clock|port|inst|hinst|pin|hpin|port_bus`	Specifies the from rise list
`-fall_from clock|port|inst|hinst|pin|hpin|port_bus`	Specifies the from fall list
`-to clock|port|inst|hinst|pin|hpin|port_bus`	Specifies pins at the end of the path exceptions that are to be reset.
`-rise_to clock|port|inst|hinst|pin|hpin|port_bus`	Specifies the to rise list
`-fall_to clock|port|inst|hinst|pin|hpin|port_bus`	Specifies the to fall list
`-through port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus`	Specifies the pins that the path to be reset goes through.

`-rise_through port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus`	Specifies the through rise list
`-fall_through port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus`	Specifies the through fall list
`-setup`	Resets only the setup paths
`-hold`	Resets only the hold paths
`-all`	Removes all path exceptions. This option cannot be used with the -from, -through, and -to options.
`-type` *`string`*	Removes only path exceptions of the specified type

Example

- The following command removes the set path exception:

```
reset_path_exception -from R1/CK -rise_to R2/D -type false_path
```

set_path_adjust

```
set_path_adjust
{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
|-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
|-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-paths string][-view analysis_view] -delay delay
[-setup] [-hold] [-name name]
[-not_through {inst|hinst|external_delay|clock|port|pin|hpin}..]
```

Description

Specifies the delay constraint value by which the path has to be adjusted. It calls the Genus native command `path_adjust`.

Path Adjust is a feature that allows users to specify the delay constraint value, overwriting the calculated value. The advantage of using path adjust is that it is not written out in the SDC constraints, so the place-and-route tool sees the timing.

Use the `set_path_adjust` command selectively to improve timing for certain paths in timing-critical designs because it might increase area.

Options and Arguments

-delay delay	Specifies the delay adjust value, the value by which the path has to be adjusted. It is specified in picoseconds. A positive adjustment relaxes the clock constraint and a negative adjustment tightens it.
-from {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that start from the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
-hold	Specifies the adjust value for early paths.

	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	Default: setup
`-name` *name*	Specifies the name of the timing exception you want to create.
	Default: `adj_`*n*
`-not_through {`*pin*|*hpin*|*external_delay*|*clock*|*port*|*inst*|*hinst*`}`	
	Specifies a Tcl list of points not to include. Points to avoid can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances, external delays or clocks.
`-paths` *string*	Specifies a path identifier obtained as a result from the `specify_paths` command.
`-setup`	Applies the path adjust constraint for setup analysis.
	Default: setup
`-through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths that traverse through the specified points. The points to traverse can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list of end points.
`-view` *analysis_view*	Specifies the analysis view named *analysis_view* to which the paths are restricted.

Examples

- The following commands tighten the reg2reg paths:

  ```
  set_path_adjust -delay 200 -from [all_register] -to [all_register] -name pa_c2c
  ```

- Remember to remove path adjusts before timing reports to normalize timing reports using the following command:

  ```
  delete_obj [get_db exceptions pa_*]
  ```

Related Information

Related command:

path_adjust

set_path_adjust_group

```
set_path_adjust_group
{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
|-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
|-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-paths string] [-name name]
```

Description

Specifies the delay constraint value by which the path has to be adjusted. It calls the Genus native command `path_adjust`.

Use the `set_path_adjust_group` command selectively to improve timing for certain paths in timing-critical designs because it might increase area.

> This command is available only if '`timing_set_path_adjust_unit`' attribute is `ns`.

Options and Arguments

`-from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that start from the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-name` *name*	Specifies the name of the path adjust group you want to create.
`-paths` *string*	Specifies a path identifier obtained as a result from the `specify_paths` command.
`-through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths that traverse through the specified points. The points to traverse can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list of end points.

Example

- The following command creates a group for reg2reg paths:

```
set_path_adjust_group -name AdJGROUP_1 -from [all_register] -to [all_register]
```

Related Information

Related commands:

reset_path_adjust
set_path_adjust

set_path_specification

```
set_path_specification
[-view analysis_view] [-domain clock_domain]
[ -from {pin|hpin|port|clock|external_delay|inst|hinst}...
| -from_rise_clock {pin|hpin|port|clock|external_delay|inst|hinst}...
| -from_fall_clock {pin|hpin|port|clock|external_delay|inst|hinst}...
| -from_rise_pin {pin|hpin|port|clock|external_delay|inst|hinst}...
| -from_fall_pin {pin|hpin|port|clock|external_delay|inst|hinst}...]
[ -through {pin|hpin|port|inst|hinst}...
| -through_rise_pin {pin|hpin|port|inst|hinst}...
| -through_fall_pin {pin|hpin|port|inst|hinst}...] ...
[ -to {pin|hpin|port|clock|external_delay|inst|hinst}... |
| -to_rise_clock {pin|hpin|port|clock|external_delay|inst|hinst}...
| -to_fall_clock {pin|hpin|port|clock|external_delay|inst|hinst}...
| -to_rise_pin {pin|hpin|port|clock|external_delay|inst|hinst}...
| -to_fall_pin {pin|hpin|port|clock|external_delay|inst|hinst}...]
[ -capture_clock_pins {pin|hpin}...
| -capture_clock_pins_rise_clock {pin|hpin}...
| -capture_clock_pins_fall_clock {pin|hpin}...
| -capture_clock_pins_rise_pin {pin|hpin}...
| -capture_clock_pins_fall_pin {pin|hpin}... ]
[-lenient]
```

Description

Creates a string that indicates the path of a particular timing exception. You must use the `set_path_specification` command as an argument to the `-paths` option in any of the timing exception commands. Paths can be selected using the `-from`, `-through`, `-to`, or `-paths` options. You must provide at least one of these three options.

The `set_path_specification` command gives you more detailed control when specifying timing exceptions than the `-from`, `-through`, `-to` options in these timing exceptions could provide.

Options and Arguments

```
-capture_clock_pins {pin | hpin}
```

	Specifies a Tcl list of sequential clock pins that capture data. This option can be useful with complex sequential cells such as RAMs that have multiple clock pins.
`-capture_clock_pins_fall_clock {`*pin* `|` *hpin*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the falling edge of the ideal clock waveform. This option can be useful with complex sequential cells such as RAMs that have multiple clock pins.
`-capture_clock_pins_fall_pin {`*pin* `|` *hpin*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the fall transitions of the specified sequential clock pin. This option can be useful with complex sequential cells such as RAMs that have multiple clock pins.
`-capture_clock_pins_rise_clock {`*pin* `|` *hpin*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the rising edge of the ideal clock waveform.
`-capture_clock_pins_rise_pin {`*pin* `|` *hpin*`}`	
	Specifies a Tcl list of sequential clock pins that capture data at the rise transitions of the specified sequential clock pin.
`-domain` *clock_domain*	
	Specifies a clock domain the paths should be restricted to.
`-from {`*pin* `|` *hpin* `|` *port* `|` *clock* `|` *external_delay* `|` *inst* `|` *hinst*`}`	
	Specifies a Tcl list of start points for the paths. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which specified external delay timing exceptions apply.
	Also, if you specify both the `-from` option on an enable pin of a latch and the `-lenient` option, the paths starting on the D pin will also be included in the timing exception.
`-from_fall_clock {`*pin* `|` *hpin* `|` *port* `|` *clock* `|` *external_delay* `|` *inst* `|` *hinst*`}`	

	Specifies a Tcl list of start points for the paths. The paths are restricted to launch at the falling edge of an ideal clock waveform. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which specified external delay timing exceptions apply.
`-from_fall_pin {`*`pin`*` | `*`hpin`*` | `*`port`*` | `*`clock`*` | `*`external_delay`*` | `*`inst`*` | `*`hinst`*`}`	
	Specifies a Tcl list of start points for the paths. The paths are restricted to fall transitions at the start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which specified external delay timing exceptions apply.
`-from_rise_clock {`*`pin`*` | `*`hpin`*` | `*`port`*` | `*`clock`*` | `*`external_delay`*` | `*`inst`*` | `*`hinst`*`}`	
	Specifies a Tcl list of start points for the paths. The paths are restricted to launch at the rising edge of an ideal clock waveform. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which specified external delay timing exceptions apply.
`-from_rise_pin {`*`pin`*` | `*`hpin`*` | `*`port`*` | `*`clock`*` | `*`external_delay`*` | `*`inst`*` | `*`hinst`*`}`	
	Specifies a Tcl list of start points for the paths. The paths are restricted to rise transitions at the start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, a combination of these, instances, or input ports to which specified external delay timing exceptions apply.

`-lenient`	Also, if you specify both the `-from` option on an enable pin of a latch and the `-lenient` option, the paths starting on the D pin will also be included in the timing exception.
	If a `-from` option is provided that is not a valid start point or a `-to` option is provided that is not a valid end point, then a warning is issued but the rest of the command succeeds. These invalid points are stored in the exception, but will not affect timing analysis results. A warning is also issued when using the `report_timing` command in this situation.
`-through {`*`pin`*` | `*`hpin`*` | `*`port`*` | `*`inst`*` | `*`hinst`*`}`	
	Specifies a Tcl list of points that a path must traverse. Points to traverse can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances.
	You can repeat the `-through` option to require that a path first must traverse one of the objects in the first set, then pass through one of the objects in the second set, and so on.
`-through_fall_pin {`*`pin`*` | `*`hpin`*` | `*`port`*` | `*`inst`*` | `*`hinst`*`}`	
	Specifies a Tcl list of points that a path must traverse. The path is restricted to fall transitions at the indicated points. Points to traverse can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances.
	You can repeat the `-through` option to require that a path first must traverse one of the objects in the first set, then pass through one of the objects in the second set, and so on.
`-through_rise_pin {`*`pin`*` | `*`hpin`*` | `*`port`*` | `*`inst`*` | `*`hinst`*`}`	
	Specifies a Tcl list of points that a path must traverse. The path is restricted to rise transitions at the indicated points. Points to traverse can be ports, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances.
	You can repeat the `-through` option to require that a path first must traverse one of the objects in the first set, then pass through one of the objects in the second set, and so on.

`-to {`*`pin	hpin	port	clock	external_delay	inst	hinst`*`}`	
	Specifies a Tcl list of end points for the paths. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which specified external delay timing exceptions apply.
`-to_fall_clock {`*`pin	hpin	port	clock	external_delay	inst	hinst`*`}`	
	Specifies a Tcl list of end points for the paths. The paths are restricted to capture at the falling edge of an ideal clock waveform. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which specified external delay timing exceptions apply.
`-to_fall_pin {`*`pin	hpin	port	clock	external_delay	inst	hinst`*`}`	
	Specifies a Tcl list of end points for the paths. The paths are restricted to fall transitions at the end points. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which specified external delay timing exceptions apply.
`-to_rise_clock {`*`pin	hpin	port	clock	external_delay	inst	hinst`*`}`	
	Specifies a Tcl list of end points for the paths. The paths are restricted to capture at the rising edge of an ideal clock waveform. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which specified external delay timing exceptions apply.
`-to_rise_pin {`*`pin	hpin	port	clock	external_delay	inst	hinst`*`}`	
	Specifies a Tcl list of end points for the paths. The paths are restricted to rise transitions at the end points. The end points can be output ports of your design, input pins of flip-flops, clock objects, or a combination of these, instances, or output ports to which specified external delay timing exceptions apply.
`-view` *`analysis_view`*	
	Specifies the analysis view to which the paths should be restricted.

Related Information

Affects these commands:

path_adjust
path_group
report_qor
report_timing
write_design
write_sdc

Affects this attribute:

paths

set_socv_rc_variation_factor

```
set_socv_rc_variation_factor
[-early] [-late] [-views analysis_view] float
```

Description

Applies global variation as percentage of the nominal delay. To enable this feature, set the `timing_socv_rc_variation_mode` global variable to `true`.

Options and Arguments

`-early`	Specifies the variation factor for early interconnect delays.
`float`	Specifies scaling value on interconnect delay. You can specify a number from 0 (minimum value) to 1 (maximum value)
`-late`	Specifies the variation factor for late interconnect delays.
`-views` *`analysis_view`*	
	Specifies to apply the SOCV interconnect variation factor to the specified list of analysis views.

Examples

- The following example apply interconnect variation factor to late interconnect delays:

  ```
  set_socv_rc_variation_factor -late -views view1 0.1
  ```

- The following example apply interconnect variation factor to early interconnect delays:

  ```
  set_socv_rc_variation_factor -early -views view1 0.1
  ```

- The following example apply interconnect variation factor to early and late interconnect delays:

  ```
  set_socv_rc_variation_factor -views view1 0.1
  ```

6

Customization

- create_command_help
- create_msg
- define_attribute
- define_msg
- define_proc
- parse_options

create_command_help

```
create_command_help command_name help [category]
```

Description

Adds a short help message for a new command to Genus's online help system.

Options and Arguments

category	Specifies the category to which this command should be added. You can specify an existing category or a new one.
	To see a list of all existing categories, type help.
command_name	Specifies the name of the command. It must be a Tcl procedure that you defined previously.
help	Lists the help text to be displayed when the `help` command is used. Use a string.

create_msg

```
create_msg message string [-caller string]
   [-obj_info string] [-file_info string]
   [-newline]
```

Description

Accesses the various native messages of Genus and the messages that were created with the `define_msg` command.

Options and Arguments

`-caller string`	Specifies the name of the calling procedure.
`-file_info string`	
	Specifies to which file the message applies.
message	Identifies the message to be sent.
	The identification is in the form of *group*-*id*. Refer to define_msg#96115define_msg for an explanation of *group* and *id*
`-newline`	Prints a new (empty) line before the message
`-obj_info string`	
	Prints the object type and name for the message
string	Describes the context-specific help.

define_attribute

```
define_attribute string
-category string -data_type string -obj_type string
[-hidden | obsolete] [-help_string string]
[-check_function string] [-compute_function string]
[-set_function string] [-default string]
[-skip_in_db] [-quiet]
```

Description

Creates a new, user-defined attribute with the specified characteristics.

Options and Arguments

-category *string*	
	Defines the category of the attribute. Categories group attributes that perform similar functions whereas object types describe where in the design an attribute is valid. You can specify any category name: both new and existing category names are valid.
-check_function *string*	
	Specifies a previously defined Tcl procedure's name in order to ensure that the newly defined attribute is valid. The Tcl procedure should be of the form:
	`proc {`*object value*`}`
	The Tcl procedure returns `1` for a valid value, and `0` for an invalid value.
-compute_function *string*	
	Specifies a previously defined Tcl procedure's name in order to get the newly defined attribute's value later (with the `get_db`) command. The Tcl procedure should be of the form:

	`proc {`*`object`*`}`
	When you use this option, the attribute becomes a read-only attribute because its value is always computed.
`-data_type` *`string`*	
	Defines the data type of the attribute. Possible data types are:
	• bool
	• coord
	• double
	• in_dir
	• in_file
	• int
	• object
	• object*
	• out_dir
	• out_file
	• point
	• polygon
	• rect
	• string

`-default` *string*	
	Specifies a default value for the attribute.
`-help_string` *string*	
	Specifies the help text for the attribute.
`-hidden`	Specifies whether the defined attribute is a hidden attribute.
`-obj_type` *string*	
	Specifies the object type of the attribute. All valid object types can be found by typing `vfind -help` at the Genus prompt.
`-obsolete`	Specifies that the defined attribute is obsolete.
`-quiet`	Suppresses all the generated messages.
`-set_function` *string*	
	Specifies a previously defined Tcl procedure's name. This option allows you to override user-defined values provided it conforms to the parameters in the Tcl procedure you created. The Tcl procedure should be of the form:
	`proc {`*object new_value current_value*`}`
`-skip_in_db`	Prevents the `write_db` command from writing out the defined attribute.
string	Specifies the name of the attribute.

define_msg

```
define_msg -group string -id number
-short_description string
-long_description string
[-internal_group]
[{-error|-warning|-info_priority number}]
```

Description

Creates a custom message that can subsequently be accessed with the `create_msg` command.

Options and Arguments

`-error`	Creates an error message.
`-group` *string*	Specifies the group of messages that the new message belongs to. A `group` groups messages that apply to a certain engine of the tool. For example, the `MAP group` groups messages issued by the mapper.
	If you want to create a message for an internal group, you must specify an existing group name.
`-id` *integer*	Specifies an identification number for the message. The number must be unique for the specified group. If the specified number already exists, you will overwrite the existing message.
	Default: `1`
`-info_priority` *integer*	
	Creates an info message with the specified priority. You can specify a number between 2 and 8.
`-internal_group`	Specifies whether the message belongs to an internal group.
`-long_description` *string*	
	Defines the help of the message.
`-short_description` *string*	
	Specifies the title or the description of the message.
`-warning`	Creates a warning message.

Related Information

Affects this command:

create_msg

define_proc

proc_name
argument_definition
proc_body
[-description *string*]
[-extended_help *formatted text*]
[-category *category name*]
[-returns {*type description*}]

Description

Creates a new, user-defined attribute with the specified characteristics.

Options and Arguments

proc_name	
	Specifies the name of the procedure
argument_definition	
	Specifies the list of traditional Tcl arguments or Extended syntax arguments
proc_body	
	Specifies the body of the code for evaluation.
-description *string*	
	Specifies the short description of the procedure to be generated as part of -help queries to the procedure.
-extended_help *formatted text*	
	Specifies the preformatted text to be generated as part of -help queries to the procedure.
-category *category name*	
	Specifies the command category name.
-returns {*type description*}	
	Specifies an optional *type descriptor* to check the return values from the procedure against and a description.

Example

```
define_proc my_status -description "Print a status message."
{
   traditional_var
   { positional_var ""       {positional_var} {string}
{required} "Example of a positional var." }
   { lucky_numbers  {}       {-lucky_numbers} {integer *}
 {optional} "List of lucky numbers." }
```

```
   { season         {spring} {-season}        {enum winter spring summer autumn}
{optional {requires moon}} "The season" }
   { moon           {full}   {-moon}          {string}
{optional} "Positional arg - Moon description." }
   { dir            {[pwd]}  {-directory}     {string}
{optional} "Directory. (defaults to current dir.)" }
   { file_ptr       {stdout} {>}              {filename}
{optional} "filename for output. (defaults to stdout)" }
  } {
    puts $file_ptr "Greeting: '$traditional_var $positional_var'"
    puts $file_ptr "Lucky Numbers are: [join $lucky_numbers {,}]"
    puts $file_ptr "The season is '$season' and the moon is '$moon'."
    if { $file_ptr ni {stdout stderr} } {
      close $file_ptr
    }
    return $dir
  } -returns {string}

  % my_status "Hello," "world!" -moon blue -lucky_numbers {10 22 45} -season winter
  Greeting: 'Hello, world!'
  Lucky Numbers are: 10,22,45
  The season is 'winter' and the moon is 'blue'.

  % mystatus "Hello to file." > some_file.gz
```

parse_options

parse_options *cmd file_var* [*args*] [*code var*]...

Description

Interfaces to the Genus internal command option parser. The Genus argument parser provides the following features:

- Checking the correctness for arguments and types. Appropriate messages are issued when

the input is incorrect.

- No specific order of arguments is required. For example, `vls -long -attribute` behaves just like `vls -attribute -long`.
- Unique abbreviations of arguments is supported. For example, `vls -l` behaves just like vls `-long`.
- Online help is provided if `-help` is specified.
- Optional file redirection is supported. For example, `report_gates >> design.rpt` causes output to be appended to the file `design.rpt`.
- Genus objects can be implicitly searched for based on their type.

You can use the command option parser to make a Tcl procedure behave just like a built-in Genus command by providing on-line help, finding objects automatically, checking for required options, handling unique argument abbreviations, and handling file redirection.

This command can return one of the following values:

- -2: You asked for help and help was provided. The command returns normally.
- 0: Your options were invalid and the command aborts.
- 1: Your options were valid and the command continues normally.

Options and Arguments

args	Specifies a list containing the options that the user sends to your command. Usually your procedure will be defined like this:
	`proc your_procedure {args} {... (code that implements the procedure)}`
	You would then pass `$args` as the args parameter to `parse_options` within your procedure.
cmd	Specifies the name of the command whose options to parse.
	This name appears in the help listing if a user calls your procedure with `-h` or if a user does not provide valid arguments.

code	Specifies a string that describes the command argument: flag name, whether it is required or optional, what type of data it accepts, and a short help message about it. The string must be in the form:	
	`"(-<name>)?<x><y><z>(<dirtypes>)? <help>"`	
	The question marks in the above string mean that these fields of the string are optional.	
	You cannot specify multiple string values if you are specifying a flag name. That is, if you are specifying a flag, you cannot also specify the `som` (string, optional, multiple values) or `srm` (string, required, multiple values) combinations.	
	<name> is the name of the flag. For example, `-number`.	
	<x> is a single character indicating the type of the option:	
	`b`	Boolean
	`d`	directory object
	`e`	enum
	`f`	float
	`n`	integer number
	`s`	string
	<y> is a single character indicating whether the option is optional or required	
	`o`	optional
	`r`	required
	`x`	obsolete
	<z> is a single character indicating whether lists are accepted	
	m	Accepts multiple values: lists OK
	s	Accepts single value only: no lists

	<dirtypes> is a string indicating the types of directory objects the option accepts. This string is required for all arguments that have the *<x>* field set to `d` and cannot be specified otherwise.
	The list of specified directory types can only be separated by vertical bars (|), that is, no spaces allowed, and must be enclosed in parentheses.
	<help> is a string that indicates to the user of your command what the purpose of the option is.
file_var	Specifies the name of a variable that holds the file handle if the user calls your procedure with > *file* or >> *file*. Your procedure should always check to see if this variable has been set to something other than ‘stdout’. If it has, then your command should send its output to that file handle instead of ‘stdout’ and you should close the file handle once your command is complete.
var	Specifies the name of the variable that will be set with the parsed result for that argument.

7

Design for Test

- add_analyzed_test_points
- add_core_wrapper_cell
- add_hard_repair
- add_jtag_boundary_scan
- add_jtag_macro
- add_lbist
- add_lockup_element
- add_los_pipeline
- add_opcg
- add_opcg_hold_mux
- add_pmbist
- add_pmbist_access_method
- add_ptam
- add_scan_power_gating
- add_shadow_logic
- add_shift_register_test_points
- add_test_compression
- add_test_point
- add_user_test_point
- add_wir_signal_bits
- add_wrapper_cell_aware_clock_gating
- analyze_atpg_testability
- analyze_initmode_processing
- analyze_scan_compressibility
- analyze_test_points
- check_atpg_rules
- check_dft_pad_cfg
- check_dft_rules
- check_dft_setup
- compress_scan_chains
- connect_dft_hier_test_cores
- connect_opcg_segments
- connect_scan_chains
- connect_serial_scan_chains
- convert_to_opcg_scan
- convert_to_scan
- create_chains_database
- define_dft_cfg_mode
- define_hier_test_core_concat_order
- define_hier_test_scan_mapping
- define_jtag_boundary_scan_segment
- define_jtag_instruction
- define_jtag_instruction_register
- define_jtag_macro
- define_jtag_tap_port
- define_mbist_clock
- define_opcg_domain
- define_opcg_domain_macro_parameters
- define_opcg_mode
- define_opcg_osc_source
- define_opcg_trigger
- define_pmbist_direct_access
- define_pmbist_port
- define_scan_abstract_segment
- define_scan_chain
- define_scan_fixed_segment
- define_scan_floating_segment
- define_scan_preserved_segment
- define_shift_register_segment
- define_test_bus_port
- define_test_clock
- define_test_signal
- fix_dft_violations
- fix_pad_cfg
- fix_scan_path_inversions
- identify_domain_crossing_pins_for_cgic_and_scan_a
- identify_multibit_cell_abstract_scan_segments
- identify_shared_wrapper_cells_in_design
- identify_shift_register_scan_segments
- identify_test_mode_registers
- map_dft_unmapped_logic
- map_pmbist_cgc_to_cgic
- map_pmbist_ffn
- map_pmbist_ffsync
- map_pmbist_map2mux
- prepare_for_test_point_analysis
- read_dft_abstract_model
- read_dft_jtag_boundary_file
- read_dft_pmbist_interface_files
- read_pmbist_memory_view
- reset_opcg_equivalent
- reset_scan_equivalent
- set_compatible_test_clocks
- set_opcg_equivalent
- set_scan_equivalent
- update_scan_chains
- write_dft_abstract_model
- write_dft_atpg
- write_dft_atpg_other_vendor_files
- write_dft_bsdl
- write_dft_compression_macro
- write_dft_compression_test_points
- write_dft_constraints

- write_dft_deterministic_test_points
- write_dft_jtag_boundary_file
- write_dft_jtag_boundary_verification
- write_dft_lbist_macro
- write_dft_lbist_test_points
- write_dft_lbist_testbench
- write_dft_pmbist_interface_files
- write_dft_pmbist_testbench
- write_dft_rtl_model
- write_dft_rtl_opcg_domain_macro
- write_dft_rtl_opcg_trigger_macro
- write_dft_rtl_unified_lbist_macro
- write_memory_physical_map
- write_modus_ctl_tp_file
- write_modus_no_tp_file
- write_scandef

The following commands are common with Analysis and Reporting commands:

- report_scan_registers
- report_scan_setup
- report_test_power
- report_scan_chains
- report_scan_compressibility
- report_chains_longest_wires
- report_chains_total_wirelength
- report_chains_wires
- report_core_wrapper_cell
- report_dft_analyzed_test_points
- report_dft_trace_back
- report_dft_violations
- report_opcg_clock_domain_info
- report_opcg_equivalents
- report_scan_compression_logic

add_analyzed_test_points

```
add_analyzed_test_points
     [-input_test_point_file file]
     [-max_number_of_test_point_nodes integer]
     [-max_number_of_test_point_flops integer]
     [-compatibility_matrix_file string]
     [-min_slack integer]
     [-share_observe_flop integer]
     [-control_only]
     [-share_control_flop integer]
     [-observe_only]
     [-share_tp_flop integer]
     [-test_clock_pin {port|pin|hpin}]
     [-test_control test_signal]
     [-test_control_for_control_tp test_signal]
     [-test_control_for_observe_tp test_signal]
     [-gate_clock_test_control test_signal]
     [-gate_clock_for_observe test_signal]
     [-write_data_for_physical_opt string]
     [-dont_map]
     [-verbose]
     [design]
     [-preview]
```

Description

Inserts selected test points analyzed by Modus Compression, LBIST, or Deterministic Testpoint Analysis commands. Test points can be inserted that have minimal impact on the slack and which target a minimum specified fault count.

Options and Arguments

`-control_only`	Specifies to insert only control type test points.
design	Specifies the name of the top-level design on which you want to perform testpoint insertion.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_map`	Prevents the inserted test points from being mapped even if the design is already mapped to the target library.
`-gate_clock_for_observe` *test_signal*	
	Specifies the test signal to be used to gate the clock of the observation flop.
`-gate_clock_test_control` *test_signal*	
	Specifies the test signal to be used to control the gating logic of the clock of the test point for control flop.
`-input_test_point_file` *file*	
	Specifies the name of the file containing the test point locations.
	The file is specified in Modus format.
`-max_number_of_test_point_flops` *integer*	
	Specifies the number of test point flops to be considered for insertion.
	You must specify an integer value greater than `0` when attempting to insert test points from a file using the `-input_test_point_file` option.
	By default, all test points are inserted.
`-max_number_of_test_point_nodes` *integer*	
	Specifies the number of test point nodes to be inserted.
	You must specify an integer value greater than `0` when attempting to insert test points from a file using the `-input_test_point_file` option.
	By default, all test points are inserted.
`-compatibility_matrix_file` *string*	
	The compatibility matrix file used when sharing the testpoints.
`-min_slack` *integer*	Limits the insertion of a test point to those nodes that have the specified minimum slack (in ps).
	Default: `-10000000`
`-observe_only`	Specifies to insert only observation type test points. In this case, you do not need to specify a test control signal.
`-preview`	Specifies to preview the run command.
`-share_observe_flop` *integer*	
	Specifies the number of observation test nodes that can share an observation flop through an XOR tree.
	Default: 4
`-share_control_flop` *integer*	
	Specifies the number of control testpoints that can share a control flop.
	Default: 4
`-share_tp_flop` *integer*	Specifies the number of observe or control testpoints that can share a control flop. It is overridden by `-share_observe_flop`/`-share_control_flop` if used together.

	Default: `1`
`-test_clock_pin {`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies the test clock that drives the clock pin of the inserted test points during test mode operation. Specify a port or pin that drives the test clock.
	If this option is not specified, the tool uses the test clock pin of the first flop in the fanin cone. If the tool cannot find any test clock pin in the fanin cone, it uses the first test clock in the `dft/test_clock_domains` directory.
`-test_control` *`test_signal`*	
	Specifies the test signal for controlling any type of test points.
	You must have specified the test signal using the `define_test_signal -function test_mode` constraint.
`-test_control_for_control_tp` *`test_signal`*	Specifies the test signal for controlling the control test points.
`-test_control_for_observe_tp` *`test_signal`*	Specifies the test signal for controlling the observe test points.
`-write_data_for_physical_opt` *`string`*	Specifies the write test point related information in a file that is later used for physical optimization.
`-verbose`	Specifies to print test point details.

Examples

- The following command inserts all of the test points, and when possible 3 test point locations will be shared through an XOR-tree to a common observation flop.

```
add_analyzed_test_points \
-input_test_point_file rc_et_dfa/myDesign.dfa.tpfile \
-test_control test_mode1 \
-test_clock_pin clk \
-share_observe_flop 3
```

- The following example inserts the first 500 test points whose fault count is greater than or equal to 3 from the specified test point file `myfile`.

```
add_analyzed_test_points -max_number_of_test_points 500 \
 -fault_threshold 3 -test_control tm -test_clock_pin clk \
-input_test_point_file myfile
```

- The following example inserts all test points for test nodes with a minimum slack of 3ps, and will share 3 observation test nodes through an XOR tree from the specified test point file `myfile`.

```
add_analyzed_test_points -min_slack 3 -share_observe_flop 3 \
-test_control tm -test_clock_pin clk -input_test_point_file myfile
```

- The following command inserts a maximum of 10 DFA test points whose nodes have a minimum slack of 1000 ps. Also, when possible, three test point locations will be shared through an XOR-tree to a common observation flop.

```
add_analyzed_test_points \
  -input_test_point_file rc_et_dfa/myDesign.dfa.tpfile \
  -min_slack 1000 \
  -test_control test_mode1 \
  -test_clock_pin clk \
  -share_observation_flop 3 \
  -max_number_of_test_points 10
```

- The following command inserts a maximum of 10 test points for locations whose fault count is greater than or equal to 15:

```
add_analyzed_test_points \
  -input_test_point_file rc_et_dfa/myDesign.dfa.tpfile \
  -test_control test_mode1 \
  -test_clock_pin clk \
  -max_number_of_test_points 10
  -fault_threshold 15
```

Related Information

Affected by these constraints:	define_test_clock
	define_test_signal -function test_mode

add_core_wrapper_cell

```
add_core_wrapper_cell

[-locations {pin|hpin|port}...]
[-floating_location_allowed]
[-dont_wrap_floating_locations_for_hierarchical_pins]
[-skip_locations_variable Tcl_variable]
[-shared_through {buffer|buffers|inverters_and_buffers|combinational} [-no_mux_before_shared_cell]]
[-wck {pin|hpin|port}]
-wsen_in {pin|hpin|port}
-wsen_out {pin|hpin|port}
[-wsen_reuse_or_for_shared_wrapper]
{-input_select_cfi {pin|hpin|port} |-wint {pin|hpin|port|constant} } {-output_select_cfi {pin|hpin|port} |-wext {pin|hpin|port|constant}[-
wcap {pin|hpin|port}]}
[-guard {0|1} -input_guard_control {pin|hpin|port} -output_guard_control {pin|hpin|port}]
[-dont_reuse_input_wrappers_for_output_ports]
```

```
[-exclude {pin|hpin|port}...]
[-exclude_clock test_clock]
[-exclude_combinational_feedthru_paths]
[-input_shared_threshold integer]
[-output_shared_threshold integer]
[-override_threshold_use_dedicated {pin|hpin|port}...]
[-override_threshold_use_shared {pin|hpin|port}...]
[-respect_dft_constants {test_setup_shiftenable|test_setup|tied|none}]
[-wrap_tied_constant_ports]
[-inside_core]
[-map_to_mux_for_dedicated_wrapper]
[-use_power_domain_of_ports_for_dedicated]
[-dont_map_to_mux_for_shared_wrapper_cell]
[-add_xor_feedback]
[-wint_in_out {pin|hpin|port|constant}]
[-name segment_prefix]
[-preview]
[-inside_sink_hierarchy]
[-no_wrapper_in_hierarchy string]
[-no_toggle_inverter_for_shared_wrapper]
[-add_mux_feedback]
[-design design]
```

Description

Selects a built-in IEEE 1500 standard wrapper cell based on the given specifications, inserts it at the specified location, and hooks it up to the specified control signals.

The wrapper cell logic is automatically identified as preserved wrapper-cell segments. You can use these segments in other segments (within nested segments) or specify the segments as elements when building the scan chains.

The command returns the directory path to the `scan_segment` objects that it creates. If multiple locations are specified, the command returns multiple segments.

> Any segment inserted by this command cannot be removed.

Options and Arguments

`-add_xor_feedback {pin|hpin|port}`	
	Specifies to add xor feedback on hold path and use pin or port as an input to xor gate.
-add_mux_feedback	Specifies to add feedback mux to both shared and dedicated wrapper cell.
`-design design`	Specifies the design in which you want to insert the wrapper cell. This option is required if multiple designs are loaded.
`-dont_reuse_input_wrappers_for_output_ports`	
	Allows to insert dedicated wrapper cells in the fanin of output ports that are being fed by wrapper cells previously inserted for input ports.
`-dont_wrap_floating_locations_for_hierarchical_pins`	
	Prevents wrapper cells from being inserted for hierarchical floating locations.
`-exclude {pin|hpin|port}`	
	Specifies the list of pins or ports to be excluded from wrapper insertion.
`-exclude_clock test_clock`	
	Specifies the test clock to be excluded for dedicated wrapper cells.
`-exclude_combinational_feedthru_paths`	
	Specifies whether to exclude pins or ports from wrapper insertion if combinational logic is found on the path from input to output.
`-floating_location_allowed`	

	Specifies to insert a wrapper cell even if the specified pin (location) is floating.
`-guard {0|1}`	Specifies the guard for safe_ value out of a wrapper cell. The safe value prevents testing of one block from interfering with another block.
	If this option is not specified, no guard logic will be included in the wrapper cell.
`-input_guard_control` {*pin*|*hpin*|*port*}	
	Specifies the in-guard control source pin or port name for an inward-facing wrapper cell.
	This signal will typically be controlled by the `-wext` control signal.
`-input_select_cfi` {*pin*|*hpin*|*port*}	
	Specifies the source pin name of the decoded control logic for the `select_cfi` signal for input bounding wrapper cells.
	Use this option to connect an already decoded signal. Otherwise, control decoder logic may have to be inserted for each cell, and extra control wires will have to be hooked up to each wrapper cell.
`-input_shared_threshold` *integer*	
	Specifies the maximum number of scannable flops that can be shared in wrapper cells. When the number of scannable flops exceeds the specified threshold value, a single dedicated wrapper cell will be inserted for the input port.
	Default: `10`
`-inside_core`	Inserts the wrapper cells associated with hierarchical pins inside the core module.
`-inside_sink_hierarchy`	
	Specifies to insert dedicated wrapper cell inside the sink hierarchy.
`-locations` {*pin*|*hpin*|*port*}	
	Specifies one or more pins that identify where the wrapper cell must be inserted.
`-map_to_mux_for_dedicated_wrapper`	
	Specifies to map the mux logic (associated with the CFI to CFO path and the optional Guard control path in the dedicated wrapper cell) to binary muxes from the target library. The binary muxes will be marked preserved (`preserve` attribute on libcell is set to `true`) for optimization. The instance names given to the muxes inserted for the CFI to CFO path and Guard control path are `dwc_cfi_mux` and `dwc_guard_mux` respectively.
	If you omit this option, the technology mapper will select the appropriate cell type(s) which satisfy the timing constraints.
`-dont_map_to_mux_for_shared_wrapper_cell`	
	Specifies not to keep the binary muxes intact inside the shared wrapper cell.
`-name` *segment_prefix*	
	Specifies the segment name prefix.
	If you specified a single location pin, and there is no name conflict, the segment name will correspond to the specified prefix, otherwise a unique name will be generated for each segment that is derived from the specified prefix.
`-no_mux_before_shared_cell`	
	Specifies to prevent the insertion of the hold mux on the test input of shared wrapper cells.
	Use this option when the wrapper cells are not configured for at-speed testing using delay test. The option applies to those wrapper cell configurations in which the input bounding wrapper cells (and output bounding wrapper cells) are always shifting in an INTEST mode (and EXTEST mode) during test (both scan-shift and functional capture).

	This option can only be specified when inserting the 1500 wrapper cells using the `-shared_through` approach.
`-no_wrapper_in_hierarchy` *string*	
	Specifies to have no wrapper inside hierarchy.
`-no_toggle_inverter_for_shared_wrapper`	
	Specifies to not add toggle inverter on the hold path of shared wrapper cell
`-output_guard_control` {*pin*|*hpin*|*port*}	
	Specifies the out-guard control source pin or port name for an outward-facing wrapper cell.
	This signal will typically be controlled by the `-wint` control signal.
`-output_select_cfi` {*pin*|*hpin*|*port*}	
	Specifies the source pin name of the decoded control logic for the `select_cfi` signal for output bounding wrapper cells.
	Use this option to connect an already decoded signal. Otherwise, control decoder logic may have to be inserted for each cell, and extra control wires will have to be hooked up to each wrapper cell.
`-output_shared_threshold` *integer*	
	Specifies the maximum number of scannable flops that can be shared in wrapper cells. When the number of scannable flops exceeds the specified threshold value, a single dedicated wrapper cell will be inserted for the output port.
	Default: `10`
`-override_threshold_use_dedicated` {*pin*|*hpin*|*port*}	
	Specifies the list of pins or ports for which a dedicated wrapper cell must be inserted.
`-override_threshold_use_shared` {*pin*|*hpin*|*port*}	
	Specifies the list of pins or ports for which a shared wrapper cell must be inserted.
`-preview`	Shows a preview of the wrapper cell that would be inserted.
`-respect_dft_constants{test_setup_shiftenable | test_setup | tied}`	
	Controls the propagation of constants and test signal values to prune the fanin (and fanout) paths of ports to find the sequential endpoints in the functional path for shared wrapper cell insertion. Possible values are:
	`test_setup_shiftenable`: Propagates shift_enable values
	`test_setup`: Propagates constant values and test_mode values
	`tied`: Propagates constant (1'b0, 1'b1) values
	If you omit this option, the command will perform full path tracing using a structural approach.
`-shared_through {buffer|buffers|inverters_and_buffers|combinational}`	
	Specifies whether the functional flop in the wrapper cell must be shared. If a shared cell is inserted, the command traces through the logic to identify a shareable functional flop (or flops).
	A functional flop in a wrapper cell can be shared if
	1. It is directly connected to the core pin through
	◦ a series of buffers (`buffer`)
	◦ a series of inverters and buffers

	◦ complex combination logic (`combinational`)
	1. It is mapped to a scan flip-flop for DFT purposes.
	1. The clock pin to the flop is controllable; that is, the flip-flop must pass the DFT rules.
	1. The flop has no connected enable pin.
	1. The functional flop is not already shared with another wrapper cell.
	If you use this option and the functional flop cannot be shared, the Genus-DFT engine issues a message and inserts a dedicated cell if you specified the `-wck` option, otherwise an error message is given.
`-skip_locations_variable` *Tcl_variable*	
	Writes the locations where no wrapper cells can be inserted to the specified Tcl variable. If you omit this option, the command fails if you have any such locations specified.
`-use_power_domain_of_ports_for_dedicated`	
	Causes dedicated wrapper cells inserted at the top hierarchy to inherit the power domain of the port which is covered by the wrapper cell.
	By default, the dedicated wrapper cell inherits the power domain of the hierarchy in which it is inserted.
`-wcap` {*pin*|*hpin*|*port*}	
	Specifies the capture control source pin or port name.
`-wck` {*pin*|*hpin*|*port*}	
	Specifies the test clock source pin or port name. If specified, this pin will connect to the clock pin of the inserted dedicated wrapper cells.
	If this option is not specified, the tool will identify a local test clock to be used to connect to the clock pin of the inserted dedicated wrapper cells.
`-wext` {*pin*|*hpin*|*port*|*constant*}	
	Specifies the control source pin or port name, or constant value for outward facing test mode.
	This option is ignored if you specified the `-output_select_cfi` option.
`-wint` {*pin*|*hpin*|*port*|*constant*}	
	Specifies the control source pin or port name, or constant value for inward-facing test mode.
	This option is ignored if you specified the `-input_select_cfi` option.
`-wint_in_out` {*pin*|*hpin*|*port*|*constant*}	
	Specifies the control source pin or port name, or constant value for inward as well as outward facing test mode.
`-wrap_tied_constant_ports`	
	Specifies to insert dedicated wrapper cells on output ports driven by a logic 0 or logic 1 constant.
`-wsen_in` {*pin*|*hpin*|*port*}	
	Specifies the shift-enable signal for input bounding wrapper cells. Specify the shift-enable source pin or port name.

`-wsen_out {pin|hpin|port}`	
	Specifies the shift-enable signal for output bounding wrapper cells. Specify the shift-enable source pin or port name.
`-wsen_reuse_or_for_shared_wrapper`	
	Specifies to use a single top level or for shared wrapper cells.

Examples

- The following example specifies to run the command in preview mode for ports `in[0]` and `out[0]`. The tool will indicate two dedicated wrapper cells would be inserted for the ports in the wrapper insertion summary report:

```
add_core_wrapper_cell -locations "in[0] out[0]" -wsen_in WSEN -wsen_out WSEN \
-wint WINT -wext WEXT -preview
```

- The following example inserts dedicated wrapper cells for ports `in[0]` and `out[0]` and uses a dedicated wrapper clock as the test clock for the wrapper cells. The two dedicated wrapper cells will be created as segments whose object names will start with the string `wrap`:

```
add_core_wrapper_cell -locations "in[0] out[0]" -wsen_in WSEN -wsen_out WSEN \
-wint WINT -wext WEXT -wck WRCK -name wrap
```

- The following example traces through the combinational logic in the fanout of the input ports and in the fanin of the output ports and includes the core flops on the boundary as the test flops in the shared wrapper cells. For shared wrapper cells, the `select_cfi` control signal for input bounding cells (`WINT` + `WSEN_in`) and for output bounding cells (`WEXT` + `WSEN_out`) will be built locally for each shared wrapper cell. The TCL variable `skipPorts` is created by the command and will hold the set of ports which are not wrapped. These ports include test ports, wrapper control ports and ports which are formally excluded from wrapping:

```
set inPorts [get_db ports * -if {.direction==in}]

set outPorts [get_db ports * -if {.direction==out}]

add_core_wrapper_cell -locations "$inPorts $outPorts" -wsen_in WSEN \
-wsen_out WSEN -wint WINT -wext WEXT -name wrap \
-shared_through combinational -skip_locations_variable skipPorts
```

- The following example builds the `select_cfi` control for input bounding and output bounding shared wrapper cells once, and uses the inserted logic to control the `scan_enable` pin of the shared wrapper cells. You should continue to specify the controls for "`-wsen_in`, `-wsen_out`, `-wint`, `-wext`" in case the tool needs to insert dedicated wrapper cells, and for hold mux control for shared wrapper cells. Additionally, the mux logic in the wrapper cells will be mapped to binary muxes as opposed to being built using discrete logic gates:

```
add_core_wrapper_cell -locations "$inPorts $outPorts" -wsen_in WSEN \
-wsen_out WSEN -wint WINT -wext WEXT -name wrap -shared_through combinational \
-skip_locations_variable skipPorts -map_to_mux_for_dedicated_wrapper \
-map_to_mux_for_shared_wrapper_cell -wsen_reuse_or_for_shared_wrapper_cell
```

- The following example specifies that the `select_cfi` control signals to be used for input bounding shared wrapper cells (`WINT` + `WSEN_in`) and for output bounding shared wrapper cells (`WEXT` + `WSEN_out`) have already been built in the user RTL. The internal pins will be used for shared wrapper cell insertion instead of building the control signals locally. You should continue to specify the controls for "`-wsen_in`, `-wsen_out`, `-wint`, `-wext`" in case the tool needs to insert dedicated wrapper cells, and for hold mux control for shared wrapper cells. If dedicated wrapper cells are inserted, the tool will select a local test clock to be used for the dedicated wrapper cell:

```
add_core_wrapper_cell -locations "$inPorts $outPorts" -wsen_in WSEN \
-wsen_out WSEN -wint WINT -wext WEXT -name wrap -shared_through combinational \
-skip_locations_variable skipPorts -map_to_mux_for_dedicated_wrapper \
-map_to_mux_for_shared_wrapper_cell \
-input_select_cfi <InternalDrivingPinCFI> \
-output_select_cfi <InternalDrivingPinCFO>
```

- The following example specifies the set of functional ports and ports involved in combinational logic paths to be excluded from wrapping. Additionally, the input and output thresholds used to direct the tool to insert dedicated wrapper cells are also specified; where the default tool thresholds are set to 10.

```
add_core_wrapper_cell -locations "$inPorts $outPorts" -wsen_in WSEN \
-wsen_out WSEN -wint WINT -wext WEXT -name wrap -shared_through combinational \
-skip_locations_variable skipPorts -wsen_reuse_or_for_shared_wrapper_cell \
```

```
-map_to_mux_for_dedicated_wrapper -map_to_mux_for_shared_wrapper_cell \
-exclude "iso*  ret*" -exclude_comb_feedthru_paths \
-input_shared_threshold  15 -output_shared_threshold 15
```

- The following example specifies to insert the shared wrapper cells without the built-in hold mux. You can use this approach for delay test when the wrappers will be shifting at-speed. You should continue to specify the controls for "`-wsen_in`, `-wsen_out`, `-wint`, `-wext`" in case the tool needs to insert dedicated wrapper cells:

```
add_core_wrapper_cell -locations "$inPorts $outPorts" -wsen_in WSEN \
-wsen_out WSEN -wint WINT -wext WEXT -name wrap -shared_through combinational \
-skip_locations_variable skipPorts -wsen_reuse_or_for_shared_wrapper_cell \
-exclude "iso*  ret*" -exclude_comb_feedthru_paths \
-input_shared_threshold  15 -output_shared_threshold 15 \
-no_mux_before_shared_cell
```

Related Information

Affects this command:

connect_scan_chains

Sets this attribute:

core_wrapper

add_hard_repair

```
add_hard_repair

-config_file config_file
[-connect_to_jtag | -dont_create_hri_ports |-direct_access_only [-dont_create_hri_ports]]
[-dont_scan]
[-alt_dft_cfg_mode mode]
[-dft_cfg_mode mode]
[-allow_unused_instructions]
[-schedule_instruction string]
[-check_instruction string]
[-test_plan_instruction string]
[-constraint_instruction string]
[-module_prefix string]
[-periods string]
[-preview]
[-select_pin]
[-write_pin]
[-fuse_cif_wrapper string]
[-repair_instruction string]
[-directory directory]
[design]
```

Description

Inserts hard repair for the PMBIST fuse control unit (FCU) for memories based on the repair channel definitions specified in the configuration file (specified with the `-config_file` option).

Support is currently limited to block-level repair channel binding.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-allow_unused_instructions`	
	Enables the creation of all possible `jtag_instruction_decode*` ports for PMBIST instructions for the block being processed by this command. This option assures a common interface on the block (independent of the functions enabled within this block).
`-alt_dft_cfg_mode` *mode*	
	Specifies the configuration mode in which the PMBIST FCU will be operating for non-JTAG mode of operation.
	PMBIST FCU test pin verification must be done in this mode as well.
	Default: `pmbist`
`-check_instruction` *string*	
	Specifies the `FCUCHK` instruction that must be loaded into the IEEE 1149.x TAP controller to check the results of PMBIST FCU execution.
	Default: `FCUCHK`
`-config_file` *config_file*	
	Specifies the file that contains the user-defined configuration parameters which selects the PMBIST repair channel features and controls for the insertion of the FCU for targeted non-volatile memories.
`-connect_to_jtag`	Connects PMBIST logic's JTAG interface pins to a pre-instantiated JTAG macro (either from Cadence or a Third party).
	Use this option when the design is the top level of the chip, and you have instantiated the JTAG macro and will use it to access the PMBIST FCU engines.
`-constraint_instruction` *string*	
	Specifies the `FCUAMR` instruction that must be loaded into the IEEE 1149.x TAP controller to access the FCU programmable algorithm memory and test conditions.
	Default: `FCUAMR`
design	Specifies the name of the top-level design.
`-dft_cfg_mode` *mode*	
	Specifies the configuration mode in which the PMBIST FCU will be operating.
	PMBIST FCU test pin verification must be done in this mode as well.
	Default: `pmbist`
`-direct_access_only`	Specifies whether to insert PMBIST FCU without the support from the JTAG Macro. The memory repairs are controlled and examined directly from the PMBIST FCU direct access ports or pins without using the JTAG macro. In this case, any information for the JTAG macro is ignored even when a JTAG macro is specified in the design.
`-directory` *directory*	
	Specifies the directory in which PMBIST must create all intermediate FCU files for its internal use.
	Default: *current_working_directory*`/hri_`*design*
`-module_prefix`	Specifies an additional character string appended to the default prefix, 'tem', placed on all modules created and inserted by this command.
`-dont_create_hri_ports`	
	Prevents creation of PMBIST FCU control ports for subsequent JTAG macro and direct access connection at the top-level instance.
	Use this option when the instance is the top level of the chip, and you have not yet instantiated the JTAG macro, or you use the direct access method and these ports were predefined.
`-dont_scan`	Allows the insertion of non-scannable PMBIST FCU logic.
	Use this option if ATPG is not required for the PMBIST FCU logic or for the design. This implies that test signals do not need to be specified for the PMBIST FCU logic.

`fuse_cif_wrapper` *`string`*	Specifies the name of common hard repair interface for which the testplans need to be generated.
`-periods` *`string`*	Specifies the list of fcu clock periods (ps) for which the fuse cell algorithms/testplans needs to be generated.
`-preview`	Specifies to generate only the configuration file.
`-select_pin` *`string`*	Specifies the name of the select pin to be used in generating common interface algorithms.
`-write_pin` *`string`*	Specifies the name of the write pin to be used in generating common interface algorithms.
`-repair_instruction` *`string`*	
	Specifies the JTAG instruction that must be loaded into the IEEE 1149.x TAP controller for repair register access.
	Default: `FCURR`
`-schedule_instruction` *`string`*	
	Specifies the `FCUSCH` instruction that must be loaded into the IEEE 1149.x TAP controller to schedule repair channels for PMBIST FCU execution.
	Default: `FCUSCH`
`-test_plan_instruction` *`string`*	
	Specifies the `FCUTPN` instruction that must be loaded into the IEEE 1149.x TAP controller for test plan selection during PMBIST FCU execution.
	Default: `FCUTPN`

Example

The following command inserts the PMBIST FCU logic as described in the configuration file, by default, to the design module at the highest level of hierarchy of the design.

```
add_hard_repair -config_file ../hri_inputs/my_hri_configuration.txt
```

Related Information

Affected by these constraints:	define_dft_cfg_mode
	define_mbist_clock
	define_test_signal -function test_mode
Affects these commands:	add_jtag_boundary_scan
	add_jtag_macro
Related commands:	add_pmbist
	read_pmbist_memory_view
	read_dft_pmbist_interface_files
	write_dft_pmbist_interface_files
	write_dft_pmbist_testbench
Affected by this attribute:	pmbist_use
Related attributes:	mbist_enable_shared_library_domain_set
	pmbist_enable_multiple_views
	pmbist_instruction_set
	memory_data_bit_structure Attributes
	memory_lib_cell Attributes
	memory_lib_pin_action Attributes
	memory_lib_pin_alias Attributes
	memory_spare_column Attributes
	memory_spare_column_map_address Attributes
	memory_spare_column_map_data Attributes
	memory_spare_row Attributes
	memory_spare_row_map_address Attributes
	write_mask_bit Attributes

add_jtag_boundary_scan

```
add_jtag_boundary_scan [-design design]
[-comp_enables_high port [port]...]
[-comp_enables_low port [port]...]
[-exclude_ports port [port]...]
[-functional_clocks port [port]...]
[-custom_cell_directory string]
[-boundary_cell_location {inst|hinst|design}] [-jtag_macro_location {inst|hinst}] [-generate_wrck] [-no_trst_test_signal] [-pinmap_file file | -physical] [-power_on_reset {pin|hpin|port}] [-tck port] [-tdi port] [-tdo port] [-tms port] [-trst port] [-dont_map] [-preview] [-preserve_tdo_connection] [-skip_checking_tap_port_iocell]
```

Description

Inserts boundary scan cells and the corresponding JTAG Macro (if it is not yet instantiated in the design).

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-boundary_cell_location {`*`inst`*`|`*`hinst`*`|`*`design`*`}`	
	Specifies the instance in which to insert the boundary scan cells. By default, the boundary scan cells are inserted in the same hierarchy as their respective pad cells in the design.
`-comp_enables_high (-comp_enables_low)` *`port...`*	
	Specifies that the compliance value for the specified ports is active high (low) during functional mode. The compliance enable value is the value that a test port (test mode, shift enable) is tied to during the functional mode of the chip.
	The ports with their compliance value are added to a BSDL `COMPLIANCE_PATTERNS` statement.
`-custom_cell_directory` *`string`*	
	Specifies the path to the directory that contains the files describing the custom boundary cells. Each cell must be described as a Verilog module in its own file. The basename of the file must match the name of the cell in the module description.
`-design` *`design`*	Specifies the name of the design in which you want to insert boundary scan logic. This option is required if you have loaded multiple designs.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_map`	Prevents the inserted boundary scan logic from being mapped to technology gates even if the design is already mapped.
`-exclude_ports` *`ports`*	
	Excludes the specified ports from being considered for boundary scan logic insertion.
`-functional_clocks` *`ports`*	
	Specifies the ports that are seen as clocks for ATPG. These ports include
	• async set and reset ports
	• clocks used in functional mode, but not in scan shift mode
`-jtag_macro_location {`*`inst`*`|`*`hinst`*`}`	
	Specifies a hierarchical instance into which to insert the JTAG macro. By default, the JTAG macro is inserted in the top-level design.
`-generate_wrck`	Creates a `JTAG_WRCK` output pin on the JTAG macro. This output port gates the test clock of the JTAG macro (TCK) for all JTAG FSM states other than `Run-Test-Idle`, `CAPTUREDR`, `SHIFTDR`, or `UPDATEDR`.
	The `JTAG_WRCK` pin can be used to facilitate interfacing an IEEE 1500 wrapper serial control (WSC) bus to a JTAG macro.
`-no_trst_test_signal`	
	Prevents defining an active-low test signal on the TRST port and causes a formal verification constraint with a value of '0' to be added for the TRST port for the revised design.
`-physical`	Specifies to build the boundary scan register using physical information to minimize its wire length.
	The physical placement information is obtained from the DEF file read in with the `read_def` command
	This option is mutually exclusive to the `-pinmap` option.
`-pinmap_file` *`file`*	Specifies the name of the file containing the mapping between the design ports and the actual package pins. This mapping is also used to determine the boundary scan order.
	Refer to Pinmap File Format for more information.

	This option is mutually exclusive to the `-physical` option.
`-power_on_reset {`*`pin`* `|` *`hpin`* `|` *`port`*`}`	
	Specifies the power-on-reset pin which will be connected to the `JTAG_POR` input pin on the `JTAG_Macro` subdesign. This connection is made when the boundary scan logic is inserted in the design.
	If the power-on-reset signal is applied to a top-level port, you also need to exclude this port from inclusion into the boundary scan register using the `-exclude_ports` option.
`-preview`	Shows the potential changes, without making any modifications to the netlist.
`-preserve_tdo_connection`	
`-tck` *`port`*	Specifies the port name of the driver for the test clock of the JTAG macro. Specify this option when the existing JTAG port does not use the standard name.
`-skip_checking_tap_port_iocell`	Preserves the existing net connection to from-core and tristate enable pins of the TDO pad cell. If you do not specify this option, the existing net connections will be broken and new net connections will be made during boundary scan insertion from the `JTAG_TDO` and `JTAG_ENABLE_TDO` pins to the from-core and tristate enable pins of the TDO pad cell, respectively. If you preserve the TDO connections, such that the net is driven by user logic other than a pre-instantiated JTAG macro, boundary scan insertion will insert a JTAG macro and leave its `JTAG_TDO` pin unconnected in the netlist. Prevents the tool from checking for the pad cell on TAP ports in case of block-level insertion, *Default*: `TCK`
	Default: `TDI`
`-tdo` *`port`*	Specifies the port name of the test data (scan) output of the JTAG macro. Specify this option when the existing JTAG port does not use the standard name.
	An existing TDO port must have a tristate I/O pad.
	Default: `TDO`
`-tms` *`port`*	Specifies the port name of the test mode select input of the JTAG macro. Specify this option when the existing JTAG port does not use the standard name.
	Default: `TMS`
`-trst` *`port`*	Specifies the port name of the (asynchronous) test reset of the JTAG macro. Specify this option when the existing JTAG port does not use the standard name.
	Default: `TRST`
`-tdi` *`port`*	Specifies the port name of the driver for the test data (scan) input of the JTAG macro. Specify this option when the existing JTAG port does not use the standard name.

Related Information

See the following sections in *Genus Design for Test Guide*: • Inserting Boundary-Scan Logic • Physical Boundary-Scan Insertion • JTAG-Controlled Scan Modes	
Related constraints:	define_jtag_instruction_register
	define_jtag_macro
	define_test_clock
	define_test_signal -function <shift_enable | test_mode>
Affects these commands:	compress_scan_chains
	add_ptam
Sets these attributes:	boundary_type
	jtag_port Attributes

add_jtag_macro

```
add_jtag_macro
[-boundary_type {IEEE_11491|IEEE_11496}]
[-dont_map]
[-jtag_macro_location {inst|hinst}] [-generate_wrck] [-add_without_pad_logic] [-create_ports] [-dont_create_dft_tdo_enable_port]] [-preserve_tdo_connection] [-tck port] [-tdi port] [-tdo port] [-tms port] [-trst port] [-no_trst_test_signal] [-power_on_reset {pin|hpin|port}] [-design design]
```

Description

Inserts a JTAG Macro (if it is not already instantiated in the design).

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-add_without_pad_logic`	
	Specifies whether to insert the JTAG Macro into a design that does not have pad logic.
	Unless you specify the `-dont_create_dft_tdo_enable_port` option, an additional port, *prefix*`TDO_ENABLE` will be created for the enable signal used to control the tristate pin of the top-level TDO pad.
`-boundary_type {IEEE_11491 | IEEE_11496}`	
	Specifies the boundary scan architecture to use for the inserted JTAG Macro.
	Default: `IEEE_11491`
`-create_ports`	Specifies whether to create the TAP ports if they do not exist.
	If you do not specify already existing TAP signals using the `-tdi`, `-tdo`, `-tms`, `-trst`, and `-tck` options, the missing ports are created and named as *prefix*`tdi`, *prefix*`tdo`, *prefix*`tms`, *prefix*`trst`, and *prefix*`tck`, where *prefix* is the value of the `dft_prefix` root attribute.

	Use this option with the `-add_without_pad_logic` option.
`-design` *design*	Specifies the name of the design in which you want to insert the JTAG Macro. This option is required if you have loaded multiple designs.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_create_dft_tdo_enable_port`	
	Prevents the creation of the block-level *prefix*`TDO_ENABLE` port which controls the tristate enable pin of the top-level TDO pad.
	As a result, the `JTAG_MODULE/JTAG_ENABLE_TDO` output pin will be left unconnected in the netlist. You will need to connect this internal output pin to the appropriate logic to control the three-state enable pin of the JTAG TDO pad to pass boundary scan verification.
	Use this option with the `-add_without_pad_logic` option.
`-dont_map`	Prevents the inserted JTAG Macro from being mapped to technology gates even if the design is already mapped.
`-generate_wrck`	Creates a `JTAG_WRCK` output pin on the JTAG macro. This output port gates the test clock of the JTAG macro (TCK) for all JTAG FSM states other than `Run-Test-Idle`, `CAPTUREDR`, `SHIFTDR`, or `UPDATEDR`.
	The `JTAG_WRCK` pin can be used to facilitate interfacing an IEEE 1500 wrapper serial control (WSC) bus to a JTAG macro.
`-jtag_macro_location` *inst*|*hinst*	
	Specifies a hierarchical instance in which to insert the JTAG Macro.
`-no_trst_test_signal`	
	Prevents defining an active-low test signal on the TRST port and causes a formal verification constraint with a value of '0' to be added for the TRST port for the revised design.
`-power_on_reset` {*pin* | *hpin* | *port*}	
	Specifies the power-on-reset pin name.
`-preserve_tdo_connection`	
	Preserves the existing net connection to from-core and tristate enable pins of the TDO pad cell. If you do not specify this option, the existing net connections will be broken and new net connections will be made from the JTAG Macro `JTAG_TDO` and `JTAG_ENABLE_TDO` pins to the from-core and tristate enable pins of the TDO pad cell, respectively. If you preserve the TDO connections, such that the net is driven by user logic other than the JTAG Macro, the `JTAG MacroJTAG_TDO` pin will be left unconnected in the netlist.
`-tck` *port*	Specifies the port name of the driver for the test clock of the JTAG Macro. Specify this option when the existing JTAG port does not use the standard name. Default: TCK
`-tdi` *port*	Specifies the port name of the driver for the test data (scan) input of the JTAG Macro. Specify this option when the existing JTAG port does not use the standard name.
	Default: `TDI`
`-tdo` *port*	Specifies the port name of the test data (scan) output of the JTAG Macro. Specify this option when the existing JTAG port does not use the standard name.
	An existing TDO port must have a tristate I/O pad.
	Default: `TDO`

`-tms` *port*	Specifies the port name of the test mode select input of the JTAG Macro. Specify this option when the existing JTAG port does not use the standard name.
	Default: `TMS`
`-trst` *port*	Specifies the port name of the (asynchronous) test reset of the JTAG Macro. Specify this option when the existing JTAG port does not use the standard name.
	Default: `TRST`

Examples

- The following example inserts the JTAG Macro into the top-level netlist, creates TAP ports on the top-level netlist, and preserves the existing net connection to from-core and tristate enable pins of the TDO pad cell.

  ```
  add_jtag_macro -create_ports design_name -preseve_tdo_connection \ -add_without_pad_logic
  ```

- The following example inserts the JTAG Macro into a specified hierarchical instance and connects it to TAP ports without pad logic.

  ```
  add_jtag_macro -jtag_macro_location instance_level design_name \ -add_without_pad_logic
  ```

Related Information

See the following sections in *Genus Design for Test Guide*: • Inserting a JTAG Macro • JTAG-Controlled Scan Modes	
Related constraints:	define_jtag_instruction
	define_jtag_instruction_register
Affects these commands:	compress_scan_chains
	add_ptam
Sets these attributes:	jtag_port Attributes

add_lbist

```
add_lbist
-lbist_clock port [-lbist_clock_hookup {pin|hpin}]
[-lbist_clock_osc_period integer]
[-lbist_clock_ref_period  integer]
[ -scan_clock port -test_control test_signal
  [-scan_clock_hookup {pin|hpin}]
| -dont_scan_lbist_flops]
[-inside {inst|hinst}] [-tester_controlled_pins port...] [-free_running_clocks test_clock...] [-control_inputs_to_0 port...] [-control_inputs_to_1
port...] [-direct_access] [-programmable_direct_access] [-programmable_direct_access_parallel_load_enable port] [-direct_reset port] [-direct_enable port] [-
direct_reset_negedge] [-direct_done_output port] [-direct_pass_output port] [-direct_fail_output port] [-scan_patterns integer] [-scan_pattern_counter_length
integer  [-scan_pattern_counter_parallel_load_pins {pin|port|hpin}...]] [-set_patterns integer] [-set_pattern_counter_length integer [-
set_pattern_counter_parallel_load_pins {pin|port|hpin}...]] [-async_set_signals test_signal] [-reset_patterns integer] [-reset_pattern_counter_length integ
er  [-reset_pattern_counter_parallel_load_pins {pin|port|hpin}...]] [-async_reset_signals test_signal] [-static_patterns integer] [-
static_pattern_counter_length integer  [-static_pattern_counter_parallel_load_pins {pin|port|hpin}...]] [-dynamic_patterns integer] [-
dynamic_pattern_counter_length integer  [-dynamic_pattern_counter_parallel_load_pins {pin|port|hpin}...]] [-scan_channel_counter_length integer  [-
scan_channel_counter_parallel_load_pins {pin|port|hpin}...]] [-scan_window integer] [-scan_window_counter_length integer  [-
scan_window_counter_parallel_load_pins {pin|port|hpin}...]] [-scan_window_pulse_value integer] [-scan_enable_delay integer] [-
scan_enable_delay_counter_length integer  [-scan_enable_delay_counter_parallel_load_pins {pin|port|hpin}...]] [-capture_window integer] [-
capture_window_counter_length integer  [-capture_window_counter_parallel_load_pins {pin|port|hpin}...]] [-capture_window_capture_value integer] [-
capture_window_launch_value integer] [-set_reset_test_window integer] [-set_reset_pulse_width integer] [-capture_clock_stagger_ratio integer] [-
scan_clock_stagger_ratio integer][-add_masking] [-parallel_load_mask [-mask_parallel_load_pins {pin|port|hpin}...]] [-parallel_load_misr [-
misr_parallel_load_pins {pin|port|hpin}...] [-misr_parallel_unload_pins {pin|port}]] [-parallel_load_control_bits [-control_bits_parallel_load_pins
{pin|port|hpin}...] [-parallel_load_prgp [-prgp_parallel_load_pins {pin|port|hpin}...]] [-auto_create_programmable_direct_access_parallel_ports] [-
dont_use_opcg] [-preview] [design]
```

Description

Inserts Logic Bist (LBIST) logic into the design.

When the length of any LBIST counter is explicitly specified using the corresponding `*_counter_length` option, that counter will be programmable in `SETBIST` and programmable direct-access LBIST. Otherwise it will always be internally initialized to its default value, same as for `RUNBIST` and (non-programmable) direct-access.

If internal pins are used to drive the programmable direct-access LBIST parallel load interface or the MISR parallel unload interface is either left unconnected or is connected to internal pins, the LBIST simulation environment that is generated by default will not work. You will need to use your own simulation environment where both LBIST as well as the proprietary parallel interface used to operate it, must be functional

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-add_masking`	Inserts the LBIST macro including the optional channel masking logic (only available with SETBIST support or the programmable direct-access LBIST support).
`-async_set_signals test_signal`	
	Specifies the list of asynchronous set signals that are controlled during set testing.
`-async_reset_signals test_signal`	
	Specifies the list of asynchronous reset signals that are controlled during reset testing.
`-auto_create_programmable_direct_access_parallel_ports`	
	Creates ports to connect to all pins of the programmable direct-access parallel load and unload interfaces of the LBIST macro for which no pins have been specified through the corresponding option.
`-capture_clock_stagger_ratio integer`	

	Controls staggering of the capture clocks. The specified value determines the number of bistck pulses after which the capture clock is staggered. Specify 0, 1, 2, 4 or 8. If you specify 0, the capture clocks will not be staggered.
`-capture_window` *integer*	
	Specifies the default value of the capture window.
`-capture_window_capture_value` *integer*	
	Specifies the value of the capture window (down) counter at which the (first) capture clock pulse is issued.
`-capture_window_counter_length` *integer*	
	Specifies the length of the capture window counter that is part of the LBIST macro.
`-capture_window_counter_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the capture window counter.
	This option must be specified together with the `-capture_window_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-capture_window_launch_value` *integer*	
	Specifies the value of the capture window (down) counter at which the launch clock pulse is issued.
`-control_bits_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the LBIST control bits.
	This option must be specified together with the `-parallel_load_control_bits` option.
	If omitted, constants are used to initialize the LBIST control bits to their default values, unless port auto-creation is also selected.
`-control_inputs_to_0` *port_list*	
	Specifies the list of ports that must be controlled to a logic zero value during LBIST mode.
	Use this option if no boundary scan cell or wrapper cell is inserted on these ports.
`-control_inputs_to_1` *port_list*	
	Specifies the list of ports that must be controlled to a logic one value during LBIST mode.
	Use this option if no boundary scan cell or wrapper cell is inserted on these ports.
design	Inserts the LBIST logic in the specified top-level design.
`-direct_access`	Specifies whether to insert LBIST without the support from the JTAG Macro.
`-direct_done_output` *port*	
	Specifies the top-level port to use to monitor LBIST completion.
`-direct_enable` *port*	
	Specifies a dedicated top-level port used to enable the direct-access LBIST. This option is required for the direct access support.
`-direct_fail_output` *port*	
	Specifies the top-level port used to monitor if LBIST fails.
`-direct_pass_output` *port*	
	Specifies the top-level port used to monitor if LBIST passes.
`-direct_reset` *port*	Specifies a dedicated top-level port used to reset the direct-access LBIST. This option is required for the direct access support.
`-direct_reset_negedge`	
	Specifies whether the reset signal for the direct-access LBIST is active low.

`-dont_scan_lbist_flops`	
	Inserts the LBIST macro without converting its flops to scan flops, connecting them in scan chains and prepending them to the full scan chains.
`-dont_use_opcg`	Inserts the LBIST macro without using the inserted OPCG logic.
`-dynamic_pattern_counter_length` *integer*	
	Specifies the length of the dynamic pattern counter that is part of the LBIST macro.
`-dynamic_pattern_counter_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the dynamic pattern counter.
	This option must be specified together with the `-dynamic_pattern_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-dynamic_patterns` *integer*	
	Specifies the default number of dynamic set test patterns to be executed.
`-free_running_clocks` *test_clock_list*	
	Specifies the free running test clocks in the design. These clocks are intercepted in LBIST mode by special mux structures to prevent clock glitches.
`-inside` {*inst*|*hinst*}	
	Specifies the instance in which to instantiate the LBIST logic.
	By default, the LBIST logic is inserted as a hierarchical instance in the top-level of the design.
`-lbist_clock` *port*	Specifies the port to which the LBIST oscillator clock is applied. You can specify either a top-level dedicated port or an oscillator reference clock.
`-lbist_clock_hookup` {*pin*|*hpin*}	
	Specifies the hookup pin for the LBIST oscillator clock.
	Usually the PLL output is used as hookup for the LBIST oscillator clock.
`-lbist_clock_osc_period` *integer*	
	Specifies the period of the LBIST oscillator clock in ps.
`-lbist_clock_ref_period` *integer*	
	Specifies the period of the LBIST reference clock in ps.
`-mask_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the mask.
	This option can only be specified together with the `-add_masking` and `-parallel_load_mask` options.
	By default, constants will be used to initialize the mask to its default value, unless port auto-creation is also selected.
`-misr_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the MISR.
	This option must be specified together with the `-parallel_load_misr` option.
	By default, constants will be used to initialize the MISR to its default value, unless port auto-creation is also selected.
`-misr_parallel_unload_pins` {*pin*|*port*|*hpin*}	

	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel unload interface of the LBIST macro to observe the MISR signature.
	By default, the MISR outputs will be left unconnected for the user to connect, unless port auto-creation is also selected.
`-parallel_load_control_bits`	
	Inserts programmable direct-access LBIST with a parallel load interface for the LBIST control bits.
	By default, the LBIST control bits will be internally initialized to their default values.
`-parallel_load_mask`	
	Inserts programmable direct-access LBIST with a parallel load interface for the mask.
	Specify this option with the `-add_masking` option.
	By default, the mask will be internally initialized to its default value (all 1's) which allows all channels to be observed by the MISR.
`-parallel_load_misr`	
	Inserts programmable direct-access LBIST with a parallel load interface for the MISR.
	By default, the MISR will be internally initialized to its default value (all 0's).
`-parallel_load_prpg`	
	Inserts programmable direct-access LBIST with a parallel load interface for the PRPG.
	If omitted, the PRPG will be internally initialized to its default value (all 1's).
`-preview`	Shows the potential changes without making any modifications to the netlist.
`-programmable_direct_access`	
	Provides programmable direct access support.
`-programmable_direct_access_parallel_load_enable`	
	Specifies the programmable direct access parallel load enable port.
`-prpg_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the PRPG.
	This option can only be specified together with option '-parallel_load_prpg'.
	By default, constants will be used to initialize the PRPG to its default value, unless port auto-creation is also selected.
`-reset_patterns` *integer*	
	Specifies the default number of reset test patterns to be executed.
`-reset_pattern_counter_length` *integer*	
	Specifies the length of the reset pattern counter that is part of the LBIST macro.
`-reset_pattern_counter_parallel_load_pins` {*pin*|*port*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the reset pattern counter.
	This option must be specified together with the `-reset_pattern_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-scan_channel_counter_length` *integer*	
	Specifies the length of the scan channel counter that is part of the LBIST macro.

`-scan_channel_counter_parallel_load_pins` {*pin*|*port*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the scan channel counter.
	This option must be specified together with the `-scan_channel_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-scan_clock` *port*	Specifies the port to which the test clock is applied that drives the LBIST flops when they are prepended to full scan chains.
`-scan_clock_hookup` {*pin*|*hpin*}	
	Specifies the hookup pin for the test clock that drives the LBIST flops when they are prepended to the full scan chains.
`-scan_clock_stagger_ratio` *integer*	
	Controls staggering of the scan clocks. The specified value determines the number of bistck pulses after which the scan clock is staggered. Specify 0, 1, 2, 4 or 8. If you specify 0, the scan clocks will not be staggered.
`-scan_enable_delay` *integer*	
	Specifies the default delay after the scan enable signal toggles.
`-scan_enable_delay_counter_length` *integer*	
	Specifies the length of the scan enable delay counter that is part of the LBIST macro.
`-scan_enable_delay_counter_parallel_load_pins` {*pin*|*port*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the scan enable delay counter.
	This option must be specified together with the `-scan_enable_delay_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-scan_pattern_counter_length` *integer*	
	Specifies the length of the scan pattern counter that is part of the LBIST macro.
`-scan_pattern_counter_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the scan pattern counter.
	This option must be specified together with the `-scan_pattern_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-scan_patterns` *integer*	
	Specifies the default number of scan test patterns to be executed. The specified integer must be a multiple of 64.
`-scan_window` *integer*	
	Specifies the default value of the scan window.
`-scan_window_counter_length` *integer*	
	Specifies the length of the scan window counter that is part of the LBIST macro.
`-scan_window_counter_parallel_load_pins` {*pin*|*port*|*hpin*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the scan window counter.
	This option must be specified together with the `-scan_window_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-scan_window_pulse_value` *integer*	

	Specifies the value of the scan window (down) counter at which the (first) scan clock pulse is issued.
`-set_pattern_counter_length` *integer*	
	Specifies the length of the set pattern counter that is part of the LBIST macro.
`-set_pattern_counter_parallel_load_pins` {*pin*|*port*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the set pattern counter.
	This option must be specified together with the `-set_pattern_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-set_patterns` *integer*	
	Specifies the default number of set test patterns to be executed.
`-set_reset_pulse_width` *integer*	
	Specifies the width of the asynchronous set/reset pulse for the set/reset tests.
`-set_reset_test_window` *integer*	
	Specifies the value of the test window for the asynchronous set/reset tests.
`-static_pattern_counter_length` *integer*	
	Specifies the length of the static pattern counter that is part of the LBIST macro.
`-static_pattern_counter_parallel_load_pins` {*pin*|*port*}	
	Specifies the ports or pins (from MSB to LSB) to be connected to the programmable direct-access parallel load interface of the LBIST macro to initialize the static pattern counter.
	This option must be specified together with the `-static_pattern_counter_length` option.
	If omitted, constants are used to initialize LBIST to the specified default value, unless port auto-creation is also selected.
`-static_patterns` *integer*	
	Specifies the default number of static set test patterns to be executed.
`-test_control` *test_signal*	
	Specifies the test signal that, when active, brings the LBIST logic into its ATPG test mode in order to test it.
`-tester_controlled_pins` *port*	
	Specifies the ports that are available on the tester and that therefore can be skipped for test point insertion. These ports are written out to the pinassign file if defined as test signal and test clocks.

Related Information

Inserting LBIST Logic in *Genus Design for Test Guide*.	
Affected by these constraints:	define_test_signal -function test_mode
Related command:	write_dft_lbist_macro

add_lockup_element

```
add_lockup_element
actual_scan_chain [actual_scan_chain]...
[-terminal_lockup]
```

Description

Inserts a lockup element as needed in the specified analyzed scan chains.

Analyzed scan chains are chains whose connectivity is traced by the Genus-DFT engine when you define them using the `-analyze` option of the `define_scan_chain` command.

Use this command on mapped netlists whose existing (configured) scan chains were either created by hand or using a third-party DFT insertion tool.

> The Genus-DFT engine determines the type of lockup element to be inserted from the value of the `dft_lockup_element_type` design attribute.

Options and Arguments

actual_scan_chain	Specifies the name of an analyzed scan chain.
`-terminal_lockup`	Allows to add a terminal lockup element to the specified analyzed scan chains.

Example

The following example inserts lockup elements as needed in all analyzed scan chains.

```
get_db actual_scan_chains * -if {.analyzed=="true"} -foreach \ {add_lockup_element $object}
```

Related Information

Analyzing Chains in a Scan-Connected Netlist in *Genus Design for Test Guide*

Affected by this attribute:	dft_lockup_element_type

add_los_pipeline

```
add_los_pipeline
[-fanout_limit integer] [-location {inst|hinst|design}] [-los_mode test_signal] [design] [-scan_clock test_clock] [-scannable] [-
keep_connected_shift_enable]
```

Description

Inserts launch-off-shift pipelined logic to flop's shift enable.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

design	Specifies the name of the design for which to insert pipeline logic.
	This option is only required if multiple designs are loaded.
	If there is only one top-level design, you can omit the design name. In this case, the tool will use the top-level design of the design hierarchy.
-fanout_limit *integer*	
	Specifies the maximum number of flops driven by each launch-off-shift pipeline.
	The default value for this is all the scan elements that share common test-clock, clock polarity, shift-enable and scan enable pin polarity.
-keep_connected_shift_enable	
	Specifies to retain any existing connection to shift enable signals.
-location {*inst*|*hinst*|*design*}	
	Specifies the location to place the pipeline logic.
	The default location of each pipeline instance is the logical parent hierarchy common to all the scan elements associated with this pipeline instance.
-los_mode *test_signal*	
	Specifies the launch-off-shift specific test signal.
	The test signal must have been previously defined by a `define_test_signal -function test_mode` constraint and must have its function specified as `opcg_los_enable`.
	If there is only one test signal with function specified as `opcg_los_enable`, you can omit this option.
-scannable	Specifies that the pipeline flops are scannable.
-scan_clock *test_clock*	
	Specifies to pipeline flops and segments using this scan clock.

Example

```
add_los_pipeline -fanout_limit 1000 -los_mode los_mode_enable
```

Related Information

Related commands:	define_test_signal -function <shift_enable | test_mode>
Related attributes:	dft_exclude_flop_from_los_pipeline
	dft_exclude_abstract_segment_from_los_pipeline

add_opcg

```
add_opcg
-opcg_load_clock test_clock
-scan_clock test_clock
-test_enable test_signal
[-opcg_enable test_signal]
[-preview] [design]
```

Description

Inserts OPCG logic that generates on-chip launch and capture clocks for testing of at-speed delay defects.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-design` *design*	Specifies the name of the design for which to insert OPCG logic.
	This option is only required if multiple designs are loaded.
	If there is only one top-level design, you can omit the design name. In this case, the tool will use the top-level design of the design hierarchy.
`-opcg_enable` *test_signal*	
	Specifies the test signal to be used to enable the OPCG mode.
	The test signal must have been previously defined by a `define_test_signal -function test_mode` constraint.
`-opcg_load_clock` *test_clock*	
	Specifies test clock that will be used to clock the side scan chains (scan chains in the OPCG logic).
	The test signal must have been previously defined by a `define_test_clock` constraint.
`-preview`	Shows the potential changes for the OPCG logic, without making any modifications to the netlist.
`-scan_clock` *test_clock*	
	Specifies the test clock that must be used for shifting the OPCG logic in fullscan mode.
	The test signal must have been previously defined by a `define_test_clock` constraint.
`-test_enable` *test_signal*	
	Specifies the test signal to be used to gate the OPCG enable.
	The test signal must have been previously defined by a `define_test_signal -function test_mode` constraint.

Example

```
add_opcg -opcg_enable OPCGMODE  -opcg_load_clock opcg_load_clk \-scan_clock scanclk -test_enable testmode
```

Related Information

Inserting the OPCG Logic in *Genus Design for Test Guide*

Related constraints:	define_test_clock
	define_test_signal -function <shift_enable | test_mode>

add_opcg_hold_mux

```
add_opcg_hold_mux
-edge_mode test_signal
-instance {inst|hinst}[design]
```

Description

Replaces the specified domain crossing scan flop instance with its OPCG-equivalent if you specified OPCG-equivalence mappings with the `set_opcg_equivalent` command. If the OPCG-equivalency mappings are not available, or if the scan cell type of the instance does not have an entry in the OPCG-equivalency table, the command adds a hold mux before the scan cell to convert it to an OPCG-equivalent cell and makes all necessary connections.

This command is used for designs with OPCG logic when the OPCG domain blocking attribute is enabled in the flow. Domain crossing flops are identified when the scan chains are built. A domain crossing flop will have its instance level attribute '`dft_opcg_domain_crossing flop`' set to `true`.

Options and Arguments

design	Specifies the design in which you want to replace an instance.
`-edge_mode` *test_signal*	
	Specifies the global `edge_mode` signal to connect to the OPCG toggle muxes protecting domain crossings. A shift enable signal may be used as long as there are no flops that are on both functional and scan chain domain crossings.
`-instance {`*inst*`|`*hinst*`}`	
	Specifies the instance to be replaced.

Related Information

Affected by these commands:	reset_opcg_equivalent
	set_opcg_equivalent

add_pmbist

```
add_pmbist
[-config_file config_file] [-preview] [-preview_area]
[-connect_to_jtag] [-dont_create_pmbist_ports][-direct_access_only][-dont_create_pmbist_ports][-dont_scan] [-module_prefix string] [-
dont_scan_repair_registers] [-alt_dft_cfg_mode mode] [-dft_cfg_mode mode] [-allow_unused_instructions] [-schedule_instruction string] [-check_instruction stri
ng] [-test_plan_instruction string] [-constraint_instruction string] [-rom_instruction string] [-ra_solution_group_failure_rate string] [-
diagnostics_instruction string] [-rom_diagnostics_instruction string] [-redundancy_instruction string] [-rom_path string] [-rom_data_files string][-
rom_data_map_file string] [-technology integer] [-amu_location {design|module|inst|hinst}] [-directory directory_path] [design]
```

Description

Inserts Programmable Built-In-Self-Test (PMBIST) logic in the design for memories based on the definitions in the configuration file specified using the `-config_file` option.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-allow_unused_instructions`	
	Enables the creation of all possible `jtag_instruction_decode*` ports for PMBIST instructions for the block being processed by this command. This option assures a common interface on the block (independent of the functions enabled within this block).
`-alt_dft_cfg_mode` *mode*	
	Specifies the configuration mode in which PMBIST will be operating for non-JTAG mode of operation. PMBIST test pin verification must be done in this mode as well.
	Default: pmbist

-amu_location {*design*|*module*|*inst*|*hinst*}	
	Specifies the location for the algorithm memory unit which includes the programmable algorithm information.
	Default: top-level design
-check_instruction *string*	
	Specifies the MBISTCHK instruction that must be loaded into the IEEE 1149.x TAP controller to check the results of PMBIST execution.
	Default: MBISTCHK
-config_file *config_file*	
	Specifies the file that contains the user-defined configuration parameters which selects the PMBIST features and controls for the insertion of the test logic for targeted memories.
-connect_to_jtag	Connects PMBIST logic's JTAG interface pins to a pre-instantiated JTAG macro (either Cadence's or Third party).
	Use this option when the design is the top level of the chip, and you have instantiated the JTAG macro and will use it to access the PMBIST engines.
-constraint_instruction *string*	
	Specifies the MBISTAMR instruction that must be loaded into the IEEE 1149.x TAP controller to access programmable algorithm memory and test conditions.
	Default: MBISTAMR
design	Specifies the name of a top-level design.
-dft_cfg_mode *mode*	
	Specifies the configuration mode in which PMBIST will be operating. PMBIST test pin verification must be done in this mode as well.
	Default: pmbist
-diagnostics_instruction *string*	
	Specifies the JTAG instruction that must be loaded into the IEEE 1149.x TAP controller when the design contains any memory that needs to be tested using the diagnostic isolation options.
	Default: MBISTDIAG
-direct_access_only	Specifies whether to insert PMBIST without the support from the JTAG Macro. The memory testing is controlled and examined directly from the PMBIST direct access ports/pins without using the JTAG macro. In this case, any information for the JTAG macro is ignored even when a JTAG macro is specified in the design.
-directory *directory_path*	
	Specifies the directory in which PMBIST must create all intermediate files for its internal use.
	Default: *current_working_directory*/pmbist_*design*
-dont_create_pmbist_ports	
	Prevents creation of PMBIST control ports for subsequent JTAG macro and direct access connection at the top-level instance.
	Use this option when the instance is the top level of the chip, and you have not yet instantiated the JTAG macro, or you use the direct access method and these ports were predefined.
-dont_scan	Allows the insertion of non-scannable PMBIST logic.
	Use this option if ATPG is not required for the PMBIST logic or for the design. This implies that test signals do not need to be specified for the PMBIST logic.
-dont_scan_repair_registers	
	Allows repair registers to be non-scannable.
-module_prefix *string*	

	Specifies an additional character string to append to the default prefix `tem`. The string is placed on all modules created and inserted by the `add_pmbist` command.
	This option is required in case of a PMBIST block-level insertion flow.
`-preview`	Requests the creation of a configuration file template without performing insertion. The template file can be used to review configuration file content or as a basis to further edit content.
`-preview_area`	Specifies to estimate the PMBIST logic area only and not to add PMBIST logic.
`-ra_solution_group_failure_rate` *string*	
	Specifies the maximum number of memory bit-cells that can be grouped together with atmost 1-bit failure rate.
`-redundancy_instruction` *string*	
	Specifies the JTAG instruction that must be loaded into the IEEE 1149.x TAP controller for redundancy analysis.
	Default: `MBISTRAR`
`-rom_data_files` *string*	
	Specifies the list of ROM data map files located in one of the directories specified with the `-rompath` option.
	This option is especially important when hardwired testplans are used for ROM testing as the signature is hardwired into the PMBIST logic.
`-rom_data_map_file` *string*	
	Specifies the file providing mapping of ROM instances to their associated data map files.
`-rom_diagnostics_instruction` *string*	
	Specifies the JTAG instruction that must be loaded into the IEEE 1149.x TAP controller when the design contains ROMs that need to be tested using the predefined diagnostic test plan
	Default: `MBISTROMDIAG`
`-rom_instruction` *string*	
	Specifies the JTAG instruction that must be loaded into the IEEE 1149.x TAP controller when the design contains ROMs that need to be tested using programmable test plans.
	Default: `MBISTROM`
`-rom_path` *string*	Specifies a list of directory paths where the ROM data map files reside.
	This option is especially important when hardwired testplans are used for ROM testing as the signature is hardwired into the PMBIST logic.
`-schedule_instruction` *string*	
	Specifies the `MBISTSCH` instruction that must be loaded into the IEEE 1149.x TAP controller to schedule targeted memories for PMBIST execution.
	Default: `MBISTSCH`
`-technology` *integer*	
	Specifies the technology node for which configuration file template is to be generated.
`-test_plan_instruction` *string*	

	Specifies the `MBISTTPN` instruction that must be loaded into the IEEE 1149.x TAP controller for test plan selection during PMBIST execution.
	Default: `MBISTTPN`

Examples

- The following command analyzes the netlist to identify memory instances and generates a PMBIST configuration file template for this design.

  ```
  add_pmbist  -preview
  ```

- The following command inserts the PMBIST logic as described in the configuration file, by default, to the design module at the highest level of hierarchy of the design.

  ```
  add_pmbist -config_file ../et_inputs/my_configuration.txt
  ```

Related Information

Affected by these constraints:	define_dft_cfg_mode
	define_mbist_clock
	define_test_signal -function test_mode
Affects these commands:	add_jtag_boundary_scan
	add_jtag_macro
Related commands:	read_pmbist_memory_view
	read_dft_pmbist_interface_files
	write_dft_pmbist_interface_files
	write_dft_pmbist_testbench
Affected by this attribute:	pmbist_use
Related attributes:	mbist_enable_shared_library_domain_set
	pmbist_enable_multiple_views
	pmbist_instruction_set
	memory_data_bit_structure Attributes
	memory_lib_cell Attributes
	memory_lib_pin_action Attributes
	memory_lib_pin_alias Attributes
	memory_spare_column Attributes
	memory_spare_column_map_address Attributes
	memory_spare_column_map_data Attributes
	memory_spare_row Attributes
	memory_spare_row_map_address Attributes
	write_mask_bit Attributes

add_pmbist_access_method

```
add_pmbist_access_method -asu_vectors_file string
[-module_prefix string] [-alt_dft_cfg_mode dft_config_mode]
[-dont_scan] [-directory string] [-fcu_instance hinst]
[-amu_instance hinst] [design]
```

Description

Adds Programmable Built-In-Self-Test (PMBIST) capabilities to test the memories. It requires a vector file generated by `write_dft_pmbist_testbench` or `create_embedded_test` command. The access method hardware needs to be inserted at block level after PMBIST specific hardware insertion before moving on to top-level PMBIST insertion (bottom-up).

Control and monitor signals for power-on-self-test are defined using `define_pmbist_direct_access` command.

Useful Signals:

- `mda_run` - enables power-on-self-test
- `mda_run_mask` - to control what needs to be tested
- `mda_run_done`, `mda_run_ovfl` and `mda_run_fail` - to monitor the results of the testing

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-alt_dft_cfg_mode` *dft_config_mode*	
	Specifies the configuration mode in which netlist back-tracing is to be done.
	Default: pmbist_od
`-amu_instance` *hinst*	
	Specifies the name of the associated algorithm memory unit instance.
`-asu_vectors_file` *string*	
	Specifies the vector file for application specific unit generation.
design	Specifies the name of a top-level design.
`-directory` *string*	Specifies the working directory.
	Default: ./pmbist_od_<designname>
`-dont_scan`	Allows access method insertion flow without test signals when the access method flops are not scan replaced.
`-fcu_instance` *hinst*	
	Specifies the name of the associated fuse control unit instance.
`-module_prefix` *string*	
	Specifies an additional character string to append to the default prefix `tem`. The string is placed on all modules created and inserted by this command.

Related Information

Related commands:	define_pmbist_direct_access
	write_dft_pmbist_testbench

add_ptam

```
add_ptam
-instruction jtag_instruction
-power_test_enable {pin|hpin|port}
[-power_test_enable_active {low|high} ]
[-shift_enable test_signal ]
[-connect_to_jtag]
[-directory string ] [-preview]
[-dont_map ] [-dont_check_dft_rules ]
```

Description

Inserts Power Test Access Mechanism (PTAM) logic to facilitate chip power management during test.

Some files may require customization according to the setup requirements.

To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-connect_to_jtag`	
	Connects the PTAM logic to the JTAG macro instance. The JTAG macro instance must first be created using either the `add_jtag_boundary_scan` command or the `add_jtag_macro` command. If `-connect_to_jtag` is not specified and a JTAG macro instance is detected in the current session, a warning message will be issued to notify of the JTAG macro instance's existence, otherwise there will be no attempt to connect to a JTAG macro instance.
`-directory` *string*	Specifies the directory to which the mode initialization and pin assign files are written.
	Default: *current_working_directory*`/ptam`
`-dont_check_dft_rules`	
	Prevents the DFT rules from being automatically checked after PTAM insertion.
`-dont_map`	Prevents the inserted logic from being mapped even if the design is already mapped to the target library.
`-instruction` *jtag_instruction*	
	Specifies the name of the instruction which must be loaded into the IEEE 1149.x TAP controller instruction register to access the PTAM test data register.
`-power_test_enable {`*pin*`|`*hpin*`|`*port*`}`	
	Identifies the power-test-enable pin or port. Asserting this pin to an active state will enable the PTAM logic to override the design's power manager control pins. This serves as a master mode control signal.
	Note: This cannot be the same pin or port identified by `define_test_signal -function test_mode` that control pin sharing if the PTAM I/O are shared
`-power_test_enable_active {`high `|` `low`}`	
	Specifies the active value for the power-test-enable pin or port.
	Default: `high`
`-preview`	Shows the potential changes without making any modifications to the netlist.
`-shift_enable` *test_signal*	
	Designates a shift-enable test signal used to override the PTAM gating logic of the power manager output control signals during scan test.
	If you omit this option, the default shift-enable signal specified using `define_test_signal -function shift_enable` is used as selected by its `default_shift_enable` attribute.

Example

The following command inserts the PTAM logic into design `chip_top`.

```
add_ptam -instruction PTAM -power_test_enable /chip_top/PwrTe \-directory ${workdir}/testmode_data
```

Related Information

Inserting Power Test Access Mechanism (PTAM) Logic in *Genus Design for Test Guide*.	
In the *Common Power Format Language Reference*: • create_isolation_rule • create_power_domain • create_state_retention_rule	
Affected by these commands:	read_power_intent
	define_test_signal -function shift_enable

add_scan_power_gating

```
add_scan_power_gating
-max_number_of_test_points integer [-min_slack integer]
{-test_control test_signal | -preview}
[-report_virtual_scan_power [-power_mode mode]
 "clockFreq [flopTogglePercent]" -preview]
[-input_test_point_file file] [-exclude <inst|hinst|module>+] [-output_test_point_file file] [-dont_check_dft_rules] [design]
```

Description

Inserts gating logic at selected flop outputs to minimize switching power during scan shift. This command must be run prior to building the actual scan chains in the design.

The usage of this command and the flows that it can be used with, are documented in the section: Gating Functional Paths to Reduce Scan Shift Power in *Genus Design for Test Guide*.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

design	Specifies the name of the top-level design on which to perform test point insertion.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
-dont_check_dft_rules	
	Prevents the DFT rules from being automatically checked after inserting scan power gating.
-input_test_point_file *file*	
	Specifies the name of the file containing the test point locations. The file is specified in Modus format.
	If you omit this option, the test point locations are read from:
	• The file specified with the -output_tp_file option
	• The following file in the working directory if you did not specify any file:
	TB/testresults/TestPointInsertion.ASSUMESCAN.expt.
-max_number_of_test_points *integer*	
	Specifies the maximum number of test points to be inserted. Selection is based on the specified number of identified test points and the weight of the point resulting from logic cone analysis.The weight indicates the probability of a higher power level if toggling occurs.
-min_slack *integer*	Limits the insertion of a test point to those nodes that have the specified minimum slack (in ps).
	Default: 2000
-output_test_point_file *file*	
	Specifies the name of the generated output file containing the recommended test points. If you do not specify this option, the test point locations are written to the following file in the working directory:
	TB/testresults/TestPointInsertion.ASSUMESCAN.expt
-power_mode *mode*	Specifies the power mode for which the test power must be reported.
	Specify this option with the -report_virtual_scan_power option when the design has multiple power modes.
-exclude	Specifies to exclude the insts/hinsts from testpoint insertion.
-preview	Reports the test points to be added, without modifying the design.
-report_virtual_scan_power "*clockFreq* [*flopTogglePercent*]"	
	Performs a *virtual* insertion of the test points into the design and measures their impact on the reduction of scan power. Use a floating value to specify the test clock frequency (in MHZ) and the flop-toggle percentage to use in the measurement. If the flop toggle percentage is not specified, it will default to 50% of the test clock frequency.
	This option is only valid with the -preview option.
-test_control *test_signal*	
	Specifies the test signal to enable the test point. This option is not required when the command is run with the -preview option.
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.

Related Information

Gating Functional Paths to Reduce Scan Shift Power in *Genus Design for Test Guide*.	
Affected by this constraint:	define_test_signal -function <shift_enable | test_mode>

add_shadow_logic

```
add_shadow_logic
{-around {inst|hinst}... [-test_control test_signal] {-mode bypass |-mode no_share -test_clock_pin {port|pin|hpin} [-rise |-fall] |-mode share -
test_clock_pin {port|pin|hpin} [-fall | -rise] } [-exclude string | -only string ] [-group string]... [-balance] |-auto [-min_shadow_logic_pins integer] [-
exclude_shadow_logic {inst|hinst}] [-test_control test_signal] [-test_clock_pin {port|pin|hpin}] } [-dont_map] [-preview] [design]
```

Description

This command either

- Automatically inserts shadow logic around logic abstract and timing models (by adding observable flops in non-share mode) when you use the `-auto` option.
- Manually inserts bypass logic and scannable logic with or without register sharing when you use the `-around` option.
  You can insert two basic types of DFT shadow logic around a particular instance: bypass and scannable logic. Each shadow logic flip-flop can implement one control point and one observation point at the same time.
  If you want to share observation and control points, either by setting `-mode` to `share` or `bypass`, the following sharing criteria are observed:
  - If you specify `-group`, the specified inputs and outputs are grouped together as indicated
  - For the remaining inputs and outputs that are not listed with `-group`, the first input will share the flip-flop with or be connected to the first output, the second input with the second output, and so on. The order is that specified in the HDL interface declaration.

Test points are added for unidirectional pins only. Bidirectional pins and pins associated with test clock objects are skipped.

Options and Arguments

`-around {`*inst*`|`*hinst*`}`	
	Specifies the instances around which the DFT shadow logic must be inserted. Specify a hierarchical instance name.
`-auto`	Automatically inserts shadow logic around logic abstract and timing models (by adding observable flops in non-share mode).
`-balance`	Groups unmatched input and output pins to have a balanced number of groups.
design	Specifies the name of the top-level design in which to insert shadow logic.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_map`	Prevents the inserted logic from being mapped even if the design is already mapped to the target library.
`-exclude` *string*	Prevents the specified pins from being considered for shadow logic insertion.
`-exclude_shadow_logic {`*inst*`|`*hinst*`}`	
	Excludes automatic shadow logic insertion for the specified instances. You can only specify instances of blackboxes or timing models.
`-group` *string*	Specifies the pins to group when also using `-mode share` or `-mode bypass`. Each group can have multiple input pins and multiple output pins. Format the groups as follows:
	`{`*inputi ... outputj...*`}`

	Separate the pins by spaces. If you have more than one group, you must specify multiple `-group` options.	
	If `-mode` is set to `bypass`, each group must have at least one input and one output. Otherwise, the number of inputs or outputs can be equal or larger than zero.	
`-min_shadow_logic_pins` *integer*		
	Limits shadow logic insertion to logic abstract or timing models that have more pins (inputs and outputs) than the specified value.	
	Default: `10`	
`-mode`	Specifies the type of shadow logic to insert.	
	`bypass`	Implements bypass logic. If you specify this option, you must balance the number of inputs and outputs.
	`no_share`	Inserts one scannable observation test point per input and one scannable control test point per output.
	`share`	Pairs each input with an output and uses one scannable control and observation test point for each pair. If there are a different number of inputs and outputs, uses one scannable observe (or control) test point is used for each remaining input (or output).
`-only` *string*	Restricts the pins to be considered for shadow logic insertion to the specified ones.	
`-preview`	Shows the potential changes, without making any modifications to the netlist.	
`[-rise|-fall]`	Specifies the edge of the test clock that is active during test mode operation. These options are only valid in conjunction with `-test_clock_pin`.	
	Default: `-rise`	
`-test_clock_pin` {*pin*|*hpin*|*port*}		
	Specifies the test clock that drives the clock pin of the shadow flip-flops. You can specify a port or pin that drives the test clock.	
`-test_control` *test_signal*		
	Specifies the test signal to use to control DFT logic (the multiplexers after the controlling points).	
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.	

Examples

In the following examples, the logic before the ATPG-untestable module is not observable and the logic after it is not controllable. Following is the Verilog input code for the ATPG-untestable module and its instantiation:

```
module blackbox (i1,i2, o1,o2,o3)
    input i1,i2;
    output o1,o2,o3;
...
blackbox U1 (.i1(n_1), .i2(n_2), .o1(n_3), .o2(n_4), .o3(n_5));
...
```

- Using the following examples, bypass logic is used to make the two inputs observable and the three outputs controllable. The first command pairs input `i1` to output `o1`, and input `i2` to output `o3` (skipping `o2`).The second command pairs input `i1` and output `o2`.

  ```
  define_test_signal -function test_mode -name my_TM -active high TM add_shadow_logic -around U1 -test_control my_TM -mode bypass \ -exclude o2 add_shadow_logic -around U1 -test_control my_TM -mode bypass \ -only {i1 o2}
  ```

- The following example uses scannable test points and shares these test points as control and observation points.

  ```
  define_test_signal -function test_mode -name my_TM -active high TM add_shadow_logic -around U1 -test_control my_TM -test_clock_pin CK \ -mode share
  ```

- The following example uses scannable test points but does not share these test points for control and observation points.

```
define_test_signal -function test_mode -name my_TM -active high TM add_shadow_logic -around U1 -test_control my_TM -test_clock CK \ -mode
no_share
```

- The following example uses scannable test points, shares these test points for control and observation points, and controls by grouping which pins share a common test point. More specifically, `i1` and `o2` share a test point, and `i2` and `o1`. In addition, no control point is inserted for the net driven by `U1/o3`.

```
define_test_signal -function test_mode -name my_TM -active high TM add_shadow_logic -around U1 -test_control my_TM -test_clock CK \ -exclude
o3 -mode share -group {i1 o2} -group {i2 o1}
```

- The following example automatically inserts shadow logic around all logic abstract and timing model instances.

```
genus:root:> add_shadow_logic -auto -test_control my_tm

INFO: Using test clock pin 'port:data_ram_fj/CK' for testpoint insertion.
WARNING: pin 'U_BIST_CONTROLLER/I_BIST_WR_ENABLE_REG/CP' is skipped from shadow DFT insertion since it is driven by a clock

WARNING: pin 'U_BIST_CONTROLLER/I_BIST_WR_ENABLE_REG/QN' is skipped from shadow DFT insertion because it has no load

Total number of test points inserted: 6
....
WARNING: pin 'U_REAL_DATA_RAM3/CK' is skipped from shadow DFT insertion since it is driven by a clock


Total number of test points inserted: 79

  Mapping shadow DFT logic...
...
652
```

The command returns the total number of test points inserted.

Related Information

Inserting DFT Shadow Logic in *Genus Design for Test Guide*

Affects these commands:	check_dft_rules
	report_scan_registers
Related constraints:	define_test_clock
	define_test_signal -function <shift_enable | test_mode>

add_shift_register_test_points

```
add_shift_register_test_points [-preview]
[-write_to_log_file file] [-shift_enable test_signal]
[-max_number_of_test_points integer]
[-incremental] [-design design]
```

Description

Allows you to add shift register test points in a mapped netlist to identify additional shift registers. You can insert the test points in one of the following ways:

- By automatically identifying the test points and manually selecting the test points.
- By automatically identifying the test points and inserting them in one step.

Options and Arguments

`-design` *design*	Specifies the name of the design in which to insert the test points.
`-incremental`	Identifies shift registers in incremental mode.
`-max_number_of_test_points` *integer*	
	Specifies the maximum number of test points that can be inserted.
`-preview`	Performs test point identification in preview mode to check the suggested test point locations.
`-shift_enable` *test_signal*	
	Specifies the controlling shift enable signal.
`-write_to_log_file` *file*	
	Specifies the file to which to write out the test points. Use this option when you want to manually select the test points.

Related Information

Inserting Test Points to Create Shift Registers in a Mapped Netlist before Creating the Scan Chains in *Genus Design for Test Guide*.	
Related constraint:	define_test_signal -function shift_enable

add_test_compression

```
add_test_compression
[-use_existing_channels actual_scan_chain...]
| -build_new_scan_chains integer
  [-asymmetric_scan_ins integer]] [-decompressor {broadcast | xor }] [ -compressor xor [-mask {wide1|wide2}] | -compressor hybrid [-mask {wide0|wide1|wide2}]]
[ -use_existing_wrapper_channels actual_scan_chain... [-use_wir_macro inst|hinst] [-bypass_register]] [-mask_sharing_ratio integer] [-auto_create] [-inside
hinst] [-top] [-2d_compression] [-dont_map] [design]
```

Description

Inserts the test logic infrastructure for scan compression and hierarchical test. This includes inserting and configuring the compressor, de-compressor and x-masking logic, and connecting them to existing or newly created compression mode scan chains (also called STUMPS channels). The DFT logic can be optionally controlled from instructions decoded from the core's IEEE 1500 wrapper instruction register (WIR).

For the Unified Compression flow, use `help dft_compression_*` command to see all available attributes of that flow.

Options and Arguments

`-2d_compression`	Specifies the 2D compression.
`-asymmetric_scan_ins` *integer*	
	Specifies the number of scan data input pins for the compression macro. You only need to specify this option if the number differs from the number of scan data output pins.
	Default: The number of scan chains specified with the `-build_new_scan_chains` option.
`-auto_create`	Automatically creates the necessary compression control signals as top-level ports.
	For the names of the auto-created ports, refer to Hierarchical Test.
	You can also define the test bus ports using the `define_test_bus_port` command.

`-build_new_scan_chains` *integer*	
	Specifies the total number of top-level scan chains that is desired.
`-bypass_register`	Inserts a single-bit bypass register to the scan paths in 1500 bypass mode for the hierarchical test flows.
	This option can only be specified with the `-use_existing_wrapper_channels` option.
`-compressor {xor | hybrid}`	
	Specifies the type of compression logic to be built:
	• `xor` specifies to build an XOR-based compressor
	• `hybrid` specifies to build a `MISR` compression with `MISR` bypass capability. Bypassing the MISR allows you to perform compression using just the XOR compressor.
	Default: xor
`-decompressor {broadcast | xor}`	
	Specifies the type of decompression logic to be built:
	• `xor` specifies to build an XOR-based spreader network in addition to the broadcast-based decompression logic
	• `broadcast` specifies to build a broadcast-based decompression logic (simple scan fanout).
	Default: broadcast
design	Specifies the name of the top-level design.
`-dont_map`	Prevents the inserted macros from being mapped even if the design is already mapped to the target library.
`-inside` *hinst*	Specifies the hierarchical instance in which to instantiate the compression logic.
`-mask {wide0 | wide1 | wide2}`	
	Inserts scan channel masking logic of the specified type.
	The masking types that can be used depend on the compressor type specified with the `-compressor` option.
	By default, no masking logic is inserted.
`-mask_sharing_ratio` *integer*	
	Specifies the number of internal scan channels sharing a mask register.
`-top`	Specifies to insert the hierarchical compression logic at the top level.
	It is used for top-level compression and TIM insertion in a hierarchical test flow after the lower-level blocks have been synthesized, DFTs are inserted and scan abstract created. It connects the blocks together via the top TIM.
`-use_existing_channels` *actual_scan_chain...*	
	Specifies the names of the actual scan chains to be used as compression channels.
`-use_existing_wrapper_channels` *actual_scan_chain...*`]`	
	Specifies the names of the actual scan chains to be used as wrapper compression channels.
`-use_wir_macro` *inst*`|`*hinst*	
	Specifies the path name of the wrapper instruction register (WIR) instance.
	This option can only be specified with the `-use_existing_wrapper_channels` option.

Related Information

Hierarchical Test Flow: Preparing a Core in *Genus Design for Test Guide*

Related command:	define_test_bus_port

add_test_point

```
add_test_point -location {pin|hpin|port}... [-test_control test_signal]

[-gate_clock_test_control test_signal

]

[-gate_clock_for_observe

test_signal] -type { control_0 | control_1 } | {{async_0 | async_1 | async_any | bypass_scan | control_node -node {pin|hpin|port} | control_observe_0 |
control_observe_1 | control_observe_node -node {pin|hpin|port} | control_scan | observe_scan [-max_observe_share integer] | scan | sync_0 | sync_1 | sync_any
} -test_clock_pin {pin|hpin|port} [-rise|-fall] } [-dont_map]
```

Description

Allows you to manually specify a control or observation test point to be added to the design. Control test points always require the specification of a test-mode signal. Test points that use scannable flip-flops to observe or control a node always require a test-clock signal.

For all of the scannable test points, you need to run check_dft_rules after the test point is inserted.

The command returns the path name of the inserted test point when it is a flip-flop.

> You can only specify multiple locations when you request to insert a test point of type `observe_scan`. In this case, the tool builds a balanced XOR-tree with all the specified location pins. Additionally, you can control the maximum number of pin locations to be observed by the same observation flop by specifying the `-max_observe_share` option. If a test-control signal is also specified, the tool builds the XOR-tree after each input is AND-ed or OR-ed with the test-control signal. This prevents switching along the XOR-tree when not it test mode. If the test control is active high, gating happens by AND-ing, otherwise by OR-ing. The output of the last XOR-gate is fed to the D input of the observation flip-flop.

Options and Arguments

`-dont_map`	Prevents the inserted test point from being mapped even if the design is already mapped to the target library.
`-gate_clock_for_observe` *test_signal*	
	Specifies the test signal to gate the clock for the observe flop.
`-gate_clock_test_control` *test_signal*	
	Specifies the test signal to gate the clock for the control flop.
`-location` {*pin*|*hpin*|*port*}	
	Specifies the location of the control point or observation point. Specify an existing hierarchical pin name or a top-level port. For observation test points, the pin can be an input or output pin. For control test points, the result is different depending on the direction of the location.
	> You can only specify multiple locations for a test point of type `observe_scan`.

	If you specify a bidirectional pin, no logic will be inserted unless you specify the direction of the pin.	
`-max_observe_share` *integer*		
	Specify the maximum number of locations to be shared for an observe test point.	
	This option applies only when you set `-type` to `observe_scan`.	
`-node` {*pin*|*hpin*|*port*}		
	Specifies the pin or port to insert when `-type` is set to `control_node`, `control_observe_node` or `bypass_scan` and when the signal specified by `-test_control` is active.	
`[-rise | -fall]`	Specifies the edge of the specified test clock that is active during test mode operation. These options are only valid in conjunction with `-test_clock`.	
	You must use the same clock edge when inserting a control flip-flop and an observation flip-flop.	
	Default: `-rise`	
`-test_clock_pin` {*pin*|*hpin*|*port*}		
	Specifies the test clock that drives the clock pin of the inserted flip-flops during test mode operation. You can specify a port or pin that drives the test clock.	
	This option is required for all type point types set using the `-type` option except those of type `control_0` or `control_1`.	
`-test_control` *test_signal*		
	Specifies the test signal to use to control or observe the specified location point.	
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.	
	Test points of type `observe_scan` do not require a test signal.	
`-type`	Specifies the type of test point to insert at the specified location when the signal specified by `-test_control` is active. Possible values are:	
	`async_0`	Inserts an asynchronous control test point that forces the control point to the value 0.
	`async_1`	Inserts an asynchronous control test point that forces the control point to the value 1.
	`async_any`	Inserts an asynchronous control test point that forces the control point to take either the original value or the inverted value.
	`control_0`	Inserts a constant value `0`.
	`control_1`	Inserts a constant value `1`.
	`control_node`	Inserts an arbitrary node.
	`control_observe_0`	Inserts a control and an observation point. The control point is forced to the value 0.
	`control_observe_1`	Inserts a control and an observation point. The control point is forced to the value 1.
	`control_observe_node`	Inserts a control and an observation point. The control point is forced to the value of the node specified by `-node`.

	`control_scan`	Inserts a flip-flop to force a particular value at the specified location during test mode operation. The flip-flop must be remapped to a scan flop before connecting it to a scan chain later on. This option requires you to specify the `-test_clock_pin` option.
	`double_cntrol_0`	Inserts two asynchronous control test points that forces the control point to the value 0.
	`double_cntrol_1`	Inserts two asynchronous control test points that forces the control point to the value 1.
	`double_cntrol_observe_0`	Inserts an asynchronous control test point that forces the control point to the value 0 and inserts a flip-flop to observe the specified location.
	`double_cntrol_observe_1`	Inserts an asynchronous control test point that forces the control point to the value 1 and inserts a flip-flop to observe the specified location.
	`observe_scan`	Inserts a flip-flop to observe the specified location. The flip-flop must be remapped to a scan flip-flop before connecting it to a scan chain later on. This option requires you to specify the `-test_clock_pin` option.
	`scan`	Inserts a scannable control and observation test point. This option requires you to specify the `-test_clock_pin` option.
	`sync_0`	Inserts a synchronous control test point that forces the control point to the value 0.
	`sync_1`	Inserts a synchronous control test point that forces the control point to the value 1.
	`sync_any`	Inserts a synchronous control test point that forces the control point to take either the original value or the inverted value.

Examples

- The following example inserts a scannable observation test point, using `CLK` to drive:

```
add_test_point -location X/out -test_clock_pin CLK -type observe_scan
```

- The following example inserts a control-1 and scannable observation point:

```
add_test_point -location X/out -test_control TM \
-test_clock_pin CLK -type control_observe_1
```

- The following example inserts a scannable control point:

```
add_test_point -location X/out -test_control TM \
-test_clock_pin CLK -type control_scan
```

- The following example inserts a scannable control and observation test point:

```
add_test_point -location X/out -test_control TM \
-test_clock_pin CLK -type scan
```

- The following example inserts an async control-0 test point at hierarchical pin `X/out`:

```
add_test_point -location X/out -test_control TM -type async_0 \
-test_clock_pin CK -fall.
```

- The following example inserts a synchronous control test point that forces the control point to the value 1 at hierarchical pin `X/out`:

```
add_test_point -location X/out -test_control TM -type sync_1 \
-test_clock_pin CK -fall.
```

- The following example inserts two observation test points, one for `pin1`, `pin2` and `pin3`, and the other for `pin4`, `pin5` and `pin6`.

```
add_test_point -type observe_scan -max_observe_share 3 \
-location pin1 pin2 pin3 pin4 pin5 pin6
```

Related Information

Inserting a Control and Observation Test Point in *Genus Design for Test Guide*.	
Affects these commands:	check_dft_rules
	connect_scan_chains
	syn_map
	syn_opt
Related constraints:	define_test_clock
	define_test_signal -function <shift_enable | test_mode>
Related attributes:	test_clock Attributes
	test_signal Attributes

add_user_test_point

```
add_user_test_point
-location {pin|hpin|port|hport} -cell {design|module|lib_cell} {-cfi {pin|hpin|port}} | -no_cfi} [-cfo {pin|hpin|port}]
 -connect string [-connect string]... [-unconnect pin_list] [-name name]
```

Description

Inserts a user-defined test point at the specified location, and hooks it up to the specified pins.

Options and Arguments

`-cell {`*`design`*`|`*`module`*`|`*`lib_cell`*`}`	
	Specifies the module or library cell to instantiate. The module can be loaded as a parallel design, or as a module.
`-cfi {`*`pin`*`|`*`hpin`*`|`*`port`*`}`	
	Specifies the cell functional input (CFI) pin or port name (name of the input pin or port that will be automatically connected to the input side of the new broken net of the -location option).
`-cfo {`*`pin`*`|`*`hpin`*`|`*`port`*`}`	
	Specifies the cell functional output (CFO) pin or port name (name of the output pin or port that will be automatically connected to the output side of the new broken net of the -location option).
`-connect` *`string`*	Specifies a string consisting of a cell pin and the corresponding source-signal pin to which the cell pin must be connected.
	This string has the following format:
	`{`*`cell_pin source_pin`*`}`
	Use this option to specify most connections to the cell, except for the connections to the CFI and CFO pins.
`-location {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`hport`*`}`	
	Specifies a pin, port or subport that identifies where the test point must be inserted.
`-name` *`name`*	Specifies the instance name to be given to the user-defined test point.
`-no_cfi`	Specifies that the user test point cell has no CFI pin, and therefore no input pin of the newly added cell will be automatically connected.
`-unconnect` *`pin_list`*	
	Specifies a list of pins on the test point that are intentionally left unconnected. When you specify the unconnected pins with this option, the tool will not issue a warning.

Examples

- The following example inserts design `MyUserTI` in design `top` at the `in1[2]` input port. Port `MyCFI` will be connected to input port `in1[2]`.

```
add_user_test_point -location top/in1[2] -cell design:MyUserTI \
-cfi MyCFI -cfo MyCFO -connect {MyShiftEn se} -connect {MyWRCK wck}
```

- Assume pins `D`, `S1` and `C` of the user test point are not connected.
  - If you specify the command without the `-unconnect` option, the tool will issue warnings:

```
genus:root: xx> add_user_test_point -cell MX4X1 -location en -cfi A -cfo Y\ -connect {B se} -connect {S0 tm}

Warning : The testpoint pin is not connected. [DFT-296]
        : Pin 'D' of testpoint will be left unconnected.
        : Use 'connect' option to connect the pin or use 'unconnect' option to suppress the warning message.
Warning : The testpoint pin is not connected. [DFT-296]
        : Pin 'S1' of testpoint will be left unconnected.
Warning : The testpoint pin is not connected. [DFT-296]
        : Pin 'C' of testpoint will be left unconnected.
```

  - If you specify the command with the `-unconnect` option, the tool will only issue a warning for any unconnected pin, not mentioned in the pinlist of the `-unconnect` option:

```
genus:root: xx> add_user_test_point -cell MX4X1 -location en -cfi A -cfo Y\ -connect {B se} -connect {S0 tm} -unconnect {D S1}

Warning : The testpoint pin is not connected. [DFT-296]
        : Pin 'C' of testpoint will be left unconnected.
        : Use 'connect' option to connect the pin or use 'unconnect' option to suppress the warning message.
```

Related Information

See the following sections in *Genus Design for Test Guide*:

- Inserting a User-Defined Control and Observation Test Point
- Inserting Scannable Test Points in Existing Scan Chains

Affects these commands:	check_dft_rules
	connect_scan_chains
	syn_map
	syn_opt

add_wir_signal_bits

```
add_wir_signal_bits
[-create_wrapper_shift_enables_for_delay_test test_signal]
[-create_cgic_enable_for_wrap test_signal]
[-preview] [-design design] [-inside design|hinst]
```

Description

Inserts a wrapper instruction register or WIR, the first time you invoke the command, and adds the WIR signals defined thus far by the `define_test_bus_port -wir_signal` command. Each time you invoke this command, it adds the WIR signals that you defined since the last time you invoked this command.

In addition, the command connects the WIR to the ports corresponding to the Wrapper Serial Control (WSC) signals and Wrapper Serial Input (WSI). The WSC and WSI signals are defined with the `define_test_bus_port` command with specific function values.

The command is used when building an IEEE 1500 wrapper around a design block. The usage of this command is documented in *Genus Design for Test Guide* in the section titled Inserting Wrapper Instruction Register and Defining WIR Signal Bits in chapter Inserting Core-Wrapper Logic.

Options and Arguments

`-create_cgic_enable_for_wrap` *test_signal*	
	Specifies the test signal to be gated by the inverse of the decoded `EXTEST` signal. You can specify this gated signal (`DFTCGENI`) as the `lp_clock_gating_test_signal` to control the test pin of the functional clock gates which feed the core flops. After inserting the core wrapper logic, the test pins of the functional clock gates which feed the shared wrapper cells can be controlled by the `DFTWSEO` signal by rewiring the test pin connections.
`-create_wrapper_shift_enables_for_delay_test` *test_signal*	
	Specifies the shift-enable signal to use to create two new shift-enable signals: the wrapper shift-enable signal used for `INTEST` mode (gated by the decoded `INTEST` signal), and the wrapper shift-enable signal used for `EXTEST` mode (gated by the decoded `EXTEST` signal).
`-design` *design*	Specifies the design in which you want to insert the wrapper instruction register.
	This option is required if multiple designs are loaded.
-inside *design*|*hinst*	
	Specifies the location to insert the WIR. Default: design
`-preview`	Shows a preview of the WIR signals that would be added to the WIR along with their bit index in the WIR.

Related Information

Inserting the Wrapper Cells in *Genus Design for Test Guide*

Related command:	define_test_bus_port

add_wrapper_cell_aware_clock_gating

```
add_wrapper_cell_aware_clock_gating
 [-wrap_te] [-core_te] [-core_en]  [-test_signal_pin pin|hpin|port][-test_signal_polarity string][-verbose][-preview][-hierarchy hinst|module+][-dont_clone][-dont_gate_wrap_te][-dont_gate_core_te][-dont_gate_core_en][-cloning_threshold integer][-dont_rewire_rtl_icg][-design design]
```

Description

Clones and rewires Genus inserted clock gates to ensure that the core flops do not receive any clock during extest mode of operation.

The command is expected to be run after wrapper insertion and mapping.

Options and Arguments

`-wrap_te`	Specifies the test signal source pin for wrapper ICG test enable.
`-core_te`	Specifies the test signal source pin for core ICG test enable.
`-core_en`	Specifies the test signal source pin for core ICG enable.
`-test_signal_pin` *pin|hpin|port*	
	Specifies the test signal source pin.
`-test_signal_polarity` *String*	
	Specifies the test signal polarity. The default value is high.
`-verbose`	Allows printing the information in detail.
`-cloning_threshold` *integer*	
	Specifies the minimum number of flops in the clock gate fanout cone for cloning to happen. *Default:1*
`-dont_clone`	Specifies that the clock gates should not be cloned.
`-dont_rewire_rtl_icg`	
	Specifies not to rewire the rtl ICG.
`-dont_gate_wrap_te`	Avoids gating of wrapper ICG test enable.
`-dont_gate_core_te`	Avoids gating of core ICG test enable.
`-dont_gate_core_en`	Avoids gating of core ICG enable.
`-design` *design*	Specifies the top block. This option is required if multiple designs are loaded.
`-preview`	Specifies the tool analysis report before cloning and rewiring the ICG cell.

Related Information

Related Command	add_core_wrapper_cell

analyze_atpg_testability

```
analyze_atpg_testability [-library string]
[-atpg_options string]
[-build_faultmodel_options string]
[-build_model_options string]
[-build_testmode_options string]
[-fault_sample_size integer]
[-atpg_log file] -directory string [design]
```

Description

Invokes Modus to perform Automatic Test Pattern Generator (ATPG) based testability analysis in either assume or fullscan mode.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-atpg_options` *string*	
	Specifies a string containing the extra options to run ATPG-based testability analysis.
	Use the following format for the *string*:
	"*-option1 value -option2 value ...*"
	For more information on these options, refer to the `create_logic_tests` or `create_logic_delay_tests` command in the Modus documentation.
`-atpg_log` *file*	Specifies the name of the Modus log file. This file will be generated in the specified directory.
	Default: `atpg.log`
`-build_faultmodel_options` *string*	
	Specifies a string containing the extra options to build a fault model.
	Use the following format for the *string*:
	"*-option1 value -option2 value ...*"
	For more information on these options, refer to the `build_faultmodel` command in the *Command Line Reference* (of the Modus documentation).
`-build_model_options` *string*	
	Specifies a string containing the extra options to build a model.
	Use the following format for the *string*:
	"*-option1 value -option2 value ...*"
	For more information on these options, refer to the `build_model` command in the Modus documentation.

`-build_testmode_options` *string*	
	Specifies a string containing the extra options to build the test modes which define the scan structure and active logic for testing.
	Use the following format for the *string*:
	`"-option1 value -option2 value ..."`
	For more information on these options, refer to the `build_testmode` command in the Modus documentation.
design	Specifies the name of the top-level design on which you want to perform test analysis and test-point selection.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	
	Specifies the working directory for Modus.
`-fault_sample_size` *integer*	
	Specifies the approximate number of faults simulated to predict the test coverage. This number affects the number of partitions or slices to be run depending on the total number of faults in the design.
	The default sample size gives a good estimation of fault coverage while limiting the run time. Use a higher number to get a better accuracy, or a smaller number to reduce your run time.
	Defaults: `20000`. For full ATPG, `-1`.
`-library` *string*	Specifies the list of Verilog structural library files, for example, *file1 file2*. Refer to the `write_dft_atpg` -library option description for additional information.
	This option is only required when you invoke this command on a mapped netlist.
	Verilog libraries are only needed if the design is mapped to technology gates - otherwise we will use simulation primitives for generic gates, and pass an RTL module description from Genus for the flops and MUXes. Can be used on a mapped design in an assume scan mode. Accuracy improves if analysis is run after you have built the scan chains. You do not need to do a full ATPG analysis: can still do 20k faults. But with the actual chains in place, the results obtained are more accurate than if the analysis was done in an assume scan mode (i.e., without the chains, assuming all the DFFs are in scan chains).

Examples

- The following example performs only ATPG-based testability analysis. The command generates a report on the fault coverage in the log file.

  ```
  analyze_atpg_testability
  ```

- The following command instructs to build a model using the IEEE standard Verilog parser.

  ```
  analyze_atpg_testability -build_mode_options "vlogparser=IEEEstandard"
  ```

- The following command uses the information from Verilog structural library files and writes out ATPG files in the directory specified:

  ```
  analyze_atpg_testability -directory atpg_test -library "./LIMS/atpg_library.v"
  ```

Related Information

Using Modus to Analyze Testability in *Genus Design for Test Guide*.

Affected by these constraints:	define_test_clock
	define_test_signal -function test_mode

analyze_initmode_processing

```
analyze_initmode_processing

[design  design]
[-mode_init  string]
[-iclfile string]
[-pdlfile string]
[-pdlentryfunction string]
[-file string]
[-library string]
[-ncsim_library string]
[-continue_with_severe_warnings]
[-fixed_value_register_file string]
[-init_value_register_file string]
[-skip_apply_init_on_ports]
[-skip_apply_init_on_flops]
[-filter_hier string]
[-exclude_hier string]
[-modus_xterm]
[-atpg_log string]
[-generate_scripts_only]
[-report_fixed_value_register]
[-macro_proc string]
[-preview]
[-prepare_testmode_sequences_override string]
```

Description

Configures Modus environment for identifying fixed value or init value registers, and auto-defines them as test mode signals.

The fixed-value registers can be identified:

- From an existing Modus log file
- By invoking Modus and simulating a mode initialization sequence Note: In the latter case, you need to have the Modus software installed and your operating system PATH environment variable must include the path to the Modus software.

  This command is not generally used and is intended for a custom flow where a special test mode initialization sequence is executed in Modus to identify all fixed-value registers that hold their value constant during the entire test mode.

  The fixed-value registers and their constant values can be fed back to a Genus DFT synthesis session to simplify the DFT setup. The alternative is for the user to manually identify all internal test mode fixed-value and initialized-value registers.

Options and Arguments

`-design` *design*	Specifies the name of the top-block. This is required if there are multiple top designs.
`-mode_init` *string*	Specifies the stil TBDSeqPatt(s) file which includes the mode initialization sequence. If there are multiple files, they are sequenced in the order provided.
`-iclfile` *string*	Specifies the stil ICL file(s) to be used for processing. For example, "file1 file2"
`-pdlfile string`	Specifies the stil PDL file(s) to be used for processing. For example, "file1 file2"
`-pdlentryfunction` *string*	Specifies the stil file which includes the mode initialization sequence.
`-file` *string*	Specifies the stil file which includes the mode initialization sequence
`-library` *string*	Specifies the list of Verilog files for the library. For example, "file1 file2"
`-ncsim_library` *string*	Specifies the list of files for NCSIM. For example, "file1 file2"
`-continue_with_severe_warnings`	Allows continuing the Modus run with severe warnings
`-fixed_value_register_file` *string*	Specifies Modus log file with list of fixed value registers
`-init_value_register_file` *string*	Specifies Modus log file with list of initialized latches and pins
`-skip_apply_init_on_ports`	Skips initialized ports in Modus log file
`-skip_apply_init_on_flops`	Skips initialized flops in Modus log file
`-filter_hier` *string*	User specified scope to filter hierarchies
`-exclude_hier` *string*	User specified scope to exclude hierarchies
`-modus_xterm`	Specifies the use of xterm while running Modus
`-atpg_log` *string*	Specifies the log file for Modus
`-generate_scripts_only`	Specifies not to execute Modus but just write scripts
`-report_fixed_value_register`	Reports only fixed value registers in report test structures command
`-macro_proc` *string*	Specifies the name of the mode initialization sequence to execute default: *test_setup*
`-preview`	Reports only the fixed value latches. It does not set test modes.
`-prepare_testmode_sequences_override` *string*	Allow users to specify options for Modus ICL/PDL processing

Examples

- The following command produce runmodus.init.log and invoke Modus.

```
genus@root:> analyze_initmode_processing -directory ./wodzro_test_log -mode_init ./elab_modus_cell/TBDseqPatt.test_setup_dc_force -
```

```
pdlfile ./PDL/seraph.pdl -iclfile ./ICL/seraph.icl -pdlentryfunction seraph_01L_seraph_01C_1_AC_BR

       Identifying internal registers with fixed value or initialized outputs under test_mode setup.

       WARNING : No user defined shift enable signal found.
       ATPG interface file may contain incomplete information

       ------------------------------------------------------------
       ------------------------------------------------------------
```

- The following command reads the log file from Modus and automatically runs the DFT rule checker.

```
   genus@root:>analyze_initmode_processing -fixed_value_register_file ./log_report_test_cell
      INFO: Setting active low test mode signal on iatom/icif/ccke_tkmaskr_reg/QB
      INFO: Setting active low test mode signal on icherub/che_ins0/iatom/icif/ciif_cifscantk_reg/QB
      INFO: Setting active low test mode signal on icherub/che_ins0/princ[1].ins/arch[1].ins/angel[1].ins/data1_reg[27]/q
      Info    : Initiating DFT command. [DFT-145]
              : Initiating check_dft_rules command.
                Checking DFT rules for 'seraph' module under 'muxed_scan' style

                Detected 301 DFT rule violation(s)
                  Summary of check_dft_rules
                    *************************
      genus@root:> vls dft/test_signals
      design:seraph.dft.test_signals:
      TMS
      TRST
      iatom__icif__ccke_tkmaskr_reg__QB
      icherub__che_ins0__iatom__icif__ciif_cifscantk_reg__QB
      icherub__che_ins0__princ[1].ins__arch[1].ins__angel[1].ins__data1_reg[27]__q
      ivirtue__vir_ins0__princ[2].ins__neti_reg[1][9]__q
```

Related Information

Related constraints:	define_test_signal -function <shift_enable | test_mode>

analyze_scan_compressibility

```
analyze_scan_compressibility
-library string [design]
[-scanins integer...] [-scanouts integer...] [-channel_lengths integer...]
[-min_scanned_flop_percentage integer]
[-compression_method {add_test_compression | compress_scan_chains}]
[-compressor {xor | misr | mimic_bidi_misr}]
[-decompressor {xor | broadcast}]
[-mask {wide1 | wide0 | wide2 | none}] [-serial_misr_read]
[-fault_sample_ratio fault_sample_ratio]
[-ratios list_of_compression_ratios] [-delay]
[-directory directory] [-dont_run_atpg]
[-atpg_options string] [-build_model_options string]
[-build_faultmodel_options string]
[-build_testmode_options string]
[-verify_test_structures_options string] [-verbose]
```

Description

Analyzes a design for scan-based compressibility and produces actual compression results for each compression setting. Analysis is done based on the scan chain information in the design.

For the `compress_scan_chains` compression method:

1. If actual scan chains exist, analysis is performed based on the number of actual scan chains.
2. If actual scan chains do not exist, analysis is performed based on the number of user-defined scan chains.
3. If user-defined scan chains do not exist, analysis is performed based on the number of scan chains specified using the `-scanins` option.

When using the `compress_scan_chains` method, you are expected to provide a list of ratios using the `-ratios` option. When the `-ratios` option is not specified for this compression method, the command runs the analysis using the `-ratios` option with the default values of `"20 50 100"`

For the `add_test_compression` compression method:

1. If actual scan chains exist, analysis is performed based on the number of actual scan chains (internal compression chains) and specifying one of the additional options `-scanins` or `-ratios`.
   For each value provided to the `-scanins` option, the configuration is built and the resulting ratio is reported.
   If instead, a list of values is provided to the `-ratios` option, then each configuration is built and the computed number of scan in pins is reported.
2. If actual scan chains do not exist, analysis is performed based on specifying two of the three options `-scanins`, `-ratios` and `-channel_lengths`. One of these options must be provided as a list of values and a single value must be provided for the other. The third option will be determined automatically from the specified options. For example:
   The tool will determine the `channel_lengths` for

   ```
   -scanins 8 -ratios "5 10 15"
   ```

   The tool will determine the number of `scanin` pins for

   ```
   -channel_lengths 100 -ratios "5 10 15"
   ```

   The tool will calculate the scan compression ratio for

   ```
   -channel_lengths 100 -scanins "5 10 15"
   ```

The command usages above assume the compression analysis is performed for *symmetric* compression, where the number of scan input signals equals the number of scan output signals. Symmetric compression analysis is performed without specifying the number of scan output signals.

Using the `add_test_compression` compression method, you can also perform an *asymmetric* compression analysis by specifying the `-scanouts` option. This analysis is used to verify the impact of aliasing by varying the number of scan-out ports. You must invoke `analyze_scan_compressibility` with `-atpg_options "aliascheck=yes"` for Modus ATPG to enable aliasing checking during pattern generation which can lead to longer ATPG runtimes.

- When compression chains do not exist, the tool determines the `channel_lengths` and will vary the `scanouts`:

  ```
  -scanins 8 -ratios 2 -scanouts "2 4"
  ```

- When compression chains exist, you can specify either scan-ins or ratios and vary the scan-outs

  ```
  -scanins 8 -scanouts "2 4"
  ```

  ```
  -ratios  2 -scanouts "2 4"
  ```

To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-atpg_options` *string*	
	Specifies a string containing the extra options to run ATPG-based testability analysis.
	Use the following format for the *string*:
	`"-option1 value -option2 value ..."`
	For more information on these options, refer to the `create_logic_tests` or `create_logic_delay_tests` command in the Modus documentation.
`-build_faultmodel_options` *string*	
	Specifies a string containing the extra options to build a fault model.
	For more information on these options, refer to the `build_faultmodel` command in the Modus documentation.

`-build_model_options` *string*	
	Specifies extra options to apply when building the Modus model.
	Use the following format for the *string*:
	"-option1 value -option2 value ..."
	For more information on these options, refer to the `build_model` command in the Modus documentation.
`-build_testmode_options` *string*	
	Specifies extra options to apply when building the test mode for Modus.
	Use the following format for the *string*:
	"-option1 value -option2 value ..."
	For more information on these options, refer to the `build_testmode` command in the Modus documentation.
`-channel_lengths` *integer*	
	Specifies the lengths of the scan compression channels to be built using the `add_test_compression` compression method. This option is only required if no actual scan chains exist.
	The specified number must be equal to or larger than one.
`-compression_method {compress_scan_chains|add_test_compression}`	
	Specifies the compression method to use:
	• `add_test_compression` adds compression logic for designs starting with scan compression (internal) chains
	• `compress_scan_chains` adds compression logic for designs starting with Fullscan chains
	Default: `add_test_compression`
`-compressor {xor | misr | mimic_bidi_misr}`	
	Specifies the type of compression logic for analysis:
	• `xor` analyzes an XOR-based compressor
	• `misr` analyzes a `MISR-based` compressor using the `compress_scan_chains` compression method
	• `mimic_bidi_misr` analyzes an on-product MISR compressor using the `compress_scan_chains` compression method and mimics the behavior of the design as though the pads can be configured bidirectionally.
	Default: `xor`
`-decompressor {broadcast | xor}`	
	Specifies the type of decompression logic for analysis:
	• `broadcast` specifies broadcast-based decompression logic (simple scan fanout).
	• `xor` specifies an XOR-based spreader network in addition to the broadcast-based decompression logic.
	Default: `xor`
`-delay`	Specifies to create the Modus run scripts for Delay Test ATPG. This option must also be used when the design includes OPCG logic.

design	Specifies the name of the top-level design on which to perform analysis.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *atpg_directory*	
	Specifies the directory to run and store ATPG results.
	ATPG results tend to consume high amounts of disk space for larger designs.
	Default: *current_working_directory*`/asc`
`-dont_run_atpg`	Specifies to only insert the different compression options and produces a list of the ATPG jobs to be run.
	This option can be used in combination with the `report_scan_compressibility` command as the list of ATPG jobs is produced. The ATPG jobs may be run with either of the following methods:
	• Run on a different workstation.
	• Run in parallel mode to reduce runtime if a Load Sharing Facility (LSF) environment is available.
	Neither of the preceding methods are automatically invoked by the `analyze_scan_compressibility` command or the `report_scan_compressibility` command.
`-fault_sample_ratio` *float*	
	Specifies the percentage of faults used to estimate the test coverage.
	This option will be ignored when the command is specified with the `-delay` option.
	Defaults: `100.0` for full ATPG, valid values between `0.01` and `50.0`
`-library` *string*	Specifies the list of Verilog structural library files.
	You can specify the files explicitly or you can specify an *include* file that lists the files. You can also specify directories of Verilog files but you cannot reference directories in an include file.
	For example, assume the following Verilog files are required:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	You can specify the files in either of the following ways:
	1. Explicitly:
	`analyze_scan_compressibility -library "./padcells.v \ ./stdcells.v ./memories ./ip_blocks" ...`
	1. Using an include file.
	`analyze_scan_compressibility \-library "include_libraries.v ./memories \ ./ip_blocks" ...`
	where you created a file `include_libraries.v` with the following contents:
	`` `include "./padcells.v"`include "./stdcells.v" ``
	If you specify a relative path, the command interprets the path to be the location from where Modus will be run.

`-mask {wide1 | wide0 | wide2 | none}`	
	Specifies the scan channel masking logic type for analysis. The masking types that can be used depend on the compressor type specified with the `-compressor` option.
	By default, `wide1` masking logic is inserted.
	Genus provides three types of channel masking: `wide0`, `wide1`, and `wide2`.
	• For XOR-based compression, two types of masking are supported: `wide1` and `wide2`
	• For `MISR`-based compression, three type of masking are supported: `wide0`, `wide1`, and `wide2`.
	When requested to insert `wide1` masking, one mask register bit is added per channel. When requested to insert `wide2` masking, two mask register bits are added per channel.
`-min_scanned_flop_percentage` *integer*	
	Specifies the minimum percentage of scannable flops required to allow the scan compression analysis to proceed.
	Default: `95`
`-ratios` *integer_list*	
	Specifies the compression ratio or the list of compression ratios to be analyzed.
	Default: `"20 50 100"` using `compress_scan_chains` compression method
`-scanins` *integer*	Specifies the number of top-level scan input pins or the list of numbers of top-level scan input pins to be analyzed.
	The specified number must be larger than one.
`-scanouts` *integer*	Specifies the number of top-level scan output pins or the list of numbers of top-level scan output pins to be analyzed.
	The specified number must be larger than one.
	Default: Equal to the number of specified, or calculated top-level scan input pins for symmetric compression.
`-serial_misr_read`	Performs analysis for serial `MISR` compression using `compress_scan_chains` compression method.
	You can only specify this option for a `MISR`-based compression (`-compressor` option is set to `misr`).
	You cannot specify this option when the `-decompressor` option is set to `xor`.
`-verbose`	Displays all messages during analysis of the design for scan-based compressibility.
`-verify_test_structures_options` *string*	
	Specifies extra options to apply when performing test structure verification for Modus.
	Use the following format for the *string*:
	`"-option1 value -option2 value ..."`
	For more information on these options, refer to the `verify_test_structures` command in the Modus documentation.

Examples

- The following command performs XOR-based compression analysis with `wide1` masking for a ratio list of `"20 50 100"`. The command does not perform the ATPG runs of the different compression schemes because the `-dont_run_atpg` option was specified.

```
genus:root:> analyze_scan_compressibility -chains 8 -compressor xor \
-mask wide1 -ratios "20 50 100" -dont_run_atpg -library mylibrary \
-directory asc
```

```
Starting scan compressibility analysis for module 'test'

Creating work directory asc .....

Analyzing the design with the following configuration
-----------------------------------------------------
        Design       - test
        Decompressor - broadcast
        Compressor   - xor
        mask         - wide1
        ratio        - 20 50 100

Scan Registers Status
---------------------
        (Number of Scannable Registers) / (Total Register Count) =
        --> 999 / 999 = 100%

        (Number of flops mapped to scan) = 999
        (Number of flops passing tdrc)   = 999

        (Number of Scannable Registers)
        (Number of flops scan mapped and passing tdrc) +
        (Number of flops in shift-register segments) +
        (Number of flops in abstract scan segments) =
        --> 999 + 0 + 0 = 999

        (Total Register Count)
        (Number of registers) +
        (Number of flops in abstract scan segments) =
        --> 999 + 0 = 999

        (*) The percentage of scannable registers is 100%.

Building the Fullscan Chains
----------------------------

        Building balanced scan chains assuming all test clocks are compatible
        and mixing of clock edges in scan chains is allowed.
        If you prefer a different set of constraints, please build the scan
        chains before calling analyze_scan_compressibility.

        Configuring 8 chains for 999 scan f/f
                ...chain01 of length: 125
                ...chain02 of length: 125
                ...chain03 of length: 125
                ...chain04 of length: 125
                ...chain05 of length: 125
                ...chain06 of length: 125
                ...chain07 of length: 125
                ...chain08 of length: 124

        Modus scripts for ATPG have been written
        for fullscan mode to directory 'asc/et_scripts_fullscan'.
        .... Invoke the script from OS command line as 'et -e
        asc/et_scripts_fullscan/runet.atpg'.

Inserting Scan Compression Logic
--------------------------------

        Compressing scan chains with ratio 20
        Tool achieved ratio: 17.8

        Modus scripts for ATPG have been written
        for compression mode with fullscan top-off to directory
       'asc/et_scripts_ratio_20'.
        .... Invoke the script from OS command line as 'et -e
        asc/et_scripts_ratio_20/runet.atpg'.
```

```
        ----------------------------------------

        Compressing scan chains with ratio 50
        Tool achieved ratio: 41.6

        Modus scripts for ATPG have been written
        for compression mode with fullscan top-off to directory
        'asc/et_scripts_ratio_50'.
        .... Invoke the script from OS command line as 'et -e
        asc/et_scripts_ratio_50/runet.atpg'.
        ----------------------------------------

        Compressing scan chains with ratio 100
        Tool achieved ratio: 62.5

        Modus scripts for ATPG have been written
        for compression mode with fullscan top-off to directory
       'asc/et_scripts_ratio_100'.
        .... Invoke the script from OS command line as 'et -e
        asc/et_scripts_ratio_100/runet.atpg'.

    Skipping ATPG because option '-dont_run_atpg' was specified Completed scan compressibility analysis for module 'test'
```

- The following command performs an XOR-based compression analysis, with `wide1` masking, with an `broadcast` decompressor, and a ratio list of `"10 20 30"`.

```
    genus:root:>  analyze_scan_compressibility -chains 1 \
    -library  rules/tsmc13.v -ratios "10 20 30" -directory $outdir/newdirnr2
```

```
Starting scan compressibility analysis for module 'DLX_CORE'
using existing work directory: './newdirnr2'
Encounter(TM) Test product version information:
Encounter(R) Test and Diagnostics xxxx

Analyzing the design with the following configuration
-----------------------------------------------------
       Design       - DLX_CORE       Decompressor - broadcast       Compressor   - xor       mask         - wide1       ratio        - 10 20 30
Scan Registers Status
---------------------
       Replacing scan        Scan flip-flops mapped for DFT          1348        100.00%       (Number of Scannable Registers) / (Total Register Count) =
--> 1348 / 1348 = 100%
       (Number of flops mapped to scan) = 1348       (Number of flops passing tdrc)   = 1348
       (Number of Scannable Registers)       (Number of flops scan mapped and passing tdrc) +       (Number of flops in shift-register segments) +       (Number
of flops in abstract scan segments) =       --> 1348 + 0 + 0 = 1348
       (Total Register Count)       (Number of registers) +       (Number of flops in abstract scan segments) =       --> 1348 + 0 = 1348
       (*) The percentage of scannable registers is 100%.
Building the Fullscan Chains
----------------------------

       Building balanced scan chains assuming all test clocks are compatible and mixing of clock edges in scan chains       is allowed. If you prefer a
different set of constraints, please build the scan chains before calling       analyze_scan_compressibility.
       Configuring 1 chains for 1348 scan f/f              ...chain01 of length: 1348
       Modus scripts for ATPG have been written       for fullscan mode to directory './newdirnr2/et_scripts_fullscan'.
Inserting Scan Compression Logic
--------------------------------

       Compressing scan chains with ratio 10       Tool achieved ratio: 9.9
       Modus scripts for ATPG have been written       for compression mode with fullscan top-off to directory'./newdirnr2/et_scripts_ratio_10'.       ----------
------------------------------
       Compressing scan chains with ratio 20       Tool achieved ratio: 19.8
       Modus scripts for ATPG have been written       for compression mode with fullscan top-off to directory './newdirnr2/et_scripts_ratio_20'.       ---------
-------------------------------
       Compressing scan chains with ratio 30       Tool achieved ratio: 29.9
       Modus scripts for ATPG have been written       for compression mode with fullscan top-off to directory './newdirnr2/et_scripts_ratio_30'.
Running Modus ATPG
--------------------------
       Running ./newdirnr2/et_scripts_fullscan/runet.atpg .....       Running ./newdirnr2/et_scripts_ratio_10/runet.atpg .....       Running
./newdirnr2/et_scripts_ratio_20/runet.atpg .....       Running ./newdirnr2/et_scripts_ratio_30/runet.atpg .....
```

```
----------------------------------------
Results of analyze_scan_compressibility
----------------------------------------
       Design       - DLX_CORE       Decompressor - broadcast       Compressor   - xor       mask         - wide1
Achieved compression table with fullscan top-off vectors
-----------------------------------------------------------
IC    TATR   TDVR    ATCov.   CL       PAT-comp  PAT-fs    Cycles        Runtime    Gatecount    Area-------------------------------------------------------------
-----------------------------------------fs    1.0    1.0     99.99%   1348     -         305       414762        00:00:03   -            -  10    5.6    5.6
99.72%   135      301       22        73532         00:00:04   55           680.65699999999220    5.5    5.5     99.45%   68       324       37        75711
00:00:05   112          1357.919999999998330    4.4    4.4     97.83%   45       299       57        95084         00:00:13   155          1941.826000000001
Achieved compression table without fullscan top-off vectors
-----------------------------------------------------------
IC    TATR   TDVR    ATCov.   CL       PAT-comp  PAT-fs    Cycles-----------------------------------------------------------------------fs    1.0    1.0     99.99%
1348     -         305       41476210    9.8    9.8     99.72%   135      301       -         4221220    17.6   17.6    99.45%   68       324       -
23586 30    27.8   27.8    97.83%   45       299       -         14946

Total ATPG runtime: 00:00:25 (hrs:min:sec).

IC        - Inserted compressionTATR      - Test application time reductionTDVR      - Test data volume reductionATCov.    - ATPG test mode coverageCL.       -
Channel LengthPAT-comp  - Number of compression test patternsPAT-fs    - Number of fullscan test patternsRuntime   - ATPG runtimefs        - Uncompressed
(Fullscan) run
Completed scan compressibility analysis for module 'DLX_CORE'
```

- The following command performs MISR-based compression analysis, with `wide1` masking, a ratio list of `2`, sixteen scan chains.

```
genus:root:>  analyze_scan_compressibility -library mylibrary -chains 16 \
-compressor misr -ratios 2 -mask wide1 -fault_sample_size 5000 -dir asco
```

Related Information

Analyzing and Reporting Scan Compressibility in *Genus Design for Test Guide.*

Affected by these commands:	connect_scan_chains
	define_scan_chain
Related command:	report_scan_compressibility

analyze_test_points

```
analyze_test_points

[-goal string]
[-test_point_type string]
[-max_number_of_test_points integer]
[-test_coverage float]
[-max_num_control integer]
[-max_num_observe integer]
[-input_test_point_file string]
[-input_sharing_compatibility_file string]
[-prepare_for_analysis]
[-dont_run_analysis]
[-share_tp_flop integer]
[-share_observe_flop integer]
[-share_control_flop integer]
[-tp_flop_percent float]
[-dont_reset_attrs]
```

Description

This command analyzes test point based on goal by invoking modus.

Points to note:

- Option goal is only required when running test point analysis using MODUS
- Generates MODUS scripts for getting test point file and test point sharing file based on goal.
- Updates Genus database and attributes with test point data extracted from these files or user provided MODUS generated test point files.

- Runs Modus directly and interpret the test point files to prepare them for insertion via `add_analyzed_test_points` command

Options and Arguments

`-goal` *string*	Specifies the type of test point coverage (`lbist_coverage` or `atpg_pattern_count`)
`-test_point_type` *string*	Specifies the test point type.
`-max_number_of_test_points` *integer*	Specifies the maximum number of test points.
`-test_coverage` *float*	Specifies the test coverage goal.
`-max_num_control` *integer*	Specifies the maximum number of controllable test points.
`-max_num_observe` *integer*	Specifies the maximum number of observable test points.
`-input_test_point_file` *string*	Specifies the test point file from MODUS (skips running MODUS).
`-input_sharing_compatibility_file` *string*	Specifies the sharing compatibility file from MODUS(skips running MODUS).
`-prepare_for_analysis`	Specifies the force rerun of `prepare_for_test_point_analysis` command.
`-dont_run_analysis`	Specifies to generate MODUS scripts but does not run MODUS.
`-share_tp_flop` *integer*	Specifies the number of test points to share a flop. (Default: 4)
`-share_observe_flop` *integer*	Specifies the number of observable test points to share a flop.
`-share_control_flop` *integer*	Specifies the number of controllable test points to share a flop.
`-tp_flop_percent` *float*	Specifies the percentage of scan flops to be identified as test point flops. (Default: 1)
`-dont_reset_attrs`	Specifies not to reset existing test point attributes.

Examples

- The following command produces run_modus_analyze_tp.tcl and invokes Modus.

```
genus@root:> analyze_test_points -goal lbist_coverage
```

- The following command populates the Genus attributes using MODUS generated expt_1.tpfile

```
genus@root:> analyze_test_points -input_test_point_file expt_1.tpfile
```

Related Information

Related Commands
prepare_for_test_point_analysis
add_analyzed_test_points

Related Attributes
dft_testpoint_type
dft_testpoint_analyze_options
dft_testpoint_analyze_sharing_options
dft_testpoint_strength

check_atpg_rules

```
check_atpg_rules [-library string] [-compression]
[-directory string] [-unix_shell] [design]
```

Description

Generates a template script to run Modus to verify if the design and its test structures are ATPG-ready. More specifically, the generated script allows you to check for

- Tristate drivers for contention
  These conditions might cause manufacturing test problems
- Feedback loops
  Failure to break combinational feedback loops might cause reduced test coverage.
- Clock signal races
  Checks for any flip-flop with a potential race condition between its data and clock signal.
- Test clock control

This command generates the following files:

- `modus.exclude`—A file listing objects to be excluded from the ATPG analysis
- `modus.modedef`—A file describing the test mode when running ATPG in `assumed scan` mode
- *`topmodulename`*`.ASSUMED.pinassign--`A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock) and their test function used to build the testmode before actual scan chains exist in the design
- *`topmodulename`*`.FULLSCAN.pinassign--`A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock and scan data IOs) and their test function used to build the testmode when actual scan chains exist in the design
- If the ATPG-based testability analysis is run in compression mode, the following pin-assignment files are generated in addition to the *`topmodulename`*`.FULLSCAN.pinassign` file. In this case, All three files include the compression test signals with their appropriate test functions to validate their specific test mode:
  - *`topmodulename`*`.COMPRESSION_DECOMP.pinassign`—A file generated *only* when inserting XOR-based decompression logic
  - *`topmodulename`*`.COMPRESSION.pinassign`—A file generated to verify the broadcast-based decompression logic
- `runmodus.tsv.tcl`—A template script file for Modus to verify the design and its test structures.

Options and Arguments

`-compression`	Performs additional checks if the design has compression logic.
`design`	Specifies the name of the top-level design for which you want to check if it ATPG-ready.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *`string`*	
	Specifies the working directory for Modus.
	If the script files already exist, rerunning the command will overwrite them.
	Default: `./check_atpg_rules`
`-library` *`string`*	Specifies the list of Verilog structural library files, for example, *`file1 file2`*. Refer to the `write_dft_atpg` -library option description for additional information.

	This option is only required when you invoke this command on a mapped netlist.
`-unix_shell`	Writes the output in shell format.

check_dft_pad_cfg

```
check_dft_pad_cfg [design] [> file]
```

Description

Checks and reports the data direction control of the pad logic for the test I/O ports.

Options and Arguments

design	Specifies the name of the top-level design to be checked. You should specify this name in case you have multiple top designs loaded.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
file	Specifies the file to which to redirect the detailed output.
	If no file is specified, the output is written to standard out (`stdout`) and to the log file.

Related Information

Checking the Pad Configuration in *Genus Design for Test Guide*.

check_dft_rules

```
check_dft_rules [design] [-advanced]
[-max_list_violations integer] [> file]
[-max_list_registers integer]
[-max_list_fanin integer][-verbose]
[-dft_cfg_mode dft_config_mode_name]
```

Description

Evaluates the design for DFT-readiness. Flip-flops that pass the DFT rule checks are later mapped to scan flip-flops during synthesis and included in a scan chain during scan connection. Flip-flops that fail the DFT rule checks `and flip-flops` marked with either a `dft_dont_scan` attribute or a `preserve` attribute, or flip-flops instantiated in lower-level blocks marked with a `preserve` attribute, are not mapped to scan flip-flops and are `excluded from the scan chains.` The DFT rule checker also analyzes the libraries and reports on the valid scan cells, and updates the DFT database with all the necessary information about newly added DFT logic and/or DFT signals. Therefore it is considered essential to run the DFT rule checker after every major DFT insertion or definition step.

Table 15-1 lists the DFT rule violations that this command checks.

DFT Rule Violation	Checked?	Auto-Fixed?
Uncontrollable asynchronous set or reset signals	Checked	Yes
Gated clocks and derived clocks	Checked	Yes
Flip-flop's clock port connected to tied lines	Checked	Yes
Conflicting clock and asynchronous set or reset signals	Checked	No
Tristate contention for internal and external nets	Checked(*)	Yes
Same asynchronous set or reset data race conditions	Checked(*)	Yes

Clock and data race conditions	Checked(*)	No
Floating nets violations	Checked(*)	No
X-source violations	Checked(*)	Yes
Floating conditions	Not checked	No

(*) indicates that the check is performed using the `-advanced` option.

To maximize fault coverage, you should try to fix any DFT rule violations, so that all flip-flops can be included in a scan chain. You can either modify the RTL or use the DFT fix capabilities using the fix_dft_violations command.

To include the RTL file name and line number at which the DFT violation occurred in the messages produced by `check_dft_rules`, set the `hdl_track_filename_row_col` root attribute to `true` before elaboration.

Options and Arguments

`-advanced`	Specifies to perform checking for tristate contention, same asynchronous set or reset data race conditions, clock and data race conditions, x-source generators, and floating nets violations.
design	Specifies the name of the top-level design to be checked. You should specify this name in case you have multiple top designs loaded.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dft_cfg_mode` *dft_configuration_mode_name*	
	Specifies the object name of the scan mode to be checked. A scan mode is defined using the define_dft_cfg_mode command.
	Scan modes can be used to build the top-level scan chains with specific elements in different modes of operation (multi-mode), or when concatenating default scan chains into a single longer scan chain in a different scan mode of operation. After finishing the execution of this command, `report_dft_trace_back` will use this mode by default for all further tracing until another `check_dft_rules` or `set_dft_configuration_mode` is run.
	`check_dft_rules` run as a part of a flow is less verbose by default, to save logfile space. Thus, after having run `check_dft_rules`, use `report_dft_violations` when a more verbose report is needed.
file	Specifies the file to which to redirect the detailed output.
`-max_list_fanin` *integer*	
	Limits the number of pins and ports reported in the fanin cone for any violation.
	This option affects DFT checks performed for same asynchronous set or reset data race conditions, clock and data race conditions, and x-source generators.
	Default: `20`
`-max_list_registers` *integer*	
	Limits the number of registers reported for any violation.
	Default: `20`
`-max_list_violations` *integer*	
	Controls the number of DFT violations for which the details are printed to the screen and log file. Specify `-1` to write the details of all violations to the log file.
	Default: `20`
`-verbose`	Produces a detailed report.

Examples

- The following example defines the shift-enable signal and its active polarity, then runs the DFT rule checker. The output of the `check_dft_rules` command is written to the `DFT.rules` file. The return value of the command (2) corresponds to the number of violations found.

```
define_test_signal -function shift_enable scan_en -active high
check_dft_rules > DFT.rules
  Checking DFT rules for 'top' module under 'muxed_scan' style

  Checking DFT rules for clock pins
  ...
  Checking DFT rules for async. pins
  ...
Detected 2 DFT rule violation(s)
    ... see the log file for more details
Number of user specified non-Scan registers:   0
      Number of registers that fail DFT rules:   4
      Number of registers that pass DFT rules:   1
  Percentage of total registers that are scannable: 20%
sh more DFT.rules

Checking DFT rules for 'top' module under 'muxed_scan' style
Processing techlib techlib_25
  Identified a valid scan cell 'SDFFHQX1'
         active clock edge:  rising
  Identified a valid scan cell 'SDFFHQX2'
         active clock edge:  rising
...
Identified 60 valid usable scan cells
Detected 2 DFT rule violation(s)
        Summary of check_dft_rules
        **************************
        Number of usable scan cells: 60
Clock Rule Violations:
----------------------
        Internally driven clock net: 1
            Tied constant clock net: 0
                 Undriven clock net: 0
        Conflicting async/clock net: 0
                    Misc. clock net: 0

Async. set/reset Rule Violations:
---------------------------------
        Internally driven async net: 1
              Tied active async net: 0
                 Undriven async net: 0
                    Misc. async net: 0

   Total number of DFT violations: 2
Clock Violation
# 0: internal or gated clock signal in module 'top', net 'Iclk', inst/pin 'g4/z' (file: test3.v, line 11) [CLOCK-05]
  Effective fanin cone:
    clk
    en
Async Violation
# 1: async signal driven by a sequential element in module 'top', net 'Iset', inst/pin 'Iset_reg/q' (file: test3.v, line 13) [ASYNC-05]
  Effective fanin cone:
    Iset_reg/q
        Violation # 0 affects  4 registers
```

```
        Violation # 1 affects  4 registers

        Note - a register may be violating multiple DFT rules
Total number of Test Clock Domains: 1
  DFT Test Clock Domain: clk
        Test Clock 'clk' (Positive edge) has   1 registers
Number of user specified non-Scan registers:   0
    Number of registers that fail DFT rules:   4
    Number of registers that pass DFT rules:   1
Percentage of total registers that are scannable: 20%
```

- The following example shows tristate net checking with the `check_dft_rules -advanced` option.

```
check_dft_rules -advanced
Checking DFT rules for 'test' module under 'muxed_scan' style
    Processing techlib tsmc_25 for muxed_scan scan cells
    Identified 60 valid usable scan cells
  Checking DFT rules for clock pins
Info    : Added DFT object. [DFT-100]
        : Added test clock domain 'clk'.
Info    : Added DFT object. [DFT-100]
        : Added test clock 'clk'.
  Checking DFT rules for async. pins
  Checking DFT rules for shift registers.
  Checking DFT rules for tristate nets.
  Checking DFT rules for clock data race conditions.
  Checking DFT rules for set reset data race conditions.
  Checking DFT rules for x-sources.
Detected 1 DFT rule violation(s)
        Summary of check_dft_rules
        **************************
        Number of usable scan cells: 60
Clock Rule Violations:
----------------------
          Internally driven clock net: 0
              Tied constant clock net: 0
                   Undriven clock net: 0
        Conflicting async & clock net: 0
                      Misc. clock net: 0

Async. set/reset Rule Violations:
---------------------------------
        Internally driven async net: 0
              Tied active async net: 0
                 Undriven async net: 0
                    Misc. async net: 0

Advanced DFT Rule Violations:
-----------------------------------
         Tristate net contention violation: 1
        Potential race condition violation: 0
                        X-source violation: 0

   Total number of DFT violations: 1


Warning : DFT Tristate net contention Violation. [DFT-315]
        : # 0 <vid_0_tristate_net>: tristate net 'tbus' connected to pin
 'out_reg/d' potentially driven by conflicting values   [TRISTATE_NET-01]
 : To remove the net contention violation in scan-shift mode, either modify the RTL, or use the '-tristate_net' option of the
```

```
'fix_dft_violations' command.

    Tristate net drivers:
      i_block2/g1/z
      i_block1/g1/z
      i_block3/g1/z

    Violations sorted by type and number of affected registers
    Note - a register may be violating multiple DFT rules.

        There are '1' tristate net violations.

---------------------
```

Related Information

See the following sections in *Design for Test Guide*:

- Running the DFT Rule Checker
- Advanced DFT Rule Checking
- Defining Scan Configuration Modes
- Concatenating Scan Chains

Affected by these constraints:	define_scan_abstract_segment
	define_dft_cfg_mode
	define_test_clock
	define_test_signal -function <shift_enable | test_mode>
Affects these commands:	connect_scan_chains
	fix_dft_violations
	report_scan_registers
	syn_map
	syn_opt
Affected by these attributes:	dft_controllable
	dft_dont_scan
	dft_identify_test_signals
	dft_identify_top_level_test_clocks
	dft_scan_style
	preserve
	dft_identify_xsource_violations_from_timing_models
Sets these attributes:	dft_status
	dft_violation
	type
	violation Attributes

check_dft_setup

```
check_dft_setup
[-2d_compression] [design] [-compression]
```

Description

Checks the DFT setup of the specified features for correctness and compatibility.

For the Unified Compression flow, use `help dft_compression_*` command to see all available attributes of that flow.

Options and Arguments

`-2d_compression`	Checks the DFT setup of 2D compression.
`-compression`	Checks the DFT setup of 1D compression.
design	Specifies the design for which to check the DFT setup.

Related Information

Affects this command:	add_test_compression

compress_scan_chains

```
compress_scan_chains {-ratio integer | -channel_length integer]
[-chains actual_scan_chain]... [-auto_create] [-power_aware] [-decompressor {broadcast | xor [-spread_enable test_signal]}] [-compression_enable test_signal] [-target_period integer] [-master_control test_signal] [ -compressor xor -mask {wide1|wide2} [-mask_clock {port|pin|hpin} [-allow_shared_clocks]] [-mask_load test_signal] [-mask_enable test_signal] [-create_mask_or_misr_chain -mask_or_misr_sdi {port|pin|hpin} -mask_or_misr_sdo {port|pin|hpin} [-shared_output]] [-mask_sharing_ratio integer] [-apply_timing_constraints [-timing_mode_names mode_list]] [-write_timing_constraints file] [-low_pin_compression [-low_pin_comp_control shift_enable] [-shift_enable shift_enable]] | -compressor misr [-serial_misr_read [-misr_obs test_signal]] [-misr_clock {port|pin|hpin}] {-misr_reset_enable test_signal | -misr_reset_clock test_signal} [-misr_read test_signal | -use_all_scan_ios_unidirectionally] [-misr_shift_enable test_signal] [-mask {wide0 | wide1 | wide2} [-mask_sharing_ratio integer  [-mask_clock {port|pin|hpin} [-allow_shared_clocks]] [-mask_load test_signal] [-mask_enable test_signal_list] [-create_mask_or_misr_chain -mask_or_misr_sdi {port|pin|hpin} -mask_or_misr_sdo {port|pin|hpin} [-shared_output]] | -compressor hybrid [-misr_bypass test_signal] [-serial_misr_read] [-misr_obs test_signal] [-misr_clock {port|pin|hpin}] {-misr_reset_enable test_signal | -misr_reset_clock test_signal} [-misr_shift_enable test_signal] [-mask {wide0 | wide1 | wide2}] [-mask_sharing_ratio integer] [-mask_clock {port|pin|hpin}] [-mask_load test_signal] [-mask_enable test_signal_list] [-create_mask_or_misr_chain -mask_or_misr_sdi {port|pin|hpin} -mask_or_misr_sdo {port|pin|hpin} [-shared_output]]] | -compressor smartscan_xor -smartscan_ratio integer  [-gate_shared_compression_clock] [-smartscan_no_update_stage] [ -smartscan_serial_only | -smartscan_serial_scan_ins {ports|pins|hpins}] [-smartscan_parallel_access test_signal] [-smartscan_enable test_signal] [-smartscan_pulse_width_multiplier {1|2|4}] [-mask {wide1|wide2}] [-mask_clock {port|pin|hpin}] [-mask_load test_signal] [-mask_sharing_ratio integer] [-apply_timing_constraints [-timing_mode_names mode_list]] [-write_timing_constraints file] ] [-dont_exploit_inout_scan_io] [-preview] [-inside {inst|hinst}] [design] [-jtag_control_instruction jtag_instruction] [-allow_multiple_jtag_control]] [-share_mask_enable_with_scan_in]
```

Adds decompression and compression logic to reduce the effective length of the actual scan chains. This is usually invoked in a top-down design flow that does not use hierarchical compression and requires flat ATPG.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-allow_multiple_jtag_control`	
	When controlling the compression testmode from a JTAG macro, (that is, the `-jtag_control_instruction` option is specified), `compress_scan_chains` checks for the presence of other compression macros that are also JTAG controlled. Genus-DFT does not automatically support such a configuration and therefore insertion of a second JTAG controlled compression macro is disallowed by default. Specify this option to bypass this check and insert additional JTAG controlled compression macros. When this option is specified, the tool assumes you will manually perform any additional stitching needed and will appropriately modify any files generated for Modus.
	You must also specify the `-jtag_control_instruction` option with this option.
`-allow_shared_clocks`	
	Allows the mask or `MISR` clock to be shared with an existing full-scan test clock.
	The mask or `MISR` clock can only be shared with a test clock if you added gating logic which prevents the scan flops from pulsing during the channel mask load or `MISR` reset sequences. Since functional clocks are typically used for scanning, this requirement means that the functional clocks must be gated during test.
	When specified with the `-auto_create` option, a `-mask_load` pin is automatically created. The `mask_load` pin must additionally be used to gate off the clock being shared. If this gating logic is not added, the mask loading or MISR reset procedure will corrupt the test data in the design.
`-apply_timing_constraints`	
	Applies timing constraints to the appropriate compression control signals to prevent the mapper from considering these paths for timing optimization.
	Timing constraints will be applied in all user-specified timing modes.
	If your design has multiple timing modes but you did not specify the `-timing_mode_names` option to list the timing modes for which to write the constraints, no additional constraints are applied.
`-auto_create`	Automatically creates the necessary test pins as top-level ports.
	If you omitted any of the following options `-compression_enable`, `-spread_enable`, `-mask_clock`, `-mask_load`, `-mask_enable`, `-mask_or_misr_sdi`, `-mask_or_misr_sdo`, `-misr_clock`, `-misr_obs`, `-misr_reset_enable`, `-misr_read`, `-misr_bypass`, `-smartscan_enable`, and `-smartscan_parallel_access`, the appropriate ports will be created and named using the following format:
	`prefixOption`
	For example, *`prefix`*`compression_enable`, where *`prefix`* is the value of the `dft_prefix` root attribute.
`-chains` *`actual_scan_chain...`*	
	Specifies the names of the actual scan chains to be compressed.
	By default all actual scan chains are compressed.
	When inserting `MISR` logic, all scan chains must eventually be compressed.
{`-channel_length` *`integer`* | `-ratio` *`integer`*}	

	Controls the maximum length of the internal scan channels. You can either specify the maximum length directly or the tool can derive the length of the internal scan channels by dividing the longest actual scan chain length by the ratio.
	The `-channel_length` option is recommended for the hierarchical compression flow.
`-compression_enable` *`test_signal`*	
	Specifies the name of the test signal that enables configuring the actual scan chains in compression mode.
	If you do not specify the `-auto_create` or `-jtag_control_instruction` options, this test signal must have been defined using the `define_test_signal -function test_mode` command. If you request to build the compression logic with a master control signal (`-master_control`), the input port driving the compression enable signal can be an existing functional pin (specified through the `-shared_in` option of `define_test_signal -function test_mode`). If you do not specify the `-master_control` option, you must define the compression enable signal without the `-shared_in` option.
`-compressor {xor | misr| hybrid | smartscan_xor}`	
	Specifies the type of compression logic to be built:
	• `xor` specifies to build an XOR-based compressor
	• `misr` specifies to build a `MISR-based` compressor
	• `hybrid` specifies to build a `MISR` compression with `MISR` bypass capability. Bypassing the MISR allows you to perform compression using just the XOR compressor.
	• `smartscan_xor` specifies to include smartscan logic in the compression macro.
	Default: xor
`-create_mask_or_misr_chain`	
	Specifies to build a separate full-scan chain for the mask and `MISR` registers.
	To place the mask or `MISR` registers in a separate chain, an additional scan data input pin and scan data output pin are needed. You can either specify these pins using the `-mask_or_misr_sdi` and `-mask_or_misr_sdo` options, or make sure that the `-auto_create` option is specified.
`-decompressor {broadcast | xor}`	
	Specifies the type of decompression logic to be built:
	• `xor` specifies to build an XOR-based spreader network in addition to the broadcast-based decompression logic
	• `broadcast` specifies to build a broadcast-based decompression logic (simple scan fanout).
	Default: broadcast
`design`	Specifies the name of the top-level design whose scan chains must be compressed. You should specify this name in case you have multiple top designs loaded.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_exploit_inout_scan_io`	
	Prevents bidirectional scan compression.
`-gate_shared_compression_clock`	

	Specifies that the clock to the compression macro is shared with the test clock to the scan chains and that the tool must insert gating logic on the test clock path. The test clock will be gated using the `TEST_CLOCK_ENABLE` pin on the compression macro.
	The `TEST_CLOCK_ENABLE` pin will always exist on the smartscan compression macro and can be used for the clock gating. If this gating logic is not added, the scan chains will be disturbed during the mask load and/or smartscan register load operations.
	If you omit this option, you must manually insert the gating logic. The tool will print a message to warn you that this connection must be made but will not perform any subsequent checks to ensure the gating logic has been correctly added.
`-inside {`*`inst`*`|`*`hinst`*`}`	
	Specifies the instance in which to instantiate the compression logic.
	By default, the compression logic is inserted as a hierarchical instance in the top-level of the design.
`-jtag_control_instruction` *`jtag_instruction`*	
	Specifies which JTAG instruction will be used to target the compression macro's test data register.
`-low_pin_comp_control` *`shift_enable_signal`*	
	Specifies a shift-enable signal used to control low pin count compression.
	You cannot specify the same shift-enable signal for both the `-low_pin_comp_control` and `-shift_enable` options.
	If you omit this signal, the tool will use one of the default shift-enable signals.
	If no shift-enable pin exists, the tool creates an LPC_CONTROL shift-enable pin if the `-auto_create` option is specified.
`-low_pin_compression`	
	Reduces the number of compression control pins required by using encoded control signals.
`-mask {wide0 | wide1 | wide2}`	
	Inserts scan channel masking logic of the specified type.
	The masking types that can be used depend on the compressor type specified with the `-compressor` option.
	By default, no masking logic is inserted.
	The syntax indicates which types are available for each of the compressor types.
`-mask_clock {`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies the clock that controls the mask registers.
	The input port associated with this option can be an existing functional pin. This clock cannot be shared with an existing full-scan test clock pin unless you also specify the `-allow_shared_clocks` option.
`-mask_enable` *`test_signal`*	
	Specifies the name of the test signal that controls whether mask bits should be applied during the current scan cycle.
	For `wide2` masking, two mask enable signals must be specified.
	This option cannot be specified when the `-compressor` option is set to `smartscan_xor`.

	If you do not specify the -auto_create option, this test signal must have been defined using the define_test_signal -function test_mode command. The input port driving this test signal can be an existing functional pin (specified through the -shared_in option of the define_test_signal -function test_mode command).
-mask_load *test_signal*	
	Specifies the name of the test signal that enables loading of the mask data into the mask data registers.
	If the mask_clock is dedicated (that is, is only used for mask register loading), this signal is not needed. If this signal is shared (is used to clock the MISR or other logic in the circuit), this signal is needed to gate non mask load clock pulses from corrupting the mask registers. If the mask_clock is shared with other logic, you can use this signal to protect the shared logic from corruption during the mask load sequence.
-mask_or_misr_sdi {*port*|*pin*|*hpin*}	
	Specifies the scan data input pin or port of the mask or MISR chain.
	The input port associated with this option can be an existing functional pin.
-mask_or_misr_sdo {*port*|*pin*|*hpin*}	
	Specifies the scan data output pin or port of the mask or MISR chain.
	If the output port associated with this option is an existing functional pin, you must specify the -shared_output option.
-mask_sharing_ratio *integer*	
	Specifies the number of internal scan channels sharing a mask register. The specified integer may not exceed the value specified for the compression ratio.
	This option is only valid with wide1 and wide2 masking.
-master_control *test_signal*	
	Specifies the master control signal that gates the compression enable signal used for compression.
	This test signal must be dedicated for test and must have been defined using the define_test_signal -function test_mode command.
-misr_bypass *test_signal*	
	Specifies the test signal used to bypass the MISR-based logic. This test signal is required in hybrid compression mode.
	If you do not specify the -auto_create or -jtag_control_instruction options, this test signal must have been defined using the define_test_signal -function test_mode command. The input port driving this test signal can be an existing functional pin (specified through the -shared_in option of the define_test_signal -function test_mode command).
-misr_clock {*port*|*pin*|*hpin*}	
	Specifies the clock that controls the MISR registers.

	This input port cannot be shared with an existing full-scan test clock unless it is only used to accumulate the MISR signature or if the `-allow_shared_clocks` option is specified. The `-misr_clock` option is only used to accumulate the MISR signature if there are separate `-mask_clock` and `-misr_reset_clock` signals specified.
`-misr_obs` *`test_signal`*	
	Specifies the test signal used to select Serial MISR Read. This is required when the `-serial_misr_read` option is specified unless `-auto_create` or `-jtag_control_instruction` is also specified.
	You must also specify the `-serial_misr_read` option with this option.
`-misr_read` *`test_signal`*	
	Specifies the test signal to configure any bidirectional scan I/O pads for MISR compression.
	This option is mutually exclusive with the `-use_all_scan_ios_unidirectionally` option. Using scan I/O bidirectionally during MISR compression is only available with the -compressor misr option. When using the `-compressor hybrid` option, all scan I/O are used unidirectionally.
`-misr_reset_clock` *`test_signal`*	
	Specifies a separate dedicated test signal that is used to asynchronously reset the MISR.
	This option is mutually exclusive with the `-misr_reset_enable` option.
`-misr_reset_enable` *`test_signal`*	
	Specifies the test signal used to reset the `MISR` registers.
	Notes:
	• This option is mutually exclusive with the `-misr_reset_clock` option.
	• If you do not specify the -`auto_create` option, this test signal must have been defined using the `define_test_signal -function test_mode` command. The input port driving this test signal can be an existing functional pin (specified through the `-shared_in` option of the `define_test_signal -function test_mode` command).
`-misr_shift_enable` *`test_signal`*	
	Specifies the test signal used to enable MISR accumulation during scan shifting.When this signal is de-asserted, the contents of the MISR register will not change.
	If this option is omitted, the default shift-enable test signal for the design is used. If this option is specified, you must have defined this test signal using the `define_test_signal -function shift_enable` command. The input port driving this test signal can be an existing functional pin (specified through the `-shared_in` option of the `define_test_signal -function shift_enable` command).
`-power_aware`	Ensures that the mux logic added during compression obeys the boundaries of the power domains defined in the CPF file.
`-preview`	Reports the requested ratio, the maximum original scan chain length, the maximum subchain length, and the number of internal scan channels that would be created without making modifications to the netlist. Use this option to verify your compression architecture prior to inserting the compression logic.
`-serial_misr_read`	Specifies to include support for reading MISR bits serially through the scan data pins.

`-share_mask_enable_with_scan_in`	
	Allows to share the mask enable port with a scan data in port. When you specify this option you enable insertion of an asymmetrical compression macro.
`-shared_output`	Specifies that the scan data output port of the dedicated mask or `MISR` chain must be shared with a functional port.
	The tool creates the additional logic required to share the port.
`-shift_enable` *shift_enable*	
	Specifies the shift-enable signal to be used for low pin count compression.
	If you omit this option, the tool will use the default shift-enable signal.
`-smartscan_enable` *test_mode_signal*	
	Specifies the test mode signal that enables the smartscan mode. You can omit this option if you specified the `-auto_create` option. The tool will create the `smartscan_enable` port on the compression macro and connect it to the primary input for this test mode signal.
	When this signal is active it ensures that the smartscan mode is ON. Currently when this signal is inactive, the smartscan flops are part of the fullscan chains only. When inactive, and in the compression modes, the smartscan flops are *not* part of the compression channels.
`-smartscan_no_update_stage`	
	Prevents the insertion of update registers between the deserializer and the decompressor. In this case, lockup latches are inserted between the deserializer flops and the decompressor.
	You cannot specify this option when you have set the `-smartscan_pulse_width_multiplier` option to either `2` or `4`.
	By default, the tool inserts update registers between the deserializer and the decompressor.
`-smartscan_parallel_access` *test_mode_signal*	
	Specifies the test mode signal to use for parallel access to the smartscan flops. You can omit this option if you specified the `-auto_create` option. The tool will create the `smartscan_parallel_access` port on the compression macro and connect it to the primary input for this test mode signal.
`-smartscan_pulse_width_multiplier {1|2|4}`	
	Determines whether to add clock divider logic to widen the clock pulse going to the scan chains. You can specify the following values: • 1—no logic added • 2—increases the scan path through the SmartScan clock controller with 1 bit • 4—increases the scan path through the SmartScan clock controller with 2 bits
	Default: `1`
`-smartscan_ratio` *integer*	
	Specifies the number of parallel scan data input pins that correspond to a single serial scan data input pin. The number of defined (fullscan) chains must be an integral multiple of the specified smartscan ratio.
`-smartscan_serial_only`	
	Specifies to only insert the smartscan serial-only interface. The number of deserializer (and serializer) registers will match the number of the defined chains. When building the model for Modus (during the `build_model` step in the `write_dft_atpg` scripts), the tool will create the pseudo pins for the parallel interface.
`-smartscan_serial_scan_ins {`*port*`|`*pin*`|`*hpin*`}`	
	Specifies the pins or ports to be used as serial scan data inputs for the SmartScan serial and parallel flow.
	The scan data outputs of the scan chains starting at these scan data inputs will be used as serial scan data outputs.
	The number of pins or ports specified must match the number of serial scan data inputs required for the SmartScan architecture as determined by the number of fullscan chains and the `-smartscan_ratio`. You cannot specify a pin or port that is not a scan data input of any of the scan chains that are part of the compression. Make sure that all scan data input ports for OPCG-side scan chains are specified with this option.

	This option cannot be specified together with the `-smartscan_serial_only` option.
`-spread_enable` *`test_signal`*	
	Specifies the name of the test signal that enables applying the input test data to an XOR-based spreader network.
	Use this option when `-decompressor` is set to `xor`.
	If you do not specify the `-auto_create` or `-jtag_control_instruction` options, this test signal must have been defined using the `define_test_signal -function test_mode` command. The input port driving this test signal can be an existing functional pin (specified through the `-shared_in` option of the `define_test_signal -function test_mode` command).
`-target_period` *`integer`*	
	Specifies a target clock period (in picoseconds) used to optimize the compression macro. If the value zero (`0`) is specified, synthesis is performed with a low effort compile, and without applying external (input/output delay) constraints.
	Default: value for `test_clock` period of the scan chains being compressed
`-timing_mode_names` *`mode_list`*	
	Specifies the timing modes for which to generate the additional timing constraints that apply to the compression control signals.
	The timing modes are taken into account when you specify the `-apply_timing_constraints` and `-write_timing_constraints` options.
	This applies only to multi-mode designs. Modes are created with the `create_mode` command.
`-use_all_scan_ios_unidirectionally`	
	Disables use of bidirectional scan I/O for a `MISR-based` compressor.
	This option is mutually exclusive with the `-misr_read` option.
`-write_timing_constraints` *`file`*	
	Specifies the file to which to write the timing constraints applied to the appropriate compression control signals to prevent the mapper from considering these paths for timing optimization. The timing constraints are not written if you specify the `-preview` option.
	Timing constraints will be applied in all user-specified timing modes.
	If your design has multiple timing modes but you did not specify the `-timing_mode_names` option to list the timing modes for which to write the constraints, constraints for all modes are written to the file.

Examples

- The following command requests XOR-based compression logic without masking, and requests creation of the necessary ports for the required test signals. In this case only a compression enable signal is required and thus one test port is created.

```
genus:root:> compress_scan_chains -ratio 5 -compressor xor -auto_create
Will create a test port for 'compression_enable'
Checking out license 'Encounter_Test_Architect'... (1 seconds elapsed)
...
```

- The following command requests XOR-based compression logic with masking logic of type `wide1`, and requests creation of the necessary ports for the required test signals. In this case three extra test signals are required for the masking logic and thus four test ports are created.

```
genus:root:> compress_scan_chains -ratio 5 -compressor xor -mask wide1 -auto_create
Will create a test port for 'compression_enable'
Will create a test port for 'mask_load'
Will create a test port for 'mask_enable'
Will create a test port for 'mask_clock'
Checking out license 'Encounter_Test_Architect'... (1 seconds elapsed)
....
```

- The following command requests an XOR-based compression with masking logic of type `wide1`, with decompression logic of type `xor`, and requests creation of the necessary ports for the required test signals. Compared to the second example, one extra test signal— the `spread_enable` signal—is required for the decompression logic, and thus five test ports are created.

```
genus:root:> compress -ratio 5 -compressor xor -decompressor xor -mask wide1 \ -auto_create
Will create a test port for 'compression_enable'
Will create a test port for 'spread_enable'
Will create a test port for 'mask_load'
Will create a test port for 'mask_enable'
Will create a test port for 'mask_clock'
Checking out license 'Encounter_Test_Architect'... (2 seconds elapsed)...
```

- The following command requests a `MISR`-based compression with masking logic of type `wide1`, and requests creation of the necessary ports for the required test signals. Compared the two second example, one extra test signal—the `misr_reset_enable` signal—is required to reset the `MISR` registers, and thus five test ports are created.

```
genus:root:> compress -ratio 5 -compressor misr -mask wide1 -auto_create
Will create a test port for 'compression_enable'
Will create a test port for 'misr_reset_enable'.
Will create a test port for 'misr_clock'
Info - Defaulting the misr shift enable signal to the default shift_enable test_signal:dft/SE
Will create a test port for 'mask_load'
Will create a test port for 'mask_enable'
Info - will share the 'misr_clock' and the 'mask_clock' ...
```

Related Information

See the following sections in *Genus Design for Test Guide*:

- Inserting Test Compression Logic
- Low Pin Count Compression Using Encoded Compression Signals
- Reducing Pin Count for Compression
- Using Asymmetrical Scan Compression

Affected by these constraints:	define_jtag_instruction
	define_test_signal -function <shift_enable | test_mode>
Affected by these commands:	connect_scan_chains
Affects this command:	report_scan_chains
Sets these attributes:	compressed

	dft_compression_signal
	dft_mask_clock
	dft_misr_clock
	type

connect_dft_hier_test_cores

```
connect_dft_hier_test_cores [-scan_in_pipeline_depth integer]
 [-scan_out_pipeline_depth integer] [-inside hinst|design]
 [-mask_enable_pipeline_depth integer] [-debug] [design]
```

Description

Connects the top-level test signals to the top level Test Interconnect Macro and the test signals defined for the scan abstracted cores as per the scan mapping defined by the user in the Hierarchical Test flow. This command is run after inserting the test compression logic, defining scan mapping and concatenation order and is one of the final processing steps before writing the Modus database in the Hierarchical Test Flow: Integrating the Top level.

Options and Arguments

`-scan_in_pipeline_depth integer`	
	Specifies the length of the input pipeline.
`-scan_out_pipeline_depth integer`	
	Specifies the length of the output pipeline.
`-inside <hinst|design>`	
	Specifies the insert logic inside hierarchy.
`-mask_enable_pipeline_depth integer`	
	Specifies the core channel mask enable pipeline depth.
`-debug`	Enables debug
`design`	Specifies the name of the design for which to connect the top-level test interconnect macro. If you omit the design name, the top-level design of the current directory of the design hierarchy is used. This option is required if multiple designs are loaded.

connect_opcg_segments

```
connect_opcg_segments
[-chains actual_scan_chains] [-side_scan_only]
[-use_ports_for_side_scan_connections port_list]
[-preview] [design]
```

Description

Connects the scan segments associated with the OPCG logic (OPCG programming registers and OPCG domain macros) into the actual scan chains so that OPCG side chains are created in order to load/program the OPCG. You must run this command after you have connected the scan chains.

There is no need to run this command if compression has been inserted using `add_test_compression` when a `test_signal` with function `opcg_load` was present.

Options and Arguments

design	Specifies the name of the design for which to connect the OPCG segments.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
-chains *actual_scan_chains*	
	Specifies the names of the actual scan chains to which the OPCG segments must be prepended.
-preview	Shows the potential connections for the OPCG segments, without making any modifications to the netlist.
-side_scan_only	Connects just the OPCG side-scan while the OPCG scan segments are included in the scan chains by executing `connect_scan_chains -include_opcg_segments`.
-use_ports_for_side_scan_connections *port_list*	
	Specifies the ports to be used as scan data input pins when making the connections for the side scan chains.
	Use this option for RTL inserted scan compression flows in which the actual chains are built with respect to scan compression channels and not as fullscan chains. In this flow, the specified ports will be used to connect the side scan chains instead of using the internal scan-data input pins specified to build the scan compression channels.
	You can also use this option in a flow in which you build your fullscan chains followed by scan compression. The specified ports will be used to connect the side scan chains instead of using the scan-data input pins defined for the fullscan chains.

Related Information

Connecting the OPCG Segments in *Genus Design for Test Guide*

Affected by these commands	connect_scan_chains
	add_opcg

connect_scan_chains

```
connect_scan_chains [design]
[-preview]
[-preview_scan_element_order][-auto_create_chains] [-allow_physical_balancing]
[-incremental][-chains chain_list]
[-elements element_list]
[-keep_connected_shift_enable]
[-dont_exceed_min_number_of_scan_chains]
[-pack | -create_empty_chains]
[-dft_cfg_mode dft_config_mode_name]
[-physical][-cluster_aggressively_high]
[-update_placement][-include_opcg_segments]
[-power_domain power_domain_list][-zipper_stitch]
```

`[-partitions string]`

Description

Configures and connects scan flip-flops which pass the DFT rule checks into scan chains. This command works at the current level of the hierarchy and all lower hierarchies instantiated in this module. The design must be mapped to the target library before connecting scan chains in a design.

The command returns the number of scan chains that the scan configuration engine creates (or would create if you use the `-preview` option).

Options and Arguments

`-auto_create_chains`	Allows the scan configuration engine to add new chains that are not defined through a `define_scan_chain` constraint.
	Without this option, the scan configuration engine reports an error if it needs more scan chains than have been defined with the `define_scan_chain` command.
`-allow_physical_balancing`	Gives more priority to balance chains without loosing physical scan benefit.
`-chains chain_list`	Connects only the specified user-defined chain names. If the list is empty, none of the user-defined chains can be connected at this time. New chains are created if you specify the `-auto_create_chains` option.
	The specified user-defined chains must have been defined using a `define_scan_chain` constraint.
	If omitted, all user-defined chains can be connected.
`-cluster_aggressively_high`	
	Uses the physical location of the scan flops to build scan chains for improved scan chain balancing and scan wirelength reduction. This option must be specified with the `-physical` option.
`-create_empty_chains`	
	Allows the scan configuration engine to create empty scan chains by making a direct connection from their scan data input to their scan data output if the number of scan chains to be configured is less than the minimum number of scan chains required in the design.
	If this option is not specified, the scan configuration engine will move scan flops between compatible chains to satisfy the minimum number of scan chains requirement. This can result in configured scan chains having a sequential depth of one element. Do not use this option when configuring scan chains to be used as internal scan channels that are loaded and unloaded using on-chip compression logic.
`-dft_cfg_mode dft_configuration_mode_name`	
	Specifies the scan mode for which to build the scan chains.
`-dont_exceed_min_number_of_scan_chains`	
	Specifies to use the exact same number of scan chains as specified by the `dft_min_number_of_scan_chains` attribute when building the scan chain.
design	Specifies the name of the top-level design to be checked. You should specify this name in case you have multiple top designs loaded.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-elements element_list`	
	Considers only the specified elements for scan chain connection. An element can be a flip-flop, segment, or a hierarchical instance.
	If you specify a hierarchical instance, all flops in this hierarchical instance that pass the DFT rule checker and that are mapped to scan for DFT, will be added to the chains.

	If some of the scan flops in a hierarchical instance belong to a segment that crosses the boundary of this instance, these scan flops will only be connected if the remaining elements of the segment are also specified with the `-elements` option—either directly or indirectly through another hierarchical instance.
	If you specify this option with the `-power_domain` option, the specified elements must belong to the specified power domains.
`-incremental`	Adds new chains in incremental mode, without changing already connected scan chains
	Do not use user-defined chains with the same names as the `actual_scan_chains`.
`-include_opcg_segments`	
	Allows you to include the OPCG segments into the scan chains.
	This option must be used when OPCG has been inserted into the design and scan compression is to be added later using the `add_test_compression` command in the non-Unified Compression Flow or in a Unified Compression Flow when the scan channels are pre-built as internal scan chains.
`-keep_connected_shift_enable`	
	Ensures that a flop that is already connected to a shift-enable signal keeps this connection when connected in a scan chain.
	If you omit this option, the tool can break the existing connection to the shift enable and connect the shift-enable pin of the flop to either the default shift-enable signal or to the shift-enable signal specified for the chain.
`-pack`	Packs the scan chains to their maximum limit instead of balancing the chains (that is, attempting to create chains with similar lengths).
	You can specify a chain-specific constraint using the `-max_length` option of the define_scan_chain command or a global constraint by setting the value of the `dft_max_length_of_scan_chains` attribute.
`-partitions` *string*	
	Specifies a list of partitions.
`-physical`	Specifies to use the placement locations of the scan flops to connect the scan chains.
	The placement information is obtained from the DEF file read in with the `read_def` command.
`-power_domain` *power_domain_list*	
	Considers only the scan flops that belong to the specified power domain(s) for scan chain connection.
`-preview`	Reports how the scan chains will be connected, but makes no modifications to the netlist. Use this option to verify your scan-chain architecture prior to connecting the scan chains.
`-preview_scan_element_order`	
	Reports the scan element order in preview mode.
`-update_placement`	Specifies to update the placement of the DFT logic that is added during the scan chain connection process.
	Use this option only in the physical flow after you have already placed the design using `syn_opt -spatial`. The option will be ignored if the design is not placed.
`-zipper_stitch`	Attempts to connect all abstract segments defined on the same instance sequentially in a scan chain.
	This option is ignored when the scan chains are built using the `-physical` option

Related Information

See the following sections in *Genus Design for Test Guide*:

- Controlling Scan Configuration
- Connecting the Scan Chains
- Library-Domain Aware Scan Chain Configuration
- Power-Domain Aware Scan Chain Configuration
- Physical Scan Chain Synthesis
- Defining Scan Configuration Modes

Related commands:	write_dft_compression_macro
Affected by these constraints:	define_scan_abstract_segment
	define_dft_cfg_mode
	define_scan_fixed_segment
	define_scan_floating_segment
	define_scan_preserved_segment
	define_scan_chain
	define_test_signal -function shift_enable
	define_shift_register_segment
	set_compatible_test_clocks
Affected by these commands:	check_dft_rules
	fix_dft_violations
	syn_map
	syn_opt
Affects this command:	connect_serial_scan_chains
	report_scan_chains
Sets these attributes:	actual_scan_chain Attributes
	actual_scan_segment Attributes
Affected by these attributes:	decoded_pin
	dft_lockup_element_type
	dft_max_length_of_scan_chains
	dft_min_number_of_scan_chains
	dft_mix_clock_edges_in_scan_chains
	dft_prefix
	dft_scan_map_mode

connect_serial_scan_chains

```
connect_serial_scan_chains
-name string
-chains actual_scan_chains
[-dft_cfg_mode dft_config_mode_name]
[-preview][design]
```

Description

Inserts muxing logic into the scan path of actual scan chains such that the scan chains are concatenated to become a single, longer scan chain in a specific test mode of operation.

An actual scan chain may be specified only once per test mode, that is, multiple configurations of the same scan chain in the same test mode are disallowed.

To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-chains` *scan_chains*	
	Specifies the names of the actual scan chains to be concatenated; where the scan chains are concatenated in the specified order.
design	Specifies the name of the top-level design for which to concatenate the scan chains. Specify this name if you have multiple top designs loaded.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dft_cfg_mode` *dft_configuration_mode_name*	
	Specifies the object name of the scan mode used to concatenate the actual scan chains.
`-name` *string*	Specifies the name of the concatenated chain.
`-preview`	Reports how the actual scan chains would be concatenated, but does not perform the concatenation.

Related Information

See the following sections in *Genus Design for Test Guide*:

- Concatenating Scan Chains
- Controlling Scan Configuration

Affected by these constraints:	define_dft_cfg_mode
	define_test_signal -function <shift_enable | test_mode>
	set_compatible_test_clocks
Affected by this command:	check_dft_rules
Affects these commands:	report_scan_chains
	write_dft_abstract_model
	write_scandef
Sets these attributes:	actual_scan_chain Attributes
	actual_scan_segment Attributes
Affected by these attributes:	decoded_pin
	dft_lockup_element_type
	dft_mix_clock_edges_in_scan_chains

convert_to_opcg_scan

```
convert_to_opcg_scan
-edge_mode test_signal
[-dont_map][-effort {high | low}][design]
```

Description

Replaces domain crossing scan flops with their OPCG-equivalent flops by adding a hold mux and inverter (referred to as a toggle mux) in the scan data path. This command is used for designs with OPCG logic when the OPCG domain blocking attribute is enabled in the flow. If the library includes OPCG scan flops with the toggle mux logic built into the liberty cell, you should specify these scan flops using the `set_opcg_equivalent` command ahead of the `convert_to_opcg_scan` command to use these cells as the OPCG-equivalent flops. The `convert_to_opcg_scan` command should be run after the test compression logic has been added to the design.

Options and Arguments

design	Specifies the design in which you want to replace domain-blocking scan flip-flops.
-dont_map	Prevents the inserted logic from being mapped even if the design is already mapped to the target library.
-edge_mode *test_signal*	
	Specifies the global `edge_mode` signal to connect to the OPCG toggle muxes protecting domain crossings. A shift enable signal may be used as long as there are no flops that are on both functional and scan chain domain crossings.
-effort {high | low}	
	Specifies the approach to inserting and mapping the toggle mux and inverter logic to be inserted for the `OPCG` domain crossing flops.
	• Using a `low` effort level, the toggle mux and inverter logic will be mapped directly to gates from the target technology library and connected in the design.
	Use this approach when you have thousands of at-speed domain crossing flops to be processed as it offers better runtime at the expense of timing accuracy.
	• Using a `high` effort level, the toggle mux and inverter logic will be mapped to generic components first, and the newly added logic will then be mapped to gates from the target technology library.
	Default: `high`

Related Information

See the following sections in *Genus Design for Test Guide*

- Inserting the Toggle Muxes
- Top-Down Test Synthesis Flow with OPCG Logic Insertion

Affected by these commands:	reset_opcg_equivalent
	set_opcg_equivalent

convert_to_scan

```
convert_to_scan [-to_non_scan]
[-dont_check_dft_rules] [design]
```

Description

This command either

- Replaces non-scan flops with their scan-equivalent flip-flops if the design was previously mapped.
  In this case, the `dft_scan_map_mode` design attribute must be set to either `tdrc_pass` or `force_all`. If set to `tdrc_pass`, you must have run the DFT rule checker.
- Replaces all scan flops that are part of shift register segments with non scan flops except for the first element of each shift register segment.

This command should be run on the mapped design and prior to building the scan chains in either a logical or physical flow. The presumption being that the scan elements will be properly mapped for DFT and included into the scan chains before physical synthesis is performed on the design.

Options and Arguments

design	Specifies the design in which you want to replace regular flip-flops.
-dont_check_dft_rules	
	Prevents the DFT rules from being automatically checked.
-to_non_scan	Replaces all scan flops that are part of shift register segments to non scan flops except for the first element in the segments.

Related Information

See the following sections in *Genus Design for Test Guide*: • Defining Scan-Equivalency between Non-Scan and Scan Flops to Map to Scan • Controlling Mapping to Scan in a Mapped Netlist • Identifying Shift Registers in a Mapped Netlist before Creating the Scan Chains	
Affected by these commands:	check_dft_rules
	identify_test_mode_registers
	identify_test_mode_registers
	set_scan_equivalent
Affected by these attributes:	dft_scan_map_mode
	Test Clock Attributes

create_chains_database

```
create_chains_database
[-chains actual_scan_chains] [-scandef string]
[-design design]
```

Description

Creates an internal not-persistent database that pre-processes the scan chain flops location data for the `report_chains_*` commands.

This command needs to be rerun each time the scan chains change.

This command can be used on a placed design that has connected scan chains.

> The information in this chains database is not saved in the Genus database.

Options and Arguments

`-design` *design*	Specifies the name of the design for which to create the internal database.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
`-chains` *actual_scan_chains*	
	Specifies the names of the actual scan chains for which the scan chain flop location must be preprocessed.
	By default, the database will be created for all active scan chains.
`-scandef` *string*	Specifies the scanDEF file path.

Related Information

See the following sections in *Genus Design for Test Guide*:

- Controlling Scan Configuration
- Connecting the Scan Chains
- Library-Domain Aware Scan Chain Configuration
- Power-Domain Aware Scan Chain Configuration
- Physical Scan Chain Synthesis
- Defining Scan Configuration Modes

Affected by these commands:	report_chains_longest_wires
	report_chains_total_wirelength
	report_chains_wires

define_dft_cfg_mode

```
define_dft_cfg_mode
[-name scan_mode_name]
[-mode_enable_high test_signal ...]
[-mode_enable_low test_signal...]
[-jtag_instruction jtag_instruction]
[-type {scan |
  wrapper [-usage { extest | intest | mission}] }
[design]
```

Description

Defines a scan mode. Scan modes can be used to build the top-level scan chains with specific elements in different modes of operation (multi-mode, 1500 wrapper insertion), or when concatenating default scan chains into a single longer scan chain in a different mode of operation.

Options and Arguments

design	Specifies the name of the top-level design for which the scan mode is defined. Specify this name if you have multiple top designs loaded.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-jtag_instruction` *jtag_instruction_name*	
	Specifies the JTAG instruction which controls the scan chains in the current mode.
`-mode_enable_high` *test_signal...*	

	Name of the test signal(s) set to a `logic_1` value that controls scan chains in the current mode.
`-mode_enable_low` *test_signal...*	
	Name of the test signal(s) set to a `logic_0` value that controls the scan chains in the current mode.
`-name`	Name of the scan mode.
`-type {scan | wrapper}`	
	Specifies the type of the scan mode.
`-usage {extest | intest | mission}`	
	Specifies the usage of the scan mode for IEEE 1500 core wrapper insertion.
	This option is only valid with `-type wrapper`.

Example

The following example defines scan mode `scanModeA`. Test signal `test1` is specified to have an active high logic value and test signal `test2` is specified to have an active low logic value in this mode of operation:

```
define_dft_cfg_mode -name scanModeA design \
    -mode_enable_high test1 -mode_enable_low test2
```

To use the `-jtag_instruction` command option you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Related Information

See the following sections in *Genus Design for Test Guide*:

- Defining Scan Chain Configuration Modes
- JTAG-Controlled Scan Modes
- Defining the Wrapper Configuration Modes
- Inserting Core Wrapper Logic

Affects these commands:	report_scan_chains
	check_dft_rules
	connect_serial_scan_chains
	connect_scan_chains
	define_scan_abstract_segment
	write_dft_atpg_other_vendor_files
	write_dft_abstract_model
	write_dft_atpg
	write_dft_jtag_boundary_verification
	write_dft_lbist_test_points
	write_scandef
Affects these attributes:	dft_configuration_mode Attributes

define_hier_test_core_concat_order

```
define_hier_test_core_concat_order
-serial_concat_order <lib_cell|module|inst|hinst>
[-parallel_concat_order string] [-design design]
```

Description

Defines the concatenation order of cores in the Hierarchical Test flow.

Options and Arguments

-serial_concat_order {*lib_cell*|*module*|*inst*|*hinst*}	
	Spcifies the ordered list of instances for serial concatenation.
-parallel_concat_order *string*	
	Specifies the ordered list of instances for parallel concatenation.
-design *design*	Specifies the name of the top-block. This is required when there are multiple top designs

define_hier_test_scan_mapping

```
define_hier_test_scan_mapping
[-si_map port] [-so_map port]
[-cme_map port] [-module module]
[-libcell lib_cell|base_cell] [-instance inst|hinst+]
[-reset] [-design design]
```

Description

Defines the scan mapping order of cores in the Hierarchical Test flow.

Options and Arguments

-si_map *port*	Specifies the top level `compress_sdi` ports corresponding to cores `compress_sdi` test bus port index (highest to lowest) in an order.
-so_map *port*	Specifies the top level `compress_sdo` ports corresponding to cores `compress_sdo` test bus port index (highest to lowest) in an order.
-cme_map *port*	Specifies the top-level mask enable port corresponding to cores mask enable test bus port.
-module *module*	Specifies the core/subsystem module for pinmapping.
-libcell {*lib_cell*|*base_cell*}	
	Specifies the core/subsystem libcell for pinmapping.
-instance {*inst*|*hinst*}	
	Specifies the core/subsystem instance for pinmapping.

-reset	Resets the existing information for hier test scan mapping.
-design *design*	Specifies the name of the top-block. This is required if there are multiple top designs.

define_jtag_boundary_scan_segment

define_jtag_boundary_scan_segment [-name *segment_name*]
{-module *module* | -instance {*inst*|*hinst*{ | -lib_cell *cell*} {-bsdl_string *string* | -bsdl_file *file*} [-differential_pair {*positive_leg_pin negative_leg_pin*}] [-mode_a *mode_a_pin*]... [-mode_b *mode_b_pin*]... [-mode_c *mode_c_pin*]... [-highz *highz_pin*] -tdi *tdi_pin* -tdo *tdo_pin* [-clockdr *clockdr_pin*] [-capturedr *capturedr_pin*] [-updatedr *updatedr_pin*] [-shiftdr *shiftdr_pin*] [-index *bsr_position*] [-acdcsel_11496 *pin*][-acpclk_11496 *pin*] [-acpsen_11496 *pin*] [-acptrenbl_11496 *pin*] [-acpulse_11496 *pin*]

Description

Defines a boundary scan segment with its associated pins for connection along the TDI-TDO path in the boundary scan register, connection to the JTAG_Macro, and optionally defines its position in the boundary scan register.

You can define multiple boundary scan segments on the same instance, as long as the TDI and TDO pins of the segments are unique.

The command returns the directory path to the object that it creates.

Options and Arguments

-acdcsel_11496 *pin*	
	Specifies the name of the pin on the boundary scan segment that will be driven by the JTAG_ACDCSEL test signal.
-acpclk_11496 *pin*	
	Specifies the name of the pin on the boundary scan segment that will be driven by the JTAG_ACPSCLK test signal.
-acpsen_11496 *pin*	
	Specifies the name of the pin on the boundary scan segment that will be driven by the JTAG_ACPSEN test signal.
-acptrenbl_11496 *pin*	
	Specifies the name of the pin on the boundary scan segment that will be driven by the JTAG_ACTRENBL test signal.
-acpulse_11496 *pin*	
	Specifies the name of the pin on the boundary scan segment that will be driven by the JTAG_ACPULSE test signal.
-bsdl_file *file*	Specifies the file containing the BSDL abstract string for the boundary scan segment.
-bsdl_string *string*	
	Specifies the BSDL abstract string for the boundary scan segment.
-capturedr *capturedr_pin*	
	Specifies the name of the CAPTURE_DR pin on the boundary scan segment.
-clockdr *clockdr_pin*	
	Specifies the name of the CLOCK_DR pin on the boundary scan segment.
-differential_pair {*positive_leg_pin negative_leg_pin*}	
	Specifies the differential pair in the form of a positive leg pin name and negative leg pin name on the boundary scan segment.
-highz *highz_pin*	Specifies the name of the HIGHZ pin on the boundary scan segment
-index *bsr_position*	

	Specifies the relative position of the boundary scan segment in the BSR. Specify an integer value of zero or greater.
`-instance {inst|hinst}`	
	Specifies the instance name of the module for which the boundary scan segment is defined.
`-lib_cell cell`	Specifies the library cell for which the boundary scan segment is defined. This option applies to library cells that are implemented as timing models and whose description includes the `JTAG_Macro` related pins specified on the command line when defining the boundary scan segment.
`-mode_a mode_a_pin`	
	Specifies the name of the `MODE_A` pin on the boundary scan segment
`-mode_b mode_b_pin`	
	Specifies the name of the `MODE_B` pin on the boundary scan segment
`-mode_c mode_c_pin`	
	Specifies the name of the `MODE_C` pin on the boundary scan segment
`-module module`	Specifies the subdesign (module) to which the element belongs.
`-name segment_name`	Defines a name for the boundary scan segment.
`-shiftdr shiftdr_pin`	
	Specifies the name of the `SHIFT_DR` pin on the boundary scan segment.
`-tdi tdi_pin`	Specifies the name of the `TDI` pin on the boundary scan segment.
`-tdo tdo_pin`	Specifies the name of the `TDO` pin on the boundary scan segment.
`-updatedr updatedr_pin`	
	Specifies the name of the `UPDATE_DR` pin on the boundary scan segment.

Related Information

Defining Boundary Scan Segments in *Genus Design for Test Guide*

Affects these commands:	add_jtag_boundary_scan
	write_dft_bsdl
Sets these attributes:	bcell_segment
	differential

define_jtag_instruction

```
define_jtag_instruction -name string -opcode string
[-register string] [-length integer]
[-register_tdi {pin|hpin|port}] [-register_tdo {pin|hpin|port}] [-register_shiftdr {pin|hpin|port}] [-register_shiftdr_state {pin|hpin|port}] [-
register_shiftdr_inverted] [-register_reset_inverted] [-register_capturedr {pin|hpin|port}] [-register_capturedr_state {pin|hpin|port}] [-register_clockdr {p
in|hpin|port}] [-register_updatedr {pin|hpin|port}] [-register_updatedr_state {pin|hpin|port}] [-register_tck {pin|hpin|port}] [-register_reset
{pin|hpin|port}] [-register_runidle {pin|hpin|port}] [-register_decode {pin|hpin|port}] [-capture string] [-tap_tdo {pin|hpin|port}] [-tap_decode {pin|h
pin|port}] [-use_for_ijtag] [-private] [-design design]
```

Description

Defines a user-defined instruction that is serially loaded into a boundary scan device.

The command returns the directory path to the object that it creates.

Options and Arguments

`-capture` *string*	Specifies the values that must be captured into a register during the `CaptureDR` state.
`-design` *design*	Specifies the name of the design for which the JTAG instruction is defined.
	If you omit the design name and multiple designs are loaded, the top-level design of the current directory of the design hierarchy is used.
`-length` *integer*	Specifies the length of the custom test data register (TDR).
`-name` *string*	Specifies the name of the user-defined instruction.
`-opcode` *string*	Specifies the binary code for this instruction.
`-private`	Specifies that the defined instruction is private.
`-register` *string*	Specifies the name of the custom test data register (TDR).
`-register_capturedr {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_CAPTUREDR` pin on the `JTAG_MACRO` subdesign.
`-register_capturedr_state {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_CAPTUREDR_STATE` pin on the `JTAG_MACRO` subdesign.
`-register_clockdr {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_CLOCKDR` pin on the `JTAG_MACRO` subdesign.
`-register_decode {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_INSTRUCTION_DECODE_`*instruction* pin on the `JTAG_MACRO` subdesign, where *instruction* is the name that you specified through the `-name` option.
`-register_reset {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_RESET` pin on the `JTAG_MACRO` subdesign.
`-register_reset_inverted`	
	Specifies that the `JTAG_RESET` pin of the custom test register (TDR) has an active low polarity.
`-register_runidle {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_RUNIDLE` pin on the `JTAG_MACRO` subdesign.
`-register_shiftdr {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_SHIFTDR` pin on the `JTAG_MACRO` subdesign.

`-register_shiftdr_state {pin|hpin|port}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_SHIFTDR_STATE` pin on the `JTAG_MACRO` subdesign.
`-register_shiftdr_inverted`	
	Specifies that the `JTAG_SHIFTDR` pin of the custom test register (TDR) has an active low polarity.
`-register_tck {pin|hpin|port}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_TCK` pin on the `JTAG_MACRO` subdesign.
`-register_tdi {pin|hpin|port}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_TDI` pin on the `JTAG_MACRO` subdesign.
`-register_tdo {pin|hpin|port}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_register_TDO` pin on the `JTAG_MACRO` subdesign, where *register* is the name that you specified through the `-register` option
`-register_updatedr {pin|hpin|port}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_UPDATEDR` pin on the `JTAG_MACRO` subdesign.
`-register_updatedr_state {pin|hpin|port}`	
	Specifies the name of the pin on the custom test data register (TDR) that must be connected to the `JTAG_UPDATEDR_STATE` pin on the `JTAG_MACRO` subdesign.
`-tap_decode {pin|hpin|port}`	
	Specifies the name of the instruction-specific decode pin that must be created on the `JTAG_MACRO` subdesign.
`-tap_tdo {pin|hpin|port}`	
	Specifies the name of the instruction-specific test data output (TDO) pin that must be created on the `JTAG_MACRO` subdesign.
-use_for_ijtag	Specifies that the JTAG instruction is used in iJTAG network selection.

Example

The following example defines a private instruction `PROGRAM_TCB` for custom test data register `TCB_REG` that has a length of 8 bits. The opcode for the instruction is `0101`.

```
define_jtag_instruction -name PROGRAM_TCB -opcode 0101 -register TCB_REG -length 8 -private
```

Related Information

See the following sections in *Genus Design for Test Guide*:

- Defining the Instructions
- Inserting a JTAG Macro
- JTAG-Controlled Scan Modes

Affects these commands:	add_jtag_boundary_scan
	add_jtag_macro
Related command:	define_jtag_instruction_register
Sets these attributes:	jtag_instruction Attributes

define_jtag_instruction_register

```
define_jtag_instruction_register
-name string
[-length integer] [-capture string]
[-design design]
```

Description

Customizes the instruction register to allow adding user-defined instructions.

The command returns the directory path to the object that it creates.

Options and Arguments

`-capture string`	Specifies the capture value of the instruction register. According to the IEEE 1149.1 standard the last two bits of the capture value must be `01`.
	Default: 01
`-design design`	Specifies the name of the design for which the instruction register is customized.
	If you omit the design name and multiple designs are loaded, the top-level design of the current directory of the design hierarchy is used.
`-length integer`	Specifies the length of the instruction register. The length of the register is determined by the number of user-defined instructions that you want to add. If n is the number of bits in the instruction register, a total of 2_n instructions can be defined including the four mandatory instructions.
	Default: 2
`-name string`	Specifies the name of the user-defined instruction register.

Example

The following example defines instruction register `INSTR_REGISTER` with length 3. A register of length 3 allows you to create 2^3 instructions, or four user-defined instructions besides the four mandatory instructions.

```
define_jtag_instruction_register -name INSTR_REGISTER -length 3
```

Related Information

See the following sections in *Genus Design for Test Guide*:

- Defining the Instruction Register
- Inserting a JTAG Macro
- JTAG-Controlled Scan Modes

Related commands:	define_jtag_instruction
Affects these commands:	add_jtag_boundary_scan
	add_jtag_macro
Related attributes:	jtag_instruction_register Attributes

define_jtag_macro

```
define_jtag_macro
[-module module] [-lib_cell lib_cell]
[-instance {inst|hinst}] [-bsr_shiftdr {pin|hpin|port|subport}] [-bsr_clockdr {pin|hpin|port|subport}] [-bsr_updatedr {pin|hpin|port|subport}] [-reset
{pin|hpin|port|subport}] [-runidle {pin|hpin|port|subport}] [-clockdr {pin|hpin|port|subport}] [-shiftdr {pin|hpin|port|subport}] [-updatedr {pin|hpi
n|port|subport}] [-capturedr {pin|hpin|port|subport}] [-shiftdr_state {pin|hpin|port|subport}] [-updatedr_state {pin|hpin|port|subport}] [-
capturedr_state {pin|hpin|port|subport}] [-exitldr_state {pin|hpin|port|subport}] [-mode_a {pin|hpin|port|subport}] [-mode_b {pin|hpin|port|subport}]
[-mode_c {pin|hpin|port|subport}] -tdi {pin|hpin|port|subport} -tdo {pin|hpin|port|subport} -tck {pin|hpin|port|subport} -tms {pin|hpin|port|subpor
t} [-trst pin|hpin|port|subport] [-boundary_tdo {pin|hpin|port|subport}] [-tdo_enable {pin|hpin|port|subport}] [-highz {pin|hpin|port|subport}] [-
dot6_ac_dc_select {pin|hpin|port|subport}] [-dot6_ac_pulse {pin|hpin|port|subport}] [-dot6_preset_clock {pin|hpin|port|subport}] [-dot6_trcell_enable {p
in|hpin|port|subport}] [-power_on_reset {pin|hpin|port|subport}] [-name string]
```

Description

Identifies a pre-instantiated JTAG Macro.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-boundary_tdo {pin|hpin|port|subport}`	
	Specifies the boundary register `TDO output` pin on the JTAG macro.
`-bsr_clockdr {pin|hpin|port|subport}`	
	Specifies the clock data register (`CLOCKDR`) output pin for the boundary-scan register.
`-bsr_shiftdr {pin|hpin|port|subport}`	
	Specifies the shift data register (`SHIFTDR`) output pin for the boundary-scan register.
`-bsr_updatedr {pin|hpin|port|subport}`	
	Specifies the update data register (`UPDATEDR`) output pin for the boundary-scan register.
`-clockdr {pin|hpin|port|subport}`	
	Specifies the clock data register (`CLOCKDR`) output pin for the custom test data register.
`-capturedr {pin|hpin|port|subport}`	
	Specifies the capture data register (`CAPTUREDR`) output pin for the custom test data register.
`-capturedr_state {pin|hpin|port|subport}`	
	Specifies the capture data register (`CAPTUREDR_STATE`) output pin for the custom test data register.
`-dot6_ac_dc_select {pin|hpin|port|subport}`	
	Specifies the logical OR of the decoded `EXTEST_PULSE` and `EXTEST_TRAIN` instructions. The tool connects this signal to the AC Mode pin of all test receivers when it inserts the boundary scan logic. This signal also controls the multiplexer which is added to the output boundary scan cells.
`-dot6_ac_pulse {pin|hpin|port|subport}`	
	Specifies the AC test signal output of the JTAG macro.
`-dot6_preset_clock {pin|hpin|port|subport}`	

	Specifies the `preset_clock` output pin that provides a positive-active edge-sensitive clock signal to test receivers that have edge-sensitive initialization.
`-dot6_trcell_enable {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the logical OR of `EXTEST`, `EXTEST_PULSE` and `EXTEST_TRAIN` used to enable the test receiver cells.
`-exit1dr_state {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the exit1dr data register (EXIT1DR) state output pin on the JTAG macro.
`-highz {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the `HIGHZ` output pin to place the I/O pads in their `HIGHZ` state.
`-instance {`*`inst`*`|`*`hinst`*`}`	
	Specifies the path name of the `JTAG_MACRO` instance.
`-lib_cell` *`cell`*	Specifies the library cell for the `JTAG_MACRO` instance.
`-mode_a {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the `mode_a` output pin to configure boundary cells in the boundary-scan register.
`-mode_b {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the `mode_b` output pin to configure boundary cells in the boundary-scan register.
`-mode_c {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the `mode_c` output pin to configure boundary cells in the boundary-scan register.
`-module` *`module`*	Specifies the subdesign (module) name of the `JTAG_Macro` instance.
`-name` *`string`*	Specifies the name of the `JTAG_MACRO`. If not specified, an object will be added under the `boundary_scan/jtag_macros` vdir with a default name of `jtag_macro_`*`n`*.
`-power_on_reset {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the power-on reset input pin on the JTAG macro.
`-reset {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the reset output pin indicating that the JTAG macro is in the Test-Logic-Reset state.
`-runidle {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the `JTAG_RUNIDLE` output pin indicating that the JTAG macro is in the Run-Test-Idle state.
`-shiftdr {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the shift data register (`SHIFTDR`) output pin for the custom test data register.
`-shiftdr_state {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the shift data register (`SHIFTDR_STATE`) output pin for the custom test data register.
`-tck {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the TAP controller `TCK` input pin on the JTAG macro.
`-tdi {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the TAP controller `TDI` input pin on the JTAG macro.
`-tdo {`*`pin`*`|`*`hpin`*`|`*`port`*`|`*`subport`*`}`	
	Specifies the TAP controller `TDO` output pin on the JTAG macro.

`-tdo_enable {pin|hpin|port|subport}`	
	Specifies the enable output pin which drives the JTAG `TDO` output enable pin.
`-tms {pin|hpin|port|subport}`	
	Specifies the TAP controller `TMS` input pin on the JTAG macro.
`-trst {pin|hpin|port|subport}`	
	Specifies the TAP controller `TRST` input pin on the JTAG macro.
`-updatedr {pin|hpin|port|subport}`	
	Specifies the update data register (`UPDATEDR`) output pin for the custom test data register.
`-updatedr_state {pin|hpin|port|subport}`	
	Specifies the update data register (`UPDATEDR_STATE`) output pin for the custom test data register.

Examples

- The following example defines a third party TAP controller with a specified instance location.

```
genus:root:> define_jtag_macro -instance user_defined_jtag \
-highz MY_JTAG_INSTRUCTION_DECODE_CTRL_HIGHZ \
-bsr_clockdr MY_JTAG_BOUNDARY_CLOCKDR -bsr_shiftdr MY_JTAG_BOUNDARY_SHIFTDR \
-bsr_updatedr MY_JTAG_BOUNDARY_UPDATEDR \
-mode_a  MY_JTAG_INSTRUCTION_DECODE_MODE_A \
-mode_b MY_JTAG_INSTRUCTION_DECODE_MODE_B \
-mode_c MY_JTAG_INSTRUCTION_DECODE_MODE_C -tdi MY_JTAG_TDI -tdo MY_JTAG_TDO \
-tms MY_JTAG_TMS -tck MY_JTAG_TCK -trst MY_JTAG_TRST \
-tdo_enable MY_JTAG_ENABLE_TDO -boundary_tdo MY_JTAG_BOUNDARY_TDO \
-power_on_reset MY_JTAG_POR -name JM1
```

- The following example defines a third party TAP controller without a `TRST` input pin.

```
genus:root:> define_jtag_macro -module MY_JTAG_MACRO \
-highz MY_JTAG_INSTRUCTION_DECODE_CTRL_HIGHZ \
-bsr_clockdr MY_JTAG_BOUNDARY_CLOCKDR -bsr_shiftdr MY_JTAG_BOUNDARY_SHIFTDR \
-bsr_updatedr MY_JTAG_BOUNDARY_UPDATEDR \
-mode_a MY_JTAG_INSTRUCTION_DECODE_MODE_A \
-mode_b MY_JTAG_INSTRUCTION_DECODE_MODE_B \
-mode_c MY_JTAG_INSTRUCTION_DECODE_MODE_C -tdi MY_JTAG_TDI -tdo MY_JTAG_TDO \
-tms MY_JTAG_TMS -tck MY_JTAG_TCK -tdo_enable MY_JTAG_ENABLE_TDO \
-boundary_tdo MY_JTAG_BOUNDARY_TDO -power_on_reset MY_JTAG_POR -name JM1
```

Related Information

See the following sections in *Genus Design for Test Guide*: • Defining a Pre-Existing JTAG Macro • JTAG-Controlled Scan Modes	
Sets these attributes:	jtag_macro Attributes
Related command:	add_jtag_macro

define_jtag_tap_port

```
define_jtag_tap_port {pin|hpin|port} [-create_port] -type {tck | tdi | tdo | tdo_enable | tms | trst} [-hookup_pin {pin|hpin|port}] [-hookup_polarity
string] [-tck_period integer] [-no_trst_test_signal] [design]
```

Description

Defines a TAP port (JTAG signal). Specifies the internal hookup pin for a JTAG signal. These hookup pins will be used when inserting boundary scan logic, JTAG macro, MBIST logic and PTAM logic, to make connections from its DFT logic to the hookup pins specified for each of the JTAG signals.

> You can only specify this command before inserting the boundary scan logic or the JTAG macro. When defining an existing JTAG macro using define_jtag_macro, the tap ports must be defined with the `define_jtag_tap_port` command.

Options and Arguments

`-create_port`	Creates the port if it does not exist.
design	Specifies the design for which the tap ports are being defined.
`-hookup_pin {`*pin*`|`*hpin*`|`*port*`}`	
	Specifies the core-side hookup pin to be used for the top-level JTAG signal during DFT synthesis.
`-hookup_polarity {non_inverted	inverted}`	
	Specifies the polarity of the JTAG signal at the core-side hookup pin.
	By default, a non inverted polarity is assumed, but if the assumed value at the hookup pin conflicts with the propagated value, the tool will issue an error message.
`-no_trst_test_signal`	
	Prevents defining an active-low test signal on the TRST port and causes a formal verification constraint with a value of '0' to be added for the TRST port for the revised design.
{*pin*`|`*hpin*`|`*port*}	Specifies the driving pin or port for the JTAG signal.
`-tck_period` *integer*	
	Specifies the frequency (in picoseconds) of the Test Clock (TCK port) on the TAP Controller.
	> You can only specify this option when you have set the `-type` option to `tck`.
	Default: `50000`
`-type {tck	tdi	tdo	tdo_enable	tms	trst}`	
	Specifies the type of TAP port being created.

Related Information

See the following sections in *Genus Design for Test Guide*: • Hookup Requirements for TAP Ports • JTAG-Controlled Scan Modes	
Sets these attributes:	tap_port Attributes
	dft_tap_tck_period

define_mbist_clock

```
define_mbist_clock -name mbist_clock
[-design design] [-internal_clock_source]
-period integer
[-domain domain]
[-hookup_pin {pin|hpin}] [-hookup_period integer] [-hookup_polarity {non_inverted | inverted}] [-is_jtag_tck] [is_srclk] port
```

Description

Defines a PMBIST clock and associates a clock waveform with the clock. The clock waveform can be different from the system clocks. You must define all PMBIST clocks that are referenced in the PMBIST configuration file to enable the DFT rules checking associated with PMBIST insertion in the design.

> The system function for the boundary cell associated with the external clock source must be `clock`.

The command returns the virtual directory path to the mbist object that it creates.

Options and Arguments

`-design` *design*	Specifies the name of the design for which the PMBIST clock is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-domain` *domain*	Specifies the PMBIST clock domain to which the defined PMBIST clock belongs.
	Default: PMBIST clock name.
`-hookup_period` *integer*	
	Specifies the period interval at the hookup pin. The hookup period is specified in picoseconds.
	Default: value of `-period` option
`-hookup_pin {`*pin*`|`*hpin*`}`	
	Specifies the core-side hookup pin to be used by the `add_pmbist` command to make the PMBIST clock connection.
`-hookup_polarity {non_inverted | inverted}`	
	Specifies the polarity of the PMBIST clock signal at the core-side hookup pin relative to the specified port.
	Default: `non_inverted`
`-internal_clock_source`	
	Specifies that the PMBIST clock is either driven by a clock source internal to the design or by an external clock source, design port, that has a free-running clock applied at the tester.
`-is_jtag_tck`	Specifies that the PMBIST clock is defined as a JTAG clock.

`-is_srclk`	Specifies that the PMBIST clock will be used as repair clock for all memories (which require repair).
	If no clock is defined as repair clock, the repair clock of a memory will be the same as the one used for running the memory test for that particular memory.
`-name` *mbist_clock*	Specifies the name of the PMBIST clock that is being defined.
	Each clock object in your design must have a unique name. If you define a new PMBIST clock with the same name as an existing clock, an error message will be issued.
	The clock name is referenced within the PMBIST configuration file to access this clock object. The clock name also allows you to search for the clock later or to recognize it in reports.
`-period` *integer*	Specifies the clock period interval at the specified port. The clock period is specified in picoseconds.
	Default: `50000` (20 MHz test clock)
port	Specifies the PMBIST clock input port, or the *test time control* port in case of an internal PMBIST clock.
	The test time control port is any port on the design which can toggle during memory test to control the number of test cycles executed to ensure the internally clocked PMBIST test completes.

Example

The following example defines three PMBIST clocks, the second one needing a hookup pin to avoid a PLL and the third one specifying an internal clock source.

```
define_mbist_clock -name CLK1X -period 20000 CLK1
define_mbist_clock -name CLK2X -period 20000 -hookup_pin PLLOUTA  \
    -hookup_period 10000 CLK2
define_mbist_clock -name CLK3I -period 30000 -hookup_pin OSCOUT \
    -internal_clock_source CLK2
```

For the PMBIST clock object `CLK2X` there is a clock frequency multiplication factor of 2 through the PLL and it is in phase with the port CLK2. For the PMBIST clock object `CLK3I` the clock is driven by an on-chip oscillator which reuses design port `CLK2` as the test time control port.

Related Information

Affects this command:	add_pmbist
Sets these attributes:	mbist_clock Attributes

define_opcg_domain

```
define_opcg_domain [-name name]
-osc_source osc_source
{ [-domain_macro_parameter domain_macro_parameter]  [-location {hport | port | pin | hpin}] | [-counter_length integer] [-max_num_pulses integer]} -
opcg_trigger opcg_trigger -min_domain_period float [-divide_by integer] [-scan_clock scan_clock][-instance name][-shift_enable test_signal] [-design des
ign]
```

Description

Defines an `OPCG` (clock) domain.

The command returns the directory path to the opcg_domain object that it creates.

Options and Arguments

-counter_length *integer*	
	Specifies the number of bits in the down counter register.
	Specify a number between 4 and 7, or specify 0 if no counter should be inserted.
	This option cannot be specified with the -domain_macro_parameter and -location options.
-design *design*	Specifies the name of the design for which the domain is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
-divide_by *integer*	Specifies to build an internal clock divider circuit which divides the oscillator source frequency by the specified number.
-domain_macro_parameter *domain_macro_parameter*	
	Specifies the domain macro parameter set to be used for the OPCG domain.
	The domain macro parameter set must have been previously defined by the define_opcg_domain_macro_parameters command.
-instance *name*	Specifies the name of the OPCG domain instance.
	This option cannot be specified with the -domain_macro_parameter, -location and -divide_by options.
-location {*hport* | *port* | *pin* | *hpin*}	
	Specifies where to insert the domain macro. Specify an existing top-level port, subport, or hierarchical pin name.
-max_num_pulses *integer*	
	Specifies the number of high speed pulses the domain macro should be able to generate. The maximum number you can specify is 8.
	Default: 2
	This option cannot be specified with the -domain_macro_parameter and -location options.
-min_domain_period *float*	
	Specifies the minimum period (in picoseconds) at which the OPCG domain is assumed to operate.
	The frequency at which the domain is supposed to operate (specified using this option) must be greater than the minimum frequency of the oscillator source, specified using the -max_output_period of the define_opcg_osc_source command.
-name *name*	Specifies the opcg_domain object name of the OPCG domain.

	The name allows you to search for the object later or to recognize it in reports.
	Default: `OPCG_DOMAIN_n`
`-opcg_trigger` *opcg_trigger*	
	Specifies the trigger signal that will enable high speed pulses to be generated by the domain macros controlled by this trigger.
	The trigger signal must have been previously defined with the `define_opcg_trigger` constraint.
`-osc_source` *osc_source*	
	Specifies the oscillator source to be used for the `OPCG` domain.
	The oscillator source must have been previously defined by a define_opcg_osc_source constraint.
`-scan_clock` *scan_clock*	
	Specifies the test clock that will be used for shifting the OPCG domain macro logic in fullscan mode.
	If you omit this option, the scan clock specified with the `add_opcg` command is used.
`-shift_enable` *test_signal*	
	Specifies the shift-enable (SE) test signal to be connected to the specified OPCG domain when OPCG is inserted, giving you control over the SE signal to use for each domain.

Examples

- The following command inserts the domain macro after output pin `Y` of buffer instance `i_buf1`.

```
define_opcg_domain -name OD1 -osc_source OSC1 \
-domain_macro_parameter DOMAIN_MACRO_PARAMETER_1 -opcg_trigger TRIGGER1 \
-location i_buf1/Y -min_domain_period 1000 -divide_by 6
```

- The following command inserts the domain macro before input pin `A` of buffer instance `i_buf1`.

```
define_opcg_domain -name OD1 -osc_source OSC1 \
-domain_macro_parameter DOMAIN_MACRO_PARAMETER_1 -opcg_trigger TRIGGER1 \
-location i_buf1/A -min_domain_period 1000 -divide_by 6
```

- The following command inserts the domain macro after pin `out1` of subdesign instance `i_core`.

```
define_opcg_domain -name OD1 -osc_source OSC1 \
-domain_macro_parameter DOMAIN_MACRO_PARAMETER_1 -opcg_trigger TRIGGER1 \
-location i_core/out1 -min_domain_period 1000 -divide_by 6
```

- The following command inserts the domain macro within the subdesign for instance `i_core` prior to output pin `out1`.

```
define_opcg_domain -name OD1 -osc_source OSC1 \
-domain_macro_parameter DOMAIN_MACRO_PARAMETER_1 -opcg_trigger TRIGGER1 \
-location i_core/out1 -min_domain_period 1000 -divide_by 6
```

- The following command inserts the domain macro before the `in1` pin of subdesign instance `i_core`.

```
define_opcg_domain -name OD1 -osc_source OSC1 \
-domain_macro_parameter DOMAIN_MACRO_PARAMETER_1 -opcg_trigger TRIGGER1 \
-location i_core/in1 -min_domain_period 1000 -divide_by 6
```

- The following command inserts the domain macro within the subdesign for instance `i_core` after the input pin `in1`.

```
define_opcg_domain -name OD1 -osc_source OSC1 \
-domain_macro_parameter DOMAIN_MACRO_PARAMETER_1 -opcg_trigger TRIGGER1 \
-location  i_core/in1 -min_domain_period 1000 -divide_by 6
```

Related Information

Defining OPCG Clock Domains in *Genus Design for Test Guide*.	
Affected by these constraints:	define_opcg_domain_macro_parameters
	define_opcg_osc_source
	define_opcg_trigger
Affects this command:	add_opcg
Sets these attributes:	opcg_domain Attributes

define_opcg_domain_macro_parameters

```
define_opcg_domain_macro_parameters [-name name]
[-max_num_pulse integer]
[-counter_length integer | -max_trigger_delay float]
[-min_target_period float] [-design design]
```

Description

Defines a set of domain macro parameters used to configure and build the OPCG domain macro logic.

You must specify the command with either -counter_length or -max_trigger_delay.

The command returns the directory path to the domain_macro_parameter object that it creates.

Options and Arguments

-counter_length *integer*	
	Specifies the number of bits in the down counter register.
	Specify a number between 4 and 7, or specify 0 if no counter should be inserted.
-design *design*	Specifies the name of the design for which the domain macro parameter set is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
-max_num_pulse *i*nteger	
	Specifies the number of high speed pulses the domain macro should be able to generate. The maximum number you can specify is 8.
	Default: 2
-max_trigger_delay *float*	
	Specifies the time (in picoseconds) after which the first pulse must be issued by the domain macro when it is triggered by the enabled by the TRIGGERRUN output signal generated by the trigger macro.
-min_target_period *float*	
	Specifies the minimum target period (in picoseconds) at which the domain macro must operate.

	This option is required when inserting OPCG logic into a technology mapped design. It is used to specify the `opcg_pllclk` period in the tool-generated SDC constraints file used to synthesize the domain macro logic.
	Default: `1000`
`-name` *name*	Specifies the `domain_macro_parameter` object name of the domain macro parameter set.
	Default: `DOMAIN_MACRO_PARAMETER_n`

Related Information

Defining OPCG Domain Macro Parameters in *Genus Design for Test Guide*.	
Affects this constraint:	define_opcg_domain
Affects this command:	add_opcg
Sets these attributes:	domain_macro_parameter Attributes

define_opcg_mode

```
define_opcg_mode
-mode_init file
[-jtag_controlled]
[-osc_source_parameters string]...
[-design design]
```

Description

Defines an OPCG mode for Modus ATPG.

```
The command returns the directory path to the opcg_mode object that it creates.
```

Options and Arguments

`-design` *design*	Specifies the name of the design for which the `OPCG` mode is defined.
	This option is only required if multiple designs are loaded.
	If there is only one top-level design, you can omit the design name. In this case, the tool will use the top-level design of the design hierarchy.
`-jtag_controlled`	Specifies whether a JTAG instruction is used to lock the PLLs for OPCG operation.
`-mode_init` *file*	Specifies the name of file containing the `OPCG` mode initialization sequence that ensures that PLLs are properly initialized and locked on the input oscillators.
`-osc_source_paremeters` *string*	
	Specifies the oscillator source parameters specific for this `OPCG` mode.
	Use the following format:

	{*name osc_source_output_period ref_clock_period*}
	where *name* is the name of the osc_source object.

Example

The following command defines an OPCG mode whose name defaults to DFT_opcg_mode_1.

```
define_opcg_mode -osc_source_parameters {OSC1 52.5 270.3} \  -osc_source_parameters {OSC2 52 290} -mode_init opcgMode1.mode_init
```

Related Information

See the following sections in *Genus Design for Test Guide*:

- Defining the OPCG Mode
- Writing the Scripts and Setup Files to Perform ATPG
- Generate Files for ATPG and Simulation

Affects this command:	write_dft_atpg
Related constraint:	define_opcg_osc_source
Sets these attributes:	opcg_mode Attributes
	osc_source_reference Attributes

define_opcg_osc_source

```
define_opcg_osc_source [-name osc_source]
-ref_clock_pin {pin|hpin|port} -min_input_period integer -max_input_period integer -min_output_period integer -max_output_period integer {pin|hpin} [-design design]
```

Description

Defines an oscillator source on the output pin of a PLL instance that will drive the OPCG logic.

```
The command returns the directory path to the osc_source object that it creates.
```

Options and Arguments

-design *design*	Specifies the name of the design for which the oscillator clock is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
-max_input_period *float*	
	Specifies the maximum period of the reference (input) clock. Specify the clock period in picoseconds. The value of this option must be larger than the value of the -min_input_period option.
-max_output_period *float*	

	Specifies the maximum period of the generated (output) clock. Specify the clock period in picoseconds. The value of this option must be larger than the value of the -`min_output_period` option.
`-min_input_period` *float*	
	Specifies the minimum period of the reference (input) clock. Specify the clock period in picoseconds.
`-min_output_period` *float*	
	Specifies the minimum period of the generated (output) clock. Specify the clock period in picoseconds.
`-name` *osc_source*	Specifies the name of the `osc_source` object for the oscillator source.
	Default: `OSC_SOURCE_`*n*
pin|*hpin*	Specifies the output pin of the PLL.
`-ref_clock_pin` {*pin*|*hpin*|*port*}	
	Specifies the reference (input) clock pin or port for the PLL.

Example

The following command defines an oscillator source for instance `instPLL1` whose reference (input) clock has a period between 20000 and 4000 ps (or clock frequency between 50 and 250 MHz) and whose output clock has a period between 2000 and 500 ps (or a clock frequency between 500 and 2000 MHz).

```
define_opcg_osc_source -name instPLL1
    -ref_clock_pin REFCLOCKPORT1 \
    -min_input_period 4000 -max_input_period 20000
    -min_output_period 500 -max_output_period 2000 \
    PLL1/Z1
```

Related Information

Affects these commands	define_opcg_domain
	define_opcg_mode
	define_opcg_trigger
	add_opcg
Sets these attributes:	osc_source Attributes

define_opcg_trigger

```
define_opcg_trigger [-name name]
-osc_source osc_source
[-active {low|high}]
[pin|hpin|port] [-create_port] [-inside {inst|hinst}] [-scan_clock scan_clock] [-delay_cycles integer | -instance {inst|hinst}] [-test_signal test_signa
l][-design design]
```

Description

Defines the trigger signal that will enable the trigger macro associated with the specified oscillator source.

> Multiple trigger macros, when inserted, can share the same trigger signal pin.

The command returns the directory path to the opcg_trigger object that it creates.

Options and Arguments

-active {low | high}	
	Specifies the active value for the OPCG trigger signal.
-create_port	Specifies whether to create the port if it does not exist.
-delay_cycles *integer*	
	Specifies an additional OPCG trigger delay to be applied internally to ensure that the scan enable has fully propagated in the design before any OPCG domain clocks are pulsed when the scan enable signal is re-used as the OPCG trigger
	This option is required when you use the -test_signal option.
	This option cannot be specified with -instance option.
-design *design*	Specifies the name of the design for which the trigger enable is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
-inside {*inst*|*hinst*}	
	Specifies the hierarchical instance in which the OPCG trigger must be inserted. This instance will be uniquified if needed.
	Default: inserted at the top-level of the design.
-instance {*inst*|*hinst*}	
	Specifies the name of the OPCG trigger instance.
-name *name*	Specifies the opcg_trigger object name of the OPCG trigger signal.
	Default: OPCG_TRIGGER_*n*
-osc_source *osc_source*	
	Specifies the object name of the oscillator source whose trigger macro is enabled by the defined signal.
	The oscillator source must have been defined with define_opcg_osc_source command.
{*pin*|*hpin*|*port*}	Specifies the driving (input) port for the OPCG trigger signal. To define the opcg_trigger on a pin or hpin, first define a test_signal with the option -function opcg_trigger and then use define_opcg_trigger with the -test_signal option.
	If multiple designs are loaded and you did no specify the -design option, you can also specify the full path to the driver.
-scan_clock *scan_clock*	
	Specifies the test clock that must be used for shifting the OPCG trigger logic in fullscan mode.
	If you omit this option, the scan clock specified with the add_opcg command is used instead.
-test_signal *test_signal*	
	Specifies the test signal for the OPCG trigger. The test signal must have the opcg_trigger or shift_enable function.

Example

The following command defines the opcg_trigger object associated with oscillator source OSC on port GO. This port will be created if it does not exist.

```
define_opcg_trigger -name TRIGGER -create_port  -active low  -osc_source OSC GO
```

Related Information

Affects this command:	add_opcg
Related constraints:	define_opcg_osc_source
	define_opcg_domain
	define_opcg_mode
Sets these attributes:	opcg_trigger Attributes

define_pmbist_direct_access

```
define_pmbist_direct_access -function string
-active {low | high}
[-hookup_pin pin ]
[-hookup_polarity {non_inverted | inverted}]
{mbist_clock_domain | port} [-fcu]
[-design design]
```

Description

Defines programmable MBIST direct access interface pins or ports which can be used as an alternative access mechanism for PMBIST. This method can be used as a supplement to or replacement for the JTAG interface for controlling MBIST.

> All signals are level-sensitive. The patterns that you generate by running the `create_embedded_test` command in Modus properly stimulate and monitor the defined signals if they are directly accessible from design ports.

Options and Arguments

`-active {low|high}`	Specifies the active value of the signal.
`-design` *design*	Specifies the name of the design for which the domain macro parameter set is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
`-fcu`	Specifies that the function is associated with the fuse control unit.
`-function` *string*	Defines the functionality of the direct access pin. The option can have one of the following values:
	• `mda_done`
	Specifies the pin or port where the PMBIST logic will assert the signal when memory testing is complete for direct access or the first full loop has completed for burnin patterns.

	• `mda_fail`
	Specifies the pin or port where the PMBIST logic will assert the signal when a failure occurs during memory testing. The signal is asserted only for direct access patterns and not for burnin patterns.
	• `mda_reset`
	Specifies the pin or port to asynchronously reset the PMBIST logic for direct access. This can be JTAG signal for `trst` if TRST is not held active during functional mode. This function is mandatory.
	• `mda_tck`
	Specifies the defined MBIST clock which is used to pass control and data into and extract results from the PMBIST logic on the direct access interface. This works in the same fashion as that `tck` signal in JTAG mode of operation. This function is mandatory.
	• `mda_tdi`
	Specifies the data and control input port or pin for direct access PMBIST. This single signal works in a similar fashion as the combination of JTAG `tms` and `tdi` in JTAG mode of operation. This function is mandatory.
	• `mda_tdo`
	Specifies the pin or port where the output of direct access PMBIST Test Data Registers can be monitored. This allows for more detailed failure information to be extracted from the `MBISTCHK` Test Data Register.
	• `mda_ovfl`
	Specifies the pin or port where the PMBIST logic will assert the signal when failures overflow during memory testing for direct access.
	• `mda_run`
	Specifies the input port or pin for on-demand PMBIST. This single signal works as a 'go' signal for on-demand PMBIST (like power-on-self-test).
	• `mda_run_mask`
	Specifies the input port or pin which is used to control testplans for on-demand PMBIST.
	• `mda_run_done`
	Specifies the pin or port where the PMBIST logic will assert the signal when memory testing is complete for on-demand PMBIST.
	• `mda_run_fail`

	Specifies the pin or port where the PMBIST logic will assert the signal when a failure occurs during memory testing. The signal is asserted only for on-demand PMBIST.
	• `mda_run_ovfl`
	Specifies the pin or port where the PMBIST logic will assert the signal when failures overflow during memory testing for on-demand PMBIST.
{*mbist_clock_domain* |*port*}	
	Specifies the source port or mbist clock object for this function. For function `mda_tck`, it must be an `mbist_clock` object. For rest of the functions, it can be a unidirectional port for the block level flow and it can be any pin or a port for the chip level flow.
	Any port can be shared for functional uses provided the defined and applied PMBIST DFT configuration mode enables PMBIST usage during PMBIST operations.
	When you specify this option, the Genus-DFT engine does not validate the controllability of any logic between the top-level signal and its designated hookup pin under test-mode setup.
`-hookup_polarity {inverted|non_inverted}`	
	Specifies the polarity of the signal at the core-side hookup pin.
	By default, a non inverted polarity is assumed, but if the assumed value at the hookup pin conflicts with the propagated value, the tool will issue an error message.

Related Information

Affects this command:	add_pmbist
Sets these attributes:	programmable_direct_access_function Attributes

define_pmbist_port

```
define_pmbist_port

-function string
[-design design]string
```

Description

This command allows to define a user-specified name for a particular function for PMBIST at block level.

For Fuse Control Unit ports:

1. If the port exists in the design, the design port range and the port range defined using the `define_pmbist_port' command must match and must use the format `n:1'. The number of channels requested in the hard repair configuration file must also be `n

2. If the port doesn't exist in the design, the port range might differ from the input port range defined in this command based on the number of channels specified in the hard repair configuration file.

3. The input vector port can be specified in one of the two ways:

    i) port name without range (for example: temfcu_rdi)

    ii) port name with range (for example: temfcu_rdi[4:1])

Options and Arguments

`-design` *design*	Specifies the name of the design for which the port is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
	This option is required if multiple designs are loaded.
`-function` *string*	Specifies the function of the port.
	These include (`tdi` | `tdo` | `reset` | `shiftdr` | `runidle` | `capturedr` | `updatedr` | `fcu_rdi` | `fcu_rdo` | `fcu_clk` | `fcu_active` | `fcu_transfer` | `temmda_tdo` | `temmda_tdi` | `retention_pause_active` | `retention_pause_continue` | `decode_<default instruction name>`)
string	Specifies the name of the port to be used for the specified function.

Related Information

Affected attributes:	programmable_direct_access_function Attributes

define_scan_abstract_segment

```
define_scan_abstract_segment
[-name segment_name] {-module module|-instance {inst|hinst}|-lib_cell cell}
-sdi {pin|hpin}
[-inversion]
-sdo {pin|hpin}
-scan_cells string
[-tail_inversion]
-clock_port {pin|hpin}
[-rise|-fall]
[-off_state {high|low}]
[ -tail_clock_port {pin|hpin}
[-tail_edge_rise | -tail_edge_fall]
[-tail_clock_off_state {high|low} [-other_clock_port {pin|hpin} [-other_clock_edge {rise|fall}]]]...
{ { -shift_enable_port {pin|hpin} -active {high|low} | -connected_shift_enable }
[ -test_mode_port {pin|hpin} -test_mode_active {low|high} ]...
-length integer
[-cg_shift_enable_active {high|low}]...
[-cg_shift_enable_port {pin|hpin}...
[-skew_safe {-skew_flop|-skew_latch}]
[-head_skew_safe]
[-dft_cfg_mode dft_config_mode_name]
```

Description

Defines an abstract segment. An abstract segment can be defined for objects of type blackbox, logic abstract module, or libcell timing model.

An abstract segment is a user-specified scan segment used at the next level of integration to define the sets of scan chains previously created for the object.

The command returns the directory path to the object that it creates.

Options and Arguments

`-active {low|high}`	Specifies the active value for the shift-enable port.
`-cg_shift_enable_active {low|high}`	
	Specifies the active value of clock gating shift enable port.
`-cg_shift_enable_port {`*`pin`*`|`*`hpin`*`}`	
	Specifies the clock gating shift enable port connected to the test enable pin of clock gates in the abstract segment.
`-clock_port {`*`pin`*`|`*`hpin`*`}`	
	Specifies the clock port—at the boundary of the blackbox or logic abstract module—driving the flip-flops at the head of the segment.
`-connected_shift_enable`	
	Indicates that the shift enable port of the module boundary is driven by external logic (preconnected) or that the shift enable signal is internally generated within the module boundary. In either case, the external logic connected to the shift enable pin of the module, or the internal logic driving the shift enable pins of the flip-flops in the module will not be modified by the scan configuration engine.
	This option cannot be specified together with the `-shift_enable` option.
`-dft_cfg_mode` *`dft_configuration_mode_name`*	
	Specifies in which scan mode the abstract segment will be connected in the scan chains.
`-head_skew_safe`	Indicates whether the abstract segment has a data lockup element connected at the head of its scan chain.
`-instance {`*`inst`*`|`*`hinst`*`}`	
	Specifies the instance name of the module for which the abstract segment is defined.
`-inversion`	Indicates an inversion at the scan data input pin of the abstract segment.
`-length` *`integer`*	Specifies the length of the abstract segment.
`-lib_cell` *`cell`*	Specifies the library cell for which the abstract segment is defined. This option applies to library cells that are implemented as timing models and whose description includes the relevant test-related pins (such as scan data input and output, clock, shift-enable) to infer the scan chain architecture.
`-module` *`module`*	Specifies the subdesign (module) to which the element belongs.
`-name` *`segment_name`*	Defines a name for the segment that you can use to reference in the define_scan_chain constraint.
`-off_state {high|low}`	
	Specifies the off state of the system clock specified through the `-clock_port` option.
	This option applies only to the clocked LSSD scan style.
`-other_clock_edge {rise | fall}`	
	Specifies the active edge of the clock specified through the `-other_clock_port` option.

	Default: `rise`
`-other_clock_port {pin|hpin}`	
	Specifies the clock port—at the boundary of the blackbox or logic abstract module—of another system clock used to drive some flip-flops in the abstract segment.
	This option is only required if the clock is different from the clock specified through the `-clock_port` option or the `-tail_clock_port` option.
`{-rise | -fall}`	Specifies the active edge of the clock specified through the `-clock_port` option.
	Default: `-rise`
	This option applies only for the clocked LSSD scan style.
`-scan_cells`	Specifies the list of scan cell names from the CTL file.
`-sdi (-sdo) {pin|hpin}`	Specifies the scan data input (scan data output) of the segment.
	• For a segment in a blackbox, specify a hport (port of the blackbox or logic abstract module).
	• For a segment defined for a lib_cell, specify a pin of the lib_cell.
`-shift_enable_port {pin|hpin}`	
	Specifies the shift enable port at the boundary of the blackbox or logic abstract module to which the segment belongs. Specify this option if you want the `connect_scan_chains` command to make the connection from the top-level shift-enable signals to the shift-enable ports of the modules.
	This option cannot be specified together with the `-connected_shift_enable` option.
`-skew_flop`	Specifies an edge-triggered flop has been inserted as the terminal lockup element for the skew-safe segment.
	This option must be specified with the `-skew_safe` option.
`-skew_latch`	Specifies a level-sensitive latch has been inserted as the terminal lockup element for the skew-safe segment.
	This option must be specified with the `-skew_safe` option.
`-skew_safe`	Indicates whether the abstract segment has a data lockup element connected at the end of its scan chain.
`-tail_clock_off_state {high|low}`	
	Specifies the off state of the system clock specified through the `-tail_clock_port` option.
	This option applies only to the clocked LSSD scan style.
`-tail_clock_port {pin|hpin}`	
	Specifies the clock port—at the boundary of the blackbox or logic abstract module—driving the flip-flops at the tail of the segment. This option is only required if the clock used at the tail of the abstract segment is different from the clock specified through the `-clock_port` option.
`-tail_edge_rise | -tail_edge_fall`	
	Specifies the active edge of the clock specified through the `-tail_clock_port` option.
	Default: `-tail_edge_rise`
`-tail_inversion`	Indicates an inversion at the scan data output pin of the abstract segment.
`-test_mode_active {low | high}`	

	Specifies the active value for the test-mode port.
	This option should immediately follow the corresponding `-test_mode_port` option.
`-test_mode_port {`*`pin`*`|`*`hpin`*`}`	
	Specifies the test mode port at the boundary of the blackbox module to which the segment belongs.
	Specify this option when the block-level design includes test-mode activated logic.
	When the test-mode signals are specified, the propagated values of the top-level test-mode signals must match the expected block-level test-mode values and the segment must also pass the clock-controllability rule checks in order for the segment to be included into a top-level scan chain.
	The tool does not make connections to the test-mode ports of the block-level design. The connections should already exist in the netlist.

Examples

- The following example defines an abstract segment with length 3 in blackbox module `b`. The clock driving the flip-flops at the tail of the segment is the same as the clock driving the first elements in the segment.

```
genus:root:> define_scan_abstract_segment -name a1 -module b -length 3 \
-sdi {p4[0]} -sdo {p5[0]} -shift_enable {p6[0]} -active high -clock p3 -rise
```

- The following example defines an abstract segment in a timing model reference `COMBELEM` with length 20.

```
genus:root:> define_scan_abstract_segment -name combElem_seg \
-lib_cell COMBELE -sdi A -sdo Z -shift_enable_port B -active hi \
-clock_port D2  -rise -length 20
```

- The following example defines an abstract segment `ABS` with length 10 in instance `o1`. The same clock `clk` with active rising edge is used for all flip-flops of the segment.

```
define_scan_abstract_segment -instance o1  -sdi sdi -sdo sdo \
-shift_enable_port se -active hi -clock_port clk -rise -length 10 -name ABS
```

Related Information

Affects this constraint:	define_dft_cfg_mode
	define_scan_chain
Affects these commands:	check_dft_rules
	connect_scan_chains
	fix_scan_path_inversions
	report_scan_chains
Sets these attributes:	scan_segment Attributes

define_scan_chain

```
define_scan_chain [-name name]
{-sdi {hport|port|pin|hpin} -sdo {hport|port|pin|hpin} [-create_ports] [-shared_input]
  {-shared_output [-shared_select test_signal] |
   -non_shared_output}
 [-hookup_pin_sdi {pin|hpin}] [-hookup_pin_sdo {pin|hpin}]
 [-shift_enable test_signal]
 [-head segment] [-tail segment] [-body segment]
 [-complete | -max_length integer]
 [-domain test_clock_domain [-edge {rise|fall}]]
 [-terminal_lockup {level_sensitive|edge_sensitive}]
 [-cfg_pad {tm_signal | se_signal} ]
|-analyze -sdo sdo [-sdi sdi] [-dont_overlay] [-internal]
  {-shared_output  | -non_shared_out} }
```

Description

Creates a scan chain or analyzes an existing chain with the specified input and output scan data ports.

If you created a scan chain, the command returns the directory path to the scan_chain object that it creates.

If you successfully analyzed an existing scan chain, the command returns the directory path to the actual_scan_chain object that it creates.

Options and Arguments

-analyze	Analyzes the connectivity of an existing top-level scan chain in a structural netlist compiled in a previous session of the tool. You must at least specify the scan data output pin and optionally the scan data input pin to identify the chain.
-body *segment*	Indicates that the specified segment (an ordered set of scan flip-flops) is part of the body of the scan chain. The segment must have been previously defined with a define_scan_xxx_segment constraint, where xxx is either abstract, fixed, floating, preserved, or shift_register.
-complete	Specifies that the defined chain is complete and no other flip-flops should be added to the chain.
-create_ports	Specifies whether to create the scan data input and output ports if they do not exist.
	If you do not specify the scan data input or output signals using the -sdi and -sdo options, the ports can be created and named as *prefix*_sdi_*num* and *prefix*_sdo_*num*, where *prefix* is the value of the dft_prefix attribute.
-domain *test_clock_domain*	
	Specifies the DFT clock domain to associate with the scan chain. This clock domain must have been previously identified by the check_dft_rules command or defined with the define_test_clock constraint.
	If you omit this option, and segments have been defined for the chain, the scan chain is automatically associated with the appropriate DFT clock domain. In the absence of segments, the scan chain can be assigned to any DFT clock domain.
	This option only applies to the muxed scan style.
-dont_overlay	Prevents that tool's reassociates (or overlays) user-defined segments of type preserve or fixed to its analyzed scan chains. Consequently, the segments are not re-established as a fixed entity (and hence are reorderable) when determining how to partition the chains for physical-based reordering. If these segments include multi-input combinational logic gates in the scan data path, the write_scandef command uses these gates to partition the analyzed scan chains into n-reorderable segments (referred to as scanDEF chains). The register before a multi-input combinational logic gate becomes the STOP point for one scanDEF chain, while the register after a combinational gate becomes the START point of another scanDEF chain.

`-edge {rise|fall}`	Specifies whether to use the falling or rising edge of the test clocks in the specified DFT clock domain. You can specify this option only if you specified `-domain`.
	If you omit this option, the scan flip-flops triggered by the different active edges of the test clocks will be placed on their own scan chain.
	This option is ignored if you enabled the `dft_mix_clock_edges_in_scan_chains` attribute.
	This option only applies to the muxed scan style.
`-head` *segment*	Indicates that the specified segment (an ordered set of scan flip-flops) must be placed at the head (closest to the scan data input) of the scan chain. The segment must have been previously defined with a `define_scan_xxx_segment` constraint, where xxx is either `abstract`, `fixed`, `floating`, `preserved`, or `shift_register`.
`-hookup_pin_sdi` *pin*`|{`*pin*`|`*hpin*`}`	
	Specifies the core-side hookup pin to be used for the scan data input signal during scan chain connection.
	When you specify this option, the DFT engine does not validate the control ability of any logic between the top-level scan data input signal and its designated hookup pin under test-mode setup.
`-hookup_pin_sdo` *pin*`|{`*pin*`|`*hpin*`}`	
	Specifies the core-side hookup pin to be used for the scan data output signal during scan chain connection.
	When you specify this option, the DFT engine does not validate the control ability of any logic between the top-level shift_enable signal and its designated hookup pin under test-mode setup.
`-max_length` *integer*	
	Specifies the maximum length that you allow for this scan chain.
	If you omit this option, the maximum length defaults to the value of the `dft_max_length_of_scan_chains` design attribute.
	This option is ignored if the scan chain is defined with the `-complete` option, or if the number of flip-flop instances in a head, body, or tail segment exceeds the maximum value.
`-name` *name*	Specifies the name of the scan chain.
	If you omit this option, a default name is used.
`-sdi {`*hport*`|`*port*`|`*pin*`|`*hpin*`}`	
	Specifies the scan data input signal.
	• If you want to *create* a chain, specify a top-level port or a hierarchical pin name in case of an existing port or pin. If you want the tool to create the port, use the `-create_ports` option and a primary input port with the specified name will be created.
	• If you want to *analyze* an existing chain, specify a top-level port, a hierarchical pin, subport, or a non-sequential instance pin.
`-sdo {`*hport*`|`*port*`|`*pin*`|`*hpin*`}`	
	Specifies the scan data output signal.
	• If you want to *create* a chain, specify a top-level port or a hierarchical pin name in case of an existing port or pin. If you want the tool to create the port, use the `-create_ports` option and a primary output port with the specified name will be created.

	• If you want to analyze an existing chain, specify a top-level port, a hierarchical pin, subport, or a non-sequential instance pin.
`-shared_input`	Specifies whether an existing functional input port can be used as scan data input port.
`-shared_select` *test_signal*	
	Specifies the select control signal to the mux inserted for a shared output port.
	Default: The default shift-enable signal or chain-specific shift-enable signal is used as the select control signal to the mux.
{`-shared_output | -non_shared_output`}	
	Specifies whether an existing functional output port can be used as scan data output port. If the functional port can be used for scan data purposes, a mux is inserted in the scan data path by the `connect_scan_chains` command.
	One of these options must be specified when the specified scan data output signal is already connected in the circuit.
`-shift_enable` *test_signal*	
	Designates a chain-specific shift-enable port or pin.
	If you omit this option, the default shift-enable signal specified using a `define_test_signal -function shift_enable` constraint is used.
`-tail` *segment*	Indicates that the specified segment (an ordered set of scan flip-flops) must be placed at the tail (closest to the scan data output) of the scan chain. The segment must have been previously defined with a `define_scan_xxx_segment` constraint, where xxx is either `abstract`, `fixed`, `floating`, `preserved`, or `shift_register`.
`-terminal_lockup {level_sensitive | edge_sensitive}`	
	Specifies the type of lockup element that configuration can insert at the tail end of the chain to connect to the specified scan data output signal.
	If this option is not specified, no terminal lockup element will be inserted.
	This option only applies to the muxed scan style.
`-internal`	Specifies that the anchor buffers will be inserted for this internal scan chain.

`-cfg_pad` {*tm_signal* | *se_signal*}	
	Specifies the test signal (test mode or shift enable signal) that the DFT engine must use if it needs to configure the pad connected to the scan data input or output signal to control the data direction during test mode.
	If the scan data input and output pin are shared with functional pins, you should use the shift enable test signal to configure pads. This will allow the pads to shift-in and shift-out data when shift-enable signal is active (scan-shift mode), and will allow the pads to operate in functional mode when shift-enable signal is inactive (capture mode).
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.

Examples

- The following example creates a chain containing three segments previously defined:

```
define_scan_chain -sdi in[0] -sdo out[0] \
-shared_output -head segHead -tail segTail -body segBody -name chain1

...
Info    : Added scan chain. [DFT-151]
        : scan chain successfully defined.
scan_chain:test/chain1
```

- The following example analyzes an existing scan chain:

```
define_scan_chain -name topChain -sdi SI -sdo SO -analyze

...
Info    : Added scan chain. [DFT-151]
        : scan chain successfully defined.
actual_scan_chain:test/topChain
```

Related Information

Affected by this constraint:	define_scan_abstract_segment
	define_scan_fixed_segment
	define_scan_floating_segment
	define_scan_preserved_segment
	define_test_signal -function shift_enable
	define_shift_register_segment
	define_test_clock
Affects these commands:	connect_scan_chains
	report_scan_chains
Affected by this attribute:	dft_max_length_of_scan_chains
	dft_mix_clock_edges_in_scan_chains
	dft_prefix
Sets these attributes:	scan_chain Attributes

define_scan_fixed_segment

```
define_scan_fixed_segment [-name scan_segment]
{pin|hpin|port|inst|hinst|scan_segment} ...
```

Description

Defines a fixed segment. In a fixed segment, the elements will be connected in the specified order during scan chain connection; they cannot be reordered by a physical scan reordering tool.

A fixed segment is a user-specified scan segment which can be associated with either

- A user-defined top-level chain—created using the define_scan_chain command
- A tool-created scan chain—created using the connect_scan_chains command

The command returns the directory path to the object that it creates.

Options and Arguments

{*pin*|*hpin*|*port*|*inst*|*hinst*|*scan_segment*}	
	Specifies an element in the scan segment being defined. List the elements in the order they should appear in the scan chain (in shift order, that is, left-most corresponds to first bit shifted-in). An element can be hierarchical pin, a port, a flip-flop instance, a combinational gate, or a scan segment.
	If the segment goes through a multi-input/output combinational gate, you must indicate the scan path through the gate by specifying its input and output pin as two separate consecutive elements.
-name *scan_segment*	Defines a name for the segment that you can use to reference in the define_scan_chain constraint.

Examples

- The following example defines the fixed segment used in the example for define_scan_chain.

```
define_scan_fixed_segment -name segBody *seq/out_reg_1 *seq/out_reg_3
```

- The following example defines a fixed segment that contains two combinational components, four sequential registers, a scan abstract segment, and a combinational component endpoint.

```
define_scan_fixed_segment -name fixedSeg \
 i_core/i_anor1/B0 i_core/i_anor1/Y i_core/i_anor2/B0 i_core/i_anor2/Y \
 i_core/i_flop11 i_core/i_flop22 i_core/i_flop33 i_core/i_flop44 \
 absSeg \
 i_core/bufToAnchorSeg/A i_core/bufToAnchorSeg/Y
```

Related Information

Affects this constraint:	define_scan_chain
Affects these commands:	connect_scan_chains
	report_scan_chains
Sets these attributes:	scan_segment Attributes

define_scan_floating_segment

```
define_scan_floating_segment [-name segment_name]
{instance|hinstance|scan_segment} ...
```

Description

Defines a floating segment. In a floating segment, the order of the elements can be changed during scan configuration and by a physical scan reordering tool.

A floating segment is a user-specified scan segment which can be associated with either

- A user-defined top-level chain—created using the define_scan_chain command
- A tool-created scan chain—created using the connect_scan_chains command

The command returns the directory path to the object that it creates.

Options and Arguments

-name *segment_name*	Defines a name for the segment that you can use to reference in the define_scan_chain constraint.
{*instance*|*hinstance*|*scan_segment*}	
	Specifies an element in the scan segment being defined. List the elements in the order they should appear in the scan chain (in shift order, that is, left-most corresponds to first bit shifted-in).

Example

The following example defines the floating segments used in the example for define_scan_chain

```
genus:root:design:test> define_scan_floating_segment -name segHead \*seq/out_reg_4 *seq/out_reg_5

genus:root:design:test> define_scan_floating_segment -name segTail \*seq/out_reg_0
```

Related Information

Affects this constraint:	define_scan_chain
Affects these commands:	connect_scan_chains
	report_scan_chains
Sets these attributes:	scan_segment Attributes

define_scan_preserved_segment

```
define_scan_preserved_segment [-name segment_name]
{ {inst|hinst|segment_name}... [-sdi {pin|hpin|port|hport}] [-sdo pin|hpin|port|hport}] | -analyze -sdi {pin|hpin|port|hport} -sdo {pin|hpin|port|hport} } [ -connected_shift_enable ] [-allow_reordering] [-hookup_pin_sdi {pin|hpin|port|hport}] [-hookup_pin_sdo {pin|hpin|port|hport}]
[<scan_segment|inst|hinst|port|hpin|pin>+]
```

Description

Defines a preserved segment. In a preserved segment, the elements of the mapped segment are already connected in the specified order, and they cannot be reordered by a physical scan reordering tool.

A preserved segment is a user-specified scan segment which can be associated with either

- A user-defined top-level chain—created using the define_scan_chain command
- A tool-created scan chain—created using the connect_scan_chains command

The command returns the directory path to the object that it creates.

Options and Arguments

-allow_reordering	Indicates whether the order of the elements can be changed during scan configuration and by a physical scan reordering tool.
-analyze	Analyzes the connectivity of the scan segment, and returns the elements of the segment given its endpoints.

	If the segment includes degenerated scan flops in its element list, you must also specify the `-connected_shift_enable` option for the command to complete successfully.
`-connected_shift_enable`	
	Indicates that the shift enable port of the module boundary is driven by external logic (preconnected) or that the shift enable signal is internally generated within the module boundary. In either case, the external logic connected to the shift enable pin of the module, or the internal logic driving the shift enable pins of the flip-flops in the module will not be modified by the scan configuration engine.
	You must specify this option to successfully analyze preserved segments (such as shift-register segments) into the Genus session when these segments include degenerated scan flops (or scan flops whose instances have their SI and SE pins are tied-off).
`-hookup_pin_sdi` | `-hookup_pin_sdo`	
	Specifies the hookup pin for SDI (or SDO).
{*inst*|*hinst*|*segment_name*}	
	Specifies an element in the scan segment being defined. List the elements in the order they should appear in the scan chain (in shift order, that is, left-most corresponds to first bit shifted-in). An element can be a flip-flop instance, a combinational instance, or a scan segment.
	Additionally, the hierarchical scan data input and output pins of the segment can be specified using the `-sdi` and `-sdo` options respectively. If the hierarchical `SDI` and `SDO` pins of the segment are both at the boundary of the same lower module, the Genus-DFT engine also traces the shift-enable signal back from the scan registers in the segment to the same module boundary. It hooks up the shift-enable signal at the module boundary whenever applicable (only if you did not specify the `-connected_shift_enable` option).
`-name` *segment_name*	Defines a name for the segment that you can use to reference in the define_scan_chain constraint.
`-sdi` | `-sdo`	Specifies the scan data input (scan data output) of the segment. Specify a pin name or port.
-scan_segment|*inst*|*hinst*|*port*|*hpin*|*pin*	
	Specifies elements composing scan segments.

Examples

- The following example defines a pre-existing segment in instance `u_a` using its scan data input and output pins.

  ```
  genus:root: xx> define_scan_preserved_segment -name segmenta -analyze \ -sdi */u_a/SIa -sdo */u_a/SOa
  ```

- The following example defines a pre-existing segment by specifying its hierarchical scan data input and output pins, and its elements consisting of two combinational components, four sequential registers, a scan abstract segment, and a combinational component endpoint.

  ```
  define_scan_preserved_segment -name preservedSeg \ -sdi i_core/i_anor1/B0 -sdo i_core/bufToAnchorSeg/Y \ i_core/i_anor1 i_core/i_anor2
  i_core/i_flop11 i_core/i_flop22 \ i_core/i_flop33 i_core/i_flop44 absSeg i_core/bufToAnchorSeg
  ```

Related Information

Affects this constraint:	define_scan_chain
Affects these commands:	connect_scan_chains
	report_scan_chains
Sets these attributes:	scan_segment Attributes

define_shift_register_segment

```
define_shift_register_segment [-name segment_name]
-start_flop {inst|hinst} -end_flop {inst|hinst}
```

Description

Defines a shift register. Because a shift register is a shiftable scan chain segment, the Genus-DFT engine can use the functional path of the shift register as the scan path by only scan-replacing the first flop in the shift register segment, while maintaining the existing connectivity of the flops.

> A shift register segment can only contain flops driven by the same clock and same clock edge.

A shift register is a user-specified scan segment which can be associated with either

- A user-defined top-level chain—created using the define_scan_chain command
- A tool-created scan chain—created using the connect_scan_chains command

```
The command returns the virtual directory path to the created shift register segment.
```

> Shift register segments are only supported for the muxed scan style.

Options and Arguments

`-end_flop {inst|hinst}`	
	Specifies the last flop in the shift register. Specify the hierarchical instance name of the flop.
`-name segment_name`	Defines a name for the segment that you can use to reference in the define_scan_chain constraint.
`-start_flop {inst|hinst}`	
	Specifies the first flop in the shift register. Specify the hierarchical instance name of the flop.

Example

The following example defines a shift register.

```
define_shift_register_segment -name myreg \-start_flop *seq/out_reg_0 -end_flop *seq/out_reg_7
```

Related Information

Affects this constraint:	define_scan_chain
Affects these commands:	connect_scan_chains
	report_scan_chains
Related command:	identify_shift_register_scan_segments
Sets these attributes:	scan_segment Attributes

define_test_bus_port

```
define_test_bus_port
[-name string] -function string [-index integer]
[-hookup_pin {pin|hpin}] [-hookup_polarity string] [-lib_cell base_cell] [-active {high|low}] [-create_port]{pin|hpin|port}
```

Description

Defines a test bus port on any port.

Use the `-function` option to specify the functionality of the port in test mode, such as scan data input, scan data output, scan control pin, and so on. For multiple test bus ports with the same function, use the `-index` option to distinguish them.

Options and Arguments

`-active {high|low}`		
	Specifies the active value.	
`-create_port`	Creates the port if it does not exist.	
`-function` *string*	Specifies the function of the test bus port. Following values can be specified	
	Function	**Purpose**
	`async_set_reset`	`Test-mode signal that controls the asynchronous set/reset pin of a flop`
	`aux_scan_clock`	Reserved for future use
	`broadcast_enable`	Enable of the broadcast-based decompression logic
	`capture_wr`	`WIR` capture control
	`codec_pipe_bypass`	Reserved for future use
	`codec_clock_enable`	
	`comp_pipe_bypass`	Reserved for future use
	`compress_sdi`	Common scan data input (broadcast to multiple cores)
	`compress_sdo`	Common scan data output
	compressor_clock	
	`compression_clock`	Test clock used for compression mask (and MISR if present)
	`compression_enable`	Control to enable compression mode
	`cpi_over_npi`	Reserved for future use
	`custom`	User specified control signal
	`decomp_pipe_bypass`	Reserved for future use
	dft_clock	
	`elastic_reset`	Reserved for future use
	`first_core`	Reserved for future use
	fullscan_enable	
	icg_async_hold	
	`in_pipe_bypass`	Reserved for future use
	ijtag_reset	IJTAG network reset control signal

	ijtag_select	IJTAG network selection control signal
	ijtag_capture	IJTAG network capture control signal
	ijtag_shift	IJTAG network shift control signal
	ijtag_tck	Clock used for iJTAG network
	ijtag_update	IJTAG network update control signal
	`lbist_direct_access`	Reserved for future use
	`lbist_test_enable`	Reserved for future use
	lbist_direct_done	
	lbist_direct_fail	
	lbist_direct_pass	
	lbist_direct_enable	
	lbist_setbist	
	lbist_runbist	
	lbist_run	
	los_enable	
	lp_gating_enable	
	lp_gating_load	
	lpc_sdi	
	lpc_sdo	
	lpc_clock	
	`lssd_scan_clock_a`	Reserved for future use
	`lssd_scan_clock_b`	Reserved for future use
	`mask_enable`	Data input to enable channel masking
	`mask_load`	Control to enable mask loading mode
	`mbist_retention_test_out`	Output test signal defined with `pmbist_use` attribute indicating `retention_pause_active`
	`misr_bypass`	Reserved for future use
	`misr_reset_enable`	Control signal to reset the MISR (misr compression only)
	misr_reset	
	`opcg_enable`	Reserved for future use
	`opcg_load`	Reserved for future use
	`opcg_load_clock`	Reserved for future use
	`opcg_trigger`	Reserved for future use
	`out_pipe_bypass`	Reserved for future use
	opcg_pllclk_capture	
	opcg_pllclk_force	
	opcg_los_enable	
	`parallel_sdi`	Reserved for future use

	parallel_sdo	Reserved for future use
	pipe_sen	Control pin that enables the pipeline flops. It is usually a shift enable pin.
	pipe_reset	
	scan_clock	Reserved for future use
	segment_bypass	
	select_bypass	Select 1500 bypass mode
	select_serial	Select 1500 serial mode
	select_wir	Select Wrapper Instruction Register (WIR) access control signal
	serial_sdi	1500 Wrapper Serial Input
	serial_sdo	1500 Wrapper Serial Output
	serial_wrck	Clock used for inserted 1500 wrapper instruction registers (WIR) and 1500 bypass registers
	shift_enable	Test signal that controls the shifting of data in a scan chain when asserted to its active value.
	shift_wr	WIR shift control
	smartscan_broadcast	Reserved for future use
	smartscan_enable	Reserved for future use
	spread_enable	Control to enable the xor decompressor
	test_clock	Reserved for future use
	test_mode	Test signal that controls the design in test mode
	update_wr	WIR update control
	wext	Wrapper Extest control signal
	wint	Wrapper Intest control signal
	wrck	
	wir_test	Test-mode signal to put the WIR in its ATPG-test mode
	wrapper_and_compression_clock	Shared compression and wrapper clock
	wrapper_clock	Wrapper instruction and dedicated wrapper cell clock
	wrapper_reset	Wrapper instruction register reset control
	Some restrictions apply for the functions.	
-hookup_pin {*pin*|*hpin*}		
	Specifies the core-side hookup pin to be used for the top-level signal during DFT synthesis.	
	When you specify this option, the Genus-DFT engine does not validate the controllability of any logic between the top-level signal and its designated hookup pin under test-mode setup.	
-hookup_polarity {non_inverted | inverted}		
	Specifies the polarity of the signal at the core-side hookup pin.	
	By default, a non inverted polarity is assumed. The Genus-DFT engine does not validate the polarity at the specified hookup pin.	

`-index` *integer*	Specifies the index of the test bus port.
	When more than one test bus port exists with the same function, they will have different indexes. Examples are:
	• In the case of an N-bit scan data input bus, each port has the `compress_sdi` function, but the index varies between `0` and `N-1`.
	• In the case of `wide2` masking, two `mask_enable` test bus ports are defined, the first with an index of 0, the second with an index of 1.
	You cannot specify the `-index` option when the `-wir_signal` option is specified.
`-lib_cell` *base_cell*	
	Specifies the *base_cell* where the test bus port is defined.
`-name` *string*	Specifies the name of the test bus port.
	Default: *dft_prefix_port*
	where *dft_prefix* is the value of the `dft_prefix` root attribute and port is the name of the port on which the test bus port is defined.
{*pin*|*hpin*|*port*}	Specifies the driver for the test bus port.

Related Information

Affects this command:	add_test_compression
Sets these attributes:	test_bus_port Attributes

define_test_clock

```
define_test_clock-name test_clock
[-design design][-domain test_clock_domain]
[-period integer] [-divide_period integer]
[-rise integer] [-divide_rise integer]
[-fall integer] [-lpc_signal][-divide_fall integer]
[-hookup_pin {pin|hpin}][-hookup_polarity string]] [-cfg_pad {tm_signal|se_signal}] [-create_port string][-controllable] {pin|hpin|port}
[{pin|hpin|port}]... [-function string][-controllable_from port]
```

Description

Defines a test clock and associates a test (scan shift) clock waveform with the clock. The test clock waveform can be different from the system clocks.

If you do not define test clocks, the DFT rule checker automatically analyzes the test clocks and creates these objects with a default waveform. The waveform information is useful in determining how to order scan flip-flops in a chain, and where to insert data-lockup elements in the chain.

Test clock waveforms are used to order flip-flops that belong to the same DFT test clock domain to minimize the addition of lockup elements. Flip-flops that are triggered first are ordered and connected last in a chain.

By default this test clock will be exported as a -ES signal in Modus Test Solution pinassign file generated from `write_dft_atpg` command.

The command returns the directory path to the `test_clock` object that it creates.You can find the objects created by the `define_test_clock` constraints in:

```
/designs/design/dft/test_clock_domains
```

Options and Arguments

`-cfg_pad {`*`tm_signal`* `|` *`se_signal`*`}`	
	Specifies the test signal that the DFT engine must use if it needs to configure the pad connected to the test clock signal to control the data direction during test mode.
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.
`-controllable`	When specifying an internal pin for a test clock, this option indicates that the internal clock pin is controllable in test mode (for example, Built-in-Self-Test (BIST)). If you do not specify this option, the rule checker must be able to trace back from the internal pin to a controllable top-level clock pin.
	If you specify an internal pin as being controllable, you need to ensure that this pin can be controlled for the duration of the test cycle. The tool will *not* validate your assumption.
`-controllable_from` *`port`*	
	When specifying an internal pin for a test clock, this option indicates that the internal clock pin is controllable from this source hookup.
`-create_port` *`string`*	
	Specifies to create a port if it does not already exist.
`-design` *`design`*	Specifies the name of the design for which the test clock is defined.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-divide_fall` *`integer`*	
	Together with the -fall option, determines the time that the falling edge occurs with respect to the beginning of the clock period. The time is specified as a percentage of the period and is derived by dividing `-fall` by `-divide_fall`.
	Default: `100`
`-divide_period` *`integer`*	
	Together with the -period option, determines the clock period interval. The clock period is specified in picoseconds and is derived by dividing `-period` by `-divide_period`.
	Default: `1`
`-divide_rise` *`integer`*	
	Together with the -rise option, determines the time that the rising edge occurs with respect to the beginning of the clock period. The time is specified as a percentage of the period and is derived by dividing `-rise` by `-divide_rise`.
	Default: `100`
`-domain` *`test_clock_domain`*	
	Specifies the DFT clock domain associated with the test clock.
	Clocks belonging to the same domain can be mixed in a chain.
	If you omit this option, a new DFT clock domain is created and associated with the test clock.
	Flip-flops belonging to different test clocks in the same domain can be mixed in a chain. Lockup elements can be added between the flip-flops belonging to different test clocks.
`-fall` *`integer`*	Together with the -divide_fall option, determines the time that the falling edge occurs with respect to the beginning of the clock period. The time is specified as a percentage of the period and is derived by dividing `-fall` by `-divide_fall`.
	Default: `90`

`-function` *string*	Specifies the type of the clock. `(test_clock|dft_clock|compressor_clock|ijtag_tck|wrck|lpc_clock)`
	`compressor_clock`—Reserved for future use
	`dft_clock`—Test clock used to drive DFT logic such as the OPCG programming register, the Elastic register, the mask register, the MISR, the pipelines, the parallel bypass register, the scan chains within the LBIST macro. Currently only used in the 2D compression flow and unified compression flow.
	`test_clock`—Clock used for shift and capture
	Default: `test_clock`
`-hookup_pin {`*pin*`|`*hpin*`}`	
	Specifies the core-side hookup pin to be used for the top-level test clock during DFT synthesis. This pin will be used for all connections to or from the test signal.
	When you specify this option, the DFT engine does not validate the controllability of any logic between the top-level hookup pin under test-mode setup.
`-hookup_polarity {inverted | non_inverted}`	
	Specifies the polarity of the test clock signal at the core-side hookup pin.
	By default, a non inverted polarity is assumed, but if the assumed value at the hookup pin conflicts with the propagated value, the tool will issue an error message.
`-name` *test_clock*	Specifies the name of the test clock that is being defined.
	Each clock object in your design must have a unique name. If you define a new test clock with the same name as an existing clock, an error message will be issued.
	The clock name allows you to search for the clock later or to recognize it in reports.
`-lpc_signal`	Specifies that the test clock will be driven off a LPC macro.
`-period` *integer*	Together with the -divide_period option, determines the clock period interval. The clock period is specified in picoseconds and is derived by dividing `-period` by `-divide_period`.
	Default: `50000` (20 MHz test clock)
`{`*pin*`|`*hpin*`|`*port*`}`	Specifies the test clock input pin or port.
	If you specify multiple pins, these pins are assumed to have zero skew: they can be mixed without lockup latches.
`-rise` *integer*	Together with the -divide_rise option, determines the time that the rising edge occurs with respect to the beginning of the clock period. The time is specified as a percentage of the period and is derived by dividing `-rise` by `-divide_rise`.
	Default: `50`

Example

The following example defines three test clocks, four test clock ports and two DFT clock domains.

```
define_test_clock -name CLK1X -domain domain_1 -period 20000 CLK1define_test_clock -name CLK2X -domain domain_1 -period 20000 CLK2 CLK2b define_test_clock -name CLK3X -domain domain_2 -period 20000 CLK3
```

The four test clock ports are CLK1, CLK2, CLK2b, and CLK3. Test clock CLK2X comprises equivalent test clocks CLK2 and CLK2B (they can be mixed in the same scan chain without any lockup element). Test clocks CLK1X, CLK2X belong to the same DFT clock domain and are compatible (requires lockup elements between compatible chain segments triggered by the different test clocks). Test clock CLK3X belongs to its own domain.

Related Information

See the following sections in *Design for Test Guide*: • Defining Test Clock Waveforms • Defining Internal Clock Branches as Separate Test Clocks • Defining Equivalent Test Clocks for Different Top-level Clock Pins • Defining an Internal Pin as Test Clock	
Affects these commands:	check_dft_rules
	connect_scan_chains
	fix_dft_violations
	report_scan_chains
Sets these attributes:	test_clock Attributes

define_test_signal

```
define_test_signal
```

```
{port|hport|pin|hpin}
-function string
[-index integer]
[-name string]
[-active {high|low}]
[-ideal]
[-lpc_signal]
[-scan_shift]
[ [-hookup_pin {pin|hpin} [-hookup_polarity string]]
[-cfg_pad {tm_signal|se_signal}] | [-create_port | -shared_input | -shared_output| -test_only] ]
[-default_shift_enable]
[-wir_signal [-wir_reset_value {low|high}]
[-wir_tm_value {low|high}] ]
[-pipeline_depth integer]
[-lec_value {auto | 0 | 1 | no_value }]
[-port_bus {port|port_bus|hport_bus}]
[-multi_mode]
[-design design]
```

Description

Defines a test signal on a pin or port for DFT processing.

Use the `-function` option to specify the functionality of the port in test mode, such as scan data input, scan data output, scan control pin, and so on. For multiple test signals with the same function, use the `-index` option to distinguish them.

You can find the objects created by the `define_test_signal` constraints in:

```
/designs/design/dft/test_signals
```

Options and Arguments

`-active {low | high}`	
	Specifies the active value for the test signal.
	Specify this option for test signals whose function type is `shift_enable`, `test_mode`, `async_set_reset`, `custom`.
	All other function types will be populated with the pre-determined active value of `high` except for `wrapper_reset` will be populated with an active value of `low`.

`-cfg_pad {`*`tm_signal`* `|` *`se_signal`*`}`			
	Specifies the test signal that the DFT engine must use if it needs to configure the pad connected to the test signal to control the data direction during test mode.		
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.		
`-create_port`	Creates the port if it does not exist.		
`-default_shift_enable`			
	Designates the specified test signal as the default shift-enable signal for user-defined scan chains for which you omit the `-shift_enable` option.		
`-design` *`design`*	Specifies the name of the design for which the test signal is defined.		
`-function` *`string`*	Specifies the function type of the test signal.		
	Many of the test signals are used in the different Compression flows as indicated in its DFT Feature column. The different Compression flows are: • **Unified** - Enabled by default; setup is defined using the `dft_compression` attributes, and the compression logic is inserted using `add_test_compression` command. • **Non-Unified** - Enabled by setting the root attribute `dft_add_test_compression_new_flow false`. The compression logic is inserted using `add_test_compression` command with its command line options.		
	Following values can be specified		
	Function	**Purpose**	**DFT Feature**
	`async_hold`	Reserved for future use	Advanced DFT
	`async_set_reset`	Test signal that controls the asynchronous set/reset pin of a flop	DFT control signal for DFT rule checks
	`aux_scan_clock`	Reserved for future use	clocked_lssd scan style
	`broadcast_enable`	Broadcast versus Daisy chain mode control signal	Unified Non-Unified Hierarchical Test
	`capture_wr`	`WIR` capture control	WIR Not used in Cadence JTAG to 1500 WIR interface
	`codec_pipe_bypass*`	Bypasses the decompressor and compressor pipelines (if implemented)	Unified Internal Pipelines
	`codec_clock_enable`		
	`comp_pipe_bypass`	Bypasses the compressor pipelines (if implemented)	Non-Unified Internal Pipelines
	`compress_sdi`	The scan input ports, shared for all scan operations	Fullscan, Compression, Mask Load, OPCG Load, LPG Load

	compress_sdo	The scan output ports, shared for all scan operations	Fullscan, Compression, Mask Load, OPCG Load, LPG Load
	compression_clock	Test clock used for compression mask (and MISR if present)	Non-Unified Hierarchical Test
	compression_enable*	Control to enable compression mode; Switches between Compression and Fullscan mode (if implemented)	Unified Non-Unified Legacy
	cpi_over_npi	Reserved for future use	Non-Unified Hierarchical Test
	custom	User specified control signal; Can also be a WIR bit; user needs to program to value for ATPG	WIR
	decomp_pipe_bypass	Bypasses the decompressor pipelines (if implemented)	Non-Unified Internal Pipelines
	elastic_reset	Resets the Elastic register and pipelines (if implemented)	Unified Non-Unified Decompressor
	first_core	Reserved for future use	Hierarchical Test
	fullscan_enable		
	in_pipe_bypass*	Bypasses the scan input and mask enable pipelines (if implemented)	Unified External Pipelines
	ijtag_reset	IJTAG network reset control signal	
	ijtag_select	IJTAG network selection control signal	
	ijtag_capture	IJTAG network capture control signal	
	ijtag_shift	IJTAG network shift control signal	
	ijtag_update	IJTAG network update control signal	
	ijtag_tck	IJTAG test clock signal	
	lbist_direct_access	Reserved for future use	LBIST Direct Access
	lbist_direct_done	Test signal that indicates the (programmable) direct access LBIST test run status	LBIST Direct Access
	lbist_direct_enable	Control signal to start (programmable) direct access LBIST execution	LBIST Direct Access
	lbist_direct_fail	Test signal that indicates if the (programmable) direct access LBIST test run has failed	LBIST Direct Access
	lbist_direct_pass	Test signal that indicates if (programmable) direct access LBIST test run has passed	LBIST Direct Access
	lbist_direct_reset	Test signal that resets the LBIST registers	LBIST Direct Access
	lbist_run	Control signal to start running of LBIST	LBIST 1500 Driven Interface

	lbist_runbist*	Selects the LBIST RUNBIST TDR (if and when inserted) between the serial scan input and scan output ports. Must be defined before compression insertion for the correct selection logic to be inserted for LBIST to connect to.	LBIST Unified Non-Unified 1500 Driven Interface
	lbist_setbist*	Selects the LBIST SETBIST TDR (if and when inserted) between the serial scan input and scan output ports. Must be defined before compression insertion for the correct selection logic to be inserted for LBIST to connect to.	LBIST Unified Non-Unified 1500 Driven Interface
	lbist_test_enable*	Selects (only) the scan chains within LBIST macro (if and when inserted) between the scan input and scan output ports. Must be defined before compression insertion for the correct selection logic to be inserted for LBIST to connect to.	LBIST Unified Non-Unified
	los_enable		
	lp_gating_enable	Activates the LPG logic (if implemented) to gate a channel head to a constant value	Unified Non-Unified Low Power Gating
	lp_gating_load	Selects the LPG register (if implemented) for loading and multiplexes it on the scan output(s) for observation/concatenation	Unified Non-Unified Low Power Gating
	lp_staggered_shift_mode	Reserved for future use	Unified Non-Unified
	lpc_sdi		
	lpc_sdo		
	lssd_scan_clock_a	Reserved for future use	clocked_lssd scan style
	lssd_scan_clock_b	Reserved for future use	clocked_lssd scan style
	mask_enable	Activates masking logic (if implemented) to block a channel tail from being observed	Unified Non-Unified Legacy Channel masking
	mask_load	Control to enable channel mask load (if implemented) to select the mask register for loading and multiplexes it on the scan output(s) for observation/concatenation	Unified Channel masking
	mbist_retention_test_out	Output test signal defined with pmbist_use attribute indicating retention_pause_active	PMBIST
	misr_bypass*	Bypasses the MISR, effectively reverting back to an XOR compressor	Unified Non-Unified Compressor
	misr_reset (async)	Resets the MISR (if implemented)	Unified Non-Unified Compressor
	misr_reset_enable (sync)	Resets the MISR (if implemented)	Compressor

	opcg_enable	Control signal to put the design in an OPCG mode	OPCG
	opcg_load	Selects OPCG programming registers (if implemented) for loading and multiplexes it on the scan output(s) for observation/concatenation. Must be defined before OPCG insertion.	OPCG
	opcg_load_clock	Test clock used to load the OPCG programming registers	Non-Unified Legacy OPCG
	opcg_los_enable	Control signal to enable pipelining of scan_enable signal (support for both default shift_enable signal and OPCG Domain specific shift_enable signals)	OPCG
	opcg_pllclk_capture	Test signal which selects the clock connected to the PLLCLK_CAPTURE input pin of the OPCG Domain Macros	Hierarchical Test OPCG
	opcg_pllclk_force		OPCG
	opcg_trigger	Control signal to enable OPCG operation	OPCG
	out_pipe_bypass*	Bypasses the scan output pipelines (if implemented)	Unified Non-Unified External Pipelines
	parallel_sdi	Reserved for future use	Non-Unified Hierarchical Test
	parallel_sdo	Reserved for future use	Non-Unified Hierarchical Test
	parallel_reset		Unified Non-Unified XOR Decompression
	pipe_reset	Resets the pipe registers	
	pipe_sen	Control signal to enable the pipelines (if implemented) to shift	Non-Unified
	scan_clock	Not recommended for general use	Advanced DFT
	segment_bypass	Control signal to bypass the segment scan-in data to bypass mux output pin	Advanced DFT
	select_bypass*	Select 1500 bypass mode; Implements a bypass mode and switches to it, deselecting the current design for Hierarchical Test	1500 Hierarchical Test
	select_serial*	Select 1500 serial mode; Implements a serial mode and switches to it, enabling the selected register to be accessed serially through the serial scan input and output ports	Serial Scan, 1500 WIR Affects: Bypass, Fullscan, Mask register, LPG register, OPCG programming register
	select_wir	Select WIR access control signal; Enables the WIR to shift and update, and selects it to be observed through the serial scan output port	WIR
	serial_sdi	The serial scan input port, used to load the selected register serially	WIR, Bypass, Fullscan, Mask register, LPG register, OPCG programming register, etc

	serial_sdo	The serial scan output port, used to unload the selected register serially	WIR, Bypass, Fullscan, Mask register, LPG register, OPCG programming register, etc
	serial_wrck	Clocks the WIR, (serial) bypass register (if implemented)	WIR
	shift_enable	Controls the scan_enable pin of a flop	DFT control signal for DFT rule checks
	shift_wr	WIR shift control signal	WIR
	smartscan_broadcast	Bypasses the Smartscan deserializer (if implemented) of the Elastic decompressor and broadcasts the scan inputs, effectively increasing the compression ratio	Unified Non-Unified Decompressor
	smartscan_enable	Enables the Smartscan deserializer (if implemented) of the Elastic decompressor, effectively creating more internal scan inputs and decreasing the compression ratio.	Unified Non-Unified Decompressor
	spread_enable*	Control to enable the xor decompressor	Non-Unified Legacy
	swizzle	Reserved for future use	Compressor
	test_clock	Not recommended for general use	Advanced DFT
	test_mode	Control signal that enables the DFT test mode	DFT control signal for DFT rule checks
	update_wr	WIR update control signal	WIR
	wext*	Switches wrappers to EXTEST mode, selecting only the wrapper channels and the corresponding subset of mask registers, MISRs, etc	WIR 1500 Hierarchical Test
	wint*	Switches wrappers to INTEST mode, selecting the core and wrapper channels and the mask registers, MISRs, etc	WIR 1500 Hierarchical Test
	wir_test	Reserved for future use	WIR Hierarchical Test
	wrapper_and_compression_clock	Reserved for future use	Non-Unified 1500
	wrapper_clock	Test Clock used to clock the 1500 Dedicated Wrapper Cells (DWC)	Non-Unified 1500 Hierarchical Test
	wrapper_reset	Control signal used to reset the WIR	WIR
	wrck	Clock used for inserted 1500 wrapper instruction registers (WIR) and 1500 bypass registers	WIR
	Some restrictions apply for the functions.		
	* These test signals can also be added as WIR signals by specifying the `-wir_signal` option instead of specifying a pin or port.		
`-hookup_pin {`*pin*`|`*hpin*`}`			
	Specifies the core-side hookup pin to be used for the top-level port during DFT synthesis.		

	When you specify this option, the DFT engine does not validate the controllability of any logic between the top-level port and its designated hookup pin under DFT setup.
`-hookup_polarity {non_inverted | inverted}`	
	Specifies the polarity of the signal at the core-side hookup pin.
	By default, a non inverted polarity is assumed. The DFT engine does not validate the polarity at the specified hookup pin.
`-ideal`	Marks the test signal as ideal. This prevents buffering of the test signal network during optimization.
	Default: The test signal is marked non-ideal.
	If the test signal is marked as ideal, the tool sets the `ideal_network` attribute to `true` on the pin or port for the test signal.
`-index` *integer*	Specifies the index of the test signal.
	When more than one test signal exists with the same function, they will have different indexes. Examples are:
	• In the case of an N-bit scan data input bus, each port has the `compress_sdi` function, but the index varies between `0` and `N-1`.
	• In the case of `wide2` masking, two `mask_enable` test bus ports are defined, the first with an index of 0, the second with an index of 1.
	You cannot specify the `-index` option when the `-wir_signal` option is specified.
`-lec_value {auto | 0 | 1 | no_value}`	
	Specifies the approach to constraining the pin for LEC validation by the `write_do_lec` command. You can specify any of the following values:
	• `auto`—Writes the opposite of the test signal active value as the "add pin constraint" in the do file
	• 0—Writes a logic `0` for the `add pin constraint` in the do file
	• 1—Writes a logic `1` for the `add pin constraint` in the do file
	• no_value—Does not write the test signal as an "add pin constraint" in the do file
	Default: `auto`
`-lpc_signal`	Specifies that the test signal will be driven off an LPC Macro.
`-multi_mode`	Indicates that the test signal is used for multi-mode configuration purposes.
`-name` *string*	Specifies the name of the test signal.
	Default: *`dft_prefix_port`*
	where *`dft_prefix`* is the value of the `dft_prefix` root attribute and port is the name of the port on which the test signal is defined.
{*port*|*hport*|*pin*|*hpin*}	
	Specifies the driver for the test signal.
`-port_bus {`*port*|*port_bus*|*hport_bus*`}`	
	Specifies the driving port or port_bus.
`-pipeline_depth` *integer*	

	Specifies the number of external pipleline stages to be inserted for this test signal.
	This option should only be used with Non-Unified Compression for test signals whose functions are `compress_sdi`, `compress_sdo`, `mask_enable`, `in_pipe_bypass`, `out_pipe_bypass`
`-scan_shift`	Indicates that this test signal should only be held to its active value during the scan shift operation of the tester cycle. This option is used to designate those test signals which must be held to their non-controlling functional values to prevent the state of the flip-flops from being asynchronous set or reset while ATPG data is being shifted into or out of the scan chains.
	This option also instructs the `write_dft_atpg` command to write the signal as a clock using (+/-) SC instead of (+/-) TI in the Modus pin assign file.
`-shared_input`	Specifies whether the input port is also used as a functional port.
	By default, the signal applied to the specified driving pin or port is considered to be a dedicated for test purposes.
	This option cannot be specified with the `-create_port` option.
	Specify this option to ensure that the shared test signal will not be included as a pin constraint in the `write_do_lec` dofile. Not specifying this option for a shared test signal will result in over constraining the `write_do_lec` dofile which can lead to false EQs.
	Additionally, the `-`shared_input option sets the `dedicated_pin` attribute for the test signal to `false` which prevents a constraint from being applied to the shared functional port in the tool-generated NON-DFT SDC timing MODE.
`-shared_output`	Specifies whether the output port is also used as a functional port.
	By default, the signal applied to the specified load pin or port is considered to be a dedicated for test purposes.
	This option cannot be specified with the `-create_port` option.
	Specify this option to ensure that the shared test signal will not be included as a pin constraint in the `write_do_lec` dofile. Not specifying this option for a shared test signal will result in over constraining the `write_do_lec` dofile which can lead to false EQs.
	Additionally, the `-`shared_output option sets the `dedicated_pin` attribute for the test signal to `false` which prevents a constraint from being applied to the shared functional port in the tool-generated NON-DFT SDC timing MODE.
`-test_only`	Specifies that the input port will only be used for test purposes.
`-wir_reset_value {low|high}`	
	Specifies the value that the test signal will be driven to when the Wrapper Instruction Register (WIR) is reset.
	You can only specify this option if you specified the `-wir_signal` option.
	Default: `high` for test signals with the function types of `select_serial` or `select_bypass` function, and `low` for all other functions.
`-wir_signal`	Specifies that the test signal will be driven by a Wrapper Instruction Register (WIR) output. When you invoke the `add_wir_signal_bits` command, a bit is inserted in the WIR register for this test signal.
	You can only specify this option for the following function types: `compression_enable`, `select_bypass`, `select_serial`, `spread_enable`, `wext`, `wint`, `misr_bypass`, `codec_pipe_bypass`, `in_pipe_bypass`, `out_pipe_bypass`, `lbist_test_enable`, `lbist_runbist`, `lbist_setbist` and `custom`.

`-wir_tm_value {low|high}`	
	Specifies the value that the test signal will be forced to when the port with `wir_test` function (if one exists) is driven `high` and the Wrapper Instruction Register (WIR) is ATPG-tested.
	You can only specify this option with the `-wir_signal` option.
	Default: the value of the `-wir_reset_value` option

Related Information

See the following in in *Design for Test Guide*: • Hierarchical Test Flow: Preparing a Core • Inserting Core-Wrapper Logic	
Affects this command:	add_test_compression
Sets these attributes:	test_bus_port Attributes

fix_dft_violations

```
fix_dft_violations
{ -clock -test_control test_signal
  [-scan_clock_pin {pin|port} [-rise | -fall]] | {-async_set | -async_reset | -async_set -async_reset} -test_control test_signal [-async_control
test_signal] [-add_observe_scan -scan_clock_pin {pin | hpin | port} [-rise | -fall]]} [-violations violation_object_id_list] [-tristate_net] [-xsource [-
exclude_xsources inst|hinst...]] [-preview] [-dont_check_dft_rules] [-dont_map] [-design design]
```

Description

Automatically fixes either

- *All* DFT violations of the specified types (clock, asynchronous set, asynchronous reset, tristate, or xsource).
  Only the specified violation types are fixed. If you allow to fix asynchronous set and reset violations using the same test mode signal, you can request both types to be fixed at the same time.
- The *identified* violations
  You can further limit the violations that Genus must fix to by specifying the violation ID (through the `-violations` option).

> Currently, clock violations are only fixed for the muxed scan style.

Options and Arguments

`-add_observe_scan`	
	Inserts a flip-flop for observability. If you fix DFT violations in a generic netlist, the flip-flop is mapped to a scan flip-flop during synthesis. If you fix DFT violations in a mapped netlist, the flip-flop is mapped to a scan flip-flop. You need to rerun the `check_dft_rules` command to update the DFT status of the observability flop prior to connecting it in a scan chain.
	> This option can only be used when fixing an asynchronous set reset violation and requires the `-scan_clock_pin` option.
`-async_control test_signal`	

	Specifies the name of the test signal to use to control the asynchronous violations to be fixed.
`-async_reset`	Fixes the asynchronous reset violations on all instances.
`-async_set`	Fixes the asynchronous set violations on all instances.
`-clock`	Fixes the clock violations on all instances.
`-design` *design*	Specifies the name of the design whose violations must be checked.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_check_dft_rules`	
	Prevents the DFT rules from being checked automatically after fixing violations.
`-dont_map`	Prevents the inserted logic from being mapped even if the design is already mapped to the target library.
`-exclude_xsources` *inst*|*hinst*	
	Specifies instances to exclude from automatic fixing of X-source violations.
{*pin* | *port*}	Specifies the test signal pin or port to use to control the set or reset. By default, the set or reset are controlled by the `-test_control` option.
	To specify this option, the pin or port must first be declared as a `test_mode` signal using the `define_test_signal -function test_mode` constraint.
`-preview`	Reports how the violations will be fixed, without making modifications to the netlist.
[`-rise` | `-fall`]	Specifies to use the rising or falling edge of the test clock to fix the DFT violation during test-mode operation.
	Default: `-rise`
`-scan_clock_pin` {*pin* | *hpin* | *port*}	
	Specifies the test clock signal to be used. Specify the pin or port from where the clock signal originates. In most applications, the clock pin or port is identified when checking the DFT rules.
	This option is optional when fixing clock violations. By default, the Genus-DFT engine performs a clock trace to identify a controllable test clock that appears in the fanin cone of the clock violation and uses this test clock to fix the actual clock violation.
	This option is required when you want to insert observability flip-flops when fixing async violations. In this case, the test clock signal drives the clock pin of the observation flip-flops during test-mode operation.
`-test_control` *test_signal*	
	Specifies the name of the test signal to use to fix the violation.
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.
`-tristate_net`	Specifies to fix tristate net contention design rules violations.
`-violations` *violation_object_id_list*	
	Fixes the violations that are identified by their object name.
	The `check_dft_rules` command creates a `violations` directory in the design hierarchy under `violation:`*design/*. The objects in this directory correspond to the violations found during the last execution of the `check_dft_rules` command. Use the `report_dft_violations` command to list all remaining violations in the design.
`-xsource`	Specifies to fix X-Source design rules violations.

Examples

- The following example instructs Genus to fix all clock, async set, and async reset violations using test mode signal `tm` and test clock `CK1` using the rising edge as the active edge.

  ```
  fix_dft_violations -clock -async_set -async_reset
  -test_control tm -add_observe_scan -scan_clock_pin CK1 -rise
  ```

  If you do not want to use the same test clock to fix the clock violations and to drive the clock pin of the observation flip-flops, you need to enter two commands. For example,

  ```
  fix_dft_violations -clock -test_control tm
  ```

  ```
  fix_dft_violations -async_set -async_reset -test_control tm
  -add_observe_scan -scan_clock_pin CK1 -rise
  ```

  In this case, the Genus-DFT engine automatically determines which test clock to use to fix the clock violations.
- The following example instructs Genus to fix all clock, async set, and asynch reset violations using test mode signal `tm` and test clock `CK1` using the rising edge as the active edge.

  ```
  fix_dft_violations -clock -async_set -async_reset
  -test_control tm -scan_clock_pin CK1 -rise
  ```

- The following example instructs Genus to fix violations `vid_1` and `vid_3` if they are violations of type `async_set`.

  ```
  fix_dft_violations -violations {vid_1 vid_3} -async_set -test_control tm
  ```

Related Information

Affected by these constraints:	define_test_clock
	define_test_signal -function <shift_enable | test_mode>
Affects these commands:	check_dft_rules
	report_scan_registers
	report_dft_violations
	syn_map
Sets these attributes:	dft_status
	dft_violation
	violation Attributes

fix_pad_cfg

```
fix_pad_cfg -mode {input | output | tristate}
[-test_control test_signal] {port|pin|hpin}
```

Description

Inserts the required logic to configure the data direction control for a bidirectional or tristate pad during test mode. This command can be used after running the `check_dft_pad_cfg` command to fix the pads configuration.

> This command can configure a generic pad.

Options and Arguments

`-mode {input | output | tristate}`		
	Specifies in which mode the pad must be configured in test mode.	
	`input`	Specifies to configure the pad in input mode.
	`output`	Specifies to configure the pad in output mode.
	`tristate`	Specifies to disable the pad.
{port|pin|hpin}	Specifies the top-level port that is connected to the I/O pad that the Genus-DFT engine needs to configure.	
`-test_control` *test_signal*		
	Specifies the test signal to use to control the pad.	
	You must have specified the test signal using either 'define_test_signal -function shift_enable' or 'define_test_signal -function test_mode'.	

Related Information

Affected by this constraint:	define_test_signal -function <shift_enable | test_mode>
Affects these commands:	check_dft_rules
	connect_scan_chains

fix_scan_path_inversions

`fix_scan_path_inversions` *actual_scan_chain...*

Description

Fixes inversions for every scan element in the scan path. The command inserts inverters as required in a scan chain to remove inversions along the scan data path.

Options and Arguments

actual_scan_chain	
	Specifies the scan chain(s) to be analyzed and to insert inverters.

identify_domain_crossing_pins_for_cgic_and_scan_abstracts

```
identify_domain_crossing_pins_for_cgic_and_scan_abstracts
[-cgic] [-bbox] [-add_fencing]
[-global_blocking_shift_enable test_signal][design]
```

Description

Identifies domain crossing pins for blackboxes and integrated clock-gating instances in the specified design and adds blocking logic. This is needed as Modus ATPG assumes by default that all clock pulses in the same cycle have their rising edges aligned, whereas in reality due to different speeds and delays this is not true especially in cases where the enable logic for a clock is controlled by enable logic from a different clock. This can lead in simulation mis-compares where the clock propagation stops in for example, a CGIC. By gating the CGIC Test Enable pin, we make sure that clocks are free to propagate as Modus expects.

Options and Arguments

`-add_fencing`	Specifies to add fencing for the identified domain crossing pins using the global Shift Enable.
`-bbox`	Specifies to identify domain crossing pins for blackboxes.
`-cgic`	Specifies to identify domain crossing pins for integrated clock-gating instances, and potentially gate their Test Enable pins if `-add_fencing` is also used.
design	Specifies the design in which you want to identify domain crossing pins.
`-global_blocking_shift_enable` *test_signal*	
	Specifies the global blocking shift enable signal.

Related Information

Related command:	add_opcg

identify_multibit_cell_abstract_scan_segments

```
identify_multibit_cell_abstract_scan_segments
[-dont_check_dft_rules] [-preview]
[-lib_cells lib_cell...] [-design design]
```

Description

Identifies multi-bit scan cells in the design, and defines scan abstract segments for each instance of the multi-bit scan cell.
Multi-bit scan cells implemented using either a parallel or serial bit approach are supported by the tool.

Refer Mapping to Multibit Scan Cells in the Other Flows Chapter in *Design for Test Guide*.

Options and Arguments

`-design` *design*	Specifies the name of the top-level design on which to identify abstract segments for multi-bit scan cells.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_check_dft_rules`	
	Prevents the DFT rules from being automatically checked after identifying abstract segments for multi-bit scan cells.
`-lib_cells` *lib_cell*	
	Specifies the multi-bit library cells on which to perform abstract segment identification.
`-preview`	Reports the identified abstract scan segments without defining them.

Examples

- The following example identifies a parallel multi-bit scan cell with two bits; where each bit would be defined as an abstract scan segment:

  ```
  identify_multibit_cell_abstract_scan_segments -preview
  ```
  Would execute command:
  ```
  define_scan_abstract_segment -length 1 -sdi SI1 -sdo Q1
  -shift_enable_port SE1 -active high -clock_port CP -rise
  -lib_cell/libraries/tcbn65ulp_c070701wc2/libcells/DUALSDFQD0 -name DUALSDFQD0
  ```
  Would execute command:
  ```
  define_scan_abstract_segment -length 1 -sdi SI2 -sdo Q2
  -shift_enable_port SE2 -active high -clock_port CP -rise
  -lib_cell /libraries/tcbn65ulp_c070701wc2/libcells/DUALSDFQD0 -name DUALSDFQD0
  ```
- The following example identifies a serial multi-bit scan cell of length 4; where each instance of the cell would be defined as an abstract scan segment:

  ```
  identify_multibit_cell_abstract_scan_segments -preview
  ```
  Would execute command:
  ```
  define_scan_abstract_segment -length 4 -sdi SI -sdo Q4
  -shift_enable_port SM -active high -clock_port CK -rise
  -lib_cell /libraries/cs60ale_uc_scan/libcells/YSDM4ALU1 -name YSDM4ALU1
  ```

identify_shared_wrapper_cells_in_design

```
identify_shared_wrapper_cells_in_design
[design]
```

Description

Identifies shared wrapper flops in a **mapped** design. Genus marks these shared wrapper flops for exclusion from shift register identification and multibit merging. These flops also have their `dft_shared_wrapper_flop` attribute set to `true`.

Options and Arguments

design	Specifies the name of the design in which the shared wrapper cells must be identified.
	This option is required if multiple designs are loaded.

Related Information

Affected by this attribute:	dft_identify_shared_wrapper_cells

identify_shift_register_scan_segments

```
identify_shift_register_scan_segments
[-preview] [-incremental] [-design design]
```

Description

Identifies all shift registers in the design.

The Genus-DFT engine uses the following naming convention for automatically identified shift-register segments:

`DFT_AutoSegment_n`

> A shift register segment contains flops driven by the same clock and same clock edge.

A shift register is a scan segment which can be associated with either

- A user-defined top-level chain—created using the define_scan_chain command
- A tool-created scan chain—created using the connect_scan_chains command

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-design` *design*	Specifies the name of the design in which the shift registers must be identified.
	If you omit the design name and multiple designs are loaded, the top-level design of the current directory of the design hierarchy is used.
`-incremental`	Adds new shift register segments in incremental mode, without changing already identified shift register segments stored in `scan_segment:`*design*
	Without this option, the existing identified shift register segments will be removed and new segments will be identified based on the new values specified for the command options.
`-preview`	Reports which shift register segments will be identified, without adding them to the list of scan segments.

Related Information

Sets this attribute:	user_defined_segment

identify_test_mode_registers

```
identify_test_mode_registers
{ -fixed_value_register_file file
| {-file file [-macro_proc string] | -mode_init file}
  [-library string] [-atpg_log string]
  [-continue_with_severe_warnings] }
[-design design] [-preview]
```

Description

Identifies all internal registers whose output signals must remain constant during test mode and generates the corresponding test-mode signals required for the Genus-DFT engine.

The fixed-value registers can be identified

- From an existing Modus log file

- By invoking Modus and simulating a mode initialization sequence

  In the latter case, you need to have the Modus software installed and your operating system `PATH` environment variable must include the path to the Modus software.

This command is not generally used and is intended for a custom flow where a special test mode initialization sequence is executed in Modus to identify all fixed-value registers that hold their value constant during the entire test mode.

The fixed-value registers and their constant values can be fed back to a Genus DFT synthesis session to simplify the DFT setup. The alternative is for the user to manually identify all internal test mode fixed-value registers.

Options and Arguments

`-atpg_log` *`string`*	Specifies the log file for Modus.
`-continue_with_severe_warnings`	
	Continues the Modus run even when severe warnings are issued.
`-design` *`design`*	Specifies the name of the design for which to define the test-mode signals.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-file` *`file`*	Specifies the name of the STIL file that contains the mode initialization sequence.
`-fixed_value_register_file` *`file`*	
	Specifies a log file from a previous run that contains a list of fixed value registers.
	When this option is specified, Genus can parse this file instead of invoking Modus to get the list of fixed value registers.
	This option cannot be specified with the `-stil`, `-mode_init`, `-macro`, `-library`, `-atpg_log` and `-continue_with_severe_warnings` options.
`-library` *`string`*	Specifies the list of Verilog structural library files. Specify the list in a quoted string. Refer to the `write_dft_atpg` -library option description for additional information.
	This option is only required when you invoke this command on a mapped netlist.
`-macro_proc` *`string`*	
	Specifies the name of the macro in the STIL file that contains the initialization vectors to be simulated.
	Default: `test_setup`
`-mode_init` *`file`*	Specifies the name of the file that contains the mode initialization sequence in TBDpatt format.
`-preview`	Reports the fixed value registers but does not set the test mode values.

Examples

- The following command creates a report of the list of the registers with fixed values.

```
identify_test_mode_registers -file newStil -prev

Identifying internal registers with fixed value outputs under test_mode setup.
... Creating intermediate files

WARNING : No user defined shift enable signal found.
ATPG interface file may contain incomplete information
Cadence Design Systems RC file: Cadence ATPG file created successfully.

----------------------------------------
... Identifying the fixed value registers
----------------------------------------

Test Function    Block Name
+TI              tc/ts_reg[0]
+TI              tc/ts_reg[1]
+TI              tc/ts_reg[2]
----------------------------------------
Note:  The +/-TI Test Function flag denotes the active logic value that a signal
is to be held to during test mode.
       +TI denotes a logic value 1; -TI denotes a logic value 0
```

- The following command creates the test signals and automatically reruns the DFT rule checker.

```
genus:root:> identify_test_mode_registers -file newStil
Identifying internal registers with fixed value outputs under test_mode setup.
... Creating intermediate files

WARNING : No user defined shift enable signal found.
ATPG interface file may contain incomplete information
Cadence Design Systems RC file: Cadence ATPG file created successfully.

----------------------------------------
... Identifying the fixed value registers
----------------------------------------
INFO: Setting active high test mode signal on tc/ts_reg[0]/q
INFO: Setting active high test mode signal on tc/ts_reg[1]/q
INFO: Setting active high test mode signal on tc/ts_reg[2]/q
----------------------------------------
  Checking DFT rules for 'top' module under 'muxed_scan' style

...
genus:root:> vls dft/test_signals
/designs/top/dft/test_signals:
./                          rst                         tc__ts_reg[1]__q
incr                        tc__ts_reg[0]__q            tc__ts_reg[2]__q
```

Related Information

Affected by these constraints:	define_test_signal -function test_mode
	define_test_clock
Sets this attribute:	user_defined_signal

map_dft_unmapped_logic

```
map_dft_unmapped_logic
[-update_placement][-map_to_scan]{design|module}
```

Description

Maps any logic that was added by previous DFT commands, but not mapped at the time of the insertion. For example, logic added by test point insertion, scan chain connection, and so on.

Options and Arguments

`{design|module}`	Specifies the name of the design or the module for which the added DFT logic should be mapped.
`-map_to_scan`	Specifies to map flops to their scan counterparts.
`-update_placement`	Specifies to also update the placement of the added DFT logic.

Related Information

Related commands:

- add_jtag_boundary_scan
- add_jtag_macro
- add_pmbist
- add_ptam
- add_shadow_logic
- add_test_compression
- add_test_point
- convert_to_opcg_scan
- fix_dft_violations

map_pmbist_cgc_to_cgic

```
map_pmbist_cgc_to_cgic
[-design design] [-module_prefix_list string]
[-clock_gating_cell lib_cell]
```

Description

Maps programmable MBIST clock gating logic to integrated clock gating cells.

Options and Arguments

`-clock_gating_cell lib_cell`	
	Specifies the name of the `latch_posedge_precontrol` clock gating cell to be used. This option has the highest priority.

	The libcell specified with this option should have the attribute `clock_gating_integrated_cell` set equal to `latch_posedge_precontrol`.
	If this option is not specified with the command, then the libcell is retrieved from the attribute `lp_clock_gating_cell` set on the module containing the clock gating instance. If such a libcell is found, then that libcell is used, otherwise libraries are searched to find a suitable `lp_clock_gating_cell` which has the attribute `clock_gating_integrated_cell` set equal to `latch_posedge_precontrol`.
`-design` *design*	Specifies the name of the top-level design.
`-module_prefix_list` *string*	
	Specifies a list of prefix strings provided to `add_pmbist` command at various levels.

Related Information

Affected by this attribute:	lp_clock_gating_cell

map_pmbist_ffn

```
map_pmbist_ffn
[-design design]
```

Description

Maps programmable MBIST negative active flop modules to proper library cells.

Options and Arguments

`-design` *design*	Specifies the name of the top-level design.

Related Information

Affected by this attribute:	pmbist_ffn_cell

map_pmbist_ffsync

```
map_pmbist_ffsync
[-design design]
```

Description

Maps programmable MBIST synchronizer flop modules to proper library cells.

Options and Arguments

`-design` *design*	Specifies the name of the top-level design.

Related Information

Affected by this attribute:	pmbist_ffsync_cell

map_pmbist_map2mux

```
map_pmbist_map2mux[-design design]
```

Description

Maps programmable MBIST map-to-mux modules to proper library cell specified using `pmbist_map2mux_cell` attribute.

Related Information

Affected by this attribute:	pmbist_map2mux_cell

Options and Arguments

`-design design`	Specifies the name of the top-level design.

prepare_for_test_point_analysis

```
prepare_for_test_point_analysis
 [-generate_script_only]
 [design]
```

Description

command will prepare test point analysis by invoking modus

Options and Arguments

`-generate_script_only`	Generates only the modus script .
design	Specifies the name of the top block. This is required if there are multiple top designs.

read_dft_abstract_model

```
read_dft_abstract_model [-ctl][-ctl_block string][-use_scan_structures_shift_enable_only] [-override_scan_lib_cell][-advanced] [-segment_prefix string] [-instance inst|hinst]
[-libcell string] [-assume_connected_shift_enable]file
```

Description

Reads in the scan abstract model of a design that is used as a core or IP block in the current design. The scan abstract model defines the scan chain architecture of the subdesign and is used as scan chain segments in the configuration of the top-level scan chains of the current design.

Options and Arguments

`-advanced`	Instructs the CTL reader to parse the IsConnected constructs for feed-through pins and apply these to the design as dft_controllable pins. In addition, any signal listed as a DataType Constant will be defined as a test mode signal with its ExpectState value used as the active test mode.
	This option must be specified with the `-ctl` option
	The writing of feedthrough pins to the scan abstract models using `dft_controllable` construct in native abstract model, and `IsConnected` construct in CTL abstract model is controlled using the `dft_include_controllable_pins_in_abstract_model` root attribute. The writing of signals as constant values in the native abstract and CTL abstract model is controlled through the `dft_include_test_signal_outputs_in_abstract_model` root attribute.
`-assume_connected_shift_enable`	
	Indicates that the shift-enable port specified in the DFT abstract model for the block being read in is already connected to logic external to this block. Therefore the scan configuration engine does not need to modify the existing connection.
	If you specify this option and the shift-enable pin is *not* connected, the scan configuration engine will *not* make the connection.
	If you do not specify this option, the scan configuration engine will make the connection to the shift-enable port specified in the DFT abstract model. If a connection already existed, it will be first removed.
`-ctl`	Specifies that the scan abstract model was written using the Core Test Language (CTL) format (IEEE format `P1450.6`).
	If you omit this option, the scan abstract model is assumed to consist of a list of define_scan_abstract_segment commands, one scan segment per scan chain in the subdesign.
`-ctl_block` *`string`*	Specifies the CTL block to process.
`file`	Specifies the file that contains the abstract model description.
`-instance` *`inst`*`|`*`hinst`*	
	Applies the scan abstract model to the specified hierarchical instance.
	If you read in an abstract model written in native tool format, this instance must be an instantiation of the subdesign specified through the `-module` option in the abstract model.
	If you read in an abstract model written in CTL format, this instance must be an instantiation of the subdesign specified in the `Environment` section of the CTL file.
	If this option is omitted, the scan abstract model is applied to all instances of the subdesign.
`-libcell` *`string`*	Specifies the libcell of the segment.
`-override_scan_lib_cell`	
	Allows you to redefine the `test_cell` definition of a scan cell in the Liberty library using the test information applied from the CTL file.
	For example, you can use this option to update the sequential length of the scan primitive from a single bit to multi-bit cell when the intended usage of this cell is a multi-bit scan flop synchronization cell.
	This option must be specified with the `-ctl` option.
`-segment_prefix` *`string`*	
	Adds the specified string as a prefix to the
	• Segment name defined in the native tool format file

	• Chain name defined in the CTL file
`-use_scan_structures_shift_enable_only`	
	Indicates that the shift-enable signal for an abstract segment must be read from its `ScanChain` definition in the `ScanStructures` block in the CTL file.
	If a shift-enable signal is not specified, the abstract segment is created using the `-connected_shift_enable` option of the `define_scan_abstract_segment` command
	This option must be specified with the `-ctl` option.

Related Information

Affected by these commands:	check_dft_rules
	connect_scan_chains
Related commands:	write_dft_abstract_model
	write_hdl (`-abstract`)
Sets these attributes:	scan_segment Attributes

read_dft_jtag_boundary_file

`read_dft_jtag_boundary_file` *`iospeclist_file`*

Description

Reads in the specified IOSpecList input file to be used for boundary scan insertion.

The IOSpecList input file is only required to provide information that cannot be inferred from the design, and the command-line options of the `add_jtag_boundary_scan` command.

You need an IOSpecList input file if

- The I/O pad cells in your library do not use the standard pin names
- Your design has pin sharing logic to shared functional output signals that was inserted before you insert boundary scan logic
- You want to customize the location of the boundary cells in the boundary register

You can also use an IOSpecList input file if

- You want to use custom boundary cells
- You want to use user-defined TAP instructions (such as those required for MBIST or PTAM) and use specific opcodes specified using `JTAG_Inline` syntax

  You can also use the `define_jtag_instruction` command to enter user-defined instructions.

Options and Arguments

`iospeclist_file`	Specifies the IOSpecList input file.

Related Information

Refer IOSpecList File Format Specifics for Boundary Scan in chapter Inserting Boundary-Scan Logic of the *Genus Design for Test Guide*.

Affects these commands:	add_jtag_boundary_scan
	write_dft_jtag_boundary_file

read_dft_pmbist_interface_files

```
read_dft_pmbist_interface_files
 -directory string[-tdo_pin string]{design|module|base_cell|lib_cell}
```

Description

Reads interface files for Programmable Memory Built-In-Self-Test (PMBIST) generated by an earlier call to the `write_dft_pmbist_interface_files` command correlated to a particular *design* or *subdesign*.

These interface files represent an abstract model of the previous PMBIST insertion process, supporting not only a bottom-up flow for incremental PMBIST insertion but also the generation of patterns to exercise the PMBIST logic from Modus `create_embedded_test` command.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

{*design*|*module*|*base_cell*|*lib_cell*}	
	Specifies the name of the design, module or base_cell or lib_cell for which the interface files are read.
`-directory` *string*	Specifies the directory that contains the interface files.
`-tdo_pin` *string*	Specifies the user defined tdo pin on the PMBIST inserted sub-design or libcell based on which stitched blocks are ignored.

Related Information

Refer to *PMBIST Interface Files* in the Genus PMBIST Guide.

Related commands:	write_dft_pmbist_interface_files

read_pmbist_memory_view

```
read_pmbist_memory_view
[-cdns_memory_view_file string]
[-cdns_memory_template_file string][-memory string]
[-bitmap string][-verilog string][-overwrite][-mbif string]
[-mbif_setup_file string] [-mbif_modus_tracing_setup_file string]
[-ctl string][-directory string][-preview]
[-dont_save_all_memory_views][-update_existing_views] [-fuse_cif_wrapper string][design]
```

Description

Loads the memory and macro views file for the memories contained within the design.

With `-preview` option, it generates the memory and macro view file template to be corrected and read back in for the memories and macros contained within the design. It also converts the memory views specified in Core Test Language (CTL) to CDNS specific format.

Without `-preview` option, it loads the memory an macro views file for the memories contained within the design.

The `cdns_memory_view_file` can supply memory views containing only the module sections for a design which include all targeted memory port descriptions, address range, read access delay, physical information including data bit order, write mask assignment and memory cell array layout, redundant features and wrapper information. The `cdns_memory_view_file` may also supply macro views containing macro interface definitions. Macro views provide a means to specify shared test bus interface to be used for memory testing, logical memories that can be accessed by a particular interface, physical memories contained within logical memories and their access mechanism. After the view file is loaded, Genus updates the design hierarchy with the views information and prints summary

tables.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-cdns_memory_view_file` *file*	
	Specifies the file containing the physical view of the memories and macros in Cadence format. This file can overload the .lib file, providing information that is missing in the .lib files.
`-cdns_memory_template_file` *string*	
	Specifies the memory template file to generate memory view file in preview mode.
`-memory` *string*	
	Specifies the name of the memory cell for which the memory view file is to be generated.
`-bitmap` *string*	
	Specifies the bitmap file containing physical information of the memory cell for generating the memory view.
`-verilog` *string*	
	Specifies the Verilog file of the memory cell for generating the memory view.
`-ctl` *file*	Specifies a file written using the Core Test Language (CTL) format (IEEE Std 1450.6.2-2014) that contains information on memory modeling.
design	Specifies the name of the top-level design.
`-directory` *string*	Specifies the directory where the template file must be generated.
	Default:. `/pmbist_`*design*
`-mbif` *file*	Specifies a customer-specific MBIST information file for use in generating macro views when the `-preview` option is specified.
`-mbif_modus_tracing_setup_file` *file*	
	Specifies the name of a Tcl file that contains the Modus commands to perform circuit tracing and determine the logical to physical mapping of the memory views.
	Modus has many different tracing options. The default tracing option values are chosen to complete tracing successfully. You can override these values using the Modus commands in the specified file.
	This option must be specified with the `-mbif` option.
`-mbif_setup_file` *file*	
	Specifies the name of a Tcl file that contains the Genus commands to be used during the generation of the macro view.
	This option must be specified with the `-mbif` option.
-overwrite	Overwrites existing simulation log files to generate memory views.
`-preview`	Requests the generation of a template file for the memory or macro views.
`-dont_save_all_memory_views`	
	Saves only those memory views that are used in the design.
	Specify this option if the design contains macros which are black boxes and physical memories reside inside these macros. Normally memory views are only retained within Genus if they are detected within the design. As these memory views must be read prior to reading macro views and memories are not visible within black-boxed macros, the use of this option ensures that all required memory views are available to support macro view physical memory bindings.

`-update_existing_views`	
	Specifies to update existing memory views.
`-fuse_cif_wrapper` *string*	
	Generates common hard repair interface view file with the given wrapper name.

Related Information

Affects this command:	add_pmbist
Related attributes:	memory_data_bit_structure Attributes
	memory_lib_cell Attributes
	memory_lib_pin_action Attributes
	memory_lib_pin_alias Attributes
	memory_spare_column Attributes
	memory_spare_column_map_address Attributes
	memory_spare_column_map_data Attributes
	memory_spare_row Attributes
	memory_spare_row_map_address Attributes
	write_mask_bit Attributes

reset_opcg_equivalent

`reset_opcg_equivalent [lib_cell]...`

Description

Removes the specified scan library cells from the OPCG-equivalency table which was previously defined using a (number of) `set_opcg_equivalent` command(s). If you do not specify any library cells, the command removes all OPCG-equivalent mappings.

Options and Arguments

Related Information

Affects this command:	convert_to_opcg_scan
Related command:	set_opcg_equivalent

reset_scan_equivalent

```
reset_scan_equivalent [lib_cell]...
```

Description

Removes the specified non-scan library cells from the scan-equivalency table which was previously defined using a (number of) `set_scan_equivalent` command(s).

If you do not specify any library cells, the command removes all scan-equivalent mappings.

Options and Arguments

Example

The following example removes the `snl_ffqx1` cell from the scan-equivalency table.

```
reset_scan_equivalent snl_ffqx1
```

Related Information

Affects this command:	convert_to_scan
Related command:	set_scan_equivalent

set_compatible_test_clocks

```
set_compatible_test_clocks
{-all | -none | list_of_test_clocks}
[-dont_check_dft_rules][-domain_name string] [-design design]
```

Description

Specifies the compatible test clocks whose related scan flip-flops can be merged into a single scan chain using lockup elements in between. By default, no test clocks are assumed compatible unless they are defined as independent test clocks in the same test clock domain.

> This command applies only to the muxed scan style.

Test clocks that are declared compatible belong to the same DFT clock domain.

> Only those test clocks with the same clock period can be made compatible as independent test clocks in the same test clock domain.

Options and Arguments

`-all`	Specifies that all test clocks are compatible.
`-design` *design*	Specifies the design for which you want to specify compatible test clocks.
`-dont_check_dft_rules`	
	Prevents the DFT rules from being checked automatically after specifying the compatible test clocks.
`-domain_name` *string*	Specifies the name of the new test clock domain.
list_of_test_clocks	Specifies the compatible test clocks. You must specify the test clock object name.
	To allow combining flip-flops from the same DFT domain—which are triggered by either edge of the same test clock—on the same scan chain, you need to set the `dft_mix_clock_edges_in_scan_chains` root attribute to `true`. By default, only same edge clocks are mixed.
`-none`	Specifies that none of the test clocks are compatible.

Related Information

See the following sections in *Genus Design for Test Guide*: • Mixing Different Test Clocks in the Same Scan Chain • Concatenating Scan Chains	
Affects this command:	connect_scan_chains
Related attribute:	dft_mix_clock_edges_in_scan_chains

set_opcg_equivalent

```
set_opcg_equivalent
-scan_cell lib_cell -opcg_cell lib_cell
[-tieoff_pins string] [-pin_map list_of_pin_groups]
-edge_mode_pin string -loop_back_pin string
```

Description

Controls the `OPCG`-equivalent cell type that is used during the conversion of a scan flip-flop to an `OPCG` cell by the `convert_to_opcg_scan` command.

The command creates an `OPCG`-equivalency table.

An `OPCG` cell is a special scannable cell with an additional (hold) mux in its scan-data path. Depending upon the value of the `-edge_mode_pin`, the mux will either circulate or loopback the inverted value of its output pin to the `-loop_back_pin`, or capture the scan-in data from the scan-data path.

Options and Arguments

`-edge_mode_pin` *string*	
	Specifies the edge mode pin of the `OPCG` cell to connect to.
`-loop_back_pin` *string*	
	Specifies the loopback pin of the `OPCG` cell to connect to.

`-opcg_cell lib_cell`	Specifies the `OPCG` library cell to map to.
`-pin_map list_of_pin_groups`	
	Indicates how to map a pin from the scan flop to a pin in the `OPCG` cell when the pin names in the cells do not match.
	The *`list_of_pin_groups`* has the following format:
	`{{`*`scan_pin opcg_pin`*`} {`*`scan_pin opcg_pin`*`}...}`
`-scan_cell lib_cell`	Specifies a scan flip-flop library cell to be replaced.
`-tieoff_pins string`	Specifies the tie-off value for extra pins on the `OPCG` cell.
	The *`string`* has the following format:
	`{{`*`pin tie_off_value`*`} {`*`pin tie_off_value`*`} ...}`
	The value can be a logic `0` or `1`.

Example

The following example assumes that the pin names in the scan and `OPCG` flip-flops match, and that there are no extra pins in the `OPCG` flop to be tied off.

```
set_opcg_equivalent -scan_cell SDFFQ_X1M -opcg_cell S2DFFQQN_X1M \
    -edge_mode_pin TEL -loop_back_pin TI
```

Related Information

Inserting the Toggle Muxes in *Genus Design for Test Guide*.	
Affects this command:	convert_to_opcg_scan
Related command:	reset_opcg_equivalent

set_scan_equivalent

```
set_scan_equivalent
-non_scan_cell lib_cell -scan_cell lib_cell
[-tieoff_pins string] [-pin_map list_of_pin_groups]
```

Description

Controls the scan-equivalent cell type that is used during the conversion of a non-scan flip-flop which passes the DFT rule checks to a scan flop. Use the `convert_to_scan` command to perform the actual conversion to scan.

> The Genus-DFT engine automatically derives the scan data input, scan data output, and other test signals from the `test_cell` description of the scan flop in the target library.

In most cases Genus will automatically identify scan-equivalent cells in the library from the function definition in the `test_cell` section. However, if the library description is incomplete this command can be used to force equivalence.

Options and Arguments

`-non_scan_cell` *lib_cell*	
	Specifies a non-scan flip-flop library cell.
`-pin_map` *list_of_pin_groups*	
	Indicates how to map a pin from the non-scan flop to a pin in the scan flop when the pin names in the cells do not match.
	The *list_of_pin_groups* has the following format:
	{{*non_scan_pin scan_pin*} {*non_scan_pin scan_pin*}...}
`-scan_cell` *lib_cell*	Specifies a scan flip-flop library cell.
`-tieoff_pins` *string*	Specifies the tie-off value for extra scan cell pins.
	The *string* has the following format:
	{{*pin value*} {*pin value*} ...}
	The value can be a logic `0` or `1`.

Example

The following example assumes that the pin names in the non-scan and scan flip-flops match, and that there are no extra pins in the scan flop to be tied off.

```
set_scan_equivalent -non_scan_cell snl_ffqx1 -scan_cell snl_sffqx1
```

Related Information

See the following sections in *Genus Design for Test Guide*:

- Defining Scan-Equivalency between Non-Scan and Scan Flops to Map to Scan
- Controlling Mapping to Scan in a Mapped Netlist

Affects this command:	convert_to_scan
Related command:	reset_scan_equivalent

update_scan_chains

```
update_scan_chains
[-flops inst|hinst] [-chains actual_scan_chains] [-type {dfa|rrfa|user|compression_testpoints}] [-max_list_flops integer] [-preview] [design] [> file]
```

Description

Inserts the identified scannable test points in the existing scan chains. The scan chains must be connected before inserting these test points.

> Test points can only be added to scan chains that have not been compressed.

Options and Arguments

`-chains` *actual_scan_chains*	
	Specifies the names of the actual scan chains to be updated

	By default, all actual scan chains are updated.
design	Specifies the design in which you want to update the scan chains.
-flops *inst|hinst*	Specifies the test point instances to be added.
	This option can't be used together with the -type option.
-max_list_flops *integer*	
	Specifies the maximum number of test point flops to report.
	By default, all test points will be reported.
-preview	reports what would be done, but does not update the scan chains
-type {dfa|rrfa|user|compression_testpoints}	
	Specifies the type of test points to add to the scan chains.
	Test points can be inserted after DFA or RRFA analysis or can be user-inserted.
	This option can't be used together with the -flop option.
	By default, all test point types are inserted.

Example

The following command shows a preview of the scan chains after they would be updated with the test points inserted by the DFA analysis.

```
update_scan_chains -type dfa -preview \
    -chains [get_db design:DLX_CORE actual_scan_chains *]
```

Related Information

Inserting Scannable Test Points in Existing Scan Chains in *Genus Design for Test Guide*

Affected by these commands:	add_analyzed_test_points
	add_test_point
	add_user_test_point
Related command:	connect_scan_chains

write_dft_abstract_model

```
write_dft_abstract_model[-ctl]
[-dft_cfg_mode dft_config_mode]
[-include_domain_blocking_information][-include_internal_pins][-write_as_libcell]
[design][> file]
```

Description

Writes a scan abstract model for all the top-level scan chains configured in the design.

Besides the command options, following root attributes also affect the information written to the abstract models:

- The `dft_include_controllable_pins_in_abstract_model` attribute allows for feedthrough pin connections to be written using the `dft_controllable` construct to the native abstract model, and using the `IsConnected` construct to the CTL model.

- The `dft_include_test_signal_outputs_in_abstract_model` attribute allows output signals whose values are constant in test setup, and output signals assigned to tied constant values, to be written as test mode signals in the native abstract model, and as constants in a CTL abstract model.

Currently, this command is not supported for the clocked LSSD scan style with the `-ctl` option.

Options and Arguments

`-ctl`	Writes out a scan abstract model in the Core Test Language (CTL) format (IEEE format `P1450.6`).
	If you omit this option, the scan abstract model is written in native Genus format that consists of a list of define_scan_abstract_segment commands, one per top-level scan chain.
design	Specifies the design for which to write out the scan abstract model of the scan chains.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dft_cfg_mode` *dft_configuration_mode*	
	Writes scan chain information related to the specified scan mode name.
file	Specifies the file to which the output must be written.
	You can write out the CTL file in compressed format by specifying a file name with the `.gz` extension.
	Default: output is written to the screen
`-include_domain_blocking_information`	
	Adds the required OPCG clock domain information created during block-level processing in a domain-blocking flow.This information is applied to the block at the next level of integration and used when inserting OPCG logic with domain blocking at the top-level design.
`-include_internal_pins`	Writes out the clock domain information.
`-write_as_libcell`	Writes out the abstract information as a libcell.

Example

In the following example, the different active edges of the different test clocks in the same test clock domain are allowed to be mixed on the same scan chains. Following shows the configuration result and the scan abstract models for the scan chains:

```
genus:root:> connect_scan_chains    Configuring 1 chains for 27 scan f/f  Configured 1 chains for Domain: 'clkAll', edge: 'mixed'         AutoChain_1
(DFT_sdi_1 -> DFT_sdo_1) has 27 registers; Domain:clkAll, edge: mixed  Processing 1 scan chains in 'muxed_scan' style.  Using default shift enable signal
'SE': 'port:test/SE' activehigh.    Connecting scan chain 'AutoChain_1' with 27 flip-flops.Mapping DFT logic introduced by scan chain connection...Mapping
DFT logic done.Reporting 1 scan chain Chain 1: AutoChain_1  scan_in:      DFT_sdi_1  scan_out:     DFT_sdo_1    shift_enable: SE (active high)
clock_domain: clkAll (edge: mixed)  length: 27    bit 1       out1_reg_4 <test_clk1/fall>    ...    bit 5       out1_reg_8 <test_clk1/fall>    llatch 5
DFT_lockup_g1    bit 6       out2_reg_4 <test_clk2/fall>    ...    bit 10      out2_reg_8 <test_clk2/fall>    llatch 10   DFT_lockup_g348    bit 11
out3_reg_4 <test_clk3/fall>    ...    bit 18      out1_reg_2 <test_clk1/rise>    bit 19      out1_reg_3 <test_clk1/rise>    llatch 19   DFT_lockup_g349
bit 20      out2_reg_0 <test_clk2/rise>    ...    bit 23      out2_reg_3 <test_clk2/rise>    llatch 23   DFT_lockup_g350    bit 24      out3_reg_0
<test_clk3/rise>    ...    bit 27      out3_reg_3 <test_clk3/rise>------------------------
genus:root:> write_dft_abstract_model
scan style is muxed_scan

# writing abstract model for 1 scan chain

  define_scan_abstract_segment -module test \    -name test_AutoChain_1 \    -sdi DFT_sdi_1 -sdo DFT_sdo_1 \    -shift_enable_port SE -active high \    -
clock_port clk1 -fall \    -tail_clock_port clk3 -tail_edge_rise \    -length 27
```

To avoid naming collisions when reading in a scan abstract model, the segment names are prefixed with the module name.

Related Information

Affected by these commands:	connect_scan_chains
Affected by these attributes:	actual_scan_chain Attributes
	dft_include_controllable_pins_in_abstract_model
	dft_include_test_signal_outputs_in_abstract_model
Related command:	read_dft_abstract_model
	write_hdl (`-abstract`)

write_dft_atpg

```
write_dft_atpg
[-ncsim_library string [-library string]]
[-directory string] [-run_from_directory]
[-delay] [-generate_config_file]
[-compression]
[-build_faultmodel_options string]
[-build_model_options string]
[-build_testmode_options string]
[-atpg_options string]
[-verify_test_structures_options string] }
[-force] [-continue_with_severe] [-serial]
[-core_verilog string] [-atpg_testmodes string]
[-hier_test_core] [design]
```

Description

Writes out the necessary files and the template run scripts to run Automatic Test Pattern Generator (ATPG) using the Modus or Modus software. By default, files and scripts are written using Modus software.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

The generated scripts depend on the setting of the dft_true_time_flow root attribute. There is one additional file called `tt_setup` that is written out when you set `dft_true_time_flow` attribute to true. Use the Modus `true_time` command to regenerate the `runmodus.atpg.tcl` script after making modifications to the `tt_setup` file.

This command generates the following files:

- *topmodulename*`.et_netlist.v`—A netlist for Modus
- `runmodus.atpg.tcl` —A template script file to run the requested tasks such as pattern generation or testability analysis
- *topmodulename*`.`*TESTMODE*`.pinassign`—A pin-assignment file that captures the top-level scan-related signals and their applicable test function used to build and validate the specific testmode. For example the test signals for FULLSCAN testmode include shift-enable, test-mode, test-clock and scan data IOs. For OPCG- based testmodes, this file also includes the OPCG test signals with their appropriate test functions and the oscillator and domain related information.
- *topmodulename*`.`*TESTMODE_OPCGModeName*`.seqdef`—A sequence definition file generated when OPCG logic is inserted in the design and the command is run with the `-delay` option. For the `true_time` flow, add the `-opcg_mode` option as well. This file is used to initialize the PLL, establish the OPCG mode, and validate their specific test mode.
- `run_`*TESTMODE*`_sim[_`*OPCGModeName*`]` — An Xcelium simulator run file used to simulate the test patterns created by Modus for the given test mode on the design containing the applicable test logic.
- `run_`*TESTMODE*`_sim[_`*OPCGModeName*`]_sdf` — An Xcelium simulator run file used to simulate the test patterns created by Modus with SDF back annotated for the given test mode on the design containing the applicable test logic.

When using the -unix_shell option, a similar set of files as listed above are generated for use with Encounter Test. The name of the netlist file is *topmodulename.et_netlist.v*. The Encounter Test run script is named runet.atpg. In addition, an et_checks.sh file is generated.

Options and Arguments

-atpg_options {*-option1 value -option2 value*}	
	Specifies extra ATPG analysis options to include for create_logic_tests commands in the runmodus.atpg.tcl script.
	Specify the extra options as "*option1*=*value option2*=*value*" when using -unix_shell argument as part of the command.
	This option does not apply to the true time flow.
-atpg_testmodes *string*	
	Specifies to use ATPG testmodes flow.
-build_faultmodel_options {*-option1 value -option2 value*}	
	Specifies a string containing the extra options to build a fault model in the runmodus.atpg.tcl script.
	Specify the extra options as "*option1*=*value option2*=*value*" when using -unix_shell argument as part of the command.
	For more information on these options, refer to the build_faultmodel command in the Modus documentation.
	This option does not apply to the true time flow.
-build_model_options {*-option1 value -option2 value*}	
	Specifies extra options to apply when building the model in the runmodus.atpg.tcl script.
	Specify the extra options as "*option1*=*value option2*=*value*" when using -unix_shell argument as part of the command.
	For more information on these options, refer to the build_model command in the Modus documentation.
	This option does not apply to the true time flow.
-build_testmode_options {*-option1 value -option2 value*}	
	Specifies extra options to apply when building the test mode.
	Specify the extra options as "*option1*=*value option2*=*value*" when using -unix_shell argument as part of the command.
	For more information on these options, refer to the build_testmode command in the Modus documentation.
	This option does not apply to the true time flow.
-compression	Generates additional files to run ATPG and to verify compression logic.
-continue_with_severe	

	Allows the continuation of the Modus script even when severe warnings occur during execution of the commands in the script.
`-core_verilog` *string*	
	Specifies the core level Verilog path for ATPG testmodes flow.
`-delay`	Specifies to generate additional files for Modus to run delay tests. In the default flow, the design contains OPCG logic.
	• In the default flow, you cannot combine this option with the `-cfg_mode_order` option.
	• In the true time flow, you can combine this option with the `-cfg_mode_order` option. In this flow, this option specifies to write out delay tests even when no OPCG logic was inserted.
design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	
	Specifies the directory to which the output files must be written.
	Default: *current*_synthesis_run_directory`/test_scripts`
`-force`	Specifies to skip certain checks to validate the design and test setup are complete and ready for ATPG. Allows the `write_dft_atpg` command to continue even if the DFT logic appears to be incomplete.
-generate_config_file	
	Generates the configuration file for Modus.
`-hier_test_core`	Instructs to write out the files needed to generate migratable patterns for a hierarchical test core flow.
	The design must have been taken through the "Hierarchical Test Flow: Preparing a Core" flow.
`-library` *string*	Specifies the list of Verilog structural library files required to run Modus ATPG. For mapped design, this option is required.
	You can specify the files explicitly or you can specify an *include* file that lists the files. You can also specify directories of Verilog files but you cannot reference directories in an include file.
	For example, if the Verilog files required to run ATPG are:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	`write_dft_atpg` can be used in either of the following ways:
	1. If specifying files separately on the command line:
	`write_dft_atpg -library "./padcells.v ./stdcells.v \ ./memories ./ip_blocks" ...`
	1. If using an include file. Create an include file named `include_libraries.v` containing:
	`` `include "./padcells.v" `include "./stdcells.v" ``
	And then specify the following:
	`write_dft_atpg -library "include_libraries.v ./memories \ ./ip_blocks" ...`
	If you specify a relative path, the command interprets the path to be the location from where Modus will be run.
`-ncsim_library` *string*	

	Specifies the list of library files required for the Xcelium simulation of the generated vectors. If this option is not used, then the list of libraries specified with `-library` option are used.
	For more information on how to specify the list of files, refer to the `-library` option.
`-run_from_directory`	
	Allows to run the Automatic Test Pattern Generator (ATPG) from the working directory specified with the `-directory` option.
	Default: directory where scripts were written from, that is, directory where Genus was run.
`-serial`	Generates additional files for Modus to verify serial testmodes: `SERIAL_INTEST`, `SERIAL_EXTEST`, and `SERIAL_BYPASS` for use with hierarchical test/ATPG flows.
`-verify_test_structures_options {-option1 value -option2 value}`	
	Specifies extra options to apply when performing test structure verification for Modus.
	Specify the extra options as "*option1*=*value option2*=*value*" when using `-unix_shell` argument as part of the command.
	This option does not apply to the true time flow.

Examples

- The following command generates the files to run an ATPG-based testability analysis.

```
write_dft_atpg -directory atpg -library $sim/mylib.v

Examining the atpg directory that was generated shows the following files: genus:root:> shell ls atpg dig_top.test_netlist.v
dig_top.FULLSCAN.pinassign run_fullscan_sim run_fullscan_sim_sdf runmodus.atpg.tcl
```

- The following command uses the `-cfg_mode_order` option to generate the files to run an ATPG-based testability analysis first using the `OPMISRPLUS_DECOMP` compression mode and then with the `FULLSCAN` mode.

```
write_dft_atpg -cfg_mode_order {OPMISRPLUS_DECOMP FULLSCAN} \
 -directory rc_et
```

- The following command generates files to run ATPG for compression mode with fullscan top-off. This example is for a design with an XOR compression design implemented with an XOR-decompressor.

```
write_dft_atpg -directory atpg -library $sim/mylib.v -compression

Examining the atpg directory that was generated shows the following files:
genus:root:>shell ls atpg
dig_top.COMPRESSION_DECOMP.pinassign
dig_top.COMPRESSION.pinassign
dig_top.FULLSCAN.pinassign
dig_top.test_netlist.v
run_compression_decomp_sim
run_compression_decomp_sim_sdf
run_compression_sim
run_compression_sim_sdf
run_fullscan_sim
run_fullscan_sim_sdf
runmodus.atpg.tcl
```

- The following command generates files to run ATPG for compression mode with fullscan top-off. This example is for a design with an XOR compression design implemented with broadcast (Illinois scan).

```
write_dft_atpg -directory atpg -library $sim/mylib.v -compression -delay

Examining the atpg directory that was generated shows the following files:
genus:root:>shell ls atpg
dig_top.COMPRESSION_DFT_opcg_mode_1.pinassign
dig_top.COMPRESSION_DFT_opcg_mode_1.seqdef
dig_top.COMPRESSION.pinassign
```

```
dig_top.FULLSCAN_DFT_opcg_mode_1.pinassign
dig_top.FULLSCAN_DFT_opcg_mode_1.seqdef
dig_top.FULLSCAN.pinassign
dig_top.test_netlist.v
run_compression_sim_DFT_opcg_mode_1
run_compression_sim_DFT_opcg_mode_1_sdf
run_fullscan_sim
run_fullscan_sim_DFT_opcg_mode_1
run_fullscan_sim_DFT_opcg_mode_1_sdf
run_fullscan_sim_sdf
runmodus.atpg.tcl
```

- The following command uses the -cfg_mode_order option to generate the files to run an ATPG-based testability analysis first using the OPMISRPLUS_DECOMP compression mode followed by the FULLSCAN mode.

  ```
  write_dft_atpg -cfg_mode_order "OPMISRPLUS_DECOMP FULLSCAN"  -directory atpg
  ```

- The following command is run prior to inserting scan chains. The files generated are for an ASSUMED scan mode for ATPG testability analysis. ASSUMED scan mode is only supported for non-true-time flow.

  ```
  write_dft_atpg -directory atpg -library $sim/mylib.v
  ```

  ```
  Examining the atpg directory that was generated shows the following files:
  genus:root:>shell ls atpg
  dig_top.ASSUMED.pinassign
  dig_top.test_netlist.v
  runmodus.atpg.tcl
  test.exclude
  test.modedef
  ```

`test.exclude` is a file listing objects to be excluded from the ATPG analysis in an assumed scan mode. `test.modedef` is a mode definition file that describes the test mode in ASSUMED scan mode.

Related Information

See the following sections in *Genus Design for Test Guide*:

- Using Modus to Analyze Testability in *Genus Design for Test Guide*.
- Writing the Scripts and Setup Files to Perform ATPG in "Exporting the Design"
- Defining the Oscillator Sources in "Inserting On-Product Clock Generation Logic"
- Generate Files for ATPG and Simulation in "Inserting On-Product Clock Generation Logic"
- Compression Logic Verification and Test Vector Generation Using Modus in "Inserting Scan Chain Compression Logic"
- Using Modus to Perform a Deterministic Fault Analysis on a Scan Connected Netlist
- Hierarchical Test Flow: Preparing a Core in "Hierarchical Test"
- Hierarchical Test Flow: Integrating the Top Level in "Hierarchical Test"
- ICL/PDL for Core and Top Level in "Hierarchical Test"

write_dft_atpg_other_vendor_files

```
write_dft_atpg_other_vendor_files
{ -mentor | -stil} [> file]
[-decimals_ok] [-picoseconds]
[-test_clock_waveform test_clock]
[-apply_inputs_at integer]
[-apply_bidirs_at integer]
[-strobe_outputs_at integer]
[-strobe_width integer] [design]
```

Description

Writes out the scan-chain information for an Automatic Test Pattern Generator (ATPG) tool in a format readable by the designated ATPG tool.

The ATPG tool uses this information to generate appropriate test patterns. The file extension given to the interface file(s) is determined by the selected tool.

The interface file is useful only to the third-party tool if the test synthesis tool has connected the scan chain. Therefore, you should use this command only if the test synthesis tool has connected the scan chains.

Options and Arguments

`-apply_bidirs_at` *integer*	
	Specifies when in the test clock cycle to apply the bidirectional signals. Specify this time as a percentage of the test clock period.
	Default: Same value as specified for `-apply_inputs_at`
`-apply_inputs_at` *integer*	
	Specifies when in the test clock cycle to apply the input signals. Specify this time as a percentage of the test clock period.
	Default: 0
`-decimals_ok`	Writes out decimal numbers. By default, time values are rounded off to integer numbers because many ATPG tools do not accept decimal numbers for test waveform time values. Use the `-picoseconds` option to minimize round-off errors.
design	Specifies the top module for which to write ATPG.
file	Specifies the file to which the output must be written.
	If no file is specified, the output is written to standard out (`stdout`) and to the log file.
`-mentor`	Creates an interface file in the format used by the Mentor Graphics ATPG tool. Files generated:
	• `top_module.testproc`
	• `top_module.dofile`
`-picoseconds`	Specifies to use picoseconds for the time unit. Use this option to minimize the round-off errors when rounding-off test waveform time values to integers.
	Default: `nanoseconds`
`-stil`	Creates an interface file in the IEEE Standard Test Interface Language (STIL) format (IEEE format 1450.1).
	The generated STIL format is TetraMAX compatible.
`-strobe_outputs_at` *integer*	

	Specifies when in the test clock cycle to strobe the outputs. Specify this time as a percentage of the test clock period.
	Default: 40
`-strobe_width` *integer*	
	Specifies how long the outputs are valid during the test clock cycle. Specify this time as a percentage of the test clock period.
	Default: 0
`-test_clock_waveform` *test_clock*	
	Specifies to use the clock waveform of the specified test clock.
	Default: first test clock object found

Related Information

Affected by this command:	compress_scan_chains
	connect_scan_chains
	define_test_clock
Affected by these attributes:	actual_scan_chain Attributes

write_dft_bsdl

```
write_dft_bsdl
[-pin_map_file file]
[-bsdl_package_name files]
[-file file]
[-include_private_instructions]
[-expose_ports_with_pin_map]
-directory string [-design design]
```

Description

Generates a file describing the boundary scan architecture of the design in Boundary Scan Description Language (BSDL), along with two VHDL package files, `STD_1149_1_2001` and `CDNDFT_1149_1_2001`, which contain the supported boundary cell descriptions that were used during boundary scan insertion.

> Dedicated test-related signals (such as shift-enable, and test-mode signals defined without the `-shared_in` option) are also written to the BSDL file along with their respective compliance enable values.

The BSDL file is required by board and system test software to test the interconnects between chips and modules. It is also used by several applications in the Modus DFT Software product such as Boundary Scan Verification (BSV) and MBIST insertion and verification. More details are available in the section titled Writing a BSDL File in *Genus Design for Test Guide*.

Options and Arguments

`-bsdl_package_name` *files*	
	Specifies the name of a VHDL file or a comma-separated list of VHDL files, each containing one or more custom boundary cell descriptions that were used during boundary scan insertion.
	The name of each package file is added to the BSDL file in a `use` statement.
	This option is required if you used custom boundary cells in the design.
`-design` *design*	Specifies the design for which to generate the BSDL file.
`-file` *file*	Specifies the name of the BSDL file to be generated.
	If you omit this option, the output file is named using the *topmodulename*`.bsdl`.
`-directory` *string*	Specifies the directory to which the output files must be written.
`-expose_ports_with_pin_map`	
	Specifies to only expose functional and test ports with package pinmap information to the output BSDL file.
	Additionally, this option prevents the writing of other ports connected in the boundary-scan register without package pinmap information. The names of these other ports will not appear in the `BOUNDARY_REGISTER` section of the BSDL file. Rather, these port names will be represented with an asterisk (`*`) and their associated boundary-scan cells will be represented as `INTERNAL` as shown in the following BSDL snippet:
	`attribute BOUNDARY_REGISTER of test: entity is ... "8 (BC_OUT, *, INTERNAL, X)," &`
	When the `-expose_ports_with_pinmap` is not specified, all ports in the design will be written to each relevant section of the BSDL file, regardless of whether any pinmap information has been provided for any ports.
	Package pinmap information may be provided during boundary-scan insertion or when writing the BSDL file using the `-pinmap_file` option.
`-include_private_instructions`	
	Specifies to include register access information for private instructions in the BSDL file.
`-pin_map_file` *file*	
	Specifies the name of the pinmap file to be used to create a BSDL file.
	This file can have fewer pin-to-pad bonding requirements than the pinmap file specified for the boundary scan insertion.
	Refer to Pinmap File for more information.

Example

The following command creates a BSDL file named `bsdlout`. The name of the package file for the custom boundary cells is added to the BSDL file.

```
write_dft_bsdl -directory .-bsdl_package_name MY_BIDIR_PKG -bsdlout bsdlout
```

This causes the following line to be added to file `bsdlout`:

```
use MY_BIDIR_PKG.all ;
```

Related Information

Affected by this command:	add_jtag_boundary_scan
Related constraints:	define_test_clock
	define_test_signal -function <shift_enable | test_mode>

write_dft_compression_macro

```
write_dft_compression_macro
{-chains integer | -scan_in integer -scan_out integer}
-sub_chains integer
[-decompressor {broadcast | xor }]
[-compressor
 { xor [-scan_in_pipeline_depth integer]
       [-scan_out_pipeline_depth integer]
       [-compressor_pipeline_depth integer]
 | opmisr | hybrid
 | smartscan_xor -smartscan_ratio integer
   [-smartscan_no_update_stage] [-smartscan_serial_only]
   [-smartscan_pulse_width_multiplier {1|2|4}] }]
[-mask {wide0 | wide1 | wide2}]
[-mask_sharing_ratio integer ]
[-no_full_scan_muxing] [-jtag_control]
[-serial_misr_read]
[-shared_scan_in_pins string
  -asymmetric_scan_in integer]
[-block_select] [-compressed_chains integer]
[-separate_mask_ports]
[-allow_removal_of_cancelling_terms_for_hierarchical_flow]
[-low_pin_compression] [-bitwise_compressor]
[-file file] [-info_file file]
```

Description

Generates the RTL for a customized scan compression macro.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-allow_removal_of_cancelling_terms_for_hierarchical_flow`	
	Generates a macro which prevents cancellation of compression channels within higher-level compression macros. Each scan channel is observed at an odd number of scan outputs.
`-asymmetric_scan_in` *integer*	
	Specifies the number of scan in pins for an asymmetric compression macro.
	This option is only supported with `wide1` and `wide2` types of channel masking.
	For wide1 type of masking, the number of scan data input pins must equal the number specified for the `-chains` option minus one (*number_of_chains*`-1`), while for wide2 type of masking, the number must equal the specified number of chains minus two (*number_of_chains*`-2`).
`-bitwise_compressor`	Generates a compressor with bitwise xoring of the channels (that is, each channel feeds only a single SO).
	Use this option for leaf or mid-level blocks during hierarchical compression to avoid simulation issues.

`-block_select`	Adds additional logic to bypass the block for compression inserted at the block level. In compression mode, the scan outputs are forced to zero. In uncompressed scan mode, the scan outputs are fed directly from the scan inputs.
	An extra `BLOCK_SELECT` pin—to control the additional block select logic—is added to the compression macro.
	If you specify this option with the `-separate_mask_ports` option, the mask registers are also bypassed by feeding the `MASK_OUT` ports directly from the `MASK_IN` ports.
	The `BLOCK_SELECT` pin should be held low to bypass the block and should be held high otherwise.
`-chains` *integer*	Specifies the number of top level scan data input/scan data output pairs. Typically, this is the number of uncompressed scan chains. For MISR-based compression, the `-chains` option must be greater than or equal to 16.
	You cannot specify this option when you specify the `-scan_in` and `-scan_out` options. When these options are specified, the number of chains equals the lesser of the following: the scan_in plus the number of shared control pins, or `scan_out`.
`-compressed_chains` *integer*	
	Specifies the number of scan chains in lower-level blocks that are already compressed.
	The command adds the `CCHAN_SI` and `CCHAN_SO` ports to the macro to connect to the compressed chains at a lower block directly. The `CCHAN_SO` port feeds the channel data directly into the compressor.
	This option cannot be specified when the `-compressor` option is set to `smartscan_xor`.
`-compressor {xor | misr| hybrid | smartscan_xor}`	
	Specifies the type of compression logic to be built:
	• `xor` specifies to build an XOR-based compressor
	• `misr` specifies to build a `MISR`-based compressor
	• `hybrid` specifies to build `MISR` compression with `MISR` bypass capability to effectively result in an XOR-based compressor.
	• `smartscan_xor` specifies to add smartscan logic to the XOR-based compression macro.
	Default: `xor`
`-compressor_pipeline_depth` *integer*	
	Specifies the number of pipeline stages to be inserted within the compressor block between the last flop in the scan channels and the compression macro output ports.
	Applies only to an XOR-based compressor.
`-decompressor {broadcast | xor}`	
	Specifies the type of decompression logic to be built:
	• `xor` specifies to build an XOR-based spreader network in addition to the broadcast-based decompression logic. This is the default.
	• `broadcast` specifies to build a broadcast-based decompression logic (simple scan fanout).
`-file` *file*	Specifies the filename where the compression macro RTL will be written. If not specified, the RTL will be written to `stdout`.
`-info_file` *file*	Specifies a file containing more detailed information about the compression macro. This script can be sourced into the current session to provide more information about the compression macro to commands such as `write_et` so they can generate accurate input files for Modus.

`-jtag_control`	Specifies to include a JTAG-controlled test data register (TDR) which generates compression test signals to configure the compression testmode.
`-low_pin_compression`	
	Enables low pin count compression.
`-mask {wide0 | wide1 | wide2}`	
	Inserts scan channel masking logic of the specified type.
	The masking types that can be used depend on the compressor type specified with the `-compressor` option.
	Default: No masking support.
`-mask_sharing_ratio` *integer*	
	Specifies the number of internal scan channels sharing a mask register. The specified integer may not exceed the value specified for the compression ratio.
	This option is only valid with `wide1` and `wide2` masking.
`-no_full_scan_muxing`	
	Specifies to exclude additional muxing logic to the compression macro. By default, additional muxing is added to the compression macro to concatenate the compressed scan channels into uncompressed fullscan chains. If such muxing exists outside the compression macro, specify this option to exclude this logic from the compression macro.
`-scan_in` *integer*, `-scan_out` *integer*	
	Specify the width of the `RSI_SI` and `DSO_SO` ports (the number of scan data inputs and scan data outputs to the compression macro) respectively.
	These options apply for MxN compression in any of the following cases:
	• M<N
	• M<N-1 and wide1 masking is used: one of the scan data inputs can be shared with the CME pin
	• M<N-2 and wide2 masking is used: two of the scan data inputs can be shared with the CME0 and CME1 pins
	• M>N
`-scan_in_pipeline_depth` *integer*	
	Specifies the number of pipeline stages required at the scan data input side.
`-scan_out_pipeline_depth` *integer*	
	Specifies the number of pipeline stages required at the scan data output side.
`-separate_mask_ports`	
	Creates separate `MASK_IN` and `MASK_OUT` ports that are used for block level compression processing in the hierarchical compression flow.
	The number of `MASK_IN/MASK_OUT` ports that is added, is the same number as the number of scan data input ports.
`-serial_misr_read`	Specifies to include support for reading MISR bits serially through the scan data pins.
`-shared_scan_in_pins` *string*	
	Specifies the pins that can be shared with the scan data input pins. Separate the pin names using one or more blanks.
	• For `-mask wide1`, specify *CME*

	• For `-mask wide2`, you can specify the following values: `CME0,CME1,` or `CME0 CME1`
	This option enables the use of an asymmetrical compression macro.
	This option will be ignored and a warning will be given if you also specified the `-scan_in` and `-scan_out` options and the value for `scan_in` is larger than the value for `scan_out`.
`-smartscan_no_update_stage`	
	Prevents the insertion of update registers between the deserializer and the decompressor. In this case, lockup latches are inserted between the deserializer flops and the decompressor.
	You cannot specify this option when you have set the `-smartscan_pulse_width_multiplier` option to either `2` or `4`.
	By default, the tool inserts update registers between the deserializer and the decompressor.
`-smartscan_pulse_width_multiplier {1|2|4}`	
	Determines whether to add clock divider logic to widen the clock pulse going to the scan chains. You can specify the following values: • 1—no logic added • 2—increases the scan path through the SmartScan clock controller with 1 bit • 4—increases the scan path through the SmartScan clock controller with 2 bits
	Default: `1`
`-smartscan_ratio` *integer*	
	Specifies the number of parallel scan data input pins that correspond to a single serial scan data input pin. The number of defined (fullscan) chains must be an integral multiple of the specified smartscan ratio.
`-smartscan_serial_only`	
	Specifies to only insert the smartscan serial-only interface. The number of deserializer (and serializer) registers will match the number of the defined chains. When building the model for Modus you will need to add the pseudo pins for the parallel interface.
`-sub_chains` *integer*	
	Specifies the number of compressed scan channels that exist in the design or that you will build.

Examples

- The following command writes an XOR-based compression macro without masking to the file `xor1.v`.

```
genus:root:> write_dft_compression_macro -compressor xor -chains 8 \
-sub_chains 88 -file xor1.v
Checking out license 'Encounter_Test_Architect'... (1 seconds elapsed)
...
```

- The following command writes an XOR-based compression macro with masking logic of type `wide1`. The -`no_full_scan_muxing` option is specified so the logic to concatenate the compressed `sub_chains` into fullscan chains will be excluded.

  > Since the `-no_full_scan_muxing` option is specified, the number of sub_chains is no longer required to be evenly divisible by the number of chains.

```
genus:root:> write_compression_macro -compressor xor -mask wide1 -chains 8 \
-sub_chains 85 -no_full_scan_muxing -file xor2.v
Checking out license 'Encounter_Test_Architect'... (1 seconds elapsed)....
```

- The following command writes an MISR-based compression macro with masking logic of type `wide1`, with decompression logic of type `xor`. The compressor type is `hybrid` which means the MISR can be bypassed resulting in XOR compression.

```
genus:root:> write_dft_compression_macro -compressor hybrid \
-decompressor xor -mask wide1 -chains 16 -sub_chains 512 -file hybrid1.v
Checking out license 'Encounter_Test_Architect'... (2 seconds elapsed)
...
```

- The following command writes a MISR-based compression macro with masking logic of type `wide2`. The `-info_file` option is also specified.

```
genus:root:> write_dft_compression_macro -compressor opmisr -mask wide2 \
 -chains 20  -sub_chains 500 -file opmisr1.v -info_file opmisr1.info
....
```

- The following command generates an asymmetric pipelined XOR-based compression macro, with masking logic of type `wide2`, with five top-level chains, of which two are shared with the mask control signals, with separate mask ports, with additional logic to bypass the block for compression inserted at the block level, and with two pipeline stages at the scan data output side.

```
write_sdt_compression_macro -chains 5 -sub_chains 9 -asymmetric_scan_in 3 \
-shared_scan_in_pins "CME0 CME1" -mask wide2 -separate_mask_ports \
-block_select -scan_out_pipeline_depth 2 -file my_comp_macro.v
```

- The following command generates a pipelined XOR-based compression macro, with masking logic of type `wide1`, with two top-level chains, with one pipeline stage at the scan data input side, six pipeline stages at the scan data output side, and without additional muxing logic.

```
write_dft_compression_macro -chains 2 -sub_chains 5 -mask wide1 \ -scan_in_pipeline_depth 1 -scan_out_pipeline_depth 6 -
no_full_scan_muxing
```

- The following command generates an asymmetric SmartScan-based compression macro with both parallel and serial interface, with masking logic of type `wide1`, with eight fullscan chains, 32 compression channels and a SmartScan ratio of 4.

```
write_dft_compression_macro -chains 8 -sub_chains 32 -mask wide1 \
-compressor smartscan_xor -decompressor xor  -smartscan_ratio 4
```

- The following command generates an asymmetric SmartScan-based compression macro with serial interface only, with masking logic of type `wide1`, with two fullscan chains, 32 compression channels and a SmartScan ratio of 4.

```
write_dft_compression_macro -chains 2 -sub_chains 32 -mask wide1 \
-compressor smartscan_xor -decompressor xor  -smartscan_ratio 4 \
-smartscan_serial_only
```

write_dft_compression_test_points

```
write_dft_compression_test_points
[design][-library string]
[-directory string][-run_from_directory]
[-compression_testpoints_options string] [-compression_testpoints_options_file string][-build_model_options string]
[-build_faultmodel_options string]
[-build_testmode_options string]
[-verify_test_structures_options string]
[-min_slack_for_no_tp_file integer]
[-dont_create_compatibility_matrix]
[-force][-continue_with_severe][-intest]
```

Description

Writes out the necessary files and the template run scripts to run scripts to run Compression Testpoint Analysis using the Modus software.

This command generates the following files:

- `test.exclude`—A file listing objects to be excluded from the test point analysis in an `ASSUMED` scan mode.
- `test.modedef`—A mode definition file, a text file that describes the test mode in `ASSUMED` scan mode
- *topmodulename*`.ASSUMED.pinassign`—A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock) and their test function used to build the testmode before actual scan chains exist in the design
- *topmodulename*`.FULLSCAN.pinassign`—A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock and scan data IOs) and their test function used to build the testmode when actual scan chains exist in the design
- *topmodulename*`.noTpFile`—A file listing objects that have been constrained, thus preventing test point insertion on them.
- *topmodulename*`.test_netlist.v`—A netlist for Modus.
- `runmodus.compression_testpoints.tcl`—A script file to run compression test point analysis in Modus.

> Some files can be customized according to the setup requirements.

Options and Arguments

`-build_faultmodel_options "-option1 value -option2 value"`	
	Specifies a string containing the extra options to build a fault model.
`-build_model_options "-option1 value -option2 value"`	
	Specifies extra options to apply when building the Modus model.
`-build_testmode_options "-option1 value -option2 value"`	
	Specifies extra options to apply when building the test mode for Modus.
`-compression_testpoints_options "-option1 value -option2 value"`	
	Specifies the extra options to run Compression Testpoints Analysis in a string.
`-compression_testpoints_options_file` *string*	
	Specifies the file where each line contains extra options to run Compression Testpoints Analysis. Multiple testpoint analysis files will be generated (one for each line). Requires MOD400 license.
`-continue_with_severe`	
	Allows the continuation of the test script even when severe warnings occur during execution of the commands in the script.
design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	Specifies the directory to which the output files must be written.

	Default: `current_working_directory/test_scripts`
`-force`	Specifies to continue even if the JTAG macro used to enable the JTAG-controlled compression macro is not found.
`-intest`	Specifying test point analysis in fullscan intest testmode.
`-library string`	Specifies the list of Verilog structural library files.
	The Verilog libraries required to run Modus ATPG of the generated vectors must be provided using the `-library` option of the `write_dft_compression_test_points` command. These libraries must be in a structural format. The files can be specified separately on the `write_dft_compression_test_points` command line or can be referenced using an *include* file. Directories of Verilog files can also be specified but they cannot be referenced in the include file.
	For example, if the Verilog files required to run ATPG are:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	`write_dft_compression_test_points` can be used in either of the following ways:
	1. If specifying files separately on the command line:
	`write_dft_compression_test_points -library "./padcells.v ./stdcells.v \ ./memories ./ip_blocks" ...`
	1. If using an include file. Create an include file named `include_libraries.v` containing:
	`` `include "./padcells.v" `include "./stdcells.v" ``
	And then specify the following:
	`write_dft_compression_test_points -library "include_libraries.v \ ./memories ./ip_blocks" ...`
	If you specify a relative path, the command interprets the path to be the location from where Modus will be run.
`-min_slack_for_no_tp_file integer`	
	Prevents test point insertion on a pin, if the pin does not have the specified minimum slack.
`-run_from_directory`	Allows to run the `runmodus.compression_testpoints.tcl` script from the Modus working directory.
	Default: Directory where scripts were written from, that is, directory where Genus was run.
`-dont_create_compatibility_matrix`	
	Specifies not to generate the compatibility matrix.
`-verify_test_structures_options "-option1 value -option2 value"`	
	Specifies extra options to apply when performing test structure verification for Modus.

Example

The following command generates the files to run a compression-based testability analysis.

```
write_dft_compression_test_points  -directory comp  -library $sim/mylib.v
```

Related Information

See the following sections in *Genus Design for Test Guide*

- Using Modus to Automatically Select and Insert Test Points

- Exporting the Design for Test Point Selection

Affected by this attribute	dft_generate_atpg_no_testpoint_file
	dft_run_test_point_analysis_for_compression

write_dft_constraints

```
write_dft_constraints
[-strict | -dc | -true_time]
[-version {1.1|1.3|1.4}]
[design] [> file]
```

Description

Generates test design constraints including expanding compressed SDCs into a format usable for Modus `read_sdc` command.

Some constraints (such as set_input_delay, set_output_delay, and so on) are written out in SDC (Synopsys Design Constraints) format while others (such as set_false_path, set_disable_timing) are written out in Modus format.

Options and Arguments

design	Specifies the design for which the constraints must be generated.
file	Specifies the name of the file to which the constraints must be written.
-dc	Writes the constraints that are DC and PT compatible, which means that commands not listed in the SDC specification may be written out.
-strict	Writes only constraints in SDC format.
-true_time	Writes constraints in the True Time language instead of DC or DCTCL.
-version {1.1|1.3|1.4}	
	Specifies the SDC version to use to write SDC constraints.

write_dft_deterministic_test_points

```
write_dft_deterministic_test_points
-library string
[-build_model_options string]
[-build_faultmodel_options string]
[-build_testmode_options string]
[-atpg_options string] [-dfa_options string]
[-include_redundant_faults]
[-verify_test_structures_options string]
[-directory string] [-run_from_directory]
[-min_slack_for_no_tp_file integer]
[-dont_create_compatibility_matrix]
[-continue_with_severe] [-unix_shell]
[design]
```

Description

Writes out the necessary files and scripts to run Deterministic Fault Analysis (DFA) using the Modus software. The template script will only be written if actual scan chains exist in the design. DFA analysis is not supported in `ASSUMED` scan mode.

This command generates the following files:

- *topmodulename*`.FULLSCAN.pinassign`—A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock and scan data IOs) and their test function used to build the testmode when actual scan chains exist in the design.
- `runmodus.dfa.tcl` —A template script file to run the requested deterministic fault analysis.

- *topmodulename*`.test_netlist.v`—A netlist for Modus.
- *topmodulename*`.noTpFile`—A file listing objects that are constrained, thus preventing test point insertion on them.

Some files can be customized according to the setup requirements.

Options and Arguments

`-atpg_options "-option1 value -option2 value"`	
	Specifies extra ATPG analysis options.
`-build_faultmodel_options "-option1 value -option2 value"`	
	Specifies a string containing the extra options to build a fault model.
`-build_model_options "-option1 value -option2 value"`	
	Specifies extra options to apply when building the Modus model.
`-build_testmode_options "-option1 value -option2 value"`	
	Specifies extra options to apply when building the test mode for Modus.
`-dont_create_compatibility_matrix`	
	Specifies not to generate the compatibility matrix.
`-continue_with_severe`	
	Allows the continuation of the test script even when severe warnings occur during execution of the commands in the script.
design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dfa_options "-option1 value -option2 value"`	
	Specifies extra options for deterministic fault analysis.
`-directory` *string*	Specifies the directory to which the output files must be written.
	Default: *current_working_directory*`/test_scripts`
`-include_redundant_faults`	
	Specifies to include the redundant faults during DFA analysis.
`-library` *string*	Specifies the list of Verilog structural library files.
	The Verilog libraries required to run Modus ATPG must be provided using the `-library` option of the `write_dft_dfa` command. These libraries must be in a structural format. The files can be specified separately on the `write_dft_dfa` command line or can be referenced using an *include* file. Directories of Verilog files can also be specified but they cannot be referenced in the include file.
	For example, if the Verilog files required to run ATPG and Xcelium simulation are:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	`write_dft_dfa` can be used in either of the following ways:
	1. If specifying files separately on the command line:
	`write_dft_dfa -library "./padcells.v ./stdcells.v  ./memories ./ip_blocks" ...`

	1. If using an include file. Create an include file named `include_libraries.v` containing:
	```` ```include "./padcells.v"```include "./stdcells.v" ````
	And then specify the following:
	```write_dft_dfa -library "include_libraries.v ./memories ./ip_blocks" ...```
	If you specify a relative path, the command interprets the path to be the location from where Modus will be run.
`-min_slack_for_no_tp_file` *integer*	
	Prevents test point insertion on a pin, if the pin does not have the specified minimum slack.
`-run_from_directory`	
	Allows to run the Deterministic Fault Analysis from the working directory specified with the `-directory` option.
	Default: current working directory of Genus.
`-unix_shell`	Generates the files in shell format for use with Modus.
`-verify_test_structures_options "-option1 value -option2 value"`	
	Specifies extra options to apply when performing test structure verification for Modus.

Example

The following command generates the files to run deterministic fault analysis.

```
write_dft_deterministic_test_points -directory dfa -library $sim/mylib.v
```

Related Information

Related command:	add_analyzed_test_points
Affected by this attribute	dft_generate_atpg_no_testpoint_file

write_dft_jtag_boundary_file

```
write_dft_jtag_boundary_file > iospeclist_file
[-supplemental_file file]
```

Description

Writes out the IOSpecList output file.

The IOSpeclist output file describes the boundary scan architecture of the design. The file contains all ports (functional, test, and TAP) in the design, specifies the type and location of the boundary cells to be inserted on all the functional ports, and lists the instructions (both mandatory and user-defined) to be built in the JTAG Macro.

Options and Arguments

iospeclist_file	Specifies the name of the file to be written.
`-supplemental_file` *file*	
	Specifies the name of the supplemental file to write out. This file and its corresponding IOSpeclist file are used to define the boundary scan objects prior to inserting boundary scan logic. The supplemental file lists the boundary scan segments and the JTAG-instruction definitions for its related objects written to the IOSpeclist file. When using an IOSpeclist file to define the order of the boundary scan register, the supplemental file should be included into the Genus session before reading the IOSpeclist input file. Both the supplemental and the IOSpeclist files need only be written if your intention is to insert boundary-scan logic in a new Genus session.
	The recommended approach to completely restoring the DFT setup (including the boundary scan objects) in a new Genus session is to use the `read_netlist` approach.

Related Information

Affected by these commands:	define_jtag_instruction
	define_jtag_instruction_register
	add_jtag_boundary_scan
	add_ptam
Related command:	read_dft_jtag_boundary_file

write_dft_jtag_boundary_verification

```
write_dft_jtag_boundary_verification -library string
[-bsdl_file file [-bsdl_package_path string]
[-bsdl_package_name files]]
[-build_model_options string]
[-directory string] [-run_from_directory]
[-continue_with_severe]
[-unix_shell] [design]
```

Description

Writes out the necessary files and the template run scripts to run boundary scan verification.

This command generates the following files:

- *topmodulename*`.bsdl`—A BSDL file
- *topmodulename*`.test_netlist.v`—A netlist for Modus
- `runmodus.bsv.tcl`—A Modus run file to run Boundary Scan Verification (BSV).

Some files can be customized according to the setup requirements.

Options and Arguments

`-bsdl_file` *file*	Specifies the name of the BSDL file to be used for the boundary scan verification.
	If you omit this option but you specified the `-bsv` option, this command will automatically run the `write_bsdl` command to generate the BSDL file.
	You must use this option if you inserted custom boundary cells in the design. Additionally, the BSDL file should be written using the `write_bsdl` command with the `-bsdl_package_name` option to list the custom package file to be used during boundary scan verification.
`-bsdl_package_name` *files*	
	Specifies a package file or a comma-separated list of package files that describe the custom boundary cells used in the design.
	This option is only required if you used custom boundary cells in the design. This option cannot be specified without the `-bsdl` option.
`bsdl_package_path` *string*	
	Specifies the UNIX directory or a comma-separated list of directories that indicate(s) where to find the package file(s). You can use dot (.) to refer to the current working directory.
	This option is only required if you used custom boundary cells in the design. This option cannot be specified without the `-bsdl` option.
`-build_model_options {`*-option1 value -option2 value*`}`	
	Specifies extra options to apply when building the Modus model.
`-continue_with_severe`	
	Allows the continuation of the Modus script even when severe warnings occur during execution of the commands in the script.
design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	Specifies the directory to which the output files must be written.
	Default: *current_working_directory*`/test_scripts`
`-library` *string*	Specifies the list of Verilog structural library files.
	The Verilog libraries required to run Modus ATPG and Xcelium simulation of the generated vectors must be provided using the `-library` option of the `write_dft_jtag_boundary_veriification` command. These libraries must be in a structural format. The files can be specified separately on the `write_dft_jtag_boundary_veriification` command line or can be referenced using an *include* file. Directories of Verilog files can also be specified but they cannot be referenced in the include file.
	For example, if the Verilog files required to run ATPG and Xcelium simulation are:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	`write_dft_jtag_boundary_veriification` can be used in either of the following ways:
	1. If specifying files separately on the command line:
	`write_dft_jtag_boundary_veriification \-library "./padcells.v ./stdcells.v \ ./memories ./ip_blocks" ...`
	1. If using an include file. Create an include file named `include_libraries.v` containing:

	`include "./padcells.v"`include "./stdcells.v"
	And then specify the following:
	write_dft_jtag_boundary_veriification \ -library "include_libraries.v ./memories \ ./ip_blocks" ...
	If you specify a relative path, the command interprets the path to be the location from where Modus will be run.
-run_from_directory	
	Allows to run boundary scan verification from the working directory specified with the `-directory` option.
	Default: parent directory of the specified work directory.
-unix_shell	Generates the files for use with Modus.

Example

The following command generates the files to run an ATPG-based testability analysis.

```
write_dft_jtag_boundary_verifcation -directory bsv -library $sim/mylib.v
```

write_dft_lbist_macro

```
write_dft_lbist_macro
-channels integer -max_length_of_channels integer
[-clocks integer]
[-add_runbist_support] [-add_setbist_support]
[-add_direct_access_support]
[-add_programmable_direct_access_support]
[-add_masking]
[-scan_patterns integer] [-scan_pattern_counter_length integer]
[-set_patterns integer] [-set_pattern_counter_length integer]
[-reset_patterns integer] [-reset_pattern_counter_length integer]
[-static_patterns integer] [-static_pattern_counter_length integer]
[-dynamic_patterns integer]  [-dynamic_pattern_counter_length integer]
[-scan_channel_counter_length integer]
[-scan_window integer] [-scan_window_counter_length integer]
[-scan_window_pulse_value integer]
[-scan_enable_delay integer] [-scan_enable_delay_counter_length integer]
[-capture_window integer] [-capture_window_counter_length integer]
[-capture_window_capture_value integer]
[-capture_window_launch_value integer]
[-set_reset_test_window integer] [-set_reset_pulse_width integer]
[-capture_clock_stagger_ratio integer]
[-scan_clock_stagger_ratio integer]
[-parallel_load_control_bits] [-parallel_load_prpg]
[-parallel_load_misr] [-parallel_load_mask]
[-info_file string] [> file]
```

Description

Writes out the structural netlist using generic logic for the LBIST macro. The generated macro has a PRPG, MISR, 1149.1 Interface, and a BIST Controller finite state machine. If you do not insert an LBIST macro with direct access support, the macro can be initialized by the JTAG macro using the `RUNBIST` and `SETBIST` instructions.

To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

For more information about LBIST, refer to chapter Inserting Logic Built-In-Self-Test Logic of the *Genus Design for Test Guide*.

Options and Arguments

`-add_direct_access_support`	
	Inserts the LBIST macro with (non-programmable) direct-access support.
`-add_masking`	Inserts the LBIST macro including the optional channel masking logic (only available with SETBIST support or programmable direct-access support.).
	This option can only be specified with the `-add_setbist_support` or `-add_programmable_direct_access_support` option.
`-add_programmable_direct_access_support`	
	Inserts the LBIST macro with programmable direct-access support.
`-add_runbist_support`	
	Inserts the LBIST macro with JTAG RUNBIST support.
`-add_setbist_support`	
	Inserts the LBIST macro with JTAG SETBIST support.
`-capture_clock_stagger_ratio` *integer*	
	Controls staggering of the capture clocks. The specified value determines the number of bistck pulses after which the capture clock is staggered. Specify 0, 1, 2, 4 or 8. If you specify 0, the capture clocks will not be staggered.
`-capture_window` *integer*	
	Specifies the default value of the capture window.
`-capture_window_capture_value` *integer*	
	Specifies the value of the capture window (down) counter at which the (first) capture clock pulse is issued.
`-capture_window_counter_length` *integer*	
	Specifies the length of the capture window counter that is part of the LBIST macro.
`-capture_window_launch_value` *integer*	
	Specifies the value of the capture window (down) counter at which the launch clock pulse is issued.
`-channels` *integer*	Specifies the number of compressed scan channels. It can include additional channels required to include the boundary scan chain in the macro too.
`-clocks` *integer*	Specifies the number of staggered scan and capture clocks.
`-dynamic_patterns` *integer*	
	Specifies the default number of dynamic test patterns to be executed.
`-dynamic_pattern_counter_length` *integer*	
	Specifies the length of the dynamic pattern counter that is part of the LBIST macro.
`-info_file` *file*	Specifies the file that contains more detailed information about the LBIST macro. It has details of the MISR and PRPG that can be used when writing out the scripts to run Modus.
`-max_length_of_channels` *integer*	

	Specifies the length of the longest compressed scan channel.
`-parallel_load_control_bits`	
	Makes the LBIST control bits programmable at run-time through the programmable direct-access LBIST parallel load interface. For more information on the LBIST control bits refer to the LBIST chapter in *Genus Design for Test Guide*.
`-parallel_load_mask`	
	Makes the LBIST mask programmable at run-time through the programmable direct-access LBIST parallel load interface.
	This option can only be specified with the `-add_masking` option.
`-parallel_load_misr`	
	Makes the LBIST MISR programmable at run-time through the programmable direct-access LBIST parallel load interface.
`-parallel_load_prpg`	
	Makes the LBIST PRPG programmable at run-time through the programmable direct-access LBIST parallel load interface.
`-reset_patterns` *integer*	
	Specifies the default number of reset test patterns to be executed.
`-reset_pattern_counter_length` *integer*	
	Specifies the length of the reset pattern counter that is part of the LBIST macro.
`-scan_channel_counter_length` *integer*	
	Specifies the length of the scan channel counter that is part of the LBIST macro.
`-scan_clock_stagger_ratio` *integer*	
	Controls staggering of the scan clocks. The specified value determines the number of bistck pulses after which the scan clock is staggered. Specify 0, 1, 2, 4 or 8. If you specify 0, the scan clocks will not be staggered.
`-scan_enable_delay` *integer*	
	Specifies the default delay after the scan enable signal toggles.
`-scan_enable_delay_counter_length` *integer*	
	Specifies the length of the scan enable delay counter that is part of the LBIST macro.
`-scan_patterns` *integer*	
	Specifies the default number of scan test patterns to be executed.
`-scan_pattern_counter_length` *integer*	
	Specifies the length of the scan pattern counter that is part of the LBIST macro.
`-scan_window` *integer*	
	Specifies the default value of the scan window.
`-scan_window_counter_length` *integer*	
	Specifies the length of the scan window counter that is part of the LBIST macro.
	Default: `8`
`-scan_window_pulse_value` *integer*	
	Specifies the value of the scan window (down) counter at which the (first) scan clock pulse is issued.
`-set_patterns` *integer*	
	Specifies the default number of set test patterns to be executed.

`-set_pattern_counter_length` *integer*	
	Specifies the length of the set pattern counter that is part of the LBIST macro.
`-set_reset_pulse_width` *integer*	
	Specifies the width of the asynchronous set/reset pulse for the set/reset tests.
`-set_reset_test_window` *integer*	
	Specifies the value of the test window for the asynchronous set/reset tests.
`-static_patterns` *integer*	
	Specifies the default number of static test patterns to be executed.
`-static_pattern_counter_length` *integer*	
	Specifies the length of the static pattern counter that is part of the LBIST macro.

Related Information

Related command:	add_lbist

write_dft_lbist_test_points

```
write_dft_lbist_test_points
[-lbist_testpoints_options string] [-lbist_testpoints_options_file string] [-build_faultmodel_options string]
[-build_model_options string]
[-build_testmode_options string]
[-verify_test_structures_options string]
[-library string]
[-directory string][-run_from_directory]
[-min_slack_for_no_tp_file integer]
[-dont_create_compatibility_matrix]
[-force][-continue_with_severe]
[-unix_shell][-intest] [design]
```

Description

Writes out the necessary files and the template run scripts to run a Random Resistant Fault Analysis (RRFA) based testability analysis using the Modus software.

This command generates the following files:

- `test.exclude`—A file listing objects to be excluded from the ATPG analysis in `ASSUMED` scan mode.
- `test.modedef`—A mode definition file, a text file that describes the test mode in `ASSUMED` scan mode.
- *topmodulename*`.ASSUMED.pinassign`—A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock) and their test function used to build the testmode before actual scan chains exist in the design.
- *topmodulename*`.FULLSCAN.pinassign`—A pin-assignment file that captures the top-level scan-related signals (shift-enable, test-mode, test-clock and scan data IOs) and their test function used to build the testmode when actual scan chains exist in the design
- *topmodulename*`.noTpFile`—A no test point listing objects that are constrained, thus preventing test point identification on them.
- *topmodulename*`.test_netlist.v`—A Verilog netlist for use with Modus.
- `runmodus.rrfa.tcl`—A script to run random resistant fault analysis in Modus.
- `runet.rrfa`—A script to run random resistant fault analysis in Modus.

Some files can be customized according to the setup requirements.

Options and Arguments

`-build_faultmodel_options "-option1 value -option2 value"`	
	Specifies a string containing the extra options to build a fault model.
`-build_model_options "-option1 value -option2 value"`	
	Specifies extra options to apply when building the Modus model.
`-build_testmode_options {-option1 value -option2 value}`	
	Specifies extra options to apply when building the test mode for Modus.
`-continue_with_severe`	
	Allows the continuation of the test script even when severe warnings occur during execution of the commands in the script.
design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	Specifies the directory to which the output files must be written.
	Default: *current_working_directory*`/test_scripts`
`-force`	Specifies to continue even when the DFT logic appears to be incomplete.
`-intest`	Specifies test point analysis in fullscan intest testmode.
`-lbist_testpoints_options "-option1 value -option2 value"`	
	Specifies the extra options to run LBIST testpoint analysis.
`-lbist_testpoints_options_file` *string*	
	Specifies the file where each line contains extra options to run LBIST Testpoints Analysis. Multiple testpoint analysis files will be generated (one for each line). Requires MOD400 license.
`-library` *string*	Specifies the list of Verilog structural library files. This option is required for mapped designs.
	The Verilog libraries required to run Modus ATPG must be provided using the `-library` option of the `write_dft_lbist_test_points` command. These libraries must be in a structural format. The files can be specified separately on the `write_dft_lbist_test_points` command line or can be referenced using an *include* file. Directories of Verilog files can also be specified but they cannot be referenced in the include file.
	For example, if the Verilog files required to run ATPG are:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	`write_dft_lbist_test_points` can be used in either of the following ways:
	1. If specifying files separately on the command line:
	`write_dft_lbist_test_points -library "./padcells.v ./stdcells.v ./memories ./ip_blocks" ...`
	1. If using an include file. Create an include file named `include_libraries.v` containing:
	`` `include "./padcells.v"`include "./stdcells.v" ``
	And then specify the following:
	`write_dft_lbist_test_points -library "include_libraries.v ./memories  ./ip_blocks" ...`

	If you specify a relative path, the command interprets the path to be the location from where Modus will be run.
`-min_slack_for_no_tp_file` *integer*	
	Prevents test point insertion on a pin, if the pin does not have the specified minimum slack.
`-dont_create_compatibility_matrix`	
	Specifies not to generate the compatibility matrix.
`-run_from_directory`	
	Allows to run either Automatic Test Pattern Generator (ATPG) or Random Resistance Fault Analysis (RRFA) from the working directory specified with the `-directory` option.
	Default: parent directory of the specified work directory.
`-unix_shell`	Generates the files for use with Modus.
`-verify_test_structures_options "-option1 value -option2 value"`	
	Specifies extra options to apply when performing test structure verification for Modus.

Example

The following command generates the files to run an LBIST-based testability analysis.

```
write_dft_lbist_test_points -directory lbist -library $sim/mylib.v
```

Related Information

Affected by this attribute	dft_generate_atpg_no_testpoint_file
	dft_run_test_point_analysis_for_lbist

write_dft_lbist_testbench

```
write_dft_lbist_tesbench
[-build_model_options string]
[-build_testmode_options string]
[-build_faultmodel_options string]
[-verify_test_structures_options string]
[-library string] [-ncsim_library string]
[-directory string] [-run_from_directory]
[-unix_shell] [design]
```

Description

Writes out the necessary data and script files for LBIST signature and pattern generation.

Options and Arguments

`-build_faultmodel_options {-option1 value -option2 value}`	
	Specifies extra options to apply when building the fault model.
`-build_model_options {-option1 value -option2 value}`	
	Specifies extra options to apply when building the Modus model.
`-build_testmode_options {-option1 value -option2 value}`	
	Specifies extra options to apply when building the test mode for Modus.
design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	Specifies the directory to which the output files must be written.
	Default: *current_working_directory*`/lbist_testbench_scripts`
`-library` *string*	Specifies the list of Verilog structural library files.
`-ncsim_library` *string*	
	Specifies the list of library files required for the Xcelium simulator to simulate the test patterns created by Modus for LBIST.
`-rrfa`	Generates the Modus files for use with rrfa run.
`-run_from_directory`	
	Allows to run LBIST from the working directory specified with the `-directory` option.
	Default: parent directory of the specified work directory.
`-unix_shell`	Generates the files for use with Modus.
`-verify_test_structures_options {-option1 value -option2 value}`	
	Specifies extra options to apply when performing test structure verification for Modus.

write_dft_pmbist_interface_files

```
write_dft_pmbist_interface_files
-directory string [-rtl] [-overwrite] [design]
```

Description

Writes out the necessary interface files for programmable Memory Built-In-Self-Test (PMBIST) related to this design.

Genus optimizations and changes to the design which affect the data within the internal representation of the interface files update these files until written. Therefore, you should execute the command just prior to writing out a design and starting PMBIST design verification.

These interface files represent an abstract model of the current PMBIST insertion process, supporting not only a bottom-up flow for incremental PMBIST insertion but also the generation of patterns to exercise the PMBIST logic from Modus `create_embedded_test` command.

This command generates the following files:

- *design*`_pattern_control.txt`—A file containing the PMBIST external interface and pattern generation controls derived from the configuration file.
- *design*`_test_def.txt`—A file containing information on testplans, algorithm constraints, and user-defined algorithms for this design.
- *design*`_mbistsch_tdr_map.txt`—A file identifying the `MBISTSCH TDR` test data register contents.
- *design*`_mbistchk_tdr_map.txt`—A file identifying the `MBISTCHK TDR` test data register contents.
- *design*`_mbisttpn_tdr_map.txt`—A file identifying the `MBISTTPN TDR` test data register contents.

- *design*_mbistamr_tdr_map.txt—A file identifying the MBISTAMR TDR test data register contents when programmable testplans ate required.
- *design*_mbistrar_tdr_map.txt—A file identifying the MBISTRAR TDR test data register contents. This is generated when redundancy analysis or fault tolerance is requested. This is used to determine the failure details for redundancy analysis, fault tolerance, and repair.
- *design*_mbistdiag_tdr_map.txt—A file identifying the MBISTDIAG TDR test data register contents. This is generated when diagnostics is requested for memories.
- *design*_mbistrom_tdr_map.txt—A file identifying the MBISTROM TDR test data register contents when ROMs are present in the design and programmable testplans target them.
- *design*_mbistromdiag_tdr_map.txt—A file identifying the MBISTROMDIAG TDR test data register contents when ROM diagnostics is required.

If hard repair is also performed then this command generates the following additional files:

- *design*_fcu_pattern_control.txt—A file containing the PMBIST hard repair specific external interface and pattern generation controls derived from the configuration file.
- *design*_fcu_test_def.txt—A file containing information on testplans, algorithm constraints, and user-defined algorithms for this design for hard repair.
- *design*_fcusch_tdr_map.txt—A file identifying the FCUSCH TDR test data register contents. This is used to schedule the fuse macro for hard repair.
- *design*_fcuchk_tdr_map.txt—A file identifying the FCUCHK TDR test data register contents. This is used to check the results of the hard repair.
- *design*_fcutpn_tdr_map.txt—A file identifying the FCUTPN TDR test data register contents. This contains information on testplans used for hard repair.
- *design*_fcuamr_tdr_map.txt—A file identifying the FCUAMR TDR test data register contents when programmable testplans are required for hard repair.
- *design*_fcurr_tdr_map.txt—A file identifying the FCURR TDR test data register contents. This contains information on channel binding for hard repair.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

design	Specifies the design for which to write out the PMBIST interface files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
-directory *string*	Specifies the directory to which the PMBIST interface files must be written.
-overwrite	Overwrites existing interface files for the current design.
	Absence of this flag leads to an error if interface files for the current design already exist in the provided directory.
-rtl	Writes out the PMBIST interface files for the RTL flow. The interface files for RTL flow may be different than the gate level flow due to addition of labels for for and if.

Related Information

Related command:	read_dft_pmbist_interface_files

write_dft_pmbist_testbench

```
write_dft_pmbist_testbench
[-create_embedded_test_options string]
[-create_fcu_patterns][-add_asu_testbench]
[-irun_options string] [-sim_options string] [-ncsim_library file_list]
[-testbench_directory string] [-directory string]
[-create_deposit_script][-no_cibforce_script] [-deposit_all_flops]
[-amu_instance hinst] [-fcu_instance hinst]
[-sim_with_deposit_script] [-sim_without_cibforce_script]  [-sim_without_msie] [-aux_source string] [-aux_top string]
[-no_build_model] [-script_only] [-no_globals_in_sim_script] [-rtl] [-sim_without_assertions] [-runmask integer][-create_fcu_patterns] [-test_pattern_directory string]
[-build_model_netlist string] [-downgrade_assertion_severity][design]
```

Description

Writes out the necessary files and the template run scripts to create Verilog test benches to validate PMBIST, and executes the scripts using the Modus software.

If your design contains ROMs, the `rompath` and `romcontentsfile` keywords in the `create_embedded_test_options` string are set by default.

Typically a single memory BIST instruction set is implemented in the design. In this case, the `interfacefilelist` keyword to `create_embedded_test` is set by default. If multiple memory BIST instruction sets are implemented in the design, include the `interfacefilelist` keyword indicating the interface files for a single instruction set in the `create_embedded_test_options` string.

After inserting PMBIST into the design and optionally a JTAG macro, write out the modified design to file. Then, use this command to generate a PMBIST Verilog test bench to validate the PMBIST functionality through simulation.

This command is designed to generate the following patterns by default:

- `production` patterns— if JTAG control has been implemented for memory BIST in the design
- `directaccess` and `burnin` patterns—if direct access has been implemented for PMBIST in the design

These default generated test benches are created with a schedule intended to run all devices in parallel.

This command generates the following files prior to executing the run scripts in Modus:

- *design*`_abstract.v`—A Verilog generic logic gates description of the netlist for Modus
- `runmodus.write_mbist_testbench.tcl`—A template script file to run `create_embedded_test` in Modus, generating patterns for PMBIST. This is automatically executed by `write_dft_pmbist_testbench`, unless the -`script_only` option has been specified.
- `runmodus.write_vectors.tcl`—A template script file to run `wr`ite_vectors in Modus to write the PMBIST patterns as Verilog test benches. This is automatically executed by `write_dft_pmbist_testbench`, unless the -`script_only` option has been specified. If the customexperiment option is specified in the -`create_embedded_test_options` keyword, then this script will be suffixed with the experiment name. Example, if -`create_embedded_test_options "customexperiment=myexp"` is specified, then the script name will be `runmodus.write_vectors.tcl.myexp`.
- `irun.simscript.`*`testbench_pattern_name`*—One or more template scripts to compile and simulate the PMBIST test bench pattern within the Xcelium simulator. By default, these scripts will be for production patterns when JTAG is used, and for `directaccess` and `burnin` patterns when direct access is used. These scripts are created in the directory specified by the -`testbench_directory` keyword. If a directory is not specified, it defaults to an `mbist_testbench` directory created in the directory specified by the -`directory` option.
- `irun.depositscript.`*`testbench_pattern_name`*—One or more Xcelium simulator scripts to initialize all PMBIST and JTAG logic to known states prior to executing the Verilog test benches. Genus logic optimizations may permit "X" state propagation. These scripts can be used to overcome this situation when desired by including `-input` keyword selecting this file in the appropriate irun script. Note these scripts include simulator run and exit commands within them. This file is generated by default, use the -`no_deposit_script` option to not generate this file. By default the `irun.simscript.`*`testbench_pattern_name`* scripts will not simulate using the `irun.depositscript.`*`testbench_pattern_name`* file. Specify the -`sim_with_deposit_script` to simulate with the deposit script.

You can customize some files according to the setup requirements.

> To use this command you need an additional license. For more information on the exact product requirements, refer to Modus Product Requirements for Advanced Features in *Genus Design for Test Guide*.

Options and Arguments

`-add_asu_testbench`	Specifies to enable generation of the ASU Verilog driver for testing on demand functions.
`-amu_instance` *`hinst`*	
	Specifies the name of the associated algorithm memory unit instance.
`-aux_source` *`string`*	Specifies a list of auxiliary design files for simulation. For example, `"mod1.v mod2.v"`
`-aux_top` *`string`*	Specifies a list of auxiliary top level design modules for simulation. For example, `"mod1 mod2"`
`-build_model_netlist` *`string`*	
	Specifies the netlist to use for build_model.
`-create_embedded_test_options {`*`-option1 value -option2 value`*`}`	
	Specifies extra options to apply when running the `create_embedded_test` command in Modus.
	The default options are to generate bypass and production patterns (-createpatterns bypass,production) if MBIST is controlled by JTAG and directaccess/burnin patterns (-createpatterns directaccess, burnin) if MBIST is controlled by direct access pins. By default all devices on all PMBIST engines will be run in parallel, where possible.
`-create_fcu_patterns`	
	Creates patterns for hard repair. By default, `write_dft_pmbist_testbench` creates MBIST patterns for logic inserted by `add_pmbist`. If hard repair support has been added by running `add_hard_repair`, then use this option to generate hard repair patterns. Hard repair patterns and non hard repair patterns can't be generated at the same time.
	When -`create_fcu_patterns` is used, the `customexperiment` and `customtestplans` options must be specified in the -`create_embedded_test_options` keyword. Also, only one hard repair pattern type can be created at a time.The `createpatterns` keyword must also be specified, and set to either `jtag_fcu_repair` (for JTAG) or `pmda_fcu_repair` (for direct access).
	Example: `write_dft_pmbist_testbench -create_fcu_patterns -create_embedded_test_options "customexperiment=myexp customtestplans= tp1,tp2,tp3 createpatterns=jtag_fcu_repair"`
`-deposit_all_flops`	Deposits a value for all flops in the deposit script. This can be useful when it's necessary to ensure a known value (non X value) is present in all sequential elements prior to starting simulation.
`design`	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *`string`*	Specifies the directory to which the output files (scripts, test data and abstract model) must be written.
	Default: *`current_working_directory/`*wmt
`-downgrade_assertion_severity`	Downgrades assertion severity so that assertion failure does not stop the batch mode.
`-fcu_instance` *`hinst`*	
	Specifies the name of the associated fuse control unit instance.
`-irun_options {`*`-option1 value -option2 value ...`*`}`	
	Specifies the options to be passed to the `irun` command for simulation purposes. These options will be placed at the beginning of the irun script, and indicated by the comments "## User specified options: ".
`-ncsim_library` *`file_list`*	
	Specifies the list of library files required for the Xcelium simulation.
	You can specify the files explicitly or you can specify an include file that lists the files. You can also specify directories of Verilog files but you cannot reference directories in an include file.
	For example, if the Verilog files required to run simulation are:
	`./padcells.v./stdcells.v./memories/*.v./ip_blocks/*.v`
	`write_dft_pmbist_testbench` can be used in either of the following ways:

	1. If specifying files separately on the command line:
	`write_dft_pmbist_testbench -ncsim_library "./padcells.v ./stdcells.v ./memories ./ip_blocks" ...`
	1. If using an include file. Create an include file named `include_libraries.v` containing:
	`` `include "./padcells.v" `include "./stdcells.v" ``
	And then specify the following:
	`write_dft_pmbist_testbench -ncsim_library \"include_libraries.v ./memories  ./ip_blocks" ...`
	If you specify a relative path, the command interprets the path to be the location from where the Xcelium simulator will be run.
`-no_build_model`	Prevents to execute the `build_model` command.
	This can be useful if the Modus model has already been built by a prior `write_dft_pmbist_testbench` invocation and does not need to be built again, especially if it's a large design and takes a significant amount of time to build.
`-create_deposit_script`	
	Prevents that the deposit script is generated and used during simulation.
`-no_cibforce_script`	
	Prevents generation of cib force deposit script that is used during simulation.
`no_globals_in_sim_script`	
	Specifies not to use global variables in the sim script.
`-rtl`	Targets the necessary RTL files for simulation in the irun script for the RTL flow.
`-runmask` *integer*	Specifies the runmask bit.
`-script_only`	Creates the `runmodus.write_mbist_testbench.tcl` script without executing it.
	By default, the tool creates and executes the `runet.write_mbist_testbench` script, and then creates and executes the `runet.write_vectors` script.
`-sim_options {`*`-option1 value1 -option2 value2 ...`*`}`	
	Specifies the command line options to be added to the simulation command in the generated simulation scripts (example "-option1 <string> -option2
`-sim_with_deposit_script`	
	Specifies to run simulation with a deposit script. Genus logic optimizations may permit "X" state propagation. Simulating with a deposit script can be used to overcome this situation when desired.
`-sim_without_cibforce_script`	
	Runs simulation without using a cib force script.
`-sim_without_msie`	Runs simulation without using msie.
`-sim_without_assertions`	
	Specifies to run simulation without using assertions.
`-testbench_directory` *directory*	
	Specifies the directory to which the generated Verilog test benches must be written.
	Default: *current_working_directory*`/mbist_testbench`

`-test_pattern_directory` *string*	
	Specifies the output directory for TBDpatt files, assign files and mode initialization files.

Related Information

Affected by these commands:	add_jtag_boundary_scan
	add_jtag_macro
	add_pmbist
Related commands:	read_hdl
	write_dft_pmbist_interface_files
Related attribute:	pmbist_instruction_set

write_dft_rtl_model

```
write_dft_rtl_model
-directory string [design] [-overwrite]
```

Description

Writes an RTL model of the design in Verilog if the DFT RTL insertion flow has been enabled and DFT insertion commands supporting the RTL update flow have been run. DFT RTL insertion flow is enabled using attribute `dft_rtl_insertion`. The commands that support RTL insertion of DFT logic are `add_jtag_macro`, `add_pmbist` and `add_hard_repair`.

This command analyzes and writes all the design files. The written design files contain three kinds of files.

1. *design_name*_eq.v file - This is a new file. This file contains the definition of DFT modules which were added to the user design.
2. Modified RTL source files - These are the files which are modified by this command. The modifications include the instantiation of DFT logic and their connections to existing design modules. The command prints the list of modified files on the standard output.
3. Original design files - These are the files which are not modified by this command.

The directory structure of the original design source is kept intact while writing the design files under user supplied location. Pure Verilog designs and mixed designs are supported with this command as long as the modifications are required for Verilog modules only.

> The attribute `dft_rtl_insertion` must be set prior to running `elaborate` command on the design so that all the design changes can be tracked and applied properly while writing the DFT inserted RTL design.

The DFT inserted RTL design now becomes the new golden design which can be used for synthesis. While treating DFT inserted RTL as new golden design, the attribute `dft_rtl_insertion` must be disabled so that synthesis can be performed.

Options and Arguments

design	Specifies the design for which to write out the Modus input files.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-directory` *string*	Specifies the directory to which the updated RTL files must be written.
`-overwrite`	Allows to overwrite any existing directory.

Related Information

Affected by this attribute	dft_rtl_insertion

write_dft_rtl_opcg_domain_macro

```
write_dft_rtl_opcg_domain_macro
 [-name string]
 [-domain_name string]
 [-max_num_pulses integer]
 [-counter_length integer]
 [-domain_blocking] [-los]
 [-file string]
```

Description

Writes out the RTL of an OPCG domain.

By default, Genus creates a gate level version of the OPCG macro and instantiates it in the design. This is optimized during subsequent synthesis optimizations. Some users may want the RTL version of this macro to use in RTL level simulations. This would typically be done for performance reasons as RTL simulation is much faster than gate level. The simulation of a gate level OPCG macro with the rest of the RTL can significantly degrade the simulation performance.

Options and Arguments

`-name string`	Specifies the name of OPCG domain module.
`-domain_name string`	
	Specifies the name of OPCG domain.
`-max_num_pulses integer`	
	Specifies the number of high speed pulses to be generated by the domain macro. The default is 2 and the maximum is 8.
`-counter_length integer`	
	Specifies the number of bits in the down counter register. The default is 0 (no counter to be inserted) and the possible values can range from 4 to 7.
`-domain_blocking`	Implements OPCG domain blocking.
`-los`	Implements Launch-Off-Shift (LOS) OPCG.
`-file string`	Specifies the file to write out OPCG domain macro. The default is 'stdout'.

Related Information

Related References:	RTL OPCG section in Genus Design for Test Guide.

write_dft_rtl_opcg_trigger_macro

```
write_dft_rtl_opcg_trigger_macro
[-name string]
[-trigger_name string]
[-delay_cycles integer]
[-file string]
```

Description

Writes out the RTL of an OPCG trigger macro.

Options and Arguments

`-name string`	Specifies the name of OPCG domain module.
`-trigger_name string`	
	Specifies the name of OPCG trigger.
`-delay_cycles integer`	
	Specifies the number of extra cycles to be used to delay the trigger signal inside the trigger macro. Default is 'stdout'.
`-file string`	Specifies the file to which the OPCG trigger macro is to be written. The default value is 0.

Related Information

Related References:	RTL OPCG section in Genus Design for Test Guide.

write_dft_rtl_unified_lbist_macro

```
write_dft_rtl_unified_lbist_macro

-channels integer
-max_length_of_channels integer
[-directory string]
```

Description

Writes out the behavioral RTL for the Unified Logic BIST macro.

Options and Arguments

`-channels integer`	Specifies the number of channels.
`-max_length_of_channels integer`	Specifies the maximum length of the channels.
`-directory path`	Specifies the directory to write unified lbist macro RTL files.

Related Information

See, Inserting LBIST Logic in Genus Design for Test Guide.

Related commands:	write_dft_lbist_macro

write_memory_physical_map

```
write_memory_physical_map
-memory string [-directory string] [design]
```

Description

Writes the physical memory layout of the memory cell provided with the `-memory` option to a file named *memory*`.mca` in a directory provided by the `-directory` option.

The physical memory layout can be generated only after reading memory information via read_pmbist_memory_view command.

This *ex_memory*`.mca` file represents the physical layout of the memory, that is, how the memory is physically structured into banks, data-bits, columns and rows. It also depicts the order of banks, order of rows and columns in a bank etc. The format of this layout structure matches the format given in the documentation under address_partition section in Genus PMBIST Guide.

Options and Arguments

design	Specifies the design on which to perform the memory layout.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-memory` *string*	Specifies the name of the memory module in the specified design.
`-directory` *string*	Specifies the directory path where to store the memory file created.
	Default: `./pmbist_`*designname*

Related Information

Affected by this command:	read_pmbist_memory_view

write_modus_ctl_tp_file

```
write_modus_ctl_tp_file
[-directory string]
[design]
```

Description

Generates a *design*.ctlFile file which contains a list of instances that will be considered as fully controllable or observable during test point analysis.

This file is passed to Modus via the ctlblockfile keyword of the `analyze_lbist_testpoints` and `analyze_compression_testpoints` commands. With this information, Modus will consider instances as fully controllable/observable.

Instances included in this file should fulfil any of the following conditions:

- The instance has the `dft_tpi_full_control_observe` attribute set to true.
- The instance has been scan abstracted (using either native or CTL model)

Options and Arguments

`-directory` *string*	Specifies the directory where CTL block file gets created.
design	Specifies the design on which to perform the test point selection. If you omit the design name, the tool selects the top-level design from the current directory in the design hierarchy.

write_modus_no_tp_file

```
write_modus_no_tp_file
[-min_slack integer] [-directory string]
[-auto_plus_user string] [design]
```

Description

Generates a *de*sign.noTpFile file which contains a list of subdesigns, instances, nets or pins that have been constrained. As a result, no test points can be inserted on these objects.

This file is passed to Modus via the notpfile keyword of the analyze_random_resistance and analyze_deterministic_faults commands. With this information, Modus does not generate test points for these subdesigns, instances, pins or nets, thereby providing a better set of test points.

Subdesigns and instances included in this file satisfy any of the following conditions:

- The subdesign or instance has the dft_dont_scan attribute set to true
- The subdesign or instance has the preserve attribute set to true, 1, or size_ok
- The instance has the hard_region att'ribute set to true
- The subdesign or instance has been scan abstracted (using either native or CTL model)

The pins that are included in the file satisfy any of the following conditions:

- The pin has a slack less than the min_slack
- The pin has the preserve attribute set to true, 1, or size_ok

In addition, any net that does not have its preserve attribute set to false, is written to the file.

If a subdesign is already written into the file, then the instances, pins or nets in the subdesign are not written out again. Similarly, if the instance is included in the file, then its pins and nets are not written out to the file.

Options and Arguments

-auto_plus_user *strings*	
	Specifies the user defined no testpoint file.
design	Specifies the design on which to perform test point selection.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
-directory *string*	Specifies the working directory.
-min_slack *integer*	
	Prevents insertion of a test point if the slack is smaller than the specified number (in ps).

write_scandef

```
write_scandef
[-partition partition -chains chain [chain]...]...
[-version {5.5|5.4}]
[-end_chains_before_lockups] [-dont_split_by_library_domains] [-dont_split_by_power_domains] [-dont_use_timing_model_pins] [-
end_chains_before_mm_fanout_flops] [-split_by_combo_segments] [-start_stop_at_anchor_buffers] [design] [ > file]
```

Description

Writes the scanDEF description of the top-level scan chains configured in the design for reordering using a physical design tool.

Options and Arguments

`-chains` *chain*	Specifies the scan chains that must be grouped in the same partition by the physical design tool. Use the `report_scan_chains` command to obtain a list of valid chain names.
	The tool ensures that chains or chain segments that are not compatible are not added to the same partition, but are further partitioned by adding the test clock name and test clock edge to the final partition name.
	This option can be used only with the `-partition` option
	Requires version 5.5.
design	Specifies the design for which to write out the scanDEF description.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-dont_split_by_library_domains`	
	Indicates not to split the chains at the scan data input pin of the last flop in the originating (or from) library domain and at the scan data output pin of the first flop in the destination (or to) library domain.
	By default, the chains will be split based on the library domains. If the design has no library domains, the chains will not be split.
`-dont_split_by_power_domains`	
	Indicates not to split the chains at the scan data input pin of the last flop in the originating (or from) power domain and at the scan data output pin of the first flop in the destination (or to) power domain.
	By default, the chains will be split based on the power domains. If the design has no power domains, the chains will not be split.
`-dont_use_timing_model_pins`	
	Prevents using the user-designated lib_cell timing model pins as the scanDEF chain `START` and `STOP` points. Instead an outward trace is performed to identify and use the first flip-flop scan data output pin and last flip-flop scan data input pin and use these pins as `START` and `STOP` pins in the scanDEF chains.
`-end_chains_before_lockups`	
	Terminates the scan segment at the scan data input pin of the scan flop which precedes the lockup element in the scan DEF chain.
`-end_chains_before_mm_fanout_flops`	
	Specifies to use the SI pin of the FF that fans out to multiple chains as the termination point for chain.
file	Specifies the file to which the output must be written. To write out the scanDEF file in compressed format, specify a file name with the `.gz` extension.
	Default: output is written to the screen
`-partition` *partition*	
	Specifies the name of a user-defined partition.
	Requires version 5.5.
`-split_by_combo_segments`	

	Specifies to split chain at SI pin of flop preceding segment with starting combinational logic and at Q pin of flop following segment with ending combinational logic.
`-start_stop_at_anchor_buffers`	
	Specifies to start and stop chain at anchor buffers.
`-version {5.5|5.4}`	Specifies which DEF version to write out. Version 5.5 writes out the `MAXBITS` and `PARTITION` keywords as regular statements (that is, uncommented).
	Default: `5.5`

Related Information

Affected by this command:	connect_scan_chains
Affected by these attributes:	actual_scan_chain Attributes

8

Design Manipulation

- add_tieoffs
- bitblast_all_ports
- bitblast_ports
- change_link
- connect
- create_design
- create_hinst
- create_hnet
- create_hport
- create_hport_bus
- create_inst
- create_module
- create_port
- create_port_bus
- create_primitive
- dedicate_module
- delete_obj
- delete_unloaded_undriven
- disconnect
- flatten_complex_ports
- group
- remove_cdn_loop_breaker
- rename_obj
- reset_design
- ungroup
- uniquify
- update_names

add_tieoffs

```
add_tieoffs
[-high_low string| -high string| -low string]
[-always_on_high_low string | -always_on_high string | -always_on_low string]
[-allow_inversion][-max_fanout integer] [-all][-skip_unused_hier_pins][-place_cells]
[-verbose][module | design]
```

Description

Ties the constants `1'b0` and `1'b1` in the netlist to tie high and tie low cells, respectively. In multiple supply voltage (MSV) designs, this command inserts cells by domain. It skips scan pins, preserved pins, preserved nets, and modules by default. Scan pins can be connected by using the `-all` option.

Use the `ui_respects_preserve` root attribute to override preserve settings.

> Ensure that the specified tie high_low lib_cell, or tie high and tie low lib_cells are usable. Make sure that both the `preserve` and `avoid` libcell attributes are set to `false` on the specified cells.

This command should only be run on uniquified designs.

Options and Arguments

`-all`	Inserts tie high/low cells without skipping scan pins.
`-allow_inversion`	Allows to use a tie cell with inverter if either the tie high or tie low cell cannot be found in the library.
`-always_on_high_low` *string* | `-always_on_high` *string* | `-always_on_low` *string*	
	Specifies the always-on lib_cell(s) to replace constant 1s and constant 0s. Either specify one lib_cell to be used for both constant 1s and constant 0s, or specify a specific lib_cell to be used for the constant 1s and one to be used for the constant 0s.
	Default: If you specified neither the `-always_on_high_low` option, nor the `-always_on_high` and `-always_on_low` options, the tool will use the first appropriate always-on cell from the library.
`-high_low` *string* | `-high` *string* | `-low` *string*	
	Specifies the lib_cell(s) to replace constant 1s and constant 0s. Either specify one lib_cell to be used for both constant 1s and constant 0s, or specify a specific lib_cell to be used for the constant 1s and one to be used for the constant 0s
	Default: If you specified neither the `-high_low` option, nor the `-high` and `-low` options, the tool will use the first appropriate cell from the library.
`-max_fanout` *integer*	Specify the maximum fanout allowed per tie cell.
	If this option is not specified, there is no constraint on the fanout.
`-place_cells`	Places the inserted tie cells.

`-skip_unused_hier_pins`	
	Skips hierarchical constant connected pins which are not used inside the module.
`{module | design}`	
	Specifies the design or module in which to insert constants.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used.
`-verbose`	Provides detailed information of the preserved and scan pins that were skipped in the tie high/low cell insertion process.

Related Information

Related attributes:

ui_respects_preserve
use_tiehilo_for_const

bitblast_all_ports

```
bitblast_all_ports {design|module}...
```

Description

Bitblasts all ports of the specified design or module. This command is available after elaboration. The name of the bitblasted ports will follow the nomenclature specified by the `bit_blasted_port_style` attribute. The default bit blasted port style is:

```
%s_%d
```

Options and Arguments

Example

In the following example, the Verilog design `top` has a four-bit input port named `AI`:

```
AI[0:3]
```

The `bitblast_all_ports` command is issued on the design `top`, bitblasting the `AI` port:

```
...

genus@root:> elaborate

...

genus@root:> bitblast_all_ports top

genus@root:> vls design:top.ports

AI_0 AI_1 AI_2 AI_3
```

Related Information

Affected by this attribute:

bit_blasted_port_style

Related Attributes:

write_vlog_bit_blast_bus_connections
write_vlog_bit_blast_mapped_ports

bitblast_ports

```
bitblast_ports
{port_bus | hport_bus}
{design | hinst}
```

Description

Bitblasts the specified port_bus (or hport_bus) of the specified design (or hierarchical instance). This command is available after elaboration. The name of the bitblasted ports will follow the nomenclature specified by the `bit_blasted_port_style` attribute. The default style is:

```
%s_%d
```

Options and Arguments

design	Specifies the design to which the port_bus belongs.
{port_bus|hport_bus}	Specifies the name of the port_bus or hport_bus to be bitblasted.

Related Information

Affected by this attribute:

bit_blasted_port_style

Related Attributes:

write_vlog_bit_blast_bus_connections

write_vlog_bit_blast_mapped_ports

change_link

```
change_link -instances {inst|hinst|pcell}...
{ -design_name {inst | hinst | module | design}
| -lib_cell lib_cell}
[-pin_map string] [-lenient]
[-no_name_change] [-base_cell base_cell]
[-change_in_non_uniq_subdesign]
[-copy_attributes] [-retain_exceptions] [-keep_module]
```

Description

Changes the reference of a hierarchical instance to the specified module or design. The command also supports libcell to libcell reference changes as well as a hierarchical instance to libcell changes.

Options and Arguments

`-base_cell` *base_cell*	
	Specifies a *base_cell* which will be used to link the instance.
`-change_in_non_uniq_subdesign`	
	Changes the link of the instance(s) in all instantiations of a non-uniquified subdesign.
`-copy_attributes`	Copies the attributes of the original leaf instance(s) to the new leaf instances.
`-design_name` {*inst*|*hinst* | *module* | *design*}	
	Specifies the design, module, or hierarchical instance to which the link has to be changed.
`-instances` {*inst*|*hinst*|*pcell*}	
	Specifies the instance(s) whose reference has to be changed.
`-keep_module`	Specifies to keep the module if there are no references.
`-lenient`	Leaves the pins floating if a pin map is not found.
`-lib_cell` *lib_cell*	Specifies the library cell to which the instance link has to be changed.
`-no_name_change`	Prevents renaming of the subdesign name.
`-pin_map` *string*	Specifies, as a Tcl list of lists, the required pin mapping for swapping.
`-retain_exceptions`	Keeps the original exceptions of the instance after replacing it with another link.

Examples

- The following command changes the reference of sub-block instance to the specified design. This can be useful in a hierarchical flow if the sub-block is synthesized separately and later integrated in the top-level design.

  ```
  change_link -instances [get_db hinsts  hinst:top/sub_block/i1] \
  -design_name [vfind / -design SUB_BLOCK_DESIGN]
  ```

- The following command changes the reference of the hierarchical instances top/A, top/B, and top/C to patch.

  ```
  change_link -instances {inst:top/A inst:top/B inst:top/C} \
  -design_name [vfind / -design patch]
  ```

- The following example changes the reference of the hierarchical instance add_0 to the design add:

  ```
  change_link -design_name design:add -instances inst:test/add_0
  ```

connect

```
connect [-net_name string]
{constant|hport|port|pin|hpin|pg_pin}
[-hnet hnet] [-net net] [-in_prefix string]
[-out_prefix string]  [-constant integer]
```

Description

Connects two objects in different levels of the hierarchy.

> You should specify the driving pin before the load pin.

Options and Arguments

`-constant` *integer*	Specifies to connect the object to a constant.
{*constant*|*hport*|*port*|*pin*|*hpin*|*pg_pin*}	
	Specifies the list of objects to connect.
	Only one of the objects can be a `pg_pin` object.
`-hnet` *hnet*	Specifies the `hnet` object to be used for connections.
`-in_prefix` *string*	Specifies the prefix for new input ports.
`-net` *net*	Specifies the `net` object to be used for connections.
`-net_name` *string*	Specifies the name for the net connection.
`-out_prefix` *string*	
	Specifies the prefix for new output ports.

Examples

Use the following command to add connection between the pins of two different instances inside the module:

```
create_module -name module1
create_hinst -name hinst1 -module module1
create_inst -name inst1 -base_cell INVX12HS -parent hinst1
create_inst -name inst2 -base_cell INVX12HS -parent hinst1
create_net -name net1
connect -net_name net1 /
        [get_pins -filter direction==in -of_objects hinst1/inst1] /
        [get_pins -filter direction==out -of_objects hinst1/inst2]
```

Use the following command to add connection between the pins of two different instances inside the top module:

```
create_design -name module1
current_design design:top
create_inst -name inst1 -base_cell INVX12HS module1
create_inst -name inst2 -base_cell INVX12HS module1
create_net -name net1 connect -net_name net1 /
        [get_pins -filter direction==in -of_objects inst1] /
        [get_pins -filter direction==out -of_objects inst2]
```

Related Information

Related command:	disconnect

create_design

`create_design -name` *`string`* `[-quiet]`

Description

Creates a new design at the same level as the existing top-level design.

The new design is created in the `designs` directory. Once the design is created, you can specify instances, `port_bus`, and so on.

Options and Arguments

`-name` *`string`*	Specifies the name of the new design.
`-quiet`	Suppresses the warning messages.

Related Information

Related command:

create_port_bus

create_hinst

```
create_hinst -name string
[-filename string]
[-parent {design|module|hinst}]
-module {design|module} [-quiet]
```

Description

Creates a new instance type in a specified level of design hierarchy.

> You cannot instantiate objects that require a change to a preserved module.

Options and Arguments

`-filename string`	Specifies the file name for the new instance.
`-module {module}`	Specifies the name of the design or subdesign to use for the new instance.
`-name string`	Specifies the name of the new instance.
`-parent {design|module|hinst}`	
	Specifies the higher level object to contain the new instance.
`-quiet`	Suppresses the warning messages.

Examples

- The following command returns newly created hinst under design test:

```
create_hinst -name hinst -module module:test/mod1

hinst:test/hinst
```

- The following command creates an instance of type gor in design gbox:

```
create_hinst -name inst -module module:gbox/gor -parent design:gbox
```

create_hnet

```
create_hnet [-name string]
[-parent {design|module|hinst}]
[-bus string]
```

Description

Creates a new hnet object in the specified level of design hierarchy.

Options and Arguments

`-bus` *string*	Specifies the bit range of bus nets in the form {*msb*:*lsb*}
`-name` *string*	Specifies the name of the new `hnet` object.
`-parent {`*design*`|`*module*`|`*hinst*`}`	
	Specifies the higher level object to contain the new `hnet`.

Examples

- The following command returns newly created hnets with bus bits:

```
create_hnet -name NEW_NET_WITH_BUS -bus 3:0

{hnet:des1/NEW_NET_WITH_BUS[0]} {hnet:des1/NEW_NET_WITH_BUS[1]}
{hnet:des1/NEW_NET_WITH_BUS[2]} {hnet:des1/NEW_NET_WITH_BUS[3]}
```

- The following command returns single hnet:

```
create_hnet -name NEW_NET
```

Related Information

Related commands:

create_inst

create_hport

```
create_hport -name string [-parent {design|module|hinst}]
[-bus string] -direction {in|out|inout}
```

Description

Creates `hport` in a design and also its corresponding `hpins` for the parent object.

Options and Arguments

`-bus` *`string`*	Specifies the bit range of `hport` in the form {*`msb`*:*`lsb`*}
`-name` *`string`*	Specifies the name of the new `hport`.
`-direction {in|out|inout}`	
	Specifies the direction of the new `hport`.
`-parent {`*`design`*`|`*`module`*`|`*`hinst`*`}`	
	Specifies the higher level object to contain the new `hport`.

Example

The following is the usage of the command:

```
create_hport -name NEW_PORT -bus 0:0 -direction in -parent
module:gbox/gorhport_bus:gbox/u3/NEW_PORT

create_hport -name NEW_PORT -bus 0:0 -direction in -parent design:gboxport_bus:gbox/NEW_PORT

create_hport -name NEW_PORT -bus 0:0 -direction in -parent
hinst:gbox/u4hport_bus:gbox/u4/NEW_PORT

create_hport -name gnor/NEW_PORT2 -direction outhport_bus:gbox/u1/NEW_PORT2
```

Related Information

Related command:

create_inst

create_hport_bus

```
create_hport_bus -name string
[-left_bit integer] [-right_bit integer]
{-input|-output}
{inst|hinst|design|module}
```

Description

Creates a `hport_bus` in a design. The command can also create a single subport by omitting both the `-left_bit` and `-right_bit` options.

Options and Arguments

`-input`	Creates an input `hport_bus`.
{*inst*|*hinst*|*design*|*module*}	
	Specifies the name of the object for which to create the `hport_bus`
`-left_bit` *integer*	Specifies the leftmost bit index of the `hport_bus`.
`-name` *string*	Specifies the name of the new `hport_bus`.
`-output`	Creates an output `hport_bus`.
`-right_bit` *integer*	
	Specifies the rightmost bit index of the `hport_bus`.

Examples

- The following is the usage of the command:

```
create_hport_bus -name in1 -input -left_bit 0 -right_bit 2 [get_db hinsts *inst2]

hport_bus:top/inst1/inst2/in1
```

- This command is applicable only to hierarchical instances; if used for any other instance, you get the following error [TUI-254]:

```
create_hport_bus -name in1 -input -left_bit 0 -right_bit 2
inst:top/inst1/inst2/add_51_22/g1

Error: The instance specified is not hierarchical. Specify a hierarchical
instance. [TUI-254] [create_hport_bus]
     : The instance is inst1/inst2/add_51_22/g1
```

Related Information

Related command:

create_inst

create_inst

```
create_inst [-name string] [-filename string]
[-base_cell {base_cell}...]
[-parent {design|hinst|module}] [-dbu]
[-location float...]
[-place_status enum] [-physical]
[-orient enum]
```

Description

Creates a specified instance type in a specified level of design hierarchy. You can instantiate inside a top-level design or a subdesign.

- You cannot instantiate objects that require a change to a preserved module.
- You cannot create a hierarchical loop. That is, if subdesign `A` contains subdesign `B`, you cannot instantiate `A` underneath `B`.

Options and Arguments

`-base_cell` *base_cell*	
	Specifies the `cell` to use for the new instance.
`-location` *float*	Specifies the location of the placed instance. It is specified as a point {*x y*}.
`-name` *string*	Specifies the name of the new instance.
`-filename` *string*	Specifies the file name of the new instance
`-parent` {*design*|*module*|*hinst*}	
	Specifies the object that will contain the newly instantiated instance.
`-dbu`	Specifies to use Data Bus Units (DBUs) for coordinates.
`-physical`	Specifies that the created instance is a filler physical instance.
`-place_status` *enum*	
	Specifies the placement status.
	{`fixed`|`soft_fixed`|`placed`|`unplaced`|`cover`}
`-orient` *enum*	
	Specifies the orientation.
	{r0 | r90 | r180 | r270 | mx | mx90 | my | my90}

Related Information

Related command:

create_hport_bus

create_module

```
create_module -name string
```

Description

Creates a new `module` object.

Options and Arguments

`-name string`	Specifies the name of the new module.

Example

The following command returns newly created module under design test:

```
create_module -name mod1
```

```
module:test/mod1
```

Related Information

Related command:

create_inst

create_port

```
create_port -name string
[-bus string] [-direction {in|out|inout}]
[-design design]
```

Description

Creates a `port` object in a design.

Options and Arguments

`-bus` *`string`*	Bus bits for the new port {msb:lsb}.
`-design` *`design`*	Specifies the name of the design for which to create the `port`.
	The design name can be omitted if there is only one top-level design.
`-direction {in|out|inout]`	
	Specifies the direction of the new port.
`-name` *`string`*	Specifies the name of the new port.

Example

The following command creates a new port with direction `out`:

```
create_port -name port1 -direction out
```

Related Information

Related command:

create_design

create_port_bus

```
create_port_bus -name string
[-left_bit integer] [-right_bit integer]
{-input|-output} [-direction in|out|inout]
[design]
```

Description

Creates a `port_bus` object in a design. The command can also create a single port by omitting both the `-left_bit` and `-right_bit` options.

Options and Arguments

design	Specifies the name of the design for which to create the `port_bus`.
	The design name can be omitted if there is only one top-level design.
`-direction` *in* | *out* | *inout*	Specifies the direction of the new port.
`-input`	Creates an input `port_bus`.
`-left_bit` *integer*	Specifies the leftmost bit index of the `port_bus`.
`-name` *string*	Specifies the name of the new `port_bus`.
`-output`	Creates an output `port_bus`.
`-right_bit` *integer*	
	Specifies the rightmost bit index of the `port_bus`.

Related Information

Related command:

create_design

create_primitive

```
create_primitive [-name string] [-inputs integer]
[-quiet] -function logic_function
-inside {design|module}
```

Description

Creates an unmapped primitive cell in a design or a subdesign.

Options and Arguments

`-function` *logic_function*			
	Specifies the logic function of the primitive cell. You can specify any of the following:		
	`and`	`dc`	`notif0`
	`bmux`	`latch`	`notif1`
	`buf`	`mux`	`or`
	`bufif0`	`nand`	`xnor`
	`bufif1`	`nor`	`xor`
	`d_flop`	`not`	
`-inputs` *integer*	Specifies the number of input pins to create for the primitive cell.		
`-inside {`*design*`|`*module*`}`			
	Specifies the name of the design or subdesign in which you want to instantiate the primitive cell.		
`-name` *string*	Specifies the name of the new primitive cell.		
	If you specify same name for two primitive cells created under the same hierarchy, then Genus will automatically add a number to the suffix of the second primitive cell name along with a warning regarding naming conflicts, which can be suppressed by using the `-quiet` option.		
`-quiet`	Suppresses the warning messages.		

Examples

- The following command creates a 2-input xor primitive cell inside the top-level design:

  ```
  create_primitive -function xor -inside design:top -name xor_gate -inputs 2
  inst:top/xor_gate
  ```

- The following command creates a D flip-flop primitive cell inside the sub-design sum:

  ```
  create_primitive -function d-flop -inside module:top/sum -inputs 1 -name dflop
  inst:top/inst1/dflop
  ```

- The following command creates a not primitive cell inside the top-level design and adds a number to the suffix of 'not_gate' as there is already a non-primitive cell with name 'not_gate' inside top-level design:

  ```
  create_primitive -function not -inside design:top -name not_gate -inputs 1

  Warning: The requested object name is already in use.[TUI-266]
         :The requested name was 'not_gate' but the new primitive instance is named
  'not_gate1'.
         :A different name for the object is chosen to avoid  name  conflict.  You
  can  store  the  newly created object in a variable to directly access it without
  having to look for it by name:
         set var [edit_netlist ...]
         inst:top/not_gate1
  ```

dedicate_module

```
dedicate_module

 {inst|hinst} [inst|hinst]...[-force]
```

Description

Creates a dedicated sub-module for a particular hierarchical instance, when a subdesign is instantiated multiple times in the design.

This command is applicable for hierarchical instances but not for leaf instances.

The `uniquify` command creates dedicated sub-module for all the instances, where as `dedicate_module` creates a separate module definition only for the instances mentioned with the command.

Options and Arguments

inst|hinst	Specifies the name of a hierarchical instance for which you want a dedicated module.
	You must specify a list of instances that share the same module.
	This option accepts TCL lists and wildcards (*, ?).
`-force`	Forces to dedicate after tracing and dedicating the parent hierarchy.

Example

Some usage examples:

```
get_db hinst:dedicate_2/g?hinst:dedicate_2/g1  hinst:dedicate_2/g2  hinst:dedicate_2/g3

dedicate_module [get_db hinst:dedicate_2/g?]module:dedicate_2/sub

dedicate_module { hinst:dedicate_2/g1 hinst:dedicate_2/g2 }module:dedicate_2/sub_2

set s { hinst:dedicate_2/g1 hinst:dedicate_2/g2 hinst:dedicate_2/g3
}hinst:dedicate_2/g1  hinst:dedicate_2/g2  hinst:dedicate_2/g3

dedicate_module $smodule:dedicate_2/sub

dedicate_module [get_db hinst *]module:dedicate_2/sub
```

delete_obj

```
delete_obj object+ [-quiet]
```

Description

Removes an object from the design hierarchy. This command is similar to its UNIX counterpart.

Deletes a list of existing objects from the database. Refer to command help for the current list of objects that can be removed.

If the hierarchical pin or port bus object has a net connection, the net is disconnected first and then the object is removed.

If you remove a design, the power intent-related information will also be removed from the design hierarchy.

> This command does not work on the `pin` or `port` object.

Options and Arguments

object+	Specifies the objects to be deleted.
	The command help lists the valid removable object types.
`-quiet`	Specifies to keep quiet unless there is a problem.

Example

The following command deletes all clock constraint objects from the design:

```
delete_obj [get_db clocks *]
```

Related Information

Related commands:

rename_obj
report_obj

delete_unloaded_undriven

```
delete_unloaded_undriven
[-disconnect] [-force_bit_blast] [-all]
[-inout] design [> file][-unconstrained]
```

Description

Disconnects subports and hierarchical pins connected to constants and that do not fanout to anything, and deletes unloaded and undriven subports from the design. Use this command as a post-processing step to remove any unused subports from the netlist.

By default the command skips individual bits of a bus that are connected to constants or that are unused.

Options and Arguments

`-all`	Additionally, deletes unloaded and undriven top-level ports from the design.
design	Specifies the design from which to remove the unused ports or subports.
`-disconnect`	Only disconnects the subports and hierarchical pins that are connected to constants and that do not fanout to anything.
file	Specifies the name of the file to which the output of the command should be redirected.
`-force_bit_blast`	Bitblasts modules that have individual bus bits that are unused or connected to constants and deletes the unused bus bits.
`-inout`	Specifies to delete disconnected inout pins too.
`-unconstrained`	Specifies to skip hpin with constraint or an exception.

disconnect

```
disconnect
{pin|hpin|pg_pin|port|hport}... [-keep_dangling_net]
```

Description

Disconnects multiple pins, ports, or hierarchical ports from each object they are connected to. For example, if `A`, `B`, and `C` are connected together and you disconnect `A`, then `B` and `C` remain connected to each other, but `A` is now left unconnected.

This command also accepts Tcl lists and wildcard entries as input.

You cannot disconnect an object that would require changes to a preserved module.

You cannot disconnect a generic constant (1 or 0) pin of the module but you can disconnect the loads from that pin.

You can disconnect an object that is not currently connected to anything else. In that case nothing happens.

Options and Arguments

hpin	Specifies the name of the pin of a hierarchical instance to disconnect.
hport	Specifies the name of a port on a hierarchical instance) to disconnect.
-keep_dangling_net	
	Specifies to keep the net object even if it has no loads or drivers.
pg_pin	Specifies the name of the power or ground pin to disconnect.
pin	Specifies the name of an instance pin to disconnect.
port	Specifies the name of a design port to disconnect.

Examples

- The following command disconnects pins in_0 and in_1 of instance g1:

  ```
  disconnect {pin:top/isub/g1/in_0 pin:top/isub/g1/in_1}
  ```

- The following command disconnects all pins of instance g1:

  ```
  disconnect pin:top/isub/g1/*
  ```

Related Information

Related command:

connect

flatten_complex_ports

```
flatten_complex_ports
[module | design] ...
```

Description

Converts two-dimensional port arrays into one-dimensional port arrays.

Options and Arguments

module | *design*	Specifies the module(s) or design for which the port conversion needs to be done.
	By default, all ports of all modules including top module get converted.

Example

The following command converts the two-dimensional array `my_port[7:0][3:0]` to one-dimensional array `my_port[31:0]` if `my_port` is a port of module A.

```
flatten_complex_ports A
```

group

```
group [-name group_name]
{inst | hinst} [inst | hinst]...
```

Description

Creates a level of the design hierarchy by grouping the specified instances. You can only group instances that belong to the same hierarchy.

Options and Arguments

`-name` *group_name*	
	Specifies the name of the module that groups the specified instances.
	By default, the resulting module will have an instance name consisting of the specified group name with the suffix `i`. The suffix is used to indicate that this hierarchy is the result of a `group` command.
	You can change the suffix with the `group_instance_suffix` attribute.
{*inst* | *hinst*}	Specifies the name of an instance to add to the specified group. You need to specify at least one instance.

Example

A sample usage of the group command is shown below:

```
group -name group_module inst:test/*

hinst:test/group_module
```

Related Information

Related command:

ungroup

Affected by these attributes:

group_generate_portname_from_netname
group_instance_suffix

remove_cdn_loop_breaker

```
remove_cdn_loop_breaker
[-instances {inst|hinst}...] [design]
```

Description

Removes the specified loop breaker buffers added by the timing engine and restores the loop.

> As Genus relies on the absence of uncut combinational feedback loops to time the design, invoking the timer through any method (such as `report_timing`) will cause the loop breaker cells to be re-inserted unless they are cut by some other means.

Options and Arguments

design	Specifies the design for which the loop breakers must be removed.
-instances {*inst*|*hinst*)	
	Specifies the loop breakers instances that you want to remove.
	If this option is not specified, all instance loop breakers will be removed.

Related Information

Related command:

report_loop

rename_obj

```
rename_obj [-flexible] [-slash_allowed] object new_name
```

Description

Renames a design object.

Options and Arguments

-flexible	Specifies that the new name is flexible for renaming in case there is a name collision.
new_name	Specifies the new name for the object.
object	Specifies the object to be renamed.
-slash_allowed	Specifies that the destination name can have embedded slashes.

Example

The following command renames instance object inst1 to inst1_mod:

```
rename_obj [get_db insts inst1] inst1_mod
```

Related Information

Related command:

update_names

reset_design

```
reset_design
[-timing] [-dft]
[-verbose] [design]
```

Description

Removes all the user-specified timing and DFT objects, as well as all the floorplan objects, and returns all attributes to their default values for the specified design. Alternatively, this command removes every clock domain, cost group, exception, external delay, scan chain, scan segment, test clock domain, and test signal while returning all attributes on the design to their default values. This allows you to reload the design in the same session while debugging some problem and provides faster debugging when repeatedly loading files to debug problems inside one session.

It does not reset the internal state of applications or global settings, so it is not reliable enough to use in a real flow. For example, it does not free collections but only invalidates them. If you try to access collections saved in TCL variables, you will not be able to see those collections. If you have saved pointers to DB objects in TCL variables, they will be invalid and may even cause a crash if used.

If you have any unexpected problems, you should start a new session without using `reset_design`.

> This command should be only used for debugging purposes in real implementation flow. Currently, it does not work on MMMC designs.

Options and Arguments

design	Specifies the particular design to reset when there are multiple designs.
-dft	Removes only the DFT constraints.
-timing	Removes only the timing constraints.
-verbose	Prints messages indicating that the command was successful.

Example

A sample flow using reset_design command is given below:

```
init_design (or read_db)

<read some files that have problems (such as a Tcl script, MMMC file, LEF file)>

<fix the files based on error messages>

reset_design

init_design (or read_db)

<read modified files to see if they are ok now>
```

If there are still errors, make more modifications, and go back to `reset_design`. If the final result is good, you should start over in a new session and repeat it without `reset_design` to continue with the flow.

Related Information

Related commands:	reset_power_intent
	reset_db

ungroup

```
ungroup
[ -all | -flatten instance...
| -threshold integer]
[-simple][-prefix string]
[-exclude instance...]
[-only_user_hierarchy] [-keep_timer_on][-force] [-verbose]
```

Description

Ungroups the specified instances. Ungrouping dissolves the hierarchy and moves the contents of a subdesign into its parent directory. By default, an instance is named by concatenating its name to its parent's name.

Options and Arguments

`-all`	Ungroups all instances at the current level.
`-exclude` *instance*	Specifies a list of instances that should not be ungrouped.
`-flatten`	Recursively ungroups all the specified instances.
`-force`	Forces to ungroup all hierarchies.
	Before mapping, this will include non-user hierarchies.
instance	Specifies the instance or instances to be ungrouped.
`-only_user_hierarchy`	Ungroups only those hierarchies created by the user. The hierarchies created by the tool are preserved. This option applies after mapping.
	If you execute this command before you execute the `syn_map` command, ungrouping will by default only apply to user-created hierarchies unless you specify the `-force` option.
`-keep_timer_on`	Keeps the timer on while ungrouping.
`-prefix` *string*	Adds the specified prefix to the names of the wires, nets, and instances created as a result of flattening the hierarchical instance(s).
`-simple`	Prevents the use of a complex new instance name during ungrouping. This option has the same effect as the `ungroup` command.
`-threshold` *integer*	Ungroups only those hierarchical instances that have a cell count equal to or less than the specified integer.
`-verbose`	Prints the names of all hierarchies being ungrouped.

Related Information

Related command:

group

uniquify

```
uniquify
{module|design} [-verbose]
```

Description

Uniquifies the instances under the specified design or subdesign. Uniquification is the process of creating a new subdesign for an instance or a group of instances. The newly created subdesign is merely a copy of the subdesign to which the original instance or group of instances were associated. That is, an instance or a group of instances will now be a part of their own, *unique* subdesign. The newly created subdesign will usually maintain the original design or subdesign name followed by a number suffix.

The naming style of uniquified subdesigns is controlled by the attribute `uniquify_naming_style`.

> Preserved modules cannot be uniquified. By default, `uniquify` command will uniquify all modules except its first occurrences, as it will be unique after other modules are uniquified.

Options and Arguments

{*design*|*module*}	
	The instances under the specified design or subdesigns will be uniquified.
`-verbose`	Prints out the uniquified instances and their corresponding subdesign names.

update_names

```
update_names[-hnet | -inst | -design | -module | -port_bus | -hport_bus]...
[-local] [-force] [-lec] [-quiet]
[-vhdl] [-verilog] [-system_verilog] [-unresolved_ok]
[-prefix string][-name_collision]
[-suffix string][-name_collision]
[-first_restricted string][-restricted string]
[-last_restricted string][-replace_str string]
[-reserved_words string] [-max_length number]
[-map string] [-allowed string...][-exclude [inst | hinst | design | hnet | module | port |
hport | port_bus | hport_bus][-regexp]
[-check_internal_net_name][-collapse_name_space]
[-dummy_net_prefix string][-skip_bus_net]
[-nocase][-lower_to_upper][-upper_to_lower]
[-log file][-append_log][inst | hinst | design]
```

Description

Changes names of nets, busses, instances, designs, subdesigns, ports, port buses, and subport buses. You can specify one of more object types. If no object type is specified, the requested change applies to all object types unless otherwise specified. There is no restriction on the length of the name.

By default, all changes are global: changes are made to all objects (instances of the specified object types) in the design. You can specify the name of a hierarchical instance to restrict the changes to only the objects in that instance.

> This command does not affect ILMs, including the name of its module, instances, hports, and hnets.

Options and Arguments

`-allowed` *`string`*	Specifies the characters that are allowed in names. Any characters that are not in the allowed list will be ignored in the resulting names. The minimum specification is 10 characters. To allow all the letters from a to z in capital and lower case letters, you must specify all of them. That is, you cannot use a dash ("-") to indicate inclusion.
`-append_log`	Appends the information of the last `update_names` command to the logfile specified with the `-log_changes` option
	If you omit this option, the information of the last `update_names` command overwrites the current information in the logfile.
	You can only specify this option if you specified the -`log_changes` option.
`-check_internal_net_name`	
	Adds the suffix `_int` to any net whose name matches that of a port or subport but is not connected to that port or subport.
`-collapse_name_space`	
	Adds the suffix `_design` to either the port, subport, net, or subdesign in a hierarchy only if they have similar Verilog names.
`-design`	Changes the names of design objects.
`-dummy_net_prefix` *`string`*	
	Specifies the prefix to use for the names of dummy nets when writing out the netlist or HDL.

	This option does not change the names in the design hierarchy.
`-first_restricted` *string*	
	Specifies the characters that cannot be used as first character in a name.
`-force`	Forces the name change even if the object name is preserved.
hinst `|` *inst* `|` *design*	
	Specifies the name of the hierarchical instance or the design to which the changes must be applied.
`-hnet`	Changes the names of net objects.
`-hport_bus`	Changes the names of hport bus objects.
	You cannot change the left bracket, "[", and the right bracket, "]", because they are a part of the bus name when referencing individual bits of the bus.
`-inst`	Changes the names of instance objects.
`-last_restricted` *string*	
	Specifies the characters that cannot be used as last character in a name.
`-lec`	Captures the changes in the log file in LEC preferred format.
`-local`	Restricts the changes to the current directory.
	Default: global changes

`-lower_to_upper`	Changes the names of all objects from lowercase to uppercase. This applies to objects of type instance, port, hnet, subport, design, and subdesign.
`-log` *file*	Specifies the name of the logfile that shows which names were changed using the `update_names` command and the result of the changes.
`-map {{"`*from*`" "`*to*`"}...}`	Maps the specified *from* string to the specified *to* string. Enclose each string in double quotes and separate the strings with a space. Enclose each set in braces. If you specify several sets, separate them with spaces and enclose the list of all sets with braces.
`-max_length` *integer*	
	Limits the length of the changed name to the specified number. If the resulting names are longer than the specified integer, the last letters will be truncated.
`-module`	Changes the names of subdesign objects.
`-name_collision`	Indicates to only use the specified prefix or suffix values to change object names if a name clash would occur while executing the `update_names` command using other options.
`-nocase`	Does not take case sensitivity into account.
`-port_bus`	Changes the names of the top-level port_bus objects.

	You cannot change the left bracket, "[", and the right bracket, "]", because they are a part of the bus name when referencing individual bits of the bus.
`-prefix` *`string`*	Adds a prefix to the names of the objects to be changed.
`-quiet`	Specifies to disable messages related to change names. The disabled message IDs are CHNM-102, CHNM-103 and CHNM-110.
`-regexp`	Allows you to specify character ranges with the `-allowed` option or `-first_restricted` option and allows you to specify full regular expression with the `-map` option
`-replace_str` *`string`*	
	Specifies the replacement string. Specify NULL for a null string.
	Default:_
`-reserved_words` *`string`*	
	Specifies words to be avoided, such as "begin end".
`-restricted` *`string`*	
	Specifies the list of strings that cannot be used in a name. Separate independent patterns to be replaced with a "space." These strings are replaced with the `-replace_str` *`string`*.
`-skip_bus_net`	Skips nets that are part of busses are changed. Only scalar nets can be changed.

`-suffix`	Adds a suffix to the names of the objects to be changed.
`-system_verilog`	Replaces SystemVerilog reserved words.
`-unresolved_ok`	Changes the names of unresolved modules.
`-upper_to_lower`	Changes the names of all objects from uppercase to lowercase. This applies to objects of type instance, port, hnet, subport, design, and subdesign.
`-verilog`	Replaces Verilog reserved words.
`-vhdl`	Replaces VHDL reserved words as well as strings that start with a digit, an underscore, continuous (two or more) underscores, and end with an underscore. You do not need to specify the `-no_case`, `-inst`, or `-module` option if you specify the `-vhdl` option.
`[-exclude [`*`inst	hinst	design	hnet	module	port	hport	port_bus	hport_bus`*`]`	
	Specifies the list of objects to exclude.

Examples

- The following example adds a suffix `_t` to the design name:

  ```
  genus@root:>update_names -design -suffix _t
  ```

  If the module name of the design was `SAMPLE`, it will be renamed to `SAMPLE_t`.

- The following example replaces all lowercase "`n`" with uppercase "`N`", and all underscores "_" with hyphens "-" in all instance names.

  ```
  genus@root:> update_names -instance -map {{"n" "N"} {"_" "-"}}
  ```

- The following example replaces all lowercase "`a`" with uppercase "`A`" on all modules and hports.

  ```
  genus@root:> update_names -map {{"a" "A"}} -module -hport_bus
  ```

- The following example replaces any instances of `ab`, `bc`, or `ca` with "@" in all object names.

  ```
  genus@root:> update_names -restricted "ab bc ca" -replacec_str "@"
  ```

- The following example specifies the maximum length of all subdesign names to be 12 characters. Issue this command after elaboration or before writing out the netlist.

  ```
  genus@root:> update_names -max_length 12 -module
  ```

- The following two commands both change the brackets ("[" and "]") in the instance name to "`x`"s:

  ```
  update_names -instance -restricted {[ ]} -replace_str "x"
  ```

  ```
  update_names -instance -restricted "\[ \]" -replace_str "x"
  ```

  > There is no need to escape special characters when enclosing them in braces ({}). If you added the escape character inside the braces, the tool would try to replace this character as well with "x".

- The following example allows all capital and lower case letters, numbers, underscores, backslashes, and brackets:

  ```
  genus@root:> update_names -allowed \
  ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789_\[\]
  ```

  > You cannot use the dash "-" to indicate inclusion. That is, the following example is not allowed:

  ```
  genus@root:> update_names -allowed a-zA-Z0-9_[]
  ```

  However, the following does work:
  genus@root:> update_names -regexp -allowed {a-zA-Z0-9_}

- The following example creates a separate `update_names` log file, `test.log`, that reflects the module name change:

  ```
  genus@root:> update_names -map {{"SUB" "HERO_SUB"}} -module -log test.log
  ```

- The following example indicates that the `d` cannot be used as the first character in a net name and that when a name collision would occur, the prefix `xx` can be used.

```
update_names -name_collision -first_res d -prefix xx
```

- The following example shows a part of a netlist that includes VHDL reserved words:

  ```
  generate u1(.A (eg1[0] ), .B (B[0]), .Y (Y[0]));

  open u2(.A (eg1[1] ), .B (B[1]), .Y (Y[1]));

  endmodule  u3(.A (eg1[2] ), .B (B[2]), .Y (Y[2]));
  ```

  To change, or eliminate, the names of the VHDL reserved words, use the `-vhdl` option.

  ```
  genus@root:> update_names -vhdl
  ```

  Now the netlist does not contain any VHDL keywords:

  ```
  generate_cn u1(.A (eg1[0] ), .B (B[0]), .Y (Y[0]));
  open_cn u2(.A (eg1[1] ), .B (B[1]), .Y (Y[1]));
  endmodule  u3(.A (eg1[2] ), .B (B[2]), .Y (Y[2]));
  ```

Related Information

Related command:

write_hdl

9

Elaboration and Synthesis

- add_assign_buffer_options
- add_buffer
- create_base_cell_set
- delete_buffer_tree
- elaborate
- get_remove_assign_options
- insert_buffer
- merge_to_multibit_cells
- remove_assigns_without_opt
- remove_clock_reconvergence
- syn_generic
- syn_map
- syn_opt

add_assign_buffer_options

```
add_assign_buffer_options
[-design {design | module}] [-ports ports]
[ [-buffer_or_inverter lib_cell]
  [-allow_unloaded_buffers]
  [-no_buffers_use_inverters]
  [-use_inverted_signal]
  [-ignore_preserve_setting]
  [-ignore_preserve_map_size_ok]
  [-include_local_constant_assign]
  [-preserve_dangling_nets] [-skip_ideal_nets]
  [-dont_respect_boundary_optimization]
  [-dont_skip_unconstrained_paths]
  [-skip_hierarchical_subdesigns]
  [-skip_remove_assign_on_clock_path]
  [-verbose] [-reset] [-clock_buffer_or_inverter lib_cell]
```

Description

Controls the aspects of the replacement of `assign` statements in the design with buffers or inverters, which is controlled by the `remove_assigns` root attribute. The actual replacement happens during the

next incremental optimization run.

The replacement of `assign` statements is performed

- Only on the objects (design or modules) specified with the `-design` option
- Only once during each incremental optimization run operation
  If your script contains multiple occurrences of the `add_assign_buffer_options` command with different settings for the same object, the last settings before the next synthesis command (`syn_*` commands) will prevail on the objects in the order of their corresponding `add_assign_buffer_options` commands.
- The specified options apply to all subsequent incremental optimization runs unless you reset the options with the `-reset` option.

Options and Arguments

`-allow_unloaded_buffers`	
	Prevents unloaded nets from being deleted and allows unloaded buffers to be added on these nets to fix the assigns in the netlist.
`-buffer_or_inverter` *`lib_cell`*	
	Specifies the buffer or inverter to be used to replace the `assign` statements. The specified cell must be part of a library that was loaded.
	If this option is omitted, Genus will determine which buffers or inverters to insert. If a particular library domain does not have any buffers or inverters, no buffers or inverters will be added in that domain. However, buffers or inverters will be added in other domains depending on availability.
`-clock_buffer_or_inverter` *`lib_cell`*	
	Specifies the clock `lib_cell` to be used to replace the `assign` statements. The specified cell must be part of a library that was loaded.
`-design {`*`design`* | *`module`*`}`	
	Indicates that the specified command options apply to the specified design or module. If neither is specified, the options apply to the entire design.
`-dont_respect_boundary_optimization`	

	Allows Genus to disconnect the unloaded subports of a module, if the `boundary_opto` attribute on the module was set to `false`.
`-dont_skip_unconstrained_paths`	
	Specifies to remove assigns on unconstrained paths. These paths can be paths driven by a constant, false paths with assigns, or paths without any timing constraints.
	By default, Genus skips unconstrained paths.
	This option has no effect in the Genus-Physical flow.
	For place and route tools removing `assign` statements is only necessary on constrained paths because these paths might not be optimized well with assigns present.
	Removing `assign` statements on unconstrained paths like constant driven nets/hierarchical ports might cause placement challenges during legalization in the Genus-Physical flow as these nets and ports don't have any anchors to guide the placement engine.
	Consequently the tool does not remove assign statements on unconstrained paths. This should not have any effect on the place and route tools. Genus handles assign statements natively and does not require assign removal.
`-ignore_preserve_map_size_ok`	
	Allows `assign` statements in the module to be removed if the `preserve` attribute on the module is set to `map_size_ok`.
`-ignore_preserve_setting`	
	Allows `assign` statements in the module to be removed independent of the setting of the `preserve` attribute on the module.
`-include_local_constant_assign`	
	Allows to remove local constant driven assigns that have multiple drivers and do not drive hierarchical ports.
`-no_buffers_use_inverters`	

	Specifies to search for inverters and to use them in case the library or library domain does not have any buffers. If buffers exist they will be used, irrespective of whether the library has inverters or not.
`-ports` *ports*	Limits the replacement of `assign` statements to the specified ports.
`-preserve_dangling_nets`	
	Specifies to preserve a net that is driven by a buffer or inverter that is inserted when `assign` statements are replaced and if that net does not have a fanout in the next level of hierarchy and was not driven by a constant before adding the buffer or inverter.
`-reset`	Specifies to use the command with the default settings.
`-skip_hierarchical_subdesigns`	
	Restricts the replacement of `assign` statements to one level only. By default, the replacement of `assign` statements is done recursively.
`-skip_ideal_nets`	Allows Genus to skip ideal nets from buffer insertion.
`-use_inverted_signal`	
	Allows to use a single inverter with an inverted signal instead of a buffer if the inverter cell is smaller in area.
`-skip_remove_assign_on_clock_path`	
	Allows Genus to skip remove `assign` on clock path.
`-verbose`	Displays all messages during `assign` removal.

Related Information

Affected by this command:	syn_opt
Affected by this attribute:	remove_assigns

add_buffer

```
add_buffer       [-lib_cell lib_cell...]       [-base_cell lib_cell| base_cell|..]
      [-inst_name string]
      [-net_name string]
      [-use_inverters]
      [-location lowconn | highconn]      [-pins hport|port|pin|hpin+]      net|hnet|hport|port|pin|hpin
```

Description

Adds buffer or inverter pair in the netlist on the mentioned pin, port, or net.

Options and Arguments

`-lib_cell lib_cell`	Specifies the `lib_cell(s)` to be used for the new buffer or inverters.
`-base_cell lib_cell | base_cell`	Specifies the base_`cell(s)` to be used for the new buffer or inverters.
`-inst_name string`	Specifies the name of the new buffer or inverter instance.
`-net_name string`	Specifies the name of the new net name to be used in connection.
`-use_inverters`	Adds inverters
`-location lowconn | highconn`	Specifies the location of insertion.
net|hnet|hport|port|pin|hpin	
	Specifies the object or name of the object where buffer or inverters are to be added.
`-pins pin|hpin|hport|port+`	
	Specifies the list of pins on whose common driver needs buffer.

Examples

- The following command will insert a buffer of type 'buf1' on the given pin:

  ```
  add_buffer -inst_name new_buff -lib_cell buf1 [get_pins in1]
  ```

- The following command will insert two inverters at pin Y:

  ```
  add_buffer -inst_name "MY_INV1 MY_INV2" -net_name MY_INV_NET pin:top/m1/i1/Y -
  use_inverters
  ```

- The following command will insert a buffer on the given pin A:

  ```
  add_buffer pin:top/m2/i1/A -location highconn
  ```

create_base_cell_set

```
create_base_cell_set -name string
[-base_cells {base_cell|lib_cell}...]
```

Description

Creates a `base_cell_set` object that is used as the `dont_use_base_cell_set` attribute data value.

The new object is created in the libraries directory.

Options and Arguments

`-name` *`string`*	Specifies the name of the new set.
`-base_cells {`*`base_cell`*`|`*`lib_cell`*`}`	
	Specifies the base cells of this set.
	If this option is not specified, an empty `base_cell_set` is created.

Related Information

Related attribute:	dont_use_base_cell_set

delete_buffer_tree

```
delete_buffer_tree
[ -drivers {port|pin|hpin}...
| -exclude {port|pin|hpin}...]
[-cone] [-verbose] [design]
```

Description

Removes the buffers and inverter pairs exhaustively or partially (starting from the specified drivers). This command can for example be used to remove any buffering done on clock paths, or on the paths of DFT control signals.

Options and Arguments

`-cone`	Removes the buffers and inverter pairs starting from the specified drivers to the timing end points. This option is effective only with the `-drivers` option.
	If you omit this option, deletion of buffers and inverter pairs stops at the first logic instances or sequential instances from the specified drivers.
design	Specifies the name of the design from which buffers and inverter pairs need to be removed.
`-drivers`	Specifies the pins and ports from where the buffer tree needs to be removed.
	By default, all drivers are considered.
`-exclude`	Specifies the drivers from which the buffer tree should not be removed. The buffer tree will be removed from all other drivers.
`-verbose`	Prints information about the buffer trees that are removed.

elaborate

`elaborate [-parameters` *string*`] [`*module*`]...`
`[-lib_path` *path*`]... [-lib_extension` *extension*`]...`

Description

Creates a design hierarchy consisting of a top-level design and its referenced subdesigns from Verilog or SystemVerilog modules or VHDL entity/architectures. If the top-level design is not specified, it elaborates all the modules not instantiated by any other module as top-level and elaborates them all along with their subdesigns.

The command transforms each module/architecture to be represented as a design in the form of a structural netlist. It also performs semantic checking, sequential register (flops/latches) inferencing, and high-level HDL optimizations. Instances of undefined modules or entities are marked as unresolved and the corresponding modules/entities reported as blackboxes. You can set the attribute `hdl_error_on_latch` to `true`, to error out for latches inferred during elaborate and set the attribute `hdl_error_on_blackbox` to `true`, to error out for blackboxes during elaborate

The command returns the directory path to the top-level design(s) that it creates.

Before elaborating a design, load your library using the `library` attribute and load your design using the `read_hdl` command into the Genus shell.

Options and Arguments

`-lib_extension` *extension*	
	Specifies the extension of the Verilog library files.
	User-specified extensions overwrite the default extensions.
	Default: `.v` and `.V`
`-lib_path` *path*	Specifies the search path to be used to look for unresolved Verilog module instances.
	You can specify a relative path with respect to the working directory (.) or an absolute path.
	`~` is currently not supported.
module...	Specifies the name of either the top-level module or the VHDL configuration to elaborate.
	If you specify the configuration name of a module instead of the module name, elaboration will occur according to the specified configuration.
	If you omit *module*, all top-level modules or configurations are elaborated.
	When reading in a structural file using the read_hdl `-netlist` command, use this option to specify the top module name.
`-parameters` *string*	

	Changes the values of Verilog parameters or VHDL generics that are used within the top level code to the specified values.
	VHDL generics can be specified in the following forms: integer: `3, -10` Verilog bit string: `1'b1, 8'hff, 9'so777` boolean: `true, false, 1'b1, 1'b0, 1, 0` string literal: `{"hello, world"}`
	Specify the new values in the same sequence as the parameter definitions appear in the code to ensure proper substitution during elaboration.
	Genus supports pure and sized integer specifications, and specifications of parameters for submodules and interfaces.

	The following example shows a sized integer specification. In this case, 6 is the size (number of bits), d the format (decimal), and 43 the value.
	`elaborate -parameters {6'd43}`
	You can specify a parameter positionally, as an integer in the parameter list, or by name, as a two-element Tcl list. The first element is the name and the second element is the value. For example, you can do either:
	`elaborate -param {5 10}`
	`elaborate -parameters {{width 5} {depth 10}}`
	The following example shows the specification of parameters for submodules or interface instances:
	`elaborate -parameters {{b2.m 0} {b1.w 7'd111}}`

	This option also supports complex parameters like structures, multi-dimensional packed-unpacked arrays, enumerated-valued parameters, assignment patterns.

	Some examples of complex parameter usage:
	`unpacked array: elab -parameters {test '{3, 4, 5, 7}}`
	`structure: elab -parameters '{2{'{3'b101,4'b1010,5'b10011}}}`

Examples

- In the following example, the top-level code contains the following parameters in this order:

```
parameter data_width1 =  8  ;
parameter data_width2 = 12  ;
parameter averg_period = 4 ;
```

The following command changes `data_width1` to 14, `data_width2` to 12, and `averg_period` to 8 during elaboration:

```
elaborate -parameters {14 12 8} TOP
```

- The following is an example of a complex parameter usage:

```
module test_01(out1, out2, out3, out4, out5);
  output out1, out2, out3, out4, out5;
  parameter                p1 = 1;
  parameter                p2 = 2+p1;
  parameter                p3 = 3+p2;
  parameter int test [0:3] = '{1, 2, 3, 4}; //unpacked array
  parameter st_1  off_set = '{2{'{3'b101,4'b1000,5'b10001}}}; //structure

  test2 #('{p1, p2}) t1_p (out1, out2);
  assign out3 = (p1 == 5);
  assign out4 = (test[1] == 4);
  assign out5 = (test[3] == 7);
endmodule
```

For these complex parameters, the elaborate command passes the parameters as:

```
elaborate -parameters {{p1 5} {test '{3, 4, 5, 7}} {p3 9} {off_set
'{2{'{3'b101,4'b1010,5'b10011}}}}}

OR

elaborate -parameters {5 7 9 {3,4,5,7} '{2{'{3'b101,4'b1010,5'b10011}}}}
```

- The following example reads in file `top.v` which has an instance of module `sub`, but file `top.v` does not contain a description of module `sub`.

```
set_db hdl_search_path { ../src ../incl }

elaborate -lib_path ../mylibs -lib_path /home/verilog/libs -lib_extension ".h"
-lib_extension ".v" top.v
```

The latter command is equivalent to

```
elaborate -lib_path { ../mylibs /home/verilog/libs } -lib_extension { ".h" ".v" }
top.v
```

First, `elaborate` looks for the `top.v` file in the directories specified through the `hdl_search_path` attribute. After `top.v` is parsed, `elaborate` looks for undefined modules (such as `sub`) in the directories specified through the `-lib_path` option. First, the tool looks for a file that corresponds to the name of the module appended by the first specified file extension (`sub.h`). Next, it looks for a file that corresponds to the name of the module appended by the next specified file extension (sub.v), and so on.

- Consider the following VHDL example:

```
entity abc is
 generic (str : string := "hello";
    x : bit_vector := "1111";
    w : integer := 5;
    bool : boolean := true);
…
end abc;
```

The following command changes `x` to `"1010"`, `str` to `"hello world"`, `w` to `5` and `bool` to `false`:

```
elaborate -parameters { {x "1010"} {str {"hello world"}} {w 5}  {bool false} } abc
```

The following command specifies the name of the configuration to elaborate.

```
elaborate $config_name
```

Related Information

Affected by this command:	read_hdl
Affected by this attribute:	library

get_remove_assign_options

```
get_remove_assign_options
[-all | -design {design|module]
```

Description

Allows you to check which options you set for the replacement of assign statements on a design or module.

If you did not set any options on a module, this command will not return anything for this module. However if you did set options for the design, these options will be used for all modules unless you specified the `-skip_hierarchical_subdesigns` option.

Options and Arguments

`-all`	Returns settings for the design and all modules.
`-design {`*`design	module`*`}`	
	Returns the settings for the specified design or specified module.

Related Information

Affected by these commands:

remove_assigns_without_opt
add_assign_buffer_options

insert_buffer

```
insert_buffer -buffer lib_cell
[-name string] [-pin {pin|hpin|port|hport}]
```

Description

Inserts the given buffer at the specified pin.

Options and Arguments

`-buffer` *lib_cell*	Specifies the library cell to be used.
`-name` *string*	Specifies the name of buffer instance.
`-pin {`*pin*`|`*hpin*`|`*port*`|`*hport*`}`	
	Specifies the pin object to insert the buffer.

Example

The following command will insert a buffer of type `buf1` on the given pin:

```
insert_buffer -name new_buff -buffer buf1 -pin in1
```

Related Information

Related command:

add_assign_buffer_options

merge_to_multibit_cells

```
merge_to_multibit_cells
[[-sequential] [-combinational] [-iso_ls] | -all]
[-logical] [-single_thread] [design]
[-subdesign module|design]
```

Description

Performs standalone mapping of single bit cells to multibit cells without performing any other optimizations.

The recommended multibit merge flow is to do multibit merge in `syn_map` stage. Set the `use_multibit_cells` attribute to `true` and other multibit attributes before `syn_map`, then `syn_map` will do multibit merge at the last stage of `syn_map`.

Genus multibit merge is timing driven.

To perform standalone multibit merge, use the `merge_to_multibit_cells` command.

You can set different multbit attributes before `merge_to_multibit_cells` command to control the multibit merging. For example, you can define `dont_merge_multibit` attribute on the single bit cells for which merging in not required.

After completion of `merge_to_multibit_cells` command, command `report_multibit_inferencing -not_merged_instance_list` can be used to report all the not merged single bit cells and their corresponding reason for not getting merged by `merge_to_multibit_cells` command. Not merged single bit cells might have multiple reasons for not getting merged but, `report_multibit_inferencing -not_merged_instance_list` command will report only one reason per not merged single bit cell.

`multibit_rejection_reason` attribute can be checked on not merged single bit cells to report the reason for not getting merged by `merge_to_multibit_cells` command.

> This command can only be run on a mapped design.

Options and Arguments

`-all`	Allows multibit mapping of combinational and sequential cells, and isolation cells and level-shifter cells.
design	Specifies the name of the design for which to perform multibit mapping.
`-combinational`	Allows multibit mapping of combinational cells.
`-iso_ls`	Allows multibit mapping of isolation cells and level-shifter cells.
`-logical`	Allows logical-only merging, thereby ignoring placement data even if it is available.
	By default, the tool performs placement-based multibit merging if the design is placed.
`-sequential`	Allows multibit mapping of sequential cells.
`-single_thread`	Performs single-threaded multibit mapping.
	By default, the tool performs super-threaded multibit mapping if the number of sequential instances is larger than 256.
`-subdesign` *module*|*design*	
	Allows multibit merging on specified module.

Example

```
set_db use_multibit_cells truemerge_to_multibit_cells -logical $DESIGN
```

Related Information

Affects this command:

report_multibit_inferencing

Affected by these attributes:

force_merge_seqs_into_multibit_cells
multibit_cells_from_different_busses
multibit_auto_exclude_registers_with_exceptions
multibit_allow_unused_bits
multibit_allow_async_phase_map
multibit_mapping_effort_level
multibit_rejection_reason
physical_aware_multibit_mapping
use_multibit_cells
use_multibit_combo_cells
use_multibit_seq_and_tristate_cells

remove_assigns_without_opt

```
remove_assigns_without_opt
[-buffer_or_inverter lib_cell]
[-skip_remove_assign_on_clock_path]
[-no_buffers_use_inverters]
[-allow_unloaded_buffers]
[-use_inverted_signal]
[-ignore_preserve_map_size_ok]
[-ignore_preserve_setting]
[-include_local_constant_assigns]
[-dont_respect_boundary_optimization]
[-preserve_dangling_nets]
[-dont_skip_unconstrained_paths]
[-skip_hierarchical_subdesigns] [-skip_ideal_nets]
[-ports ports] [-internal] [-verbose]
[-design {module|design}] [-clock_buffer_or_inverter lib_cell]
```

Description

Controls the aspects of the replacement of `assign` statements in the design with buffers or inverters without performing incremental optimization.

To avoid incremental optimization when removing assign statements, make sure that the `remove_assigns` attribute is set to `false`, and specify this command after synthesis.

> Make sure to uniquify the design before you run this command, otherwise the tool will issue an error message.

In the Physical flow, Genus will by default not remove the assign statements inserted by Innovus on constants, because inserting extra buffers to remove assigns on constants could lead to higher congestion.

To remove assigns on constants after the Physical flow, use the following command:

```
remove_assigns_without_opt -dont_skip_unconstrained_paths -design design
```

> The command will not remove assignment statements when this could lead to an NEQ.

Options and Arguments

`-allow_unloaded_buffers`	
	Prevents unloaded nets from being deleted and allows unloaded buffers to be added on these nets to fix the assigns in the netlist.
`-buffer_or_inverter` *lib_cell*	
	Specifies the buffer or inverter to be used to replace the `assign` statements. The specified cell must be part of a library that was loaded.
	If this option is omitted, Genus will determine which buffers or inverters to insert. If a particular library domain does not have any buffers or inverters, no buffers or inverters will be added in that domain. However, buffers or inverters will be added in other domains depending on availability.
`-skip_remove_assign_on_clock_path`	
	Skips remove assigns on clock path.

`-clock_buffer_or_inverter` *lib_cell*	
	Specifies the clock buffer or inverter for removal from clock path. The specified cell must be part of a library that was loaded.
`-design {`*design* `|` *module*`}`	
	Indicates that the specified command options apply to the specified design or module. If neither is specified, the options apply to the entire design.
`-dont_respect_boundary_optimization`	
	Allows Genus to disconnect the unloaded subports of a module, if the `boundary_opto` attribute on the module was set to `false`.
`-dont_skip_unconstrained_paths`	
	Specifies to remove assigns on unconstrained paths. These paths can be paths driven by a constant, false paths with assigns, or paths without any timing constraints.
	By default, Genus skips unconstrained paths.
`-ignore_preserve_map_size_ok`	
	Allows assign statements in the module to be removed if the `preserve` attribute on the module is set to `map_size_ok`.
`-ignore_preserve_setting`	
	Allows assign statements in the module to be removed independent of the setting of the `preserve` attribute on the module.
`-include_local_constant_assigns`	
	Allows to remove local constant driven assigns that have multiple drivers and do not drive hierarchical ports.
`-no_buffers_use_inverters`	
	Specifies to search for inverters and to use them in case the library or library domain does not have any buffers. If buffers exist they will be used, irrespective of whether the library has inverters or not.
`-ports` *ports*	Limits the replacement of `assign` statements to the specified ports.

`-preserve_dangling_nets`	
	Specifies to preserve a net that is driven by a buffer or inverter that is inserted when assign statements are replaced and if that net does not have a fanout in the next level of hierarchy and was not driven by a constant before adding the buffer or inverter.
`-skip_hierarchical_subdesigns`	
	Restricts the replacement of `assign` statements to one level only. By default, the replacement of `assign` statements is done recursively.
`-skip_ideal_nets`	Allows Genus to skip ideal nets from buffer insertion.
`-use_inverted_signal`	
	Allows the tool to use an inverter with inverted signal if the following conditions are met: • The assign statement is driven by a constant • The area of the inverter is smaller than the area of the buffer
`-internal`	Specifies that the remove assigns is called internally in the flow.
`-verbose`	Displays all messages during assign removal.

Examples

- The following example removes assigns on unconstrained ports:

  ```
  remove_assigns_without_opt -dont_skip_unconstrained_paths
  ```

- The following example removes assigns at port out1:

  ```
  remove_assigns_without_opt -port out1
  ```

Related Information

Affected by this attribute:	remove_assigns

remove_clock_reconvergence

```
remove_clock_reconvergence
[-clocks clock...]  [-effort {low|high}]
[-ports port_list] [design]...
```

Description

Optimizes the clock path by removing clock path reconvergence for all clocks. You can also remove the reconvergence for the specified clock only.

Options and Arguments

`-clocks` *clock*	Specifies the name of the clocks for which to remove the clock path reconvergence.
	By default, the command optimizes the networks for all clocks.
design	Specifies the design for which to perform the removal of clock reconvergence.
`-effort {low|high}`	
	Specifies the effort level during optimization.
	`low`—Performs clock reconvergence removal.
	`high`—Performs clock reconvergence removal and in addition fixes common clock inputs and constant inputs.
	Default: `low`
`-ports` *port_list*	Removes the reconvergence through the specified ports.

Examples

- The following command removes the re-convergence for clock clk1 and, in addition, fixes common clock inputs and constant inputs:

```
remove_clock_reconvergence -clocks [get_db clocks *clk1] -effort high
```

- The following command removes the re-convergence through the pcm_sclk and pcm_lrclk ports:

```
remove_clock_reconvergence -ports "[get_db ports *pcm_sclk] \
 [get_db ports *pcm_lrclk]"
```

syn_generic

```
syn_generic
[-physical][-create_floorplan][design]...[-estimate_flop_bits]
```

Description

Takes an elaborated and fully constrained design as input and synthesizes it into a netlist of generic gates by doing high-level RTL and datapath optimizations.

Main steps include:

- Optimization of datapath components (adders, multipliers, shifters, etc.) to implement arithmetic modules that meet the required performance goal while minimizing area and power. You can control the global trade-off between timing and area by using the `dp_analytical_opt` attribute. You can have more control over specific datapath transformations through datapath attributes. Refer Datapath Attributes for a list of them or use `get_db * -category dp` on the Genus prompt.
- Implementation of mux structures and choosing the best architecture (binary or one-hot) for them.
- Using enable conditions in the fanin of registers to implement clock-gating logic and thus reducing power. The main attribute to enable this feature is `lp_insert_clock_gating` (default is off). For details refer to the other related attributes in the `lp_cg` category.
- Apply retiming to improve overall maximum performance of the design. This technique is applied to all modules tagged with the retime attribute. Further control is possible via attributes of the retime category.

It is recommended to use the `check_design` command in advance to find major design problems like

unresolved modules and multi-driven or undriven nets or pins. Also, the command `check_timing_intent` helps to identify issues related to the timing constraints read into the tool.

If the `-physical` option is specified, Genus will also place the generic netlist and compute an initial congestion map based on actual global routing that enables the user to rate the quality of the given floorplan. Top paths are physically optimized, so timing bottlenecks of the design can be identified quickly. The resulting database can be used to analyze congestion and critical path placement even before mapping to the technology library. Use the `check_floorplan` command in advance to ensure the given floorplan data is of high quality.

The effort of this command is controlled via the `syn_generic_effort` attribute.
Use the `write_snapshot` command to capture relevant metrics after running syn_generic.

Large parts of generic optimization are either multi-threaded or executed via partition-based synthesis, distributed across multiple machines and/or CPUs. By increasing the number of CPUs the user can enable Genus to better balance the workload and improve runtime further.

Options and Arguments

`-create_floorplan`	Specifies to enable flow without DEF file, creates a square floorplan with density of 0.7.
design	Specifies the name of the design that will be synthesized.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is used. If multiple top-level designs exist, all are synthesized.
`-physical`	Instructs Genus to do physical-aware generic optimization, including global placement and routing of the design. If not specified, Genus will do logical optimization only.
	If the design attribute `physical_aware_mapping` is set to true, the tool does accurate long wire prediction that influences datapath structuring and cell selection for convergent timing closure.
	This option is available only with the Genus_Physical_Opt License Key.
	If the effort is low, the `-physical` option is ignored for `syn_generic` and a warning is issued that the physical option is ignored for low effort.
`-estimate_flop_bits`	Specifies to exit after optimization, sufficient to enable accurate sequential bit count estimation (for purposes such as DFT architecture).

Example

The following example runs physical-aware generic optimization on the top-level design in memory:

```
syn_generic -physical
```

Related Information

Performing Synthesis in *Genus User Guide.*

Genus Datapath Synthesis Guide

Retiming the design in the *Genus User Guide*

Affects these commands:

report_area
report_dp
report_gates
report_power
report_summary
report_timing

Related commands:

report_dp
syn_map
syn_opt

Affected by these attributes:

syn_generic_effort
dp_analytical_opt
lp_insert_clock_gating
retime

syn_map

```
syn_map
[-physical] [design]...
```

Description

Maps a design from generic gates to a technology library while optimizing for best performance, power and area. Genus evaluates multiple implementations for a given logic cone and chooses the one that meets timing constraints while minimizing area and power. After an initial mapping phase, further refinement is done by incrementally optimizing the netlist to recover area and power while maintaining timing.

Main steps include:

- Structuring and mapping of generic logic to meet specified goals. Certain features of optimization can be controlled via attributes. For a full list of these attributes, refer to the `opt` category using `get_db * -category opt`.
- Selective ungrouping of modules to improve optimization for best timing or area. This can be controlled via the `auto_ungroup` attribute.
- Optimizing for leakage and dynamic power. The effort spent on reducing leakage power is specified via the `design_power_effort` attribute. By default, Genus does not optimize for power.
- Mapping of single-bit to multi-bit library cells. By default this feature is turned off, but it can be enabled via the `use_multibit_cells` attribute.

The library cell set available to the mapper can be restricted by applying the `dont_use` attribute to library cell objects. It is recommended to only apply this attribute in case certain library elements are not allowed to be used under any circumstances and not as an overall guidance to the mapping engine.

If the `-physical` option is specified, Genus will perform physical-aware mapping and restructuring of logic to account for physical effects like long wires in the design. Placement locations of registers are maintained throughout mapping to allow for physical-aware multi-bit merging of flip flops and physical-aware scan insertion post `syn_map`. It is a prerequisite to run physical-aware mapping if `syn_generic` was run in physical mode too.

The effort to use during optimization is controlled by the `syn_map_effort` root attribute. Use the `write_snapshot` command to capture relevant metrics after running `syn_map`.

Large parts of mapping are either multi-threaded or executed via partition-based synthesis, distributed across multiple machines and/or CPUs. By increasing the number of CPUs, you can enable Genus to better balance the workload and improve runtime further.

Options and Arguments

design	Specifies one or multiple designs to be synthesized.
	If you omit the design name, the top-level design of the current directory of the design hierarchy is synthesized.
-physical	Instructs Genus to perform physical-aware mapping. If not specified, Genus will only do logical optimization.
	If the effort is low, the -physical option is ignored for syn_map and a warning is issued that the physical option is ignored for effort low. This will happen only when syn_gen also was in low_effort.
	This option can be used if and only if syn_gen -physical was done previously
	This option is only available with the Genus_Physical_Opt License Key.

Example

The following example runs physical-aware mapping on the top-level design in memory:

```
syn_map -physical
```

Related Information

Performing Synthesis in *Genus User Guide.*	
Related commands:	syn_generic
	syn_opt
	write_snapshot
	report_qor
	report_area
	report_gates
	report_power
	report_summary
	report_timing
	report_design_rules
Related attributes:	syn_map_effort
	auto_ungroup
	design_power_effort
	use_multibit_cells

syn_opt

```
syn_opt
 [-logical] [-spatial] [-incremental]
 [design]...
```

Description

Takes a mapped design as input and incrementally optimizes timing, area, and power.

When run with no arguments, other than design, syn_opt will do a pure logical optimization. This

version of logical-only optimization is OBSOLETE and will be removed in a future release, see 'syn_opt -logical'. Depending on the effort level specified by syn_opt_effort, it first optimizes worst and total negative slack, then applies certain transforms to recover area and power. The higher the effort, the more iterations of optimization steps are done.

When run with the [-logical] option, the design is transfered to Innovus as a service to perform logical optimization. This optimization does not do or use placement information. The result of this may be passed to Innovus or may be used for a subsequent 'syn_opt -spatial' call. This option may also be combined as a preoptimization to iSpatial 'syn_opt -logical -spatial'.

When run with the [-spatial] option, the design is transfered to Innovus as a service to perform "iSpatial" placement and optimization. The result is predictive of area and timing that will be seen if the result is further processed in Innovus implementation flows.

Options and Arguments

design	Specifies one or multiple designs that will be synthesized.
	If omitted, the top-level design of the current directory in the design hierarchy is used. If multiple designs exist, all of them are synthesized.
-spatial	Instructs Genus to run the spatial flow to get an accurate prediction of pre-CTS performance of the design. This option requires a full Innovus installation to be present in your search path.
-logical	Instructs Genus to run Innovus as a service to perform logic only optimization. This option may be used by itself or with "-spatial".
-incremental	Performs incremental optimization of the mapped gates for iSpatial flow.

Example

- Run a pure logical optimization using Innovus services

  ```
  syn_opt -logical
  ```

- Run iSpatial predictive flow

  ```
  syn_opt -spatial
  ```

- Run logical preoptimization before iSpatial predictive flow

  ```
  syn_opt -logical -spatial
  ```

Related Information

Affects these commands:

report_area
report_dp
report_gates
report_power
report_summary
report_timing

Related commands:

remove_assigns_without_opt

Affected by these attributes:

syn_opt_effort
iopt_force_constant_removal
iopt_sequential_resynthesis
iopt_sequential_resynthesis_min_effort

10

Flowkit

- check_flow
- create_flow
- create_flow_step
- define_feature
- delete_flow
- delete_flow_config
- delete_flow_step
- edit_flow
- end_flow
- get_feature
- get_flow_config
- is_feature
- is_flow
- read_flow
- reset_flow
- run_flow
- schedule_flow
- set_feature
- set_flow_config
- set_flowkit_read_db_args
- set_flowkit_write_db_args
- write_flow
- write_flow_template

check_flow

```
check_flow
[-fail string] [-flow string] [-proc string]
[-step string] [-from string] [-to string]
```

Description

Creates a single flow_step object and fills-in the required, and some optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

> Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Options and Arguments

`-fail` *string*	Reports a problem. This is not used for analysis.
	It can only be used from Tcl called as part of static checking. Calling from outside static checking results in error.
`-flow` *string*	Runs the specified flow (created with the `create_flow` command). The `-from` and `-to` options are based on the flow steps.
	Default: `flow_step_order`
`-from` *string*	Specifies the name of the step to start from. Specify a step in the flow.
	By default, the flow starts from the first step that has not been run, or the first step if all steps have been run previously.
`-proc` *string*	Checks the specified procedure instead of analyzing a flow step.
`-step` *string*	Checks just the specified step.
	This option takes priority over the -`from` and -`to` options.
`-to` *string*	Specifies the name of the last step to be executed.
	By default, runs to the end of the last step in the flow. Specify a step in the flow, later than the `-from` step (if any).

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

create_flow

```
create_flow -name name
[-tool tool] [-tool_options options]
[-owner string] [-skip_metric] [steps]
```

Description

Creates a new, named flow object. `run_flow`, then allows the execution of the named flow (in addition to its existing functionality).

Options and Arguments

`-name` *name*	Specifies the name of the flow object to create.
`-owner` *owner*	Specifies the owner of the flow.
`-skip_metric`	Specifies not to create a metric snapshot for the flow.
steps	Specifies the Tcl flow steps.
`-tool` *tool*	Specifies the name of the tool with which to run this flow.
	This option allows you to create flows for other tools than the one currently running.
	By default, the flow will be run in the current tool.
`-tool_options` *options*	
	Specifies the tool options to use when launching this flow.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

create_flow_step

```
create_flow_step
-name name  body [-copy step] [-owner string]
[-begin beginTcl] [-check checkTcl] [-end endTcl]
[-categories string] [-description string] [-write_db]
[-skip_metric] [-exclude_time_metric]
```

Description

Creates a single `flow_step` object and fills in the required and optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify its attributes, or delete the step and recreate it.

> Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Options and Arguments

`-begin` *beginTcl*	Specifies a block of Tcl to run at the start of the step.
body	Specifies a block of Tcl that is run as the main action of the step.
`-categories` *string*	
	Defines the metric categories to calculate the flow step.
`-check` *checkTcl*	Specifies a block of Tcl to run step-specific static checks.
`-copy` *step*	Copies the specified step to this step.
`-end` *endTcl*	Specifies a block of Tcl to run at the end of the step.
`-exclude_time_metric`	
	Prevents that CPU time and wall time for this step are included in the parent steps.
`-name` *name*	Specifies the name of the step. The name can consist of alphanumeric characters and underscore only, and may not start with a digit.
	The following names have special meanings and cannot be used for step names:
	• next
	• end
`-owner` *owner*	Specifies the owner of the flow step.
`-skip_metric`	Skips generating metrics when the step is complete. By default, every step saves the design and runs metrics when completed.
`-description` *string*	
	Specifies the purpose for the flow step.
`-write_db`	Automatically saves a database when the flow step is complete.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

define_feature

```
define_feature
feature
[-obj string] [-group string]
[-description string]
[-type {boolean|string|enum}]
[-possible_values string]
[-default string] [-required]
```

Description

Defines a flow feature.

Options and Arguments

`-default` *string*	Specifies the default value of the feature.
`-description` *string*	
	Specifies the detailed description of the flow feature. If it matches multiple features, then these features are listed along with a short description for each.
	If it matches exactly one feature, a full description of this feature is printed. Matching no features results in an error.
feature	Specifies the name of the feature.
`-group` *string*	Outputs the exclusive group that contains features.
`-obj` *string*	Specifies the flow or flow_step for a flow-specific feature. The features can be limited to a flow/step, or be global and usable in all the flows or steps.
	Both the enabled and disabled code are included, with conditionals based on a new user attribute.
`-possible_values` *string*	
	Specifies the list of values that are valid for the features.
`-required`	Indicates that this feature is mandatory.
`-type {boolean|string|enum}`	
	Specifies the type of feature value. The type can be boolean, a string, or an enumerated type

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

Related commands:

get_feature

set_feature

delete_flow

```
delete_flow
-name name
```

Description

Deletes the flow that is cached in the database.

Options and Arguments

`-name` *name*	Specifies the name of the flow to be deleted.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

delete_flow_config

```
delete_flow_config
 string
```

Description

Deletes the flow configuration for the specified key.

Options and Arguments

string	Specifies to name of a key in the data structure.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

delete_flow_step

```
delete_flow_step
-name name
```

Description

Deletes an existing `flow_step` object. If the step is present in the flow order, it is removed and an information message is displayed.

Options and Arguments

`-name` *name*	Specifies the name of the step to be deleted.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

edit_flow

```
edit_flow [-append string] [-prepend string]
[-reset] [-after string] [-before string]
[-enabled_feature string]
[-owner string] [-unique]
```

Description

Edits the flow or `flow_step` objects.

Options and Arguments

`-after` *string*	Specifies to add the flow or `flow_step` objects at the end of the given flow or `flow_step` objects.
`-append` *string*	Specifies to append the flow or `flow_step` objects at the start or end of the given flow or `flow_step` objects.
`-before` *string*	Specifies to add the flow or `flow_step` objects at the start of the given flow or `flow_step` objects.
`-enabled_feature` *string*	
	Specifies to only apply the `edit_flow` if the listed features are enabled.
`-owner` *owner*	Specifies who edited the flow or flow step.
`-prepend` *string*	Specifies to prepend the flow or `flow_step` objects at the start or end of the given flow or `flow_step` objects.
`-reset`	Specifies to reset the customization of flow or `flow_step` objects at the start or end of the given flow or `flow_step` objects.
`-unique`	Specifies not to add the given flow or `flow_step` objects if they are already present on the given flow or `flow_step` objects..

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

end_flow

```
end_flow [flow | flow_step]
```

Description

Accepts a flow or a flow step name, and immediately ends its execution and returns normally.

Options and Arguments

`flow	flow_step`	Specifies the name of the `flow` or `flow_step` to end and skip.
	Calling `end_flow` command outside of a `flow_step` inside `run_flow` (that is, `init_flow plugin`, `flow_header_tcl` or as a plain command) will result in an error.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

get_feature

```
get_feature
[string]
[-obj {flow | flow_step}]
```

Description

Returns the value of the specified flow feature.

Options and Arguments

`-obj {`*`flow`* `|` *`flow_step`*`}`	
	Specifies to name of the flow or flow step to which the feature applies.
	The features can be limited to a flow/step, or be global and usable in all the flows/steps.
`string`	Specifies name of the feature.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

get_flow_config

```
get_flow_config
string... [-quiet]
```

Description

Returns the value for a key. If you specify multiple strings, the command interprets it as a set of keys that describe how to navigate the hierarchy of a data structure.

Options and Arguments

`-quiet`	Suppresses error messages if the specified key does not exist.
`string`	Specifies to name of a key in the data structure.

Examples

- The following command retrieves the value of the design_process_node.

  ```
  get_flow_config design_process_node
  ```

- The following command retrieves the library files of a library in a library set.

  ```
  get_flow_config library_sets slow_0v990 library_files
  ```

  This corresponds to looking in a data structure similar to:

  ```
  library_sets
   slow_0v990
    library_files {fill.lib pllclk_slow.lib}
    socv_files ...
   fast_0v792
    library_files ...
    socv_files ....
  ```

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

Related command:

set_flow_config

is_feature

```
is_feature
[string] [-obj {flow | flow_step}]
```

Description

Checks if the specified flow feature exists.

Options and Arguments

-obj {*flow* | *flow_step*}	
	Specifies to name of the flow or flow step to which the feature applies.
	The features can be limited to a flow/step, or be global and usable in all the flows/steps.
string	Specifies name of the feature.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

is_flow

```
is_flow [-after string] [-before string]
[-inside string]
[-hierarchical][-quiet]
```

Description

Queries if the current execution matches the specified condition. This helps you add actions depending on the context.

Options and Arguments

`-after` *string*	Specifies the current execution is after the flow or flow_step objects.
`-before` *string*	Specifies the current execution is before the flow or flow_step objects.
`-hierarchical`	Allows a search to go beyond the current flow boundary. This is done by flattening the entire flow execution graph into a linear chain of events (including all flow customization or scheduling). This enables to form more complex conditional statements especially when considering flows which consist of multiple sub-flows. The `-hierarchical` option allows before/after checks to be done on this type of structure.
`-inside` *string*	Specifies the current execution is within the flow or flow_step objects.
`-quiet`	Ensures that errors are reported for missing flow or flow_step objects.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

read_flow

`read_flow` *string* `[-no_check]`

Description

Loads the content for the current flow and debugs the flow in the tool. The command accepts single argument as the name of a Tcl or yaml file.

Options and Arguments

`-no_check`	Skips the error checks (`yamllint` and `check_tcl`) prior to running. By default, the checks are run.
`string`	Specifies the flow script file to read.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

reset_flow

```
reset_flow
[-all] [-flows] [-steps] [-features] [-edits] [-history]
```

Description

Resets the content for the current flow by clearing any existing flows, steps, or features.

Options and Arguments

`-all`	Specifies to reset all flows, steps, features, edits and history.
`-edits`	Specifies to delete all flow edits.
`-features`	Specifies to delete all features and values.
`-flows`	Specifies to delete all flow objects.
`-history`	Specifies to reset all attributes storing flow history state.
`-steps`	Specifies to delete all flow step objects.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

Related commands:

check_flow
create_flow_step
edit_flow
edit_flow
report_flow
run_flow
schedule_flow
write_flow

run_flow

```
run_flow
[-directory string]
[-yaml string] [-flow string]
[-no_check] [-no_summary] [-reset]
[-step string] [-from string] [-to string] [-predict {summary|tcl}]
```

Description

Runs one or more flow steps by default, in the order specified in a single flow.

If more than one flow is defined, the `run_flow` command must be run with the `-flow` option. However, if `-step` option is used, that step does not have to be part of a flow.

Options and Arguments

`-directory` *string*	Specifies the working directory in which to run the flow. If a directory is not specified, the current directory is used.
	Any relative paths in attributes such as `flow_report_directory` are relative to this directory.
`-flow` *string*	Runs the specified flow (created with the `create_flow` command). The `-from` and `-to` options are based on the flow steps.
	Default: `flow_step_order`
`-from` *string*	Specifies the name of the step to start from accepting a full dotted canonical path to a flow step object.
	By default, the flow starts from the step immediately after the last successfully run step (determined by examining the value of the `flow_history` root attribute), or the first step if that attribute is not set.
`-no_check`	Skips the error checks prior to running. By default, the checks are run.
`-no_summary`	Skips printing the full summary. By default, the summary is printed and mailed if the `flow_mail_to` root attribute is set.
`-predict {summary|tcl}`	
	Shows a prediction of what will run.
`-reset`	Before running the command, it clears the flow history before running. After this, the flow starts from the first step if no explicit `-from` step is specified.
`-step` *string*	Runs just the specified step.
	This option takes priority over the -`from` and -`to` options.
`-to` *string*	Specifies the name of the last step to run to (inclusive; in other words, the named step is executed) accepting a full dotted canonical path to a flow step object.
	By default, runs to the end of the last step in the flow. Specify a step in the flow, later than the `-from` step (if any).

`-yaml` *string*	Specifies the `yaml` file to source before running the flow.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

schedule_flow

```
schedule_flow -flow string
[-branch string] [-db string] [-defer]
[-directory string]
[-no_db] [-no_sync | -sync]
```

Description

Schedules a flow to be run after the current flow. The details of scheduled flows are stored in the `flow_schedule` attribute.

Options and Arguments

`-branch` *string*	Specifies the branch for the new (hierarchical) flow.
`-db` *string*	Specifies the starting database of the flow.
	If not supplied, the final database of the preceding flow (the one currently running) is used.
`-defer`	Specifies not to run the scheduled flow until requested.
`-directory` *string*	Specifies the working directory to use for the flow.
	By default, the current directory is used.
`-flow` *string*	Specifies the flow to be scheduled.
`-no_db`	When specified, flowtool does not use the database saved at the end of the step as the starting database for this flow.
`-no_sync`	When specified, flowtool does not wait for this flow to complete before continuing.
`-sync`	When specified, flowtool waits for all flows in the current branch to complete before starting.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

set_feature

```
set_feature
-value string [name]
[-obj {flow | flow_step}]
```

Description

Specifies the value of the specified flow feature.

Options and Arguments

`-obj {`*`flow	flow_step`*`}`	
	Specifies to name of the flow or flow step to which the feature applies.
	The features can be limited to a flow/step, or be global and usable in all the flows/steps.
`name`	Specifies name of the feature.
`-value` *`string`*	Specifies the value of the feature.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

Related commands:

define_feature
get_feature

set_flow_config

`set_flow_config`
`string string...`

Description

Specifies the value for a key. If you specify multiple strings, the command interprets it as a set of

keys with a single value, where the keys describe how to navigate the hierarchy of the data structure.

You can use these commands to set up the data structure to configure a flow.

Options and Arguments

string	Specifies to name of a key in the data structure.
string	The last string specified specifies the value for a key.

Examples

- The following command modifies the design_process_node.

  ```
  set_flow_config design_process_node 7
  ```

- The following command operates on an individual library set.

  ```
  set_flow_config library_sets slow_0v990 library_files {fill.lib pllclk_slow.lib}
  ```

  This corresponds to operating on a data structure similar to:

  ```
  library_sets
   slow_0v990
    library_files {fill.lib pllclk_slow.lib}
    socv_files ...
   fast_0v792
    library_files ...
    socv_files ....
  ```

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

Related command:

get_flow_config

set_flowkit_read_db_args

`set_flowkit_write_db_args`

Description

Sets the options for the `read_db` command when running the flow.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

set_flowkit_write_db_args

`set_flowkit_write_db_args`

Description

Allows you to save databases through flow-kit in the Open Access format and specify the library or cell or view information. The `set_flowkit_write_db_args` command can be used to extend the options used to save a design on flow-steps, which are marked with `skip_db false`. This command needs to be placed inside the flow-step `body_tcl` of interest since it only affects that flow-step and resets back to default state when that step completes. The format of this command is similar to what would be used for `write_design`:

```
create_flow_step -name save {set_flowkit_write_db_args -rc -lib2ldb}
```

This saves a database named save, which uses the -`rc` and -`lib2ldb` options for `write_db`. Previous and subsequent flow-steps use default `write_design` options unless overridden using the `set_flowkit_write_db_args` command.

> This command supports all the parameters supported by `write_db`.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

"Overview" chapter in the *Innovus Common UI Migration Guide*

Related command:

write_db

write_flow

```
write_flow
[-out_file string]
[-include_state] [-directory string]
[-flow_tcl] [-flow_yaml] [-setup_yaml]
```

Description

Generates a Tcl script that may be sourced to reproduce the current flow. The script includes commands to:

- Set the value of all top-level attributes with non-default values to their current values
- Recreate all flow step objects with their current attributes, aside from those relating to run history (specifically, "`run_count`" and "`status`").

Sourcing this script reproduces (but not runs) the current flow in a fresh session, and forms a user-readable and editable record of the flow (compared to the flow data saved in the DB, which should not be read or edited directly).

Options and Arguments

`-directory` *string*	Specifies the directory in which all files must be written.
`-flow_tcl`	Writes only the `flow.tcl` file in the directory.
`-flow_yaml`	Writes only the `flow.yaml` file in the directory.
`-include_state`	Writes out the current flow state. This can be used to restore the flow.
`-out_file` *string*	Specifies the name of the output file to generate.
`-setup_yaml`	Writes only the `setup.yaml` file in the directory.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

write_flow_template

```
write_flow_template [-enable_feature string]
[-optional_feature string]
[-tools string] [-directory string]
[-list] [-describe string]
[-type string] [-version string]
[-template_file string] [-tech_steps string]
[-ip_steps string] [-user_steps string]
```

Description

Creates a set of template TCL files to be used as the basis for a flow.

Options and Arguments

`-describe` *string*	Specifies a detailed description of the available flow features of a template.
`-directory` *string*	Specifies the directory for saving the resulting flow.
`-enable_feature` *string*	
	Specifies the list of features required in the flow.
`-ip_steps` *string*	Specifies the base IP steps to overlay on the flow.
`-list`	Specifies the list of the available flow templates.
`-optional_feature` *string*	
	Specifies the list of features that are optional in the flow.
`-tech_steps` *string*	Specifies the basic technology steps to overlay on the flow.
`-template_file` *string*	
	Specifies the flow template file.
`-tools` *string*	Specifies the list of tools whose scripts should be included in the flow.
`-type` *string*	Specifies the type of the flow to use
`-user_steps` *string*	Specifies the base design steps to overlay on the flow.
`-version` *string*	Specifies the version of the flow to use.

Related Information

"Flow" chapter in the *Innovus Common UI Migration Guide*

11

GUI Text

Most of the GUI commands are available in both the new GUI and legacy GUI. A note was added for commands or options that only apply to the legacy GUI.

- General GUI Text Commands
- Schematic Viewer GUI Text Commands
- Layout Viewer GUI Text Commands

- deselect_obj
- gui_clear_pv
- gui_clear_sv
- gui_configure_pi_schematic
- gui_hide
- gui_highlight_hier_instances_pv
- gui_highlight_pv
- gui_highlight_sv
- gui_legend
- gui_open_schematic
- gui_pv_airline_add
- gui_pv_airline_delete
- gui_pv_airline_display
- gui_pv_airline_raw_add
- gui_pv_connectivity_airlines
- gui_pv_display_collection
- gui_pv_draw_net
- gui_pv_selection
- gui_pv_steiner
- gui_raise
- gui_redo_pv
- gui_redraw_pv
- gui_reset
- gui_select_highlighted_pv
- gui_selection
- gui_show
- gui_snapshot_pv
- gui_snapshot_sv
- gui_sv_get_instance
- gui_sv_load
- gui_undo_pv
- gui_update
- gui_zoom_fit_pv
- gui_zoom_fit_sv
- gui_zoom_pv
- select_obj

deselect_obj

```
deselect_obj [-all]
{fpobject | region | blockage | hnet | port | pin | hpin | inst | hinst}...
```

Description

Deselects the specified objects in the Layout Viewer.

Options and Arguments

`-all`	Specifies to deselect all objects.
{fpobject | region | blockage | hnet | port | pin | hpin | inst | hinst}	
	Specifies the objects to deselect.

Related Information

Related command:

select_obj

gui_clear_pv

```
gui_clear_pv [-airline] [-highlight] [-selection]
[-shape] [-index integer]
[{inst|hinst|pin|hpin|port|hnet|extraction_grid|
    gcell|pcell|blockage|region|row}...]
```

Description

Clears airlines or highlighting from the Layout Viewer, or deselects objects in the Layout Viewer. When no options are specified, everything is cleared.

Options and Arguments

`-airline`	Clears only airlines.
`-highlight`	Clears only highlighting.
`-index integer`	Specifies the index at which the object needs to be cleared.
	min=1, max=64
{*blockage* | *extraction_grid* | *gcell* | *pcell* | *inst* | *hinst* | *hnet* | *pin* | *hpin* | *port* | *region* | *row*}	
	Specifies the object to clear if it is in the scope of the current Layout Viewer.
`-selection`	Deselects all selected objects.
`-shape`	Clears only shapes.

Related Information

Related command:

gui_redraw_pv

gui_clear_sv

```
gui_clear_sv [-index integer]
[{inst|hinst|pin|hpin|port|hport|hnet}...]
```

Description

Clears highlighting and selection lists in the Schematic Viewer.

Options and Arguments

{*inst* | *hinst* | *hnet* | *pin* | *hpin* | *port* | *hport*}	
	Specifies the object to clear if it is in the scope of the current Schematic Viewer.
`-index integer`	Specifies the index at which the object needs to be cleared.
	min=1, max=64

Related Information

Related command:

- gui_highlight_sv

gui_configure_pi_schematic

```
gui_configure_pi_schematic -schematic_id string [-type {instance|pin|port}] -remove
-callback_report string
```

Description

Clears highlighting and selection lists in the Schematic Viewer.

Configure Power Intent Schematic Viewer. You can register a callback process on vdir object of type `{instance|pin|hpin|port|net|hnet}`. On right click on that vdir object, the registered process can be seen as a context menu option. Clicking on that process option will execute the process.

Options and Arguments

`-callback_report string`	
	Specifies to register callback tcl process to display and execute the context menu request. Process template: `proc abc {sch_id item}. schematic_id: schematic id, Item: vdir` of object on which Custom Menu is requested.
`-type {instance| pin|port}`	
	Specifies the object to clear if it is in the scope of the current Schematic Viewer.
`-remove`	Specifies to remove registered callback.
`-schematic_id string`	
	Specifies the Schematic ID.

Related Information

Related command:

- gui_highlight_sv

gui_hide

```
gui_hide
```

Description

Hides the GUI. Type the `gui_show` command to re-display the GUI.

Related Information

Related commands:

- gui_raise
- gui_show

gui_highlight_hier_instances_pv

```
gui_highlight_hier_instances_pv [-threshold integer]
[-depth integer] [-under inst|hinst] [-instances instance_list] [-fromgui] [design]
```

Description

Highlights instances in the specified hierarchical instance in the Layout Viewer.

Options and Arguments

`-depth integer`	Specifies the search depth.
design	Specifies the design containing the instances to be highlighted.
`-fromgui`	Indicates whether the call comes from the GUI.
	This option is not available in the legacy GUI.
`-instances instance_list`	
	Specifies the instances to be highlighted.
`-threshold integer`	
	Specifies the threshold of the hierarchical instance count.
`-under inst|hinst`	Specifies the parent hierarchical instance that contains the instances to be highlighted.
	Options `-under` and `-instances` are mutually exclusive.

gui_highlight_pv

```
gui_highlight_pv [-color color] [-auto_color] [-collect] [-append]
[-group name] [-pattern {cross|none|solid}] [-zoom] {blockage | extraction_grid | gcell | pcell | region | row | inst | hinst | pin | hpin | port | hnet}... [-index integer]
```

Description

Highlights objects in the Layout Viewer.

Options and Arguments

`-append`	Appends the object to highlight.
`-auto_color`	Specifies object color automatically.
{*blockage* | *extraction_grid* | *gcell* | *pcell*| *inst* | *hinst* | *hnet* | *pin* | *hpin* | *port* | *region* | *row*}	
	Specifies the object to highlight if it is in the scope of the current Layout Viewer.
`-collect`	Collects objects to highlight
`-color` *color*	Specifies the color for highlighting.
	Possible values: {red|blue|green|yellow|magenta|cyan|lightpink|purple|teal|olive|plum|navy|pink|lime|orange|brown|lightblue|gold|chocolate|lightgreen|maroon|sa
	Default: red
`-group` *name*	Specifies the group name for the object.
`-index` *integer*	Specifies the index of the object in the Physical Viewer to which to apply the command.
	min=1, max=64
`-pattern {cross | none | solid}`	
	Specifies the stipple pattern.
	Default: `none`
`-zoom`	Zooms the highlighted objects.

gui_highlight_sv

```
gui_highlight_sv [-append] [-color color]
[-from {port|pin|hpin}] [-to {port|pin|hpin}] [-zoomto]
{port|hport|pin|hpin|hnet|inst|hinst}
```

Description

Highlights the specified object in the Schematic Viewer.

Options and Arguments

`-append`	Highlights the specified object, retaining existing highlighted objects.
	If you omit this option, all existing highlighting is cleared and only the specified object in highlighted by default
`-auto_color`	Specifies object color automatically.
`-color` *color*	Specifies the color for highlighting an object.
`-from {`*port*`|`*pin*`|`*hpin*`}`	
	Specifies the starting point from where the object needs to be highlighted.
`-index` *integer*	Specifies the index of the object in the Physical Viewer to which to apply the command.
	min=1, max=64
`-to {`*port*`|`*pin*`|`*hpin*`}`	
	Specifies the end point from where the object needs to be highlighted.
{port|hport|pin|hpin|hnet|inst|hinst}	
	Specifies the object type to be highlighted.
`-zoomto`	Zooms the highlighted objects.

Examples

- The following TCL procedure highlights inverters in the Schematic Viewer

```
# highlight inverters
proc highlight_inverters {idir type odir} {
 # only proceed if no object under cursor
 if {$type != "none"} return
 # clear any highlight and selection
 gui_clear_sv
 _find_inverters $idir
}

proc _find_inverters {idir} {
 # search for inverters
 foreach inst [vfind $idir -maxdepth 2 -inst *comb/*] {
  if {[get_db inverter $inst] == "true"} {
   gui_highlight_sv $inst -append -color green
  }
 }
}
```

- The following example highlights the falling clock edge flip-flops

```
# highlight falling clock edge flip-flops
proc highlight_falling_edge_ff {idir type odir} {
 # only proceed if no object under cursor
 if {$type != "none"} return
 # clear any highlight and selection
 gui_clear_sv
 _find_falling_edge_ff $idir
}

proc _find_falling_edge_ff {idir} {
 # search for sequential gates with falling clock edge
 foreach inst [vfind $idir -maxdepth 2 -inst *seq/*] {
  set libcell [get_db libcell $inst]
  if {$libcell == ""} continue
  if {[get_db flop $libcell] == "true"} {
   foreach libpin [vfind $libcell -libpin *] {
    if {[get_db output $libpin] == "true"} {
     foreach arc [get_db incoming_timing_arcs $libpin] {
      set liberty [get_db liberty_attributes $arc]
      if {[lsearch $liberty falling_edge] != -1} {
       gui_highlight_sv $inst -append
      }
     }
    }
   }

  }
 }
}
```

gui_legend

`gui_legend -title` *`titlename`* `[-cancel` *`string`*`] [-physical]` *`string`*

Description

Creates a GUI legend. This is useful if you need a simple legend to color code information.

You can use this command with the Layout Viewer and Schematic Viewer.

Options and Arguments

`-cancel` *string*	Specifies a Tcl proc that can be run when the Close button is pressed.
`-physical`	Specifies to show the legend for *Layout Viewer*.
string	Specifies the data set to display in the legend.
	The format should be a list of 'color note' pairs, where possible colors are: {red|blue|green|yellow|magenta|cyan|lightpink|purple|teal|olive|plum|navy|pink|lime|orange|maroon|lightblue|gold|choco late|lightgreen|brown|salmon|chartreuse|darkcyan|royalblue|darkgreen|tomato|violet|wheat|deepskyblue|darkorange|dark red|white}
`-title` *titlename*	Specifies the legend dialog title.

Example

The following commands show how to create a legend. The first set of commands create the data set for the legend:

```
set entry1 [list red   "This is entry 1"]

set entry2 [list green "This is entry 2"]

set entry3 [list blue  "This is entry 3"]

set data [list $entry1 $entry2 $entry3]
```

The next command specifies the TCl procedure to run when the Close button is pressed:

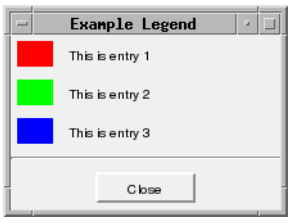
```
proc foo {} {     puts "Legend cancel"}
```

The last command specifies the data set and title to display in the legend, and the procedure to run when the Close button is pressed:

```
gui_legend -title "Legend Example" -cancel foo $data
```

The following figure shows the resulting GUI legend:

Figure -1 GUI Legend



gui_open_schematic

```
gui_open_schematic [-append] [-name string] {inst|hinst|net|hnet|pin|hpin|port|hport|hpin_bus|port_bus|hport_bus}
```

Description

Specifies to display the specified object into a new schematic cone viewer.

This command is the text equivalent for Open in 'Schematic Viewer (cone)' command in the Schematic Viewer, which is available when an object is selected.

Options and Arguments

`-append`	Appends the new object to the last open cone schematic viewer or to the specified cone schematic viewer by `-name` option.
{*port_bus* | *hport_bus* | *hpin_bus* | *inst* | *hinst* | *net* | *hnet* | *pin* | *hpin* | *port* | *hport*}	
	Specifies the object to be added in the schematic view. You can also use `get_db` selected to pass the selected object.
`-name` *string*	Specifies a unique window name for the schematic view to be opened.
	Data_type: `string`

gui_pv_airline_add

```
gui_pv_airline_add [-name name] [-id integer]
-from {inst|hinst|pin|hpin|port|pinstance} -to {inst|hinst|pin|hpin|port|pinstance} [-color color] [-glyph {arrow|circle|none}]
[-nodisplay] [-width integer]
```

Description

Adds the specified airline between the two objects in the Layout Viewer.

Options and Arguments

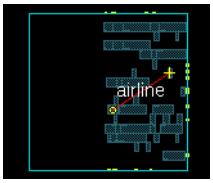
`-color` *color*	Specifies the airline color.
	The color can have following values:
	`blue, brown, cyan, green, magenta, orange, pink, purple, red, yellow, white.`
	Default: `blue`
`-from` {*inst*|*hinst*|*pin*|*hpin*|*port*|*pinstance*}	
	Specifies the from object.
`-id` *integer*	In case of multiple Layout Viewers, specifies the ID of the Physical Viewer to which to apply the command.
	This option is only available in the legacy GUI.
`-glyph {arrow | circle | none}`	
	Specifies the glyph type.
	This option is not available in the legacy GUI.
`-name` *name*	Specifies the airline name.
`-nodisplay`	Specifies not to display the airline.
`-to` {*inst*|*hinst*|*pin*|*hpin*|*port*|*pinstance*}	
	Draws an airline from the center of the `from` object to the center of the `to` object.
`-width` *integer*	Specifies the airline width.

Example

The following command creates a `red` airline named `test` from the `g442` port to the `g412` port, as shown in the figure below.

```
gui_pv_airline_add -from g442 -to g412 -color red -name test
```

Figure -2 Specified Airline in the Layout Viewer



Related Information

Related command:

gui_redraw_pv

gui_pv_airline_delete

`gui_pv_airline_delete` *`name`*

Description

Deletes airlines from the Layout Viewer with the specified *`name`*.

Options and Arguments

`name`	Specifies the airline name to be deleted.
	The airline name must have been added using the gui_pv_airline_add command.

Example

The following command removes the airline `test` from the Layout Viewer:

```
genus:root:> gui_pv_airline_delete test
```

Related Information

Related command:

gui_redraw_pv

gui_pv_airline_display

`gui_pv_airline_display` *`name`*

Description

Displays the specified airline named *`name`* in the Layout Viewer.

Options and Arguments

name	Specifies the name of the airline which is to be displayed.
	The airline name must have been added using the gui_pv_airline_add command.

Example

The following command displays the airline `test` in the Layout Viewer:

```
genus:root:> gui_pv_airline_display test
```

Related Information

Related command:

gui_redraw_pv

gui_pv_airline_raw_add

```
gui_pv_airline_raw_add [-name name]
[-color color] [-label label] [-width integer]
[-nodisplay] [-glyph {arrow | circle | none}] -fx float -fy float -tx float -ty float
```

Description

Creates an airline in the Layout Viewer between two points specified by their coordinates.

Options and Arguments

`-color` *color*	Specifies the airline color.
	The color can have following values:
	`blue, brown, cyan, green, magenta, orange, pink, purple, red, yellow, white.`
	Default: `blue`
`-fx` *float*	Specifies the x coordinate of the from object.
`-fy` *float*	Specifies the y coordinate of the from object.
`-glyph {arrow | circle | none}`	
	Specifies the glyph type.
`-label` *label*	Specifies a label to place at the center of the airline.
`-name` *name*	Specifies the airline name.
`-nodisplay`	Specifies not to display the airline.
`-tx` *float*	Specifies the x coordinate of the to object.
`-ty` *float*	Specifies the y coordinate of the to object.
`-width` *integer*	Specifies the width of the airline.

Related Information

Related command:

gui_redraw_pv

gui_pv_connectivity_airlines

```
gui_pv_connectivity_airlines
[-append] {inst | port }...
```

Description

Shows the connectivity airlines from the specified instance or port to the instances and ports it is connected to.

Options and Arguments

`-append`	Adds the new airlines to the existing ones.
{*inst* | *port*}	
	Specifies the name of the instance or port from where to draw the connectivity airlines.

Related Information

Related command:

- gui_redraw_pv

gui_pv_display_collection

```
gui_pv_display_collection [-group group]
```

Description

Displays a collection of objects in the Layout Viewer.

Options and Arguments

`-group` *group*	Specifies the name of the group to be displayed.
	A group can be created and appended by any `gui_pv_xx` command that has the `-collect` and `-group` options.

Related Information

Related commands:

- gui_pv_draw_net
- gui_redraw_pv

gui_pv_draw_net

```
gui_pv_draw_net [-delete] [hnet...]
```

Description

Adds or removes the route drawing for the specified net.

> The command only works for eGR (HUM) routed nets. If there are no eGR routes, nothing happens.

This command is not available in the legacy GUI.

Options and Arguments

`-delete`	Removes the net from the route drawing.
hnet	Specifies the net(s) for which to add the route drawing.

gui_pv_selection

`gui_pv_selection`

Description

Returns the list of selected objects in the Layout Viewer.

You can select multiple objects by pressing the Shift key while drawing a region with the left mouse button. All objects overlapping that region are selected.

Related Information

Related command:

gui_redraw_pv

gui_pv_steiner

`gui_pv_steiner`
`[-icolor string][-ocolor string]`
`{inst|hinst|pin|hpin|port}...`

Description

Shows the Steiner routing from the specified object. For output pins show the route. For input pins the route for the driver is shown. For instances both routes for output pins and routes for the driver of input pins are shown.

Options and Arguments

`-icolor string`	Species the input pin color for the layer.
	Default: `green`.
`-ocolor string`	Species the output pin color for the layer.
	Default: `yellow`.
`{inst|hinst|pin|hpin|port}`	
	Specifies the name of the instance or port or pin from where to draw the Steiner routing.

Related Information

Related command:

gui_redraw_pv

gui_raise

```
gui_raise
```

Description

Raises and keeps the GUI main window on top of all other windows.

It can be also used to reopen the GUI main window hidden by `gui_hide` command or by clicking "Minimize Window" icon at the top-right corner. The functionality is very similar to `gui_show` command. The minor difference is that the `gui_raise` command reopens the Schematic Main window (Design Browser : "Schematic View" -> "In Main" menu) as well as the GUI Main window.

Related Information

Related commands:

gui_hide
gui_open_schematic
gui_show

gui_redo_pv

```
gui_redo_pv [-win string] [-fromgui]
```

Description

Specifies to redo last floorplan edit command.

Options and Arguments

`-fromgui`	Indicates whether the call comes from the GUI.
`-win string`	Specifies the name of the window which needs to be redrawn.

gui_redraw_pv

```
gui_redraw_pv
```

Description

Redraws the contents of the Layout Viewer.

gui_reset

```
gui_reset
```

Description

Resets the busy indicator if it remains busy.

The busy indicator can remain red if the script that set the busy indicator is broken and therefore the busy indicator does not get reset or cleared.

gui_select_highlighted_pv

```
gui_select_highlighted_pv
```

Description

Specifies to select the highlighted objects in the Physical Viewer.

gui_selection

```
gui_selection [-reset] [-append]
[-quiet] [{inst|hinst|hnet|pin|hpin|port|hport}]
```

Description

Returns the selection list of the currently selected objects in the GUI. The list consists of the object path for instance, net, pin, and port objects. The list can contain multiple objects.

Options and Arguments

`-append`	Appends to the selection list.
`{port|hport|pin|hpin|hnet|inst|hinst}`	
	Specifies the object type to be highlighted.
`-quiet`	Controls print statements on the shell.Suppresses all the generated messages.
`-reset`	Resets the selection list.

Example

The following example returns a combinational instance:

```
genus:root:> gui_selection

inst:mul_clk/g14
```

gui_show

```
gui_show
```

Description

Displays the GUI main window if `gui_hide` command was used to hide it or for the first time display of GUI when the tool is launched.

If you run the `gui_show` command when the Genus main window is already displayed, the tool retains the current zoom level and displays the main window without refreshing it.

You can run the gui_hide command to hide the Genus main window.

The functionality of this command is similar to that of gui_raise command, difference being:

1. `gui_show` displays/reopens only the GUI main window and not the schematic main window.
2. It does not place the main window on top of other open windows.

Related Information

Related commands:

gui_hide
gui_raise

gui_snapshot_pv

```
gui_snapshot_pv
[-congestion]
[-overwrite][-png]file[-width integer][-height integer][-density][-min_value float][max_value float][-legend]
```

Description

Saves a snapshot image of the part of the design that is visible in the Layout Viewer.

Options and Arguments

`-congestion`	Overlays a congestion map on top of what is visible in the Layout Viewer before making a snapshot.
	This option eliminates the need to first turn on the display of the congestion map.
file	Specifies the name of the file in which to save the snapshot.
`-height` *integer*	Specifies the height of the snapshot. The value can be in the range: 100 to 20000.
`-legend`	Specifies to save the legend.
`-density`	Enables the density map overlay
`-min_value` *float*	Specifies the minimum value for density color scale (range 0.0 to 1.0)
`-max_value` *float*	Specifies the maximum value for density color scale (range 0.0 to 1.0)
`-overwrite`	Specifies to overwrite an existing file.
	If you omit this option, you'll get a message that indicates that the file exists.
`-png`	Specifies to create the file in PNG format.
	If this option is omitted, a GIF file will be created.
`-width` *integer*	Specifies the width of the snapshot. The value can be in the range: 100 to 20000.

gui_snapshot_sv

```
gui_snapshot_sv
[-overwrite] [-no_fit] [-png] file
```

Description

Saves a snapshot image of the part of the design that is visible in the Schematic Viewer.

Options and Arguments

file	Specifies the name of the file in which to save the snapshot.
-no_fit	Captures the design as is exactly displayed on the Schematic Viewer (as is zoomed or highlighted).
	If you omit this option, whole design is fit to the full screen and saved (regardless of how it is displayed).
-overwrite	Specifies to overwrite an existing file.
	If you omit this option, you'll get a message that indicates that the file exists.
-png	Specifies to create the file in PNG format.
	If you omit this option, a GIF file will be created by default.

gui_sv_get_instance

gui_sv_get_instance

Description

Returns the current instance or hierarchical instance which is currently displayed on the Schematic Viewer.

Examples

- If top design is displayed on the Schematic Viewer, gui_sv_get_instance returns the design object:

  ```
  gui_sv_get_instance
  myAlu
  ```

- If a hierarchical instance is displayed on the Schematic Viewer, `gui_sv_get_instance` returns the instance object.

  ```
  gui_sv_get_instance
  myAlu/ckGen
  ```

gui_sv_load

gui_sv_load {*design* | *inst* | *hinst*}

Specifies an hierarchical instance or instance or design to load into the main Schematic Viewer.

> The opened window should be schematic viewer in order for this command to work. If you are in any other viewer, open the schematic viewer before executing this command.

Example

Following example shows how to load an instance in Schematic Viewer.

```
gui_sv_load TDSP_CORE_INST
```

gui_undo_pv

gui_undo_pv [-win *string*] [-fromgui]

Description

Specifies to undo last floorplan edit command.

Options and Arguments

`-fromgui`	Indicates whether the call comes from the GUI.
`-win` *string*	Specifies the name of the window which needs to be refreshed.

gui_update

```
gui_update
```

Updates the GUI. It forces synchronization of primary viewers with the current design database. Clicking the 'Redraw' button on GUI Viewer also executes this command.

Related Information

Related commands:

gui_raise

gui_zoom_fit_pv

```
gui_zoom_fit_pv
```

Description

Performs a "zoom fit" in the Layout Viewer.

Related Information

Related command:

gui_update

gui_zoom_fit_sv

```
gui_zoom_fit_sv
```

Performs a "zoom fit" in the Schematic Viewer.

Related Information

Related command:

gui_update

gui_zoom_pv

```
gui_zoom_pv [-rect {llx lly urx ury}...]
[-in] [-out] [-selected]
```

Description

Performs zoom-in or zoom-out as per the specified parameters in the Layout Viewer.

Options and Arguments

`-in`	Specifies to zoom-in the Layout Viewer.		
`-out`	Specifies to zoom-out the Layout Viewer.		
`-rect {llx lly urx ury }`			
		`-selected`	Specifies to zoom the selected objects.

select_obj

```
select_obj {fpobject | region | blockage | hnet | port | pin | hpin |
 hinst |inst}...
```

Description

Selects the specified objects in the Layout Viewer.

Options and Arguments

`{fpobject|region|blockage|hnet|port|pin|hpin|hinst|nst}`	
	Specifies the objects to select.

Related Information

Related command:

deselect_obj

12

Hierarchical Flow

- assemble_design
- generate_ilm
- read_ilm
- read_ilm_from_files

assemble_design

```
assemble_design

 [-config_view_name_mapping string] [design] [-fuzzy_match]
```

Description

Integrates the interface logic models(ILMs) into the design.

Use this command after elaborating the design which includes the top level module and all the ILMs. The DEF file for the top-level design is also required before assembling the design if physical ILM are read (using the `read_ilm` command without the `-logical` option).

> In case of a multi-mode design, you must create the modes, read the top-level SDC file, and initialize the design using `init_design` before assembling the design. This can be done with the `read_mmmc` command.

Options and Arguments

`-config_view_name_mapping` *string*	
	Specifies a string for name mapping if view name given in the ILM configuration file is different from the one at TOP level. ILM view name should be mentioned first. For example: {ilm_view_1 top_view_1} {ilm_view_2 top_view_2}
design	Specifies the name of the design for which the checking needs to be done. This argument is only required if multiple designs are loaded.
`-fuzzy_match`	Enables fuzzy name matching in ILM `read_def`.

Examples

- Logical flow:

```
read_hdl top.v
read_ilm -basename ilm_dir/leon -module leon -logical
read_ilm -basename ilm_dir/dsp -module dsp -logical
elaborate top
assemble_design
```

- Physical flow:

```
read_hdl top.v
read_ilm -basename ilm_dir/leon -module leon
read_ilm -basename ilm_dir/dsp -module dsp
elaborate top
read_def -design top top.def
assemble_design
```

Related Information

Related commands:

generate_ilm

read_ilm

generate_ilm

```
generate_ilm
[-basename string |-preview]
[-latch] [-gzip_files] [design]
```

Description

Generates an interface logic model (ILM) for the design. The ILM contains:

- A Verilog structural netlist with information about the instances associated with the interface logic.
- A DEF file containing the physical information of the nets, interconnects, pins, and the other hard macros in the design interface.
  A DEF file is only generated if placement data is available.
- An SPEF file that contains parasitic information of the pins and nets at the design interface
- An SDC file that contains `set_transition` and `set_case_analysis` information for the inputs, nets and instance pins of the interface logic. This allows for accurate calculation of delays and slews.

In Genus, you should generate an ILM for the design at the end of the synthesis run.

> When generating an ILM, any logic that is not part of an I/O path will be removed from the database and the design will be reduced to an interface logic model. Therefore, you should save a snapshot of the design (`write_db`) prior to generating the ILM.

Options and Arguments

`-basename` *`string`*	Specifies the directory path name and base filename for the output data.
	This option is not required if you generate a preview.
`design`	Specifies the name of the design for which to generate the model.
	This argument is only required if multiple designs are loaded.
`-gzip_files`	Compresses the generated files in gzip format.
`-latch`	Specifies whether the ILM should stop at the first latch.
	By default, the command will include latches in the interface logic and stop at the first flop.
`-preview`	Only generates a reduction summary report which gives you a comparison between the number of instances and registers in the original design and in the to-be-generated ILM. Based on the reported reduction ratio, you can decide whether to generate the ILM for the block.

Related Information

Related commands:

assemble_design
read_ilm

read_ilm

```
read_ilm
-basename string
-module_name module
[-logical] [-lef_file file] [-upf_file file]
[-view_map string] [-corner_map string]
```

Description

Reads in the interface logic model (ILM) for the specified module.

You should read in the ILMs after reading in the RTL of the design, that is, before elaborating the design.

Options and Arguments

`-basename` *`string`*	Specifies the directory path name and base filename for the ILM data. {Genus: *`dirname/filename`*, Innovus: *`dirname`*}
`-corner_map` *`string`*	Specifies SPEF corners for mapping.
	Specified as a list of lists: `{{`*`corner1 top_corner1`*`} {`*`corner2 top_corner2`*`}...}`
`-lef_file` *`file`*	Specifies the name of the LEF file corresponding to the ILM.
`-logical`	Loads only the logical data.
`-module_name` *`module`*	
	Specifies the name of the ILM.
`-upf_file` *`file`*	Specifies the name of the UPF file corresponding to the ILM.
`-view_map` *`string`*	Specifies SDC views for mapping.
	Specified as a list of lists: `{{`*`view1 top_view1`*`} {`*`view2 top_view2`*`} ...}`

Related Information

Related commands:

assemble_design
generate_ilm

read_ilm_from_files

```
read_ilm_from_files
-module_name module [-verilog_file file]
-input_directory string [-logical] [-output_directory string] [-prefix string] [-def_file file]
[-spef_file file] [-upf_file file] [-sdc_file file] [-place_file file] [-lef_file file] [-
sdc_by_mode {{view_name_1 sdc_file} {view_name_2 sdc_file}]...}] [upf_invs_read_ilm] [-
spef_by_corner {{corner_1 spef_file_for_corner_1} {corner_2 spef_file_for_corner_2}]...}]
[-view_map string] [-corner_map string] [-innovus]
```

Description

Imports an interface logic model (ILM) for the specified module.

Provide at least an SDC file or SPEF file for the ILM, but it is recommended to specify both.

Options and Arguments

`-corner_map string`	Specifies SPEF corners for mapping.
	Specified as a list of lists: `{{corner1 top_corner1} {corner2 top_corner2}...}`
`-def_file file`	Specifies the name of the DEF file corresponding to the ILM.
`-innovus`	Specifies to load Innovus ILM or SHDB.
`-input_directory string`	
	Specifies the directory containing the ILM data to be imported
`-lef_file file`	Specifies the name of the LEF file corresponding to the ILM.
`-logical`	Loads only the logical data.

`-module_name` *`module`*	
	Specifies the name of the ILM.
`-output_directory` *`string`*	
	Specifies the directory where Genus should write out the tool-generated ILM data.
`-place_file` *`file`*	Specifies the name of the Innovus place file of the ILM.
`-prefix` *`string`*	Specifies the prefix for the ILM data generated by Genus.
	Default: The specified module name in `module_name` becomes the prefix.
`-sdc_by_mode` *`string`*	
	Specifies a list of Tcl lists. Each Tcl l list contains a view of the ILM with the corresponding SDC file.
	Example: `{{view_a sdc_file} {view_b sdc_file} ...}`
`-sdc_file` *`file`*	Specifies the name of the SDC file corresponding to the ILM.
`-spef_by_corner` *`string`*	
	Specifies a list of Tcl lists. Each Tcl l list contains a path of SPEF files for multiple rc-corners in the ILM module.
	Provide at least one SPEF or SDC file for ILM module.
	Example: `{{corner_a spef_file_for_corner_a} {corner_b spef_file_for_corner_b} ...}`
`-spef_file` *`file`*	Specifies the name of the SPEF file corresponding to the ILM.
`-upf_file` *`file`*	Specifies the name of the UPF file corresponding to the ILM.
`-upf_invs_read_ilm`	Specifies to read UPF file coming from Innovus `read_ilm` command.
`-verilog_file` *`file`*	Specifies the name of the Verilog file of the ILM.
`-view_map string`	Specifies SDC views for mapping.

	Specified as a list of lists: `{{view1 top_view1} {view2 top_view2} ...}`

13

Incremental Synthesis

- design_has_cache_replaced_content
- report_incremental_synthesis
- reset_incrsyn_preserves
- write_incremental_synthesis_cache

design_has_cache_replaced_content

`design_has_cache_replaced_content` *`design`*

Description

Returns true if a design contains content (design or subdesign) that was reused from an incremental synthesis cache.

Options and Arguments

`design`	Specifies the design name to be analyzed. If not specified, then the return value is based on inspection of the current design.

Related Information

Related attribute:

incrsyn_read_cache

report_incremental_synthesis

```
report_incremental_synthesis
[-detail]
[-only_resynthesized]
[-output file]
[-module design|subdesign]
[design]
```

Description

Reports Incremental Synthesis cache-replacement statistics, for the "incremental" run in an incremental synthesis flow.

Options and Arguments

design	Specifies the design name on which to report. If no design is specified, the report is given for the current design.
-detail	Includes additional/verbose detail in the report.
-only_resynthesized	Reports information only for modules that were re-synthesized. By-default, both re-synthesized and cache-reused modules are included in the report.
-output *file*	Writes the report to the specified file, rather than to stdout.
-module *design*|*subdesign*	Reports re-use/re-synthesis only for the specified design or subdesign.

Related Information

Affected by these attributes:	incrsyn_read_cache
	incrsyn_write_cache

reset_incrsyn_preserves

```
reset_incrsyn_preserves
[-dont_reset_net_drivers]
[design]
```

Description

Reset controls that preserve cache-reused content in an incremental synthesis flow, such that this content can be fully optimzed in subsequent flow steps. Call this command in the incremental (read-cache) run, once the flow has reached the point at which the cache was written in the baseline run.

Options and Arguments

`-dont_reset_net_drivers`	Leaves the attributes intact that tag the driving pins of nets and are fully cache-replaced. Use this option when intending to seed an Innovus POD-reuse flow with an incrementally synthesized result from Genus.
design	Specifies the design name on which to operate. If no design is specified, then the command is applied to the current design.

Related Information

Affected by these attributes	incrsyn_read_cache
	incrsyn_write_cache

write_incremental_synthesis_cache

```
write_incremental_synthesis_cache

[design]
```

Description

Writes an incremental synthesis cache, for use in a subsequent incremental re-synthesis of a design.

Options and Arguments

design	Specifies the design name for which a cache should be written. If no design is specified, then a cache is written for the current design.

Related Information

Affected by these attributes	incrsyn_write_cache

14

Input and Output

- decrypt
- encrypt
- export_oa_db
- hdl_set_vlog_file_extension
- import_oa_db
- read_clock_tree_spec
- read_db
- read_dfm
- read_hdl
- read_libs
- read_module_model
- read_netlist
- read_qrc
- read_sdc
- read_sdf
- split_db
- write_clp_script
- write_db
- write_design
- write_do_lec
- write_hdl
- write_mmmc
- write_module_model
- write_name_mapping
- write_netlist
- write_sdc
- write_sdf
- write_snapshot
- write_sv_wrapper
- write_template
- write_tempus

The following commands are common with Flowkit commands:

- write_flow
- write_flow_template

The following commands are common with DFT commands:

- read_dft_abstract_model
- read_dft_jtag_boundary_file
- read_dft_pmbist_interface_files
- write_dft_abstract_model
- write_dft_atpg

- write_dft_atpg_other_vendor_files
- write_dft_bsdl
- write_dft_compression_macro
- write_dft_compression_test_points
- write_dft_constraints
- write_dft_deterministic_test_points
- write_dft_jtag_boundary_file
- write_dft_jtag_boundary_verification
- write_dft_lbist_macro
- write_dft_lbist_test_points
- write_dft_lbist_testbench
- write_dft_pmbist_interface_files
- write_scandef

The following commands are common with Physical commands:

- read_encounter
- write_def
- read_parasitics
- read_physical
- write_congestion_map
- write_sdp_file
- read_def
- write_parasitics
- write_reports

The following commands are common with Joules commands:

- write_icgc_monitor

The following commands are common with Hierarchical Flow commands:

- read_ilm

- read_ilm_from_files

The following commands are common with Metrics commands:

- read_metric
- write_metric

The following commands are common with Low Power Synthesis commands:

- write_tcf
- write_saif

decrypt

`decrypt [-keydb path] file`

Description

Decrypts and evaluates a Tcl file that was encrypted with the `encrypt` command.

Options and Arguments

file	Specifies the name of the Tcl file to be decrypted and evaluated.
`-keydb` *path*	Sets the `NCPROTECT_KEYDB` environment variable to the directory containing the public key needed to decrypt the file.
	If you omit this option, you need to set the `NCPROTECT_KEYDB` environment variable before you run the `decrypt` command.

Related Information

Related command:

encrypt

encrypt

```
encrypt input_file_name
[-verilog | -vhdl | -tcl] [-pragma]
[> file]
```

Description

Uses the NC Protect protection scheme to encrypt the specified HDL or Tcl files.

To load encrypted HDL files, use the `read_hdl` command.

The command to source an encrypted Tcl file depends on the file extension of the encrypted Tcl file:

- If the encrypted file does not have the `.etf` extension, use the `decrypt` command.
- If the encrypted file has the `.etf` extension, you can use the `source` command.

Options and Arguments

input_file_name	Specifies the file to be encrypted.
file	Specifies the name of the encrypted file.
	By default, the encrypted file is printed to standard out (stdout).
`-pragma`	Specifies to only encrypt the text between the `protect begin` and `protect end` NC Protect pragmas. Encryption block must start with `pragma protect` comment
`-tcl`	Specifies that the file to be encrypted is a Tcl file. To hide the body of the Tcl scripts, make sure to code procedures using `hidden_proc`.
`-vhdl`	Uses VHDL style comments for NC Protect pragmas.
`-verilog`	Uses Verilog style comments for NC Protect pragmas. This is the default option.

Related Information

Related command:

decrypt

export_oa_db

```
export_oa_db
-ref_lib string
[-design_cell string] [-design_view string]
[-design_lib string] [-lib_path string]
[-invs_write_design_dir string]
```

Description

Generates the Open Access database which can be loaded back in Innovus™ Implementation System and Virtuoso®.

Options and Arguments

`-design_cell` *string*	
	Specifies the name of the Open Access design cell to export to.
	Default: *design_name*
`-design_lib` *string*	Specifies the name of the Open Access library to export to.
	If the library does not exist, the library will be created in the directory specified by the `-lib_path` option.
	If the library already exists (as a pointer in cds.lib) and the `-lib_path` option points to another directory, an error will be issued.
	Default: `oa_out_`*process_id*
`-design_view` *string*	
	Specifies the name of the Open Access view to export to.
	Default: `synthesized`
`-invs_write_design_dir` *string*	
	Specifies the directory where the `write_design -innovus` data is saved.
	Default: `.oa_export_`*process_id*
`-lib_path` *string*	Specifies the path to the library if the library must be created.
	Default: `./oa_export_`*process_id*
`-ref_lib` *string*	Specifies the list of Open Access reference libraries.

Related Information

Design For Manufacturing Flow

Related command:

import_oa_db

hdl_set_vlog_file_extension

```
hdl_set_vlog_file_extension
{-v1995 | -v2001 |-sv} string ...
```

Description

Specifies the Verilog language version that corresponds to the specified filename extension(s).

Options and Arguments

string	Specifies the filename extension.
	The filename extension must start with the "." character, and is case sensitive.
-sv	Specifies that HDL files with the specified extension conform to SystemVerilog P1800.
-v1995	Specifies that HDL files with the specified extension conform to Verilog-1995.
-v2001	Specifies that HDL files with the specified extension conform to Verilog-2001.

Example

The following `read_hdl` command automatically parses `bot1.v1` in v2001 format, and `test.v2` in System Verilog format.

```
hdl_set_vlog_file_extension -v2001 .v1
```

```
hdl_set_vlog_file_extension -sv .v2 .v3

read_hdl -mixvlog bot.v1 test.v2
```

Related Information

Using the Genus Database in *Genus User Guide*.	
Affects this command:	read_hdl

import_oa_db

```
import_oa_db
-ref_lib string
-design_lib  string
-design_cell string
-design_view string
```

Description

Reads in the Open Access database into Genus and generates the LEF and DEF files.

The generated LEF file is read internally when you execute the `import_oa_db` command. The generated DEF file is read internally post elaboration.

Options and Arguments

`-design_cell` *string*	
	Specifies the Open Access design cell name to be used.import.
`-design_lib` *string*	Specifies the Open Access design library name to be used.
`-design_view` *string*	
	Specifies the Open Access design view name to be used.
`-ref_lib` *string*	Specifies the list of Open Access reference libraries to import.

Related Information

Design For Manufacturing Flow

Related command:

export_oa_db

read_clock_tree_spec

```
read_clock_tree_spec
string
```

Description

Takes a list of files and validates that the files describe an Innovus clock tree specification. Calls to `syn_opt -spatial` command will use this specification.

Options and Arguments

string	Specifies the list of filenames.

Example

```
read_clock_tree_spec my_cts_spec.tcl
```

read_db

```
read_db
[db_file] [-from_tcl string] [-mmmc_file file]
[-quiet] [-verbose]
[-setup_views] [-hold_views]
[-dynamic_view] [-leakage_view]
```

Description

Loads the specified database file or Tcl object.

If the database contains setup information, the setup is restored as well. If the setup was written to a separate script, you must source that script before you read the database file.

Options and Arguments

db_file	Specifies the name of the file or directory to be read.
-dynamic_view	Specifies the analysis_view for dynamic power analysis
-from_tcl *string*	Specifies to read the database from the specified Tcl object.
-hold_views	Specifies a list of hold analysis_views for loading.
-leakage_view	Specifies the analysis_view for leakage power analysis
-mmmc_file *file*	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file. Use this option if you want to override the settings in the existing MMMC configuration file in the design being restored and instead load a different MMMC file.
-quiet	Suppresses any warning messages. Only error messages are printed.
-setup_views	Specifies a list of setup analysis_views for loading.
-verbose	Enables verbose output while reading in the database file.

Related Information

Design For Manufacturing Flow

Related commands:

split_db
write_db

read_dfm

read_dfm *coefficients_filename*

Description

Reads the coefficients file. You can only load one file at a time. After the coefficients file is loaded, Genus will annotate the defect probability of any matching cells between the coefficients file and the timing library.

Options and Arguments

coefficients_filename	
	Specifies the name of the coefficients file.

Related Information

Affects these commands:

- report_gates
- report_yield

Affects this attribute:

- yield

Related attribute:

- optimize_yield

read_hdl

```
read_hdl
[-language {v2001 | v1995 | sv | vhdl }]
[-library library_name[=library_name2]...]
[-netlist][-define macro=value]... file_list...[-f filename]
```

Description

Loads one or more HDL files in the order given into memory. Files containing macro definitions should be loaded before the macros are used. Otherwise, there are no ordering of constraints.

If you do not specify either the `v1995`, `v2001`, `sv` or the `vhdl` option, the default language format is that specified by the hdl_language attribute. The *default* value for the `hdl_language` attribute is `v2001`.

The HDL files can contain structural code for combining lower level modules, behavioral design specifications, or RTL implementations.

You can automatically read in or write out a compressed HDL file in gzip format.

When you load a parameterized Verilog module or VHDL architecture, each parameter in the module or architecture will be identified as an `hdl_parameter` object and located under `../architecture_name/parameters`. The default `hdl_parameter` attribute value for these parameters will be `true`.

Use the `genus -abort_on_error -f <`*your script*`>` command to specify that Genus automatically quit if a script error is detected when reading in HDL files instead of holding at the `genus:root:>` prompt.

Options and Arguments

`-define` *`macro=value`*	Defines a Verilog macro with the specified *`value`*, which is equivalent to the `'define` *`macro value`*. You can also define a macro definition list.
`-f` *`filename`*	Specifies the name of the list file for reading files from the simulation environment. This option is not supported for VHDL designs.
`file_list`	Specifies the name of the HDL files to load. If several files must be loaded, specify them in a string.
	The files can be encrypted.

	The host directory where the HDL files are looked for is specified via the `hdl_search_path` root attribute.
`-language`	Specifies the language of the HDL files. It can be any one of the following: *`sv`*, *`v1995`*, *`v2001`* (default) and *`vhdl`*.
	The `hdl_vhdl_read_version` root attribute value specifies the standard to which the VHDL files conform.
	Default: `VHDL-1993`
`-library` *`library_name`*`[=`*`library_name2`*`]...`	
	Specifies the name of the Verilog or VHDL library in which the definitions will be stored.
	A virtual directory with this name will be created in the `hdl_libraries` directory of the design hierarchy if it does not already exist.
	If you specify multiple libraries, they become multiple names (aliases) of one library. In this case, separate the names with the equal sign (=). Only one of the library names in the list becomes a virtual directory in the `hdl_libraries` directory.
	The library definitions remain in effect until elaboration, after which all library definitions are deleted.
	By specifying Verilog and VHDL library names, you can read in multiple Verilog modules and VHDL entities (and VHDL packages) with the same name without overwriting each other.
`-netlist`	Reads structural input files when parts of the input design is in the form of a structural netlist. You can read partially structural files provided the structural part of the input design is in the form of structural Verilog-1995 constructs and is contained in separate files from the non-structural (RTL) input.
	If this option is specified, `-language` option is ignored.

Examples

- The following example first loads the `example1.v` file, then the `example2.v` file:

  ```
  genus:root: XX> read_hdl {example1.v example2.v}
  ```

- The following commands with macro definitions are equivalent:
  - `read_hdl -define "A B=4 C" example.v`
  - `read_hdl -define A -define B=4 -define C ...`
- The following command loads a single VHDL file and specifies a single VHDL library.

  ```
  read_hdl -language vhdl -library lib1 test1.vhdl
  ```

- The following commands read structural Verilog files when the design includes RTL (VHDL or Verilog) files:

  ```
  read_hdl -language vhdl file1_bhv.vhdl

  read_hdl -language sv file2_bhv.v

  read_hdl -netlist file3_str.v

  elaborate
  ```

- In the following command, the `-v1995` option is ignored. Both `rtl.v` and `struct.v` are parsed in the structural mode.

  ```
  read_hdl -language v1995 rtl.v -netlist struct.v
  ```

- The following command defines VHDL libraries `lib1`, `lib2` as aliases for `lib3`.

  ```
  read_hdl -language vhdl -library lib1=lib2=lib3 test1.vhdl
  ```

- The following commands read in two Verilog files that each contain a Verilog module with the same name (`compute`) but with different functionality. To store both definitions, the `-lib` option indicates in which library to store the definition.

  ```
  read_hdl -language v2001 -library lib1 test_01_1.v
  read_hdl -language v2001 -library lib2 test_01_2.v
  ```

  Inspection of the design hierarchy shows:

  ```
  genus:root: XX> vls hdl_libraries

  /hdl_libraries: ./              DP/             DW04/           GB/             STD/
  ```

```
lib2/
    AMBIT/          DW/             DW05/           GTECH/          SYNERGY/        synthetic/
    CADENCE/        DW01/           DW06/           IEEE/           SYNOPSYS/
    CW/             DW02/           DWARE/          IEEE_SYNERGY/   lib1/
    CWTECH          DW03/
```

```
    genus:root: XX> vls /hdl_libraries/lib1/architectures/hdl_lib:lib1.architectures:./
    compute/ genus:root: XX> vls /hdl_libraries/lib2/architectures/hdl_lib:lib2.architectures:./
    compute/
```

Related Information

Reading a Partially Structural Design in *Genus User Guide.*	
Affects this command:	elaborate
Related command:	read_netlist
Affects this attribute:	hdl_parameter
Affected by these attributes:	hdl_language
	hdl_preserve_dangling_output_nets
	hdl_verilog_defines
	init_hdl_search_path

read_libs

```
read_libs [-min_libs string] [-max_libs string]
 [-min_noise_libs string] [-max_noise_libs string]
 [-aocv_libs string] [-socv_libs string]
 [-noise_libs string] [-stamp_libs string]
 [-power_ground_files string]  [-library_side_file string]
 [filenames...]
```

Description

Reads the libraries. The command will search for libraries with the specified name. If not found, it will search for the libraries in the library search path. Only the first library encountered will be loaded.

Options and Arguments

`-aocv_libs` *string*	Specifies the list of AOCV libraries to be read for Advanced On-Chip-Variable support.
filenames...	Specifies the list of libraries. When this is used, the tool will populate the minimum and maximum with the same libraries.
`-max_libs` *string*	Specifies the list of libraries for maximum analysis to be read.
	If this option is used along with the *filenames* option, it will cause an error.
`-max_noise_libs` *string*	
	Not Supported in Genus.
`-min_libs` *string*	Specifies a list of libraries for minimum analysis to be read.
	If this option is used along with the *filenames* option, it will cause an error.
	Not supported in Genus.
`-min_noise_libs` *string*	
	Not supported in Genus.
`-noise_libs` *string*	
	Not supported in Genus.
`-power_ground_files` *string*	
	Not supported in Genus.
`-socv_libs` *string*	Specifies the list of SOCV libraries to be read.
`-stamp_libs` *string*	
	Not supported in Genus.
`-library_side_file` *string*	Specifies the library side files to be read.

Related Information

Related commands:

read_mmmc
read_physical
read_netlist

read_module_model

```
read_module_model
[-type string] [-tag string] [-cell string]
```

Description

Reads the content of the module model.

Options and Arguments

`-type string`	Specifies the type of the module model.
`-tag string`	Specifies the tag name of the module model.
`-cell string`	Specifies the name of the module.

Related Information

Related Commands:

commit_module_model
set_module_model
write_module_model

read_netlist

```
read_netlist file_list
[-top top_module_name]
[-define macro=value]...
[-language {v2001|v1995}]
```

Description

Reads and elaborates a Verilog 2001 or Verilog 1995 structural netlist when the design does not include behavioral (VHDL or Verilog) modules.

A structural Verilog file contains only structural Verilog-1995 or 2001 constructs, such as module and gate instances, concurrent assignment statements, references to nets, bit-selects, part-selects, concatenations, and the unary ~ operator. Using the `read_hdl -netlist` command uses less memory and runtime to load a structural file than the `read_hdl` command.

- Use the `read_netlist` command to read and elaborate a design and create a generic netlist that is ready to be synthesized. You do *not* need to use the `elaborate` command
- Use the read_hdl `-netlist` command to read in a design that *includes* behavioral (VHDL or Verilog) files.

Options and Arguments

`-define` *`macro=value`*	
	Defines a Verilog macro with the specified *`value`*, which is equivalent to the `` `define `` *`macro value`*.
`file_list`	Specifies the name of the HDL files to load. If several files must be loaded, specify them in a string.
	The host directory where the HDL files are looked for is specified via the `hdl_search_path` root attribute.
`-language`	Specifies the language in which the netlist file is written.
	Default: `v2001`
`-top` *`top_module_name`*	
	Specifies the top-level structural Verilog module to be read and elaborated.
	If you do not specify this option and multiple top-level modules are found in the loaded netlist, the tool randomly selects one of them and deletes the remaining top-level modules.

Related Information

Related Commands:	`read_hdl -netlist`
	read_libs
	read_physical
	write_hdl
Sets this attribute:	hdl_v2001
Affected by these attributes:	hdl_preserve_dangling_output_nets
	init_hdl_search_path

read_qrc

```
read_qrc string...
```

Description

Reads the parasitic extraction technology file. This command should be issued after `read_libs` but before `read_mmmc`. This is used only in the simple mode (without MMMC).

Options and Arguments

string	Specifies the name of the parasitic extraction technology filename (QRC tech file) to be read.

Related Information

Related commands:

read_libs

read_sdc

```
read_sdc file [files]
[-stop_on_errors] [-no_compress]
[-echo] [-verbose]
{-view view_name]
```

Description

Reads a constraints file in Synopsys Design Constraint (SDC) format into Genus. Genus creates a cost group for each clock defined in the file. It does not create false paths between these clocks.

You must first elaborate the design before you can read the design constraints.

If you use the `read_sdc` command for loading a subset of timing constraints that includes native Genus commands, such as adding exceptions, the `write_sdc` command will write these exceptions out in the SDC file.

After using the `read_sdc` command, if you make hierarchy changes in Genus using the `ungroup` or `group` commands, and there are pin specific constraints, then the `write_sdc` command will reflect the change in the hierarchy. For example, if you have constraints on the hierarchy pins you have ungrouped, then the constraints are moved to buffers (that are automatically inserted by Genus when ungrouping). The SDC file will have constraints reflecting these buffers.

Options and Arguments

`-echo`	Echoes he SDC commands in the logfile and on the console.
`file` `[`*`files`*`]`	Specifies the name of the constraints files to read.
	You can also specify a file that was compressed with gzip (`.gz` extension).
`-no_compress`	Turns off advanced compression and compression of exceptions for the remainder of the session.
	The tool issues message SDC-230 if a potential runtime increase can be expected when you set this option. This message is given when the ratio of the number of exceptions to the number of instances exceeds 0.068, and the number of instances exceeds 100,000.
`-stop_on_errors`	Stops reading the remainder of the script if an error is encountered during reading of the SDC file.
`-verbose`	Displays additional information for each SDC command.
`-view` *`view_name`*	Reads view-specific constraints for a design.

Related Information

Affects these commands:	syn_generic
	syn_map
	syn_opt
Related command:	write_sdc
Affected by these attributes:	enable_break_timing_paths_by_mode
	detailed_sdc_messages
Related attributes:	embedded_script
	scale_factor_group_path_weights
	tim_ignore_data_check_for_non_endpoint_pins

Unsupported Constraints

Not all SDCs are supported. For those that are not supported, Genus will issue a warning message but store them for output for the `write_sdc` command. Genus will only store the SDCs and not manipulate any data with them.

The following SDCs are not supported:

```
set_max_area

set_propagated_clock

set_scan_style

set_signal_type

set_test_methodology

set_wire_load_min_block_size

get_references

set_connection_class

set_critical_range

set_fix_multiple_port_nets
```

set_local_link_library

read_sdf

```
read_sdf file
[-design design]
{-view view_name]
```

Description

Reads the annotated delays from a file in Standard Delay Format (SDF) and applies it to the specified design for the specified analysis view.

Options and Arguments

`-design design`	Specifies the name of the design to which the annotated delays must be applied.
file	Specifies the name of the SDF file to read.
`-view view_name`	Reads the view-specific delays for the design.

Related Information

Design For Manufacturing Flow

Related command:

write_sdf

split_db

```
split_db
input_db_file [-from_tcl file]
-script file [-to_file file]
```

Description

Reads a database with setup information and writes out the setup information in a setup script and the netlist information into a setup-free database.

If `write_db` command was used without the -`script` option, you can use the `split_db` command to extract the root attributes from the existing database. This command reads the database along with the setup information and then writes out all the root attributes to a separate tcl file specified in the -`script` option. A new database file is also created (specified in the -`to_file` option).

Options and Arguments

`-from_tcl` *file*	Specifies to read the specified Tcl object.
`-script` *file*	Specifies the name of the script to be written.
	If this option is specified without the `-to_file` option, the tool writes both the setup and the database to the script. This ensures that the Tcl script and its database are always in sync.
	Storing the database in the script takes more memory and results in a larger file.
`-to_file` *file*	Specifies the name of the new database file.
input_db_file	Specifies the name of the database file to be read.

Example

```
split_db my_org.db -to_file my_new.db -script my_new.root_attribute.tcl
```

Related Information

Design For Manufacturing Flow

Related commands:

read_db
write_db

write_clp_script

```
write_clp_script
[-design design] [-netlist string ]
[-env_var string]
[-add_iso_cell string] [-clp_out_report string]
[-ignore_level_shifter_high_to_low]
[-pre_read script] [-pre_exit script]
[-cpf_file file | -files_1801 file] [-all ] [-lp_only] [-no_exit] [-tmp_dir string] [-log_file file] [-tcl] [> file]
```

Description

Writes the required dofile for Conformal Low Power (CLP).

> This command will issue an error and will not proceed if you have multiple Common Power Format (CPF) files.

For more information on the Low Power Rule (CLP) Checks, refer to the Conformal® Low Power Reference Manual.

Options and Arguments

`-add_iso_cell string`	
	Specifies the standard cells that CLP should recognize as isolation cells.
`-all`	Specifies to perform all rule checks, whether low-power related or not. This is also the default behavior.

`-clp_out_report` *string*	
	Writes the output of the `report rule check` Conformal Low Power command to this specified file.
`-cpf_file` *file*	Specifies the CPF file to be used for low power checks.
`-design` *design*	Specifies the top-level design in Genus.
`-env_var` *string*	Specifies the names and values of UNIX environment variables to be used in the library, design, and logfile names in the generated dofile.
file	Redirects all the output to the specified file.
`-files_1801` *file*	Specifies the 1801 file to be used for low power checks.
`-ignore_level_shifter_high_to_low`	
	Indicates whether to ignore the high to low level shifter check. If this option is specified, the following CLP directive will be added to the dofile:
	`set lowpower option -ignore_high_to_low`
`-log_file` *file*	Specifies the name of the CLP logfile.
`-lp_only`	Specifies to only perform the low power rule check errors and warnings.
	By default, non low-power related issues, such as structural issues are reported as well.
`-netlist` *path*	Specifies the UNIX path that contains the netlist containing all the design's low power features (for example, level shifters, isolation cells and SRPG flops).
	Default: RTL
`-no_exit`	Indicates whether to skip the `exit` command at the end of the dofile. If this option is specified, the following command will be omitted from the dofile:
	`exit -force`

`-pre_exit` *`string`*	Specifies the name of the dofile (script) that must be sourced before the CLP `exit` command.
`-pre_read` *`string`*	Specifies the name of dofile (script) that must be sourced before the libraries and the design are read.
`-tcl`	Specifies to generate the dofile as a Tcl script.
`-tmp_dir` *`string`*	pecifies the name of the directory with the hierarchical CLP dofile.
	The contents of this directory will be cleared after finishing creating the CLP script.

Related Information

Affected by these attributes:

wclp_lib_statetable

write_db

```
write_db
 [-all_root_attributes] [-no_root_attributes]
 [db_file] [-script file] [-design design]
 [-quiet] [-common] [-legacy] [-verbose]
```

Description

This command writes out a netlist, SDC, physical information (if any), power intent information (if any) and root attributes to a database file or returns a Tcl list. But, it doesn't write out any user defined variables. By default, only root attributes affecting the QOR are written out. Root attributes with non-default settings can be included in the database or written to a script if the `-all_root_attributes` option is specified. To read this database in a different environment, you need to specify the `-script` option which helps to write all root attributes in a separate tcl file, which can be later, edited to change the library paths.

`write_db` stores absolute paths of `library`, `lef_library`, `qrc_tech_file` and `cap_table_file` attributes and source files in setup information. If you wish to use relative paths, set the `write_db_use_relative_filepath` attribute. This attributes uses path relative to `init_lib_search_path` attribute while writing out these attributes to database.

Options and Arguments

`-all_root_attributes`	
	Writes out all the root attributes with non-default settings.
`db_file`	Specifies the name of the database file to be written.
	By default, the command returns a Tcl list.
`-design` *`design`*	Specifies the design for which to write out the database information.
	If omitted, the design defaults to the current design.
`-no_root_attributes`	
	Prevents writing out of the root attributes with non-default settings.
	You cannot specify this option with the `-script` option.
`-quiet`	Suppresses any warning messages. Error messages are printed.
`-common`	Writes database in Common DB format which is suitable for both Genus and Innovus.
	Note that a design using a non-MMMC timing configuration will be upgraded to MMMC if the DB is loaded in Innovus. To load a Common DB, use 'read_db' which will auto detect the format. Storing the database as a **Common DB** takes more memory. It is not recommended for DBs that will only be loaded back into Genus.
	You cannot use this option when the -script option is used.

-legacy	Forces MMMC content to be compatible with legacy Innovus. This is used with the `-common` option.
`-script file`	Includes all root attribute settings in the specified script.
	If this option is omitted, this information is included in the database file.
	If this option is specified without the *`db_file`* argument, the tool writes both the setup and the database to the script. This ensures that the Tcl script and its database are always in sync.
	Storing the database in the script takes more memory and results in a larger file.
	You cannot use this option when the the -common option is used.
`-verbose`	Enables verbose output while writing out the database file.

Example

```
write_db -all_root_attributes -script my_root.tcl
```

Related Information

Design For Manufacturing Flow

Related commands:

define_attribute
read_db
split_db

write_design

Related Attributes:

write_db_use_relative_filepath

init_lib_search_path

write_design

```
write_design
[-base_name string][-gzip_files]
[design]
```

Description

Generates all the files needed to reload the session in Genus (for example, `.g`, `.v`. and `.tcl` files).

> When you use this command on a design that is not fully mapped, for example after `elaborate` or `syn_generic`, and then reload and map the design, the final area and timing results may differ from the results obtained in a single synthesis session.

Options and Arguments

`-base_name` *string*	Specifies the path and basename for the generated files.
design	Specifies the top-level design in Genus.
`-gzip_files`	Compresses the generated files in gzip format.
	> Since Global Timing Debug (in Encounter Timing System) does not handle compressed SDC files, the `write_design` command will not compress SDC files.

Related Information

Related Command	write_db

write_do_lec

```
write_do_lec

[-top string]
[-golden_design {rtl|fv_map|file}]
-revised_design {fv_map|file}
[-1801_golden file][-1801_revised file]
[-cpf_golden file][-cpf_revised file]
[-no_lp][-no_insert_iso_in_dof]
[-sim_lib string] [-sim_plus_liberty][-cw_sim string]
[-no_dft] [-dft_constraint_file file]
[-log_file file][-env_var string]
[-pre_read file][-pre_compare file] [-pre_verification_information string][-pre_read_design
string][-pre_read_power_intent string][-post_read_power_intent string][-post_compare string][-
pre_exit file][-refine_elab_dofile][-no_exit]
[-hierarchical_reference | -flat]
[-checkpoint file]
[-tmp_dir string][-verbose][-fastgen2fvmap][-fastelab2gen]
[-fastrtl2elab][-gzip_fv_json][-hier][-rtl_revised]
[-low_power_analysis][> filename][out_file]
```

Description

Writes a script and verification information that can by used Conformal® Logical Equivalence Checking tool to compare the specified golden and revised designs.

This command works with the Common Power Format (CPF) flow: if it detects a CPF file then it will output this information to the Conformal LEC dofile.

Options and Arguments

`-1801_golden` *file*	Specifies the name of the golden (original) 1801 file.
	This option only applies to the IEEE-1801 flow.
`-1801_revised` *file*	Specifies the name of the revised 1801 file that corresponds to the current state of the design.
	This option only applies to the IEEE-1801 flow.
`-checkpoint` *file*	Specifies the name of the checkpoint file to generate.
`-cpf_golden` *file*	Specifies the name of the golden (original) file.
	If omitted, the file will be inferred from the `cpf_files` design attribute.
	This option only applies to the CPF flow.
`-cpf_revised` *file*	Specifies the name of the revised CPF file.
	If you omit this option, the file specified for the `-cpf_golden` option will be used.
	If you expect name changes in the CPF design objects, you must generate this file using the `write_power_intent -cpf` command.
	This option only applies to the CPF flow.
`-cw_sim` *string*	Specifies the language of the simulation models that the Conformal Logical Equivalence Checker should use to infer the ChipWare components.

	You can specify any of the following values: `{V1995|V2001|SV|VHDL1987|VHDL1993}`
	Default: Language specified in the `read_hdl` command.
`-dft_constraint_file` *file*	
	Sources DFT constraints from the specified file before executing the `compare` command.
`-env_var` *string*	Specifies the names and values of UNIX environment variables to be used for directory and file names in the dofile.
`-fastrtl2elab`	Specifies an abbreviated LEC dofile for quick RTL to elaborated netlist comparison.
`-fastelab2gen`	Specifies an abbreviated LEC dofile for quick elaborated to generic netlist comparison.
`-fastgen2fvmap`	Specifies an abbreviate dofile for quick generic to `fv_map` comparisons.
filename	Specifies the name of the target LEC script.
out_file	Specifies the name of the target LEC script.
`-flat`	Performs a flattened Conformal LEC comparison.
`-golden_design {rtl|fv_map|`*file*`}`	
	Specifies the golden design to compare to. You can specify the following values:
	`rtl`—`write_do_lec` will access the `hdl_filelist` attribute to use the same set of files that were read into Genus.
	`fv_map`—`write_do_lec` will use the netlist automatically written by Genus during mapping (`syn_map`).
	file—Specifies the UNIX path to an alternative golden design.

	If the file was loaded into Genus (using either `read_hdl` or `read_netlist`), the tool knows the language format of the file. Otherwise, the tool assumes that the format of the file is Verilog-1995.
	Default: `rtl`
`-gzip_fv_json`	Specifies to compress the implementation information file.
	This option is available with Conformal 19.10-s300 or newer versions.
`-hier`	Specfies whether to do hierarchical LEC comparison or not.
`-hierarchical_reference`	
	Performs a hierarchical Conformal LEC comparison.
`-log_file` *file*	Specifies the name of the Conformal LEC logfile.
`-low_power_analysis`	Specifies to add the Conformal Low Power commands to compare the power domains between the golden and revised designs. Power intent must be present.
`-no_dft`	Prevents that DFT constraints are written to the dofile.
`-no_exit`	Does not add the `exit` command to the end of the dofile.
`-no_insert_iso_in_dof`	
	Prevents that the `-insert_isolation` option is added to the LEC `commit power intent` commands for the golden and revised design in the generated dofile.
	Use this option when you write out a dofile before the power intent is committed in Genus (before using the `commit_power_intent` command)
`-no_lp`	Prevents verification of power intent information.

`-pre_compare` *file*	Specifies a dofile that must be sourced from the LEC dofile right before the LEC `set system mode lec` or `write hier_compare dofile` command.
`-pre_exit` *file*	Specifies a dofile that must be sourced from the LEC dofile right before the LEC `exit` command.
`-pre_read` *file*	Specifies an extra dofile that must be sourced from the LEC dofile right before the library and design are read.
`-pre_verification_information` *string*	Specifies a dofile to be sourced from the LEC dofile right before '`set_verification_information`'.
`-pre_read_design` *string*	Specifies a dofile to be sourced from the LEC dofile right before '`read_design`'.
`-pre_read_power_intent` *string*	Specifies a dofile to be sourced from the LEC dofile right before '`read_power_intent`'.
`-post_read_power_intent` *string*	Specifies a dofile to be sourced from the LEC dofile right after '`read_power_intent`'.
`-post_compare` *string*	Specifies a dofile to be sourced from the LEC dofile right after the comparison.
`-revised_design {fv_map|`*file*`}`	
	Specifies the revised design to compare to the golden design. You can specify the following values:
	`fv_map`—`write_do_lec` will use the netlist automatically written by Genus during mapping (`syn_map`).
	file—Specifies the UNIX path to the revised design.
`-refine_elab_dofile`	Specifies whether to enable a simple elab dofile.
`-rtl_revised`	Specifies that the revised design is DFT inserted back-annotated RTL.
`-sim_lib` *string*	Specifies the simulation library in Verilog 1995.

`-sim_plus_liberty`	Specifies whether the simulation library is added to or replaces the synthesis library.
`-smart`	Enables advanced SMART LEC capabilities. Requires a Conformal SMARTLec license.
`-tmp_dir` *string*	Specifies the name of the directory to which the generated files must be written.
`-top` *string*	Specifies the name of the top-level design.
`-verbose`	Generates a dofile with verbose reporting.

Examples

- The following command will source the `extra_settigs.do` dofile before the `compare` command in the sample.do file.

```
write_do_lec -revised revised.v -pre_compare extra_settings.do > sample.do
```

- The following command generates a dofile that will point to the System Verilog simulation models of all ChipWare components used in the design.

```
write_do_lec -revised_design ../netlist/top_nl.v -cw_sim sv \
 -log_file cw_sim_sv.log > cw_sim.sv_lec.do
```

Related Information

See the following sections in *Genus Interface to Conformal*: • Interfacing with Conformal Logical Equivalence Checker • Performing a Low Power Equivalence Check • More About the write_do_lec dofile	
Related command:	write_power_intent
Affected by these attributes:	boundary_optimize_invert_hpins
	dp_ungroup_during_syn_map
	wlec_add_noblack_box_retime_subdesign
	wlec_analyze_abort
	wlec_analyze_setup
	wlec_auto_analyze
	wlec_compare_threads
	wlec_composite_compare
	wlec_gzip_fv_json
	wlec_hier_comp_threshold
	wlec_lib_statetable
	wlec_low_power_analysis
	wlec_set_cdn_synth_root
	wlec_uniquify
	wlec_use_lec_model
	wlec_use_smart_lec

write_hdl

```
write_hdl [design|module]...
[-suffix string]
[-abstract] [-generic] [-depth integer]
[-equation] [-pg] [-language {v1995|v2001}]
[-lec] [-use_eeq_cell_with_liberty_info][-exclude_ilm] [ > file]
```

Description

Generates one of the following design implementations in Verilog format:

- A structural netlist using generic logic
- A structural netlist using mapped logic

You can automatically read in or write out a gzip compressed Verilog file.

Options and Arguments

`-abstract`	Generates an empty top-level Verilog module definition of the specified design or module that defines the I/O pins and bit-width for all top-level functional and scan-related ports in the design or module. This empty module description is further referred to as *logic abstract model*.
`-depth` *integer*	Specifies the number of hierarchy levels to be written out, starting from the top level. A value of 0, writes out only the top-level module.
	Default: `infinite`
{*design* | *module*}	
	Specifies the design or module for which the design implementation must be generated.
`-equation`	Writes out a logic equation in an assign statement for each Verilog primitive gate.

`-exclude_ilm`	Specifies to skip writing out Verilog for ILM modules.	
file	Specifies the file to which the output must be written.	
	Default: Output is written to the screen.	
		Once the synthesize commands (`syn_generic`, `syn_map`, or `syn_opt`) have been run, you cannot recover the version of the design that was generated using this option prior to synthesis.
`-language {v1995| v2001}`		
	Specifies the language of the output Verilog file. In `v2001`, the tool writes out the Verilog-2001 attributes stored with instances, ports and subports in the design.	
	Default: `v1995`	
`-lec`	Generates an *intermediate* netlist with additional information to facilitate formal verification with Conformal® Equivalence Checking.	
`-pg`	Adds the power and ground nets to the netlist, where PG pins, net connections have come through 1801, UPF files.	
	If the PG pins are coming from a Liberty file, and are connected in RTL, they will be written out by default in the Verilog netlist generated by Genus.	
`-suffix`	Specifies the string to be appended to the name of all defined modules in the generated netlist.	

-use_eeq_cell_with_liberty_info	
	Specifies whether the Verilog Writer should write the eeq master equivalent or not.

Related Information

Affected by these commands:

elaborate
syn_generic
syn_map
syn_opt

Affected by these attributes:

optimize_merge_flops
optimize_merge_latches
optimize_merge_seq
write_sv_port_wrapper
write_vlog_bit_blast_bus_connections
write_vlog_bit_blast_constants
write_vlog_bit_blast_mapped_ports
write_vlog_bit_blast_tech_cell
write_vlog_convert_onebit_vector_to_scalar
write_vlog_declare_wires
write_vlog_empty_module_for_logic_abstract

write_vlog_line_wrap_limit
write_vlog_no_negative_index
write_vlog_port_association_style
write_vlog_preserve_net_name
write_vlog_top_module_first
write_vlog_unconnected_port_style
write_vlog_wor_wand

write_mmmc

```
write_mmmc[-dir string] [-prefix string]
[-design] [-library]
[-skip_library_domains] [-skip_sdc_update string] [-no_sdc]
[-with_target_link] [design]
```

Description

Writes the MMMC view definition file to be used across Genus, Innovus and Tempus. It also includes the updated SDC files. By default it dumps out `mmmc.tcl` and the `default_emulate_constraint_mode.sdc` files.

Options and Arguments

`-design`	Specifies to write only the design-dependent MMMC settings.
`design`	Specifies the name of the design.
`-dir` *`string`*	Specifies the directory where the MMMC and SDC files will be written out.
`-library`	Specifies to write only the library-dependent MMMC settings.
`-prefix` *`string`*	Specifies the prefix for the output files.
`-skip_library_domains`	
	Specifies not to include library domains.
`-skip_sdc_update` *`string`*	
	Specifies the list which contains information regarding which constraint modes to skip updating SDC.
`-no_sdc`	Specifies the list which contains information regarding which constraint modes to skip updating SDC.
`-with_target_link`	Prevents merging of target and link libraries in the library sets. Genus writes out the `creating_library_set` command in the MMMC file with the `-target_timing` and `-link_timing` options.
	By default, Genus writes out the `creating_library_set` command in the MMMC file with the `-timing` option, which merges the target and link libraries into a single set of libraries. This makes the generated files, Innovus compatible.
	Applies only when you use the link library feature.

Related Information

Related commands:

create_analysis_view

create_constraint_mode
create_delay_corner
create_library_set
create_opcond
create_rc_corner
create_timing_condition
set_analysis_view

write_module_model

```
write_module_model [-tag string] [-type string]
```

Description

Creates and writes a module model such as pnr and db.

Options and Arguments

`-type` *string*	Specifies the type of the module model. By default, the module model is written as 'db' type. It can also be set as 'pnr' type.
`-tag` *string*	Specifies the name of the design.

Related Information

Related commands:

commit_module_model

read_module_model
set_module_model

write_name_mapping

```
write_name_mapping
[-to_file file][-include_clock_gating][-include_duplicate]
```

Description

Writes out the RTL name and the corresponding netlist name to a file for each of the following netlist objects:

- sequential instances
- input and output ports
- blackboxes (timing models and unresolved references)

By default, the format of the name mapping information is Conformal-compatible:

```
add mapped point RTL_instance_path netlist_instance_path phase_inversion
```

For phase inverted flops, `-invert` is added.

Options and Arguments

`-to_file` *file*	Specifies the name of the file to which the name mapping information must be written.
	Default: `./name_map.rpt`
`-include_clock_gating`	Specifies to include the name mapping information for clock gating cells instantiated in the HDL.
`-include_duplicate`	Spoecifies to include the name mapping information for duplicated sequential instances.

Example

```
add mapped point iCORE/digit_out[0]
iCORE/digit_out_reg[0]_iCORE_digit_out_reg[1]_iCORE_digit_out_reg[2]
_iCORE_digit_out_reg[3]/Q1N -invert

add mapped point iCORE/digit_out[1]
iCORE/digit_out_reg[0]_iCORE_digit_out_reg[1]_iCORE_digit_out_reg[2]
_iCORE_digit_out_reg[3]/Q2 -invert

add mapped point iCORE/digit_out[2]
iCORE/digit_out_reg[0]_iCORE_digit_out_reg[1]_iCORE_digit_out_reg[2]
_iCORE_digit_out_reg[3]/Q3N
```

write_netlist

```
write_netlist [design|module]...
[-suffix string]
[-abstract] [-generic] [-depth integer]
[-equation] [-pg] [-language {v1995|v2001}]
[-lec] [-use_eeq_cell_with_liberty_info][-exclude_ilm] [ > file]
```

Description

Generates one of the following design implementations in Verilog format:

- A structural netlist using generic logic
- A structural netlist using mapped logic

You can automatically read in or write out a gzip compressed Verilog file.

Options and Arguments

`-abstract`	Generates an empty top-level Verilog module definition of the specified design or module that defines the I/O pins and bit-width for all top-level functional and scan-related ports in the design or module. This empty module description is further referred to as *logic abstract model*.
`-depth` *integer*	Specifies the number of hierarchy levels to be written out, starting from the top level. A value of 0, writes out only the top-level module.
	Default: `infinite`
{*design* | *module*}	
	Specifies the design or module for which the design implementation must be generated.
`-equation`	Writes out a logic equation in an assign statement for each Verilog primitive gate.

`-exclude_ilm`	Specifies to skip writing out Verilog for ILM modules.
file	Specifies the file to which the output must be written.
	Default: Output is written to the screen.
`-generic`	Generates an unoptimized generic logic implementation of the design that uses the generic logic gates specified within the Verilog language. Any parts of the design that are mapped will be unmapped for the `write_hdl` command without affecting the design in memory.
	Once the synthesize commands (`syn_generic`, `syn_map`, or `syn_opt`) have been run, you cannot recover the version of the design that was generated using this option prior to synthesis.
`-language {v1995| v2001}`	
	Specifies the language of the output Verilog file. In `v2001`, the tool writes out the Verilog-2001 attributes stored with instances, ports and subports in the design.
	Default: `v1995`
`-lec`	Generates an *intermediate* netlist with additional information to facilitate formal verification with Conformal® Equivalence Checking.
`-pg`	Adds the power and ground nets to the netlist, where PG pins, net connections have come through 1801, UPF files.
	If the PG pins are coming from a Liberty file, and are connected in RTL, they will be written out by default in the Verilog netlist generated by Genus.
`-suffix`	Specifies the string to be appended to the name of all defined modules in the generated netlist.
`-use_eeq_cell_with_liberty_info`	
	Specifies whether the Verilog Writer should write the eeq master equivalent or not.

Related Information

Writing Out the Design Netlist in *Genus User Guide*.

Creating a Logic Abstract Model in *Genus Design for Test Guide*

Affected by these commands:

elaborate
syn_generic
syn_map
syn_opt

Affected by these attributes:

optimize_merge_flops
optimize_merge_latches
optimize_merge_seq
write_sv_port_wrapper
write_vlog_bit_blast_bus_connections
write_vlog_bit_blast_constants
write_vlog_bit_blast_mapped_ports
write_vlog_bit_blast_tech_cell
write_vlog_convert_onebit_vector_to_scalar
write_vlog_declare_wires
write_vlog_empty_module_for_logic_abstract
write_vlog_line_wrap_limit
write_vlog_no_negative_index
write_vlog_port_association_style
write_vlog_preserve_net_name

write_vlog_top_module_first
write_vlog_unconnected_port_style
write_vlog_wor_wand

write_sdc

```
write_sdc
[-version {1.1|1.3|1.4|1.5|1.7|2.0|2.0genus}][-dc]
[-strict] [-rtl] [-no_split] [-exclude cmd_list]
[-view analysis_view] [-time_unit] [-cap_unit]
[-constraint_mode constraint_mode] [design] [> file]
```

Description

Writes out the current design constraints in Synopsys Design Constraint (SDC) format. The `write_sdc` command can also compress the SDC constraints with gzip (`.gz` extension).

When using the `write_sdc` command, Genus replaces the / character with the @ character when the `/` character is used in the name of objects. This could happen when the design is ungrouped or when the / character is used as the `ungroup_separator`.
To prevent this problem, write out the constraints using an SDC version less than 1.3 to avoid the hsc specification. For example: `genus:root:> write_sdc -version 1.1`.

For those SDCs that are not supported, Genus will issue a warning message but store them for output for the `write_sdc` command. Genus will only store the SDCs and not manipulate any data with them.

> Using the `write_sdc` command may not capture all the design information necessary to recreate the image of a design's constraints.

Options and Arguments

-cap_unit	Writes out the constraints in the specified capacitance units.
`-constraint_mode` *`constraint_mode`*	
	Specifies the name of the constraint mode that the SDC constraints apply to.
	A constraint mode groups a set of constraints to be used with a specific analysis view.
	The constraint mode must have been created with the `create_constraint_mode` command.
`-dc`	Specfies to write the constrains in DC language instead of TCL.
`design`	Specifies the name of the design for which to write the SDC constraints.
`file`	Specifies the name of the file to which to write the SDC constraints.
`-exclude` *`cmd_list`*	Specifies the commands that must not be written to the SDC file.
`-no_split`	Prevents printing the SDC commands over several lines.
-rtl	Writes only the DFT-RTL flow specific SDC constraints.
`-strict`	Writes out commands that are specifically listed in the SDC specification. If you do not use this option, the `write_sdc` command outputs commands that are DC and PT compatible, which means that commands not listed in the SDC specification may be written out.
-time_unit	Writes out the constraints in the specified time units.
`-version {1.1|1.3|1.4|1.5|1.7|2.0|2.0genus}`	
	Specifies the SDC version to use.
	Default: `2.0`
`-view` *`analysis_view`*	Specifies the analysis view, the SDC constraints apply to.

Related Information

Affected by this command:

read_sdc

write_sdf

```
write_sdf [-version {OVI 3.0 | OVI 2.1}]
[-precision non_negative_integer]
[-timescale {ps | ns}] [-delimiter character]
[-celltiming {all | none | nochecks}]
[-interconn {port | interconnect}] [-no_empty_cells]
[-edges {edged | check_edge}] [-condelse]
[-nonegchecks] [-no_escape] [-nosplit_timing_check]
[-no_input_port_nets] [-no_output_port_nets]
[-recrem [merge_always | merge_when_paired | split]
[-setuphold [merge_always | merge_when_paired | split]
[-view analysis_view] [-design string] [-exclude_ilm][> file}
```

Description

The command writes a Standard Delay Format (SDF) file that analysis and verification tools or timing simulation tools can use for delay annotation. The SDF file specifies the delay of all the cells and interconnects in the design in the Standard Delay Format. Specifically, it includes the delay values for all the timing arcs of a given cell in the design.

The `write_sdf` command can generate both plain-text and GNU zipped (.gz) SDF files.

By default, the `write_sdf` command uses the delays already cached by the system and writes only values of the triplets.

By default, non-sequential checking of arcs is disabled. However, you can enable them by setting these two attributes:

```
set_db timing_disable_library_data_to_data_checks 0
```

```
set_db timing_disable_non_sequential_checks 0
```

Use the `write_sdf` command after technology mapping (after the `syn_map` command).

Options and Arguments

`-celltiming {all | none | nochecks}`	
	Specifies the cells delays and timing checks to write out.
	`all`—Writes all cell delays and timing checks to the SDF file.
	`none`—Prevents cell delays and timing checks from being written into the SDF file.
	`nochecks`—Disables the writing of timing checks only.
	Default: `all`
`-condelse`	Writes `CONDELSE` constructs with the default value when a `COND` construct is written out.
	Use this argument with the `-version` 3.0 option.
`-delimiter` *character*	
	Specifies the hierarchy divider character to be used in the SDF file. The valid options are the "`/`" and "`.`" characters.
	This option sets the hierarchy divider of only the `write_sdf` command. If omitted, the default hierarchy divider character (.) is used as the `write_sdf` default.
`-design` *string*	Specifies the design name for which the SDF file has to be generated.
`-edges {edged | check_edge}`	
	Specifies whether to add edge specifiers on combinational arcs.
	`check_edge`—Keeps edge specifiers on timing check arcs but does not add edge specifiers on delay arcs.
	`edged`—Keeps edge specifiers on timing check arcs as well as delay arcs.
	Default: `edged`

`-exclude_ilm`	For ILM flow, it specifies to only dump out nets between top and child ports.
file	Specifies the name of the SDF output file. If omitted, this argument defaults to stdout.
`-interconn {port | interconnect}`	
	Specifies the construct to use for writing out net delays.
	`port`—Writes out the net delays using the `PORT` construct.
	`interconnect`—Writes out the net delays using the `INTERCONNECT` construct.
	Default: `port`
`-no_escape`	Writes out object names without escaping hierarchical dividers. Normally, when writing out an SDF, the hierarchical divider (/) is escaped because it is a special character.
	You can set `timing_write_sdf_no_escape_backslash` timing attribute to control the handling of escape characters when `write_sdf -no_escape` parameter is specified.
`-nonegchecks`	Specifies that negative checks are not allowed for `SETUP/HOLD/RECOVERY/REMOVAL`. Converts all negative timing check values to `0.0`.
`-no_empty_cells`	Suppresses writing out empty cell descriptions. This option should be used with `-interconn interconnect`
`-no_input_port_nets`	
	Suppresses writing out nets connected to input ports.
`-no_output_port_nets`	
	Suppresses writing out nets connected to output ports.
`-nosplit_timing_check`	
	Does not split the `TIMINGCHECK` delays (`SETUP/HOLD/RECOVERY/REMOVAL` delays) for rise and fall transitions. Instead, the maximum delay values are used.

`-precision` *non_negative_integer*	
	Controls the number of digits appearing after the decimal point in the output SDF file. The default precision is the value of the `timing_report_precision`.
`-recrem [merge_always | merge_when_paired | split]`	
	Determines how `RECOVERY` and `REMOVAL` timing checks are written.
	`merge_always`—Always writes combined `RECREM` checks, even if either of the `RECOVERY` or `REMOVAL` check does not exist.
	For example, if only a `RECOVERY` check exists, the tool writes a combined `RECREM` check in the SDF file as follows:
	`RECREM (value) (value) () ()`
	`split`—Splits the `RECREM` checks into a `RECOVERY` check and a `REMOVAL` check.
	`merge_when_paired`—Writes combined `RECREM` checks only when both `RECOVERY` and `REMOVAL` checks exist.
	For example, if both a `RECOVERY` and a `REMOVAL` check exist, the tool writes a combined `RECREM` check in the SDF file as follows:
	`RECREM (value) (value) (value) (value)`
	However, if only the RECOVERY check exists, the tool only writes the `RECOVERY` check in the SDF file:
	RECOVERY (value) (value)
	Default: `split`
`-setuphold [merge_always | merge_when_paired | split]`	
	Determines how `SETUP` and `HOLD` timing checks are written.

	`merge_always`—Always writes combined `SETUPHOLD` checks, even if either of the `SETUP` or `HOLD` check does not exist.
	For example, if only a `SETUP` check exists, the tool writes a combined `SETUPHOLD` check in the SDF file as follows:
	`SETUPHOLD (value) (value) () ()`
	`split`—Splits the `SETUPHOLD` checks into a `SETUP` check and a `HOLD` check.
	`merge_when_paired`—Writes combined `SETUPHOLD` checks only when both `SETUP` and `HOLD` checks exist.
	For example, if both a `SETUP` and a `HOLD` check exist, the tool writes a combined `SETUPHOLD` check in the SDF file as follows:
	`SETUPHOLD (value) (value) (value) (value)`
	However, if only the SETUP check exists, the tool only writes the `SETUP` check in the SDF file:
	SETUP (value) (value)
	Default: `split`
`-timescale {ps | ns}`	
	Specifies a multiplier to the delay values used in the SDF file. This option does not change the timescale setting of the SDF file. The delay values are scaled, but the units are the same.
	Default: `ps`
`-version {OVI 3.0 | OVI 2.1}`	
	Specifies whether to generate SDF version 2.1 or 3.0.
	Default: OVI 3.0
`-view` *analysis_view*	

	Uses the late delay from the specified analysis view to populate the SDF type slot, when in multi-mode multi-corner analysis mode.

Table -1 Definition of -edges Option Values

	edged value	library value	noedge value
IOPATH from a unate input pin to output pin	No edge specifier	No edge specifier	No edge specifier
IOPATH from a non-unate input pin to output pin	Edge specifier at the input pin	Edge specifier at the input pin when sdf_edges is either "both_edges" or "start_edge"	No edge specifier when merging edged paths, the maximum values are used for MAX and TYP fields; minimum values are used for MIN fields.
IOPATH of a clock/enable pin to data out pin	Edge specifier at the input pin	Edge specifier at the input pin when sdf_edges is either "both_edges" or "start_edge"	No edge specifier

Timing checks	Edge specifiers match the check arcs specified in the timing library	Overridden by value of the `sdf_edges` attribute. For `both_edges`, the timing checks will contain edge specifier on both starting and ending pins. For `start_edge`, the starting pin (for HOLD type check) or ending pin (for SETUP type check) is edge-specified. For `end_edge`, the ending pin (for HOLD style check) or the starting pin (for SETUP type check) is edge-specified. The timing checks are duplicated or combined appropriately. The `start_edge` and `end_edge` are interpreted as first and second events, respectively, of the Verilog-XL's $hold() and $setup system tasks.	Only the edge specifiers of the reference pin are written.

Examples

- The following command writes out an SDF 2.1 compliant output file named my.sdf with 4 digits after the decimal point:

  ```
  write_sdf -version 2.1 -precision 4 my.sdf
  ```

- The following example SDF files compare the output of -edges edged to the output of -edges noedge:
  Resultant SDF with -edges set to edged:

  ```
  (HOLD (posedge D) (posedge CK) (-0.14:-0.14:-0.14)) (HOLD (negedge D)
  (posedge CK)(-0.06:-0.06:-0.06)) (SETUP (posedge D) (posedge CK)
  (0.22:0.22:0.22)) (SETUP (negedge D) (posedge CK) (0.41:0.41:0.41))
  ```

  With `-edges` set to noedge only the edge specifiers of the clock pin are written.:

  ```
  (HOLD D(posedge CK) (-0.14:-0.06:-0.06))SETUPD (posedge CK)
  (0.22:0.41:0.41))
  ```

Related Information

Related attributes:

timing_disable_library_data_to_data_checks
timing_disable_non_sequential_checks

write_snapshot

```
write_snapshot -directory string -tag string
[-innovus] [-hierarchical]
[-split_db] [-no_gzip] [-levels_of_logic] [design]
```

Description

Generates design reports and database snapshots.

Options and Arguments

design	Specifies the design name on which to execute the `write_snapshot` command.
`-directory` *string*	Specifies the path to the directory where the output data will be stored.
`-hierarchical`	Writes out additional information for the hierarchical flow using interface logic models (ILMs).
`-innovus`	Specifies to write data for Innovus.
`-levels_of_logic`	Prints the number of gates on the critical path per cost group in the QoR report.
`-no_gzip`	Prevents that the generated files are compressed in gzip format.
`-split_db`	Writes out the setup information in a setup script and the netlist information into a setup-free database.
`-tag` *string*	Specifies a tag name for reports and design data generated at this stage.

Example

The following example writes out the basic output of the command:

```
write_snapshot -dir snapshot -tag counter.elab
```

```
This summary table is written to the logfile:


QoS Summary for
counter=================================================================================Metric
counter.elab============================================================================Slack
(ps):                   -714  R2R (ps):                    606  I2R (ps):                   -568
R2O (ps):                   -714  I2O (ps):               no_value  CG  (ps):
no_valueTNS (ps):                  8,594  R2R (ps):                     0  I2R (ps):
5,111  R2O (ps):                  3,483  I2O (ps):               no_value  CG  (ps):
no_valueFailing Paths:                  19Cell Area:                  3,871Total Cell Area:
3,871Leaf Instances:                122Total Instances:               122Utilization (%):
73.70Tot. Net Length (um):     no_valueAvg. Net Length (um):     no_valueRoute Overflow H (%):
0.00Route Overflow V (%):         0.00MBCI(%) (bits/gate) :         0.00Norm Cong Hotspot Area:
Max Cong:               no_value  Tot Cong:
no_value================================================================================CPU
Runtime (h:m:s):     00:00:10Real Runtime (h:m:s):     00:01:57CPU  Elapsed (h:m:s):
00:00:16Real Elapsed (h:m:s):     00:02:00Memory (MB):
581.61==========================================================================================
=============================================================Flow
Settings================================================================================Total
Runtime (h:m:s):    00:01:58Total Memory (MB):          582.61Executable Version:        versio
n=============================================================================Total Cell Area =
Cell Area + Physical Cell AreaTotal Instances =  Leaf Instances + Physical Instances


The following files are written to the snapshot
directory:counter.elab_area.rptcounter.elab_counter.dbcounter.elab_gates.rptcounter.elab_qor.rpt
counter.elab_time.rptfinal.rpt


Note: For all Genus flows, except logical-only flow, "Cell Area" mentioned in the summary table
points to the "physical_cell_area" attribute of the design.
```

Related Information

Related command:

write_db

write_sv_wrapper

```
write_sv_wrapper
[-continue_on_error]
[-module_name string] [-wrapper_name string]
[-exclude_type_definition]
[-rename_module] [design | module] [> file]
```

Description

Writes out a wrapper SystemVerilog module for a design or module. Such a wrapper module can help in a comparative simulation of the output netlist from Genus and the input RTL design, especially when the design description in the input RTL has complex ports.

When you synthesize a design `test` that has complex ports in the input RTL description, the `write_sv_wrapper` command writes out the definitions of the complex port types, as well as a System Verilog description of module `test` which has the same complex ports as the original RTL description of `test`. The module's test body consists of a single instantiation of another module named `test%s` where %s is the suffix specified with the `-module_suffix` option.

`write_sv_wrapper` is also used in Bottom up synthesis flow. If a sub-block has complex ports (e.g structure port or multidimensional array port), then in its pre-synthesized netlist, these ports may be represented by multiple ports, one each for each basic leaf field of the complex port. Synthesis process needs to be able to match each complex port name in instantiation with the corresponding multiple ports in the pre-synthesized netlist. A wrapper SV module is included in the netlist with the same name as the sub-block, its body consists of one instance of the pre-synthesized sub-block module using the modified name. This is done through the `write_sv_wrapper` command.

The ports of the wrapper SV module match, in type and complexity with the original definition of the sub-block in RTL.

> Wrapper is generated only for System Verilog design with ports of type struct, array, or interface.

Options and Arguments

`-continue_on_error`	
	Prevents returning an error code even if an error occurred during wrapper creation.
{*design*|*module*}	
	Specifies the design or module for which the wrapper SystemVerilog module must be written.
`-exclude_type_definition`	
	Prevents to write out complex types (structs, unions, interfaces, and so on) for the wrapper module.
	If SV wrappers are written out with included type definitions (which is the default nature of `write_sv_wrapper`), Genus complain of re-declaration of type definitions since both SV wrappers and top design contain the same type definitions.
file	Specifies the file to which to write the wrapper SystemVerilog module.
`-module_name` *string*	
	Specifies the suffix to append to the name of the instantiated module.
`-rename_module`	Renames the main design module and uses the value of the `-module_name` option as suffix to the design name.
`-wrapper_name` *string*	
	Specifies the suffix to append to the module name of the wrapper module.

write_template

```
write_template
[-dft] [-power] [-cpf] [-retime]
[-physical] [-performance]
[-area] [-no_sdc] [-n2n] [-yield]
[-full] [-simple] [-split]
[-multimode] -outfile file
```

Description

Generates a user-friendly template script with the commands and attributes needed to run Genus. Use the command options to include specific commands and attributes in the script.

Options and Arguments

`-area`	Writes out a template script for area-critical designs.
`-cpf`	Writes out a template script for the Common Power Format (CPF) based flow and a template CPF file (`template.cpf`). The template CPF file is read in the template script with the `read_cpf` command.
`-dft`	Writes out a template script for the test synthesis (DFT) flow.
	The template contains commands and attributes needed for basic and advanced DFT features.
`-full`	Writes out DFT, power, and retiming commands and attributes along with the basic template.
`-multimode`	Writes out a template script for multi-mode analysis.
`-n2n`	Writes out the template script for netlist to netlist optimization in Genus. Use with the `-dft` and `-power` options to include the DFT and power attributes and commands.
`-no_sdc`	Writes out clock delays and input and output delays in the Genus format.

`-outfile` *string*	Specifies the name of the file to which the template script is to be written.
`-performance`	Writes out a template script which enables some advanced optimization algorithms that can improve QoR and that have an impact on the runtime.
`-physical`	Writes out a template script for the physical flow.
`-power`	Writes out power attributes and commands.
`-retime`	Writes out retiming attributes and commands.
`-simple`	Writes out a simple template script.
	You cannot use this option with the `-split`, `-area`, `-dft`, `-cpf`, `-multimode`, `-physical`, `-power`, `-retime`, or `-full` options.
`-split`	Writes out a template script with a separate setup file which contains the root attributes and setup variables.
	If you specified the `-dft` option with the `-split` option, an additional file is created which contains the DFT design attributes, test clock and scan chain information.
	If you specified the `-power` option with the `-split` option, an additional file is created which contains the leakage and dynamic power, and clock-gating setup information.
`-yield`	Writes out a template script for yield.

Examples

- The following example writes out the basic template file with the constraints in SDC format:

  ```
  write_template -outfile template.g
  ```

- The following example writes out the basic template file with the constraints in the Genus format:

  ```
  write_template -no_sdc -outfile template.g
  ```

- The following example adds both the DFT and power related attributes to the template.g file written out:

  ```
  write_template -dft -power -outfile template.g
  ```

- The following example writes out the template script with the DFT attributes and commands for netlist to netlist optimization:

  ```
  write_template -dft -n2n -outfile template.g
  ```

- The following example writes out the template script *template.g* and the setup file *setup_template.g* that contains all the root attributes and setup variables and includes it in the *template.g* file:

  ```
  write_template -split -outfile template.g
  ```

- The following example writes out the *template.g* template script, a setup_*template.g* setup file, a dft_*template.g* file, and a power_*template.g* file and includes them in the appropriate *template.g* file:

  ```
  write_template -split -dft -power -outfile template.g
  ```

- The following example writes out a simple template script with no path and cost groups and without any variables, power, or DFT related attributes:

  ```
  write_template -simple -outfile template.g
  ```

- The following example writes out a template script for area critical designs:

  ```
  write_template -area -outfile template.g
  ```

write_tempus

```
write_tempus [-default] [-ocv] [-pre_lib_include string]
[-pre_include string] [-post_include string]
[-netlist string] [-libs string] [-no_exit]
[-sdc string] [-sdf string] [-spef string]
[> file]
```

This command is obsolete for standalone use and will be obsolete in the next major release. (TUI-37). Use `write_db -common` to exchange with Innovus and Tempus.

Description

Generates a Tempus or Encounter® Timing System (ETS) run script.

Options and Arguments

`-default`	Generates a simple ETS run script. The script will contain the following ETS commands: `read_lib`, `read_verilog`, `set_top_module`, `read_sdc`, and `report_timing`.
file	Redirects the output to the specified file.
`-libs` *string*	Specifies the library paths.
`-netlist` *string*	Specifies the UNIX path of the file containing the gate-level netlist.
`-no_exit`	Suppresses the `exit` command at the end of the run script.
`-ocv`	Adds an extra `set_timing_derate` command into the ETS run script. This option can only be specified with the `-sdf` option.
`-post_include` *string*	
	Specifies the UNIX path of the include file that contains ETS commands that need to be added after the `report_timing` command in the run file generated by `write_ets`.
`-pre_include` *string*	
	Specifies the UNIX path of the include file that contains ETS commands that need to be added before the `report_timing` command in the run file generated by `write_ets`.
`-pre_lib_include` string	
	Specifies the UNIX path of the include file that contains ETS commands that need to be added before the libraries are read in the run file generated by `write_ets`.
`-sdc` *string*	Specifies the UNIX path of the SDC file.
`-sdf` *string*	Specifies the UNIX path of the SDF file. If this option is specified, the `read_sdf`, `set_analysis_mode`, and `set_op_cond` commands will be added to the ETS run script.
`-spef` *string*	Specifies the UNIX path of the SPEF file. If this option is specified, the `read_spef`, `set_analysis_mode`, and `set_op_cond` commands will be added to the ETS run script.

15

Joules

- get_lib_domains
- report_sdb_annotation
- write_icgc_monitor

get_lib_domains

```
get_lib_domains
[-name string] [-invert] [-nocase]
```

Description

Returns the full path to any library domain(s) that matches the specified name.

Options and Arguments

`-nocase`	Indicates that the name is not case sensitive.
`-name string`	Specifies the name to match with the domain names. Wildcards are supported.
`-invert`	Filters out domains that match the name and return those domains that do not match the given name.

Examples

- `get_lib_domains` # get list of ALL library domains
- `get_lib_domains -name "*1p2*"` # get a LVT/HVT library
- `get_lib_domains -name "1P2" -nocase -invert` # get a LVT/HVT library

report_sdb_annotation

```
report_sdb_annotation
[-stims string...] [-inputs]
[-show_details obj_type[:annotation_type[:pin_prop_type]]]
[object]
```

Description

Reports the annotations on a design found in the Joules stimulus database (SDB).

Options and Arguments

`-inputs`	Reports the annotation of inputs pins. Displays inputs of flops, memories, latches, and integrated clock-gating instances (icgc).
object	Reports the annotation of the specified design or instance.
	Default: top design.
`-show_details` *obj_type*`[:`*annotation_type*`[:`*pin_prop_type*`]]`	
	Reports annotation details for the specified object, annotation type, and pin type set.
	object_type can be one of the following:
	`{all |port | port_dft | seq | comb | reg | reg_dft | mem | mem_dft | icgc | latch | bbox | pad | pm | mux | drivers | rtl_drivers}`
	Default: `all`
	annotation_type can be one of the following:
	• after `read_stimulus` and before `compute_power`
	`{asserted | unasserted | unconnected | all}`
	• after `compute_power` or `propagate_activity`
	`{all | asserted | user_asserted | computed | default |clock_source | constant | non_toggle | unasserted |unconnected}`
	Default: `all`
	pin_prop_type can be one of the following:
	`{out | in | clock | enable | all}`
	Default: `out`
`-stims` *string*	Reports the annotation of the given stims.
	Default: all stims are considered.

Examples

- `report_sdb_annotation -stims /stim#1 -show_details seq:computed:out`
- report_sdb_annotation -stims /stim#1 -show_details reg:asserted:clock

write_icgc_monitor

```
write_icgc_monitor
[-out file] [-dut_instance name]
[-max_expr_rtl_signal_count string]
```

Description

Creates a System Verilog module file for the enable signals of the clock-gating instances that were inserted during synthesis. This file can then be used during simulation to generate the activity on these enable pins.

You must have run generic synthesis before you can use this command.

> To run this command, you will need a `Joules_RTL_Power` or `Joules_Power_SP` license.

Options and Arguments

`-dut_instance` *name*	Specifies the instance name of the design under test in the testbench.
`-max_expr_rtl_signal_count` *string*	
	Specifies the maximum number of RTL signals allowed.
`-out` *file*	Specifies the name of the output file.

Related Information

Related command:

syn_generic

16

Low Power Synthesis

- add_clock_gates_obs
- add_clock_gates_test_connection
- declone_clock_gate
- delete_clock_gate
- incr_read_1801
- merge_clock_gate
- read_saif
- read_tcf
- read_vcd
- rebalance_clock_gating
- share_clock_gate
- split_clock_gate
- state_retention
- update_clock_gate
- write_saif
- write_tcf
- write_top_power_intent

The following command is common with Analysis and Reporting commands:

- report_power

add_clock_gates_obs

```
add_clock_gates_obs
[-hierarchical] [-make_obs_module]
[-max_cg integer]
[-ignore_clock_constraint]
[-exclude {inst|hinst}...] [-disable_clock] [-libcell libcell]
```

Description

Inserts and connects circuitry to improve the observability of the design after clock-gating logic is inserted. This command applies to the current design or the current hierarchical instance.

> To make sure that the enable signal of the clock-gating logic is observable, set the `lp_clock_gating_add_obs_port` design attribute to `true` before you insert the clock-gating logic.

Observability logic is inserted based on clock information. The clock information is required because only clock-gating logic driven by the same clock can share an observation flip-flop. The clock information can be derived from clock constraints or from the physical connectivity.

> This command works on a netlist whose clock-gating logic was either inserted or imported by the Genus-LP engine.

Options and Arguments

`-disable_clock`	Specifies to gate the clock of the observability flip-flops.
	One gating cell is inserted per flip-flop. The Genus-LP engine creates a separate module for each gating cell.
`-exclude {`*`inst`*`|`*`hinst`*`}`	
	Prevents insertion of observability logic in the specified hierarchical instances' list.
`-hierarchical`	Allows insertion and connection of observability logic in the current level of the hierarchy and all its children.
	By default, this command only affects the current level of the hierarchy.
`-ignore_clock_constraint`	
	Inserts observability logic based on physical connectivity.
	By default, observability logic is inserted based on clock constraints.
`-libcell` *`libcell`*	Specifies the name of a library cell to be used for gating.
	You can only specify an AND cell to gate the observability logic.
`-make_obs_module`	Creates a separate hierarchy (module) for each observation flip-flop and its associated XOR tree.
`-max_cg` *`integer`*	Specifies the maximum number of clock-gating cells that can be observed per observation flip-flop. Specify an integer between 1 and 32.
	Default: `8`

Related Information

Affected by this attribute:

lp_clock_gating_add_obs_port

add_clock_gates_test_connection

```
add_clock_gates_test_connection
[-detail] [-ignore_preserve]
[-clock_gate_types string]
```

Description

Globally connects the test input of all clock-gating logic to the test signal specified through the `lp_clock_gating_test_signal` attribute and marks this network as ideal. This command applies to the current design or the current hierarchical instance.

If the clock-gating test input is already connected, the command has no effect.

> This command works only on a netlist whose clock-gating logic was inserted by the Genus-LP engine. If clock-gating was inserted by a third party tool, the command update_clock_gate needs to be run prior to `add_clock_gates_test_connection` command.

Options and Arguments

`-detail`	Specifies to print out names of all affected clock gates at the end of the command.
`-ignore_preserve`	Specifies to ignore any kind of preserve status during test connection.
`-clock_gate_types` *string*	
	Specifies the types of clock gates that will be considered for test connection.

Related Information

Related commands:

update_clock_gate

Affected by the attribute:

lp_clock_gating_test_signal

declone_clock_gate

```
declone_clock_gate
[-hierarchical][-no_clock_tree_traversal]
[-verbose][-effort {high|low}] [-clock_gate_type]
```

Description

If the clock-gating logic of different registers in the design uses the same enable signal, Genus can merge these clock-gating instances for any such identically gated registers

The Genus-LP engine automatically removes any dangling ports.

> This command works only on a netlist whose clock-gating logic was inserted by the Genus-LP engine. It will not work on the clock gating logic created by other third party tools. After decloning, it will create the new instance named RC_CG_DECLONE_HIER_INST out of RC_CG_HIER_INSTs in the netlist.

The logfile also shows the information such as:

```
Info : Clock-gating instances are decloned. [POPT-56]
: New decloned instance is gdfcc_0/gdhb_data_out_ctrl_0/RC_CG_DECLONE_HIER_INST

     Old instances were:

     gdfcc__0/gdfcc_b_data_out_ctrl_0/RC_CG_HIER_INST19
gdfcc__0/gdfcc_b_data_out_ctrl_0/RC_CG_HIER_INST24

     Clock-gating declone status
```

```
==========================
Total number of clock-gating instances before: 128
Total number of clock-gating instances after : 64
```

Check for the [POPT-56] messages in the logfile to see if the decloning has happened or not.

Options and Arguments

`-clock_gate_type`	Specifies the types of CGs to rebalance. Default: synthesis
`-hierarchical`	Allows traversing the design hierarchy to search for clock-gating instances that can be merged. As a result, new ports can be added for the gated-clock signal.
	By default, this command only affects instances at the current level of the hierarchy.
`-no_clock_tree_traversal`	
	Prevents traversing through buffers and inverter pairs on the clock signal.
	If you do not set this option, Genus can remove buffers or inverter pairs on the clock path during optimization unless the buffers or inverter pairs are marked preserved.
`-verbose`	Gives detailed information.
`-effort {high|low}`	Specifies the effort level.
	High effort mode enables better equivalency detection but may result in longer runtime. Default effort level is low.

Related Information

Affected by this attribute:

lp_clock_gating_max_flops

delete_clock_gate

```
delete_clock_gate
[ -hierarchical] [-obs_only]
| -cg_list {inst|hinst}... ] | -flops {inst|hinst}...] [ -no_verbose] [ -effort {low|high}]
```

Description

Removes clock-gating logic inserted by the Genus-LP engine from the current design or the current hierarchical instance.

> This command works on a netlist whose clock-gating logic was either inserted or imported by the Genus-LP engine.

Options and Arguments

`-cg_list {`*`inst`*`|`*`hinst`*`}`	
	Specifies a list of clock-gating instances to be removed. Use a full path name to identify these instances.
	If you specify a clock-gating instance with an incomplete path, the tool searches for that instance from the root of the design hierarchy and might select multiple instances with the same name from different hierarchies.
`-effort {high|low}`	Specifies the effort level.
	Choosing low effort results in better runtime performance but at the cost of an area increase. In this case, the Genus-LP engine reconstructs the original MUX and feedback loop from the flip-flop to the MUX.
	For large designs high effort can result in long runtimes, but the feedback logic is optimized.
	Default: `low`
`-flops {`*`inst`*`|`*`hinst`*`}`	Removes clock gating from the specified flops (that is, recreates the feedback loop for those flops).
	If you specified all flops that are gated by the same clock-gating instance, the clock-gating instance will also be removed.
`-hierarchical`	Removes all clock-gating logic in the hierarchy of the current design or module.
	By default, this command only affects the current level of the hierarchy.
`-no_verbose`	Suppresses info and warning messages.
`-obs_only`	Removes only the observability logic.

incr_read_1801

```
incr_read_1801
[-design design] string...
```

Description

Updates the 1801 constraints in the design.

Options and Arguments

`-design` *design*	Specifies the design to which the power intent applies.
string	Specifies a list of files with updated 1801 constraints.

Related Information

Clock Gating in Genus Low Power Guide.	
Related commands:	check_power_intent
	commit_power_intent
	read_power_intent
	write_power_intent

merge_clock_gate

```
merge_clock_gate
[-hierarchical] [-max_level integer]
[-multi_fanouts] [-start_from object]
```

Description

Combines multiple stages of clock-gating logic into a single clock gating instance with a complex enable function.

> This command works on a netlist whose clock-gating logic was either inserted or imported by the Genus-LP engine.

Options and Arguments

`-hierarchical`	Allows joining of clock-gating logic down the hierarchy starting from the current directory.
	By default, this command only affects the current level of the hierarchy.
`-max_level` *integer*	Specifies the maximum levels of clock-gating instances that can be combined. If you specify n, n+1 stages can be combined.
	Default: `1` - allowing 2 levels to be combined.
`-multi_fanouts`	Allows joining even if the root stage clock-gating instance is driving multiple clock-gating instances.
`-start_from` *object*	
	Joins the clock-gating logic starting from the specified hierarchical instance.

Example

The following example shows merging clock gates starting from hierarchy u1, allowing up to 3 levels clock gates:

```
genus@root: XX> merge_clock_gate -hierarchy -start_from u1 -max_level 2
```

read_saif

```
read_saif [-scale scale_factor]
[-update [-weight weight_factor]]
[-verbose] [-ignorecase] [-instance instance]
[-scale_to_sdc_frequency] file
```

Description

Reads switching activity information in Synopsys switching activity interchange format (SAIF) and converts it internally to the Toggle Count Format (TCF) for power estimation.

The `read_saif` command can read files that have been compressed with gzip (`.gz` extension). The `.gz` file is unzipped in memory while the file is read in.

> If you read in subsequent SAIF files without the `-update` option, only the probability values and toggle counts of the pins and nets in the current SAIF file are overwritten. The other net values remain unchanged.

The following applies when *updating* the probability values and toggle counts:

- If the probability values and toggle rates were not previously user asserted, the updated probability and toggle rates are determined by the values specified in the SAIF file.
- If the probability and toggle rates were previously user asserted, the new probability and toggle rates are calculated as follows:

  ```
  prob_new = (prob_old + w * prob_spec)/(1+w)
  ```

  ```
  tr_new = (tr_old + w * tc_spec/duration_spec)/(1+w)
  ```

  where `prob_old` and `tr_old` are the stored values, and `prob_spec`, `tc_spec`, and `duration_spec` are the probably, toggle count, and duration values derived from the new SAIF file.

Options and Arguments

file	Specifies the name of the SAIF file. The file can have any name, suffix, or length.
`-ignorecase`	Ignores the case of module, net, and pin names in the SAIF file when searching for the matching module, net, or pin in the design.
	By default, case is taken into account.
	Using this option might result in increased run time.

`-instance` *instance*	Reads in the switching activities for the specified instance. The instance name can refer to an instance in the design loaded in Genus, or can refer to an instance name in the SAIF file.
	• If the instance name refers to an instance in the design loaded in Genus, the Genus-LP engine asserts switching activities on that instance in the loaded design.
	> In this case, a *complete* design is loaded in Genus. However, the SAIF file is incomplete and contains only switching activities for the specified instance.
	• If the instance name refers to an instance in the SAIF file, the Genus-LP engine asserts switching activities on the design loaded in Genus.
	> The design loaded in Genus is
	◦ a *partial* design if the top-level instance in the SAIF file corresponds to the full design, while the *specified* instance is a lower-level instance.
	◦ the full design if the *specified* instance in the SAIF file corresponds the top-level design scope in the SAIF file.
	In this case, the SAIF file is a hierarchical SAIF and contains switching activities for the full design.
	The name of the instance in the SAIF file does not need to match the name of the design.
`-scale` *scale_factor*	
	Scales the toggle counts in the SAIF file by dividing them by the specified factor. Use a positive (non-zero) floating number.
	Default: `1.0`

`-scale_to_sdc_frequency`	
	Specifies to scale activities to SDC activities.
`-update`	Indicates that you are updating the probability values and toggle counts.
`-verbose`	Prints a message for each net that is asserted.
	Default: Silent mode. Prints the percent completion messages.
`-weight` *`weight_factor`*	
	Specifies the relative weight of the probability values and toggle rates in the new SAIF file with respect to the probability values and toggle rates currently stored in the design. Use a positive floating number. This option is only valid with the `-update` option.
	Default: `1.0`

Related Information

Affects this command:

- report_power

Related command:

- write_saif

Sets these attributes:

- lp_asserted_probability
- lp_asserted_toggle_rate

Related attributes:

- lp_probability_type
- lp_toggle_rate_type

read_tcf

```
read_tcf [-scale scale_factor]
[-update [-weight weight_factor]]
[-hinst instance]
[-tcf_instance instance]
[-nocase] [-verbose] [-scale_to_sdc_frequency] file
```

Description

Reads or updates probability values and toggle counts of the pins and nets in the specified Toggle Count Format (TCF) file and stores the assertions as pin or net attributes, so they can be used for power estimation and optimization.

The `read_tcf` command can read files that have been compressed with gzip (`.gz` extension). The `.gz` file is unzipped in memory while the file is read in.

> If you read in subsequent TCF files without the `-update` option, only the probability values and toggle counts of the pins and nets in the current TCF file are overwritten. The other net values remain unchanged.

When *updating* the probability values and toggle counts, the new probability and toggle rates are calculated as follows:

```
prob_new = (prob_old + w * prob_spec)/(1+w)
```

```
tr_new = (tr_old + w * tc_spec/duration_spec)/(1+w)
```

where

- `prob_old` and `tr_old` are either the user-asserted values or the values computed using the power simulation engine.
- `prob_spec`, `tc_spec`, and `duration_spec` are the probability, toggle count, and duration values specified in the new TCF file.

> You should not execute any command (such as `update_names` or `ungroup`) that can cause changes in the name of the design objects before you read the TCF file. Otherwise, the `read_tcf` command may not find some design objects.

Options and Arguments

file	Specifies the name of the TCF file. The file can have any name, suffix, or length.
`-hinst` *instance*	
	Specifies the name of an instance in the Genus hierarchy to which the parsed TCF hierarchy (specified through the `-tcf_instance` option) corresponds.
	For example, if a *partial* design is loaded in Genus but you have a TCF file that contains switching activities for the full design, the top design in Genus will correspond to an instance in the TCF hierarchy.
	You can also have a full design loaded in Genus, but only have a partial TCF. In that case you need to specify the name of the instance in the Genus hierarchy to which the TCF file applies.
	By default, the TCF file applies to the top design in the Genus hierarchy. If multiple top designs exists, you must specify the name of the top design.
`-nocase`	Ignores the case of module, net, and pin names in the TCF file when searching for the matching module, net, or pin in the design.
	By default, case is taken into account.
	Using this option might result in increased run time.
`-scale` *scale_factor*	
	Scales the toggle counts in the TCF file by dividing them by the specified factor. Use a positive (non-zero) floating number.
	Default: `1.0`
`-scale_to_sdc_frequency`	
	Specifies to scale activities to SDC activities.
`-tcf_instance` *instance*	
	Starts parsing the TCF hierarchy from the specified instance.You can specify to start parsing from the top or from a particular instance in the TCF hierarchy.

	Default: First instance encountered is used as the top instance.
`-update`	Indicates to update the probability values and toggle counts.
`-verbose`	Prints a message for each net that is asserted.
	Default: Silent mode. Prints the percent completion messages.
`-weight` *weight_factor*	
	Specifies the relative weight of the probability values and toggle rates in the new TCF file with respect to the probability values and toggle rates currently stored in the design. Use a positive floating number. This option is only valid with `-update` option.
	Default: `1.0`

Related Information

Affects this command:

- report_power

Related command:

- write_tcf

Sets these attributes:

- lp_asserted_probability
- lp_asserted_toggle_rate

Related attributes:

- lp_probability_type
- lp_toggle_rate_type

read_vcd

```
read_vcd
[-static] [-scale scale_factor]
[-activity_profile] [-time_window time]
[-simvision] [-write_sst2 file]
[-start_time start_monitoring_time]
[-end_time end_monitoring_time]
[-hinst instance] [-vcd_scope instance]
[-nocase] [-scale_to_sdc_frequency] vcd_file
```

Description

Reads a Value Change Dump (VCD) file for power analysis. You can

- Perform static power analysis
- Build an activity profile

> If no options are specified, static power analysis is performed by default.

The `read_vcd` command can read files that have been compressed with gzip (`.gz` extension). The `.gz` file is unzipped while the file is read in.

> You should not execute any command (such as `update_names` or `ungroup`) that can cause changes in the name of the design objects before you read the VCD file. Otherwise, the `read_vcd` command may not find some design objects.

Options and Arguments

`-activity_profile`	Builds a profile of the activities for the set scope without annotating the switching activities to the design.

	By default, profiling is done for the whole design. You can limit the scope to a portion of the design by setting the `lp_dynamic_analysis_scope` attribute to `true` on those instances for which you want to build the profile.
	The Genus-LP engine captures the toggle count of objects within a given time window. The object can be a net, pin or a hierarchical instance. For a hierarchical instance, the activity will be the sum of the activities of the objects in that instance.
	By default, the `read_vcd` command performs static power analysis if neither the `-static` or `-activity_profile` option was specified.
`-end_time` *`end_monitoring_time`*	
	Specifies the time you want to end monitoring the switching activities or events. Specify a value larger than zero in picoseconds.
	By default, the activities or events are monitored till the end (last timestamp of the VCD file).
`-hinst` *`instance`*	
	Specifies the name of the hierarchical instance in the Genus hierarchy to which the parsed VCD hierarchy (specified through the `-vcd_scope` option) corresponds.
	For example, if a *partial* design is loaded in Genus but you have a VCD file that contains switching activities for the full design, the top design in Genus will correspond to an instance in the VCD hierarchy.

	You can also have a full design loaded in Genus, but only have a partial VCD. In that case you need to specify the name of the instance in the Genus hierarchy to which the VCD file applies.
	By default, the VCD file applies to the top design in the Genus hierarchy. If multiple top designs exists, you must specify the name of the top design.
`-nocase`	Ignores the case of module, net, and pin names in the VCD file when searching for the matching module, net, or pin in the design.
	By default, case is taken into account.
	Using this option might result in increased run time.
`-scale` *`scale_factor`*	
	Scales the toggle counts in the VCD file by dividing them by the specified factor. Use a positive (non-zero) floating number.
	Default: `1.0`
`-scale_to_sdc_frequency`	
	Specifies to scale activities to SDC activities.
`-simvision`	Invokes SimVision to display the activity profile.
	To use this option you need to have SimVision installed and your operating system `PATH` environment variable must include the path to SimVision.

`-start_time` *start_monitoring_time*	
	Specifies the time you want to start monitoring the switching activities or events. Specify a value larger than zero in picoseconds.
	By default, the first timestamp in the VCD file is considered as the start time to monitor.
`-static`	Calculates the switching activities of each of the nets and pins from the time you want to start monitoring the switching activities to the time you want to stop monitoring, and then stores the information as assertions on the nets and pins.
	By default, the `read_vcd` command performs static power analysis if neither the `-static` or `-activity_profile` option was specified.
`-time_window` *window*	
	Specifies the time increment, in picoseconds, in which you want the Genus-LP engine to divide the period during which the events are monitored. The specified time window must be larger than zero.
	By default, the time window is calculated based on the setting of the values of the `-start_time` and `-end_time` options.
	If the `-start_time` and `-end_time` options are not specified, the time window will correspond to the complete simulation time from the VCD file.

vcd_file	Specifies the name of the value change dump (VCD) file.		
-vcd_scope *instance*			
		-write_sst2 *file*	Specifies the prefix of the SST2 database files to generate, to view the data in other waveform viewers.

Related Information

TCF Syntax in *Toggle Count Format Reference*.	
Affects this command:	report_power
Related attribute:	lp_dynamic_analysis_scope

rebalance_clock_gating

```
rebalance_clock_gating
[-strict_min_flop_constraint][-effort high|low][-clock_gate_type]
```

Description

Regroups the flops of the clock-gating instances driven by the same inputs based on the values of the `lp_clock_gating_min_flops` and `lp_clock_gating_max_flops` attributes. As a result, the number of clock-gating instances may decrease or increase.

You can use this command after generic synthesis (when clock-gating instances are inserted) or after importing RTL clock-gating instances.

Rebalancing is performed on the full design, and on clock-gating instances inserted by Genus.

Rebalancing is done on clock-gating instances that

- are within the same hierarchy
- are not marked *preserved*
- have the same enable signal
- have the same timing exceptions
- have the same type

Options and Arguments

`-strict_min_flop_constraint`	
	Requests to strictly follow the minimum flop constraint specified by the `lp_clock_gating_min_flop` attribute.
`-effort {`*high*`|`*low*`}`	Specifies the effort level.
	High effort mode enables better equivalency detection, but may result in longer runtime. The default effort level is low.
`-clock_gate_type` { *all* | *synthesis* | *user* }	Specifies the type of CGs to rebalance. *Default*: synthesis

Related Information

TCF Syntax in *Toggle Count Format Reference*.	
Related command:	syn_generic
Related attributes:	lp_clock_gating_max_flops
	lp_clock_gating_min_flops

share_clock_gate

```
share_clock_gate
[-hierarchical] [-max_level integer]
[-max_stage {integer|string}]
```

Description

Extracts the enable function shared by clock-gating logic and inserts shared clock-gating logic with the common enable sub function as the enable signal. The resulting netlist has multiple stages of clock-gating logic.

> This command works on a netlist whose clock-gating logic was either inserted or imported by the Genus-LP engine.

Options and Arguments

`-hierarchical`	Inserts shared clock-gating logic down the hierarchy starting from the design or current hierarchical instance.
	By default, this command only affects the current level of the hierarchy.
`-max_level` *integer*	Specifies the maximum levels of logic (buffers and inverters excluded) to traverse in the enable fanin of clock-gating instances to extract the common enable function.
	Default: `5`
`-max_stage {`*integer*`|`*string*`}`	
	Specifies the maximum number of stages of shared clock-gating logic.
	To specify the same maximum number of stages for all clocks, specify an integer.
	To specify the maximum number of stages per clock, use a string. The string must have the following format:
	{{*clock integer*} {*clock integer*}...}
	If this option is not specified, no limit is applied to the number of stages.

Example

Inserting shared clock-gating logic from 'design:test'...

```
share_clock_gate -hierarchicalWarning : New clock-gating instance is created. [POPT-98]
: OBS port of this new clock-gating -instance 'inst:test/RC_CG_SHARED_HIER_L1_INST' is
not connected : Make sure to run 'clock_gating insert_obs' to connect teh OBS port of
this new clock_gating instance. Inserted 1 shared clock-gating instance.
```

split_clock_gate

```
split_clock_gate
[-hierarchical] [-max_level integer]
[-power_driven] [-start_from instance]
```

Description

Splits a single clock gating instance with a complex enable function into multiple stages of clock-gating logic.

> This command works on a netlist whose clock-gating logic was either inserted or imported by the Genus-LP engine.

Options and Arguments

`-hierarchical`	Allows splitting of clock-gating logic down the hierarchy starting from the current directory.
	By default, this command only affects the current level of the hierarchy.
`-max_level` *integer*	Specifies how many times a complex enable function can be split.
	Default: `1` allowing the complex enable function to be split into two stages.
`-power_driven`	Forces to use the signal with smallest toggle rate as the root-level enable.
	By default, the Genus-LP engine considers timing first and uses the late signal as the root-level enable.
`-start_from` *instance*	
	Splits the clock-gating logic starting from the specified hierarchical instance.

Example

Usage example:

```
split_clock_gate -max_level 2Clock-gating Split Status (round
1)=====================================================Category                        Number
Percentage----------------------------------------------------Clock-gating splitted             1
100%Clock-gating not splittedEnable non-splittable             0            0%Enable with
multiple fanouts      0            0%Non-combinational enable          0            0%Preserved
0            0%-----------------------------------------------Clock-gating before splitting
1          100%Clock-gating after splitting     2
```

```
Clock-gating Split Status (round 2)=================================================Category
Number         Percentage-----------------------------------------------------Clock-gating
splitted         1              50%Clock-gating not splittedEnable non-splittable         0
0%Enable with multiple fanouts  0               0%Non-combinational enable      1
50%Preserved                     0               0%-----------------------------------------
------Clock-gating before splitting 2             100%Clock-gating after splitting  3
```

state_retention

```
state_retention {connect_power_gating_pins | swap}
```

Description

Defines the aspects of mapping to state retention cells.

Options and Arguments

`connect_power_gating_pins`	Connects the power gating pins in a mapped netlist.
swap	Replaces sequential cells with their equivalent state retention power gating cells in a mapped netlist.

update_clock_gate

```
update_clock_gate
[-start_from {inst|hinst|design}] [-hierarchical] [-keep_name] [-detail] [-verbose]]
```

Description

Processes clock-gating instances that were either manually inserted or inserted by third-party tools to make them recognizable as clock-gating instances by the Genus-LP engine.

The command returns the total number of instances imported.

Currently, the Genus-LP engine recognizes the following structures as clock-gating instance:

- Two-input AND or NAND gates that have a clock signal driving one of the inputs

In this case, the other pin is assumed to be the enable pin.

> If the other pin is part of the test network, the Genus-LP engine does not recognize the gate as a clock-gating instance.

- Integrated clock-gating cells

Currently, the Genus-LP engine does not recognize these structures as clock-gating instances if they were defined in a separate module that is instantiated in the netlist.

The Genus-LP engine creates a new hierarchical instance (RC_CG_HIER_INST) for each clock-gating instance it recognizes and adds it to the list of clock-gating instances that the Genus-LP engine has created.

Options and Arguments

`-detail`	Prints a summary with the names of the imported clock-gating modules and the number of instances found for each clock-gating module.
`-hierarchical`	Allows traversing the design hierarchy to process clock-gating instances that were not inserted by Genus.
	By default, this command only affects instances at the current level of the design hierarchy.
`-keep_name`	Keeps the original clock-gating instance name.
`-start_from {`*`inst`*`|`*`hinst`*`|`*`design`*`}`	
	Starts processing clock-gating instances that were not inserted by Genus from the specified hierarchical instance or from the top of the design).
	By default, the process starts from the current location in the design hierarchy.
`-verbose`	Includes the list of modules that have no imported clock gates in the summary report.

write_saif

`write_saif [-duration` *`simulation_period`*`] [-computed]`
`[-boundary_only] [-include_hier_ports] [`*`design`*`][> `*`file`*`]`

Description

Writes a hierarchical SAIF file containing the user-asserted or computed (if requested) probability and toggle count of the pins in the design.

By default, the Genus-LP engine writes out the user-asserted switching activities of all leaf instance output pins and primary inputs ports.

The `write_saif` command writes out a compressed SAIF file if you add the `.gz` extension to the file name, but is not removed from the directory.

Options and Arguments

`-boundary_only`	Writes out the switching activities of the primary inputs ports and the leaf sequential instance output pins.
`-computed`	Adds the computed probability and toggle count of the pins and nets to the SAIF file. By default, only the asserted values are written out.
`design`	Specifies the name of the design in case multiple top modules are present.
`-duration` *`simulation_period`*	
	Modifies the duration for which the toggle count is written in the SAIF file. By default, toggle counts are given for a duration of one second. By modifying the duration, smaller toggle counts (numbers) can be written. For example, if the toggle rate is `3e-3/ns`, the default printed toggle count is 300000. If the simulation period is set to`1e+5 ns`, the printed toggle count will be 300.
	Do not choose the duration too small, otherwise the toggle count will be rounded to 0, because only integer numbers are written out.
	Default: `1e+9 ns`
`file`	Specifies the name of the file to which to write the probability values and toggle count values.
`-include_hier_ports`	
	Includes the (computed) switching activities for the hierarchical output ports.

Related Information

TCF Syntax in *Toggle Count Format Reference*.	
Affected by these attributes:	lp_asserted_probability
	lp_asserted_toggle_rate
Related command:	read_saif

write_tcf

```
write_tcf [-duration simulation_period] [-computed]
[-hierarchical] [-include_hier_ports] [-pin]
[-boundary_only] [design] [> file]
```

Description

Writes a TCF file containing the user-asserted or computed (if requested) probability and toggle count of the pins in the design.

The `write_tcf` command writes out a compressed TCF file if you add the `.gz` extension to the file name.

Options and Arguments

`-boundary_only`	Writes out the switching activities of the primary inputs ports and the leaf sequential instance output pins.
`-computed`	Adds the computed probability and toggle count of the pins and nets to the TCF file. By default, only the asserted values are written out.
design	Specifies the name of the design in case multiple top modules are present.
`-duration` *simulation_period*	
	Modifies the duration for which the toggle count is written in the TCF file. By default, toggle counts are given for a duration of 1s
	By modifying the duration, smaller toggle counts can be written. For example, if the toggle rate is 3e-3/ns, the default printed toggle count is 300000. If the simulation period is 1e5 ns, the printed toggle count will be 300.
	Do not choose the period too small, otherwise the toggle count will be rounded to 0, because only integer numbers are written out.
	Default: `1e+9 ns`
file	Specifies the name of the file to which to write the probability values and toggle count values.
`-hierarchical`	Writes out a TCF file in hierarchical format.
	By default, the Genus-LP engine writes out a flat TCF file.
`-include_hports`	
	Includes the (computed) switching activities for the hierarchical output ports.
`-pin`	Writes out a pin-based TCF file. This implies that switching activities on all pins are written out.
	By default, the Genus-LP engine writes out the user-asserted switching activities of all leaf instance output pins and primary inputs ports.

Related Information

Related command:

read_tcf

Affected by these attributes:

lp_asserted_probability

lp_asserted_toggle_rate

write_top_power_intent

```
write_top_power_intent
 [-dir string]
 [-design design]
 [-load_blocks]
 inst|hinst+
```

Description

Writes top partitioned power intent that integrates partitioned block which can be used to create a hier power intent.

Options and Arguments

`-dir` *string*	Specifies the directory path for the output data.
`-design` *design*	Specifies the top design for which the files are written.
`load_blocks`	With block UPF load_upf command.
-inst | hinst	Specifies the instances to exclude.

Related Information

Creates top hierarchical power_intent using specified block level UPF file. Top level UPF file will be saved as top_part.upf in the specified directory.

Related command:

define_design_partition
partition_design

17

Machine Learning Commands

- init_ml
- write_ml_design
- write_ml_timing_path

init_ml

```
init_ml
[-comment string]
[-flow_phase string]
[-name string]
[-prefix string]
[-scenario string]
```

Description

Initializes the environment to start writing data for JedAI. Run this command before using any of the following commands to ensure that content is properly labeled for big data ingestion and registration:

- write_ml_design
- write_ml_timing_path

Options and Arguments

`-comment` *string*	Specifies a comment. Data_type: string, optional
`-flow_phase` *string*	Specifies the flow phase. Data_type: string, optional
`-name` *string*	Specifies the design name. Data_type: string, optional
`-prefix` *string*	Specifies the prefix for the output files. Data_type: string, required
`-scenario` *string*	Specifies the scenario. Data_type: string, optional

Example

The following command initializes the environment to start writing data for JedAI for the Clock Tree Synthesis (CTS) flow phase with the default scenario for the DTMF design.

```
init_ml -prefix out/ -name dtmf -flow_phase cts -scenario default -comment "This is a demo."
```

Related Information

Related Commands:

write_ml_design
write_ml_timing_path

write_ml_design

```
write_ml_design
[-objects objects]
[-prefix string]
[-views views]
```

Description

Writes design-related data for JedAI to CSV files with specified prefix. It includes data that is design specific and relates to logical, physical, and timing type objects.

Options and Arguments

`-objects` *objects*	
	Specifies the list of db object(s) to be dumped, which could include one or more of the following values: `analysis_view arc base_cell base_pin blockage clock congestion constraint_mode delay_corner design gcell_grid hinst hnet hpin hport inst layer marker net netlist opcond option p2p partition patch_wire pg_base_pin pg_pin pin pin_shape port rc_corner route_rule row site skew_group special_net timing_condition track_pattern unit_parasitic via_def` Use curly braces to enclose multiple values. Default: all Data_type: tcl_obj, optional
`-prefix` *string*	Specifies the prefix for the output files. Data_type: string, required
`-views` *views*	Dumps the data for the specified analysis view(s). Default: all Data_type: tcl_obj, optional

Example

The following command writes the attributes of all design objects to CSV files with the specified prefix:

```
write_ml_design -prefix out/
```

Related Information

Related Commands:

init_ml
write_ml_timing_path

write_ml_timing_path

```
write_ml_timing_path
[-max_paths int]
[-max_slack float]
[-nworst int]
[-prefix string][-views views]
```

Description

Writes timing-related data for JedAI to CSV files.

Options and Arguments

`-max_paths` *int*	
	Specifies the `max_paths` for `report_timing`. Data_type: int, optional
`-max_slack` *float*	
	Specifies the `max_slack` for `report_timing`. Data_type: float, optional
`-nworst` *int*	Specifies the `nworst` for `report_timing`. Default: 100 Data_type: int, optional
`-prefix` *string*	Specifies the prefix for the output files. Data_type: string, required
`-views` *views*	Dumps the timing path for the specified analysis view(s). Default:all Data_type: tcl_obj, optional

Example

The following command writes out the attributes of timing paths to CSV files with the specified prefix:

```
write_ml_timing_path -prefix out/ -max_paths 10000 -nworst 1
```

Related Information

Related Commands:

init_ml
write_ml_design

18

Metrics

For usage information, refer to "Unified Metrics" chapter in the *Innovus Common UI Migration Guide*.

- create_metric_page
- create_metric_table
- create_metric_table_heading
- create_snapshot
- create_testcase
- define_metric
- delete_metric
- delete_metric_header
- delete_metric_page
- delete_metric_run
- delete_metric_table
- delete_metric_table_heading
- get_metric
- get_metric_alias
- get_metric_config
- get_metric_definition
- get_metric_header
- pop_snapshot_stack
- push_snapshot_stack
- read_metric
- read_metric_html_package
- reset_metric_config
- reset_metric_run_id
- run_metric_category
- run_parallel_commands
- set_metric
- set_metric_alias
- set_metric_header
- update_metric
- update_metric_definition
- write_metric
- read_metric_config
- write_metric_config

The following command is common with Analysis and Reporting commands:

- report_metric

create_metric_page

```
create_metric_page

-name string
[-before string] [-after string]
[-format enum {html|excel|text}]
```

Description

User defined proc.

Creates a new metric page.

Metrics are saved as part of the database and can be accessed using `get_db` command.

Options and Arguments

`-after string`	Specifies the name of the page after which the new page should be inserted
`-before string`	Specifies the name of the page before which this new page should be inserted
`-format enum {html|excel|text}`	
	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`
`-name string`	Specifies the name for the page. The page name should be unique.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

delete_metric_page
get_db

create_metric_table

```
create_metric_table

[-heading string]
[-type enum (horizontal vertical cpu dict histogram imageplot)]
[-id table_id][-auto_hide]
[-per_snapshot] [-show_label_every frequency]
[-hide_footers] [-hide_graph_footers]
[-hierarchy_separator separator]
[-format {html|excel|text}] [-key string] [-collapsible_key]
```

Description

User defined proc.

Creates a metric table beneath a heading. The metrics html output contains many different types of elements for showing data, the `create_metric_table` command is used for configuring what is shown in the html output.

Options and Arguments

`-auto_hide`	Specifies whether to auto-hide the table or not. If specified, the table auto-hides, otherwise not.
`-collapsible_key`	Specifies to include collapse key columns in the table with the option to expand them.
`-format` *`string`*	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`
`-heading` *`string`*	Specifies the heading of the table. It is appended to the specified ID of the table.
`-hide_footers`	Specifies to hide the min, average and max rows from the bottom of the table. Graph buttons are still shown
`-hide_graph_footers`	
	Specifies to hide the graph buttons from the bottom of the table.
`-hierarchy_separator` *`separator`*	
	Specifies to show the metric names with hierarchy using the provided separator.
`-id` *`table_id`*	Specifies the ID for the new table.
`-key` *`string`*	Specifies a key for any regexp.
`-per_snapshot`	Specifies to expand the table per snapshot (`true` | `false`).
`-show_label_every` *`frequency`*	
	Specifies the frequency of X-axis labels in the histogram.
`-type {horizontal | vertical | cpu | dict | histogram | imageplot}`	
	Specifies the type of component to display data in the output.
	`horizontal | vertical` - These tables allow the axes (rows/columns) to be swapped to change the orientation of displayed data.
	`cpu` - A donut chart of CPU and wall-time usage.
	`dict` - A table to show TCL-dict metrics data, generally used for capturing [report_obj].
	`histogram` - A component that shows histogram data.
	`imageplot` - A component for showing image data (typically floorplans); can also merge multiple images (for example, show DRCs over floorplan).

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

create_metric_table_heading
delete_metric_table
delete_metric_table_heading

create_metric_table_heading

```
create_metric_table_heading[-page page_name][-title string][-id string]
[-format enum {html|excel|text}][-type enum {fixed | collapse}]
```

Description

User defined proc.

Creates a metric table heading on a page which is created by the `create_metric_table` command.

Options and Arguments

`-format` *string*	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`
`-id` *string*	Specifies the ID for the new heading.
`-page` *page_name*	Specifies the page name to add the heading to.
`-title` *string*	Specifies the title to be shown in the heading.
`-type {fixed | collapse}`	
	Specifies the type of heading element, fixed (standard heading in the flow of the metrics html output) or collapse (a button containing a table, which is hidden by default).
	fixed - A fixed heading is a heading in the flow of the metrics html output.
	collapse - A collapse heading is a button which contains a table, which is hidden by default. This is used for detail numbers in a table without showing all the detail initially.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

create_metric_table
delete_metric_table
delete_metric_table_heading

create_snapshot

```
create_snapshot {-name string}
[-auto {default|min}] [-exclude_time_metric]
[-categories {all|{check|design|flow|hold|power|route|setup}...}]
[pattern] [-push] [-pop]
```

Description

Creates a snapshot of the metric values.

All commands that save metrics call `create_snapshot` to save the metrics in a consistent manner. When `create_snapshot` is called, it creates a named or labeled record for the metrics in the database. The label for super commands are the command names.

Metrics are saved as part of the database and can be accessed using `get_db` command.

Metric values should be saved such that the unit is easily known or accessible.

Options and Arguments

`-auto {default|min}`	
	Specifies which set of auto metrics to run. You can choose between the default set or a minimum set.
`-categories {all|{check|clock|design|flow|hold|power|route|setup}}`	
	Specifies the metrics categories to compute.
	If you specify all, the command selects the categories based on the current state of the design.
	You can also specify a blank-separated list of categories.
`-exclude_time_metric`	
	Prevents that CPU time and wall time for this step are included in the parent snapshot calculation.
`-name` *string*	Specifies the name of the snapshot.
pattern	If specified, it is the pattern which should be compared with the metrics. Only the metrics matching the pattern are included in the snapshot.
`-pop`	Specifies to pop the snapshot stack and create a snapshot.
	This option is equivalent to command `pop_snapshot_stack`.
`-push`	Specifies to push the snapshot onto the snapshot stack.
	This option is equivalent to command `push_snapshot_stack`.

Examples

- The following example shows simple syntax usage:

```
create_snapshot -name elab -categories "clock setup"
```

- The following example shows sample usage of `create_snapshot` command:

```
push_snapshot_stack
syn_opt
pop_snapshot_stack
create_snapshot -categories all
report_metric -format html -file syn_opt.html
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

define_metric
get_db
pop_snapshot_stack
push_snapshot_stack
read_metric
run_metric_category
set_metric
write_metric

create_testcase

```
create_testcase
-directory string
[-overwrite]
[-write_db string]
[-design design]
[-update_db string]
[-copy_libraries]
[-include_si]
[-skip_copying_libs]
```

Description

Creates a portable testcase from current database.

This utility links or copies libraries into a local directory, and updates a `split_db` tcl file to use these new locations. Optionally, the db can be saved prior to creating the testcase. Once created, the directory can be tarred up and provided as a standalone testcase for debugging.

Options and Arguments

`-directory` *string*	Specifies the directory where the testcase will be written.
`-overwrite`	Specifies to overwrite an existing directory. By default, an error will be issued if the directory already exists.
`-write_db` *string*	Specifies to first write a db (split tcl and db) under the testcase directory with the prefix *string*.
`-update_db` *string*	Update the specified `split_db` tcl file with the testcase directory locations.
-design *design*	Writes the db with the specified design. If omitted, the design defaults to the current design.
Use either `-write_db` or `-update_db` to write out the db based testcase.	
`-copy_libraries`	Copies the libraries. By default, links are created.
`-include_si`	Includes SI libraries. By default, they are skipped.
`-skip_copying_libs`	
	Specifies not to copy the libraries. This can be used if an existing testcase directory is being used with a new db.

Related Information

Related commands:

write_db

define_metric

```
define_metric -name metric [-type string]
[-tcl tcl_command_list]
[-inheritance_tcl string] [-no_inheritance]
[-inherit_no_children]
[-description string]
[-units string] [-precision num_dec_places]
[-target metric_name]
[-compare {more_better|less_better|more_better_below_target|
less_better_above_target|more_better_ignore_positive|not_equal_bad}]
[-threshold units] [-warning string] [-tools string] [-id string]
```

Description

Defines a new metric to be calculated.

To create the new metrics, you need to provide the TCL code. It can be a set of commands or a single procedure that returns a value.

The metric is calculated and stored with the other metrics using the `create_snapshot` command. All `define_metric` commands are saved and stored in the design database.

Options and Arguments

`-compare {more_better|less_better|more_better_below_target|less_better_above_target|more_better_ignore_positive|not_equal_bad}`	
	Specifies the compare function.
`-description` *`string`*	
	Specifies some description for the metrics to be defined.
`-id` *`string`*	Specifies the ID of the database.
`-inherit_no_children`	
	Specifies to run the inheritance Tcl even if there are no children values.
`-inheritance_tcl` *`string`*	
	Specifies the Tcl for combining children values, equivalent proc args is {children}.
`-name` *`metric`*	Specifies the name of the user-defined metric.
`-no_inheritance`	Disables inheritance of the metric.
`-precision` *`num_dec_places`*	
	Specifies the number of decimal places to be used for precision.
`-target` *`metric_name`*	
	Specifies the name of the metric containing the target value.
`-tcl` *`tcl_command_list`*	
	Specifies the Tcl commands or procedure to execute to generate the metric.
	It is of the format {*`value unit`*}
`-threshold` *`units`*	Specifies the threshold for numeric metrics.
`-tools` *`string`*	Specifies the list of tools which the Tcl code will run in.
`-type` *`string`*	Specifies the type of the metric being defined.
`-units` *`string`*	Specifies the units for reporting the metrics.
`-warning` *`string`*	Specifies the warning threshold or name of the metric containing the threshold value.

Example

The following example defines a Tcl function `get_state` which returns the current design state. The user-defined metric is called `state`.

```
proc get_state {} {
    return [get_db state design:cscan]}

define_metric -name state -tcl get_state
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Affects this command:

report_metric

Related commands:

delete_metric
delete_metric_run
read_metric

delete_metric

```
delete_metric
-name metric [-id string]
```

Description

Removes a previously user-defined metrics from all labels.

Options and Arguments

`-id string`	Specifies the ID of the database.
`-name metric`	Specifies the name of the user-defined metric to be removed.

Example

The following example removes the user-defined metric `state`.

```
delete_metric -name state
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Affects this command:

report_metric

Related command:

define_metric
delete_metric_run
read_metric

delete_metric_header

```
delete_metric_header
-name string [-id string]
```

Description

Deletes the metric header.

Options and Arguments

`-id string`	Specifies the ID of the database. The ID is given by the `read_metric` command.
`-name string`	Specifies the name of the metric header to be deleted.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*.

delete_metric_page

```
delete_metric_page[-name page_name][-format enum {html|excel|text}]
```

Description

User defined proc.

Deletes a metric page created by the `create_metric_page` command.

Options and Arguments

`-format string`	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`
`-name page_name`	Specifies the name of the page to be deleted.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

create_metric_page

delete_metric_run

```
delete_metric_run
-name metric
```

Description

Deletes a metric run.

Options and Arguments

`-name metric`	Specifies the name of the metric run to be deleted.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Affects this command:

- report_metric

Related commands:

- define_metric
- delete_metric
- read_metric

delete_metric_table

```
delete_metric_table[-id table_id][-format enum {html|excel|text}]
```

Description

User defined proc.

Deletes a metric table created by the `create_metric_table` command.

Options and Arguments

`-format string`	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`
`-id table_id`	Specifies the ID of the table to be deleted.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

create_metric_table
create_metric_table_heading
delete_metric_table_heading

delete_metric_table_heading

```
delete_metric_table_heading[-id heading_id][-format enum {html|excel|text}]
```

Description

User defined proc.

Deletes a metric page based on the heading ID.

Options and Arguments

`-format string`	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`
`-id heading_id`	Specifies the ID of the heading to be deleted.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

create_metric_table
create_metric_table_heading
delete_metric_table

get_metric

```
get_metric [-id string] [-names string]
 [patternlist] [-exclude_inherited]
 [-pending] [-pending_top] [-raw] [-branch string]
```

Description

Gets the current value for a metric. This command is used to query the value of a metric from the current specified list of metrics as well as from any previous state.

Options and Arguments

`-branch` *string*	Specifies a list of patterns to match snapshot branches.
`-exclude_inherited`	Specifies not to return any inherited value.
`-id` *string*	Specifies the ID of the database to output.
	ID is given by the `read_metric` command.
	Default: current database.
`-names` *string*	Specifies a list of pattern that matches the snapshot names.
	Wildcards are supported in the pattern.
	If not specified, it indicates current snapshot.
patternlist	Specifies a list of pattern to match the metrics.
	Wildcards are supported.
	Use * to get all metrics.
`-pending`	Specifies to return the metrics from the pending snapshot. This snapshot is not yet created.
`-pending_top`	Specifies to return metrics from the entire pending snapshot stack. This snapshot is not yet created.
`-raw`	Specifies to return the metrics as raw values. These are not normalized to defined units.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

read_metric
set_metric
set_metric_header
write_metric

get_metric_alias

`get_metric_alias`

Description

Returns the list of a metric aliases created by the `set_metric_alias` command.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

set_metric_alias

get_metric_config

```
get_metric_config
 [-format enum {html|excel|text}]
```

Description

User defined proc.

Gets the current configuration of a metric as a Tcl list or a dict.

Options and Arguments

`-format` *enum*	Specifies the metric configuration for `report_metric` format.
	Options: `{html excel text}`

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

reset_metric_config

get_metric_definition

```
get_metric_definition -name metric_name
[-id string]
```

Description

Gets the metric definition for the specified metric.

Options and Arguments

`-id` *string*	Specifies the database ID to output.
	ID is given by the `read_metric` command.
`-name` *metric_name*	Specifies the name of the metric.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

define_metric

get_metric_header

```
get_metric_header [-name string] [-id string]
```

Description

Returns the value of the metric header component.

Options and Arguments

`-id string`	Specifies the ID of the database.
`-name string`	Specifies the name for the metric header for which to retrieve the value.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

get_metric_definition

pop_snapshot_stack

```
pop_snapshot_stack
```

Description

Pops the metrics saved since the last push.

The `pop_snapshot_stack` command works in conjunction with `push_snapshot_stack` to create metric hierarchies. Any metrics captured inside a push/pop must be saved via create_snapshot (implicit or explicit) or they will be lost.

Example

The following example shows how to use the `pop_snapshot_stack` command:

```
push_snapshot_stacksyn_optpop_snapshot_stackcreate_snapshot -categories all -label syn_optreport_metric -format html -file
```

```
syn_opt.html
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

push_snapshot_stack
create_snapshot
report_metric

push_snapshot_stack

```
push_snapshot_stack [-name string]
```

Description

Creates a hierarchy for metric capture. All metrics captured inside the hierarchy is saved at this level during snapshot creation (implicit or explicit). Snapshot hierarchies can be nested so any number of hierarchies are supported.

> Every `push_snapshot_stack` command should have a corresponding `pop_snapshot_stack` command.

Options and Arguments

`-name string`	Specifies the name of the snapshot.

Example

The following example shows how to use the `push_snapshot_stack` command:

```
push_snapshot_stacksyn_optpop_snapshot_stackcreate_snapshot -categories all -label syn_optreport_metric -format html -file
syn_opt.html
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

pop_snapshot_stack
create_snapshot
report_metric

read_metric

```
read_metric filename [-id string]
[-previous string] [-merge]
```

Description

Loads the metrics file for querying, reporting and comparing. You can load any number of these files but each file should have a unique ID.

Options and Arguments

filename	Specifies the name of the file to read.
-id *string*	Specifies the name of the snapshot to create or overwrite.
-merge	Specifies to merge the metrics into the named snapshot instead of replacing its content.
-previous *string*	Specifies the unique ID of previous snapshots for new pending snapshot.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

get_metric
set_metric
set_metric_header
write_metric

read_metric_html_package

```
read_metric_html_package filename
```

Description

User defined proc

Loads the vivid HTML package to customize `report_metric -format vivid` output.

The `read_metric_html_package` command allows you to load a vivid HTML package into a Genus session. This will replace any previously loaded package and page configuration loaded with `read_metric_config` for the `report_metric -format vivid` command.

Options and Arguments

filename	Specifies the name of the file to read the metric package from.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

reset_metric_config

```
reset_metric_config [-format enum {html|excel|text}]
```

Description

User defined proc.

Resets the metric page configuration back to default.

Options and Arguments

`-format` *string*	Specifies the configuration for the `report_metric` format.
	Options: `{html excel text}`

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

get_metric_config

reset_metric_run_id

```
reset_metric_run_id
```

Description

User defined proc.

Clears the current metric run ID.

When you issue the `create_snapshot` command and send metrics data to a server, the job has a `run_id` to identify the data. If you load a database from another user, or want to perform experimentation, the `reset_metric_run_id` command allows you to clear the `run_id` and any related state so that it is sent with a new `run_id`.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

run_metric_category

```
run_metric_category  [-category category]
```

Description

Calculates the metrics for the specified category.

The metric values are saved in the database when you issue a `create_snapshot` command.

Options and Arguments

`-category` *category*	
	Specifies the name of the category to run.
	The category can be any of {check|clock|design|flow|hold|power|route| setup}
	When no category is specified, the `design` category is used by default. This is in alignment with the `create_snapshot` command. This command is not required to be run by the user. This command is used by the `create_snapshot` command to compute the metrics for each category. But, the difference is `run_metric_category` leaves the computed metrics as "pending", so they will get added to the next snapshot which allows them to be combined with other pending metrics in the same snapshot.
	The `hold`, `cts`, and `route` categories do not compute any metrics in Genus.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

create_snapshot

run_parallel_commands

```
run_parallel_commands [-queue string] [-execute] [-append_log] [-log_dir string] [-prefix string] [-priority integer] [> file]
```

Description

Forks the command to run in background.

Options and Arguments

`-queue` *`string`*	Adds command string to run in background.
`-execute`	Launches all queued commands.
`-append_log`	Appends `run_parallel_commands` log to the main Genus log.
`-log_dir` *`string`*	Specifies the output directory for saving forked commands logfile.
`-prefix` *`string`*	Specifies the prefix for the logfile.
`-priority` *`integer`*	Specifies the priority of the command. The default priority is 0.

Example

```
genus:root: 1> run_parallel_commands -queue report_qor -log_dir ./rpt/rpt_2 -prefix test -priority 2

Command added to the queue "report_qor". Use -execute to launch.


genus:root: 1> run_parallel_commands -queue report_timing -log_dir ./rpt/rpt_2 -prefix test -priority 2 -execute

Command added to the queue "report_timing".

Info    : All commands in the queue are launched. [RPT-303]

Waiting for job : 451171 to complete ...

Waiting for job : 451330 to complete ...

SUCCESS PID 451171 : report_timing

SUCCESS PID 451330 : report_qor


genus:root: 1> run_parallel_commands -execute -queue report_clocks

Command added to the queue "report_clocks".

Info    : -log_dir option missing. [RPT-302]

        : -log_dir option is not provided for: report_clocks, writing log in: /tmp/genus_temp_335410_sjfdcl1248_dverma_dYtCCj

Info    : All commands in the queue are launched. [RPT-303]

Waiting for job : 102642 to complete ...

SUCCESS PID 102642 : report_clocks
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Affected by the attribute

max_cpus_per_server

set_metric

```
set_metric -name string
-value string
```

Description

Sets a value for the specified metric.

Options and Arguments

`-name string`	Specifies a name for the metric.
`-value string`	Specifies the metric value.

Examples

- The following example sets the value of the worst negative setup slack (all path groups) to 0.100ns:

  ```
  set_metric -name timing.setup.wns -value
  {0.100ns}
  ```

- The following example sets the value of the total area of the design to 100 um:

  ```
  set_metric -name design.area -value {100 um}
  ```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

define_metric
set_metric_header

set_metric_alias

```
set_metric_alias
-from old_name -to new_name [-no_overwrite] [-order order_of_priority]
```

Description

Renames a metric name, prefixed with the name of an analysis view, path group, and so on, to another name. For example, you can rename `path_group:reg2reg` to `reg2reg` using the `set_metric_alias` command.

Options and Arguments

`-from old_name`	Specifies the metric object name to be changed.
`-no_overwrite`	Prevents overwriting of an already set alias.
`-order order`	Specifies the order of priority of metric object in tables.
`-to new_name`	Specifies the alias name for the metric object.

Examples

- The following command renames `path_group:reg2reg` to `reg2reg` with priority order 1.

  ```
  set_metric_alias -from path_group:reg2reg -to reg2reg -order 1
  ```

- The following command renames `path_group:reg2cgate` to `reg2cgate` with priority order 2.

  ```
  set_metric_alias -from path_group:reg2cgate -to reg2cgate -order 2
  ```

- The following command queries the set of aliases:

  ```
  get_metric_alias
  ```

  Output

  ```
  path_group:reg2reg {to reg2reg order 1} path_group:reg2cgate {to reg2cgate order 2}
  ```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related commands:

get_metric_alias
set_metric

set_metric_header

```
set_metric_header -name string
-value string [-id string]
```

Description

Sets the metric header.

Options and Arguments

`-id string`	Specifies to output the database ID.
`-name string`	Specifies a name for the metric.
`-value string`	Specifies the metric value.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

get_metric_definition

update_metric

```
update_metric -table string [-metric string] [-header string]
[-title string] [-group string] [-graph_type enum (treemap | line | none)] [-show_graph] [-link_file_metric metric_name] [-link_table
table] [-renderer html_format]
```

Description

Updates a metric table by adding a metric or a heading.

Options and Arguments

`-graph_type enum (treemap | line | none)`	
	Specifies the type of graph to be used in the table.
	Default: `line`
`-group string`	Specifies the title of the group to be used in the table.
`-header string`	Specifies the name of the header to add to the table.
`-link_file_metric metric_name`	
	Specifies the name of the metric containing the filename to open when the metric value is clicked.
`-link_table table`	Specifies the table to scroll to when the metric value is clicked.
`-metric string`	Specifies the name of the metric to add to the table.
`-renderer html_format`	
	Forces a specific HTML format renderer for the histogram metric.
`-show_graph`	Specifies whether to show this metric in a comparison graph.
`-table string`	Specifies the ID of the table to add the metric to.
`-title string`	Specifies the title to be used in the table.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

get_metric
set_metric
set_metric_header
write_metric

update_metric_definition

```
update_metric_definition -name string [-tcl command_list]
[-type string][-description string][-units string]
[-precision integer][-compare
{more_better|less_better|more_better_below_target|less_better_above_target|more_better_ignore_positive|not_equal_bad}]
[-threshold unit] [-no_inheritance][-inherit_no_children]
[-inheritance_tcl tcl_name][-target metric_name]
[-warning threshold] [-tools string] [-id string]
```

Description

Updates the specified properties of an already-existing metric definition having the specified name.

Options and Arguments

`-compare` {more_better|less_better|more_better_below_target|less_better_above_target|more_better_ignore_positive|not_equal_bad}	
	Specifies the compare function.
`-description` *string*	
	Describes the metric.
`-id` *string*	Specifies the ID of the database.
`-inheritance_tcl` *tcl_name*	
	Specifies TCL for combining children values. The metric is not executed in the global namespace.
`-inherit_no_children`	Specifies to run inheritance tcl even if there are no child values.
`-name` *string*	Specifies the name of the metric.
`-no_inheritance`	Specifies to disable the metric inheritance.
`-precision` *integer*	Specifies the number of decimal places.
`-target` *metric_name*	Specifies the target or the name of the metric containing the target value.
`-tcl` *command_list*	Specifies the tcl code to generate the metric value which is of the format {<`value`><`unit`>}.
`-threshold` *unit*	Specifies the threshold for comparing numbers in HTML.
	If the delta between the two numbers being compared is less than the threshold value, the metric will be colored "yellow". If it is outside the threshold, then it is colored red or green depending on the compare function, that is, whether more is better or less is better.
`-tools` *string*	Specifies the list of tools which the Tcl code will run in.
`-type` *string*	Specifies the type of metric.
`-units` *string*	Specifies the unit for reporting.
`-warning` *threshold*	Specifies the warning threshold value or name of the metric containing the threshold value.

Related Information

"Unified Metrics" chapter in *Innovus Common UI Migration Guide*

Related commands:

get_metric
set_metric
set_metric_header
write_metric
update_metric

write_metric

```
write_metric [-format {json|xml|csv|tcl}] [-id string] [-names string] [metric_names] [-out_file string] [-inline] [-merge]
```

Description

Writes out the current metrics to the specified file or stdout.

The `write_metric` command is used by `write_db` to write the metrics in the json format in the database directory. Super commands use `write_metric` to write metrics into the log file (stdout). This can also be used to write metrics in various formats by the user.

Options and Arguments

`-format string`	Specifies the format of the data for the return value.
	Supported formats: `json`, `xml`, `csv`, `tcl`.
	Default: `json`
`-id string`	Specifies to output the database ID.
`-inline`	Specifies to return the result to stdout instead of writing it to the file.
`-merge`	Specifies to merge the results to an existing file.
metric_names	Specifies a list of patterns to match the metrics.
	Default is all metrics (*).
`-names string`	Specifies a list of patterns to match snapshot names.
`-out_file string`	Specifies the name of the output file to write.

Example

The following code shows the usage of the `write_metric` command:

```
# ------------------------------------------------------------------# Generate a XML file for all timing, power, and flow metrics
from# from prects to postcts for a previous run (id v101)# ------------------------------------------------------------------


read_metric -id v101 run101/dbs/postcts.enc.dat/design.metrics


write_metric -id v101 -format xml
```

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:

read_metric
set_metric

set_metric_header

read_metric_config

```
read_metric_config
 [-format enum {html|excel|text|vivid}]filename
```

Description

User defined proc.

Options and Arguments

`-format` *enum*	Specifies the metric configuration for `report_metric` format.
	Options: `{html excel text vivid}`
filename	Specifies the filename to read the metric configuration from.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

write_metric_config

```
write_metric_config
 [-format enum {html|excel|text|vivid}] filename
```

Description

User defined proc.

Options and Arguments

`-format` *enum*	Specifies the metric configuration for `report_metric` format.
	Options: `{html excel text vivid}`
filename	Specifies the filename to write the metric configuration to.

Related Information

"Unified Metrics" chapter in the *Innovus Common UI Migration Guide*

Related command:	
	read_metric_config
	get_metric
	set_metric
	set_metric_header

19

Multi-Mode Multi-Corner

- all_constraint_modes
- create_analysis_view
- create_constraint_mode
- create_delay_corner
- create_library_set
- create_net
- create_opcond
- create_rc_corner
- create_timing_condition
- get_interactive_constraint_modes
- init_design
- read_mmmc
- set_analysis_view
- set_interactive_constraint_modes
- update_analysis_view
- update_constraint_mode
- update_delay_corner
- update_library_set
- update_opcond
- update_rc_corner
- update_timing_condition
- update_lef_macro

The following command is common with Analysis and Reporting commands:

- report_analysis_views

all_constraint_modes

```
all_constraint_modes
[-active] [-active_setup] [-active_hold]
```

Description

Returns a list of all the defined MMMC constraint modes.

Options and Arguments

`-active`	Returns a list of the defined constraint modes associated with all active analysis views.
`-active_hold`	Returns a list of the defined constraint modes associated with only the active hold analysis views.
`-active_setup`	Returns a list of the defined constraint modes associated with only the active setup analysis views.

Related Information

Related commands:

create_constraint_mode
update_analysis_view

create_analysis_view

```
create_analysis_view -name view
-constraint_mode constraint_mode
-delay_corner delay_corner
[-power_modes list_of_power_modes][-latency_file file]
```

Description

Defines an analysis view (object) and associates it with a constraint mode and a delay calculation corner and optionally a list of power modes.

An analysis view object provides all the information necessary to control a single multi-mode multi-corner analysis.

Options and Arguments

`-constraint_mode constraint_mode`	
	Specifies the name of the constraint mode to associate with this analysis view.

	A constraint mode groups a set of constraints to be used with a specific analysis view.
	The constraint mode must have been created with the `create_constraint_mode` command.
`-delay_corner` *`delay_corner`*	
	Specifies the name of the delay corner to associate with this view.
	A delay corner contains all of the information needed to control delay calculation for a specific analysis view.
	The delay corner must have been created with the `create_delay_corner` command.
`-latency_file` *`file`*	Specifies the name of the latency file.
`-name` *`view`*	Specifies the name of the analysis view being defined.
`-power_modes` *`list_of_power_modes`*	
	Specifies a list of power modes to use with this analysis view.
	The power mode names must exist in the power intent file.

Example

```
create_analysis_view -name missionSlow
    -constraint_mode missionSetup
    -delay_corner dcWCCOM
```

Related Information

Related commands:

create_constraint_mode
create_delay_corner
update_analysis_view

create_constraint_mode

```
create_constraint_mode -name mode
-sdc_files list_of_files
[-ilm_sdc_files list_of_files]
[-tcl_variables {{var val}...}]
```

Description

Defines a constraint mode (object) and associates the specified SDC files with the constraint mode.

Associates a list of SDC constraint files with a specified constraint mode name, for multi-mode multi-corner analysis. This constraint mode name can be referred to later when creating analysis views. A constraint mode defines one of possibly many different functional, test, or Dynamic Voltage and Frequency Scaling (DVFS) modes of a design. SDC files can be shared by many different constraint modes, and the same constraint mode can be associated with multiple analysis views.

SDC files typically contain timing analysis information, such as the clock specifications, case analysis constraints, I/O timings, and path exceptions that make each mode unique. If the ILM is being used, the timing system utilizes the files specified with -`ilm_sdc_files` during the flatten ILM mode (`flatten_ilm`); otherwise, the system uses the files specified with -`sdc_files`.

Options and Arguments

`-ilm_sdc_files` *list_of_files*	
	Specifies the list of ILM SDC files to be included in the constraint mode.
`-name` *mode*	Specifies the name of the constraint mode being defined.
`-sdc_files` *list_of_files*	
	Specifies the TCL list of SDC files to be associated in this constraint mode.
`-tcl_variables {`*vars val*`}`	
	Specifies the Tcl variables associated with this constraint mode as a list of variable-value pairs.
	`{{`*name1 val1*`} {`*name2 val2*`}....{`*nameN valN*`}}`

Examples

- The following command groups the SDC files io.sdc, mission1-clks.sdc, and mission1-except.sdc to create a mode object named missionSetup:

```
create_constraint_mode   -name   missionSetup   -sdc_files \
   [list   io.sdc   mission1-clks.sdc   mission1-except.sdc]
```

- The following command groups the SDC files io.sdc, mission1-clks.sdc, mission1-except.sdc, and dont_use.sdc to create a mode object named missionHold:

```
create_constraint_mode -name missionHold -sdc_files \
[list io.sdc mission1-clks.sdc mission1-except.sdc dont_use.sdc]
```

Related Information

Related commands:

create_analysis_view
update_analysis_view
set_interactive_constraint_modes
update_constraint_mode

create_delay_corner

```
create_delay_corner -name delay_corner
[ [-timing_condition conditional_pairs]
  [-rc_corner rc_corner]
  [-temperature_files files]
  [-irdrop_files files ]
  [-early_timing_condition conditional_pairs]
  [-late_timing_condition conditional_pairs]
  [-early_rc_corner rc_corner]
  [-late_rc_corner rc_corner]
  [-early_temperature_files files]
  [-late_temperature_files files]   [-pg_net_voltages power_net@voltage]
  [-early_irdrop_files files]
  [-late_irdrop_files files] ]
  [-si_enabled {true|false}]
  [-early_estimated_worst_irdrop_factor float]  [-late_estimated_worst_irdrop_factor float]
```

Description

Defines a delay calculation corner (object) that can be referenced when creating an analysis view. A delay calculation corner provides all of the information necessary to control delay calculation for a specific analysis view. Each corner contains information on the libraries to use, the operating conditions with which the libraries should be accessed, and the RC extraction parameters to use for calculating parasitic data. Delay corner objects can be shared by multiple top-level analysis views.

> The `-early_*` and `-late_*` parameters should not be used to represent best and worst case corners. Instead use separate delay corners to define Best-Case and Worst-Case differences. Use the `-early_*` and `-late_*` parameters within a single delay calculation corner to control on-chip variation. In this latter situation, different conditions or libraries will be used for the launch and capture paths of individual timing paths.

> A single delay calculation corner object specifies the delay calculation rules for the entire design.If a design includes power domains, the delay calculation corner can contain domain-specific sub-sections that specify the required operating condition information, and any necessary timing library rebinding for the power domain. Use the `update_delay_corner` command to add a power domain definition to a delay calculation corner.

This command determines the timing conditions for every instance in the design. You must specify the binding between power domains and timing conditions, and specify a common RC corner. The result is a fully specified delay calculation definition.

Options and Arguments

`-early_irdrop_files` *files*	
	Specifies the name of the IR drop files to apply for the early delay calculation for this delay corner.
`-early_rc_corner` *rc_corner*	
	Specifies the RC corner object to associate with the early delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis.
	The RC corner must have been defined with the `create_rc_corner` command.
`-early_temperature_files` files	
	Specifies the name of the temperature files for temperature-aware delay calculation for an early corner.

`-early_timing_condition` *conditional_pairs*	
	Specifies the early timing conditions to associate with this delay corner object. Use one of the following formats for a conditional pair:
	• timing_condition
	Specifies the default timing condition which applies to all cells not covered by any other timing condition.
	• power_domain_list@timing_condition
	The named timing condition applies to the list of all power domains (defined in a power intent file).
	• supply_set_list@timing_condition
	The named timing condition applies to all blocks tied to the specified supply set (defined in an 1801 file).
	• hinst@timing_condition
	The named timing condition applies to the specified hierarchical instance.
	The timing conditions must have been defined with the `create_timing_condition` command.
`-early_estimated_worst_irdrop_factor` *float*	
	Specifies the early estimated worst IR drop factor.
`-late_estimated_worst_irdrop_factor` *float*	
	Specifies the late estimated worst IR drop factor.
`-irdrop_files` *files*	
	Specifies the name of IR drop files to apply for both the early and late delay calculation for this delay corner. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes.

`-late_irdrop_files` *files*	
	Specifies the name of the IR drop files to apply for the late delay calculation for this delay corner.
`-late_rc_corner` *rc_corner*	
	Specifies the RC corner object to associate with the late delay corner.
	The RC corner must have been defined with the `create_rc_corner` command.
`-late_temperature_files` *files*	
	Specifies the name of the temperature files for temperature-aware delay calculation for a late corner.
`-pg_net_voltages` *power_net@voltage*	
	Specifies the list of pg_net@voltage.
`-late_timing_condition` *conditional_pairs*	
	Specifies the late timing conditions. Use one of the following formats for a conditional pair:
	• timing_condition
	Specifies the default timing condition which applies to all cells not covered by any other timing condition.
	• power_domain_list@timing_condition
	The named timing condition applies to the list of all power domains (defined in a power intent file).
	• supply_set_list@timing_condition
	The named timing condition applies to all blocks tied to the specified supply set (defined in an 1801 file).
	• hinst@timing_condition

	The named timing condition applies to the specified hierarchical instance.
	The timing conditions must have been defined with the `create_timing_condition` command.
`-name` *`delay_corner`*	Specifies the name of the delay corner being defined.
`-rc_corner` *`rc_corner`*	
	Specifies the RC corner object to associate with this delay corner for both late and early delay calculation.
	The RC corner must have been defined with the `create_rc_corner` command.
`-si_enabled`	Controls SI analysis.
	Genus does not support SI analysis. Genus saves this option and writes it out when you specify the `write_mmmc` command.
`-temperature_files` *`files`*	
	Specifies the name of the temperature files for temperature-aware delay calculation for a single corner.
`-timing_condition` *`conditional_pairs`*	
	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. Specifies the early and late timing conditions. Use one of the following formats for a conditional pair:
	• timing_condition
	Specifies the default timing condition which applies to all cells not covered by any other timing condition.
	• power_domain_list@timing_condition
	The named timing condition applies to the list of all power domains (defined in a power intent file).

	• supply_set_list@timing_condition
	The named timing condition applies to all blocks tied to the specified supply set (defined in an 1801 file).
	• hinst@timing_condition
	The named timing condition applies to the specified hierarchical instance.
	The timing conditions must have been defined with the `create_timing_condition` command.

Example

The following command assigns timing condition `TC1` for the whole chip except for power domains `PD2` and `PD3` and associates the rail `VDD` with timing condition `TC3`.

```
create_delay_corner -name DC3 -timing_condition {TC1 {PD2 PD3}@TC2 VDD@TC3}
```

Related Information

Related commands:

create_rc_corner
create_timing_condition
update_delay_corner

create_library_set

```
create_library_set -name library_set
{-timing library_files |-target_timing library_list -link_timing library_list
[-aocv list_of_files] [-socv list_of_files]
[-si  {lib1.cdb lib2.cdb lib3.udn lib4.udn…}]
[-library_side_file string]
```

Description

Associates a TCL list of timing and cdB/UDN libraries with a specified library set name.

Library sets allow a group of library files to be treated as a single entity so that higher-level descriptions (delay calculation corners) can simply refer to the library configuration by name. The same library set can be referenced multiple times by different delay calculation corners.

> The order in which you define timing libraries is important. The software considers the first library you specify in the list as the master library, with each successive library having a lower priority.

While specifying the libraries should be in the list format.Provide separate library list for each corner and IPs. If using for voltage scaling, then the library-pairs used in the command should be specified in a way that within the pair, the library which is characterized at lower voltage should be specified first.Tool does not treat IP libraries differently it just checks the nominal-voltage specified in the liberty file irrespective of the liberty content. If using `set_db` library command for voltage scaling the same rule applies.

Options and Arguments

`-aocv` *list_of_files*	
	Specifies a list of AOCV libraries (for Advanced On-Chip-Variable support) to be included in the library set.
`-link_timing` *list_of_files*	
	Specifies a list of link libraries to be included in the library set.
	The link libraries contain the descriptions of cells (macros like RAMS, pads, PLLs, and so on) in a mapped netlist.
	This option must be specified with the `-target_timing` option and cannot be specified with the `-timing` option,
-library_side_file *string*	
	Specifies the library side file.
`-name` *library_set*	Specifies the name of the library set being defined.
	This name is used to associate the library list with a specific delay calculation corner.
`-si {lib1.cdb lib2.cdb lib3.udn lib4.udn…}`	

	Specifies a list of cdB libraries and user-defined noise (UDN) models to include in the library set. The cdB and UDN libraries are required for performing signal integrity analysis.
	You can use this parameter for CCS-N or ECSM-N models, that is, for cdBs only.
`-socv` *`list_of_files`*	
	Specifies a list of SOCV libraries to be included in the library set.
`-target_timing` *`list_of_files`*	
	Specifies a list of target libraries to be included in the library set.
	The target libraries contain the cells used to generate the netlist and definitions for the design's operating conditions.
	This option must be specified with the `-link_timing` option and cannot be specified with the `-timing` option,
`-timing` *`list_of_files`*	
	Specifies a list of timing libraries (.lib files) to include in the library set.

Examples

- The following command creates a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name `IsCOM-1V`:

```
create_library_set -name IsCOM-1V \
 -timing [list stdcell_F_1V.lib ram_F.lib pad.lib] \
 -si [list stdcell_F_2.cdb ram_F.cdb pad.cdb]
```

- The following example shows the usage of target and link libraries.

```
create_library_set -name LS1 -target_timing { TL1a TL1b } \
-link_timing { LL1a_used LL1b_used LL1c_unused LL1d_unused }

create_library_set -name LS2 -target_timing { TL2a TL2b } \
-link_timing { LL2a_used LL2b_used LL2c_unused LL2d_unused }
```

- The following example shows the how to specify for voltage scaling purpose:

```
create_library_set  -name LS2 -target_timing { {lib_0.6 lib_0.7}\
   {lib_0.55 lib_0.7} }

create_library_set  -name LS2 -target_timing { {lib_0.55 lib_0.7} \
   {lib_0.6 lib_0.7} }
```

  Below is the incorrect way:

```
create_library_set  -name LS2 -target_timing { {lib_0.7 lib_0.6} \
   {lib_0.7 lib_0.5} }
```

- The following example shows how to specify lib_list if there is single voltage libraries for voltage scaling:

```
create_library_set  -name LS2 -target_timing{ {lib_0.55 lib_0.7} {lib_0.8} }\
```

  It is recommended to put the libraries in {curly braces}.

Related Information

Related command:	update_library_set
Related attributes:	library_set

create_net

```
create_net -name string
[-parent design|hinst|module]
[-bus string]
```

Description

Creates a new net object.

Options and Arguments

`-name string`	Specifies the name of the new net.
`-bus string`	Specifies the bit range for the bus nets <msb:lsb>.
`-parent design|hinst|module`	
	Specifies the module, hinst, or the design to contain the new net.

Examples

- The following command returns newly created nets with bus bits

  ```
  create_net -name NEW_NET_WITH_BUS -bus 3:0
  ```

  ```
  Output : {net:des1/NEW_NET_WITH_BUS[0]}   {net:des1/NEW_NET_WITH_BUS[1]}
  {net:des1/NEW_NET_WITH_BUS[2]} {net:des1/NEW_NET_WITH_BUS[3]}
  ```

- The following command returns single net.

  ```
  create_net -name NEW_NET
  ```

Related Information

Related command:

create_hnet

create_opcond

```
create_opcond -name operating_condition
[-process process]
[-voltage voltage]
[-temperature temperature]
{-tree_type string]
```

Description

Creates a set of virtual operating conditions in the specified library without actually modifying the library.

Creating operating condition is needed in situations when there are no predefined operating conditions in the user timing libraries, or the pre-existing operating conditions are not consistent with the user's operating environment. In such scenarios instead of actually modifying the timing libraries to add or adjust operating condition definitions, you can create a set of virtual operating conditions, to define a PVT operating point. These virtual operating conditions can then be referenced by a delay corner.

An operating condition can be associated with a library set using the `create_timing_condition` command.

Options and Arguments

`-name` *operating_condition*	
	Specifies the name of the operating condition to be created.
`-process` *process*	Specifies the process value of the operating condition.
`-temperature` *temperature*	
	Specifies the temperature of the operating condition in degrees Celsius.
`-tree_type` *string*	Specifies the tree type value. Possible values are:
	• `balanced_tree`
	• `best_case_tree`
	• `binary_tree`
	• `worst_case_tree`
`-voltage` *voltage*	Specifies the voltage value of the operating condition in volt.

Examples

- The following command creates a virtual operating condition called `PVT1`

  ```
  create_opcond -name PVT1
  ```

- The following command creates a virtual operating condition called `PVT1`

  ```
  create_opcond -name PVT1 -process 1.0 -voltage 1.2 -temperature 120
  ```

- The following command creates a virtual operating condition called `PVT2`:

  ```
  create_op_cond -name PVT2 -process 1.0 -voltage 1.2 -temperature 20
  ```

Related Information

Related commands:

create_timing_condition
update_opcond

create_rc_corner

```
create_rc_corner -name rc_corner
[-temperature temperature]
[-qrc_tech fileName ]
[-cap_table  capTable_file]
[-pre_route_cap capFactor ]
[-pre_route_res resFactor ]
[-post_route_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_res {resFactor1 resFactor2 resFactor3}]
[-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-pre_route_clock_cap capFactor ]
[-pre_route_clock_res resFactor ]
[-post_route_clock_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_clock_res {resFactor1 resFactor2 resFactor3}]
[-post_route_clock_cross_cap float+]
[-via_variation_file string]
```

Description

Creates a named RC corner (object) that can be referenced later when creating a delay calculation corner object. An RC corner object provides the software with all of the information necessary to properly extract, annotate, and use the RCs for delay calculation.

> You must use the RC corner scaling attributes when running the software in multi-mode multi-corner analysis mode. Use the `extract_rc` category attributes to set the extraction mode for all RC corner objects in the design.

> In multi-mode multi-corner mode, only the default RC corners created by the software inherit the derating factor values defined in the configuration file.

You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

```
resFactor1 resFactor2 resFactor3
```

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of 1 is

used for `medium` and `high` by default.

- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of 1 is used for `high` by default.
- Triplet: If yo specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Options and Arguments

`-cap_table` *`capTable_file`*	
	Specifies the capacitance table to use for extraction when using this RC corner.
`-name` *`rc_corner`*	Specifies the name of the RC corner being defined.
	You can choose any name for an RC corner object, but it is recommended that you use a name that can be easily recognized as referring to a specific RC corner (for example, rc-min, rc-max, c-min, c-max, and so on).
`-post_route_cap {`*`capFactor1 capFactor2 capFactor3`*`}`	
	Specifies the capacitance scale factor for RC extraction in post-route mode.
	Default: `1.0`
`-post_route_clock_cap {`*`capFactor1 capFactor2 capFactor3`*`}`	
	Specifies the scale factor for the capacitance of clock nets in post-route mode.
	When this parameter is specified, this value overrides the value specified with the `-post_route_cap` option for the clock nets.
	When this parameter is not specified, the value specified with the `-post_route_cap` option is used for pre-route extraction.
	Default: `0.0`
`-post_route_clock_cross_cap` *`float+`*	
	Specifies the triplet factors for the post-route coupling capacitances.
`-post_route_clock_res {`*`resFactor1 resFactor2 resFactor3`*`}`	
	Specifies the scale factor for the resistance of clock nets in post-route mode.

	When this parameter is specified, this value overrides the value specified with the `-post_route_res` option for the clock nets.
	When this parameter is not specified, the value specified with the `-post_route_res` option is used for pre-route extraction.
	Default: `0.0`
`-post_route_cross_cap {`*capFactor1 capFactor2 capFactor3*`}`	
	Specifies the coupling capacitance scale factor for RC extraction in post-route mode.
	Default: `1.0`
`-post_route_res {`*resFactor1 resFactor2 resFactor3*`}`	
	Specifies the capacitance scale factor for RC extraction in post-route mode.
	Default: `0.0`
`-pre_route_cap` *capFactor*	
	Specifies the capacitance scale factor for RC Extraction in pre-route mode.
	Default: `1.0`
`-pre_route_clock_cap` *capFactor*	
	Specifies the scale factor for the capacitance of clock nets in pre-route mode.
	When this option is specified, this value overrides the value specified with the `-pre_route_cap` option for clock nets.
	When this option is not specified, the value specified with the `-pre_route_cap` option is used for pre-route extraction.
	Default: `1.0`
`-pre_route_clock_res` *resFactor*	
	Specifies the scale factor for the resistance of clock nets in pre-route mode.
	When this parameter is specified, this value overrides the value specified with the `-pre_route_res` option for the clock nets.
	When this parameter is not specified, the value specified with the `-pre_route_res` option is used for pre-route extraction.
	Default: `0.0`

`-pre_route_res` *resFactor*	
	Specifies the resistance scale factor for RC Extraction in pre-route mode.
	Default: `0.0`
`-qrc_tech` *fileName*	
	Specifies the name of an QRC technology (.tch) file.
	The technology file is generated by TechGen, a built-in functionality of Quantus QRC. For most technologies you can get this file from technology vendors. For technologies that are not available through the technology vendors, you enter the fabrication process information into an ASCII-format interconnect technology (ICT) input file. The ICT file is then used as input to IceCaps, which in turn generates the technology file.
`-temperature` *temperature*	
	Specifies the temperature, in units of Celsius, to be used to derate resistance values.
	Use this option when you want to override the default temperature specified in the capacitance table or the QRC technology file.
	Default: 25 degrees Celsius
`-via_variation_file` *string*	
	Specifies the name of the via variation file.

Example

The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:

```
create_rc_corner -name rc-cbest -cap_table myTech.4_bc.CapTbl -temperature 50
```

Related Information

Related commands:

update_rc_corner

create_timing_condition

```
create_timing_condition -name timing_condition
-library_sets list
[-opcond name] [-opcond_library library]
```

Description

Defines a timing condition.

A timing condition defines a set of libraries characterized for a specific operating condition. Timing conditions are assigned to power domains or supply sets, and effectively describe the delay calculation requirements for the domains.

Options and Arguments

`-library_sets list`	Specifies the list of library sets to associate with this timing condition.
	The library sets must have been defined with the `create_library_set` command.
`-name timing_condition`	
	Specifies the name of the timing condition being defined.
`-opcond opcond`	Specifies the operating condition of the library sets.
	The operating condition must have been defined with the `create_opcond` command.
`-opcond_library library`	
	Specifies the name of the library to search for the specified operating condition.

Related Information

Related commands:

create_library_set
create_opcond
update_timing_condition

get_interactive_constraint_modes

```
get_interactive_constraint_modes
```

Description

Returns a list of the MMMC constraint modes that are in interactive constraint entry mode.

When the software is in interactive constraint entry mode (by the set_interactive_constraint_modes), any specified timing constraints take effect immediately on all active analysis views that are associated with these constraint modes.

init_design

```
init_design [-top design]
```

Description

Initializes the database and ensures that the tool is ready for full execution.

During `init_design`, the tool steps through the defined MMMC objects, the design object, and the power requirements and builds the full representation of the design. After initialization is complete, the design is in a consistent state and is ready for manipulation (floorplanning, synthesis, and so on) or timing.

The `init_design` command corresponds to the *Init* step in the Initialization flow. In this step, the full database is prepared for execution. The constraints are loaded into the correct mode, the power data is loaded and applied to the appropriate instances, and the operating conditions are applied to the power domains. Extensive error checking is performed, including checking of objects, and checking for completeness of data. After the *Init* step is complete, the database is completely consistent and ready for execution.

The `init_design` command executes the following steps:

1. Power Domain Creation

First, the power domains are populated by the design data. The design instances associated with a particular domain are defined in the Common Power Format (CPF) or 1801 file. This instance data is used to populate the power domain attributes. At the end of this step, the different power domains are resolved, and every instance in the system is associated with its power domain.

2. Power Domain Binding to Domain Corners

After the power domains are fully resolved, they can be bound to the library sets in the various create_delay_corner command. In this step, the active domain corners are processed with each instance getting assigned to the appropriate library set.

3. Power Domain Object Processing

After the power domains are associated with library sets, the power domain objects (state retention, iso cells, and level shifters) can be processed. The power object rules are processed against the defined library sets for each domain to identify the power cells to be used.

4. Loading of Libraries

The library files (.lib) are loaded for each active analysis view. The library of the default analysis view are loaded with `read_mmmc`.

5. Timing Constraint Loading

The timing constraints (.sdc files) are loaded for all active analysis views defined with the `set_analysis_view` command. The SDC files correspond to the constraint modes of the active analysis views. The SDC files of inactive analysis views are also loaded, to keep the timing constraints in sync with the design when it is synthesized. This enables the SDC files of inactive analysis views to be saved during `write_mmmc` in sync with the design changes.In addition, the RC corner defined by the required domain_corner is processed and loaded. In both cases, syntax and error checking is performed.

After a design has been initialized with `init_design`, the design is ready for execution. Execution can include incremental updates to timing.

Options and Arguments

`-top` *design*	Specifies the name of the design.

Related Information

Related commands:

read_mmmc
read_netlist
read_physical

read_mmmc

`read_mmmc [-design` *`string`*`]` *`filename`*

Description

Reads the MMMC view definition file and populates the MMMC-related attributes.

The *`filename`* can be specified with absolute paths in the MMMC, or relative to the local directory or to the library search path attributes.

Reads in the Multi-Mode, Multi-Corner (MMMC) file. MMMC refers to the full set of timing libraries, constraints, operating conditions, timing conditions, RC corners, Delay corners and the associated analysis views.

When the `read_mmmc` command is run, the MMMC file is read and processed. The majority of elements in the MMMC control file are saved in the MMMC caching attributes. When `read_mmmc` command is run, only those library files are read which are associated with the first view defined by the `set_analysis_view` command in the MMMC file. No other data is loaded into the database at this point. This command does basic syntax checking on the MMMC commands, rejecting commands that have incorrect options and commands that are not MMMC or TCL commands. However, no consistency checking is performed. For example, if a constraint mode is used in an analysis view, but is not defined, it will not be flagged.

The MMMC file contents are cached in attributes in the database. These attributes allow you to write out a full and complete MMMC file later in the flow. In addition, the names of the MMMC files are cached for querying.

The exceptions to data set load are the libraries pointed by the first `set_analysis_view` command in the MMMC file. These library sets are loaded into the database. This load uses the standard library parser, and includes standard library syntax checking.

The `read_mmmc` command corresponds to the Timing step of the Initialization flow:

In the Timing step, the MMMC objects are set up by populating the basic MMMC information attributes. As synthesis needs library information before the Design step, the Timing step also loads some library information. The library data to be loaded is pointed to the tool by the first `set_analysis_view` command in the MMMC file.

At a minimum, the MMMC file must contain one each of:

- create_library_set
- create_opcond
- create_rc_corner
- create_timing_condition
- create_delay_corner
- create_constraint_mode
- create_analysis_view
- set_analysis_view

If the MMMC file does not contain this minimum set, an error will be issued.

The library sets can be specified with absolute paths in the MMMC, or relative to the local directory or to library search path attributes. Tool first attempts to load based on the name in the MMMC. If the library is not found, the tool sweeps through the search directory list in the specified order. Only the first hit is loaded.

Options and Arguments

-design *string*	Specifies the name of the design on which the MMMC settings will be applied.
filename	Specifies the name of the MMMC file to be read.
	The filename can be specified with absolute paths in the MMMC, or relative to the local directory or to the library search path attributes.

Example

When reading an MMMC file with read_mmmc, the tool sets the MMMC attributes with the data specified in the file. When the first set_analysis_view command is processed, the tool loads in the libraries specified by this analysis view. No other data is loaded.

For example, consider an MMMC file with the following commands:

```
create_library_set -name LS1 -timing [list lib1.lib lib2.lib lib3.lib]create_library_set -name LS2 -timing
[list lib4.lib lib5.lib lib6.lib]create_library_set -name LS3 -timing [list lib7.lib lib8.lib
lib9.lib]create_rc_corner -name QX -qrc_tech file.qrccreate_timing_condition -name TC1 -library_sets
LS1create_timing_condition -name TC2 -library_sets LS2create_timing_condition -name TC3 -library_sets
LS3create_delay_corner -name DC1 -timing_condition {TC1 PD2@TC2}  -rc_corner QXcreate_delay_corner  -name
DC2 -timing_condition {TC1 PD2@TC3} -rc_corner QXcreate_constraint_mode -name CM1 -sdc_files
cond1.sdccreate_constraint_mode -name CM2 -sdc_files cond2.sdccreate_analysis_view -name AV1 -
constraint_mode CM1 -delay_corner DC1create_analysis_view -name AV2 -constraint_mode CM2 -delay_corner
```

```
DC2set_analysis_view -setup AV1  AV2 -hold AV1 AV2
```

When the `set_analysis_view` command is processed, the tool traces backwards from that command to determine the library set. In this case, the analysis view specified by set_analysis_view is AV1. The AV1 analysis view points to domain corner DC1. DC1 points to timing condition TC1 and TC2.

This timing condition is attached to a power domain which is called as powerdomain@timingcondition (PD@TC). This PD@TC is treated as a library domain which is used by the tool to find/link all the cells present in a particular power domain to its .libs.. In the current example, all the cells in the power domain PD2 will be linked to the library sets attached to timing condition TC2 i.e. LS2.

Now DC1 points to two library sets, LS1 from TC1 and LS2 from TC2. As a result, lib1.lib, lib2.lib, lib3.lib, lib4, lib5, lib6 are loaded into the database.

In this case, lib7.lib lib8.lib and lib9.lib are not loaded into the database during `read_mmmc` because it is not in the library sets required by the analysis view.

Related Information

Related Attributes:

init_lib_search_path

Related commands:

init_design
read_libs
read_physical
read_netlist

set_analysis_view

```
set_analysis_view
[-setup { default_setup_view setup_view2 ...}]
[-hold { default_hold_view hold_view2 ...}]
[-leakage { analysis_view ...}][-dynamic { analysis_view ...}]
[-inactive { analysis_view ...}][-drv { analysis_view ...}]
```

Description

Defines the analysis views to use for setup and hold analysis and optimization. These "active" views represent the different design variations that will be analyzed. Active views can be changed throughout the flow to utilize different subsets of views. Genus can handle the views concurrently or sequentially, depending on their individual capabilities. Libraries and data are loaded into the system, as required to support the selected set of active views.

When the `set_analysis_view` command is issued, it will cause a full timing reset. During a timing reset, all the interconnect parasitics, delays, and timing slacks will be recalculated.

> The order in which you specify the views with the `-setup` and `-hold` parameters is important. By default, the first views defined in the `-setup` and `-hold` lists are the default views.

> Use this command after creating multiple analysis views using the create_analysis_view command.

Options and Arguments

`-dynamic {`*`analysis_view ...`*`}`	
	Sets the active view for dynamic power analysis.
`-hold {`*`default_setup_view setup_view2`*`...}`	
	Sets the active views for hold analysis.
	Not supported by Genus.
	Genus writes out the option when you specify the `write_mmmc` command.
`-leakage {`*`analysis_view ...`*`}`	
	Sets the active view for leakage power analysis.
`-setup {`*`default_setup_view setup_view2`*`...}`	
	Sets the active views for setup analysis.
`-inactive {`*`analysis_view ...`*`}`	
	Sets the active views for passing sdc only.
`-drv {`*`analysis_view ...`*`}`	Sets the active the views for passing sdc only.

Example

The following command sets missionWCCOM and mission2WCCOM as the active views for setup analysis, and missionBCCOM and testBCCOM as the active views for hold analysis:

```
set_analysis_view -setup {missionWCCOM mission2WCCOM} \
          -hold {missionBCCOM testBCCOM}
```

Related Information

Related commands:	create_analysis_view
	report_analysis_views
	update_analysis_view

set_interactive_constraint_modes

```
set_interactive_constraint_modes
constraint_modes
```

Description

Puts the software into interactive constraint entry mode for the specified multi-mode multi-corner constraint mode objects. Any timing constraints that you specify after this command takes effect immediately on all active analysis views that are associated with the specified constraint modes. By default, no constraint modes are considered active.

The software stays in interactive mode until you exit by specifying the set_interactive_constraint_modes command with an empty list:

```
set_interactive_constraint_modes { }
```

All new assertions are saved in the SDC files of the specified constraint modes when you save the design (`write_db`).

> Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode. In min/max analysis mode, constraints are always accepted interactively.

Options and Arguments

constraint_modes	Specifies the list of constraint modes to which subsequent constraints must be applied.

Example

The following commands put the software into interactive constraint entry mode, and applies a `set_clock_latency` assertion on all views in the current session that are associated with the constraint mode func1:

```
set_interactive_constraint_modes func1set_clock_latency 0.5 [all_clocks]
```

When you save the constraints, the updated constraints file for the func1 constraint mode will include the `set_clock_latency` assertion.

Related Information

Related commands:

create_constraint_mode
update_analysis_view

update_analysis_view

```
update_analysis_view -name view
[-constraint_mode constraint_mode]
[-delay_corner delay_corner]
[-power_modes list_of_power_modes][-latency_file file]
```

Description

Updates an existing analysis view (object) with a new constraint mode, or a new delay calculation corner.

> You can update an analysis view before (or after) the multi-mode multi-corner (MMMC) definition file is loaded into the design. However, after the software processes the file, any change to an existing object (in MMMC mode) may reset the timing, delay calculation, and RC extraction data.

Options and Arguments

`-constraint_mode` *constraint_mode*	
	Specifies the name of the new constraint mode to associate with this analysis view.
	A constraint mode groups a set of constraints to be used with a specific analysis view.
`-delay_corner` *delay_corner*	
	Specifies the name of the new delay corner to associate with this analysis view.
	A delay corner contains all of the information needed to control delay calculation for a specific analysis view.
`-latency_file` *file*	Specifies the name of the latency file to be updated.
`-name` *view*	Specifies the name of the analysis view to be updated.
`-power_modes` *list_of_power_modes*	
	Specifies a list of power modes to use with this analysis view.
	The power mode names must exist in the power intent file.

Examples

- The following command creates an analysis view called missionSlow using the constraint mode missionSetup and the delay calculation corner dcWCCOM:

  ```
  create_analysis_view -name missionSlow -constraint_mode missionSetup -delay_corner dcWCCOM
  ```

- The following command changes the delay calculation corner associated with the analysis view `missionSlow` to `dcMax`:

  ```
  update_analysis_view -name missionSlow -delay_corner dcMax
  ```

Related Information

Related commands:

create_analysis_view
create_constraint_mode
create_delay_corner

update_constraint_mode

```
update_constraint_mode -name constraint_mode
[-sdc_files list_of_files]
[-ilm_sdc_files list_of_files][-tcl_variables vars]
```

Description

Updates a constraint mode object.

Changes the SDC constraint file information for the specified existing constraint mode object.

> By default the `update_constraint_mode` command updates a constraint mode without affecting other constraint modes, including interactive constraints.

> You can update a constraint mode object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Options and Arguments

`-ilm_sdc_files` *list_of_files*	
	Specifies the new SDC constraint files for the ILM flow to associate with this constraint mode.
`-name` *constraint_mode*	
	Specifies the name of the existing constraint mode object to update.
	The constraint mode must have been defined with the `create_constraint_mode` command.
`-sdc_files` *list_of_files*	
	Specifies the list of new SDC constraint files to associate with this constraint mode.
`-tcl_variables` *vars*	
	Specifies a list of tcl variables to associate with this constraint mode.
	These are specified as {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}

Examples

- The following command groups the SDC files io.sdc, mission1-clks.sdc, and mission1-except.sdc to create a mode object named missionSetup:

```
create_constraint_mode -name missionSetup -sdc_files \
        [list io.sdc mission1-clks.sdc mission1-except.sdc]
```

- The following command changes the SDC files associated with the constraint mode missionSetup to io.sdc, mission3-clks.sdc, and mission3-except.sdc:

```
update_constraint_mode -name missionSetup -sdc_files \
         {io.sdc mission3-clks.sdc mission3-except.sdc}
```

Related Information

Related commands:

create_analysis_view
create_constraint_mode
update_analysis_view

update_delay_corner

```
update_delay_corner -name delay_corner
  [-timing_condition conditional_pairs]
  [-rc_corner rc_corner]
  [-temperature_files files]
  [-irdrop_files files ]
  [-irdrop_data string]
  [-early_timing_condition conditional_pairs]
  [-late_timing_condition conditional_pairs]
  [-early_rc_corner rc_corner]
  [-late_rc_corner rc_corner]
  [-early_temperature_files files]
  [-late_temperature_files files]
  [-pg_net_voltages pg_voltage_pairs]
  [-early_irdrop_files files]
  [-late_irdrop_files files]
  [-early_irdrop_data string]
  [-late_irdrop_data string]
  [-si_enabled {true|false}]
  [-early_estimated_worst_irdrop_factor float]
  [-late_estimated_worst_irdrop_factor float]
```

Description

Modifies the parameters of an existing delay calculation corner object. You can use the update_delay_corner command to add or change attribute values for an existing delay calculation corner object, or for an existing power domain definition.

You can update a delay calculation corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You also can use the update_delay_corner command to add a power domain definition to the specified delay calculation corner Instances contained within an MSMV power domain often require a different library or operating condition that used for the default power domain. Power domain definitions within a delay calculation corner object can be used to assign specific libraries and PVT settings per power domain.

Options and Arguments

```
-early_irdrop_files files
```

	Specifies the name of the IR drop files to apply for the early delay calculation for this delay corner.
`-early_irdrop_data` *string*	
	Specifies the IR drop data for early conditions.
`-early_rc_corner` *rc_corner*	
	Specifies the RC corner object to associate with the early delay corner.
	The RC corner must have been defined with the `create_rc_corner` command.
`-early_temperature_files` files	
	Specifies the name of the temperature files for temperature-aware delay calculation for an early corner.
`-early_timing_condition` *conditional_pairs*	
	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. Use one of the following formats for a conditional pair:
	• timing_condition
	Specifies the default timing condition which applies to all cells not covered by any other timing condition.
	• power_domain_list@timing_condition
	The named timing condition applies to the list of all power domains (defined in a power intent file).
	• supply_set_list@timing_condition
	The named timing condition applies to all blocks tied to the specified supply set (defined in an 1801 file).
	• hinst@timing_condition
	The named timing condition applies to the specified hierarchical instance.

	The timing conditions must have been defined with the `create_timing_condition` command.
`-irdrop_files` *files*	
	Specifies the name of IR drop files to apply for both the early and late delay calculation for this delay corner.
`-irdrop_data` *string*	
	Specifies the IR drop data.
`-late_irdrop_files` *files*	
	Specifies the name of the IR drop files to apply for the late delay calculation for this delay corner.
`-late_irdrop_data` *string*	
	Specifies the IR drop data for late conditions.
`-late_rc_corner` *rc_corner*	
	Specifies the RC corner object to associate with the late delay corner.
	The RC corner must have been defined with the `create_rc_corner` command.
`-late_temperature_files` files	
	Specifies the name of the temperature files for temperature-aware delay calculation for a late corner.
`-late_timing_condition` *conditional_pairs*	
	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. Use one of the following formats for a conditional pair:
	• timing_condition
	> Specifies the default timing condition which applies to all cells not covered by any other timing condition.
	• power_domain_list@timing_condition

	> The named timing condition applies to the list of all power domains (defined in a power intent file).
	• supply_set_list@timing_condition
	> The named timing condition applies to all blocks tied to the specified supply set (defined in an 1801 file).
	• hinst@timing_condition
	> The named timing condition applies to the specified hierarchical instance.
	The timing conditions must have been defined with the `create_timing_condition` command.
`-name` *`delay_corner`*	Specifies the delay calculation corner to which to add the power domain definition, or to which to update the delay corner or power domain attribute values.
	The delay corner must have been defined with a `create_delay_corner` command.
`-pg_net_voltages` *`pg_voltage_pairs`*	
	Specifies pair list of pg_nets and voltage sets to associate with this delay corner object. Use the following formats for a conditional pair: `pg_net@voltage`
`-rc_corner` *`rc_corner`*	
	Specifies the new RC corner object to associate with this delay corner.
	The RC corner must have been defined with the `create_rc_corner` command.
`-si_enabled`	Enables SI analysis.
	> Genus does not support SI analysis. Genus saves this option and writes it out when you specify the `write_mmmc` command.
`-temperature_files` *`files`*	
	Specifies the name of the temperature files for temperature-aware delay calculation for a single corner.

`-timing_condition` *`conditional_pairs`*	
	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. Use one of the following formats for a conditional pair.
	• timing_condition
	> Specifies the default timing condition which applies to all cells not covered by any other timing condition.
	• power_domain_list@timing_condition
	> The named timing condition applies to the list of all power domains (defined in a power intent file).
	• supply_set_list@timing_condition
	> The named timing condition applies to all blocks tied to the specified supply set (defined in an 1801 file).
	• hinst@timing_condition
	> The named timing condition applies to the specified hierarchical instance.
	The timing conditions must have been defined with the `create_timing_condition` command.
`-early_estimated_worst_irdrop_factor` *`float`*	
	Specifies the early estimated worst IR drop factor.
`-late_estimated_worst_irdrop_factor` *`float`*	
	Specifies the late estimated worst IR drop factor.

Example

The following command assigns timing condition `TC1` for the whole chip except for power domains `PD2` and `PD3` and associates the rail `VDD` with timing condition `TC3`.

```
update_delay_corner -name DC3 -timing_condition {TC1 {PD2 PD3}@TC2 VDD@TC3}
```

Related Information

Related commands:

create_delay_corner
create_rc_corner
create_timing_condition

update_library_set

```
update_library_set -name library_set
{-timing library_files |-target_timing library_list -link_timing library_list
[-aocv list_of_files]
[-socv list_of_files]
[-library_side_file string]
[-si  {lib1.cdb lib2.cdb lib3.udn lib4.udn…}]
```

Description

Changes timing and cdB library files for the specified existing library set.

Options and Arguments

-aocv *list_of_files*	
	Specifies the new list of AOCV libraries to be included in the library set.
-library_side_file *string*	
	Specifies the library side file to be included in the library set.
-link_timing list_of_files	
	Specifies a list of link libraries to be included in the library set.
	The link libraries contain the descriptions of cells (macros like RAMS, pads, PLLs, and so on) in a mapped netlist.

	This option must be specified with the `-target_timing` option and cannot be specified with the `-timing` option,
`-name` *library_set*	Specifies the name of an existing library set to be updated.
	The library set must have been defined with the `create_library_set` command.
`-si` *list_of_files*	Specifies the new list of cdB libraries and user-defined noise (UDN) models to include in the library set.
`-socv` *list_of_files*	
	Specifies the new list of SOCV libraries to be included in the library set.
`-target_timing` *list_of_files*	
	Specifies a list of target libraries to be included in the library set.
	The target libraries contain the cells used to generate the netlist and definitions for the design's operating conditions.
	This option must be specified with the `-link_timing` option and cannot be specified with the `-timing` option,
`-timing` *list_of_files*	
	Specifies the new list of timing libraries (.lib files) to include in the library set.

Examples

- The following command creates a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name `IsCOM-1V`:

```
create_library_set  -name IsCOM-1V \
 -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]\
 -si [list stdcell_F_2.cdb ram_F.cdb pad.cdb]
```

- The following command changes the timing libraries associated with the library set `IsCOM-1V`:

```
update_library_set -name IsCOM-1V \
 -timing [list stdcell_CVF_1V.lib ram_CVF.lib pad.lib ]
 -si {stdcell_CVF_2.cdb ram_CVF.cdb pad.cdb}
```

Related Information

Related command:

create_library_set

update_opcond

```
update_opcond -name operating_condition
[-process process]
[-voltage voltage]
[-temperature temperature]
[-tree_type string]
```

Description

Updates a set of virtual operating conditions in the specified library that were created using the `create_opcond` command.

Options and Arguments

`-name` *operating_condition*	
	Specifies the name of the operating condition to be updated.
	The operating condition must have been defined with the `create_opcond` command.
`-process` *process*	Specifies the process value of the operating condition.
`-temperature` *temperature*	
	Specifies the temperature of the operating condition in degrees Celsius.
`-tree_type` *string*	Specifies the tree type value. Possible values are:
	• `balanced_tree`
	• `best_case_tree`
	• `binary_tree`
	• `worst_case_tree`
`-voltage` *voltage*	Specifies the voltage of the operating condition in volt.

Examples

- The following command creates a virtual operating condition called `PVT1`

  ```
  create_opcond -name PVT1
  ```

- The following command adds the PVT values for the operating condition `PVT1`.

  ```
  update_opcond -name PVT1 -process 1.0 -voltage 1.2 -temperature 120
  ```

Related Information

Related commands:

create_opcond

update_rc_corner

```
update_rc_corner -name rc_corner
[-temperature temperature]
[-qrc_tech fileName ]
[-cap_table  capTable_file]
[-pre_route_cap capFactor ]
[-pre_route_res resFactor ]
[-post_route_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_res {resFactor1 resFactor2 resFactor3}]
[-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-pre_route_clock_cap capFactor ]
[-post_route_clock_cross_cap float+]
[-pre_route_clock_res resFactor ]
[-post_route_clock_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_clock_res {resFactor1 resFactor2 resFactor3}]
[-via_variation_file string]
```

Description

Adds or changes attribute values for the specified existing RC corner object.

> You can edit an RC corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

For some options you can specify one, two or three values.

Where:

- Single value: If you specify one value, the scale factor applies to effort level low. Scale factor value of 1 is used for medium and high by default.
- Duplet: If you specify two values, the first value is used for effort level low and the second value is used for medium. Scale factor value of 1 is used for high by default.
- Triplet: If you specify three values, the first value is used for effort level low, the second value is used for medium, and the third value is used for high.

Options and Arguments

`-cap_table` *capTable_file*	
	Specifies the capacitance table to use for extraction when using this RC corner.
`-name` *rc_corner*	Specifies the name of the RC corner to be updated.
	The RC corner must have been defined with the `create_rc_corner` command.
`-post_route_cap {`*capFactor1 capFactor2 capFactor3*`}`	
	Specifies the capacitance scale factor for RC extraction in post-route mode.
	Default: `1.0`
`-post_route_clock_cap {`*capFactor1 capFactor2 capFactor3*`}`	
	Specifies the scale factor for the capacitance of clock nets in post-route mode.
	When this parameter is specified, this value overrides the value specified with the `-post_route_cap` option for the clock nets.
	When this parameter is not specified, the value specified with the `-post_route_cap` option is used for pre-route extraction.
	Default: `1.0`
`-post_route_clock_cross_cap` *float+*	
	Specifies the scale factor for the coupling capacitances of clock in post-route mode.
`-post_route_clock_res {`*resFactor1 resFactor2 resFactor3*`}`	
	Specifies the scale factor for the resistance of clock nets in post-route mode.
	When this parameter is specified, this value overrides the value specified with the `-post_route_res` option for the clock nets.
	When this parameter is not specified, the value specified with the `-post_route_res` option is used for pre-route extraction.
	Default: `1.0`
`-post_route_cross_cap {`*capFactor1 capFactor2 capFactor3*`}`	
	Specifies the coupling capacitance scale factor for RC extraction in post-route mode.
	Default: `1.0`
`-post_route_res {`*resFactor1 resFactor2 resFactor3*`}`	

	Specifies the capacitance scale factor for RC extraction in post-route mode.
	Default: `1.0`
`-pre_route_cap` *capFactor*	
	Specifies the capacitance scale factor for RC Extraction in pre-route mode.
	Default: `1.0`
`-pre_route_clock_cap` *capFactor*	
	Specifies the scale factor for the capacitance of clock nets in pre-route mode.
	When this option is specified, this value overrides the value specified with the `-pre_route_cap` option for clock nets.
	When this option is not specified, the value specified with the `-pre_route_cap` option is used for pre-route extraction.
	Default: `1.0`
`-pre_route_clock_res` *resFactor*	
	Specifies the scale factor for the resistance of clock nets in pre-route mode.
	When this parameter is specified, this value overrides the value specified with the `-pre_route_res` option for the clock nets.
	When this parameter is not specified, the value specified with the `-pre_route_res` option is used for pre-route extraction.
	Default: `1.0`
`-pre_route_res` *resFactor*	
	Specifies the resistance scale factor for RC Extraction in pre-route mode.
	Default: `1.0`
`-qrc_tech` *fileName*	
	Specifies the name of an QRC technology (.tch) file to use for RC extraction when using this corner.
`-temperature` *temperature*	
	Specifies the temperature, in units of Celsius, to use to derate resistance values.

	Use this option when you want to override the default temperature specified in the capacitance table or the QRC technology file.
	Default: No value
`-via_variation_file` *string*	
	Specifies the name of the via variation file.

Examples

- The following command creates an RC corner called rc_cbest that uses the QRC Tech file qrc_tech file_bc.tch, and derates the resistance values based on the temperature of 50 Celsius:

  ```
  create_rc_corner -name rc_cbest -qrc_tech qrc_techfile_bc.tch -temperature 50
  ```

- The following command changes the QRC Tech file used by RC corner rc_cbest to new_qrc_techfile_bc.tch, and the temperature by which resistance values are derated to 25 Celsius:

  ```
  update_rc_corner -name rc_cbest -qrc_tech new_qrc_techfile_bc.tch -temperature 25
  ```

Related Information

Related commands:

create_rc_corner

update_timing_condition

```
update_timing_condition -name timing_condition
[-library_sets list]
[-opcond name] [-opcond_library library]
```

Description

Updates an existing timing condition with new operating conditions or library sets.

This command performs an update on the specified timing condition. The named condition must be present, or an error is issued. The options completely replace the contents of the specified attribute.

Options and Arguments

`-library_sets` *list*	Specifies the list of library sets to associate with this timing condition.
	The library sets must have been defined with the `create_library_set` command.
`-name` *timing_condition*	
	Specifies a unique name for the timing condition. This name must exist or an error is displayed.
	The timing condition must have been defined with the `create_timing_condition` command.
`-opcond` *opcond*	Specifies a single name for the operating condition to associate with the timing condition.
	The operating condition must have been defined with the `create_opcond` command.
`-opcond_library` *library*	
	Specifies the name of the library to search for the specified operating condition.

Examples

- The following example shows the `update_timing_condition` command performs an update on the specified timing condition:

```
create_timing_condition -name TC1 -library_sets {ls1 ls2} -op_cond oc1

update_timing_condition -name TC1 -library_sets {ls3};
                                        # TC1 now only has libset ls3
```

- The following commands result in identical results. However, the first command creates the timing condition TC2 and the second command updates both the options:

```
create_timing_condition -name TC2 -library_sets {ls1 ls2} -op_cond oc1
update_timing_condition -name TC2 -library_sets {ls1 ls2} -op_cond oc1
```

Related Information

Related commands:

create_library_set
create_opcond

create_timing_condition

update_lef_macro

```
update_lef_macro
lef_file
-macros string
```

Description

Reads the physical design information from the specified LEF file and update macros.

Options and Arguments

lef_file	Specifies the LEF file to read.
-macros *string*	Specifies a list of macros to update. If not specified all macros in the LEF file will be updated.

20

Navigation

- get_logical_name
- get_object_type
- set_top_module
- vbasename
- vcd
- vdirname
- vdirs
- vfind
- vls
- vname
- vpopd
- vpushd
- vpwd

get_logical_name

```
get_logical_name [inst|hinst|net|hnet|port_bus|port|pg_pin|pin|pin_bus
|hpin_bus|hpin|hport|module|design]...
```

Description

Returns the Verilog name of the specified objects.

> You can specify a list of objects and the tool returns a list.

get_object_type

```
get_object_type
object...
```

Description

Returns a string describing the type of object passed to the command as its argument. The

command will work only if a single object is specified.

This command is mostly used for writing Tcl scripts (rather than being an interactive command). It is useful for checking the types of arguments passed to the Tcl procedures.

Options and Arguments

object	Specifies the object for which you want to know the type.

set_top_module

`set_top_module [`*design*`]`

Description

Changes to the new top module of the specified design.

Options and Arguments

design	Specifies the name of the design.

vbasename

`vbasename` *pathname*

Description

Removes the leading virtual directory (vdir) names of the virtual path name and returns only the object name. This command behaves similarly to the UNIX `basename` command.

Options and Arguments

pathname	Specifies the virtual path name of the object, including the object name.

Example

- The following example removes the virtual directory name of the `CW_absval` ChipWare component and only returns the component name:

```
genus:root:> vbasename hdl_component:CW/CW_absval

CW_absval
```

Related Commands

vdirname

vcd

`vcd [ vdir ]`

Description

Sets the current virtual directory (vdir) in the design hierarchy and navigates the design hierarchy. This command is similar to its UNIX counterpart, but used in the virtual directory.

Options and Arguments

vdir	Specifies the name of the vdir to be set as the current vdir.
	The ".", "..", and "/"have the same meaning as their UNIX counterparts (current vdir, parent vdir, and root vdir respectively).
	You can use wildcards when they do not produce an ambiguous reference (more than one match).

Examples

- The following command returns you to the top of the design hierarchy:

```
genus:root:.designs> vcd

genus:root:> vpwd

root:
```

- The following command specifies the absolute path to the target vdir:

```
genus:root:> vcd /designs/alu/timing/exceptions

genus:design:alu.timing.exceptions>
```

- The following command specifies the relative path to the target vdir:

```
genus:root:> vcd designs/alu

genus:design:alu> vcd timing/exceptions

genus:design:alu.timing.exceptions>
```

- The following command changes the current vdir to a parent vdir:

```
genus:design:alu.timing.exceptions> vcd ../../modules

genus:design:alu.modules> vpwd

design:alu.modules
```

- The following command uses wildcards in the path specification:

```
genus:design:alu.modules> vcd ../tim*/exc*

genus:design:alu.timing.exceptions> vpwd

design:alu.timing.exceptions
```

- The following command uses object name for the path specification:

```
genus:root> vcd lib_cell:default_emulate_libset_max/typical/BUFXL

genus:lib_cell:default_emulate_libset_max/typical/BUFXL>
```

Related Information

Affects these commands:

vdirs
vls
vpopd
vpushd
vpwd

vdirname

`vdirname` *`pathname`* `[-times` *`integer`*`]`

Description

Removes the object name of the specified path name and returns only the vdir name. This command behaves similarly to the UNIX `dirname` command.

Options and Arguments

`pathname`	Specifies the path name of the object, including the object name.
`-times` *`integer`*	Specifies the number of times to apply vdirname.
	Default: `1`

Examples

- The following example removes the `CW_absval` ChipWare component name and only returns its vdir name:

  ```
  genus:root:> vdirname hdl_component:CW/CW_absval

  hdl_lib:CW
  ```

- The following command applies the `vdirname` command 2 times to the specified path.

  ```
  genus:root:> vdirname -times 2 hdl_component:CW/CW_absval.implementations

  hdl_lib:CW
  ```

Related Information

Related commands:

vbasename

vdirs

```
vdirs
```

Description

Displays the contents of the design virtual directory (vdir) stack. This command is similar to its UNIX counterpart and is used in conjunction with the vpushd and vpopd commands.

Examples

- The following commands respectively add designs and libraries to the design vdir stack, then display the contents of the vdir stack:

  ```
  genus:root: xx> vpushd designs root:.designs root:

  genus:root:designs xx> vpushd libraries root:.libraries root:.designs root:

  genus:root:libraries xx> vdirs root:.libraries root:.designs root:
  ```

- The following commands respectively remove the last added directories and then display the contents of the vdir stack:

  ```
  genus:root:libraries xx> vpopd root:.designs root:

  genus:root:designs number> vpopd root:

  genus:root: xx> vdirs root:
  ```

- The following command uses object name for the path specification:

  ```
  genus:root: xx> vcd lib_cell:default_emulate_libset_max/typical/BUFXL
  genus:lib_cell:default_emulate_libset_max/typical/BUFXL xx>
  ```

Related Information

Related commands:

vls
vpopd
vpushd
vpwd

vfind

```
vfind {[root_path]
[-maxdepth integer] [-mindepth integer]
[-regexp expression] [-invert expression]
[-ignorecase] [-split] [-vname] {-option...|-*} object}
```

Description

Searches the design hierarchy for the specified types of objects and returns a Tcl list containing the full paths to any matching objects. This list can then be used by other commands to operate on groups of objects. The `vfind` command supports the `*` and `?` wildcard characters.

> The `vfind` command is very powerful but overusing it can increase the execution time or lead to memory issues. To avoid issues, be as specific as possible when specifying the object to match. For example,
>
> - Traverse the design recursively on hierarchical instances, instead of starting from the root directory
> - Use the `-vname` option where possible.

> vfind is similar to the legacy RTL Compiler `find` command and is retained for users who are comfortable with database searching in this way. However, it may not be supported by all tools. Therefore, consider using `get_db` instead.

Options and Arguments

`-ignorecase`	Ignores the case (upper case or lower case) of the parameters. Alternatively, specifies the search to be case insensitive.
`-invert` *`expression`*	
	Excludes the objects specified by the regular expression.
`-maxdepth` *`level`*	Descends no more than the specified number (non-negative integer) of levels below *`root_path`*. A level of `0` searches only the `root_path`. When using this option, you need to take into account the object containers of the virtual directory structure.
	Default: infinity
`-mindepth` *`integer`*	Skips the specified number (non-negative integer) of levels below *`root_path`* before finding objects. A level of `1` searches all objects except *`root_path`*. When using this option, you need to take into account the object containers of the virtual directory structure.

	Default: `0`
object	Specifies the name of the object to match. The name can include wildcard characters.
-option	Specifies the type of object you want to find. You can specify multiple options or look in all types by specifying *. Check the command help for a list of the valid object types.
-regexp *expression*	Specifies to find the specified regular expression.
root_path	Specifies the name of the virtual directory from where to start searching. The name can include wildcard characters (*).
	By default, an explicit search is done for the specified object. During an explicit search every object directory is searched, instead of starting from *root_path*. Specifying a *root_path* reduces the numbers of locations where the `vfind` command will search which reduces the execution time.
-split	Causes the command to return one object per line.
	This option is for interactive use only: the Tcl result is set to null and the objects are only printed to the screen.
-vname	Removes the container directories in the path name and returns Verilog style names where appropriate. This option will only work on the following objects: `pin`, `port`, `net`, `module`, and `instance`.

Examples

- The following example returns the Verilog names of all hierarchical instances up to one level below the top level hierarchical instance `top_hinst_a`:

```
genus:root:> vfind hinst:chip_top/top_hinst_a -maxdepth 2 -vname -hinst *
top_hinst_a/sub_hinst_a
top_hinst_a/sub_hinst_b
top_hinst_a/sub_hinst_c
top_hinst_a/sub_hinst_d
```

- The following example returns leaf pins matching the `Q*` pattern, such as `Q` and `QN`, which are two hierarchy levels below `top_hinst_a` but no more than 3:

```
genus:root:> vfind hinst:chip_top/top_hinst_a -mindepth 6 -maxdepth 8 -pin Q*
pin:chip_top/top_hinst_a/sub_hinst_a/ct_custom_regs/r_toggle_reg_0/Q
pin:chip_top/top_hinst_a/sub_hinst_a/ct_custom_regs/r_toggle_reg_0/QN
pin:chip_top/top_hinst_a/sub_hinst_a/ct_custom_regs/err_IN_reg_0/Q
pin:chip_top/top_hinst_a/sub_hinst_a/ct_custom_regs/err_IN_reg_0/QN

...
```

vls

```
vls [-computed] [-attribute] [-long] [-dir]
         [-width integer] [object]... [>file]
```

Description

Lists information about the objects in the design hierarchy (designs, library cells, clocks, and so on). This command is similar to its UNIX counterpart, but used in the virtual directory (vdir). The vdir contains all objects related to designs, libraries, flows, internal commands & messages, and more.

> Option `-long` and `-attribute` can be combined as `-la` or `-al`.

Options and Arguments

`-attribute`	Lists the attributes for the specified object whose values are different from the default values.
`-computed`	Lists all computed attributes. Computed attributes are potentially very time consuming to process and are therefore by default not listed.
`-dir`	Does not list contents of the vdir. Only the vdir names are listed.
`file`	Specifies the name of the file to which to redirect the information.
`-long`	Lists the contents (long listing) of the vdir.
`object`	Specifies the vdir for which you want to list information.
	Default: current vdir
`-width` *`integer`*	Specifies the width of the screen that can be used to show the information.
	Default: `80`

Examples

- The following command lists the contents of the `libraries` vdir:

```
genus:root:> vls libraries
root:.libraries:
./                          library_sets/               rc_corners/
base_cells/                 opconds/                    timing_conditions/
```

- The following command shows information of cell `buf1` in regular and long listing format:

```
genus@base_cell:NAND2X4> vls
./                      base_pins/              pg_base_pins/

genus@base_cell:NAND2X4> vls -long
Total: 3 items
./                          (base_cell)
base_pins/
pg_base_pins/
```

- The following command lists only the attributes of `buf1` whose values are different from the default values:

```
genus@base_cell:NAND2X4> vls -attribute
Total: 3 items
./                            (base_cell)
    Attributes:
      area = 3.1464
      base_class = CORE
      base_name = NAND2X4
      base_pins = base_pin:NAND2X4/A base_pin:NAND2X4/B base_pin:NAND2X4/Y
      class = CORE
      escaped_name = NAND2X4
      is_combinational = true
      is_timing_defined = true
      lib_cells = lib_cell:default_emulate_libset_max/slow_1v0/NAND2X4
      name = NAND2X4
      num_base_pins = 3
      obj_type = base_cell
      pg_base_pins = pg_base_pin:NAND2X4/VSS pg_base_pin:NAND2X4/VDD
base_pins/
pg_base_pins/
```

- The following command lists all attributes (except for the computed attributes) for `buf1`, even those with the default value:

```
genus@base_cell:NAND2X4> vls -long -attribute
Total: 3 items
./                            (base_cell)
    All attributes:
      area = 3.1464
      base_class = CORE
      base_name = NAND2X4
      base_pins = base_pin:NAND2X4/A base_pin:NAND2X4/B base_pin:NAND2X4/Y
      class = CORE
      clock_gating_integrated_cell =
      escaped_name = NAND2X4
      is_buffer = false
```

```
        is_combinational = true
        is_fall_edge_triggered =
        is_flop = false
        is_integrated_clock_gating =
        is_interface_timing = false
        is_inverter = false
        is_isolation_cell = false
        is_latch = false
        is_macro = false
        is_master_slave_flop = false
        is_master_slave_lssd_flop = false
        is_memory = false
        is_negative_level_sensitive =
        is_pad = false
        is_pll = false
        is_positive_level_sensitive =
        is_rise_edge_triggered =
        is_sequential = false
        is_timing_defined = true
        is_timing_model = false
        is_tristate = false
        level_shifter_type =
        lib_cells = lib_cell:default_emulate_libset_max/slow_1v0/NAND2X4
        name = NAND2X4
        num_base_pins = 3
        obj_type = base_cell
        pg_base_pins = pg_base_pin:NAND2X4/VSS pg_base_pin:NAND2X4/VDD
base_pins/
pg_base_pins/
```

Related Information

Affects these commands:

vdirs
vcd

vpopd
vpushd
vpwd

vname

`vname [inst | hinst | net | hnet | pin | hpin | pg_pin | port_bus | pin_bus | hpin_bus | port | port | hport | module | design]...`

Description

Returns the design hierarchical name of the specified objects. Generally, this command is used with `get_*` command.

You can specify a list of objects and the tool returns a list.

Example

Command `get_db` returns the Genus hierarchical name, use `vname` to get design hierarchical name:

```
genus:root:> get_db ports port_pad_data_out[6]{port:top/port_pad_data_out[6]}

genus:root:> vname [get_db port_pad_data_out[6]]port_pad_data_out[6]
```

vpopd

`vpopd`

Description

Removes the topmost element of the virtual directory (vdir) stack, revealing a new top element and changes the current vdir to the new top element. This command is similar to its UNIX counterpart `popd`.

If `vpopd` is issued on a vdir stack that has only one element, an appropriate warning message is printed and the `vpopd` command exits without changing the vdir stack.

Examples

- Starting from root vdir and going to the `design:top` vdir, and come back to root:

```
genus:root:> vpushd design:top
design:top root:

genus:design:top number> vpwd design:top

genus:design:top number> vpopd root:

genus:root:> vpwd
root:
```

- Error message is issued when running `vpop` command on an empty vdir stack:

```
genus:root:> vpopd

Error : Directory stack is empty. [TUI-56] [popd_internal]
      : Can't execute 'popd' command on an empty vdir stack.
Failed on popd
```

Related Information

Affects these commands:

vdirs
vls
vpushd
vpwd

vpushd

`vpushd` *vdir*

Description

Pushes the specified new target virtual directory (vdir) onto the vdir stack (as the topmost element) and changes the current vdir to that specified vdir. This command is similar to its UNIX counterpart `pushd`.

Options and Arguments

vdir	Specifies the name of the vdir to be set as target vdir. You can use wildcards when they do not produce an ambiguous reference (more than one match).

Example

Starting from root vdir and going to the `design:top` vdir:

```
genus:root:> vpushd design:topdesign:top root:
```

```
genus:design:top> vpwddesign:top
```

Going back to the root vdir:

```
genus:design:top> vpopdroot:
```

```
genus:root:> vpwdroot:
```

Related Information

Affects these commands:

vdirs
vls
vpopd
vpwd

vpwd

`vpwd`

Description

Displays the current position in the design hierarchy. This command is similar to its UNIX counterpart `pwd`.

Example

The following example shows the current virtual directory (vdir) after the vdir is changed:

```
genus:root:> vcd designgenus:root:.design> vpwdroot:.designs
```

Related Information

Related Commands:

- vdirs
- vls
- vpopd
- vpwd

21

Physical

- check_floorplan
- check_pbs_mim_module
- check_placement
- convert_polygon_to_boxes
- create_clip
- create_floorplan
- create_group
- create_place_blockage
- create_place_halo
- create_ple_model
- create_property_alias
- create_region
- create_route_blockage
- create_route_halo
- create_route_rule
- create_route_type
- create_row
- create_single_row
- create_track
- cut_power_domain_by_overlaps
- def_move
- delete_rows
- duplicate_register
- floorplan_resize_blockage
- floorplan_resize_boundary
- get_computed_shapes
- get_obj_in_area
- modify_power_domain_attr
- move_blockage
- move_boundary
- move_instance
- move_obj
- move_port
- place_dft
- place_dft_sequentials
- predict_floorplan
- read_clip
- read_congestion_map
- read_def
- read_encounter
- read_parasitics
- read_physical
- read_sdp_file
- set_cell_padding
- specify_cell_pad
- update_congestion_map
- update_group
- update_inst
- update_instance_data
- update_power_domain
- write_congestion_map
- write_def
- write_lib_lef
- write_parasitics
- write_preserves
- write_reports
- write_sdp_file

The following commands are common with Analysis and Reporting commands:

- report_congestion
- report_summary
- report_utilization

check_floorplan

```
check_floorplan
[-spatial] [-out_file string] [-error] [-detailed] [-max_per_violation string]
```

Description

Checks for the validity of the floorplan.

You can use this command in a script to ensure that there is some valid physical information available before you continue with the next steps in the flow. Run this command after reading the floorplan and before performing physical synthesis.

Options and Arguments

`-error`	Specifies to return 0 if the checks have errors.
	This option should be used along with `-spatial` option.
`-out_file` *string*	Specifies the output filename.
`-spatial`	Specifies to perform Innovus floorplan and library checks.
-detailed	Returns detailed violation report.
-max_per_violation *integer*	
	Specifies the maximum number of objects to print for each violation.
	This option should be used along with the `-detailed` option.

check_pbs_mim_module

```
check_pbs_mim_module
 [-limit integer]
  module
  string
  [file]
```

Description

Checks whether the given module can be set as a MIM module.

Options and Arguments

`-limit` *integer*	Specifies the message limit for each uniquification reason.
module	Specifies the module for which MIM requirements need to be checked.
string	Specifies the name of the file to which the output is written.

check_placement

```
check_placement [design]
[-file file] [-verbose]
```

Description

Checks the placement legality and highlights illegal objects. The command also returns some instance status statistics.

> This command applies only to the Genus-Physical flow.

Options and Arguments

design	Specifies the name of the design for which to create the group.
-file *file*	Specifies the file to which to write the report.
-verbose	Specifies to generate a verbose report.

convert_polygon_to_boxes

```
convert_polygon_to_boxes pt pt pt [pt]...
[-dbu_in] [-dbu_out]
```

Description

Converts a polygon into a list of boxes.

> This command applies only to the Genus-Physical flow.

Options and Arguments

-dbu_in	Specifies that the input coordinates are in Database Units.
-dbu_out	Specifies that the output coordinates are in Database Units.
pt pt pt [pt]...	Specifies a list of the coordinates of at least three points that define the region polygon. Use floats for the coordinates.

create_clip

```
create_clip
-outdir string -instances {inst|hinst}...
[-logical_only] [design]
```

Description

Generates timing constraints and physical data for a design clip, that allow each unit (clip) to be synthesized separately but with the full design context.

These data should be generated after an initial synthesis of the full design.

Options and Arguments

design	Specifies the name of the design.
	This parameter is only required if multiple designs are loaded.
-instances {*inst*|*hinst*}	
	Specifies a list of instances that identify the clip.
-logical_only	Skips the generation of the physical data.
-outdir *string*	Specifies the directory in which to generate the data.

Example

Following is the suggested flow usage.

```
source setup.tclread_hdl all_hdl_fileselaborate top_designread_sdc top_design.sdcread_def def_f
ileset_db <block_units>.boundary_opto false syn_generic -physicalsyn_map -physicalsyn_opt -
spatialgenerate_units ...
```

Related Information

Affects this command:

read_clip

create_floorplan

```
create_floorplan
[-core_margin_by {io|die} ][-die_size_by_io_height {min|max}]
[-flip {n|s|f}][-floorplan_origin {center | llcorner}][-match_to_site]
[-no_resize] [-no_snap_to_grid] [-overlap_same_site_row]
[-box_size float... | -core_size float... | -die_size float... |
     -core_density_size float... | -stdcell_density_size float...|
     -keep_shape_density float... ]
[ -site string | -site_only string ]
```

Description

Specifies the floorplan information for the specified design.

Specifies floorplan dimensions by size; or by die and core coordinates. The `create_floorplan` command initializes the floorplan, creates rows and adds track information to create new routing tracks.

Options and Arguments

`-box_size` *float*	Specifies the coordinates of all two boxes of the floorplan. They are in the format <*die_box core_box*>.
	The boxes are specified using the lower left and upper right coordinates. Specify the coordinates in micron.
`-core_density_size` *float*	
	Specifies the aspect-ratio, density and core to edge. It is specified as: <*aspectratio* [*rowdensity* [*left bottom right top*]]>
`-core_margin_by` {`io` | `die`}	
	Specifies the core margin to be either io or die.
`-core_size` *float*	Specifies size by core size and core to edge: <`width height left bottom right top`>
`-die_size` *float*	Specifies DIE size and core to edge <`width height left bottom right top`>
`-die_size-by_io_height {min|max}`	

	Specifies to use max or min io height for initial die size.
`-flip {n|s|f}`	Specifies to flip rows.
`-floorplan_origin {center | llcorner}`	
	Specifies to set floorplan origin at `center` or `llcorner`.
`-keep_shape_density` *float*	
	Specifies target utilization after resize.
`-match_to_site`	Specifies that the floorplan boxes are multiple times of io site or core site's width.
`-no_resize`	Specifies to check if there is a need to do floorplanning if everything is same. If they are same, it specifies not to do floorplanning again.
`-no_snap_to_grid`	Specifies not to change input floorplan boxes.
`-overlap_same_site_row`	
	Specifies to allow same site row overlapping.
`-site` *string*	Specifies the default tech site name.
`-site_only` *string*	Specifies the default tech site name and to create row for the site.
`-stdcell_density_size` *float*	
	Specifies the aspect-ratio, density and core to edge. It is specified as: <*aspectratio* [*rowdensity* [*left bottom right top*]]>

Related Information

Affects these commands:

syn_generic -physical
syn_opt

Related commands:

create_row

check_floorplan
delete_rows
predict_floorplan

create_group

```
create_group group
-name name
-type {fence | region | guide | cluster}
[-density float  [-aspect_ratio float]]
[ -polygon pt pt pt [pt]...
| -rects {{llx lly} {urx ury}...}
[-is_physical_hier] [-floating]
```

Description

Creates the physical container for a group of objects that must be placed close together. Objects to be contained can be hierarchical instances, leaf instances, or other groups. Objects need to be associated with a group by `update_group` command afterwards.

The command returns the path to the `group` object that it creates. You can find the objects created by the `create_group` command as:

```
vls /designs/design/physical/groups/
OR
get_db groups
```

> Groups are used to create a new logical hierarchy unit, and normally all power domains associate with a group which always have a boundary constraint fence. The boundary constraint fence means instances belonging to this group cannot be placed outside and only the instances belonging to this group can be placed inside. Cadence recommends that this physical-logical coherence should not be violated. The coherence requirement does not only apply for groups, but also pertains to all hinsts and groups as long as they have boundary constraint fence.

In other words, if fence A belongs to fence B in logical, then fence A must be placed inside fence B.

Options and Arguments

`-aspect_ratio` *float*	
	Specifies the aspect ratio. Specify a number from 0 to 1.
	Default: `1`
`-density` *float*	Specifies a placement density. Specify a number from 0.0 to 1.0, use the decimal format, where 0.75 is for 75% density.
`-is_physical_hier`	Specifies that the created group is a physical hierarchy. By default, the created group is not saved back to the netlist.
	Not supported in Genus.
`-name` *name*	Specifies the name of the group to be created.
`-polygon` *pt pt pt* `[`*pt*`]...`	
	Specifies a list of the coordinates of at least three points that define the group polygon. Use floats for the coordinates.
`-floating`	Specifies that the created group is a floating group.
`-rects {{`*llx lly*`} {`*urx ury*`}...}`	
	Specifies the coordinates of the rectangular boxes that define the group.
`-type`	Specifies the type of the group to be created. You can choose between fence, region, or guide.
	`fence`: Specify to create a group of fences. Instances belonging to this group need to be placed inside of the specified box/polygon shape exclusively. The other instances cannot be placed inside.
	`region`: Specify to create a group of regions. Instances belonging to this group need to be placed inside of the specified box/polygon shape as a hard constraint. However, the other instances can be also placed inside.
	`guide`: Specify to create a group of guides. Instances belonging to this group are basically placed inside of the specified box/polygon shape but are allowed to go outside of it. Guide is treated as a soft constraint.

	`cluster`: Specify to create a group of clusters.

Examples

- The following command creates group adder1:

```
create_group -name adder1
```

- The following command creates group test1 and its placement guide at the specified location:

```
create_group -name test1 -type guide -rects {2345.3 9962.1 4930.0 18244.8}
```

- To add hierarchical instance or leaf instances to this group:

```
update_group -name test1 -add -obj top/u1
```

- To confirm what groups exist

```
get_db groups
group:top/test1
```

- To confirm which instances belong to that group

```
get_db -computed group:top/test1 .members
{inst:top/u1/a...}
```

- To confirm the constraint type (`region` below is an object type, not a constraint type)

```
get_db group:top/test1 .region
test1
```

```
get_db region:top/test1 .type
guide
```

Related Information

Affects these commands:

syn_generic -physical
syn_opt

Related command:

update_group

create_place_blockage

```
create_place_blockage [-name name | -use_prefix ]
[-type {hard | macro_only | partial | soft}
[-density float] [-exclude_flops]]
[-no_cut_by_core ][-snap_to_site ]
{ -area x1 y1 x2 y2}
| -polygon {x1 y1 x2 y2 ...}
| -rects {x1 y1 x2 y2 ...} }
{ -inst inst | -hinst hinst
| -all_macros | -all_partitions}
[-cover]
[ -inner_ring_by_side {left bottom right top}
| -inner_ring_by_edge {edge1 edge2 edge3 ...} ]
[ -outer_ring_by_side {left bottom right top}
| -outer_ring_by_edge {edge1 edge2 edge3 ...} ]
```

Description

Creates cell placement blockages that can be placed even outside the core area. A placement blockage is a floorplan object used to block standard cell placement.

You can use the `delete_obj` command to delete placement blockages.

Options and Arguments

`-all_macros`	Creates a placement blockage around all macros.
	Genus does *not* support this option.
`-all_partitions`	Creates a placement blockage around all partitions.
	Genus does *not* support this option.
`-area {llx lly urx ury}`	

	Specifies the lower left and upper right coordinates of the blockage area. Use floats for the coordinates.
`-cover`	Specifies that a placement blockage of the same size as the specified block instance will be created on top of the instance.
	Specify the `-inst` option with this option.
`-density` *float*	Specifies the density for partial blockages. Enter a value between 100 and 0. For example, a partial placement percentage of 75 percent means that up to 75 percent of placement density is allowed in the area.
	Use this option with the partial blockage type specified using `-type partial`.
`-exclude_flops`	Prevents flops and latches to be placed in a partial placement blockage during global placement.
	Use this option with the partial blockage type specified using `-type partial`.
`-hinst` *hinst*	Specifies the name of the hierarchical instance on which the placement blockage must be applied.
	Genus does *not* support this option.
`-inner_ring_by_edge {`*edge1 edge2 edge3 ...*`}`	
	Creates a hollow ring of placement blockage around the inside of the object.
	Specified distance is from the object boundary to the inner edge of the ring for every edge.

	This option must be used with -inst, -hinst or -all_partitions parameters.
-inner_ring_by_side {*left bottom right top*}	
	Creates a hollow ring of placement blockage around the inside of the object.
	Specified distance is from the object boundary to the inner edge of the ring for the left, bottom, right, and top sides respectively.
	This option must be used -inst, -hinst or -all_partitions parameters.
-inst *inst*	Specifies the name of the instance on which the placement blockage must be applied.
-name *name*	Specifies the name of the blockage.
	If no name or no prefix is specified, the blockage name defaults to defScreenName_*n*
-no_cut_by_core	Specifies that placement blockages should not be cut by row area in the core. It allows creation of placement blockages that overlap the core boundary and the IO ring area.
	Genus does *not* support this option.
-outer_ring_by_edge {*edge1 edge2 edge3 ...*}	
	Creates a hollow ring of placement blockage around the outside of the object.
	Specified distance is from the object boundary to the inner edge of the ring for every edge.

	This option must be used -`inst`, -`hinst` or -`all_partitions` parameters.
-`outer_ring_by_side` {*left bottom right top*}	
	Creates a hollow ring of placement blockage around the outside of the object.
	Specified distance is from the object boundary to the inner edge of the ring for the left, bottom, right, and top sides respectively.
	This option must be used -`inst`, -`hinst` or -`all_partitions` parameters.
-`polygon` {*x1 y1 x2 y2...*}	
	Specifies the polygon vertices of the blockage, in microns.
-`rects` {*x1 y1 x2 y2...*}	
	Specifies the box vertices of the blockage, in microns.
-`snap_to_site`	Snaps the newly created placement blockage to a row site.
	Genus does *not* support this option.
-`type` {`hard` | `macro_only` | `partial` | `soft`}	
	Specifies the type of blockage to be created.
	• `hard`: The area cannot be used to place blocks or cells at any time during the session.
	• `macro_only`: This creates a macro only placement blockage area.
	• `soft`: The area cannot be used to place blocks during standard cell placement, but can be used during in-place optimization, clock tree synthesis, or ECO Placement.

	• `partial`: Sets a percentage of the area that is available for placement using the create_place_blockage command.
	Default: `hard`
`-use_prefix`	Specifies to name the placement blockages with a prefix.
	If no name or no prefix is specified, the blockage name defaults to `defScreenName_`*n*

Example

The following command creates a soft blockage, softBlockage2 in the design area.

```
create_place_blockage -area {3442.3600 3739.2000 3511.6500 5716.6300} \  -type soft -name
softBlockage2
```

Related Information

Related attribute

floorplan_default_blockage_name_prefix

Related commands:

- delete_obj
- move_blockage
- floorplan_resize_blockage

create_place_halo

```
create_place_halo
[-halo_deltas float...] [-orient string]
[-snap_to_site] [-insts {inst|hinst}] [-cell {inst|hinst}] [-all_macros] [-all_black_boxes] [-
all_commit_partitions] [-all_blocks] [-all_io_pads] [{inst|hinst}]
```

Description

Creates a placement halo around a block. A placement halo is an area that prevents the placement of blocks and standard cells within the specified halo distance from the edges of a hard macro to reduce congestion. Use this command during or after floorplanning.

You can use the `delete_obj` command to delete placement halos.

Options and Arguments

`-all_black_boxes`	Specifies that it is for all black-boxes.
`-all_blocks`	Specifies that it is for all blocks.
`-all_commit_partitions`	
	Specifies that it is for all partitions.
`-all_io_pads`	Specifies that it is for all io-pads.
`-all_macros`	Specifies that it is for all macros.
`-cell {`*`inst`*`|`*`hinst`*`}`	
	Specifies the cell name.
`-halo_deltas` *`float`*	
	Specifies a halo of four sides {`left bottom right top`}
`-insts {`*`inst`*`|`*`hinst`*`}`	
	Specifies the instance name.
{*`inst`*`|`*`hinst`*`}`	Specifies the instance around which the placement halo is created.
-`orient` *`string`*	Specifies the orientation.
`-snap_to_site`	Specifies that the halo box will snap to row site.

One DB unit is 1/1000 micron.

Related Information

Affects these commands:

syn_generic -physical

Related commands:

create_place_blockage
delete_obj
move_blockage
floorplan_resize_blockage

create_ple_model

```
create_ple_model [design] [-out_file file]
```

Description

Creates PLE correlation data for the design and stores this data in the specified file.

Net capacitances and resistances depend on technology parameters as well as the floorplan. This command refines the PLE parameters by taking both these variables into account and by comparing the PLE data with the SPEF data from Innovus. This results in a highly customized PLE equation for the given design and technology libraries.

The generated file is an encrypted file that contains

- Average Capacitance and Resistance values based on placement and default routing
- Adjustments for PLE equation parameters

The header of the generated file is readable. Check the header against the current design data to avoid miscorrelation. The header might look like:

```
# DESIGN NAME: DTMF_CHIP# TECHNOLOGY LEF: all.lef# CAP-TABLE: typical.captbl# CAP SCALE: 1.0# RES SCALE:
1.0# ASPECT RATIO: 0.9814
```

If the design data is not inconsistent, the following message will be issued:

```
Warning : Inconsistent data. [PHYS-600]        : Original design "DTMF_CHIP" not found in current
session.        : Input data used to create PLE correlation file is different from data used in this
```

```
session. This might lead to invalid results. Check design data.
```

Run this command once to generate PLE correlation data separately for each design. Source the generated file for every subsequent run that starts from RTL.

This command does not require a floorplan, though having a good floorplan is highly recommended.

Options and Arguments

design	Specifies the design for which to generate the PLE data.
-out_file *file*	Specifies the name of the output file in which to store the PLE correlation data.

Related Information

Affects these commands:

syn_generic
syn_map
syn_opt

create_property_alias

```
create_property_alias
-map string
-obj_type string
```

Description

Creates property alias.

Options and Arguments

`-map` *string*	Specifies pairs of property and its alias.
`-obj_type` *string*	Specifies the `obj_type` of the property.

create_region

```
create_region
[region] [-name string] [-area float...]
```

Description

Creates a region with the specified name. The command returns the path to the `region` object that it creates.

The user-defined name is stored in the `def_name` region attribute.

Options and Arguments

`-area` *pt pt pt pt* [*pt*]...	
	Specifies a list of the coordinates of at least four points that define the region polygon. Use floats for the coordinates.
`-name` *string*	Specifies the name of the region.
region	Specifies the name of the region to be created.

Related Information

Affects these commands:	syn_generic -physical
	syn_opt
Related commands:	move_boundary
	floorplan_resize_boundary
Related Attributes:	region Attributes

create_route_blockage

```
create_route_blockage [-name name | -use_prefix]
[-fills] [-partial integer]
[-inst inst [-cover]]
[-layers  layer_list | -all {cut|route}...
{ -polygon {x1 y1 x2 y2 x3 y3... }
| -rects {x1 y1 x2 y2 ... } }
[-except_pg_nets | -pg_nets]
[-spacing float | -design_rule_width float ]
```

Description

Creates a routing blockage on the specified layer. You can use the delete_obj command to delete routing blockages.

Options and Arguments

`-all {cut|route}`	Specifies the layer type for which all layers must be blocked.
`-cover`	Specifies that a routing blockage of the same size as the specified block instance will be created on top of the instance.

	Default: If you do not specify this parameter, the Innovus software will check if any default blockage layers have been specified with the `set_route_blockage_default_layer` command. If yes, the layers specified with the `set_route_blockage_default_layer` command will be used. If no, the default is wire3 if the number of layers is greater than 3, or wire2 if the number of layers is less than or equal to 3.
`-design_rule_width` *`float`*	
	Specifies that the blockage has a width of *`effective_width`* for the purposes of spacing calculations. If you specify `-design_rule_width`,you cannot specify `-spacing`.
	The *`effective_width`* is a real number greater than 0, in microns.
	Genus does *not* support this option.
`-except_pg_nets`	Specifies that the routing blockage is to be applied on a signal net routing and not on power or ground net routing. Use this option to block signal routing above or around a sensitive block to avoid noise from nearby signal nets but still allow power connections to go through the blockage.
	Blocking the signal net routing helps in avoiding cross talk or coupling caused by signal routes.
`-fills`	Specifies that the routing blockage is to be applied on metal fills.
`-inst` *`inst`*	Specifies the name of the instance on which the placement blockage must be applied.
`-layers` *`layer_list`*	
	Specifies the layer, list of layers, or all layers on which the routing blockage is to be applied. The allowed layers are routing layer and cut layer.

	You must specify the `layer_name` as a number or as a layer name defined in LEF. For example, `layer_name` can be specified as 2 or metal2.
`-name` *name*	Specifies the name of the routing blockage.
	If no name or no prefix is specified, the blockage name defaults to `defScreenName_`*n*
`-partial` *integer*	Specifies to create a partial routing blockage with the specified density.
	`min`=0, `max`=100
`-pg_nets`	Specifies that the routing blockage is to be applied only on power or ground net special routes and not on signal nets. This option affects only commands that create special routes, such as add_rings, add_stripes, or route_special when they are used for PG nets. It does not affect signal routing of PG nets done by `route_early_global`. NanoRoute, which is used to connect tie-high or tie-low connections or to connect to secondary standard-cell power-pin connections, is also not affected by this option.
	Use this option during power planning to prevent power routes from getting too close to block edges and, as a result, blocking signal pin access or causing congestion in narrow channels.
	Genus does *not* support this option.
`-polygon {`*x1 y1 x2 y2 x3 y3 ...*`}`	
	Specifies the polygon vertices of the blockage, in microns.
	Although `create_route_blockage` can be used to create triangular blockages, metal fill does not currently support triangular route blockages.
`-rects {`*x1 y1 x2 y2 ...*`}`	
	Specifies the coordinates of a list of blockage areas, in microns.

`-spacing` *float*	Specifies the minimum spacing allowed between the blockage and any other routing shape. If you specify `-spacing`, you cannot specify `-design_rule_width`.
	The float is a real number greater than or equal to 0, in microns.
`-use_prefix`	Specifies that the routing blockages should be named with a prefix.
	If no name or no prefix is specified, the blockage name defaults to `defScreenName_`*n*

Examples

- The following command creates a routing blockage object to block Metal3 routing in the specified area:

```
create_route_blockage -rects {2353.8000 7123.000 2653.8000 8298.2000} -layers Metal3
```

- The following command creates a routing blockage object on metal layers M1, M4, and M5:

```
create_route_blockage -rects {0 0 100 100} -layers {M1 M4 M5}
```

- The following command creates a routing blockage object on metal layers M1, M4 and blocks the signal net routing:

```
create_route_blockage -rects {10 18 1782 2300} -layers {M1 M4} -except_pg_nets
```

- The following command creates a routing blockage object having polygon vertices (10, 10), (10, 15), (15, 15), (15, 20), (20, 20), (20, 10) on the metal layer M1:

```
create_route_blockage -polygon {10 10 10 15 15 15 15 20 20 20 20 10} -layers M1
```

Related Information

Related attribute

floorplan_default_blockage_name_prefix

Related commands:

create_place_blockage
delete_obj

move_blockage
floorplan_resize_blockage

create_route_halo

```
create_route_halo
[-bottom_layer layer] [-space float] [-top_layer layer]
[-litho_halo] [-all_blocks] [-block string]
[-cells inst] [-insts inst]
[-design_halo] [inst]
```

Description

Adds a routing halo around a block. A routing halo restricts wire routing within the specified area for specific layer(s). Use this command during or after floorplanning.

You can use the `delete_obj` command to delete routing halos.

Options and Arguments

`-all_blocks`	Specifies to add halo for all blocks.
`-block` *string*	Specifies the block names to add halo to.
`-bottom_layer` *layer*	
	Specifies the lowest routing layer for which to create the halo blockage.
`-cells` *inst*	
	Specifies the cells to add halo to.
`-design_halo`	Specifies to add halo for all partitions.
`-insts` *inst*	
	Specifies the instances to add halo to.
inst	Specifies the instance around which the routing halo is created.
`-litho_halo`	Specifies to add litho halo to the instances.
`-space` *float*	Specifies the space on four sides.
`-top_layer` *layer*	Specifies the highest routing layer for which to create the halo blockage.

Related Information

Affects these commands:

syn_generic -physical
syn_opt

Related commands:

create_place_blockage
move_blockage
floorplan_resize_blockage

create_route_rule

```
create_route_rule
[-name string] [-hard_spacing] [-generate_via]
[-via string] [-add_via string] [-min_cut string]
[-use_via_cut_class string]
[ -spacing string | -spacing_multiplier string]
[ -width string | -width_multiplier string] [-init string]
```

Description

Specifies to create routing rules. This command adds a non-default rule (NDR) for objects. You can also remove vias from the NDRs. Use this command after you read the design.

Options and Arguments

`-add_via` *string*	Specifies a list of vias to add.
	The format of the string is: {*vianame1 vianame2* ...}
`-generate_via`	Specifies to auto-generate the vias.
`-hard_spacing`	Specifies hard spacing.
`-init` *string*	Specifies the name of existing non-default rule from which all values are initialized.
	Other options will overwrite these values.
`-min_cut` *string*	Specifies the minimum cut limit.
	The format of the string is: {<*starting layer*>:<*ending layer*> <*min cut*>}

`-name` *string*	Specifies the name of the non-default rule.
`-spacing` *string*	Specifies the spacing for the routing layers.
	The format of the string is: {<*starting layer*>:<*ending layer*> <*spacing*>}
`-spacing_multiplier` *string*	
	Specifies a multiplier to default spacing of routing layers.
	The format of the string is: {<*starting layer*>:<*ending layer*> <*spacing multiplier*>}
`-use_via_cut_class` *string*	
	Specifies the layer and `cut_class` list to support the LEF/DEF property.
`-via` *string*	Specifies a list of vias.
`-width` *string*	Specifies the width of the routing layers.
	The format of the string is: {<*starting layer*>:<*ending layer*> <*width*>}
`-width_multiplier` *string*	
	Specifies a multiplier to default width of routing layers.
	The format of the string is: {<*starting layer*>:<*ending layer*> <*width multiplier*>}

Example

The following command creates a rule that requires extra spacing. All other values are the same as the default rule:

```
create_route_rule -name default_2x -spacing {metal1:metal4 0.2} \ -spacing_multiplier {metal1:metal4
1.0} -width {metal1:metal4 0.2}\ -width_multiplier {metal1:metal4 1.0}
```

Related Information

Related commands:

create_route_halo
syn_generic -physical
predict_floorplan

create_route_type

```
create_route_type
[ -name string | -append_default_route ]
[ -non_default_rule string | -route_rule string ]
[-shield_net string] [-bottom_preferred_layer string]
[-top_preferred_layer string]
[ -preferred_routing_layer_effort {low|medium|high} | -route_effort {low|medium|high} ]
[-em_route_rule string] [-em_route_rule_distance float]
[-layer_mask_range string] [-mask integer]
[-bottom_mask_layer_num integer] [-top_mask_layer_num integer]
[-min_stack_layer string] [-one_side_spacing_range string]
[-prefer_multi_cut_via] [-shield_side {one_side|both_side}]
[-stack_distance float] [-input_stack_via_rule string]
[-output_stack_via_rule string] [-set_ple_data] [-populate_from_ple]
```

Description

Specifies to add the routing type.

Options and Arguments

`-name` *string*	Specifies the name of the route type.
`-append_default_route`	
	Specifies to add a new `route_type` to the list of default route types and follow default route type naming conventions.

	The default value is false.
`-non_default_rule` *string*	
	Specifies the non-default rule name.
`-route_rule` *string*	
	Specifies the non-default rule name.
`-shield_net` *string*	
	Specifies the shield net name.
`-bottom_preferred_layer` *string*	
	Specifies the bottom preferred layer.
`-top_preferred_layer` *string*	
	Specifies the top preferred layer.
`-preferred_routing_layer_effort {low|medium|high}`	
	Specifies the preferred routing layer effort
`-route_effort {low|medium|high}`	
	Specifies the preferred routing layer effort.
`-em_route_rule` *string*	
	Specifies the EM `route_rule` to associate with the route type. When routing within the distance specified in `em_route_rule_distance` from the output pin, the router will use this `route_rule` to route.
	The default rule used is `em_route_rule`.
`-em_route_rule_distance` *float*	
	Specifies the distance from the output pin, when the `em_route_rule` is applied to the net. This allows a larger width `route_rule` for a short distance to avoid EM violations near the output pin. When routing outside this distance from the output pin, the router uses the normal net `route_rule`.

`-layer_mask_range` *string*	
	Specifies the layer range that the mask constraint should be applied on. For example, 3:4 means that the constraint should be applied on M3 and M4.
`-mask` *integer*	Indicates mask number for multiple mask layer usage. Refer to layers `.numMask` attribute for valid range, 0 indicates unconstrained. Layers that do not support the specified value will be treated as unconstrained. (Legal range: 0-3)
`-bottom_mask_layer_num` *integer*	
	Specifies the bottom layer number that the mask constraint should be applied to.
`-top_mask_layer_num` *integer*	
	Specifies the top layer number that the mask constraint should be applied.
`-min_stack_layer` *string*	
	Specifies that the net should use a stacked via from output pins up to the given layer before starting normal routing. This is normally used to force the routing to higher layers with wider widths to reduce wiring resistance or avoid EM current limits for high-drive outputs. If `output_stack_via_rule` or `input_stack_via_rule` is also specified, the specified stack via rule is used for the input or output pins accordingly. If both `output_stack_via_rule` and `input_stack_via_rule` are not specified, a single-cut stacked-via will be used for the output pins only.
`-one_side_spacing_range` *string*	
	Specifies the layer range that one side spacing constraint should be applied on.
`-prefer_multi_cut_via`	
	Specifies whether or not to use multi-cut vias for connections at the net driver pins.
`-shield_side {one_side|both_side}`	
	Specifies whether to perform one sided or two sided shielding for the specified route type.
`-stack_distance` *float*	
	Specifies that the cut distance of cuts on adjacent layers in the stacked vias are defined in `min_stack_layer`.
`-input_stack_via_rule` *string*	

	Specifies a stacked via rule name, which must match one of the LEF STACKVIARULE names.
`-output_stack_via_rule` *string*	
	Specifies a stacked via rule name, which must match one of the LEF STACKVIARULE names.
`-populate_from_ple`	
	Computes the average physical characteristics using PLE LEF file, if available. The default value is false.
-set_ple_data	Specifies to use the route type to set the value of the resistance and capacitance per unit length.

create_row

```
create_row row
[-site site_name]
[-flip_first_row | -no_flip_rows]
[-no_abut | -no_abut_first ]
[-limit_rows_in_core]
[-no_check]
[-spacing  distance ]
[-area {x1 y1 x2 y2} | -rects {{x1 y1} {x2 y2}...}
| -polygon {{x1 y1} {x2 y2}...}] [design]
```

Description

Creates rows for the specified site. You can specify the row boundary or use the core area. This command can create overlapping rows. This command can only create horizontal rows. By default, the rows are flipped and abutted.

The following restrictions apply:

- Rows can be created both inside and outside the core area, but within the die.
- All new rows created in an area must be an integer multiple of any existing rows in the same area.
- Non-integer multiple height rows cannot be created outside power domains.

You can only use this command after you read the design.

Options and Arguments

`-area {x1 y1 x2 y2}`	
	Specifies the coordinates of the area in which the rows must be created.
	• x1 specifies the lower left x coordinate of the area.
	• y1 specifies the lower left y coordinate of the area.
	• x2 specifies the upper right x coordinate of the area.
	• y2 specifies the upper right y coordinate of the area.
	If you do not specify this option, the core area is used by default.
design	Specifies the name of the design for which to create the row.
`-flip_first_row`	Specifies that the first row is flipped mirrored to its x-axis.
	This option cannot be used with the `-no_flip_rows` option.
`-limit_rows_in_core`	
	Specifies that new rows must be limited inside the core.
`-no_abut`	Prevents that rows are abutted (created back to back).
	By default, the rows are abutted.
`-no_abut_first`	Specifies that the first row should not be abutted.
	This option cannot be used with the `-no_abut` option.
`-no_check`	Specifies that the row should not be aligned to the existing row and should support redo undo.
`-no_flip_rows`	Prevents that the rows are flipped.
	By default, the rows are flipped.
`-polygon {{x1 y1} {x2 y2}...}`	
	Specifies the coordinates of the polygon in which the rows must be created.

`-rects {{x1 y1} {x2 y2}...}`	
	Specifies the coordinates of the boxes in which the rows must be created.
`-spacing float`	Specifies the spacing between rows.
	Default: `0.0 um`
`-site site_name`	Specifies the name of the site to be used for creating the rows.

Related Information

Affects these commands:

syn_generic -physical

create_single_row

```
create_single_row row
-macro string
-orientation {N|S|E|W|FN|FS|FE|FW}
-llx integer -lly integer
-height integer -width integer
[-no_update] [-design design]
```

Description

Creates a row with the specified name. Coordinates and dimensions must be specified in DB units, where one DB unit is 1/1000 micron.

The command returns the path to the `row` object that it creates.

Options and Arguments

`-design` *design*	Specifies the name of the design for which to create the region.
`-height` *integer*	Specifies the height of the row.
`-llx` *integer*	Specifies the lower left X coordinate of the row.
`-lly` *integer*	Specifies the lower left Y coordinate of the row.
`-macro` *name*	Specifies the name of the LEF site to be used for the row.
`-no_update`	Prevents an update of the GUI when the new row is created.
`-orientation`	Specifies the orientation of the row to be created.
`-width` *integer*	Specifies the width of the row.
row	Specifies the name of the row to be created.

Related Information

Affects these commands:	syn_generic -physical
Related Attributes:	row Attributes

create_track

```
create_track
-start integer -step integer -num integer
-layer string... [-vertical] [-design design]
```

Description

Creates physical tracks (or routing grid) for the specified layer. This command creates non-uniformed, customized tracks in selected design area. The first track is located at an offset from the placement grid set by the OFFSET value for the layer in the LEF file. The track spacing is the PITCH value for the layer defined in LEF.

Options and Arguments

`-design` *design*	Specifies the name of the design in which the tracks must be created.
`-layer` *string*	Specifies the routing layer for which the tracks are created.
`-num` *integer*	Specifies the number of tracks to be created.
`-start` *integer*	Specifies the starting X or Y coordinate. X coordinate applies to vertical tracks Y coordinate applies to horizontal tracks.
`-step` *integer*	Specifies the spacing between the tracks.
	If not specified, step value will be the pitch of the layer defined in the techfile.
`-vertical`	Specifies to create vertical tracks.
	By default, a horizontal track is created.

Related Information

Affects these commands:

syn_generic -physical

cut_power_domain_by_overlaps

`cut_power_domain_by_overlaps`
power_domain

Description

In case of overlapping power domains, cuts the specified outer power domain in a donut shape.

> Overlapping power domains are not allowed for floorplanning. So, the inner power domain must be cut out from the outer power domain.

Options and Arguments

power_domain	Specifies the name of the outer power domain.

def_move

```
def_move
[-initialize]
[-highlight]
[-min_distance float]
```

Description

Highlights cell movement in the physical tab of the GUI.

Use this command after you run the `syn_opt -spatial` command.

Options and Arguments

`-highlight`	Highlights the cell movement with respect to their last stored location.
`-initialize`	Stores the location of the instances at the time the command is given with this option.
`-min_distance` *float*	
	Limits the highlighting to cells that have been moved more than the specified distance. Specify the distance in microns.
	If this option is omitted, highlighting is limited to those cells whose x and y locations both changed by a value greater than or equal to the row height.

Example

Following illustrates the usage:

```
def_move -initialize

syn_opt -spatial

def_move -highlight
```

Related Information

Related command:

syn_opt

delete_rows

```
delete_rows
[-all] [-selected] [-site string] [-row {row [row]}...]
```

Description

Deletes specified rows.

Options and Arguments

`-all`	Specifies to delete all the rows.
`-selected`	Specifies to delete the selected rows.
`-site string`	Specifies to delete the rows from the specified site.
`-row row`	Specifies the row(s) to be deleted.

Example

The following command deletes all existing rows from the database:

```
delete_rows [get_db rows *]
```

Related Information

Related command:

create_row

duplicate_register

```
duplicate_register
inst...
[-num_copies integer] [-fanout_list {{pin|hpin|port}...}]...
| -timing integer]
[-verbose] [-local] [-no_cross] [-score]
```

Description

Duplicates the specified register(s) to reduce the load on the register outputs and thus improving the timing. The duplicated registers will be part of the same hierarchy as the original register. Duplication is controlled by the specified options. You can run this command any time after the design has been elaborated. If the design has been placed, the placement of the leaf instances in the fanout will be taken into account.

Options and Arguments

`-fanout_list {pin|hpin|port}`	
	Specifies a list of fanouts (pins or ports) that require a dedicated register. You can specify the option multiple times.
	The endpoints do not have to be input pins of registers.
inst	Specifies the list of registers to be duplicated.
`-local`	Only allows duplication of the registers if the leaf instances in the fanout of the register also belong to the same hierarchy as the original unduplicated register.
`-no_cross`	Only allows duplication of the registers if there is no buffer or inverter tree between the register and the leaf instances.
`-num_copies`	Specifies the number of copies that must be made of each specified register.
`-score`	Only allows duplication of the registers if it improves the timing of the design.
`-timing` *integer*	Specifies the number of loads with the worst slacks for which a dedicated register must be used.
	Default: `0`
`-verbose`	Prints information about the duplicated registers.

Related Information

Affected by this attribute:

boundary_opto

floorplan_resize_blockage

`floorplan_resize_blockage` *blockage*
`[-left` *integer*`] [-right` *integer*`]`
`[-top` *integer*`] [-bottom` *integer*`]`

Description

Resizes the specified blockage. All distances must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

blockage	Specifies the name of the blockage to be resized.
-bottom *integer*	Specifies the distance over which the bottom edge of the blockage must be moved.
-left *integer*	Specifies the distance over which the left edge of the blockage must be moved.
-right *integer*	Specifies the distance over which the right edge of the blockage must be moved.
-top *integer*	Specifies the distance over which the top edge of the blockage must be moved.

Related Information

Affects these commands:

syn_generic -physical

Related commands:

create_place_blockage
create_place_halo
move_blockage

floorplan_resize_boundary

```
floorplan_resize_boundary region
[-left integer] [-right integer]
[-top integer] [-bottom integer]
```

Description

Resizes the specified region. All distances must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

`-bottom integer`	Specifies the distance over which the bottom edge of the region must be moved.
`-left integer`	Specifies the distance over which the left edge of the region must be moved.
`-right integer`	Specifies the distance over which the right edge of the region must be moved.
`-top integer`	Specifies the distance over which the top edge of the region must be moved.
region	Specifies the name of the region to be resized.

Related Information

Affects these commands:

syn_generic-physical

Related commands:

create_region
move_boundary

get_computed_shapes

```
get_computed_shapes
[-output area|hrect|polygon|rect ] [-step string]
[-maxPoint integer] [-d] [string]
```

Description

This command is used for shape manipulation.

Options and Arguments

`-output` *string*	Specifies the output format.
	It can be one of the following: {polygon rect hrect area}
	Default: rect
`-step` *string*	Specifies the step size if the output format is rect or hrect.
`-maxPoint` *integer*	Specifies the maximum number of points for output in a polygon.
	Minimum is 5 and maximum is 8000
`-d`	Specifies the unit of measurement. User specified values and return values are in database units.
	Default: `um`
string	Provides a list of shapes as per the following syntax: a polygon `{{x y} {x y} ...}`, `rect {xl yl xu yu}`. You can also provide a list of polygons and/or rects `{{{x1 y1} {x2 y2} ...} {xl yl xu yu} ...}`
	Use the following operators on the list: AND | ANDNOT | OR | XOR | SIZE | MOVE | BBOX | INSIDE | OUTSIDE | ENCLOSE | STRADDLE | HOLES | NOHOLES

get_obj_in_area

```
get_obj_in_area
-area float...
[-dbu]
-obj_type {bump|inst|pcell|pin|port|place_blockage|route_blockage|row}
[design]
```

Description

This command returns a list of objects inside a defined area.

Options and Arguments

`-area` *float*	Specifies the lower left and upper right coordinates of the search area in the format: `{llx lly urx ury}` Use floats for the coordinates.
	Default: microns
`-dbu`	Signifies that the specified values are in Database Units.
`-obj_type {bump|inst|pcell|pin|port|place_blockage|route_blockage|row}`	
	Specifies the object type to return.
design	Specifies the name of the design.

modify_power_domain_attr

```
modify_power_domain_attr

power_domain
[-design design]
[-rs_exts {T B L R}]
[-min_gaps {T B L R}]
[-box_list  {llx   lly   urx   ury}...]
[-disjoint_hinst_box_list   {{hier_inst_list region} ...}]
```

Description

Creates or modifies the physical boundary for the specified power domain.

> Braces in bold must be entered literally!

Options and Arguments

`-box_list {llx lly urx ury}`	Specifies a list of boxes (rectangular areas) within the design area that serve as the boundary for the power domain. For each box, specify the coordinates of the lower-left and upper-right corners. Use floating values.
`-design design`	Specifies the name of the design to which the power domain belongs. This option is only required if multiple designs are loaded.
`-disjoint_hinst_box_list{{hier_inst_list region}...}`	Associates a list of hierarchical instances with a region in which they can be placed. You can specify multiple combinations.
`-min_gaps {T B L R}`	Defines the distance, in microns, that must be reserved from the power domain boundary edges for power routing. Specify a list of four floating values, one for each edge.
power_domain	Specifies the name of the power domain for which the physical boundary is created or modified. The name must correspond to the power domain name given in CPF.
`-rs_exts {T B L R}`	Specifies the boundary for legal targets to be used by the power planning and routing commands, in conjunction with the power domain boundary. Specify a list of four floating values, one for each edge.

Related Information

Related attribute:

Power domain attributes

move_blockage

```
move_blockage blockage
[-x integer | -dx integer]
[-y integer | -dy integer]
```

Description

Moves the specified blockage. You can specify either absolute coordinates, or a delta change in the coordinates. Coordinates or changes in coordinates must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

`-dx integer`	Specifies to move the blockage over the specified distance in the x-direction.
`-dy integer`	Specifies to move the blockage over the specified distance in the y-direction.
`-x integer`	Specifies the new x-coordinate for the lower-left corner of the blockage.
`-y integer`	Specifies the new y-coordinate for the lower-left corner of the blockage.
blockage	Specifies the name of the blockage to be moved.

Related Information

Affects these commands:

syn_generic -physical

Related commands:

create_place_blockage
create_place_halo
floorplan_resize_blockage

move_boundary

```
move_boundary region
[-x integer | -dx integer]
[-y integer | -dy integer]
```

Description

Moves the specified region. You can specify either absolute coordinates, or a delta change in the coordinates. Coordinates or changes in coordinates must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

`-dx integer`	Specifies to move the region over the specified distance in the x-direction.
`-dy integer`	Specifies to move the region over the specified distance in the y-direction.
`-x integer`	Specifies the new x-coordinate for the lower-left corner of the region.
`-y integer`	Specifies the new y-coordinate for the lower-left corner of the region.
region	Specifies the name of the region to be moved.

Related Information

Affects these commands:

syn_generic -physical

Related commands:

create_region
floorplan_resize_blockage

move_instance

```
move_instance {pcell|inst|hinst} [-x integer | -dx integer] [-y integer | -dy integer] [-dbu]
```

Description

Moves the specified instance. You can specify either absolute coordinates, or a delta change in the coordinates. Coordinates or changes in coordinates must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

`-dbu`	Specifies to use DB Units for coordinates.
`-dx` *`float`*	Specifies to move the instance over the specified distance in the x-direction.
`-dy` *`float`*	Specifies to move the instance over the specified distance in the y-direction.
`-x` *`float`*	Specifies the new x-coordinate for the lower left corner.
`-y` *`float`*	Specifies the new y-coordinate for the lower left corner.
`pcell|inst|hinst`	Specifies the name of the instance to be moved.

Related Information

Affects these commands:

syn_generic -physical

move_obj

```
move_obj {inst|hinst} [-group] [-point float...] [-direction string] [-to string] [-distance float]
[-selected] [{fpobject|region|port|blockage|inst|hinst}...]
```

Description

Moves the specified object. All coordinates or changes in coordinates must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

`-direction` *string*	Specifies the direction to move.
	{`up down left right`}
`-distance` *float*	Specifies the distance by which objects will be moved to in the specified direction.
fpobject|*region*|*port*|*blockage*|*inst*|*hinst*	
	Specifies the objects to be moved.
`-group`	Specifies to move all specified objects as a group.
`-point` *float*	Specifies the new lower left point to where the objects will be moved.
`-selected`	Specifies to move selected object.
`-to` *string*	Specifies to move object to core box or die box boundary.

Related Information

Affects these commands:

syn_generic -physical

move_port

```
move_port port
[-x integer | -dx integer]
[-y integer | -dy integer]
[-placement_status string] [-dbu]
```

Description

Moves the specified port. You can specify either absolute coordinates, or a delta change in the coordinates. Coordinates or changes in coordinates must be specified in DB units, where one DB unit is 1/1000 micron. You must specify at least one option.

Options and Arguments

`-dbu`	Specifies to use DB Units for coordinates.
`-dx` *integer*	Specifies to move the port over the specified distance in the x-direction.
`-dy` *integer*	Specifies to move the port over the specified distance in the y-direction.
`-x` *integer*	Specifies the new x-coordinate for the center of the port.
`-y` *integer*	Specifies the new y-coordinate for the center of the port.
port	Specifies the name of the port to be moved.
`-placement_status` *string*	
	Specifies the placement status of the port. You can specify:
	`fixed`—port cannot be moved by automatic tools
	`placed`—port can be moved by automatic tools

Related Information

Affects these commands:

syn_generic -physical

place_dft

`place_dft` *design*

Description

Helps to place inserted unplaced logic without disturbing global placement. Useful in cases that after a full physical flow has been run, additional netlist altering commands are also run, resulting in small amounts of mapped but unplaced logic.

> While this command is usually used to place unplaced DFT logic in the design, it will place any unplaced logic. Also, due to the incremental nature of the command, good results will only appear if the percentage of unplaced logic is relatively low. If a big part of the design is unplaced please run a global placement.

Options and Arguments

design	Specifies the design in which the DFT logic is to be placed.

place_dft_sequentials

```
place_dft_sequentials
[-unplaced_insts instances]
[-list_failed] [-max_print integer]
[-preview]
```

Description

Places wrapper cells and test point flops.

Use this command to place wrapper cells and test points inserted after `syn_map` was run.

> This command requires a `Modus_DFT_Opt` license.

Options and Arguments

`-list_failed`	Prints the instances that could not be placed.
`-max_print` *integer*	
	Specifies the maximum number of instances for which to report the placement location.
	Default: `20`
`-preview`	Reports the instances that would be placed with their location without placing them.
`-unplaced_insts` *instances*	
	Specifies the list of DFT instances to be placed.
	By default, the tool attempts to place all unplaced flops and latches.

predict_floorplan

```
predict_floorplan
[-constraints file] [-script string] [design]
```

Description

Creates a floorplan by running Innovus Master Plan.

This command invokes Innovus to create a floorplan. By default it will create a square floorplan with an utilization target. You can use option -`constraints` and -`script` to specify constraints/settings to Innovus.

It is recommended to set `innovus_executable` before executing this command.

> This floorplan is just a rough starting point and ultimately the user should carefully create a floorplan based on the needs of the design. Things like floorplan size, port locations, macro placement, obstructions, regions can have a very significant impact on the timing, congestion and overall QOR. Hence all these should be defined carefully in a floorplan created by the user. Genus created floorplan is a very preliminary estimate of design QOR while running a Spatial/Physical flow.

Options and Arguments

`-constraints` *file*	Specifies the constraints file for Innovus planDesign.
design	Specifies the design for which to create the floorplan.
`-script` *file*	Specifies the script file for Master plan.

Example

The following commands tell Genus to create a square floorplan with utilization target 0.5.

```
set_db $your_design .utilization_threshold 0.5

predict_floorplan -script ./invs_postload.tcl
```

Related Information

Affects these commands:

syn_generic -physical
syn_opt -spatial

read_clip

```
read_clip
[-keep_existing] [-no_def] [-no_constraints]
[-input_scale float] [-output_scale float]
-directory string  [design]
```

Description

Reads the clipper data for the specified design.

Clipper data consists of

- Timing constraints for the clip in Genus Tcl format
- Physical information for the clip in DEF format
- A (binary) database representing the results of the clip operation

> If you would read the clip information by sourcing the constraints and using the read_def command, error messages could be generated if pins have changed (if you updated the RTL), By using the `read_clip` command, the messages about different pins are captured and organized.

Options and Arguments

design	Specifies the name of the design.
	If the design is not specified, Genus will search for a unique design and apply the clipper data to that design.
	This parameter is only required if multiple designs are loaded.
-directory *string*	Specifies the location of the clip data.
-input_scale *float*	
	Multiplies all input constraints by this value. This option allows you to scale input arrival delays.
	Default: 1.0
-keep_existing	Retains constraints currently applied to the design. By default, all constraints are removed prior to reading the clip data.
	For example, if you applied a clock, using this option will prevent the clip data from overwriting that clock.
-no_constraints	Prevents reading the constraints file. Use this option when only the DEF data is desired.
-no_def	Prevents reading the DEF file. Use this option when only logical analysis is needed.
-output_scale *float*	
	Multiplies all output constraints by this value. This option allows you to scale output required times.
	Default: 1.0

Related Information

Related command:

create_clip

read_congestion_map

```
read_congestion_map [-design design] file
```

Description

Restores the congestion map data from the specified file.

> This command applies only to the Genus-Physical flow.

Options and Arguments

`-design design`	Specifies the design for which the congestion map must be restored.
file	Specifies the name of the file containing the congestion data.

Related Information

Related commands:

write_congestion_map
update_congestion_map

read_def

```
read_def
[-design design]
[-create_ports] [-add_power_switch]
[-no_nets] [-no_specialnets]
[-no_physical_pins] [-no_vias]
[-keep_filler_cells] [-keep_welltap_cells]
[-keep_all_instances] [-keep_all_physical_cells] [-keep_routed_nets]
[-hierarchical] [-no_force_fixed]
[-convert_sites {old_site_name new_site_name}
[-incremental] [-ignore_errors] [-fuzzy_match] [-merge_place_blockages]
def_file
```

Description

Loads the specified DEF file.

Genus will perform a consistency check between the DEF and the Verilog netlist and issue relevant messages if necessary. The DEF file must define the die size. A warning message will be issued for any components that lie outside the die area. For better synthesis results, you should also have the pin and macro locations specified in the DEF, although it is not required.

Genus supports DEF 5.3 and above.

> The information extracted from the DEF file depends on the license you use to start the tool. In most cases, the `read_def` command will only extract the floorplan information (fixed macros, blockages, pins, regions, and so on). Detailed placement information (in particular, placed components) can only be extracted if you start the tool with a physical license.

Options and Arguments

`-add_power_switch`	Inserts power switch cells in the netlist if they are present in the DEF file.
	These components will be added to the existing netlist and will be connected.
`-convert_sites {`*old_site_name new_site_name*`}`	
	Converts the row site name specified in the DEF file to the specified new row site name.

`-create_ports`	Specifies to create an unconnected top-level netlist port for each pin in the DEF file that does not exist in the netlist and that is either of type clock, reset, scan, or signal.
`def_file`	Specifies the DEF file to read.
`-design` *`design`*	Specifies the design to which to annotate the DEF information.
`-fuzzy_match`	Enables fuzzy name matching. Genus allows matching the following: '/', '[ ]', '_'. The Examples section of this command shows some of the fuzzy matching examples.
`-hierarchical`	Specifies that the DEF file is hierarchical.
`-ignore_errors`	Ignores any inconsistency errors between the LEF and DEF.
`-incremental`	Specifies that the DEF file has incremental information. Therefore only updates of the blockages, components, groups, pins, and regions are needed.
	• If a pin or component did not have physical data in the original DEF file, physical data in the incremental DEF are used.
	• If a pin or component has physical data from the original DEF file, the physical data in the incremental DEF file will overwrite the original data.
`-keep_all_instances`	
	Stores all cells instantiated in the `DEF COMPONENTS` section but not instantiated in the netlist as physical-only cells (pcells).
`-keep_all_physical_cells`	
	Specifies to keep cells with `lib_cell` attribute `keep_as_physical` set to `true`.
`-keep_filler_cells`	
	Specifies to keep the physical-only filler cells. Filler or spacer cells are standard cells to fill in space between regular core cells.
`-keep_routed_nets`	Keeps routed nets in addition to fixed and cover nets.
`-keep_welltap_cells`	

	Specifies to keep the physical-only welltap cells. Welltap cells are standard cells that connect N and P diffusion wells to the correct power and ground wire.
-merge_place_blockages	
	Merges placement blockages.
`-no_force_fixed`	Prevents that components have their `placement_status` set to fixed in hierarchical mode (that is, when reading in a hierarchical DEF file).
`-no_nets`	Specifies to skip the `NETS` section in the DEF file.
`-no_physical_pins`	Specifies not to create a physical pin for a missing pin in the DEF file.
`-no_special_nets`	Specifies to skip the `SPECIALNETS` section in the DEF file.
`-no_vias`	Specifies to skip the `VIAS` section in the DEF file.

Examples

- The following examples shows fuzzy matching that matches '/', '[ ]', '_'.

```
netlist U1/g7 - DEF U1_g7
netlist U1_g7 - DEF U1/g7
netlist U1_g365 - DEF U1\/g365
netlist U1_ptr1_reg[6] - DEF U1/ptr1_reg\[6\]
```

- The following example loads the `point.def` DEF file:

```
genus@root:> read_def point.def
Reading and processing DEF file 'point.def'...
Parsing DEF file...

Warning : Metal fill present. [PHYS-178]
        : Metal fill on layer 'METAL1'.
Warning : Metal fill present. [PHYS-178]
        : Metal fill on layer 'METAL2'....
Warning : Component not present in the netlist. [PHYS-171]
        : The component 'U1/foo1' does not exist.
Warning : Component not present in the netlist. [PHYS-171]
        : The component 'U1/foo4' does not exist....
Info    : COVER component present. [PHYS-182]
```

```
        : The instance 'ram_bank1' is COVER.
Info    : COVER component present. [PHYS-182]
        : The instance 'U1/g20' is COVER.
Info    : COVER component present. [PHYS-182]
        : The instance 'U1/g21' is COVER.
...
Warning : Library cell not defined in the physical library. [PHYS-214]
        : Library cell 'FILL1MTR' specified for instance 'FILLER1' is
not defined in a LEF file.
Warning : Physical cell not created due to missing macro. [PHYS-211]
        : Macro 'FILL1MTR' for physical cell 'FILLER1' not found.
Warning : Specified library cell not defined in the physical library.
[PHYS-214]
        : Library cell 'FILL1MTR' specified for instance 'FILLER2' is
not defined in a LEF file.
Warning : Physical cell not created due to missing macro. [PHYS-211]
        : Macro 'FILL1MTR' for physical cell 'FILLER2' not found.
Done parsing DEF file.
Processing DEF file...
Warning : Overlapping guide detected. [PHYS-187]
        : Guide 'region_6' (DEF name: 'region_6') overlaps fence
'region_2' (DEF name: 'REGION_TWO').
Warning : Overlapping guide detected. [PHYS-187]
        : Guide 'region_8' (DEF name: 'region_8') overlaps region
'region_1' (DEF name: 'REGION_ONE').
Installing blockage router...
  Summary report for DEF file 'point.def'

    Components
    ----------
        Cover:   3
        Fixed:  10
     Physical:   2
         Bump:   0
       Placed: 591
     Unplaced:   2
        TOTAL: 608 (3 are class macro)

  There are 2 components that do not exist in the netlist.
```

```
          Pins
          ----
        Cover: 0
        Fixed: 0
     Physical: 0
       Placed: 0
     Unplaced: 0
        TOTAL: 0

          Nets
          ----
         Read: 0 (cover: 0, fixed: 0)
      Skipped: 0
        TOTAL: 0

   SpecialNets
   -----------
         Read: 0
      Skipped: 0
        TOTAL: 0

       Fences: 2
       Guides: 5
      Regions: 1
Done processing DEF file.

  ========================
  Physical Message Summary
  ========================

   2 / 2  W  PHYS-171  Component not present in netlist.
   2 / 2  W  PHYS-178  Metal fill present.
  13 / 0  I  PHYS-181  Full preserve set on instance.
   3 / 3  I  PHYS-182  Cover component present.
   2 / 2  W  PHYS-187  Overlapping guide detected.
   2 / 2  W  PHYS-211  Physical cell not created due to missing macro.
   2 / 2  W  PHYS-214  Library cell not defined in physical library.
```

```
Done reading and processing DEF file (time: 0s).
```

Related Information

Affected by this attribute:

script_search_path

Related attribute:

phys_ignore_special_nets

read_encounter

`read_encounter config` *`configuration_file`*

Description

Reads an Encounter configuration file into Genus. An Encounter configuration file is an ASCII file that contains Tcl variables that describe information such as the netlist or RTL, technology libraries, LEF information, constraints, and capacitance tables. Encounter configuration files have the `.config` extension.

After the file is loaded, the constraints and attributes specified in the configuration file will automatically be set. Hence, the design will be ready for synthesis or optimization or both.

Options and Arguments

`configuration_file`	Specifies the configuration file to load.

Related Information

Related command:

write_design

read_parasitics

```
read_parasitics spef_file
[-max_fanout integer] [-rc_corner rc_corner]
[-hierarchical] [-incremental] [-initialize]
```

Description

Reads the SPEF file and loads the resistance and grounded capacitors from the file. Gzip compressed files (`.gz` extension) can also be loaded. In Genus, the SPEF file is generated by Encounter.

Options and Arguments

`-hierarchical`	Specifies that SPEF file is hierarchical.
`-incremental`	Specifies that the SPEF file contains incremental information. Allows to read in the data without resetting the original SPEF annotated values (if any).
`-initialize`	Specifies to initialize resistance or capacitance values to zero.
`-max_fanout` *integer*	Any net with a fanout count greater than the specified value will not be annotated with the resistance and capacitance from the SPEF. Also, delay calculation will not be performed. The default value is 1000.
`-rc_corner` *rc_corner*	
	Specifies the RC corner object that the SPEF values are .associated with.
	The RC corner must have been defined with the `create_rc_corner` command.
spef_file	Specifies the SPEF file.

Related Information

Related command:

write_parasitics

read_physical

```
read_physical [-lefs string | -add_lefs string | -oa_ref_libs string]
```

Description

Reads the physical design information from the specified LEF files or open access library files. When this command is run, the physical library information is loaded into the database. Both LEF and OA are supported, but not simultaneously.

The `read_physical` command corresponds to the Physical step of the Initialization flow. During this step, the physical libraries (OA or LEF) are loaded into the tool's database using `read_physical` command. When the command is issued, the database is loaded with the specified objects, and is available for querying of attributes associated with the physical data, such as size, layer details, and so on.

The tool uses the standard OA parser, and performs full error checking on the physical data. When the command is complete, a full timing library set (via `read_mmmc`) and a physical library set would have been loaded into the database.

If `-lefs` option is used, the technology LEF must be first specified.

This command is optional for Synthesis and STA, but is required for Implementation.

The tool can support a physical-only mode when the attribute `init_physical_only` is set to `true`. This attribute must be set prior to the `read_physical` command, otherwise an error will be issued.

> This command should be issued after `read_libs` or `read_mmmc` otherwise an error will be issued.

Options and Arguments

`-add_lefs` *string*	Specifies a list of LEF files to read and to add to the original list of LEF files that were added before `init_design`, and cannot be combined with any other option.
	You can add new macros, non-default rules, sites etc. using this option, but you cannot use it to modify any existing LEF data that is already loaded. The LEF files added using this option will be used when the saved design is loaded again using `read_db`.
`-lefs` *string*	Specifies the name of the LEF files to read. Specifies the list of LEF files to be loaded. The first LEF file must have the technology section present. The file names are saved to the `init_lef_files` root attribute. This option cannot be combined with `-oa_ref_libs` option.
`-oa_ref_libs` *string*	
	Specifies the name of the open access library files to read.
	Specifies th list of OpenAccess library names to be loaded. These names must also be listed in the cds.lib file. All the abstract views in the specified libraries are loaded.
	Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded, even if some of the cells are not currently used in the netlist. The first OpenAccess library that is specified is also used to load the OpenAccess technology data.

Example

The following example uses `read_physical` command to read all `lef` files:

```
read_mmmcread_physical -lefs $your_lef_list.lstread_hdl -sv -f $your_file_list.lstread_power_intent
#optionallyinit_design...
```

Related Information

Related commands:

read_libs
read_mmmc

read_netlist

Related attributes:

init_lef_files
init_lib_phys_consistency_checks
init_min_dbu_per_micron

read_sdp_file

```
read_sdp_file -file file
[-design design] [-preserve_size_ok]
[-hier_path path] [-origin llx lly]
```

Description

Reads a relative placement file (in .sdp format) containing the SDP definitions.

Options and Arguments

`-design` *design*	Specifies the design in which to use the SDP information.
`-file` *file*	Specifies the name of the SDP input file.
`-hier_path` *path*	Specifies the hierarchical path name of the SDP elements specified in the SDP file. This path name is appended to all SDP instances defined in the file.
`-origin` *llx lly*	Specifies the coordinates of the origin of the SDP groups.
	You can use floats to specify the coordinates.
	All SDP groups use the same origin.
`-preserve_size_ok`	Sets the `preserve` attribute on the SDP instances to `preserve_size_ok`.
	Default: `preserve`

Example

The following command reads the `dtmf_chip.sdp` file. It also appends the specified hierarchical path to all elements.

```
read_sdp_file -file dtmf_chip.sdp -hierPath DTMF_CHIP/TDSP_CORE_INST
```

Related Information

SDP Flows in *Genus Physical Guide*

Related command:

write_sdp_file

set_cell_padding

```
set_cell_padding -cells {inst | lib_cell}...
[-bottom_side integer] [-left_side integer]
[-right_side integer] [-top_side integer]
[-padding integer]
```

Description

Specifies to set the padding (placement clearance) for instances of the specified library cells.

> The padding is retained during placement optimization and clock-tree synthesis.

Options and Arguments

-bottom_side integer	
	Specifies the bottom side padding.
	min=0, max=500
-cells {*inst* | *lib_cell*}	
	Specifies the leaf cells to which the padding factor needs to be added.
-left_side integer	
	Specifies the left side padding.
	min=0, max=1000
-padding integer	Specifies the padding factor to be applied.
-right_side integer	
	Specifies the right side padding.
	min=0, max=1000
-top_side integer	
	Specifies the top side padding.
	min=0, max=500

specify_cell_pad

```
specify_cell_pad
-padding string
lib_cell_list
```

Specifies the padding factor to be used to calculate the padding (placement clearance) for instances of the specified library cell types.

Description

During placement, the tool adds the padding to the right side of the instances of the specified cell types. The padding is determined by multiplying the placement SITE value of the libcells with the specified padding factor.

> The padding is retained during placement optimization and clock-tree synthesis.

Options and Arguments

lib_cell_list	Specifies a list of `lib_cell` types to which the padding factor applies.
`-padding` *string*	Specifies the padding factor to be applied.

Related Information

Affects these commands:

syn_generic -physical
syn_opt

update_congestion_map

`update_congestion_map [-pin_density] [-design` *design*`]`

Description

Updates the congestion map for the specified design. You can run this command after any command that updates the physical data.

> This command applies only to the Genus-Physical flow.

Options and Arguments

`-design` *design*	Specifies the design for which the congestion map must be updated.
`-pin_density`	Specifies to compute the pin density.

update_group

```
update_group -name group
[-add][-delete][-objs string][-overwrite]
```

Description

Adds or removes the specified objects from the specified group.

> This command applies only to the Genus-Physical flow.

Options and Arguments

-add | -delete	Adds or removes the specified objects.
-name *group*	Specifies the name of the group for which to add or remove instances.
-objs *string*	Specifies the instances, hierarchical instances or group objects to be added or removed to the specified group.
-overwrite	Overwrites any previously existing group membership.

Related Information

Related command:

create_group

update_inst

```
update_inst -name string
[-location float] [-base_cell cell] [-dbu]
[-place_status string] [-orient {enum}]
```

Description

Updates the libcell, location, and placement status of the specified instance.

Options and Arguments

`-location` *float*	Specifies the location of the placed instance. For example, point - {x y}
`-base_cell` *cell*	Specifies the `base_cell` to use for the instance.
`-dbu`	Use DBUs for coordinates.
`-name` *string*	Specifies the name of the instance to be updated.
`-place_status` *enum*	
	Specifies the placement status.
	`{fixed|soft_fixed|placed|unplaced|cover}`
`-orient` *enum*	Specifies the orientation.
	{r0|r90|r180|r270|mx|mx90|my|my90}

Examples

- Following command updates inst to new base call, location value and also `place_status` value:

```
update_inst -name inst:gbox/u4/inst -base_cell base_cell:buf1 -location \
{3.2 2.1} -place_status cover

inst:gbox/u4/inst
```

- Following command updates inst to new base call and location value:

```
update_inst -name inst:gbox/u3/inst -base_cell base_cell:nor2 \
-location {4.8 6.2}

inst:gbox/u3/inst
```

Related Information

Related command:

create_design

update_instance_data

```
update_instance_data
[-dbu] [-location_x float] [-location_y float]
[-placement_status string] [-orientation string]
{inst|hinst}
```

Description

Updates the location and placement status of the specified instance.

> This command applies only to the Genus-Physical flow.

Options and Arguments

`-dbu`	Specifies to use DBUs for the coordinates.
{*inst*|*hinst*}	Specifies the name of the instance for which the data needs to be updated.
`-location_x` *float*	Specifies the lower left corner x-coordinate.
`-location_y` *float*	Specifies the lower left corner y-coordinate.
`-orientation`	Specifies the orientation.
`-placement_status` *string*	
	Specifies the placement status.
	`{fixed|soft_fixed|placed|unplaced|cover}`

Related Information

Related command:

create_group

update_power_domain

```
update_power_domain
[-design design]
[-add_block_box float+]
[-power_extend_sides float+ | -power_extend_edges float+]
[-disjoint_hinst_box_list string]
[-gap_sides float+ | -gap_edges float+ ]
[-core_to_bottom float]
[-core_to_left float]
[-core_to_right float]
[-core_to_top float]
[-first_row_site_index integer]
[-last_row_site_index integer]
[-default_tech_site string]
[-row_flip {noflip|second|first|auto}]
[-row_pattern_site string]
[-row_space_type {0|1|2}]
[-row_spacing float]
power_domain
```

Description

Creates or modifies the physical boundary for the specified power domain.

Options and Arguments

`[-design design]`	Specifies the name of the design to which the power domain belongs. This option is only required if multiple designs are loaded.
`[-add_block_box float+]`	Adds a hard block to the disjoint power domain.
`[-power_extend_sides float+]`	Specifies the boundary for legal targets to be used by the power planning and routing commands, in conjunction with the power domain boundary. Specifies a list of four floating values, one for each side.

[-power_extend_edges *float*+]	Specifies the maximum search distance for power extension connections by domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).
[-disjoint_hinst_box_list *string*]	Associates a list of hierarchical instances with a region in which they can be placed. You can specify multiple combinations.
-gap_sides *float*+	Defines the distance, in microns, that must be reserved from the power domain boundary edges for power routing. Specify a list of four floating values, one for each edge.
-gap_edges *float*+	Specifies the minimum space value to other domain by power domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).
[-core_to_bottom *float*]	Specifies the row margin from bottom boundary.
[-core_to_left *float*]	Specifies the row margin from left boundary.
[-core_to_right *float*]	Specifies the row margin from right boundary.
[-core_to_top *float*]	Specifies the row margin from top boundary.
[-first_row_site_index *integer*]	Specifies the first row site index value.
[-last_row_site_index *integer*]	Specifies the last row site index value.
[-default_tech_site *string*]	Specifies the default tech site for power domain.
[-row_flip {noflip|second|first|auto}]	Flips the orientation of either the first or second row, then follows the flipped and abutted row pattern.
[-row_pattern_site *string*]	Specifies the site name for the row pattern.
[-row_flip {noflip|second|first|auto}]	Flips the orientation of either the first or second row, then follows the flipped and abutted row pattern.
[-row_pattern_site *string*]	Specifies the site name for the row pattern.
[-row_space_type {0|1|2}]	Determines whether the row spacing value applies between no row (0), each row (1), or each pair of rows (2).

`[-row_spacing float]`	Specifies the row spacing between each row (1) or each pair of rows (2) as specified in -row_space_type.
power_domain	Specifies the power domain name.

write_congestion_map

`write_congestion_map [-design design] file`

Description

Saves the congestion map to the specified file.

> This command applies only to the Genus-Physical flow.

Options and Arguments

`-design design`	Specifies the design for which the congestion map must be saved.
file	Specifies the name of the file to which the congestion data must be saved.

Related Information

Related commands:

read_congestion_map
update_congestion_map

write_def

```
write_def [-ignore_groups] [-ignore_placed_instances]
[-hierarchical instance] [-scan_chains]
[-exclude_ilm] [-no_die_area]
[-no_special_net] [-no_def_pin] [-no_ilm_pg_net]
[-version string] [design] [> file]
```

Description

Writes floorplan information to a DEF file. The DEF, by default will not contain netlist information (net connectivity information) aside from the power/ground nets defined in the input DEF (`SPECIALNETS` section).

You can write out the DEF in gzip format by specifying the `.gz` extension when writing out the file.

You can use the `write_def` command after importing a design and reading in a floorplan DEF or creating a floorplan with `create_floorplan` command.

To create a DEF in gzip format specify .gz extension when writing out the file.

> You can select the DEF version for your output file by setting the `def_output_version` attribute to the version number. For example, to set the version number to 5.6, type the following command:

```
set_db def_output_version 5.6
```

Options and Arguments

design	Specifies a particular design for which to write out the floorplan. Only one design can be specified at a time.
-exclude_ilm	Specifies to skip placement information of ILM modules.
file	Redirects the floorplan to the specified file.
-hierarchical *instance*	
	Specifies the hierarchical instance for which to write out the DEF.
-ignore_groups	Discards the instance groups that are defined in Genus. These come from the input DEF (GROUPS section). The output DEF will have no GROUPS or REGIONS sections.
-ignore_placed_instances	
	Only writes preplaced instances (+ FIXED tag in the DEF) to the DEF. These are objects such as macros. Without this option, the output DEF will include all the instances that are preplaced or placed. Unplaced components will never be written to the DEF. This is all in the COMPONENTS section.
-no_def_pin	Specifies to exclude def_pin objects from the written out DEF file.
-no_die_area	Specifies to exclude DIEAREA, PROPERTYDEFINITIONS and UNITS statements.
-no_special_net	Specifies to exclude special net information from the written out DEF file.
-no_ilm_pg_net	Specifies to exclude power and ground special net for ILMs.
-scan_chains	Specifies that scan chain information should be included in the floorplan.
-version *string*	Specifies the DEF version to write out.
	Default: 5.7

Related Information

Related commands:	create_floorplan
	read_def
Related attribute:	phys_ignore_special_nets
	def_output_version
	skip_in_write_def

write_lib_lef

```
write_lib_lef
[-lib file] [-lef file][-verbose][-keep_temp_dir]
[-process_node integer] [-include_power_ground][-include_power][-use_internal_supply][-views
analysis_view..][design]
```

Description

Writes out the timing model for the design in Liberty format and/or the physical model in LEF library format.

Options and Arguments

design	Specifies the design for which to write out the library data.
-lef *file*	Specifies the name of the file to which to write out the LEF data stored in the database.
-lib *file*	Specifies the name of the file to which to write out the Liberty data stored in the database.
-process_node *integer*	
	Specifies the process design node for which to write out the information.
-views *analysis_view..*	Specifies the analysis views to be included in the library model generation.
-verbose	Echoes Innovus log into Genus log.
-keep_temp_dir	Specifies that the temporary directory is not deleted after the command completes.
-include_power_ground	Allows power or ground related pin information to be extracted inside the extracted model.
-include_power	Includes information of power consumed by the extracted model.
-use_internal_supply	Specifies to use internal supply for ETM (Extracted Timing Models).

Example

Simple usage example:

```
write_lib_lef -lib top -lef top
```

Output files:

```
Timing model: top.lib
```

```
Physical model: top.lef
```

write_parasitics

```
write_parasitics
[-cap_unit {pf | fF}]
[-res_unit {ohm | kohm}]
[-view analysis_view]
[> file]
```

Description

Writes out the parasitics (resistance and capacitance information) of the design in SPEF (Standard Parasitic Exchange Format) format.

Options and Arguments

`-cap_unit {pf | fF}`	
	Specifies the unit to be used to write out the capacitance values.
	Default: `pf`
file	Specifies the file to which to write the parasitics.
`-res_unit {ohm | kohm}`	
	Specifies the unit to be used to write out the resistance values.
	Default: `ohm`
`-view` *analysis_view*	
	Specifies the analysis view for which to write out the SPEF file.

Related Information

Related command:

read_parasitics

write_preserves

write_preserves
[-out_file *string*]
[-obj_types *string*]
[-design *string*]

Description

Generates preserve information for a netlist as a script of set_db commands.

Options and Arguments

`-out_file`	Specifies the name io the output file. Default: `stdout`
-obj_types	Optional list of object-types for the entries to be written. Default: Values one or more of `design, hinst, module, inst, net, hnet, pin, hpin, base_cell`
`-design`	Specifies the name or vdir of the design to report.

Related Information

Related command:

write_design
write_db

write_reports

`write_reports -directory` *`path`* `-tag` *`string`*
`[-innovus] [-levels_of_logic]`

Description

Generates the QoS statistics table at any stage in the flow, and writes the timing, area, gates, QoR reports to the specified output directory.

> To write a separate summary table of the QoS statistics in the log file, use the report_summary command after running write_reports.

> To disable power reporting set the `qos_report_power` root attribute to `false`.

Options and Arguments

`-innovus`	Specifies to use the QoS statistics from the latest Innovus run.
	By default, the statistics are used from Genus.
`-directory` *path*	Specifies the path to the directory where the output data should be stored.
`-levels_of_logic`	Prints the number of gates on the critical path per cost group in the QoR report.
`-tag` *string*	Specifies the prefix for reports generated at this stage.

Example

```
write_reports -dir rpt -tag syngen


QoS Summary for counter
=======================================================================
Metric       syngen
=======================================================================Slack (ps):
-702  R2R (ps):                    615  I2R (ps):                   -578  R2O (ps):
-702  I2O (ps):               no_value  CG  (ps):               no_value  TNS (ps):
7,942  R2R (ps):                      0  I2R (ps):                   4,598  R2O (ps):
3,345  I2O (ps):               no_value  CG  (ps):               no_valueFailing Paths:
19Cell Area:                   2,616Total Cell Area:             2,616Leaf Instances:
58 Total Instances:                58Utilization (%):              44.62Tot. Net Length (um):
1,689Avg. Net Length (um):           24Route Overflow H (%):         0.00Route Overflow V (%):
0.00MBCI(%) (bits/gate) :         0.00Norm Cong Hotspot Area:  Max Cong:                    0.00  Tot
Cong:
0.00=======================================================================CPU  Runtime (h:m:s):
```

```
00:00:37Real Runtime (h:m:s):     00:02:04CPU  Elapsed (h:m:s):     00:00:43Real Elapsed (h:m:s):
00:02:06Memory (MB):                643.51

============================================================================

============================================================================

Flow Settings:

============================================================================Total Runtime (h:m:s):
00:10:02Total Memory (MB):           643.51Executable Version:    18.12-
s018_3============================================================================

Total Cell Area =  Cell Area + Physical Cell AreaTotal Instances =  Leaf Instances + Physical
Instances

The ouput directory 'rpt' will have these
files:rpt/syngen_area.rptrpt/syngen_gates.rptrpt/syngen_qor.rptrpt/syngen_time.rpt
```

Related Information

Affects this command:	report_summary
Affected by this attribute:	qos_report_power

write_sdp_file

```
write_sdp_file
[-file file]
[-top_group sdp_group]
```

Description

Saves the current SDP relative placement information in the specified file. Genus stores the original design data along with additional information in the SDP file in the design information hierarchy in the form of attributes.

Options and Arguments

`-file` *file*	Specifies the name of the SDP output file.
	Default: Output is written to the screen.
`-top_group` *sdp_group*	
	Specifies the SDP top group whose information to write out.

Example

The following command writes the current SDP information in SDP format in file `test.sdp`.

```
write_sdp_file  -file test
```

Related Information

SDP Flows in *Genus Physical Guide*

Related command:

read_sdp_file

`Related attributes:`

- sdp_row
- sdp_column
- sdp_instances
- sdp_group

22

SDC Commands

Currently this appendix describes the new commands added in SDC 1.7 and some other frequently used SDC commands.

SDC commands can be specified

- in an SDC file and read in with the `read_sdc` command
- at the Genus prompt

- add_to_collection
- all_clocks
- all_connected
- all_fanin
- all_fanout
- all_inputs
- all_instances
- all_outputs
- all_registers
- append_to_collection
- apply_clock_gating_network_latency_table
- compare_collections
- copy_collection
- create_clock
- create_generated_clock
- current_design
- current_design_name
- current_instance
- filter_collection
- foreach_in_collection
- get_arcs
- get_cells
- get_clock_gating_history_dictionary_entry
- get_clocks
- get_designs
- get_equivalent_cells
- get_generated_clocks
- get_leaf_cells
- get_leaf_nets
- get_leaf_pins
- get_lib_arcs
- get_lib_cells
- get_lib_pins
- get_libs
- get_nets
- get_object_name
- get_path_groups
- get_pins
- get_ports
- get_references
- getenv
- group_path
- index_collection
- list_property
- load_of
- read_clock_gating_network_latency_table
- remove_case_analysis
- remove_clock_gating_check
- remove_clock_latency
- remove_clock_transition
- remove_disable_clock_gating_check
- remove_disable_timing
- remove_from_collection
- remove_generated_clock
- remove_ideal_net
- remove_ideal_network
- remove_input_delay
- remove_output_delay
- report_property
- reset_annotated_check
- reset_annotated_delay
- reset_annotated_transition
- reset_case_analysis
- reset_clock
- reset_clock_gating_check
- reset_clock_gating_network_latency_table
- reset_clock_groups
- reset_clock_latency
- reset_clock_sense
- reset_clock_transition
- reset_clock_uncertainty
- reset_data_check
- reset_disable_clock_gating_check
- reset_disable_timing
- reset_drive
- reset_driving_cell
- reset_generated_clock
- reset_ideal_latency

- reset_ideal_network
- reset_ideal_transition
- reset_input_delay
- reset_load
- reset_max_capacitance
- reset_max_fanout
- reset_max_time_borrow
- reset_max_transition
- reset_min_capacitance
- reset_min_fanout
- reset_min_pulse_width
- reset_min_transition
- reset_mode
- reset_output_delay
- reset_path
- reset_path_group
- reset_path_group_options
- reset_resistance
- reset_sense
- reset_timing_derate
- reset_wire_load_mode
- reset_wire_load_model
- reset_wire_load_selection_group
- set_active_clocks
- set_annotated_check
- set_annotated_delay
- set_annotated_transition
- set_case_analysis
- set_clock_exclusivity
- set_clock_gating_check
- set_clock_gating_network_latency_table
- set_clock_groups
- set_clock_latency
- set_clock_sense
- set_clock_skew
- set_clock_transition
- set_clock_uncertainty
- set_constraint_mode
- set_data_check
- set_disable_clock_gating_check
- set_disable_timing
- set_dont_touch
- set_dont_touch_network
- set_dont_use
- set_drive
- set_driving_cell
- set_equal
- set_false_path
- set_fanout_load
- set_hierarchy_separator
- set_ideal_latency
- set_ideal_net
- set_ideal_network
- set_ideal_transition
- set_input_delay
- set_input_transition
- set_lib_pins
- set_load
- set_load_unit
- set_logic_dc
- set_logic_one
- set_logic_zero
- set_max_capacitance
- set_max_delay
- set_max_dynamic_power
- set_max_fanout
- set_max_leakage_power
- set_max_time_borrow
- set_max_transition
- set_min_capacitance
- set_min_delay
- set_min_fanout
- set_min_pulse_width
- set_min_transition
- set_mode
- set_module_model
- set_multicycle_path
- set_operating_conditions
- set_opposite
- set_output_delay
- set_path_group_options
- set_port_fanout_number
- set_resistance
- set_sense
- set_size_only
- set_time_unit
- set_timing_derate
- set_unconnected
- set_units
- set_wire_load_mode
- set_wire_load_model
- set_wire_load_selection_group
- sizeof_collection
- sort_collection

add_to_collection

```
add_to_collection
base_collection second_collection [-unique]
```

Description

Creates a new collection by concatenating objects from the base collection and the second collection.

- If all the objects in the base collection have the *same* type, only those objects in the second collection with the same type as those in the base collection are added in the result collection. If you specify an object list in second collection, the command converts the objects list to a single list and will add the objects with the same type as those in the base collection to result collection.
- If the objects in the base collection have *different* types, all objects in the second collection are added to the result collection.
- If the second collection is empty, the new collection will be a copy of the base collection.
- If the base collection is empty, the new collection will be a copy from the second collection.

If the software finds no objects to add, an empty collection is returned.

Options and Arguments

base_collection	Specifies the base collection (list) of objects.
	Objects from the second collection or object list are added to this collection to create the new collection.
	The base collection cannot contain an object list.
second_collection	Specifies the second collection (list) of objects.
-unique	Removes the duplicate objects from the new collection. If this option is omitted, duplicate objects should not be removed.

Related Information

Related commands:

append_to_collection
compare_collections
copy_collection
remove_from_collection

all_clocks

```
all_clocks
```

Description

Returns all clocks in the design when you do not perform multi-mode timing analysis. This command creates a collection of all the clocks based on the defined or created clocks in the design. If you do not define or create any clocks, an empty string is returned.

When performing multi-mode timing analysis, the command returns all clocks that apply to the current mode.

- When you use this constraint in an SDC file, the constraint will be applied to the mode specified with the `-mode` option of the read_sdc command that reads in the file.
- When you want to use this constraint at the Genus prompt, make sure that the mode to which this constraint needs to be applied was previously specified using set_interactive_constraint_modes command.
- If no clocks were defined, an empty string "" is returned.
- To filter clocks, use the get_clocks command to create a collection of the clocks matching a specific pattern. Assign these clocks to a variable or pass them using another command.

Related Information

Related commands:

all_inputs
create_clock
get_clocks
report_clocks

all_connected

```
all_connected [-leaf_pin] { net | pin | pg_pin | port}...
```

Description

If the specified object is a net, the command returns the list of all the pins and ports connected to the net. If the object is a pin or a port, the command returns the net connected to the pin or the port.

This command accepts a TCL list as an argument but not wildcards.

Options and Arguments

`-leaf_pin`	Returns the leaf cell pins and the ports connected to the specified net.
net	Specifies a net. The ensuing list will return a list of all the pins connected to this net.
pin	Specifies a pin. The ensuing list will return the net connected to the pin.
pg_pin	Specifies a power or ground pin. The ensuing list will return the net connected to the pin.
port	Specifies a ports. The ensuing list will return the net connected to the port.

Examples

- The following command returns a collection of all of the objects connected to the pin `b1_Sin1`:

```
genus:root:> all_connected Sin1
Sin1
```

- The following command returns a collection that contains only the flattened connection points for the net `net_1`:

```
genus:root:> all_connected -leaf [get_nets net_1]
Sin1
```

- The following command returns a collection of all of the objects connected to the pin `b1_Sin1`:

```
genus:root:> all_connected Sin1
TL2/L1/RX/D TL1/RX/Q
```

- The following command returns a list of pins/ports connected to the nets `clk3` and `n_8`:

```
genus:root:> all_connected -leaf {net:test/clk_3 net:test/n_8}
port:test/clk_3
pin:test/r1_reg/clk
pin:test/grouped_modulei/g8/in_0
pin:test/inst1/SN
pin:test/grouped_modulei/g9/z
```

all_fanin

```
all_fanin -to {pin|port|net|pin_bus|port_bus}...
[-flat] [-levels integer]
[-only_cells] [-startpoints_only]
[-trace_through {case_disable |user_disable | all}]
[-view view_name]
```

Description

Returns a list of pins, ports, or cells that exist in the fanin cone of the specified objects.

> The default behavior of the `all_fanin` command is different from that of Innovus. In Innovus, the default behavior is flat, but in Genus its hierarchical. By setting the attribute "`timing_collection_all_fanin_fanout_traversal_mode`" to "`hierarchical`" in Innovus, its behavior is matched with genus default.

Options and Arguments

`-flat`	Returns all objects in the fanin cone of the specified objects, even when they are present in a hierarchy above the specified objects.
`-levels` *integer*	Specifies the number of logic levels to go back to report the fanin cone information.
`-only_cells`	Returns only the instance names of all objects in the fanin cone.
`-startpoints_only`	Returns only the timing start points in the fanin cone.
`-trace_through {case_disable |user_disable | all}`	
	You can specify one of the following options:
	`case_disable`—Allows traversal of arcs disabled using `set_case_analysis` command. Arcs disabled using `set_disable_timing` are not traversed.
	`user_disable`—Allows traversal of arcs disabled using `set_disable_timing` command, but arcs disabled using `set_case_analysis` command are not traversed.
	`all`—Allows traversal through all the arcs.
`-to` {*pin* | *port* | *net* | *pin_bus* | *port_bus*}	
	Specifies a list of pins, ports or nets for which you want the fanin cone information.
`-view` *view_name*	Specifies the view name for which the fanin cone traversal must be performed.

Examples

- The following module is used for the examples:

```
module top_inv (in, out);
 input in;
 output out;
 wire in;
 wire out;
 inv1 inst_4(.A (in), .Y(out));
endmodule

module top(in, out, clk);
 input in, clk;
 output out;
 wire in, clk;
 wire out;
 wire w1, w2;
 fflopd inst1(.CK (clk), .D (in), .Q (w1));
 top_inv inst2(.in (w1), .out (w2));
 fflopd inst3(.CK (clk), .D (w2), .Q (out));
endmodule
```

- The following command returns the fanin of pin `A` of instance inst2/inst_4.

```
genus:root:> all_fanin -to inst2/inst_4/A
```

- The following command returns the timing start points in the fanin cone of pin `A` of instance inst2/inst_4.

```
genus:root:> all_fanin -to inst2/inst_4/A -startpoints_only
```

- The following command returns the instances in the fanin cone of pin `A` of instance inst2/inst_4.

```
genus:root:> all_fanin -to inst2/inst_4/A -only_cells
```

- The following command returns the fanin cone of pin `D` of instance `inst3`.

```
genus:root:> all_fanin -to inst3/D
```

- The following command reports only one level in the fanin cone of pin `D` of instance `inst3`.

```
genus:root:>all_fanin -to inst3/D -levels 1
```

- The following command returns the output by matching only the given hierarchy.

```
genus:root:> all_fanin -to pin:top/inst2/inst_4/A

Output

inst2/inst_4/A
```

- The output from the following command will also include the output from different hierarchies.

```
genus:root:> all_fanin -to pin:top/inst2/inst_4/A -flat

Output

inst2/inst_4/A inst1/Q inst1/CK
```

all_fanout

```
all_fanout
{-from {pin|port|net}... | -clock_tree}
[-flat [-levels integer]] [-pin_levels integer]
[-only_cells] [-endpoints_only] [-stop_at_seqs]
[-trace_through {case_disable |user_disable | all}]
[-view view_name]
```

Description

Returns a list of pins, ports, or cells that exist in the fanout cone of the specified objects.

> The default behavior of the `all_fanout` command is different from that of Innovus. In Innovus, the default behavior is flat, but in Genus its hierarchical. By setting the attribute "`timing_collection_all_fanin_fanout_traversal_mode`" to "`hierarchical`" in Innovus, its behavior is matched with genus default.

Options and Arguments

`-clock_tree`	Returns the objects in the fanout of the clock network.
`-endpoints_only`	Returns only the timing end points in the fanout cone.
`-flat`	Returns all objects in the fanout cone of the specified objects, even when they are present in a hierarchy above the specified objects.
`-from` {*pin* | *port* | *net*}	
	Specifies a list of pins, ports or nets for which you want the fanout cone information.
`-levels` *integer*	Specifies the number of logic levels to go back to report the fanout cone information.
	This option can only be specified with the `-flat` option.
`-only_cells`	Returns only the instance names of all objects in the fanout cone.
`-pin_levels` *integer*	
	Specifies the number of logic levels to go back to report the fanout cone information. It returns all objects in the fanout cone of the sink list that are present within this many pin levels of the sink object.
`-stop_at_seqs`	Stop the traversal at a pin if the next arc is a clock_edge arc.
`-trace_through {case_disable |user_disable | all}`	
	You can specify one of the following options:
	`case_disable`—Allows traversal of arcs disabled using `set_case_analysis` command and physical constants. Arcs disabled using `set_disable_timing` are not traversed.
	`user_disable`—Allows traversal of arcs disabled using `set_disable_timing` command, but arcs disabled using `set_case_analysis` command are not traversed.
	`all`—Allows traversal through all the arcs.
`-view` *view_name*	Specifies the view name for which the fanout cone traversal must be performed.

Examples

The following module is used for the examples:

```
module top_inv (in, out);
    input in;
    output out;
    wire in;
    wire out;
    inv1 inst_4(.A (in), .Y(out));
endmodule

module top(in, out, clk);
    input in, clk;
    output out;
    wire in, clk;
    wire out;
    wire w1, w2;
    fflopd inst1(.CK (clk), .D (in), .Q (w1));
    top_inv inst2(.in (w1), .out (w2));
    fflopd inst3(.CK (clk), .D (w2), .Q (out));endmodule
```

- The following command returns the fanout of pin `CK` of instance `inst1`.

  ```
  genus:root:> all_fanout -from inst1/CK
  ```

- The following command returns only the timing end points in the fanout cone of pin `CK` of instance `inst1`.

  ```
  genus:root:> all_fanout -from inst1/CK -endpoints_only
  ```

- The following command returns only the instances in the fanout cone of pin `CK` of instance `inst1`.

  ```
  genus:root:> all_fanout -from inst1/CK -only_cells
  ```

- The following command returns the output by matching only the given hierarchy.

  ```
  genus:root:> all_fanout -from pin:top/inst2/inst_4/A

  Output

  inst2/inst_4/A inst2/inst_4/Y
  ```

- The output from the following command will also include the output from different hierarchies.

  ```
  genus:root:> all_fanout -from pin:top/inst2/inst_4/A -flat

  Output -

  inst2/inst_4/A inst2/inst_4/Y inst3/D
  ```

all_inputs

```
all_inputs
[ -clock clock_list][-edge_triggered | -level_sensitive]
[-clock_domains clock_domain_list
      [-edge_triggered | -level_sensitive | -no_clocks]]
```

Description

Returns all the input ports in the design. Using the command options you can filter the ports.

Options and Arguments

`-clock` *clock_list*	
	Returns all input ports that have an input delay relative to the specified clock(s).
`-clock_domains` *clock_domain_list*	
	Returns all input ports that have an input delay relative to the clocks in the specified clock_domain(s).
`-edge_triggered`	Returns all input ports that have a non level-sensitive input delay.
	If the `-clock` option is omitted, it returns all input ports that have an edge-triggered input delay with respect to any clock in the design.
`-level_sensitive`	Returns all input ports that have a level-sensitive input delay.
	If the `-clock` option is omitted, it returns all input ports that have a level-sensitive input delay with respect to any clock in the design.
`-no_clocks`	Returns all input ports that do not have a clock definition.

all_instances

```
all_instancess
{design | lib_cell | module}
```

Description

Returns a list of instances available in the design, or all instances of the specified `lib_cell` or `module`. The search for instances is made relative to the current instance within the current design.

Options and Arguments

design	Returns the instances of the specified design.
	You cannot use this command at the top level of the design. You can use it for hierarchical modules below the top design.
lib_cell	Returns the instances of the specified library cell.
module	Returns the instances of the specified module.

Examples

- The following examples returns the instances of the library cell "AND3X12M"

```
genus:root:> all_instances lib_cell:default_emulate_libset_max/ss_g_1v08_125c/AND3X12M

     0x763
inst:dtmf_recvr_core/TDSP_CORE_INST/TDSP_CORE_GLUE_INST/g5118
inst:dtmf_recvr_core/TDSP_CORE_INST/TDSP_CORE_GLUE_INST/g5123

genus:root:> all_instances AND3X12M

     0x764
inst:dtmf_recvr_core/TDSP_CORE_INST/TDSP_CORE_GLUE_INST/g5118
inst:dtmf_recvr_core/TDSP_CORE_INST/TDSP_CORE_GLUE_INST/g5123
```

- The following examples returns the instances of the module "accum_stat"

```
genus:root:> all_instances module:dtmf_recvr_core/accum_stat

     0x765
hinst:dtmf_recvr_core/TDSP_CORE_INST/ACCUM_STAT_INST

genus:root:> all_instances accum_stat

     0x766
hinst:dtmf_recvr_core/TDSP_CORE_INST/ACCUM_STAT_INST
```

all_outputs

```
all_outputs
[-clock clock_list | -clock_domains clock_domain_list]
[-edge_triggered | -level_sensitive]
```

Description

This command creates a collection of all output ports in the current design. To filter outputs, use the `get_ports` command to create a collection of ports matching certain criteria. Assign these ports to a variable or pass them using another command. You can limit the contents of the collection to ports with output delay relative to a specific clock.

Options and Arguments

`-clock` *clock_list*	
	Returns all output ports that have an output delay relative to the specified clocks.
`-clock_domains` *clock_domain_list*	
	Returns all output ports that have an output delay relative to the clocks in the specified clock_domain(s).
`-edge_triggered`	Returns all output ports that have a non level-sensitive output delay.
	If you omit the `-clock` option, the command returns all output ports that have a non level-sensitive output delay to any clock in the design.
`-level_sensitive`	Returns all output ports that have a level-sensitive output delay.
	If you omit the `-clock` option, the command returns all output ports that have a level-sensitive output delay to any clock in the design.

Example

The following command returns the names all of the output ports with an output delay relative to an `ideal_clk`:

```
all_outputs -clock ideal_clk
```

Related Information

Related commands:

- get_ports
- report_port
- set_output_delay

all_registers

```
all_registers
[-cells]
[-clock clock...]
[-fall_clock clock...] [-rise_clock clock...]
[-clock_domains clock_domain...]
[-data_pins] [-clock_pins] [-slave_clock_pins]
[-async_pins] [-output_pins]
[-flops | -edge_triggered]
[-latches | -level_sensitive]
[-macros] [-master_slave] [-inverted_output]
[-exclude {inst|hinst}...] [-no_hierarchy]
```

Description

Returns all the sequential elements in the design. Using the command options you can extract specific subsets of elements or specific object types.

> This command was added in SDC 1.7.

Options and Arguments

`-async_pins`	Returns the asynchronous pins of all sequential elements in the design.
`-cells`	Creates a collection of cells. This is the default option.
`-clock` *clock_list*	
	Returns only those sequential elements clocked by the specified clocks.
`-clock_domains` *clock_domain_list*	
	Returns only those sequential elements that belong to the specified clock domains.
`-clock_pins`	Returns the clock pins of all sequential elements in the design.
`-data_pins`	Returns the synchronous pins of all sequential elements in the design.
`-exclude {`*inst*`|`*hinst*`}`	
	Excludes the specified instances from the list of sequential elements to be returned.
`-fall_clock` *clock_list*	
	Returns only those sequential elements clocked by the falling edge of the specified clocks.
`-flops	-edge_triggered`	
	Returns all sequential elements of type flop in the design.
`-inverted_output`	Returns all sequential elements in the design that have an inverted output.
`-latches	-level_sensitive`	
	Returns all sequential elements of type latch in the design.
`-macros`	Returns only macros.
`-master_slave`	Returns all sequential elements in the design that are master-slave flops.
`-no_hierarchy`	Returns only those sequential elements at the top-level of the design.
`-output_pins`	Returns the output pins of all sequential elements in the design.
`-rise_clock` *clock_list*	
	Returns only those sequential elements clocked by the rising edge of the specified clocks.
`-slave_clock_pins`	Returns the clock pins of the slave flops in the design.

Example

The following command will return the output pins of registers clocked by `CLK1`:

```
genus:root:> all_registers -output_pins -clock CLK1pin:test/Ireg0/Q pin:test/Ireg1/Q
```

append_to_collection

```
append_to_collection
base_collection_name second_collection
[-unique]
```

Description

Appends objects from one collection to another collection.

- If the objects in the base collection have the *same* type, only those objects in the second collection with the same type as those in the base collection are appended to the base collection. If you specify an object list as the second collection, the command converts the objects list to a single list and appends the objects with the same type as those in the base collection to the base collection.
- If the objects in the base collection have *different* types, all objects in the second collection are appended to the base collection. If the second collection has an object list than the command converts the object list to a single collection and append the objects to the base collection for this scenario.
- If the base collection is empty than the result collection is the second collection. If the second collection is empty than the result collection is the base collection.

Options and Arguments

base_collection_name	
	Specifies the variable name of (or pointer to) the base collection to which the objects of the second collection must be added. This base collection becomes the result collection.
	The base collection cannot contain an object list.
	If this argument is a collection instead of a name, its content will be ignored and the result collection will be the second collection.
second_collection	Specifies the second collection.
`-unique`	Removes the duplicate objects from the new collection. If you specify a pointer that does not refer to an existing base collection, the software creates a new collection that uses the pointer as a reference.
	If this option is omitted, duplicate objects will not be removed.

Examples

- The following command appends the nets in the specified collection to the collection `newVar`:

```
genus:root:> append_to_collection newVar [get_nets {CLK1}]
```

- The following command appends the specified nets to the collection `newVar`:

```
genus:root:> append_to_collection newVar {Sin1 CLK1 Sout1}
```

apply_clock_gating_network_latency_table

```
apply_clock_gating_network_latency_table

     [-design object]
     [-out_file string]
```

Description

Applies previously configured clock gating cell latency values to in memory timing constraints using `set_clock_latency`.

Options and Arguments

`-design` *object*	Specifies the top level design for which the latencies will be set.
`-out_file` *string*	Writes the converted commands to a script instead of applying latencies immediately.

Related Information

Related Command
set_clock_gating_network_latency_table
reset_clock_gating_network_latency_table

compare_collections

```
compare_collections
collection1 collection2
[-order_dependent]
```

Description

Compares the contents of two collections. If the objects in both collections are the same, the command returns 0, otherwise the command returns 1.

This command is also aliased to `compare_collection`.

Options and Arguments

collection1 *collection2*	Specifies the name of a collection of objects.
`-order_dependent`	Specifies that the order of the objects in both collections must be the same for the collections to be considered the same.

Examples

- The following command compares two collections without considering the order:

```
genus:root:> compare_collections [concat [get_cells *] [get_ports *]] \
==> [concat [get_ports *] [get_cells *]]

1
```

- The following command compares the same collections this time considering the order:

```
genus:root:> compare_collections [concat [get_cells *] [get_ports *]] \
==> [concat [get_ports *] [get_cells *]] -order_dependent

0
```

copy_collection

```
copy_collection
collection
```

Description

Specifies the collection to be copied. If an empty collection is specified then the command returns an empty collection.

Options and Arguments

collection	Specifies the name of a collection of objects to be copied.

Example

Define a collection col1 and copy the content to collection col2:

```
set col1 [all_registers]

set col2 [copy_collection $col1]]
```

create_clock

```
create_clock
[-add]
[-name clock] [-domain clock_domain]
-period float [-waveform float]
[-apply_inverted {port|pin|hpin}]
[port|pin|hpin] [-comment string]
```

Description

Creates a clock object and defines its waveform. If you do not specify any sources, but you specify the `-name` option, a virtual clock is created.

Options and Arguments

`-add`	Allows to specify multiple clocks on the same source for simultaneous analysis with different clock waveforms.
	If you omit this option and a clock was already defined on a pin or port, this definition would overwrite the previous one.
`-apply_inverted {`*port* | *pin* | *hpin*`}`	
	Specifies inverted sources of the clock.

	This is non-SDC option.
`-comment string`	Specifies a comment to be tagged to this command.
`-domain clock_domain`	
	Specifies the clock domain to which the clock belongs.
	Default: `domain_1`
	This is non-SDC option.
`-name clock`	Specifies the name of the clock. The specified name must be unique. If a clock with this name was already defined, the clock definition will be replaced.
	If you omit this option, the clock gets the same name as the first clock source (inverted or non-inverted). If no sources were specified for this clock, the clock name defaults to `create_clock_n`, where `n` is 1,2, and so on.
`-period float`	Specifies the length of the clock period.
{*port*|*pin*|*hpin*}	Specifies the non-inverted sources of the clock.
`-waveform float...`	Specifies the rise and fall edge times of the clock waveform over one clock period. The first value corresponds to the first rising transition after time zero. The numbers should represent one full clock period. If you omit this option, a default waveform is assumed: the leading edge occurs at 0 and the trailing edge occurs at the midpoint of the period, such that a symmetric clock is generated.

Examples

- The following command creates clock clk1 on port clkA with a period of 5 time units. The clock has a rising edge at 0 time units and a falling edge at 2.5 time units.

  ```
  create_clock [get_ports clkA]  -name clk1  -period 5  -waveform {0 2.5}
  ```

- The following command specifies a comment for the to be created clock `abc`.

  ```
  create_clock -name abc -period 1000 -comment "mycomment for clock abc"
  ```

Related Information

Related attribute:	comment

create_generated_clock

```
create_generated_clock-source {port|pin|hpin}
[-add] [-master_clock clock]
[-name clock][-domain domain][-dont_create_cost_group] [-combinational]
[-divide_by integer]
[-multiply_by integer][-duty_cycle float]
[-edges integer [integer]...][-edge_shift float [float]...]]
[-invert][-source_invert][-apply_inverted {port|pin|hpin}...][-comment string] {port|pin|hpin}...]
```

Description

Creates a new clock signal from the clock waveform of a given pin in the design.

This command must be specified with one of the following options: `-combinational`, `-divide_by`, `-multiply_by` or `-edges`.

Options and Arguments

`-add`	Allows to specify multiple generated clocks on the same source when multiple clocks must fan into the source pin.
	If you omit this option and a clock was already defined on a pin or port, this definition would overwrite the previous one.
	You must specify this option together with the `-master_clock` and `-name` options.
`-apply_inverted {`*`port`*`|`*`pin`*`|`*`hpin`*`}`	
	Specifies the pin or port on which to apply the inverted clock waveform.
	This is non-SDC option.
`-combinational`	Restricts the path search for latency to only paths which do not cross sequential devices, or the source pins of other generated clocks.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-comment` *`string`*	Specifies a comment to be tagged to this command.
`-divide_by` *`integer`*	Determines the frequency of the new clock by dividing the frequency of the source clock by this factor.
	If the frequency is divided by a certain factor, the clock period is multiplied by that same factor.
	If the factor is a power of 2 (2,4,8... except 1), the rising edges of the master clock are used to determine the edges of the generated clock, otherwise the edges are scaled from the master clock edges.
`-domain` *`domain`*	Specifies the clock domain to which the generated clock belongs.
	Default: `domain_1`
	This is non-SDC option.
`-duty_cycle` *`float`*	Specifies the duty cycle (high pulse width) as a percentage of the clock period. Specify a number between 0 and 100.
`-dont_create_cost_group`	Specifies whether the cost group has been created.
`-edge_shift` *`float`* `[`*`float`*`]...`	
	Specifies how much each edge specified with the `-edges` option should be shifted.
`-edges` *`integer`* `[`*`integer`*`]...`	
	Selects a list of edges from the source clock that form the edges of the derived clock.

	You must specify an odd number of edges. The last edge represents the first edge of the next clock period.
	You cannot specify this option with either the `-divide_by` or `-multiply_by` option.
`-invert`	Inverts the resulting waveform of the generated clock.
`-source_invert`	Specifies whether to invert the master clock waveform.
`-master_clock` *clock*	
	Derives the generated (target) clock from the specified clock.
	Use this option if multiple signals arrive at the source pin and you want to indicate a particular clock from which the target clock must be generated.
	You must specify this option together with the `-add` option.
`-multiply_by` *integer*	
	Determines the frequency of the new clock by multiplying the frequency of the source clock with this factor.
	If the frequency is multiplied with a certain factor, the clock period is divided by that same factor.
`-name` *clock*	Specifies the name of the generated clock.
	You must specify this option with the `-add` option to ensure that clocks with the same source have unique names.
{*port*|*pin*|*hpin*}	Specifies the pins and ports to which the generated clock must be applied
`-source {`*port* | *pin* | *hpin*`}`	
	Specifies the name of the pin from which the clock must be derived.

Examples

- The following command derives target clock `gen_clk1` from the clock at pin `CK` on instance `inst1` and divides the frequency by 2. The clock is applied to pin `Q` of instance `inst1`. A comment is tagged to the command.

```
create_generated_clock -name gen_clk1 -source [get_pins inst1/CK] \
-divide_by 2 [get_pins inst1/Q] -comment "mycomment for clock gen_clk1"
```

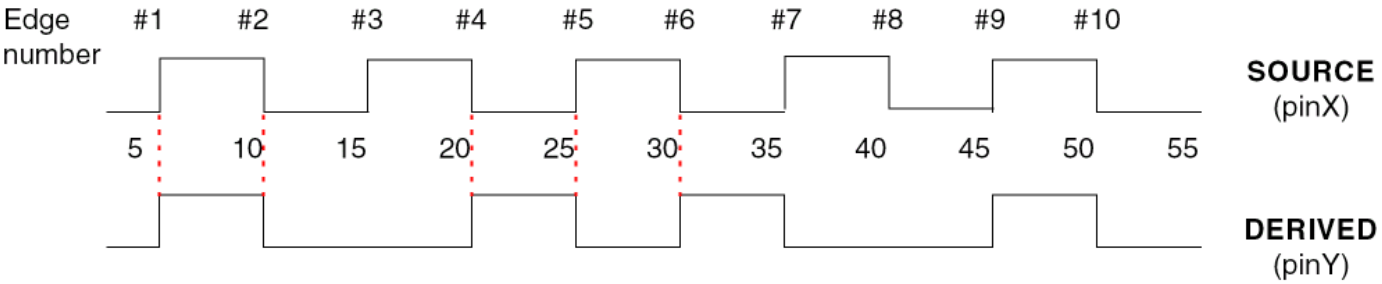
- The following command derives target clock `gen_clk1` from the clock at pin `CK` on instance `inst1` by specifying edges 1, 5 and 7. This implies a `divide_by 3` and a duty cycle of 66.66%.

```
create_generated_clock name gen_clk1 source [get_pins inst1/CK]
-edges {1 5 7} [get_pins inst1/Q]
```

- The following command shows how to create a clock by using edge numbers of the source waveform to specify the new waveform, as shown in the following figure:

```
create_generated_clock -name genclk -source pinX -edges {1 2 4 5 6} pinY
```

Figure -1 Derived Clock (Frequency Division)



Related Information

Related attributes:	comment
	divide_waveform
	waveform

current_design

```
current_design
[design]
```

Description

Changes the current directory in the design hierarchy to the specified design and returns the path to the design.

If no design is specified, the command returns the top-level design.

In some cases multiple designs may be in memory, either from create_derived_design or from reading and elaborating more than a single design's RTL. In this case an error will be issued if no single design is specified.

Options and Arguments

design	Specifies the name of the top-level design.

Examples

- The following example assumes the top design is called `test`. The following command returns the path to the top-level design.

```
genus:root:> current_design

design:test
```

- The following example shows the behavior of the command when more than one design is present.
  Script to load two designs:

```
read_hdl rtl/decoder.v
elaborate decoder

read_hdl rtl/decoder1.v
elaborate decoder1
```

  Command behavior:

```
genus:root:> current_design

Error   : Multiple designs are available. Specify the design you want to use.[TUI-17] [current_design]
 : There is no unique design here.
 : Specify a design by using the cd command to change to that design's directory or specify the design as an argument for
the command.

Failed on find_unique_design

genus:root:> current_design decoder
design:decoder

genus:design:decoder number> current_design decoder1 design:decoder1
```

current_design_name

```
current_design_name
```

Description

If the current directory is the top-level design, this command returns the design name.

If the current directory is a hierarchical instance, this commands returns its corresponding module.

In all other cases, this command returns null.

Examples

- The following command shows the top design name:

```
genus:root:> current_design_name
design:soc
```

- The following command shows the hierarchical instance:

```
genus:root:> current_design_name
inst:soc/cpu
```

- The following command shows the output when no design is present:

```
genus:root:> current_design_name
genus:root:>
```

- The following command uses current_design_name command to save current design name to a variable:

```
genus:root:> set design [current_design_name]
genus:root:> basename [set design [current_design_name]]
soc
```

current_instance

```
current_instance
[instance]
```

Description

Changes the current directory in the design hierarchy to the specified instance and returns the path to the instance.

Current instances can be set to a hierarchical instance as well as to a leaf instance.

If no instance is specified, the command returns the path to the top-level design.

Options and Arguments

instance	Specifies the name of the instance to become the current instance. If `hinst` or `inst` is not given, the current instance is set to the top instance of the hierarchy.

Example

The following command sets the current instance to `MAC/MULTI`, sets a constant on a register pin inside the hierarchical instance and then later sets the current instance back to top:

```
genus:root:> current_instance MAC/MULT1

genus:root:> set_case_analysis 0 [get_pins {reg6/SE}]

genus:root:> current_instance
```

filter_collection

```
filter_collection
[-quiet] [-regexp [-nocase]]
string...
filter_expression
```

Description

Returns a collection of objects that match the specified filter expression.

> The filter expression will be evaluated on all instances, pins, and ports in the design if the object list is empty.

Options and Arguments

filter_expression	Corresponds to a filter expression supported for the specified objects.
	> Genus will generate an error message if the returned collection is empty.
-nocase	Indicates that the specified pattern is not case sensitive.
	> This option can only be used with the -regexp option.
-regexp	Allows to specify regular expressions in the filter expressions. If this option is not specified, the filter expression can only contain wild cards (* and ?).
string	Specifies the collection of objects from which to filter objects for the new collection. The string can have any of the following types:
	{*library*|*lib_arc*|*lib_cell*|*lib_pin*|*design*| *module*|*inst*|*hinst*|*port*|*pin*|*hpin*|*hnet*|*clock*| *exception*}
-quiet	Suppresses all the generated messages.

Examples

- The following command returns all instances whose lib_cell matches `inv1`.

```
genus:root:> filter_collection [get_cells *] "ref_name==inv1"

inst:top_most/inst2         inst:top_most/inst5
```

- The following command searches all instances, pins and ports in the design for an object whose name starts with abc.

```
genus:root:> filter_collection "" "base_name==abc"
```

- The following command returns all instances whose name start with either `CPF` or `bw_reg`. The case of the specified strings must be matched.

```
genus:root:> filter_collection -regexp [get_db insts *] name=="(CPF|bw_reg).*"

inst:chip_top/CPF_ISO_HIER_INST_305     inst:chip_top/bw_reg_1
```

- The following command returns all instances whose name start with either the `cpf` or `BW_REG` string without necessarily matching the case of the specified strings.

```
genus:root:> filter_collection -regexp -nocase [get_db insts *]\
name=="(CPF|bw_reg).*"

inst:chip_top/CPF_ISO_HIER_INST_305     inst:chip_top/bw_reg_1
```

Related Information

For a list of supported filter expressions, refer to

get_cells

get_pins

get_ports

foreach_in_collection

```
foreach_in_collection
iter object_list body
```

Description

Executes a foreach loop over the elements of a collection.

Options and Arguments

body	Specifies the body of the foreach loop.
iter	Specifies the iteration variable.
object_list	Specifies the collection over which to iterate.

Example

Consider the following collection:

```
genus:root:> set list_temp [get_cells *]

instances_seq:test/U3            instances_comb:test/U0instances_comb:test/U1          instances_comb:test/U2
```

The following command prints the corresponding library cell for each instance in the collection:

```
genus:root:> foreach_in_collection icell $list_temp {puts "cell = [get_object_name [get_lib_cells -of $icell]]"}

libcell:      cell = lca300kv/FD1      cell = lca300kv/BUF8A      cell = lca300kv/BUF8A      cell = lca300kv/MUX21S
```

get_arcs

```
get_arcs [quiet] [-filter string]
[-of_objects instance | net ]
[-to pin|port...] [-from pin|port...]
```

Description

Returns a collection of arcs. Using the command options you can filter the elements that are returned.

> This command was added in SDC 1.5.

Options and Arguments

`-quiet`	Suppresses the messages.
`-filter string`	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|!=|!~}name"`
	`base_name` or `name` checks the base name of the instance.
	`-filter "from_pin{==|!=|!~}pin_name"`
	`from_pin` checks the name of the from pin of the arc.
	`-filter "obj_type{==|!=|!~}libarc"`
	`obj_type` checks if the object is a timing arc.
	`-filter "sdf_cond{==|!=|!~}sdf_cond"`
	`sdf_cond` checks the SDF condition of the arc.

	`-filter "timing_type{==|!=|!~}`*`timing_type`*`"`
	`timing_type` checks the timing type (positive_unate, negative_unate, non_unate) of the arc.
	`-filter "when{==|!=|!~}`*`when_cond`*`"`
	`when` checks the when condition of the arc.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-of_objects` *`instance | net`*	
	Specifies to return the arcs for the specified instance or connected to the specified nets (evaluated by `get_nets` *`pattern`*).
`-from` *`pin |`* `port`	Returns all the outgoing arcs of the list of pins provided.
`-to` *`pin |`* `port`	Returns all the incoming arcs of the list of pins provided.

Examples

- Sample usage:

```
genus:root> get_arcs -from [get_pins *] -filter { (is_cellarc == false ) \
&& (is_disabled == true ) }

genus:root> get_arcs -from [get_pins flop1/CK] \
-filter {when == "(((!(R))&(!(SE)))&(!(SI)))"}
```

- Sample usage using `filter_collection`:

```
genus:root> filter_collection [get_arcs -from [get_pins *] ] \
{delay_max_rise < 20}

genus:root> filter_collection [get_arcs -from [get_pins *]] {is_disabled == true && is_cellarc == true}
```

get_cells

```
get_cells [-filter string]
[ -hierarchical | -of_objects list [-leaf]
| pattern [-hsc string] [-regexp [-nocase]] ]
[-quiet]
```

Description

Returns the full path to a list of instances. Using the command options you can filter the elements that are returned.

> This command was added in SDC 1.5.

This command is also aliased to `get_cell`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "area{==|!=|<=|>=|<|>}`*float*`"`
	`area` checks the area of the instance.
	`-filter "{base_name|name}{==|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the instance.
	`-filter "{full_name|hierarchical_name}{==|!=|!~}`*vname*`"`
	`full_name` or `hierarchical_name` checks the vdir name of the instance.
	`-filter "is_black_box{==|!=}{true|false}"`
	`is_black_box` checks if the instance is a black box.
	`-filter "is_combinational{==|!=}{true|false}"`
	`is_combinational` checks if the instance is combinational.
	`-filter "is_dont_touch{==|!=}{true|false}"`
	`is_dont_touch` checks if the instance is preserved.
	`-filter "is_hierarchical{==|!=}{true|false}"`
	`is_hierarchical` checks if the instance is hierarchical.

	`-filter "is_icg{==|!=}{true|false}"`
	`is_icg` checks if the cell is an integrated clock-gating cell.
	`-filter "is_sequential{==|!=}{true|false}"`
	`is_sequential` checks if the instance is sequential.
	`-filter "object_type{==|=~|!=|!~}instance"`
	`object_type` checks if the object is an instance.
	`-filter "pin_count{==|!=|<=|>=|<|>}`*`integer`*`"`
	`pin_count` checks the pin count of the instance.
	`-filter "ref_lib_cell_name{==|=~|!=|!~}`*`lib_cell`*`"`
	`ref_lib_cell_name` checks the lib_cell name of the instance.
	`-filter "ref_name{==|=~|!=|!~}`*`lib_cell`*`"`
	`ref_name` checks the lib_cell name of the instance.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-hierarchical`	Selects all instances in the hierarchy below the current level that match the specified pattern.
`-hsc` *`string`*	Specifies the hierarchical delimiter for patterns.
	For example, if the hierarchical delimiter is @, the `sub/I1@I2` pattern matches instance `I2` within the hierarchy `sub/I1`.
`-leaf`	Returns all leaf cells found for the *nets* specified with the `-of_objects` option after traversing the hierarchical boundaries.
	Without the `-leaf` option, the command returns all hierarchical and leaf cells found at the same level of the nets specified with the `-of_objects` option without traversing the hierarchical boundaries
	This option can only be specified with the `-of_objects` option.
`-nocase`	Indicates that the pattern is not case sensitive.
	This option can only be used with the `-regexp` option.
	If `-regexp` is omitted, the pattern is assumed to be the wildcard.

`-of_objects` *list*	
	Specifies to return the instances with the specified pins (evaluated by `get_pins` *pattern)* or connected to the specified nets (evaluated by `get_nets` *pattern*).
pattern	Specifies a pattern of cell names or regular expressions.
	Default: *
`-quiet`	Suppresses all error and warning messages generated.
`-regexp`	Specifies that the pattern is a regular expression.

Example

The following command uses a complex expression to return the path to

- the list of all instances that either have lib_cell name inv1 or instance name `inst_mux`,
- the list of instances that either have lib_cell `MX2X1` or `fflopd`

```
genus:root:> get_cells -filter "((ref_name ==inv1 || full_name==inst_mux) && (ref_name==MX2X1)) || (ref_name==fflopd))
```

get_clock_gating_history_dictionary_entry

`get_clock_gating_history_dictionary_entry`

`[-design` *design*`]`
`[-id` *long*`]`
`[-all]`

Description

Gets strings from the dictionary used by the `lp_clock_gating_history` attribute.

Options and Arguments

`-design` *design*	Specifies the design if multiple designs are present.
`-id` *long*	Returns the corresponding string for a specific id.
-all	Returns a Tcl dict of all dictionary entries. This is a list of pairs and each pair consists of an id and a string.

Related Information

Related Attributes
lp_clock_gating_history

get_clocks

```
get_clocks [-filter string]
[-regexp] [-nocase] [-quiet] [pattern]
```

Description

Returns the full path to a list of timing clocks that match the pattern. This command reports all clocks, including inactive clocks.
This command is also aliased to `get_clock`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the clock.
	`-filter "full_name{==|=~|!=|!~}`*vname*`"`
	`full_name` checks the vdir name of the clock
	`-filter "has_detailed_parasitics{==|!=}{true|false}"`
	`has_detailed_parasitics` checks if the `phys_capacitance` attribute is set on the clock.
	`-filter "object_type{==|=~|!=|!~}clock"`
	`object_type` checks if the object is a clock.
	`-filter "period{==|!=|<=|>=|<|>}`*period*`"`
	`period` checks the period of the clock.
	`-filter "sources{==|=~|!=|!~}`*clock_sources*`"`
	`sources` checks the inverted and non-inverted sources of the clock
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.

	This is a non-SDC option.
`-nocase`	Specifies that pattern matching is not case sensitive.
	You must specify -regexp in order to use this parameter.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
pattern	Specifies a pattern of clock names.
	Patterns include the * and ? wildcard characters, and collections of instances and clocks.
	Default: *
`-quiet`	Suppresses all the generated messages that are generated when the `get_clocks` command is run.
`-regexpr`	Treats the specified patterns as a regular expression patterns.
	Default: Treats the specified pattern as a wild card.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

Examples

- The following command returns the path to clock `test_clock`.

```
genus:root:> get_clocks test_clock
clock:top_most/domain_1/test_clock
```

- The following command returns the path to all clocks starting with `test`:

```
genus:root:> get_clocks test*
clock:top_most/domain_1/test_clock
```

- The following command returns the path to a clock whose name is `c1`.

```
genus:root:> get_clocks -filter "full_name==c1"
clock:top/domain_1/c1
```

get_designs

```
get_designs
[-filter string] [-quiet] pattern
```

Description

Returns the full path to any design or module matching the pattern.

This command is also aliased to `get_design`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the name of the design.
	`-filter "full_name{==|=~|!=|!~}`*vname*`"`
	`full_name` checks the vdir name of the design
	`-filter "is_dont_touch{==|!=}{true|false}"`
	`is_dont_touch` checks if the design is preserved.
	`-filter "max_capacitance{==|!=|<=|>=|<|>}`*float*`"`
	`max_capacitance` checks the `max_capacitance` attribute set on the design
	`-filter "max_fanout{==|!=|<=|>=|<|>}`*integer*`"`
	`max_fanout` checks the `max_fanout` attribute set on the design
	`-filter "max_transition{==|!=|<=|>=|<|>}`*float*`"`
	`max_transition` checks the `max_transition` attribute set on the design
	`-filter "object_type{==|=~|!=|!~}design"`
	`object_type` checks if the object is a design.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
pattern	Specifies a pattern of design or module names.
`-quiet`	Suppresses all the generated messages.

Examples

- The following command returns the path to design `top_most`.

  ```
  get_designs top_most*
  design:top_most
  ```

- The following command returns a complete list of modules:

  ```
  get_designs *
  ```

- This command is useful for finding sub-designs (modules) for limiting optimization:

  ```
  get_designs latch_RC_CG*
  ```
  It returns all the sub-designs with prefixes of `latch_RC_CG`. This command can be used together with other commands to apply optimization guidelines.

get_equivalent_cells

```
get_equivalent_cells
-cell lib_cell
[-delay_corner delay_corner]
[-early_only]
[-late_only]
[-library library]
[-max]
[-min]
[-power_domain power_domain]
```

Description

Reports equivalent cells.

Options and Arguments

`-cell` *lib_cell*	Specifies the cell name for which equivalent cells need to be fetched.
`-delay_corner` *delay_corner*	Specifies to fetch the equivalent cells from libraries of a specific delay corner only.
`-early_only`	Specifies to fetch cells only from the early libraries of the specified delay corner.
`-late_only`	Specifies to fetch cells only from the late libraries of the specified delay corner.
`-library` *library*	Specifies to fetch cells only for the given library.
`-max`	Specifies to fetch equivalent cells from the max libraries only.
`-min`	Specifies to fetch equivalent cells from min libraries only.
`-power_domain` *power_domain*	Specifies to fetch cells only for the libraries of a specific power domain.

get_generated_clocks

```
get_generated_clocks
[-quiet] [-regexp] [-exact] [-nocase]
[-filter expr] [pattern]
```

Description

Returns the full path to all the generated clocks that match the pattern. Using the command options you can filter the clocks that are returned.

Options and Arguments

`-exact`	Performs exact pattern matching against the object name, instead of wild card or regular expression matching. Use this parameter when the object name includes special characters such as * , ? , and + .
	If you specify this parameter, you cannot specify -`regexp`.
`-filter` *`expr`*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*`name`*`"`
	`base_name` or `name` checks the base name of the generated clock.
	`-filter "full_name{==|=~|!=|!~}`*`vname`*`"`
	`full_name` checks the vdir name of the generated clock
	`-filter "object_type{==|=~|!=|!~}clock"`
	`object_type` checks if the object is a clock.
	`-filter "period{==|!=|<=|>=|<|>}`*`period`*`"`
	`period` checks the period of the generated clock.
	`-filter "sources{==|=~|!=|!~}`*`clock_sources`*`"`
	`sources` checks the inverted and non-inverted sources of the generated clock
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
	If `-regexp` is omitted, the pattern is assumed to be the wildcard.
`pattern`	Specifies the pattern for generated clock names to be matched. Patterns include the * and ? wildcard characters, and collections of generated clocks.
	If no pattern is specified, all generated clocks are considered.
`-quiet`	Suppresses all error and warning messages generated.
`-regexp`	Treats the specified pattern as a regular expression.
	`Default`: Treats the specified pattern as a wild card

	If you specify this parameter, you cannot specify `-exact`.

Related Information

Related command:	create_generated_clock

get_leaf_cells

```
get_leaf_cells
pattern
```

Description

Returns the full path to all the leaf cells that match the pattern.

Options and Arguments

pattern	Specifies the pattern for leaf cells to be matched.

Example

The following example matches sss*g1 cells:

```
genus:root:> get_leaf_cells sss*glinst:test/sss/glinst:test/sss/sss1/glinst:test/sss1g1
```

get_leaf_nets

```
get_leaf_nets
pattern
```

Description

Returns the full path to all the leaf nets that match the pattern.

Options and Arguments

pattern	Specifies the pattern for instance to be matched.

Example

`get_leaf_nets sss*` can match `hnet:test/sss/y` and `hnet:test/sss/a`

get_leaf_pins

```
get_leaf_pins
pattern
```

Description

Returns the full path to all the leaf pins that match the pattern.

Options and Arguments

pattern	Specifies the pattern for instances to be matched.

Example

The following example matches sss*g1 pins:

```
genus:root:> get_leaf_pins sss*g1pin:test/sss/g1/zpin:test/sss/g1/apin:test/sss/sss1/g1/zpin:test/sss/sss1/g1/a
```

get_lib_arcs

```
get_lib_arcs
[-quiet] [-to lib_pin...] [-from lib_pin...]
[-of_objects lib_cell...] [-filter string]
```

Description

Returns all timing arcs that match the specified pattern. Assign these library timing arcs to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

This command is also aliased to `get_lib_timing_arcs`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the name of the timing arc.

	`-filter "from_lib_pin{==|=~|!=|!~}`*`pin_name`*`"`
	`from_lib_pin` checks the name of the from pin of the arc.
	`-filter "object_type{==|=~|!=|!~}libarc"`
	`object_type` checks if the object is a timing arc.
	`-filter "sdf_cond{==|=~|!=|!~}`*`sdf_cond`*`"`
	`sdf_cond` checks the SDF condition of the arc.
	`-filter "timing_type{==|=~|!=|!~}`*`timing_type`*`"`
	`timing_type` checks the timing type (positive_unate, negative_unate, non_unate) of the arc.
	`-filter "when{==|=~|!=|!~}`*`when_cond`*`"`
	`when` checks the when condition of the arc.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-from` *`lib_pins`*	Returns all outgoing timing arcs for the specified library cell pin(s). You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection.If you specify this parameter with the `-to` parameter, the software creates a collection of all library arcs that exist between the `to` and `from` library pins.
	You cannot specify this parameter with the `-of_objects` parameter.
`-of_objects` *`cells`*	
	Returns all timing arcs for the specified library cells.
	You can specify a list of library cells or a list of timing arcs, but you cannot specify a list that contains both library cells and timing arcs. You can specify a pattern for the library cell or timing arc name, a collection of library cells or timing arcs, or a pattern and a collection.
	You cannot specify this parameter with the `-to` or `-from` parameter.
`-to` *`lib_pins`*	Returns all incoming timing arcs for the specified library cell pin(s).
	You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection.

	If you specify this parameter with the -`from` parameter, the software creates a collection of all library arcs that exist between the `to` and `from` library pins.
	You cannot specify this parameter with the -`of_objects` parameter.

Examples

- The following two commands returns all timing arcs for the specified library cell AND4X12

```
get_lib_arcs -of_objects\
lib_cell:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M
```

Below is the snippet of output:

```
0x750
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/A_Y_n90
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/B_Y_n90
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/C_Y_n90
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/D_Y_n90
```

```
get_lib_arcs -of_objects AND4X12M
```

Below is the snippet of output:

```
0x751
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/A_Y_n90
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/B_Y_n90
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/C_Y_n90
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/AND4X12M/D_Y_n90
```

- The following command returns all outgoing timing arcs for the specified library cell pin(s):

```
get_lib_arcs -from \
lib_pin:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK
```

Below is the snippet of output:

```
0x758
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK_CK_W00
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK_D_Sa2
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK_D_Ha1
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK_Q_ca3
```

- The following command returns all incoming timing arcs for the specified library cell pin(s):

```
get_lib_arcs  -to \
lib_pin:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/Q
```

Below is the snippet of output:

```
0x759
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK_Q_ca3
```

```
get_lib_arcs -from \
lib_pin:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK -to\
lib_pin:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/Q
```

```
0x750
lib_arc:default_emulate_libset_max/ss_g_1v08_125c/DFFQX1M/CK_Q_ca3
```

get_lib_cells

```
get_lib_cells
[-filter string] [-of_objects string] [-hsc string]
[-regexp] [-nocase] [-quiet] [pattern]
```

Description

Returns the full path to any library cell that matches the pattern. Using the command options you can filter the elements that are returned.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "area{==|!=|<=|>=|<|>}`*float*`"`
	`area` checks the area of the lib_cell
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the lib_cell.
	`-filter "is_combinational{==|!=}{true|false}"`
	`is_combinational` checks if the lib_cell is combinational
	`-filter "is_dont_touch{==|!=}{true|false}"`
	`is_dont_touch` checks if the lib_cell is preserved.
	`-filter "is_dont_use{==|!=}{true|false}"`
	`is_dont_use` checks if the lib_cell must be avoided.
	`-filter "is_sequential{==|!=}{true|false}"`
	`is_sequential` checks if the lib_cell is sequential.
	`-filter "is_tristate{==|!=}{true|false}"`
	`is_tristate` checks if the lib_cell is tristate.
	`-filter "object_type{==|=~|!=|!~}libcell"`
	`object_type` checks if the object is a lib_cell.

	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-hsc` *string*	Specifies the hierarchical delimiter for patterns.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
	If `-regexp` is omitted, the pattern is assumed to be the wildcard.
`-of_objects` *string*	Specifies the libpins or instances for which to return the lib_cells.
pattern	Specifies a pattern of lib_cell names or regular expressions.
`-quiet`	Suppresses all the generated messages.
`-regexp`	Specifies that the pattern is a regular expression.

get_lib_pins

```
get_lib_pins
[-filter string] [-of_objects string] [-hsc string]
[-regexp] [-nocase] [-quiet] [pattern]
```

Description

Returns the full path to any lib_cell pin that matches the pattern. Using the command options you can filter the elements that are returned.

This command is also aliased to `get_lib_pin`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the lib_cell pin.

	`-filter "capacitance{==|!=|<=|>=|<|>}float"`
	`capacitance` checks the average capacitance (rise+fall/2) of the lib_cell pin.
	`-filter "direction=={in|out|inout}"`
	`direction` checks the direction of the lib_cell pin.
	`-filter "is_async_pin{==|!=}{true|false}"`
	`is_async_pin` checks if the lib_cell pin is an asynchronous pin.
	`-filter "is_clear_pin{==|!=}{true|false}"`
	`is_clear_pin` checks if the lib_cell pin is a clear pin.
	`-filter "is_clock_pin{==|!=}{true|false}"`
	`is_clock_pin` checks if the lib_cell pin is a clock pin.
	`-filter "max_capacitance{==|!=|<=|>=|<|>}float"`
	`max_capacitance` checks the `max_capacitance` attribute set on the lib_cell pin.
	`-filter "max_fanout{==|!=|<=|>=|<|>}integer"`
	`max_fanout` checks the `max_fanout` attribute set on the lib_cell pin.
	`-filter "max_transition{==|!=|<=|>=|<|>}float"`
	`max_transition` checks the `max_transition` attribute set on the lib_cell pin.
	`-filter "object_type{==|=~|!=|!~}libpin"`
	`object_type` checks if the object is a libpin.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-hsc` *string*	Specifies the hierarchical delimiter for patterns.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
	If `-regexp` is omitted, the pattern is assumed to be the wildcard.

`-of_objects` *string*	
	Specifies the lib_cells or pins for which to return the lib_cell pins.
pattern	Specifies a pattern of libpin names or regular expressions.
`-quiet`	Suppresses all the generated messages.
`-regexp`	Specifies that the pattern is a regular expression.

get_libs

```
get_libs
[-filter string]
[-regexp] [-nocase] [-quiet]
{-of_objects object_list | pattern_list }
```

Description

Returns the full path to any library that matches the pattern. Using the command options you can filter the elements that are returned.

This command is also aliased to `get_lib`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the library.
	`-filter "object_type{==|=~|!=|!~}library"`
	`object_type` checks if the object is a library.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.

	If `-regexp` is omitted, the pattern is assumed to be the wildcard.
`-of_objects` *`object_list`*	
	Creates a collection of libraries based on the library cells in the given object list.
`pattern_list`	Specifies a pattern of library names or regular expressions.
`-quiet`	Suppresses all the generated messages.
`-regexp`	Specifies that the pattern is a regular expression.

get_nets

```
get_nets
[-filter string]
[-hierarchical] [-hsc string] [-regexp]
[-nocase] [-quiet] [-of_objects string | pattern]
[-segments [-top_net_of_hierarchical_group]]
```

Description

Returns the full path to any net that matches the pattern. Using the command options you can filter the elements that are returned.

This command is also aliased to `get_net`.

Options and Arguments

`-filter` *`string`*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "area{==|!=|<=|>=|<|>}`*`float`*`"`
	`area` checks the enclosed area of the net
	`-filter "{base_name|name}{==|=~|!=|!~}`*`name`*`"`
	`base_name` or `name` checks the base name of the net.
	`-filter "full_name{==|=~|!=|!~}`*`vname`*`"`
	`full_name` checks the vdir name of the net
	`-filter "is_dont_touch{==|!=}{true|false}"`
	`is_dont_touch` checks if the net is preserved.
	`-filter "object_type{==|=~|!=|!~}net"`
	`object_type` checks if the object is a net.

	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-hierarchical`	Selects all the nets in the hierarchy below the current level that match the specified pattern.
`-hsc` *string*	Specifies the hierarchical delimiter for patterns.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
	If `-regexp` is omitted, the pattern is assumed to be the wildcard.
`-of_objects` *string*	Specifies to return the nets that are connected to the specified cells (evaluated by `get_cells` *pattern*), pins (evaluated by `get_pins` *pattern*), and ports (evaluated by `get_ports` *pattern*).
pattern	Specifies a pattern of net names or regular expressions.
	Default: *
`-quiet`	Suppresses all the generated messages.
`-regexp`	Specifies that the pattern is a regular expression.
`-segments`	Returns all global segments for the given nets. Global net segments are all those physically connected segments across all hierarchical boundaries. For each net, all global segments that are part of that net are added to the resulting collection.
	This modifies the initial search that matches patterns or objects
	It is recommended to use this option with a single net.
`-top_net_of_hierarchical_group`	
	Keeps only the top net of a hierarchical group. When more than one hierarchical net of the same group is specified (local nets at various hierarchical levels of the same physical net), only the net closest to the top of the hierarchy is saved with the collection. In the case of multiple nets at the same level, the first net specified is kept.
	When you use this option together with the `-segments` option, you can isolate the highest net segment of a physical net.

Examples

- The following command returns all the segments of a net starting from port `in` driving port `A` of an inverter 2 levels below:
  `i1_top/i1_mid/i1_inv/A`

  `get_nets -segments i1_mid/in`
  Returns following segments: `in, i1_top/in, i1_mid/in`
- The following command returns the net segment closest to the top of `i1_mid/in`

  `get_nets -segments -top_net_of_hierarchical_group i1_mid/in`
  Returns following segment: `in`

get_object_name

```
get_object_name
{library|lib_cell|lib_pin|lib_arc|module|idesign|inst|hinst|port|pin
|hpin|hnet|clock|exception}...|
```

Description

This command was added for compatibility while reading in SDC files with non-SDC constraints. Generally, this command is used with the command get_* in SDC. By default, get_* commands returns a pointer of the objects like `get_object_name [get_* *]` returns object-list of the pointer. This command can also be used to get the object name without Genus hierarchal.

> Since Genus does not have a concept of collections, this command just returns the specified argument. If the object name includes `[]`, `get_object_name` return value is "`{$the_object}`".

Options and Arguments

string	Specifies the argument to be returned.

Examples

- The following examples shows that when the port port_pad_data_out[7] includes [], so the return value is {port_pad_data_out[7]}:

  ```
  genus:root:> get_object_name [get_ports port_pad_data_out[7]]
  {port_pad_data_out[7]}
  ```

- When the port int doesn't include [], the returned value is int:

  ```
  genus:root:> get_object_name [get_ports int]
  int
  ```

- The following example shows how to use get_object_name to get the object's name without Genus hierarchal.

  ```
  genus:root:> get_object_name port:dtmf_recvr_core/switch_en_out
  switch_en_out
  ```

get_path_groups

```
get_path_groups
[-quiet] [-regexp [-nocase]]
[-filter string] [pattern]
```

Description

Returns all the path groups that match the pattern. Using the command options you can filter the path groups that are returned.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the path group.
	`-filter "object_type{==|=~|!=|!~}path_group"`
	`object_type` checks if the object is an path_group.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
	If `-regexp` is omitted, the pattern is assumed to be the wildcard.
pattern	Specifies the pattern for path group names to be matched.
	If no pattern is specified all path groups are considered.
`-quiet`	Suppresses all the generated messages.
`-regexp`	Specifies that the specified pattern is a regular expression.

Related Information

Related commands:	create_generated_clock
	path_group

get_pins

```
get_pins
[-filter string] [-hierarchical] [-hsc string] [-regexp]
[-nocase] [-quiet] {-of_objects string [-leaf]| pattern}
```

Description

Returns the full path to any pin that matches the pattern. Using the command options you can filter the elements that are returned.

This command is also aliased to `get_pin`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:	
	`-filter "{base_name|pin_name|name}{==|=~|!=|!~}`*name*`"`	
	`base_name`, `pin_name` or `name` checks the base name of the pin.	
	If the pin name specified in the SDC file cannot be found in the design, Genus uses the patterns listed in the following mapping table to search for the pins:	
	Pin name in SDC	Pattern searched for in design
	`D`, `next_state`, `data_in`	`d`
	`Q`	`q`
	`CLK`, `CP`, `clocked_on`, `enable`	`clk`, `ena`
	`SEN`, `synch_enable`	`sena`
	`SCLR`, `SPRE`, `synch_preset`, `synch_clear`, `synch_toggle`	`srl`, `srd`
	`APRE`, `preset`	`apre`
	`ACLR`, `clear`	`aclr`
	`-filter "{direction|pin_direction}=={in|out|inout}"`	
	`direction` or `pin_direction` checks the direction of the pin.	
	`-filter "{full_name|hierarchical_name}{==|=~|!=|!~}`*vname*`"`	
	`full_name` or `hierarchical_name` checks the vdir name of the pin.	
	`-filter "fanin{==|=~|!=|!~}`*integer*`"`	
	`fanin` checks the fanin of the pin.	
	`-filter "fanout{==|=~|!=|!~}`*integer*`"`	
	`fanout` checks the fanout of the pin.	

	`-filter "is_clock_gating_pin{==|!=}{true|false}"`
	`is_clock_gating_ pin` checks if the lib_cell pin is a clock or enable pin of a clock_gating cell.
	`-filter "is_clock_pin{==|!=}{true|false}"`
	`is_clock_pin` checks if the lib_cell pin is a clock pin.
	`-filter "is_hierarchical{==|!=}{true|false}"`
	`is_hierarchical` checks if the pin is hierarchical.
	`-filter "is_on_clock_network{==|!=}{true|false}"`
	`is_on_clock_network` checks if the lib_cell pin if the pin is on a clock network.
	`-filter "is_pin{==|!=}{true|false}"`
	`is_pin` checks if the object is a pin.
	`-filter "is_port{==|!=}{true|false}"`
	`is_port` checks if the object is a port.
	`-filter "lib_pin_name{==|=~|!=|!~}`*`string`*`"`
	`lib_pin_name` checks if the object is a libpin.
	`-filter "object_type{==|=~|!=|!~}`*`string`*`"`
	`object_type` checks if the object is a pin.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
	This is a non-SDC option.
`-hierarchical`	Selects all pins in the hierarchy below the current level that match the specified pattern.
`-hsc` *`string`*	Specifies the hierarchical delimiter for patterns.
`-leaf`	Returns all leaf pins found for the *nets* specified with the `-of_objects` option after traversing the hierarchical boundaries.
	Without the `-leaf` option, the command returns all hierarchical and leaf pins found at the same level of the nets specified with the `-of_objects` option without traversing the hierarchical boundaries.

	This option can only be specified with the `-of_objects` option.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
`-of_objects` *string*	Specifies to return the pins of the specified instances (evaluated by `get_cells` *pattern*) and nets (evaluated by `get_nets` *pattern*).
pattern	Specifies a pattern of cell names or regular expressions.
`-quiet`	Suppresses all the generated messages.
`-regexp`	Specifies that the pattern is a regular expression.

Examples

- The following command returns the full path to all the pins of instance `g6`.

```
genus:root:> get_pins -hsc g6*
hpin:rct/g6/A hpin:/rct/g6/B hpin:rct/g6/Y
```

- The following command returns the full path to the output pin of instance `g6`.

```
genus:root:> get_pins -hsc g6* -filter direction==out
hpin:rct/g6/Y
```

- The following command returns the full path to all output pins of the instances whose name start with `g`.

```
genus:root:> get_pins -filter direction==out -of_objects g*
hpin:rct/g3/Y hpin:rct/g5/Y hpin:rct/g4/Y
```

- The following command returns the full path to all output pins to which nets `n_1` and `n_3` connect.

```
genus:root:> get_pins  -filter direction==out -of_objects {n_1 n_3}
hpin:rct/g6/Y hpin:rct/cdn_loop_breaker/Y
```

- The following command checks if the pin inst1/A specified in the filter expression exist and then returns the path to the pin:

```
genus:root:> get_pins -filter "full_name==inst1/A"
hpin:top/inst1/A
```

- The following commands return nothing:

```
genus:root:> get_pins -filter "is_port==true" *

genus:root:> get_ports -filter "is_pin==true" *
```

- The following command returns all pins connected by net `ck`.

```
genus:root:> get_pins -filter "object_type==pin" -of_object hnet:rct/aco1/ck
hpin:rct/aco1/c_reg[3]/clk hpin:rct/aco1/c_reg[0]/clk hpin:rct/aco1/c_reg[4]/clk hpin:rct/aco1/c_reg[5]/clk
hpin:rct/aco1/c_reg[2]/clk
```

- The following example shows the effect of the `-quiet` option.

```
genus:root:> get_pins invalid_pin
Warning : Could not find requested search value. [SDC-208] [get_pins]
        : The 'get_pins' command  cannot find any pins named 'invalid_pin'
        : Use the 'vcd' and 'vls' commands to browse the virtual directories to
find the object because the specified name and/or location does not exist.
genus:root:> get_pins -quiet invalid_pin
genus:root:>
```

get_ports

```
get_ports
[-filter string] [-regexp] [-nocase] [-quiet]
[-of_objects string | pattern]
```

Description

Returns the full path to any port that matches the pattern. Using the command options you can filter the elements that are returned.

This command is also aliased to `get_port`.

Options and Arguments

`-filter` *string*	Filters the result based on the specified expression evaluating to `true`. The following filter expressions are supported:
	`-filter "{base_name|name}{==|=~|!=|!~}`*name*`"`
	`base_name` or `name` checks the base name of the port.
	`-filter "{direction|port_direction}=={in|out|inout}"`
	`direction` or `port_direction` checks the direction of the port.
	`-filter "full_name{==|=~|!=|!~}`*vname_of_port*`"`
	`full_name` checks the vdir name of the port
	`-filter "is_on_clock_network{==|!=}{true|false}"`
	`is_on_clock_network` checks if the lib_cell pin if the pin is on a clock network.
	`-filter "is_pin{==|!=}{true|false}"`
	`is_pin` checks if the object is a pin.
	`-filter "is_port{==|!=}{true|false}"`
	`is_port` checks if the object is a port.
	`-filter "max_capacitance{==|!=|<=|>=|<|>}`*float*`"`
	`max_capacitance` checks the maximum capacitance value set on the port
	`-filter "max_fanout{==|!=|<=|>=|<|>}`*integer*`"`
	`max_fanout` checks the `max_fanout` attribute set on the port

	`-filter "max_transition{==|!=|<=|>=|<|>}`*`float`*`"`
	`max_transition` checks the `max_transition` attribute set on the port
	`-filter "object_type{==|=~|!=|!~}`*`string`*`"`
	`object_type` checks if the object is a port.
	Any filter option can be specified as `option` or `@option` in the expression.
	You can also combine two or more expressions using the following operators: `AND`, `OR`, `&&`, and `||`. You can use parentheses `()` to enclose simple filter expressions. However, if any of the individual filter expressions contains an `&`, `|`, `AND`, or `OR`, the command errors out.
`-nocase`	Indicates that the pattern is not case sensitive. This option can only be used with the `-regexp` option.
`-of_objects` *`string`*	Specifies to return the ports connected to the specified nets (evaluated by `get_nets` *`pattern`*).
`pattern`	Specifies a pattern of port names or regular expressions.
	Default: *
`-quiet`	Suppresses all the generated warning and error messages.
`-regexp`	Specifies that the pattern is a regular expression.

Examples

- The following command returns the full path to all input ports starting with letter `d`.

```
genus:root:> get_ports -filter direction==in d*

port:rct/din[15] port:rct/din[14] port:rct/din[13] port:rct/din[12] port:rct/din[11] port:rct/din[10] port:rct/din[9]
port:rct/din[8] port:rct/din[7] port:rct/din[6] port:rct/din[5] port:rct/din[4] port:rct/din[3] port:rct/din[2]
port:rct/din[1] port:rct/din[0] port:rct/done
```

- The following command returns the full path to all ports connected to nets whose name start with `rd_addr`.

```
genus:root:> get_ports -of_objects rd_addr*

0x15
port:rct/rd_addr[0] port:rct/rd_addr[9]} port:rct/rd_addr[11] port:rct/rd_addr[2] port:rct/rd_addr[13] port:rct/rd_addr[4]
port:rct/rd_addr[15] port:rct/rd_addr[6] port:rct/rd_addr[8] port:rct/rd_addr[1] port:rct/rd_addr[10] port:rct/rd_addr[12]
port:rct/rd_addr[3] port:rct/rd_addr[14] port:rct/rd_addr[5] port:rct/rd_addr[7]}
```

- The following command returns nothing:

```
genus:root:> get_ports -filter "is_pin==true" *
```

get_references

```
get_references
{module | inst| hinst} [-quiet]
```

Description

Returns

- The list of instances of the specified *module*
- The module of the specified *hierarchical instance*
- The lib_cell of the specified *leaf instance*

This command is also aliased to `get_reference`.

Options and Arguments

*inst	hinst*	Specifies the instance for which to get the reference. You can specify a leaf instance or a hierarchical instance.
module	Specifies the module for which to get the reference.
-quiet	Suppresses all the generated messages.

getenv

```
getenv
env_variable
```

Description

Returns the contents of the specified environment variable.

Options and Arguments

env_variable	Specifies the environment variable for which you want to display the contents.

Example

The following command displays the content of the `CDN_SYNTH_ROOT` variable.

```
genus:root:> getenv CDN_SYNTH_ROOT /net/your_install_dir/tools.lnx86
```

group_path

```
group_path
{{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-rise_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-fall_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-rise][-fall][-setup][-hold]
[-exception_name name]{-name cost_group | -default}
[-priority integer][-weight float]
[-critical_range float][-comment string]
```

Description

Assigns paths that meet the path selection criteria to a cost group. Paths can be selected using the `-from`, `-fall_from`, `-rise_from`, `-through`, `-fall_through`, `-rise_through`, `-to`, `-fall_to`, or `-rise_to` options. You must provide at least one of these options.

Options and Arguments

`-comment string`	Specifies a comment to be tagged to this command.
`-critical_range`	Specifies the margin of delay for optimization.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-default`	Associates the path group with the default cost group
`-exception_name name`	
	Specifies the name of the timing exception this path group should be associated with.
`-fall`	Limits the path selection to those paths who have a falling edge at the end points.
`-fall_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}`	
	Selects all paths whose signals have a falling edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-fall_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}`	
	Selects all paths whose signals have a falling edge on the specified through points. The through points can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-fall_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}`	
	Selects all paths whose signals have a falling edge at the specified end points.The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}`	
	Selects all paths that start from the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.

`-hold`	
	Genus ignores this option for optimization and reporting purposes. The `write_sdc` command will write the option out for consumption by downstream tools.
`-name` *cost_group*	Specifies the name of the cost group to which this path group belongs. The cost group should have been previously defined with a `define_cost_group` command.
`-rise`	Limits the path selection to those paths who have a rising edge at the end points.
`-rise_from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-rise_through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge on the specified through points. The through points can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-rise_to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge on the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-setup`	Indicates that the path groups apply for setup analysis only.
`-through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths that traverse through the specified points. The points to traverse can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-priority` *integer*	
	Specifies the user priority setting for an exception.
`-weight` *float*	Sets a weight for the cost group with which this path group is associated.

Examples

- The following example assigns the paths from all inputs to all outputs to the cost group called `I2O`.

```
group_path -from port:*/*IN* -to port:*/*OUT* \
-name I2O -comment " my path_group"
```

- The following two commands are equivalent. They both define path groups for cost group C2C that have a rising edges at the end points.

```
group_path -rise -to inst1/pin1 -name C2C
```

```
group_path -rise_to inst1/pin1 -name C2C
```

Related Information

Related commands:	define_cost_group
	path_group
	set_path_specification
Related attribute:	comment

index_collection

```
index_collection
collection integer
```

Description

Searches an ordered list of objects and creates a new collection containing the object at the specified index.

> The first object in the list is at index 0.

An error will be given if the collection is empty.

Options and Arguments

collection	Specifies the name of a collection of objects to be searched.
integer	Specifies the index of the object to be returned. Valid values range from 0 till the length of the list -1.
	If the specified index is outside of the allowed range, an error message will be returned.

list_property

```
list_property -type string
```

Description

Lists all of the properties associated with the specified object type. The supported object types are cell, pin, port, clock, lib_cell, lib_pin, lib (library), lib_timing_arc, design and net.

Options and Arguments

-type *string*	Specifies the object type for which to list the properties

Example

The following example shows the properties of library type object.

```
genus:root:> list_property -type lib

Property : nameProperty : base_nameProperty : object_typeProperty : object_class...
```

load_of

```
load_of {lib_pin | pin | hpin}
```

Description

Returns the capacitance of the specified pin.

In case of a library pin:

- The command returns the largest of the rise and fall capacitances of the lib_pin.

In case of an instance pin:

- For a pin that does have a corresponding library pin, the command returns the largest of the rise and fall capacitances of the libpin.
- For a pin that has no corresponding library pin, the command returns 0.0.

Options and Arguments

lib_pin	Specifies the library pin for which to return the capacitance
pin | *hpin*	Specifies the instance pin for which to return the capacitance

Example

The following commands return the capacitance (load) value of lib_pin, pin and hpin respectively.

```
genus:root:> load_of lib_pin:default_emulate_libset_max/slow_1v0/ACHCONX2/B0.005

genus:root:> load_of pin:test/inst1/SN0.001

genus:root:> load_of hpin:test/mux_r2_41_7/z0.0
```

read_clock_gating_network_latency_table

```
read_clock_gating_network_latency_table

[-design object]
[-convert_only string]
-filename string
[-view_mapping string]
[-clock_mapping string]
```

Description

Reads previously captured latency information from a file generated by Innovus `capture_clock_gating_network_latency_table` command.

Options and Arguments

`-design` *object*	Specifies the design for which the file is read.
`-filename` *string*	Specifies the file generated by Innovus `capture_clock_gating_network_latency_table` command.
`-convert_only` *string*	Specifies to write `set_clock_gating_network_latency_table` command to this file and not to configure any latency table.
`-view_mapping {`*source_view dest_view ...*`}`	Specifies the list of view mappings.
`-clock_mapping {`*source_clock dest_clock*`}`	Specifies the list of clock mappings.

Related Information

Related Commands:	set_clock_gating_network_latency_table
	reset_clock_gating_network_latency_table
	apply_clock_gating_network_latency_table

remove_case_analysis

`remove_case_analysis`
`{`*pin*`|`*hpin*`|`*port*`}+`

Description

Removes the case constant setting on the specified pins and ports which is set using the `set_case_analysis` command or `timing_case_logic_value` attributes.

Options and Arguments

pin`|`*hpin*`|`*port*	Removes the case constant settings from the specified pins or ports.

Examples

The following command sets and removes case analysis value at port se:

genus:root: <number>> set_case_analysis 1 se

genus:root: <number>> get_db port:top/se .timing_case_logic_value
1

genus:root: <number>> remove_case_analysis se

genus:root: <number>> get_db port:top/se .timing_case_logic_value
no_value

Related Information

Related commands:	set_case_analysis
Related attribute:	timing_case_logic_value

remove_clock_gating_check

```
remove_clock_gating_check
[-setup] [-hold]
[-rise] [-fall]
[-high | -low]
[inst | hinst | pin | hpin]...
```

Description

Removes any clock gating check value defined using the `set_clock_gating_check` command or inferred automatically by Genus.

> To write out this constraint in the output SDC file, you must set: `set write_remove_cgc 1`

Options and Arguments

`-fall`	Removes any clock gating check specified for a falling clock transition.
`-high`	Removes any clock gating check for which the non-controlling value of the clock is high.
`-hold`	Resets clock gating hold values.
	> Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
inst | *hinst* | *pin* | *hpin*	
	Removes the clock gating checks from the specified instances or pins.
`-low`	Removes any clock gating check for which the non-controlling value of the clock is low.
`-rise`	Removes any clock gating check specified for a rising clock transition.
`-setup`	Removes any clock gating check that has a setup value.

Examples

- The following command sets clock gating check at pin u1/A:

  ```
  set_clock_gating_check -setup 0.2 u1/A
  ```

- The following command removes clock gating check at pin u1/A:

  ```
  remove_clock_gating_check u1/A
  ```

remove_clock_latency

```
remove_clock_latency
[-source]
[-clock clock_list]
{clock | port | pin | hpin}...
```

Description

Removes the general and clock-specific latencies from the specified objects.

> Although supported by the `read_sdc` command, it is *not* an official SDC command.

Options and Arguments

`-clock` *clock_list*	Removes only the clock-specific latencies from the specified pins and ports with respect to the specified clocks.
	General latencies are not removed.
clock|*pin*|*hpin*|*port*	
	Specifies the objects (clocks, pins and ports) from which the latencies should be removed.
	> No clocks can be specified in the object list if the `-clock` option is specified.
`-source`	Removes the source latencies from the specified clocks, pins or ports.
	If this option is omitted, the network latencies are removed.

Examples

- The following command removes all the network latencies defined on pin `pin1` that refer to clock `clk1`:

  ```
  genus:root:> remove_clock_latency -clock clk1 pin1
  ```

- The following command removes clock source latency at port c:

  ```
  genus:root:> get_db port:top/c .source_latency_late_rise_max
  10.0

  genus:root:> remove_clock_latency -source c

  genus:root:> get_db port:top/c .source_latency_late_rise_max
  no_value
  ```

Related Information

Related command:	set_clock_latency

remove_clock_transition

```
remove_clock_transition
clock_list {-min | -max} {-rise | -fall}
```

Description

Removes the clock transition (slew) from the specified clocks.

Options and Arguments

clock_list	Removes only the clock skews from the specified clocks.
-fall	Removes the falling transition value.
-max	Removes the maximum slew value.
-min	Removes the minimum slew value.
-rise	Removes the rising transition value.

Related Information

Related command:	set_clock_transition

remove_disable_clock_gating_check

```
remove_disable_clock_gating_check
[-setup] [-hold] [-rise] [-fall] [-high | -low]
{pin | hpin | inst | hinst}...
```

Description

Restores the clock gating checks inferred on the specified pins or the enable pins of the specified instances that were present before applying the `set_disable_clock_gating_check` command.

> This command is supported only on mapped instances and pins of mapped instances. This command is not supported on the clock pins of an instance.

> To write this constraint in the output SDC file, you must set the following variable: `set write_disable_and_rmdisable_cgc 1`

Options and Arguments

`-fall`	Restores any clock gating check specified for a falling clock transition.
`-high`	Restores any clock gating check for which the non-controlling value of the clock is high.
`-hold`	
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
inst|*hinst*	Restores the clock gating check value(s) inferred at every single enable pin of the specified instance.
`-low`	Restores any clock gating check for which the non-controlling value of the clock is low.
pin|*hpin*	Restores the clock gating check value(s) on the specified pins.
`-rise`	Restores any clock gating check specified for a rising clock transition.
`-setup`	
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

Example

The following command removes disable_clock_gating_check at pin u1/A

```
remove_disable_clock_gating_check u1/A
```

Related Information

Related command:	set_disable_clock_gating_check

remove_disable_timing

```
remove_disable_timing
{ [-from pin] [-to pin] {lib_cell | inst| hinst}...
| {lib_pin | pin | hpin | port| hport}... }
```

Description

Restores the timing arcs through the specified objects that were disabled using the `set_disable_timing` command.

Options and Arguments

`-from` *pin*	Restores the timing arcs starting from this pin in the specified lib_cells or instances.
	An error is given when you specify an object type other than a lib_cell or instance with this option.
{*lib_pin* | *pin* | *hpin* | `port` | *hport*}	
	Specifies the lib_pin, pin, port, or hierarchical pin or port at which the timing path(s) must be restored.
{*lib_cell* | *inst* | `hinst`}	
	Specifies the lib_cells or instances for which you want to restore timing arcs. If you specify a lib_cell or instance without a `-from` or `-to` option, the tool restores all timing arcs for that lib_cell or instance.
`-to` *pin*	Restores the timing arcs ending at this pin in the specified lib_cells or instances.
	An error is given when you specify an object type other than a lib_cell or instance with this option.

Examples

- Following example removes disable timing constraint to pin Y of cell `BUFX1`:

  ```
  remove_disable_timing -to Y BUFX1
  ```

- Following example removes disable timing constraint from pin A to pin Y on instance g1295:

  ```
  remove_disable_timing -from A -to Y g1295
  ```

Related Information

Related command:	set_disable_timing

remove_from_collection

```
remove_from_collection
base_collection second_collection [-intersect]
```

Description

Removes any occurrence of an element in the second collection from the base collection and returns the new collection.

If the base and second collections have different objects, the result will be the base collection.

The collections cannot contain a list of objects. If either the base or second collection or both contain a list of objects, an error message will be issued.

Options and Arguments

base_collection	Specifies the base collection.
-intersect	Provides the intersection of base collection elements with second collection of objects or lists.
second_collection	Specifies the collection containing the objects that must be removed from the base collection.

Examples

- The base collection contains cells and ports. The following command indicates to remove the cells from the base collection.

```
genus:root:> remove_from_collection [concat [get_cells *] [get_ports *]] \
==> [get_cells *]

port:test/clk1 port:test/data1 port:test/sel port:test/out1
```

- In the following example, the objects in the base and second collection are different. Consequently, the resulting collection equals the base collection.

```
genus:root:> remove_from_collection [get_cells *] [get_ports *]

hinst:test/U3 hinst:test/U0 hinst:test/U1 hinst:test/U2
```

remove_generated_clock

```
remove_generated_clock
{-all | clock_list}
```

Description

Removes all the generated clock objects created by the `create_generated_clock` command in the design.

Options and Arguments

-all	Removes all generated clock objects.
clock_list	Removes the specified list of generated clocks.
	You can use wildcards to specify a list of clocks.

Examples

- The following command removes the generated clock `gen_clk`.

```
genus:root:> remove_generated_clock [get_db clocks *gen_clk]
```

- The following command removes all generated clocks whose name start with abc.

```
genus:root:> remove_generated_clock [get_db clocks *abc*]
```

Related Information

Related command:	create_generated_clock

remove_ideal_net

```
remove_ideal_net hnet_list
```

Description

Causes the specified nets to be treated as non-ideal and allows the timing and optimization engines to optimize these nets. The command sets the `ideal_driver` attribute on all the leaf-level drivers of the specified nets to `false`.

The `ideal_driver` attribute can be set manually or by using the `set_ideal_net` command.

Options and Arguments

hnet_list	Specifies the nets which must be treated as non-ideal.

Related Information

Related command:	set_ideal_net
Sets this attribute:	ideal_driver

remove_ideal_network

```
remove_ideal_network {pin | hpin | port | hnet}...
```

Description

Sets the `ideal_network` and `ideal_driver` attributes on all the leaf-level drivers of the specified net to `false`. The `ideal_network` attribute can be set manually or by the `set_ideal_network` command.

Options and Arguments

{pin | hpin | port| hnet}	
	Specifies the object to which the command applies.

Related Information

Related command:	set_ideal_network
Sets these attributes:	ideal_driver
	ideal_network

remove_input_delay

```
remove_input_delay
[-clock clock] port
```

Description

Removes the input delay on the specified port related to either the specified clock or all clocks.

> If an input delay is associated with several ports, it will not be removed.

Options and Arguments

`-clock clock`	Removes the input delay relative to the specified clock.
	If this option is omitted, the input delays relative to all clocks on this port will be removed.
	> This command accepts only one clock at a time. Wildcard card and lists are not supported with this switch.
port	Removes the input delay from the specified port.
	> Wild card or port lists are accepted with this switch.

Example

```
genus:root:> remove_input_delay -clock clk1 in1*
```

remove_output_delay

```
remove_output_delay
[-clock clock] port
```

Description

Removes the output delay on the specified port related to either the specified clock or all clocks.

> If an output delay is associated with several ports, it will not be removed.

Options and Arguments

`-clock` *clock*	Removes the output delay relative to the specified clock.
	If this option is omitted, the input delays relative to all clocks on this port will be removed.
	> It accepts only one clock at a time, wildcards and lists are not supported with this option.
port	Removes the output delay from the specified port.
	> Wild card is accepted or a list of ports is also accepted.

Example

```
genus:root:> remove_output_delay -clock clk1 out*
```

report_property

```
report_property
{ collection | list_of_collections }
-property_list property_list
```

Description

Reports all of the properties associated with each object in the specified collection.

For properties which return a populated collection, the command output displays {...}. For empty collections returned, a {} will be displayed.

> If there are different object types in the list, the property names should be common for all the objects specified in the list.

Options and Arguments

collection | *list_of_collections*	
	Specifies the collection, or list of collections for which to report.

	The collection can contain any of the following object types:
	library, lib_cell, lib_pin, lib_arc, design, module, inst, hinst, port, pin, hpin, hnet, clock, or exception.
`-property_list` *property_list*	
	Specifies a list of valid property names.

Example

- The following command shows attributes of the specified properties:

```
report_property [get_pins TU11/A] -property_list {name direction}

report_property {test/g3/* test/clk_50} -property_list {name direction}
in_0 z clk_50
in out in
```

reset_annotated_check

```
reset_annotated_check
{ [-from from_pins] | [-to to_pins]} [-clock {fall|rise} ] [-cond sdf_condition ] {[-rise] | [-fall]} {[-setup] | [-hold] } {[-nochange_high] |
[-nochange_low]} [-all] [inst | hinst]
```

Description

Removes annotated timing check values that you had set by using the set_annotated_check command or by SDF annotation.

Options and Arguments

`-all`	Specifies to reset all annotated timing check values in the design.
`-clock {fall|rise}`	
	Specifies whether the timing check applies to the clock rising or clock falling transition.
	Default: Resets checks for the clock rise and clock fall transitions.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-cond` *sdf_condtion*	
	Specifies to reset all annotated timing check values for which a condition is specified.
	The *sdf_condition* must match the syntax of the condition specified in the library.

	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-fall`	Restricts the reset to only affect the falling edge of the constrained pins.
	Default: `-rise` and `-fall`
`-from` *from_pins*	Specifies a list of leaf cell clock pins as starting points of the timing arcs for which timing checks must be reset. Generally, it specifies the beginning instance pin of the timing arcs for which the annotated check is reset. This is usually the reference (for example, clock pin) of the constraint. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option.
	If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected. You cannot use this parameter with the {*inst* | *hinst*} parameter.
inst | *hinst*	Resets the annotated checks on specified list or collection of cell instances.
`-rise`	Restricts the reset to only affect the rising edge of the constrained pins.
	Default: `-rise` and `-fall`
`-setup|-hold|-recovery|-removal|-nochange_high|-nochange_low`	
	Specifies the timing check type.
	Make sure there is a corresponding timing arc between the *from_pins* and the *to_pins* arguments.
	Genus does not support the `-nochange_high` and `-nochange_low` timing checks.
`-to` *to_pins*	Specifies a list of leaf cell clock pins as end points of the timing arcs for whcih timing checks must be annotated. Generally, it specifies the terminating instance pin of the timing arcs for which the annotated check is reset. This is usually the constrained pin (data pin) of the check. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option.
	If this option is not specified, only timing arcs whose source pins are specified using the -from option are selected. You cannot use this parameter with the {*inst* | *hinst*} parameter.

Examples

- The following commands annotates a `nochange_low` check between data low and clock low. The nochange margin before the clock is 1.3 and the nochange margin after the clock is 2.4:

```
set_annotated_check -nochange_low 1.3 -clock fall -fall -from {I1/CP} -to {I1/EN}

set_annotated_check -nochange_low 2.4 -clock fall -rise -from {I1/CP} -to {I1/EN}
```

- The following command resets all the timing check annotations in the design:

  ```
  reset_annotated_check -all
  ```

- The following command resets all timing check annotations on a particular cell:

  ```
  reset_annotated_check [get_cells BLK/BR1]
  ```

- The following command resets specific checks on a specific instance:

```
reset_annotated_check -setup -hold [get_cells BLK/BR2]
```

- The following command resets checks on a specific timing check arc:

```
reset_annotated_check -from BLK/BR1/CK -to BLK/BR1/D
```

Related Information

Related command:	set_annotated_check

reset_annotated_delay

```
reset_annotated_delay
[-all] [-from from_pins][-to to_pins]
[inst | hinst]
```

Description

Resets annotated delay assertions that have been previously specified using the set_annotated_delay command. When you reset an annotated delay, the system uses the delays which are calculated from the original libs for the arc.

Options and Arguments

`-all`	Specifies to reset all annotated delay assertions in the design.
`-from` *from_pins*	Specifies timing arcs to which the delay is reset. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option.
	If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected.
	You cannot specify hierarchical pins. If specified, an error messages is displayed.
{*inst* | *hinst*}	Resets the annotated delays on specified list or collection of cell instances.
`-to` *to_pins*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their source pins specified using the `-from` option and the sink pins specified using this option.
	If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected.
	You cannot specify hierarchical pins. If specified, an error message is displayed.

Example

The following command shows the usage of `reset_annotated_delay` options:

```
reset_annotated_delay -from {I1/IO}
```

```
reset_annotated_delay -to {I1/IO}

reset_annotated_delay {I1}

reset_annotated_delay 3.0 -all

reset_annotated_delay 3.0 -from A -to Z
```

Related Information

Related commands:	set_annotated_check
	set_annotated_delay

reset_annotated_transition

```
reset_annotated_transition{[-all] | [object_list]}
```

Description

Resets annotated transition assertions that have been previously specified using the set_annotated_transition command. When you reset an annotated transition, the system uses the transitions which are calculated from the original libs for the arc.

Options and Arguments

`-all`	Specifies to reset all annotated transition assertions in the design.
object_list	Resets the annotated delays on specified list of pins, ports or hpins.

Example

The following example shows the usage of `reset_annotated_transition` options:

```
reset_annotated_transition -all

reset_annotated_transition [get_pins A]
```

Related Information

Related commands:	set_annotated_transition

reset_case_analysis

```
reset_case_analysis
{port | pin | hpin}...
```

Description

Resets the case constant setting on the specified pins and ports which is set using the `set_case_analysis` command or `timing_case_logic_value` attributes.

Options and Arguments

port|*pin*|*hpin*	Resets the case constant settings from the specified pins or ports.

Example

The following command sets and resets case analysis value at port `se`:

```
set_case_analysis 1 port:top/se

get_db port:top/se .timing_case_logic_value#Returns a value of 1

reset_case_analysis port:top/se

get_db port:top/se .timing_case_logic_value# Returns no_value
```

Related Information

Related commands:	set_case_analysis
Related attribute:	timing_case_logic_value

reset_clock

```
reset_clock
[-all] | [{clock}] [-quiet]
```

Description

Removes previously created clock assertions.

Options and Arguments

`-all`	Resets all clocks in the design.
clock	Specifies the list of clock objects.

`-quiet`	Suppresses all error and warning messages.

Example

The following command resets all the clock assertions from a design:

```
reset_clock -all
```

Related Information

Related command:	report_clocks

reset_clock_gating_check

```
reset_clock_gating_check
[-setup] [-hold]
[-rise] [-fall]
[-high | -low]
[{inst | hinst | pin | hpin | clock}...]
```

Description

Resets the specified setup and hold values for clock gating checks to the default values.

Options and Arguments

`-high | -low`	Resets the non-controlling value (high or low) of the clock.
`-hold`	Resets clock gating hold values.
inst | hinst | pin | hpin | clock	
	Specifies a list of clocks, instances, or pins for which clock gating checks are to be removed. If this list is not specified, the clock gating check is reset on the current design.
`-rise | -fall`	Resets the specified setup (or hold) value to the rising (or falling) signals on the specified pins.
`-setup`	Resets clock gating setup values.

Examples

- The following command removes the setup value for the clock CLK1:

  ```
  reset_clock_gating_check -setup [{get_clocks CLK1}]
  ```

- The following command removes the hold value for the pin A:

  ```
  reset_clock_gating_check -hold [{get_pins g1/A}]
  ```

Related Information

Related command:	set_clock_gating_check

reset_clock_gating_network_latency_table

```
reset_clock_gating_network_latency_table
     [-design object]
```

Description

Removes all clock gating cell latency values specified by `set_clock_gating_network_latency_table`.

Options and Arguments

`-design` *object*	Specifies the design for which the reset clock gate latencies are applied

Related Information

Related Command
set_clock_gating_network_latency_table
apply_clock_gating_network_latency_table

reset_clock_groups

```
reset_clock_groups
{-physically_exclusive | -logically_exclusive | -asynchronous}
{-name name_list | -all}
```

Description

Resets clock groups set using the set_clock_groups command.

Options and Arguments

`-all`	Removes all clock groups in the design.
`-asynchronous`	Specifies to only remove asynchronous clock groups
`-logically_exclusive`	
	Specifies to only remove logically exclusive clock groups
`-name` *name_list*	Specifies to remove previously defined clock groups.

-physically_exclusive	
	Specifies to only remove physically exclusive clock groups

Examples

- The following command resets all physically exclusive clock groups in the design:

```
reset_clock_groups -physically_exclusive -all
```

- The following command resets the physically exclusive clock group named `CLKGRP0`:

```
reset_clock_groups -physically_exclusive -name CLKGRP0
```

reset_clock_latency

```
reset_clock_latency
[-source]
[-clock clock_list]
{clock | port | pin | hpin}...
```

Description

Resets assertions made by previous `set_clock_latency` commands.

Options and Arguments

-clock *clock_list*	Specifies the list of clocks from which the associated `set_clock_latency` constraints (at the pin level), are to be removed.
clock|*pin*|*hpin*|*port*	
	Specifies a list of pins, ports, or clock objects for which the assertions are to be removed.
	No clocks can be specified in the object list if the `-clock` option is specified.
-source	Removes the source latencies from the specified clocks, pins or ports.

Example

The following command removes the clock latency assertion for the clock object in a design:

```
reset_clock_latency [get_clocks WAVE]
```

Related Information

Related command:	set_clock_latency

reset_clock_sense

```
reset_clock_sense
[ -all
| [-clocks clock_list] {port|pin|hpin}...]
```

Description

Removes the clock sense constraints specified with the `set_clock_sense` command from the specified pins.

Options and Arguments

`-all`	Removes all specified clock sense constraints.
`-clocks clock_list`	
	Specifies the clocks for which the clock sense constraints must be removed.
	If no clock is specified, the command looks at all clocks defined on each of the pins and ports and changes the clock sense as specified.
{port|pin|hpin}	Specifies the name of the pins or ports for which the clock sense constraints must be removed.

Related Information

Related command:	set_clock_sense

reset_clock_transition

```
reset_clock_transition
clock_list
```

Description

Removes the clock transition (slew) assertions set on the specified list of clock waveforms.

Options and Arguments

clock_list	Specifies a list of clock waveforms for which the clock transition assertions are to be removed. The `clock_list` variable can be a Tcl list or collection of clock waveforms.

Example

The following command removes the clock transition assertion for the clock object in a design:

```
reset_clock_transition [get_clocks WAVE]
```

Related Information

Related command:	set_clock_transition

reset_clock_uncertainty

```
reset_clock_uncertainty
[-from_edge string] [-to_edge string]
[-from clock_list | -fall_from clock_list |
  -rise_from clock_list]
[-to clock_list | -fall_to clock_list |
  -rise_to clock_list]
[-hold | -setup] [clock|port|inst|hinst|pin|hpin]...
```

Description

Specifies to remove the assertions that were added with the `set_clock_uncertainty` command.

Options and Arguments

{*clock* | *pin* | *hpin* | *port* }	
	Removes the target uncertainty assertions from the specified list of clocks or pins. You can specify a list of clock names, or clock root pin or port names relative to the current module. You can also specify a collection of clocks, or clock root pins or ports.
	You cannot use this parameter when any of the -from and -to parameter pairs are specified.
-from *clock_list* | -fall_from *clock_list* | -rise_from *clock_list*	
	Removes the assertions for uncertainty value for the specified list of launching clocks.
	You must specify one of these parameters with one of the `-to`, `-rise_to`, or `-fall_to` parameters. You cannot use these parameters when pins are specified.
-from_edge {rise | fall | both}	
	Specifies the edge type of the launch clock.
	This option only applies to inter-clock uncertainties and cannot be specified with either `-fall_from` or `-rise_from`.

`-hold | -setup`	Specifies that the clock uncertainty is removed for hold or setup checks.
	If neither option is specified, the uncertainty value is removed from both checks.
`-to` *clock_list* `| -fall_to` *clock_list* `| -rise_to` *clock_list*	
	Removes the uncertainty value from the specified list of capture clocks.
	These options only apply to inter-clock uncertainties.
`-to_edge {rise | fall | both}`	
	Specifies the edge type of the capture clock.
	This option only applies to inter-clock uncertainties and cannot be specified with either `-fall_to` or `-rise_to`.

Related Information

Related commands:	path_adjust
	set_clock_uncertainty

reset_data_check

```
reset_data_check
[-from {pin|port} | -rise_from {pin|port} | -fall_from {pin|port}] [-to {pin|port} | -rise_to {pin|port} | -fall_to {pin|port}] [-setup | -hold] [-clock clock_object]
```

Description

Resets data-to-data checks from the specified pins or ports which were set using the set_data_check command. Data-to data checks are non-sequential checks that are designed to constrain a data signal with respect to another data signal.

Options and Arguments

`-clock` *clock*	Specifies the name of the clock that triggers the data event on the related (`-from`) pin.
	In the case where there are multiple data events triggered by different clock phases arriving at the `-from` pin, the `-clock` parameter allows you to specify which data event should be used for the "related pin" side of the timing check.
	If this option is not specified, the data check is clock independent.
`-from` {*pin*|*port*} `| -fall_from` {*pin*|*port*} `| -rise_from` {*pin*|*port*}	
	Specifies the pins or ports that are set as related pins for data-to-data checks.
	The pin or port list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. Leaf design cells (instances) can also be specified.

	Use only one -from, -rise_from, or -fall_from parameter per command.
	By default, the -from parameter checks both the rising and the falling edges.
	Using -rise_from parameter checks at the rising edge of the signal on the from pins.
	Using -fall_from parameter checks at the falling edge of the signal on the from pins.
	You cannot use the -from parameter to specify multiple objects.
-setup | -hold	Specifies whether the data check value is for setup or hold check.
	If you do not specify either -setup or -hold option, the value applies to both setup and hold.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The write_sdc command will write the option out for consumption by downstream tools.
-to {*pin*|*port*} |-fall_to {*pin*|*port*} |-rise_to {*pin*|*port*}	
	Specifies the pins or ports that are set as constrained pins for data-to-data checks.
	The pin or port list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. Leaf design cells (instances) can also be specified.
	If you specify the -to option, the data check is performed on the falling and rising edge of specified pin.
	Use only one -to, -rise_to, or -fall_to parameter per command.
	By default, the -to parameter checks both the rising and the falling edges.
	Using -rise_to parameter checks at the rising edge of the signal on the from pins.
	Using -fall_to parameter checks at the falling edge of the signal on the from pins.

Related Information

Related command:	set_time_unit

reset_disable_clock_gating_check

```
reset_disable_clock_gating_check
{pin | hpin | inst | hinst}...
```

Description

Restores the clock gating checks for the specified instances or pins.

Options and Arguments

inst | *hinst* | *pin* | *hpin*

	Specifies the list of disabled pins or instances that are to be enabled.

Example

The following command restores the check at pin u1/A:

```
reset_disable_clock_gating_check u1/A
```

Related Information

Related command:	set_disable_clock_gating_check

reset_disable_timing

```
reset_disable_timing
 [-from pin] [-to pin] {lib_cell | inst | hinst | lib_pin | pin | lib_arc | hpin | port| hport | arc}...  [-quiet]
```

Description

Restores the timing arcs that were disabled using the `set_disable_timing` command.

Options and Arguments

`-from` *pin* `-to` *pin*	Specifies the arc(s) that are to be restored. Specify the `-from` and `-to` options together.
	You can specify the `-from` and `-to` parameters when the `object_list` contains instance or library cell objects. Do not specify the `-from` and `-to` parameters when the `object_list` includes instance pin or library cell pin objects.s
{*lib_cell* | *inst* | *hinst* | *lib_pin* | *pin* | *hpin* | *lib_arc* | *port* | *hport* | *arc*}	
	Specifies the instance, instance pin, arc, library arc, library cell, or library cell pin objects.
	If the object list includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are restored.
	If the object list contains instance or library cell objects, and the `-from` and `-to` parameters are not specified, all arcs found in the specified instance or library cell objects are restored.
`-quiet`	Specifies not to display any error or warning messages.

Example

Following example command disables the timing arcs from CK to Q for instance U8, then resets the assertions using the instance reference:

```
reset_disable_timing u8
```

Related Information

Related commands:	set_disable_timing
	report_timing

reset_drive

```
reset_drive
{port|port_bus}...
```

Description

Resets the drive resistance on the specified input and/or bidirectional ports of the top cell.

The drive resistance models the cell driving the port. This resistance is used to compute the load dependent delay and the slew at the specified ports.

Options and Arguments

{port | port_bus}	Specifies a list of input or bidirectional ports from which the drive resistance is reset.

reset_driving_cell

```
reset_driving_cell
[-rise] [-fall] [-max | -min]
port_list
```

Description

Resets the driving cell assertions applied with the `set_driving_cell` command for the given rise or fall, minimum or maximum combination on ports.

If `-rise`, `-fall`, `-min`, or `-max` parameters are not specified, the `reset_driving_cell` command resets the assertion for all rise or fall, minimum or maximum combinations for which the assertion exists.

Options and Arguments

`-fall`	Specifies the falling drive assertions.
`-max`	Specifies the late drive assertion.
`-min`	Specifies the early drive assertion.
port_list	Specifies the port(s) to which the driving_cell assertions are reset.
`-rise`	Specifies the rising drive assertions.

Example

Following command reset the late drive assertion on the mentioned port:

```
reset_driving_cell -max [get_db ports *port_pad_data_in[5]]
```

Related Information

Related command:	set_driving_cell

reset_generated_clock

```
reset_generated_clock
{-all | clock_list}
```

Description

Removes all the generated clock assertions previously created using the `create_generated_clock` command in the design.

Options and Arguments

`-all`	Removes all generated clock objects.
clock_list	Specifies the list of clocks where the generated clock assertion are to be removed.

Examples

- The following command removes the generated clock assertions on all generated clocks

  ```
  reset_generated_clock -all
  ```

- The following command removes all generated clock assertions named GCLK:

  ```
  reset_generated_clock [get_clocks GCLK]
  ```

Related Information

Related command:	create_generated_clock

reset_ideal_latency

```
reset_ideal_latency [-min | -max]
[-rise | -fall]
pin|port....
```

Description

Specifies to reset the ideal latency set by the set_ideal_latency command.

Options and Arguments

`-fall`	Indicates that the specified value is a falling ideal latency value.

`-max`	Indicates that the specified value is a maximum ideal latency value.
`-min`	Indicates that the specified value is a minimum ideal latency value.
pin|port	Specifies the list of objects - instance pins or ports from where to remove the latency value.
`-rise`	Indicates that the specified value is a rising ideal latency value.

Related Information

Related command:	set_ideal_latency

reset_ideal_network

```
reset_ideal_network {pin | hpin | port | hnet}...
```

Description

Resets the ideal network constraints, specified using the `set_ideal_network` command, on pins, ports, or nets.

> The `reset_ideal_network` command is not view or mode-based it applies to all the active views.

Options and Arguments

{pin | hpin | port| hnet}	
	Specifies a list of objects - instance pins, ports, or nets.

Related Information

Related command:	set_ideal_network

reset_ideal_transition

```
reset_ideal_transition [-min | -max]
[-rise | -fall]
pin|port...
```

Description

Specifies to reset the ideal transition.

Options and Arguments

-fall	Indicates that the specified value is a falling ideal transition value.
-max	Indicates that the specified value is a maximum ideal transition value.
-min	Indicates that the specified value is a minimum ideal transition value.
pin|port	Specifies the list of objects - instance pins or ports from where to remove the transition value.
-rise	Indicates that the specified value is a rising ideal transition value.

reset_input_delay

```
reset_input_delay
[-clock clock]
[-clock_fall]
[-level_sensitive]
[-rise] [-fall] [-max] [-min]
{port}...
```

Description

Resets previously specified input delay assertions.

Options and Arguments

-clock *clock*	Specifies the reference clock.
	Default: The asynchronous (@) clock
	If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the -clock parameter, the constraints specified for a pin with matching -clock parameter are reset. If you do not specify the -clock parameter, all the constraints applied on the specified pin, with or without the -clock parameter specification, are reset.
-clock_fall	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter.
	Default: Removes the delay assertions relative to the rising edge.
-fall	Resets the input delay for the falling edge at the input port.
	If both -rise and -fall options are omitted, the input delay is reset on both the edges.
-level_sensitive	Resets output delay that is launched by a positive level sensitive latch. When you specify the -level_sensitive parameter, the command resets all the set_input_delay constraints specified with the -level_sensitive parameter. If -level_sensitive parameter is not specified, the constraints specified for the same port, with or without the -level_sensitive parameter, are reset.
	Note: You must specify the -clock parameter with this parameter.
-max	Removes the delay assertions that refer to the setup analysis.
	If you do not specify the -max parameter, the input delay assertions for both setup and hold analysis are removed.
-min	Removes the delay assertions that refer to the hold analysis.
	If you do not specify the -min parameter, the input delay assertions for both setup and hold analysis are removed.

{*port*}	Specifies a single or multiple ports for which the input delay is reset. To specify multiple ports, enclose the list with curly braces ({ }) and separate the pin information with white space.
-rise	Resets the input delay for the rising edge at the input port. If both -rise and -fall options are omitted, the input delay is reset on both the edges.

Example

The following command removes the input delay value on the port:

```
reset_input_delay -clock CLK1 -min [get_ports ABC]

reset_input_delay -clock CLK1 -max [get_ports ABC]
```

Related Information

Related commands:	set_input_delay

reset_load

```
reset_load
{port | net | port_bus}...
```

Description

Resets the capacitance from the specified ports and nets in the design.

Options and Arguments

{*port*	*net*	*port_bus*}	
	Specifies the port, net, or port_bus objects from which the capacitance value is reset.

Related Information

Sets these attributes:	external_pin_cap
	external_wire_cap
Related commands:	all_outputs
	report_port
	set_drive
	set_resistance

reset_max_capacitance

```
reset_max_capacitance
{design | port | pin | clock | libpin}...
```

Description

Specifies to reset the maximum capacitance that can be driven by all ports in the specified design(s) or that can be driven by the specified ports.

Resets the `max_capacitance` attribute on the specified design(s) or port(s).

Options and Arguments

clock	Specifies the name of the clock from which the maximum capacitance design rule constraint needs to be removed.
design	Specifies the name of the design from which the maximum capacitance design rule constraint needs to be removed.
libpin	Specifies the name of the library cell pin from which the maximum capacitance design rule constraint needs to be removed.
pin	Specifies the names of the pins from which the maximum capacitance design rule constraint needs to be removed.
port	Specifies the names of the ports from which the maximum capacitance design rule constraint needs to be removed.

reset_max_fanout

```
reset_max_fanout
{design | port | libpin}...
```

Description

Resets the maximum fanout load limit constraints from the specified instance pin, ports of the top cell, the specified modules, or lib pins. When the net name is not specified in the constraint, then the constraints is removed from all the nets in the design.

Resets the `max_fanout` attribute on the specified design(s) or port(s).

Options and Arguments

design	Specifies the name of the design from which the maximum fanout design rule constraint is to be removed.
libpin	Specifies the name of the library cell pin from which the maximum fanout design rule constraint is to be removed.
port	Specifies the names of the input ports from which the maximum fanout design rule constraint is to be removed.

reset_max_time_borrow

```
reset_max_time_borrow
[clock | instance | pin | port ]...
```

Description

Resets the maximum time that can be borrowed by one stage from the next logic stage which was set using the `set_max_time_borrow` command.

Resets the `latch_max_borrow` attribute.

Options and Arguments

{*clock*	*inst*ance	*pin*	*port*}	
	Specifies the object(s) from which the borrow value is reset.

Related Information

Sets this attribute:	latch_max_borrow

reset_max_transition

```
reset_max_transition
[design | clock | port | pin | libpin]...
```

Description

Resets the maximum transition design rule for the specified design or port list.

Options and Arguments

clock	Specifies the name of the clock from which the maximum transition design rule constraint needs to be removed.
design	Specifies the name of the design from which the maximum transition design rule constraint needs to be removed.
libpin	Specifies the name of the library cell pin from which the maximum transition design rule constraint needs to be removed.
pin	Specifies the name of the pin from which the maximum transition design rule constraint needs to be removed.
port	Specifies the names of the ports from which the maximum transition design rule constraint needs to be removed.

Related Information

Sets this attribute:	max_transition

reset_min_capacitance

```
reset_min_capacitance
{design | port | clock | pin | libpin}...
```

Description

Resets the minimum capacitance that can be driven by all ports in the specified design(s) or that can be driven by the specified ports.

Options and Arguments

clock	Specifies the name of the clock from which this design rule constraint needs to be removed.
design	Specifies the name of the design from which this design rule constraint needs to be removed.
libpin	Specifies the name of the library cell pin from which the minimum capacitance design rule constraint needs to be removed.
pin	Specifies the name of the pin from which the minimum capacitance design rule constraint needs to be removed.
port	Specifies the names of the ports from which the minimum capacitance design rule constraint needs to be removed.

reset_min_fanout

```
reset_min_fanout
{design | port | libpin}...
```

Description

Resets the minimum fanout that set on all input ports in the specified design(s) or the specified input ports.

> Genus ignores this command for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the command out for consumption by downstream tools.

Options and Arguments

design	Specifies the name of the design from which the minimum fanout design rule constraint needs to be removed.
libpin	Specifies the name of the library cell pin from which the minimum fanout design rule constraint needs to be removed.
port	Specifies the names of the input ports from which the minimum fanout design rule constraint needs to be removed.

reset_min_pulse_width

```
reset_min_pulse_width
[-low][-high]{pin | hpin | design | port | clock} ...
```

Description

Resets the minimum pulse width checks from the specified objects.

Options and Arguments

`-high`	Resets the maximum pulse width check from high signal levels only.
`-low`	Resets the minimum pulse width check from low signal levels only.
	If you specify neither the `-low` or the `-high` option, the constraint resets from both the low and high pulses of the clock signals
{*design* | *pin*| *port* | *clock* | *hpin*}	
	Specifies the list of pins, ports, design or clock from which the minimum pulse width constraint must be reset.

Related Information

Related commands:	report_min_pulse_width
	report_timing

reset_min_transition

```
reset_min_transition
[design | clock | port | pin | lib_pin]...
```

Description

Specifies to reset the minimum transition design rule for the specified design or port list.

Options and Arguments

clock	Specifies the name of the clock from which the minimum transition design rule constraint needs to be reset.
design	Specifies the name of the design from which the minimum transition design rule constraint needs to be reset.
lib_pin	Specifies the name of the library cell pin from which the minimum transition design rule constraint needs to be reset.
pin	Specifies the names of the pins from which the minimum transition design rule constraint needs to be reset.
port	Specifies the names of the ports from which the minimum transition design rule constraint needs to be reset.

reset_mode

```
reset_mode
[-type cell ]
list_of_modes object_list
```

Description

Resets the Liberty timing library modes for a specified instance.

Library modes are set using the set_mode command. By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive. When you reset the library mode for the instance, all library modes become active again.

Options and Arguments

list_of_modes	Specifies the name of the library modes to reset.
object_list	Specifies the instances for which to reset the library modes.
-type	Specifies the Liberty cell type of the instance.
	Setting the library mode at the instance level is the only form of `set_mode` that is currently supported. This parameter exists to support constraints coming from external sources.

Related Information

Related command:	set_mode

reset_output_delay

```
reset_output_delay
[-clock clock]
[-clock_fall]
[-level_sensitive]
[-rise] [-fall] [-max] [-min]
{port}...
```

Description

Resets previously specified output delay assertions.

Options and Arguments

-clock *clock*	Specifies the reference clock.
	Default: The asynchronous (@) clock.
	If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the -clock parameter, the constraints specified for a pin with matching -clock parameter are reset. If you do not specify the -clock parameter, all the constraints applied on the specified port, with or without the -clock parameter specification, are reset.
-clock_fall	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter.

	Default: Removes the delay assertions relative to the rising edge.
-fall	Resets the output delay for the falling edge at the output port. If both -rise and -fall options are omitted, the output delay is reset on both the edges.
-level_sensitive	Resets output delay that is launched by a positive level sensitive latch. When you specify the -level_sensitive parameter, the command resets all the set_output_delay constraints specified with the -level_sensitive parameter. If -level_sensitive parameter is not specified, the constraints specified for the same port, with or without the -level_sensitive parameter, are reset.
	Note: You must specify the -clock parameter with this parameter.
-max	Removes the delay assertions that refer to the setup analysis.
	If you do not specify the -max parameter, the output delay assertions for both setup and hold analysis are removed.
-min	Removes the delay assertions that refer to the hold analysis.
	If you do not specify the -min parameter, the output delay assertions for both setup and hold analysis are removed.
{*port*}	Specifies a single or multiple ports for which the output delay is reset. To specify multiple ports, enclose the list with curly braces ({ }) and separate the pin information with white space.
-rise	Resets the output delay for the rising edge at the output port. If both -rise and -fall options are omitted, the output delay is reset on both the edges.

Example

The following command removes the output delay value on the port:

```
reset_output_delay -clock CLK1 -min [get_ports ABC]

reset_output_delay -clock CLK1 -max [get_ports ABC]
```

Related Information

Related commands:	set_output_delay

reset_path

```
reset_path
{ -all
| [-from {clock|port|inst|hinst|pin|hpin|port_bus}...]
  [-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...]
  [-to {clock|port|inst|hinst|pin|hpin|port_bus}...]}
[-setup] [-hold] [-type string]
```

Description

Removes the exceptions of the specified type from the specified paths in the design.

The command removes the exceptions of the specified type from either all paths of from the paths selected with the -from, -through, and -to options.

reset_path works only if the expression given with -from, -to or -through exactly matches with the original expression OR if the resulting collection

of paths of the `reset_path` command are a subset of the collection of paths of original exception.

Options and Arguments

`-all`	Removes the specified exception from all paths in the design.
`-from` {*clock* | *port* | *inst* | *hinst* | *pin* | *hpin* | *port_bus*}	
	Selects all paths that start from the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-hold`	Removes only those exceptions that apply to hold analysis.
`-setup`	Removes only those exceptions that apply to setup analysis.
`-through` {*port* | *inst* | *hinst* | *pin* | *hpin* | *hnet* | *port_bus hpin_bus*}	
	Selects all paths that pass through the specified points. Specify a Tcl list.
`-to` {*clock* | *port* | *inst* | *hinst* | *pin* | *hpin* | *port_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-type` *string*	Specifies the type of exception to be removed. You can specify one of the following values: `false_path`, `multicycle` or `path_delay`.

Examples

- The following scenario is supported by Genus:

```
set_max_delay -from [get_clocks {clk}] -to [get_pins {current_state_reg[0]/clk}] 5

reset_path -from [get_clocks {clk}] -to [get_pins {current_state_reg[0]/clk}]
```

- The following scenario is not supported by Genus:

```
set_max_delay -from [get_clocks {clk}] 5

reset_path -from [get_clocks {clk}] -to [get_pins {current_state_reg[0]/clk}]
```

reset_path_group

```
reset_path_group
[-all] [-name group_name]
```

Description

Removes the exceptions from the specified path group or completely removes all the groups in the design.

Options and Arguments

`-all`	Removes all path groups.

`-name group_name`	Removes only the named path group completely. This option cannot be used with the `-all` option.

Examples

- The following command removes the group named GRPA:

  ```
  reset_path_group -name GRPA
  ```

- The following command removes all the path groups

  ```
  reset_path_group -all
  ```

Related Information

Related commands:	group_path

reset_path_group_options

```
reset_path_group_options
[-early]
[-late]string
[-effort_level]
[-skewing_slack_constraint]
[-slack_adjustment]
[-slack_adjustment_priority]
[-target_slack]
[-view analysis_view]
```

Description

Specifies options to guide timing optimization and timing analysis to reset `path_group_options`.

Options and Arguments

`-early`	Specifies the early mode to reset.
-late	Specifies the late mode to reset.
string	Specifies the name of the path group defined by the `group_path`.
-effort_level	Resets the `effort_level`.
-skewing_slack_constraint	
	Resets the `slewing_slack_constraint`.
-slack_adjustment	Resets the `slack_adjustment`.
-slack_adjustment_priority	
	Resets the `slack_adjustment_priority`.
-target_slack	Resets the `target_slack`.

-view *analysis_view*	Specifies the view names. This parameter is used when you want to specify different effort levels of the path group for different views.

Examples

- Reset reg2reg `effort_level`

  ```
  reset_path_group_options reg2reg -effort_level -view analysis_view:Top/setup_analysis_view
  ```

- Reset reg2reg path group for all views

  ```
  reset_path_group_options reg2reg
  ```

reset_resistance

```
reset_resistance
net [net]...
```

Description

Resets the wire resistance from the specified nets in the design.

Options and Arguments

net	Specifies a list of nets from which the wire resistance must be removed.

Related Information

Sets this attribute:	resistance

reset_sense

```
reset_sense
{[-all] | [-clocks clock_list]}
{port|pin|hpin}...]
```

Description

Resets clock sense constraint(s) specified using the `set_sense` command.

Options and Arguments

`-all`	Removes all specified `clock_sense` constraints.
`-clocks` *clock_list*	

	Specifies the list of clocks for which the clock sense constraints must be removed.
	The clocks must have previously been defined using the `set_sense -clock` command. If no clock is specified, the command looks at all clocks defined on each of the pins and ports and changes the clock sense as specified.
{port|pin|hpin}	Specifies the list of cell output pins or ports for which the clock sense constraints must be removed.

Related Information

Related command:	set_sense

reset_timing_derate

```
reset_timing_derate
[-delay_corner delay_corner] [-transition]
{lib_cell | inst | hnet | hinst | design}...
```

Description

This command resets the originally applied derating factors specified on a design or a list of instances (cells, net, or library cells) to the value 1.0. All the derating factors that are set either globally or on specific instances are reset to a default value of 1.0.

Options and Arguments

`-delay_corner` *delay_corner*	
	Removes the derating factors from this delay corner.
design	Specifies the design from which the derating factors must be removed.
inst | *hinst*	Specifies the instance(s) from which the derating factors must be removed.
lib_cell	Specifies the library cells from which the derating factors must be removed.
hnet	Specifies the net name from which the derating factors must be removed.
`-transition`	Resets the transition derate.

Examples

- The following example shows how to reset the timing derate on a library cell:

  ```
  reset_timing_derate [get_db lib_cells *TCHDFFX5]
  ```

- The following example shows how to check whether there are any of the pre-specified derate values:

  ```
  report_timing -user_derate
  ```

  If no derates are specified, then the default value is 1.0.

Related Information

Related command:	set_timing_derate

reset_wire_load_mode

```
reset_wire_load_mode
```

Description

Resets the `wireload_mode` root attribute value.

Related Information

Sets this attribute:	wireload_mode

reset_wire_load_model

```
reset_wire_load_model
{design | subdesign | instance | port}...
```

Description

Forces Genus to use the default wire load model for the given design, subdesign, hierarchical instance or port.

Resets the `force_wireload` attribute if the specified object is a design, subdesign or a hierarchical instance. If the specified object is a hierarchical instance, the attribute on the corresponding subdesign is reset.

Resets the `external_wireload_model` attribute if the specified object is a port.

Options and Arguments

*design	subdesign	instance| port*	
	Specifies the design, subdesign, hierarchical instance, or port for which you want to reset the wire load model.
	If no object is specified, the wire load model is reset from the top-level design.

Related Information

Sets these attributes:	force_wireload
	external_wireload_model
Related attributes:	wireload_mode
	wireload_selection

reset_wire_load_selection_group

```
reset_wire_load_selection_group
[-object_list string]
```

Description

Specifies a wire-load selection group.Resets the `wireload_selection` root attribute for the specified object.

Options and Arguments

`-object_list`	Specifies the object list from which to reset the selection table.

Related Information

Related attributes:	force_wireload
	wireload_mode
	wireload_selection

set_active_clocks

```
set_active_clocks
clock...
```

Description

Specifies the clocks that are made active, while others remain inactive. Any clock created after using the `set_active_clocks` command is active by default.

Options and Arguments

clock	Specifies the list of active clocks

Example

- The following command makes the specified clocks active while others remain inactive - only CLK1 is active:

  ```
  set_active_clocks clock:top/m1/CLK1
  ```

set_annotated_check

```
set_annotated_check
-from from_pins
-to to_pins
check_value[-clock {fall|rise}] [-cond sdf_condition] {-setup | -hold | -recovery | -removal | -nochange_high | -nochange_low} [-rise] [-fall]
[-min] [-max] [-incremental]
```

Description

Annotates the setup, hold, recovery, or removal timing check value between two or more pins on a cell in the current design.

Options and Arguments

check_value	Specifies timing check value.
-clock {fall|rise}	Specifies whether the timing check applies to the clock rising or clock falling transition.
	Default: Sets checks for the clock rise and clock fall transitions.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The write_sdc command will write the option out for consumption by downstream tools.
-cond *sdf_condtion*	Specifies the condition only if the library has a condition attached to the specified timing check arc.
	The *sdf_condition* must match the syntax of the condition specified in the library.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The write_sdc command will write the option out for consumption by downstream tools.
-fall	Specifies that the timing check applies to a data falling transition.
	Default: -rise and -fall
-from *from_pins*	Specifies a list of leaf cell clock pins as starting points of the timing arcs for which timing checks must be annotated.
-incremental	Sets the incremental checks.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The write_sdc command will write the option out for consumption by downstream tools.
-max	Specifies the maximum timing check for both data rise and data fall transitions.
	Use this option only if the design uses minimum and maximum (min_max) operating conditions.

`-min`	Specifies the minimum timing check for both data rise and data fall transitions.
	Use this option only if the design uses minimum and maximum (min_max) operating conditions.
`-rise`	Specifies that the timing check applies to a data rising transition.
	Default: -rise and -fall
`-setup|-hold|-recovery|-removal|-nochange_high|-nochange_low`	
	Specifies the timing check type.
	Make sure there is a corresponding timing arc between the *`from_pins`* and the *`to_pins`* arguments.
	Genus does not support the `-nochange_high` and `-nochange_low` timing checks.
`-to` *`to_pins`*	Specifies a list of leaf cell clock pins as end points of the timing arcs for which timing checks must be annotated.

Example

The following command annotates a 2.0 setup time between the CP clock pin and the A data pin of the instancename instance:

```
set_annotated_check -setup 2.0 -from instancename/CP -to instancename/A
```

set_annotated_delay

```
set_annotated_delay
{-net [-delta_only] | -cell [-cond sdf_condition]}
[-from from_pins][-to to_pins]
delay_value
[-fall] [-rise] [-min | -max]
[-increment]
```

Description

Annotates delay to timing arcs. If you specify the set_annotated_delay command without any parameters, the delay value specified is assumed to include both the base delay and any delta (cross-talk) delay.

Options and Arguments

`-cond` *`sdf_condition`*	
	Specifies the sdf condition of the selected timing arcs.
	Default: The delay applies to all the timing arcs selected using the `-from` and the `-to` options. The expression is ignored for net arcs.
	You cannot specify this option with the -net option.

	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
delay_value	Specifies the delay value to be annotated to the selected timing arcs.
`-delta_only`	Populates the incremental delay slot with the delay value, so that you can differentiate between the base net delay and a delay offset that is due to crosstalk. The tool reports the incremental (or delta) delay value separately in the timing report using `report_timing -output_format incr_delay`, and maintains the separate value for CPPR calculations.
	If the base delay was previously calculated or annotated from the SDF, the -`delta_only` parameter adds the specified delay to the existing base delay.
	This parameter only can be used with the -`net` parameter; you cannot specify it with the -`cell` parameter.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-fall`	Specifies that the delay applies to a falling transition.
	Default: `-rise` and `-fall` - The delay applies to both transitions.
`-from` *from_pins*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option.
	If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected.
	You cannot specify hierarchical pins. If specified, an error message is displayed.
`-increment`	Increments the specified delay value on top of the existing base delay value.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-max`	Specifies the annotated delay for a particular operating corner. You can apply this option for the maximum operating corner.
`-min`	Specifies the annotated delay for a particular operating corner. You can apply this option for the minimum operating corner.
`-net | -cell`	Specifies whether the delay should be annotated to cell arcs or net arcs.
`-rise`	Specifies that the delay applies to a rising transition.
	Default: `-rise` and `-fall` - The delay applies to both transitions.
`-to` *to_pins*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their source pins specified using the `-from` option and the sink pins specified using this option.
	If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected.
	You cannot specify hierarchical pins. If specified, an error message is displayed.

	If -from and -to options are specified together with the -cell option, then the specified pins must belong to the same leaf instance.
	If -from and -to options are specified together with the -net option, then the specified pins must be connected to the same net.
	This command is not supported on unmapped gates, that is, when the -cell option is specified, the instance must be mapped and when the -net option is specified driving pin and load pin must belong to mapped instance.

Examples

- The following example shows the usage of -cell and -net options:

```
set_annotated_delay 3.0 -cell -from {I1/IO}
```

- Use the -net arc option to select net arcs narc1 and narc2 only:

```
set_annotated_delay 3.0 -net -from {I1/IO}
```

- The following command annotates a delay of 3.0 to arcs arc1, arc2, and arc3 for the rise and fall transition at the pin I1/Z, for the two corners {min to max}:

```
set_annotated_delay 3.0 -from A -to Z -cell
```

- The following command annotates a delay of 2.5 to net arc arc4 for both the rising and the falling transition and for the two corners {min max}:

```
set_annotated_delay 2.5 -from I1/Z -to I2/A -net
```

- The following command annotates a delay of 1.5 to arc2 for the rising transition and the min corner:

```
set_annotated_delay 1.5 -from I1/A -to I1/Z -rise -min -cond  "B & C" -cell
```

- The following commands annotate a delay of 3.0 to arc1 and arc2, and a delay of 2.4 to the arc3:

```
set_annotated_delay 3.0 -from I1/A -to I1/Z -cell
set_annotated_delay 2.4 -from I1/A -to I1/Z -cond {!B | !C} -cell
```

- The following commands first annotate a delay of 3.0 to arc1, arc2 and arc3, then overwrite the value set by the first set_annotated_delay command:

```
set_annotated_delay  2.4  -from I1/A  -to  I1/Z  -cond {!B | !C} -cell
set_annotated_delay 3.0 -from I1/A -to I1/Z -cell
```

Related Information

Related command:	set_annotated_check

set_annotated_transition

```
set_annotated_transition
[-rise] [-fall] [-min] [-max]
slew {object_list}
```

Description

Specifies the transition time to be annotated on the specified pins or ports. This transition time overrides any transition time calculated by Genus.

By default, Genus considers the specified slew as the maximum transition value.

Options and Arguments

`-fall`	Specifies that the transition value is for a falling transition time.
	By default, the transition value is set for both falling and rising transition times.
`-max`	Specifies that the transition value is specified for a maximum transition time.
	The maximum values are used for timing analysis.
`-min`	Specifies that the transition value is specified for a minimum transition time.
	The minimum values are not used for timing analysis, but they will be written out when you execute the `write_sdc` command.
`object_list`	Specifies the pins, hpins or ports to be annotated.
`-rise`	Specifies that the transition value is for a rising transition time.
	By default, the transition value is set for both falling and rising transition times.
`slew`	Specifies the transition value to be annotated to the specified pins and ports.

Example

The following command sets the falling and rising (maximum) transition time of port A to 0.6.

```
set_annotated_transition 0.6 [get_ports A]
```

set_case_analysis

```
set_case_analysis
string {port | pin | hpin}...[-comment string]
```

Description

Sets constant or transitional values to a list of pins or ports for use by the timing engine. Case analysis can be used to specify a particular mode in design without structurally changing the netlist. If case analysis is specified as a constant value, the value is propagated forward through the network.For example, if one of the inputs of an AND gate is specified as a constant logic 0, that value is propagated to the AND output, which becomes a constant logic 0 value. This value 0 gets propagated to the inputs of all cells which are driven by this signal, and so on.

When single mode timing analysis is performed, this command sets the `timing_case_logic_value` attribute of the specified pin or port to the specified logic value.

When performing multi-mode timing analysis, this command *updates* the `timing_case_logic_value_by_mode` attribute of the specified pin or port with the specified logic value for the current mode. If the attribute value already contained a value for the specified mode, it will overwrite the existing value. If the attribute value did not yet contain a value for the specified mode, it will add the mode name and the corresponding logic value.

- When you use this constraint in an SDC file, the constraint will be applied to the mode specified with the `-mode` option of the read_sdc command that reads in the file.
- When you want to use this constraint at the Genus prompt, make sure that the mode to which this constraint needs to be applied was previously specified using the set_interactive_constraint_modes command.

Options and Arguments

`-comment` *string*	Specifies a comment to be tagged to this command.
{*port*|*pin*|*hpin*}	Specifies the list of pins or ports to which the logic value or transition value applies. Specify multiple pins by enclosing them with curly braces {}and separating the pin names with white space. The `pin_or_port_list` argument can be a collection.
string	Applies any of the following values to the specified pin or port.
	{0 | 1 | zero | one} — A logic value of 0 or 1 is propagated through combinational logic to disable or enable parts of the logic.
	{rising | falling | rise | fall} — Specifies a valid transition value to be considered by the timing engine. For example, if a rising value is specified at a certain pin or port, only the rising signal transition from the pin or port is considered for timing analysis and the opposite transition type (falling) is ignored.

Examples

- The following command sets the value of pin i4/A to 1 for use by the timing engine:

  ```
  genus:root:> set_case_analysis 1 i4/A
  ```

- The following example shows that the pins `P1/P2/B` and `P1/P3/CI` are considered only for a rising transition. The falling transition on these pins is ignored:

  ```
  genus:root:> set_case_analysis rising {P1/P2/B P1/P3/CI}
  ```

- When you want to use this constraint at the Genus prompt, specify the mode using the command below:

  ```
  genus:root:> set_interactive_constraint_modes -mode_func

  genus:root:> set_case_analysis 0 [get_pins A/B/Q]
  ```

Related Information

Related attribute:	timing_case_logic_value

set_clock_exclusivity

```
set_clock_exclusivity_wrapper_new
[-group string]+
[-exclude_opposite_polarity]
[-exclude_same_polarity]
[-exclude_related_input_pins]
[pin+]
```

Description

Controls filtering of mutually-exclusive clocks at strategic points in the design, for example, the output of clock multiplexers.

Options and Arguments

`-group` *string*	Specifies the clock group.
	Usage: `-group` *clock_list_1* `-group` *clock_list_2*
	At least one `-group` is required .
`-pin {`*pin*`|`*hpin*`}`	Specifies the pin or hpin name.
`-exclude_opposite_polarity`	
	Isolates positive and negative clock phases.
`-exclude_same_polarity`	
	Compares clocks from the same input on the cell.
`-exclude_related_input_pins`	
	Excludes clocks from the different input on the cell.

Example

- Usage example:

```
set_clock_exclusivity -group {clock:top/m2/CLK1 clock:top/m1/CLK2} \
-group {clock:top/m3/CLK3 clock:top/m4/CLK4} -pin pin:top/U1/Y
```

set_clock_gating_check

```
set_clock_gating_check
[-setup float] [-hold float]
[-rise | -fall]
[-high] [-low]
[design | clock | inst| pin ]...
```

Description

Performs a setup check with respect to the edge of the clock signal that changes the state of the clock pin from controlling to non-controlling.

For AND and NAND gates, the setup check is performed with respect to the rising edge of the clock input. For OR and NOR gates, the setup check is performed with respect to the falling edge of the clock input.

If several clock gating checks are specified, the clock gating check specified on a pin has a higher priority then the clock gating check specified on an instance, followed by the clock gating check specified on a clock, a module, the design.

> This command is not supported on the clock pins of an instance.

> The clock gating constraint is not propagated along the clock tree.

The setup check is performed with respect to the edge of the clock signal that changes the state of the clock pin from controlling to non-controlling. For example, for AND and NAND gates, the setup check is performed with respect to the rising edge of the clock input. For OR and NOR gates, the setup check is performed with respect to the falling edge of the clock input.

The hold check is performed with respect to the edge of the clock signal that changes the state of the clock pin from non-controlling to controlling.

For example, for AND and NAND gates, the hold check is performed with respect to the falling edge of the clock input. For OR and NOR gates, the hold check is performed with respect to the rising edge of the clock input.

If the gate logic in unknown, setup and hold checks are performed with respect to both the rising edge and the falling edge of the clock signal and the timing analyzer uses the worst case values.

Timing analysis uses the following priority rules to determine the setup (or hold) value to use for a particular clock gating check:

- Use the clock gating check assertion on the data pin if it exists.
- Otherwise, use the clock gating check assertion on the clock pin if it exists.
- Else, use the clock gating check assertion on the clock waveform for the clock pin if it exists.
- Otherwise, use the clock gating check assertion, if it exists.
- If no clock gating check assertion is present, use the default setup (or hold) value of 0.0.

> This command has no effect on a clock gating check defined as a non-sequential setup or hold check in the library and reported as `ClockGatingSetup` or `ClockGatingHold`.

Options and Arguments

`[-fall|-rise]`	Applies the specified setup (or hold) value to the rising or falling signals on the specified pins. If the clock is specified or the object_list variable is not specified, the value applies to rising or falling, setup or hold.
	If neither option is specified, the clock gating check is applied to both the rising and falling transitions.
`[-high|-low]`	Specifies the non-controlling value of the clock is high or low.
	Timing analysis performs the check on the non-controlling value of the clock.
	By default, the software determines the non-controlling value of the clock using information from the logic of the gate.
	For OR gates the non-controlling value is low. For complex gates, such as XOR and MUX, the clock is never controlling and it does not perform checks unless you specify either the `-high` or `-low` option. The specified value takes precedence over the default values. This option can only be used with pins or instances.
`-hold` *float*	Specifies the hold value.
	Default: 0.0
	Because the tool does not perform any minimum analysis, Genus just stores this value and writes it out when the `write_sdc` command is given.
design | *clock* | *inst* | *pin*	
	Specifies the design, module, clock, instance, or pin for which you want to set the clock gating check.
	If a clock is specified, the check applies to all the clock gating gates driven by this clock.
	If an instance is specified, the check applies to all input pins of the instance.
	If no object is specified, the clock gating check is applied to the current design.
`-setup` *float*	Specifies the setup value.

	Default: `0.0`

Example

```
set_clock_gating_check -setup 0.44 [{get_clocks CLK1}]

set_clock_gating_check -hold 0.11 [{get_pins g1/A}]
```

set_clock_gating_network_latency_table

```
set_clock_gating_network_latency_table
[-design object]
[-clock string]
[-view string]
-level integer
-latency_list string
-skip_latency_range_checks
```

Description

Constructs a table of clock network latency values indexed by clock gating level and fanout counts.

This command performs the configuration only. The specified latency values will be applied automatically during clock gate insertion or other clock gate related operations, or they can be applied directly using `apply_clock_gating_latency`.

Options and Arguments

`-design` *object*	Specifies the top level design for which the latencies need to be set.
`-clock` *string*	Specifies a list of patterns for the clock object(s) for which the latencies need to be set. If unspecified (default), the setting will be applied to all clocks.
`-view` *string*	Specifies a list of patterns for the analysis view(s) for which the latencies need to be set. If unspecified (default), the setting will be applied to all analysis views.
`-level` *integer*	Specifies the clock gating level for which the clock gate latencies need to be set. Clock gating level numbers are counted starting with 0 at leaf level clock gates, working back towards the clock source(s) traversing all combinational arcs.
`-latency_list` *string*	Specifies the list of pairs of fanout count range and latencies values to set the specified level. • The fanout count range is in the form of 'min-max', where min, max are both integers (unless max is 'inf'), and min >= 1, min < max. • The ranges must be in order, non-overlapping, and together cover 1-inf. • The latency values should be non-increasing as a function of stage. • The latency values are the arrival time of the clock at the CG clock input pin to use in ideal timing, which will typically be negative as the flops will have a clock arrival time of 0.
`-skip_latency_range_checks`	Skips checking the latency value for fanout ranges.

Related Information

Related Commands:	apply_clock_gating_network_latency_table
	reset_clock_gating_network_latency_table

set_clock_groups

```
set_clock_groups
-name string
[ -asynchronous [-allow_paths]
| -logically_exclusive | -physically_exclusive ]
-group clock_list [-group clock_list]... [-comment string]
```

Description

Specifies the groups of clocks between which timing analysis will not be done (does a `set_false_path` between the groups of clocks).

> This command was added in SDC 1.7.

Options and Arguments

`-allow_paths`	Allows paths between asynchronous clock groups to be traced in timing analysis. These paths are blocked by default. When you use this option, the software still considers the paths to be asynchronous for signal integrity considerations. You can only use this option with the `-asynchronous` option.
	This option is not supported for signal integrity analysis.
	> Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-asynchronous`	Defines clock group with asynchronous clocks. Asynchronous clocks have no specified phase relationship. The tool considers asynchronous nets as false paths for timing analysis. Use the `-allow_paths` option to trace these paths in timing analysis.
	This option is not supported for signal integrity analysis.
`-comment string`	Specifies a comment to be tagged to this command.
`-group clock_list`	Specifies a group of clocks. Specify at least one group.
	> A clock can belong to one group only.
	> If the number of clocks in a group equals the number of clocks in the design, no false paths are applied.

`-logically_exclusive`	
	Defines clock group with clocks that do not have any functional paths between them, but coexist in the design at the same time. The tool considers paths between these clock groups as false paths for timing analysis.
	This option is not supported for signal integrity analysis.
	Physical flows will honor it as part of Innovus initial placement. Genus does not support the signal integrity analysis implications of this command; however, for timing analysis, the timing paths between the clock groups are considered false paths. The `write_sdc` and `write_snapshot` commands will write the option out for consumption by downstream tools.
`-name` *string*	Specifies the name of the clock group.
`-physically_exclusive`	
	Defines clock group with clocks that cannot exist in the design at the same time. The tool does considers these clock groups as false paths for timing analysis.
	Genus does not support the signal integrity analysis implications of this command; however, for timing analysis, the timing paths between the clock groups are considered false paths. The `write_sdc` and `write_snapshot` commands will write the option out for consumption by downstream tools.
	If Physical aware flow is used in Genus, which calls Innovus during `syn_opt` stage, then the SDC written out for Innovus will have this option written out in SDC for Innovus to implement accordingly. Such flows could be full Physical flow, Spatial High Effort Placement (SHEP) flow or iSpatial flow.

Examples

- The following command sets a false path between clock1 and clock2, and clock1 and clock3.

  ```
  set_clock_groups -name cg_group1 -logically_exclusive -group {clock1} \
  -group {clock2 clock3}
  ```

  When executing the `write_sdc` command, the `set_clock_groups` command in the input SDC file is written out verbatim. The false path exception created by the `set_clock_groups` command is not written out.

- The following command sets a false path between clock1 and clock2, and clock1 and clock3.

  ```
  set_clock_groups -name cg_group1 -logically_exclusive -group {clock1} \
  -group {clock2 {clock3}}
  ```

- The following command results in an error because clock `clk1` is defined in two groups.

  ```
  genus:root:> set_clock_groups -group {clk1 clk2} -group {clk1 clk3}
  Error   : Invalid SDC command option combination. [SDC-204] [set_clock_groups]
          : clk1 can be specifed as part of only one group in 'set_clock_groups'
          : This option is not valid for the indicated SDC command. Check the SDC command and contact Cadence customer support
  if you believe this option combination should be supported.
  ```

- The following command will not apply any false paths constraints assuming that the designs has only those three clocks.

  ```
  set_clock_groups -group { clk1 clk2 clk3} -comment "comment for clock_group"
  ```

Related Information

Related attribute:	comment

set_clock_latency

```
set_clock_latency
     [-min ]
     [-max]
     [-rise ]
     [-fall]
     [-jitter float] [-early]
     [-late]
     [-stddev]
     [-dynamic float]
     [-source]
     [-clock clock_list]
     [-clock_gate] latency obj_list
```

Description

Specifies a clock source or network, early or late latency value of one or more clocks that propagate to the specified pin or port.

When you specify this command, it sets a pin, port or clock latency-related attribute.

- Clock network latency is the time taken by clock signal from definition point to sequential logic clock pin.
- Clock source latency is the source latency defined as time taken by a clock signal from clock source (PLL) to clock definition point

Clock value has two components - source value and network value.

Source value:

- Specified using the `-source` parameter and is the delay from the external clock generator to the clock port of the design.
- Applied when the clock is either ideal or propagated.
- `set_clock_latency` set on a clock port has higher precedence than `set_clock_latency` set on a clock waveform.
- Specified on clock waveforms with both the `set_input_delay` command and the `set_output_delay` command.

Network value (clock tree delay):

- Default value type and is the delay from the clock port to the register.
- Applied only when the clock is ideal. When the clock is propagated, this delay is replaced by the actual clock tree delays.
- Specified on a clock waveform or a clock port is ignored if the clock is propagated.
- `set_clock_latency` set on a timing pin has higher precedence than `set_clock_latency` set on a waveform.
- If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.
- Specified on internal circuit pins or hierarchical ports and used in the fanout cone of the network value pin, when a clock is ideal.

Use the `set_clock_latency` command in two ways:

- To specify clock value for a clock waveform:
  - Use the following syntax for source value associated with a clock signal: `set_clock_latency [-source] [-early|-late] [-min][-max] \ value obj_list`
  - Use the following syntax for network value associated with a clock signal: `set_clock_latency [-min] [-max] value obj_list`

- You cannot specify the -late and -early parameters for network value. Specifying value on a port or pin overrides any previous value specified on the port pin or waveform.
- The *obj_list* argument is the list of the clock waveform names as specified by the create_clock command.

Options and Arguments

{*clock*	*pin*	*hpin*	*port*}	
	Specifies a list of clock waveform names or a list of pins to associate with the clock value. The pin can be a port or an instance pin.
-clock *clock_list*	Associates clock value constraints, placed at the pin level, with a specific clock. This makes it possible to specify a different value per clock.
	If no clocks are specified, the latency value applies to all clocks that propagate to the specified pin or port.
	You cannot use the -clock parameter with clock waveform objects.
-clock_gate	Allows setting the local clock value on pin or port objects.
	This setting overwrites the existing value of clock gating cells and is used when performing timing checks against the specified pins or ports.
-dynamic *float*	Specifies the dynamic value. This is a floating number.
-early | late	Specifies clock arrival time with respect to the early or the late time of the clock signal.
	In setup analysis, launch path is the late path and capture path is the early path
	In hold analysis, launch path is the early path and the capture path is the late path.
	If neither parameter is specified, the default is both early and late. Use these parameters only with the -source parameter.
	You cannot use these parameters with network value. Specify the clock value at the given operating corner.
	The -min and -max parameters refer to the operating corner and the -early and -late parameters refer to the clock arrival time at the source. Use the -early and -late parameters with the -source parameter to specify the four different clock latencies
-fall	Indicates that the specified latency value applies to a fall edge.
-jitter *float*	Specifies the jitter value. This is a floating number.
latency	Specifies the latency value. This is a floating number.
-max	Specifies the clock value for a particular operating corner. Apply the -max parameter to the maximum operating corner.
-min	Specifies the clock value for a particular operating corner. Apply the -min parameter to the minimum operating corner.
-rise | fall	Specifies the -rise or -fall parameter to the waveform at register clock pins for ideal value and to the waveform at the source for source value. Ideal network value does not change polarity when crossing negative unate arcs.

`-source`	Specifies a source value.
	If this option is not specified, the specified latency value is considered to be a network latency.
`-stddev`	Specifies the sigma value.

Examples

- An example usage:

```
set_clock_latency -clock_gate -0.027 \
[get_pins {top/u_exfifo_artemis_cpu__RC_CG_HIER_INST34/RC_CGIC_INST/CK}] -clock [get_clocks {clk}]
```

- The following set of commands enable you to create a clock using the `create_clock` command and apply value to the created clock using the `set_clock_latency` command:

```
create_clock CLK2 -period 20 [get_ports {clkA clkB clkD}]

set_clock_latency -source 2 {clkB clkD} # Define port settings

set_clock_latency -rise 5 {clkB clkD} # Define port settings

set_clock_latency -early -late -source CLK1 # Define waveform settings

set_clock_latency  3  -early  -late -source CLK1 # Define waveform settings
```

Related Information

Related command:	remove_clock_latency
Related attributes:	source_latency_early_fall_max
	source_latency_early_fall_min
	source_latency_early_rise_max
	source_latency_early_rise_min
	use_multi_clks_latency_uncertainty_optimize
	use_multi_clks_latency_uncertainty_report

set_clock_sense

```
set_clock_sense
[-clocks clock_list] [-type string]
[ -positive | -negative | -logical_stop_propagation | -stop_propagation]
{pin|hpin|port}...
```

Description

Specifies clock phase or polarity with respect to source clock from specified pin (forward direction only). Selects which phase of the clock to filter at the specified point.

.This command was added in SDC 1.7.

Options and Arguments

`-clocks` *clock_list*	
	Specifies the list of clocks passing through the specified point to which the constraint applies.
	If no clock is specified, the command looks at all clocks defined on each of the pins and ports and changes the clock sense as specified.
`-logical_stop_propagation`	
	Stops the propagation of the specified clocks through the specified pins.
`-negative`	Specifies that the specified pins and ports are inverted sources of the clock.
	If the pin or port is a non inverting source of the clock, the clock sense on the port is changed and it becomes an inverting source of the clock.
{*pin*|*hpin*|*port*}	Specifies the list or collection of cell output pins. The unateness specified by the `set_clock_sense` command is applied to clocks propagating through the specified point.
`-positive`	Specifies that the specified pins and ports are sources of the clock.
	If the pin or port is an inverting source of the clock, the clock sense on the port is changed and it becomes a non-inverting source of the clock.
`-stop_propagation`	Stops the propagation of the specified clocks and clock source data paths through the specified pins.
`-type` *string*	Specifies to apply clock sense to any of the following objects: `all`, `clock`, `data`, `clock_source_data`

Examples

- The following command stops the propagation of clock CLK1 through the pin `CLKBUF1/Z`:

  ```
  set_clock_sense -logical_stop_propagation -clocks CLK1 CLKBUF1/Z
  ```

- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:

  ```
  set_clock_sense -positive -clocks CLK1 CLKBUF1/Z
  ```

- The following command stops clock phases starting from pin1:

  ```
  set_clock_sense pin1 -logical_stop_propagation
  ```

Related Information

Sets these attributes:	clock_sense_negative
	clock_sense_positive
	clock_sense_stop_propagation
	propagated_clocks

set_clock_skew

```
set_clock_skew
[-minus_uncertainty] [-plus_uncertainty]
float {clock | port}...
```

Description

Sets the skew on the specified clocks and ports.

Skew is internally modeled as clock uncertainty. If neither `-minus_uncertainty` or `-plus_uncertainty` are specified, the skew is applied for both setup and hold checks. However, only the setup value is used for timing analysis and optimization, while the hold value is stored and written out with the `write_sdc` command.

Options and Arguments

`{clock | port}`	Specifies the clocks and ports to which the specified skew value applies.
float	Specifies the skew value.
`-minus_uncertainty`	
	Shifts the clock edge to the left to make the clock arrive earlier.
	Used to check setup delays.
	Equivalent to:
	`set_clock_uncertainty -setup` *uncertainty* `[`*clock*`|`*port*`]...`
`-plus_uncertainty`	
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	Equivalent to:
	`set_clock_uncertainty` *uncertainty* `[`*clock*`|port]...`

Example

In the following example, the uncertainty will be applied to both setup and hold. But only setup is considered for timing analysis and hold will be ignored.

```
genus:root:> set_clock_skew 0.5 [get_clocks clk1]
```

Related Information

Related command:	set_clock_uncertainty

set_clock_transition

```
set_clock_transition
{-min | -max} {-rise | -fall} float clock_list
```

Description

Specifies the clock transition (slew) on the specified clocks.

You can specify four slew values: the minimum rise, minimum fall, maximum rise, and maximum fall slew.

The minimum values are not used for timing analysis, but they will be written out when you execute the `write_sdc` command.

Options and Arguments

clock_list	Specifies a list of clocks only with ideal mode in the design, which affects the slew times on all register clock pins in the transitive fanout of the specified clocks. Specified set_clock_transition values on register clock pins in the fanout of a clock with propagated latency are ignored.
`-fall`	Specifies the falling edge (transition value) of the register clock pins on which the slew time is asserted.
float	Specifies the slew value for the specified clock(s).
`-max`	Indicates that the specified value is a maximum slew value.
`-min`	Indicates that the specified value is a minimum slew value.
`-rise`	Specifies the rising edge (transition value) of the register clock pins on which the slew time is asserted.

Related Information

Related commands:	remove_clock_transition
	set_clock_latency
	set_clock_uncertainty
	set_input_delay

set_clock_uncertainty

```
set_clock_uncertainty
[-from_edge string] [-to_edge string]
[-from clock_list | -fall_from clock_list |
  -rise_from clock_list]
[-to clock_list | -fall_to clock_list |
  -rise_to clock_list]
[-hold | -setup] [-half_cycle_jitter]
[-full_cycle_jitter] uncertainty
[clock|port|pin|hpin]...
```

Description

Specifies the uncertainty on the clock network. You specify either a simple/intra-clock or an inter-clock uncertainty.

- Simple/intra-clock uncertainties are defined directly on a clock, port, pin, or instance. These uncertainty values are stored in attributes. When you specify a clock, it means that the uncertainty applies to all sequential elements driven by that clock. When applied on a port or pin, it applies to all clocks (and their corresponding sequential elements) in the fanout of the port or the pin.
- The inter-clock uncertainties are defined using options such as `-from`, `-to`, `-rise_from`, `-rise_to`. These are modeled as path_adjust exceptions. These inter-clock uncertainties take precedence over the simple uncertainty values, when applied on a clock object. But, when set on port, pin or inst objects, the simple uncertainty has higher priority than inter-clock uncertainty.When you specify inter-clock uncertainty, it must be specified to cover all source and destination clock combinations. For example, if you specify paths from CLK1 to CLK2, you must specify the uncertainty for CLK2 to CLK1 even if the values are the same.

Options and Arguments

{*clock* | *pin* | *hpin* | *port*}	
	Specifies the clocks, pins, and ports to which the specified uncertainty value applies. In case of instances, the uncertainty is applied on the input pins of the instance.
	These arguments only apply to simple uncertainties.
-from *clock_list* | -fall_from *clock_list* | -rise_from *clock_list*	
	Applies the uncertainty value to the specified list of launching clocks.
	These options only apply to inter-clock uncertainties.
-from_edge {rise | fall | both}	
	Specifies the edge type of the launch clock.
	This option only applies to inter-clock uncertainties and cannot be specified with either `-fall_from` or `-rise_from`.
-full_cycle_jitter	Specifies to use the full cycle jitter.

`-half_cycle_jitter`	Specifies to use the half cycle jitter.
`-hold | -setup`	Specifies that the clock uncertainty applies to hold or setup checks.
	If neither option is specified, the uncertainty value applies to both checks.
	These options apply to both types of uncertainties.
`-to` *clock_list* `| -fall_to` *clock_list* `|-rise_to` *clock_list*	
	Applies the uncertainty value to the specified list of capture clocks.
	These options only apply to inter-clock uncertainties.
`-to_edge {rise | fall | both}`	
	Specifies the edge type of the capture clock.
	This option only applies to inter-clock uncertainties and cannot be specified with either `-fall_to` or `-rise_to`.
uncertainty	Specifies the uncertainty value for the clock(s). Use a floating number.

Examples

- The following command sets a (intra-clock) setup uncertainty of 0.4 sdc units for all paths ending at `reg1` and captured by the rising transition of all clocks propagating to `reg1/CK`.

  ```
  set_clock_uncertainty -setup -rise_to 0.4 [get_db pins *reg1/CK]
  ```

- The following command sets a (inter-clock) setup uncertainty of 0.5 sdc units for all paths launched by `clk2` and captured by `clk1`.

  ```
  set_clock_uncertainty -setup -from clk2 -to clk1 0.5
  ```

- The following example specifies that all paths leading to registers or ports clocked by CLKB have setup uncertainty of 0.5 and hold uncertainty of 0.2:.

  ```
  set_clock_uncertainty -setup 0.5 [get_clocks CLKB]

  set_clock_uncertainty -hold 0.2 [get_clocks CLKB]
  ```

If there are paths from other clocks to CLKB, then unless otherwise specified, the intra-clock uncertainty also models the inter-clock uncertainty.

- The following example specifies inter-clock uncertainties between CLKA and CLKB clock domains:

  ```
  set_clock_uncertainty 1.5 -from CLKA -to CLKB

  set_clock_uncertainty 1.5 -from CLKB -to CLKA
  ```

- The following example specifies inter-clock uncertainties between the CLKA and CLKB clock domains with specific edges:

  ```
  set_clock_uncertainty 0.25 -rise_from CLKA -to CLKB

  set_clock_uncertainty 0.25 -fall_from CLKA -rise_to CLKB

  set_clock_uncertainty 1.5 -from CLKA -fall_to CLKB
  ```

- The following example specifies that when both inter-clock and intra-clock uncertainties are specified, then inter-clock takes a higher precedence:
  Intra-clock and Inter-clock Uncertainty Conflict

```
set_clock_uncertainty 0.6 -from CLKB -to CLKA

set_clock_uncertainty 0.5 [get_clocks CLKA]
```

In the example above the two uncertainty values on Clock CLKA are different. For all paths from CLKB to CLKA, the inter-clock uncertainty takes precedence, and a value of 0.6 is used. For all other paths where the destination clock is CLKA, the intra-clock value of 0.5 is used.

Related Information

Affects this command:	path_adjust
	reset_clock_uncertainty
Sets these attributes:	clock Attributes
	setup_uncertainty

set_constraint_mode

```
set_constraint_mode
mode
```

Description

Specifies the mode to which subsequent constraints—entered *interactively* at the Genus prompt—must be applied.

Specify this command first followed by the changes in the constraints for that specified mode.

Once this command is set, all constraints that need a mode specification will apply the constraint to the specified mode.

> You do not need to explicitly set `set_constraint_mode` before each constraint.

> When reading in the SDC file using the `read_sdc` command, use the `-mode` option to specify the mode to which all the constraints in the SDC file must be applied.

Options and Arguments

mode	Specifies the mode to which subsequent constraints must be applied.

Example

The following command shows the error messages issued when using interactive constraint mode:

```
set_false_path -to delay_reg[1]

Error   : SDC command requires a constraint mode specification. [SDC-235]          [retrieve_mode]        : The 'set_false_path' command
requires a constraint mode to be          specified when in MMMC.        : The indicated command requires that a constraint mode be specified.
Use the 'set_interactive_constraint_modes' to indicate the          constraint mode for the command.
```

When `set_constraint_mode` command is used the constraint is applied:

```
set_constraint_mode synth_setup_ss

mode:top/synth_setup_ss

set_false_path -to delay_reg[1]

exception:top/syn_setup_ss/dis_1
```

Related Information

Related command:	create_constraint_mode
	create_analysis_view
	set_analysis_view

set_data_check

`set_data_check`
`[-from {pin|hpin|port} | -rise_from {pin|hpin|port} | -fall_from {pin|hpin|port}][-to {pin|hpin|port} | -rise_to {pin|hpin|port} | -fall_to {pin|hpin|port}][-setup | -hold][-clock clock_object][-constrained_clock]value`

Description

Sets data-to-data checks using the specified values of setup and hold time. Data-to data checks are non-sequential checks that are designed to constrain a data signal with respect to another data signal.

In a normal sequential check, a data signal is constrained with respect to a clock signal. In these cases single-cycle timing is used - allowing one full clock cycle for the data transfer. In a non-sequential data-to-data check, there is no clock reference. Therefore zero-cycle timing rules are used. You can use the `set_multicycle_path` constraint to adjust the cycle accounting rules.

For setup checks the tool computes the maximum arrival time on the constrained pin (specified with the `-to|-rise_to|-fall_to` option) and checks it against the minimum arrival time on the related pin (specified with the `-from|-rise_from|-fall_from` option).

The arrival times on the related pin can depend on the clock that triggers the data event on this pin.

Options and Arguments

`-clock` *clock*	Specifies the name of the clock that triggers the data event on the related (`-from`) pin.
	In the case where there are multiple data events triggered by different clock phases arriving at the `-from` pin, the `-clock` parameter allows you to specify which data event should be used for the "related pin" side of the timing check.
	If this option is not specified, the data check is clock independent.
`-constrained_clock`	Specifies the clock which triggers the minimum arrival time on the launch pin.
`-from` {*pin*|*hpin*|*port*} | `-fall_from` {*pin*|*hpin*|*port*} | `-rise_from` {*pin*|*hpin*|*port*}	
	Specifies the pins or ports that are set as related pins for data-to-data checks.
	The pin or port list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. Leaf design cells (instances) can also be specified.

	Use only one -from, -rise_from, or -fall_from parameter per command.
	By default, the -from parameter checks both the rising and the falling edges.
	Using -rise_from parameter checks at the rising edge of the signal on the from pins.
	Using -fall_from parameter checks at the falling edge of the signal on the from pins.
	You cannot use the -from parameter to specify multiple objects.
-setup | -hold	Specifies whether the data check value is for setup or hold check.
	If you do not specify either -setup or -hold option, the value applies to both setup and hold.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The write_sdc command will write the option out for consumption by downstream tools.
-to {*pin*|*hpin*|*port*} |-fall_to {*pin*|*hpin*|*port*} |-rise_to {*pin*|*hpin*|*port*}	
	Specifies the pins or ports that are set as constrained pins for data-to-data checks.
	The pin or port list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. Leaf design cells (instances) can also be specified.
	If you specify the -to option, the data check is performed on the falling and rising edge of specified pin.
	Use only one -to, -rise_to, or -fall_to parameter per command.
	By default, the -to parameter checks both the rising and the falling edges.
	Using -rise_to parameter checks at the rising edge of the signal on the from pins.
	Using -fall_to parameter checks at the falling edge of the signal on the from pins.
value	Specifies the value of the setup or hold time for the check, in the time units specified with the set_time_unit command. Use a floating number.

Example

The following example demonstrates how the set_data_check command can be used to constrain signals:

Consider two control signals, say XE and SE, going to a macro block. Both of these are derived from the same state machine, and hence launched from the same clock domain. You want XE to reach the macro earlier than SE. The set_data_check constraint can meet this design intent, by applying data checks on non-sequential signals, without setting path specific limits using the following command:

```
set_data_check -from [get_pins {sub/XE}] -to [get_pins {sub/SE}] -setup 0.5
```

The set_data_check command establishes data-to-data checks using similar ideas of setup and hold time as in sequential checks, with the -from parameter denoting the reference signal (the clock in a traditional setup/hold timing check), and the -to parameter indicating the signal being constrained.

Related Information

Related command:	set_time_unit

Current_Limitations

1. Genus does not support data-to-data checks on top-level ports. Genus will not infer a check and this will also not be captured as part of `write_sdc`.
2. Genus does not support data-to-data checks specified on pins belonging to different instances. Genus will not infer such a data check.
3. Genus also does not support specifying `set_data_check` -`from`/-`to` instances.

set_disable_clock_gating_check

```
set_disable_clock_gating_check
[-setup] [-hold]
[-rise] [-fall]
[-high | -low]
{pin | hpin |inst| hinst} ...
```

Description

Disables the clock gating check inferred on the specified pins or the enable pins of the specified instance.

> This command is supported only on mapped instances and pins of mapped instances. This command is not supported on the clock pins of an instance.

> To write this constraint in the output SDC file, you must set the following variable:

```
set write_disable_and_rmdisable_cgc 1
```

Options and Arguments

`-fall`	Disables only clock gating checks specified for a falling clock transition.
`-high`	Disables any clock gating check for which the non-controlling value of the clock is high.
`-hold`	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
inst|*hinst*	Disables all clock gating checks inferred at every single enable pin of the specified instance.
	`To restore the clock gating check on an instance, use:remove_disable_clock_gating_check` *instance*
`-low`	Disables any clock gating check for which the non-controlling value of the clock is low.
pin|*hpin*	Specifies the pin on which to apply the command.
	If the pin is the enable pin of a clock gate, all clock gating checks inferred at the enable pin are disabled and the timing path is not broken at this pin.

	To restore the clock gating check on a pin, use:remove_disable_clock_gating_check *pin*
-rise	Disables any clock gating check specified for a rising clock transition.
-setup	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The write_sdc command will write the option out for consumption by downstream tools.

Related Information

Related command:	remove_disable_clock_gating_check

set_disable_timing

```
set_disable_timing
 [-from pin] [-to pin]
 {lib_pin | pin | lib_arc | port| hport | lib_cell | inst| hinst | hpin | arc}... } [-quiet]
```

Description

Disables the timing propagation through the specified objects.

- For a pin, port, subport of a hierarchical instance, Genus breaks the timing paths at this point. For a subport, the timing path is broken at the pin on the other side of the subport.
- For a pin of an unmapped instance, set_disable_timing is not supported.

Options and Arguments

-from *pin*	Disables the timing arcs starting from this pin in the specified lib_cells or instances.
	An error is given when you specify an object type other than a lib_cell or instance with this option.
{*lib_pin* | *pin* | *hpin* | *port* | *hport* | *arc*}	
	Specifies the lib_pin, pin, port, arc, or hierarchical pin or port at which the timing path(s) must be broken.
{*lib_cell* | *inst*| *hinst*}	
	Specifies the lib_cells or instances for which you want to disable timing arcs. If you specify a lib_cell or instance without a -from or -to option, the tool disables all timing arcs for that lib_cell or instance.
-to *pin*	Disables the timing arcs ending at this pin in the specified lib_cells or instances.
	An error is given when you specify an object type other than a lib_cell or instance with this option.
-quiet	Specifies not to display any error or warning messages.

Related Information

Related command:	remove_disable_timing

set_dont_touch

```
set_dont_touch
{design | module | inst| hinst | hnet | lib_cell | base_cell} ...
```

Description

Prevents the specified object from being modified during optimization. However, the specified object can be moved during optimization.

This command does not work on unmapped or unresolved objects.

When you specify a net, it cannot be buffered or become a part of the logic cone restructuring. However, you can resize the instances connected to this net because the `dont_touch` attribute set of the net does not propagate on the connected instances.

For net objects, the `preserve` attribute is only set on the specified net, and not on all the segments of the physical net (if the net traverses many hierarchies).

Options and Arguments

{*design*	*module*	*inst*	*hinst*	*hnet*	*lib_cell*	*base_cell*}	
	Specifies the list of instances, nets, library cells, base cells or submodules that you want to preserve.

Examples

- The following command prevents the specified instance from being modified:

  ```
  genus:root:> set_dont_touch [get_cells {TWA/FF1}]
  ```

- The following command prevents the specified instance explicitly from being modified:

  ```
  genus:root:> set_dont_touch /dtmf_recvr_core/DMA_INST true
  ```

set_dont_touch_network

```
set_dont_touch_network
[-no_propagate]
{clock | port | pin | hpin | port_bus | hpin_bus}...
```

Description

Prevents the combinational path connected to the given clocks, pins, or ports from being modified. If a clock object is specified, the nets and combinational logic fanning out from the clock sources are preserved.

> If a net segment has any unmapped leaf-level instances, unresolved instances, or instances which are not logical hierarchies (such as a bmux), the command will preserve the net segment leading up to the first unmapped leaf combinational instance or non-logical hierarchical instance.

`set_dont_touch_network` can have unintended consequences by limiting optimization. For example, using this command on an input port will results in a `dont_touch` on the first sequential element and the logic will not be optimized. This happens if the -`no_propagate` switch is not used.

Options and Arguments

{*clock*	*port*	*pin*	*hpin*	*port_bus*	*hpin_bus*}	
	Specifies the objects - clocks, pins, or ports - to be preserved.
-no_propagate	Preserves all the nets driven by the specified pin up to the first leaf-cell-input (combinational or sequential).
	If this option is omitted, preserving stops at the first sequential cell in the network.

Example

The following command prevents the combinational path connected to the specified pin from being modified:

```
genus:root:> set_dont_touch_network [get_pins and1/z]
```

Related Information

Sets this attribute:	preserve

set_dont_use

```
set_dont_use
 lib_cell | base_cell
```

Description

Indicates whether the specified library cell(s) should be avoided by the technology mapper. Sets the `avoid` attribute on the specified cells to the specified value.

Options and Arguments

*lib_cell	base_cell*	Specifies the library cell(s) or base cell(s) that must be marked for the technology mapper.

set_drive

```
set_drive
[-rise] [-fall]
[-min] [-max]
resistance {port|port_bus}...
```

Description

Sets the drive resistance on the specified input and/or bidirectional ports of the top cell.

The drive resistance models the cell driving the port. This resistance is used to compute the load dependent delay and the slew at the specified ports. The load-dependent delay is added to the arrival times.

In absence of these specifications, by default, all inputs are assumed to have an infinite drive strength. The default condition implies that the transition time at the input pins is 0.

The smaller the drive resistance value, the higher the drive strength. A resistance value 0 implies an infinite drive strength. As the drive value (resistance) increases, drive strength decreases and transition time increases.

The load-dependent delay and the slew are computed by multiplying the drive resistance by the wire capacitance of the net connected to the port.

The resolution is 1/1000. Genus ignores (does not use) the minimum values but the they can be passed to downstream tools through `write_sdc`. As a result, the tool sets the `external_resistance` attribute on the specified ports and removes the external driver on the specified ports.

Use the `report_port` command to view drive resistances asserted on ports.

Options and Arguments

`-fall`	Specifies that the drive resistance is to be used for the falling transition at the specified ports.
	By default, if neither the `-rise` nor the `-fall` option are specified, the drive resistance applies to both the rising transition and the falling transition.
`-min`	Specifies that the drive resistance is to be used to compute the early path delay.
	By default, if neither the `-min` option nor the `-max` option is specified, the drive resistance applies to both the early path delay and the late path delay.
`-max`	Specifies that the drive resistance is to be used to compute the late path delay.
	By default, if neither the `-min` option nor the `-max` option is specified, the drive resistance applies to both the early path delay and the late path delay.
`{`*port* | *port_bus*`}`	Specifies a list of input or bidirectional ports on which the drive resistance is set.
resistance	Specifies the value of the drive resistance. This value must be greater than or equal to zero. The drive resistance is in units found in the target technology library.
`-rise`	Specifies that the drive resistance is to be used for the rising transitions at the specified ports.
	By default, if neither the `-rise` nor the `-fall` option are specified, the drive resistance applies to both the rising transition and the falling transition.

Example

The following command sets a drive resistance of 2.0 for the early path, the late path, the rising and the falling transitions at the in2 and in2 ports:

```
set_drive 2.0 [get_ports {in1 in2}]
```

Related Information

Related commands:	all_inputs
	set_driving_cell
	set_input_transition
	set_load

set_driving_cell

```
set_driving_cell
{-cell cell | -lib_cell cell} [-library library]
[-from_pin pin] [-pin pin]
[-input_transition_rise float]
[-input_transition_fall float]
[-rise] [-fall] [-max | -min]
[-multiply_by float] [-no_design_rule]
 port_list
```

Description

Models the drive capability of an external driver connected to the input port.

To model the drive capability of the external driver, the timing analyzer computes an offset to the arrival time of an input, and changes the slew time used to compute the delay for the next cell. The timing analyzer computes the offset by subtracting the zero load delay from the total delay of the cell.

This constraint overrides any prior `set_drive` or `set_input_transition` constraint defined on the same port. If you do not specify drive or transition using any of these constraints, the timing analyzer use a default value of 0ps.

This command associates an external driving cell to the input or inout port of the current design.

> You cannot assign a buffer to drive the output port.

Options and Arguments

`-cell` *cell*	Specifies the name of the lib_cell used to drive the port.
`-fall`	Indicates that the information applies only to a falling transition on the ports.
	By default, if both the `-fall` and `-rise` options are omitted, the information applies to the rising and falling transitions on the ports.
`-from_pin` *pin*	Specifies an input pin on the specified cell. The command uses the drive of the timing arc from this pin to the pin specified with the `-pin` option.
	If this option is omitted, the transition on `-pin` is assumed to be 0.
`-input_transition_fall` *float*	
	Specifies the input falling transition time associated with the `-from_pin` option.

	Default: `0`
`-input_transition_rise` *float*	
	Specifies the input rising transition time associated with the `-from_pin` option.
	Default: `0`
`-lib_cell` *cell*	Specifies the name of the lib_cell used to drive the port.
`-library` *library*	Specifies the name of the library to which the lib_cell belongs.
	If this option is omitted, the command searches in all available libraries.
`-max`	Indicates that the driver cell information applies to setup analysis.
	By default, if both the `-max` and `-min` options are omitted, the information applies to setup analysis.
`-min`	Specifies the early drive assertion.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-multiply_by` float	
	Specifies the total number of external drivers. The number of additional external drivers is the specified number minus 1 and this value will be set on the `external_non_tristate_drivers` port attribute.
	Default: `1`
`-no_design_rule`	Indicates to ignore the design rule violations seen by the external driver pin.
`-pin` *pin*	Specifies the output pin of the lib_cell which will be used to drive the port.
	If this option is omitted, the command chooses an output pin at random.
port_list	Specifies the port(s) to which the external driver applies.
`-rise`	Indicates that the information applies only to a rising transition on the ports.
	By default, if both the `-fall` and `-rise` options are omitted, the information applies to the rising and falling transitions on the ports.

Examples

- The following command specifies the driving cell set on the input port

```
genus:root:> set_driving_cell -lib_cell BUFX4 [get_db ports *port_pad_data_in[5]]
```

- The following command shows the limitation that you cannot assign a buffer to drive the output port

```
genus:root:> set_driving_cell -lib_cell BUFX4 [get_db ports *port_pad_data_out[9]]
Warning : The given attribute is not valid on a port of this direction.\
          [TIM-126]
         :The attribute 'fixed_slew' cannot be applied to output port\
         'port_pad_data_out[9]'.
```

Related Information

Related attribute:	external_non_tristate_drivers
Related command:	remove_from_collection

set_equal

```
set_equal
port1 port2
```

Description

Declares the two specified ports in the current design to be logically equivalent. This information is used during synthesis to eliminate redundant ports and hence improving optimization quality.

he command does not work on output ports and ports with multidriven nets.

Use the `reset_design` command to remove this property from a port.

Options and Arguments

port1	Specifies the first input port.
port2	Specifies the another input port.

Example

The following example sets two input ports `A` and `B` logically equal.

```
genus:root:> set_equal A [get_ports B]
```

set_false_path

```
set_false_path
{{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_to {{clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-rise_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-fall_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-rise] [-fall] [-setup] [-hold]
[-exception_name name] [-reset_path]
[-comment string]
```

Description

Identifies false paths in a design, and breaks or disables specific instance timing arcs in a design resulting in them not being timed.

> Q pin of the flop which acts as a generated clock source is not a valid through point in false path constraints. You should exclude this pin from through point. The following error will be reported if it is specified as a through point in false path constraint. `Error: Invalid path specification. A 'through' object is invalid. [TIM-233] [set_false_path]`

> To break combinational loops, use the `set_disable_timing` command instead of the `set_false_path` command.

The `set_false_path` command differs from the `set_disable_timing` command as follows:

- `set_false_path` command selectively disables the arrival time information depending on the precedence rules.
- `set_false_path` command does not affect constant values.
- `set_disable_timing` command physically snips a timing arc such that neither arrival times nor constant values propagate through it.
- `set_disable_timing` command is applied in all cases to valid timing arcs.

The `set_false_path -from fpin` command disables all timing paths whose source pin is an `fpin`. Similarly, the `set_false_path -to tpin` command disables all timing paths whose sink pin is a `tpin`. Using the `set_false_path -from fpin -to tpin` command disables all timing paths whose source is fpin and sink is tpin.

Specify a group of pins by enclosing the group with curly braces ({}) within one `set_false_path` command.

To set a false path for any path beginning at `p1` or `p2` and terminating at `t1` or `t2`, group the pins as follows:

```
set_false_path -from {p1 p2} -to {t1 t2}
```

> This method of pin grouping reduces the number of exceptions. Having large numbers of exceptions adversely impacts the run time of the timing analysis commands.

`set_clock_groups -asynchronous` also applies a `set_false_path` in between various clock groups, however, the interpretation of that command is different than `set_false_path` for implementation tools as `set_false_path` allows SI/EM analysis as if the clocks could be pulsed concurrently. Genus will still write out the original command used when `write_sdc` is involved to export the SDC constraints.

Options and Arguments

-comment string	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
-exception_name *name*	
	Specifies the name of the timing exception you want to create. *Default*: `dis_n`
-fall	Applies the assertion at the falling edge of the signal on the path endpoints.
(-from | -fall_from | -rise_from} {*clock* | *port* | *inst*| *hinst* | *pin* | *hpin* | *port_bus* | *hpin_bus*}	
	Specifies a list of timing paths start points.
	The argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The argument can be a collection.
	The valid start points are:

	• Input port • Bidirectional port • Clock pin of sequential cell • Clock pin of sequential cell is one of the following: ◦ Structural clock pin that is part of latch or flip-flop. ◦ Inferred clock pin in cells like macro cells. • A pin is an inferred clock pin if it is associated with one of the following: ◦ Reference end of setup check ◦ Reference end of hold check ◦ Reference end of removal check ◦ Reference end of recovery check ◦ Minimum period check ◦ Minimum pulse width check ◦ Pin associated with set_input_delay
	Applies the false path constraint to all paths originating from the list. If a clock is specified as the `-from` object, all primary inputs and registers clocked by that clock are used as start points.
	By default, the `-from` option applies the constraint to both the rising and the falling edges. `-rise_from`: Applies the false path constraint at the rising edge of the signal on the from pins. `-fall_from`: Applies the false path constraint at the falling edge of the signal on the from pins.
`-fall_through | -rise_through | -through` {*port* | *inst*| *hinst* | *pin* | *hpin* | *hnet* | *port_bus* | *hpin_bus*}	
	Specifies a list of timing paths through points.
	The argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The argument can be a collection.
	Specify one of the following: `-through`:
	When you use this parameter without using the `-from` or `-to` options, the false path constraint is set for all paths that go through the specified pins. When you use it with both the `-from` and `-to` options, any path starting at any pin on the `-from` list and going through any point on the `-through` pin list and going to any point on the `-to` list is affected by the constraint.
	By default, the `-through` option applies the constraint to both the rising and the falling edges. `-rise_through`: Applies the assertion at the rising edge of the signal on the through pins. `-fall_through`: Applies the assertion at the falling edge of the signal on the through pins.
`-fall_to | -rise_to | -to` {*clock* | *port* | *inst*| *hinst* | *pin* | *hpin* | *port_bus* | *hpin_bus*}	
	Specifies a list of timing paths end points.
	The argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The argument can be a collection.
	The valid end points are:

	• Output port • Bidirectional port • Data pin of sequential cell • Data pin of sequential cell is one of the following: ◦ Structural data pin that is part of latch or flip-flop. ◦ Inferred data pin in cells like macro cells. • A pin is an inferred data pin if it is associated with one of the following: ◦ Signal end of setup check ◦ Signal end of hold check ◦ Signal end of non-sequential setup check ◦ Signal end of non-sequential hold check ◦ Signal end of removal check ◦ Signal end of recovery check ◦ Pin associated with `set_external_delay` ◦ Data pin of clock gating cell
	Applies the false path constraint to all paths ending in the -to pin list. If a clock is specified as the -to object, all primary outputs and registers clocked by that clock are used as endpoint.
	By default, the `-to` option applies the constraint to both the rising and the falling edges. `-rise_to`: Applies the false path constraint at the rising edge of the signal on the to pins. `-fall_to`: Applies the false path constraint at the falling edge of the signal on the to pins.
`-hold | -setup`	Sets false paths for either setup or hold analysis.
	Genus does not perform hold analysis and optimization. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	If you do not specify -`setup` or -`hold`, the software sets false paths for both setup and hold analysis.
`-reset_path`	Resets any other exceptions (`multicycle` and `path_delay`) on the path specified using the `-from`, `-through`, `-to` options and applies a `set_false_path` exception for these paths.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-setup`	Applies the false path constraint for setup analysis only.

Examples

- The following command applies a false path (disables the path for timing analysis) to all paths between `inst1/CK` and `inst2/D`.

  ```
  set_false_path  -from clock:inst1/CK  -to clock:inst2/D -comment "my false paths"
  ```

- The following command applies a false path (disables the path for timing analysis) through all paths that go through `inst1/A`.

  ```
  set_false_path -through inst:inst1/A
  ```

- The following command disables hold checking from ff1 to ff2:

  ```
  set_false_path -hold -from ff1 -to ff2
  ```

- The following command sets all timing paths from ff1/CK to D input of ff2 that pass through u1/Y and one or more of u2/Y and u3/Y as false paths:

  ```
  set_false_path -from ff1/CK -through {u1/Y} -through {u2/Y u3/Y} -to ff2/D
  ```

- The following command sets all timing paths from the rise of ff1/CK to ff2/D through one or more of u1/Y and u4/Y as false paths:

  ```
  set_false_path -rise_from ff1/CK -through {u1/Y u4/Y} -to ff2/D
  ```

Related Information

Related attribute:	comment
Related commands:	set_disable_timing
	set_max_delay
	set_min_delay
	set_multicycle_path
	set_clock_groups

set_fanout_load

`set_fanout_load` *`float port_list`*

Description

Specifies the fanout load on a set of ports. Library instance pins may have fanout loads set in the technology library. Fanout does not have specific units so it is assumed that fanout as used by constraints is consistent with fanout in the technology libraries. If a pin or port has no fanout attribute, either from constraints or from the technology library, the fanout load is zero.

> The fanout load is used only for design rule checking; it does not affect timing.

This command sets the `external_fanout_load` attribute on the specified output and inout ports.

Options and Arguments

`float`	Specifies the external load for the port(s).

port_list	Specifies the ports to which the fanout load is applied.

Examples

- The following command sets the fanout load of port out to 2:

  ```
  set_fanout_load 2 out
  ```

- The following command sets the fanout load of ports in1 and in2 to 1:

  ```
  set_fanout_load 1 {in1 in2}
  ```

- The following command sets the fanout load of element 3 of port out to 12:

  ```
  set_fanout_load 12 { out[3] }
  ```

- The following command sets the fanout load of all the ports to 2:

  ```
  set_fanout_load 2 [get_ports *]
  ```

Related Information

Sets this attribute:	external_fanout_load
Related commands:	report_nets
	report_port
	set_max_fanout

set_hierarchy_separator

```
set_hierarchy_separator
character
```

Description

Specifies the hierarchy delimiter character used in the SDC file. Preferably, choose your hierarchy separator to be a character, which is not used in the actual naming of any object in your design.

For example, your design does not use any escaped naming. In that case, " / " is a good hierarchy separator. When SDC refers to " a/b", there is no ambiguity. On the other hand, if your design has several objects, where " / " is used as a part of the escaped name itself, then using this same character as a hierarchy separator can cause ambiguity.

In such situations, the hierarchy separator might be changed to another character like @, using the following command:
`set_hierarchy_separator @`

With the hierarchy separator being set at @, the above ambiguity is resolved.Then, "a/b" means a design object a/b, where "/" is part of the name. If you want to refer to "b within a", you need to specify a@b.

Options and Arguments

character	Specifies the hierarchy delimiter character.

	Default: `/`

set_ideal_latency

```
set_ideal_latency [-min | -max] [-rise | -fall]
latency_value port_list
```

Description

Specifies the ideal latency to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

Options and Arguments

I

`-fall`	Indicates that the specified value is a falling ideal transition value.
latency_value	Specifies the ideal transition time for the specified port(s).
`-max`	Indicates that the specified value is a maximum ideal transition value.
`-min`	Indicates that the specified value is a minimum ideal transition value.
port_list	Specifies the list of objects - instances, pins or ports.
`-rise`	Indicates that the specified value is a rising ideal transition value.

Example

The following command specifies the ideal latency of 1.5 to the input pin A of cell `FE_OFC413_scan_se`:

```
set_ideal_latency 1.5 FE_OFB413_scan_se/A
```

set_ideal_net

```
set_ideal_net hnet_list
```

Description

Causes the specified nets to be treated as ideal and prevents the timing and optimization engines from optimizing these nets. The command sets the `ideal_driver` attribute on the leaf-level drivers of the specified nets to `true`.

Options and Arguments

hnet_list	Specifies the nets which must be treated as ideal nets.

Related Information

Related command:	remove_ideal_net
Sets this attribute:	ideal_driver

set_ideal_network

```
set_ideal_network {pin | hpin | port | hnet}...
[-dont_care_placement] [-no_propagate]
```

Description

Sets the `ideal_network` attribute on all leaf-level drivers of the specified object to `true`.

Options and Arguments

{pin | hpin | port | hnet}	
	Specifies the object(s) to which the command applies.
`-dont_care_placement`	
	Specifies to ignore placement.
`-no_propagate`	Only sets the `ideal_driver` attribute on the leaf-level drivers of the specified object to `true`. This way only the electrically connected nets are idealized.

Related Information

Related command:	remove_ideal_network
Sets these attributes:	ideal_driver
	ideal_network

set_ideal_transition

```
set_ideal_transition [-min | -max]
[-rise | -fall] float
[string]
```

Description

Specifies to set the ideal transition value for the specified port(s).

Options and Arguments

`-fall`	Indicates that the specified value is a falling ideal transition value.
`-max`	Indicates that the specified value is a maximum ideal transition value.
float	Specifies the ideal transition time for the specified port(s).
`-min`	Indicates that the specified value is a minimum ideal transition value.
`string`	Specifies the list of objects - instance pins or ports from where to remove the transition value.
`-rise`	Indicates that the specified value is a rising ideal transition value.

Example

The following command specifies the ideal transition of 0.5 on the pin `scan_se`:

```
set_ideal_transition 0.5 scan_se
```

Related Information

Related command:	reset_ideal_transition

set_input_delay

```
set_input_delay
[-clock clock]
[-clock_fall| -clock_rise]
[-level_sensitive]
[-network_latency_included]
[-source_latency_included]
[-reference_pin {pin|port}...]
[-rise] [-fall] [-max] [-min]
[-add_delay] [-name name] [-group_path group]delay {port|pin|hpin|port_bus|hpin_bus}...
```

Description

Constrains input and inout ports, port busses, and pins (that are valid start points) within the design relative to a clock edge.

If you omit both the `-min` and `-max` options, the delay is assumed to apply for both setup and hold analysis.

If you omit both the `-rise` and `-fall` options, the delay applies for both the rising and falling edges of the input signal.

> This command sets the `external_delays` attribute on the specified pins or ports.

Options and Arguments

`-add_delay`	Specifies to add the specified input delay information for the specified pins or ports.
	Use this option to capture the delay information if multiple paths lead to an input pin or port that are relative to different clocks or clock edges.
	If you omit this option, the specified delay information overwrites all existing input delays for the specified pins or ports.
`-clock` *clock*	Specifies the reference clock. The input delay is defined relative to this clock.
	If no clocks are specified, the timing paths from the specified pins and ports will be unconstrained as captured in the `report_timing` report.
`-clock_fall`	Specifies to use the falling edge of the reference clock as reference edge.
	Default: `-clock_rise`
`-clock_rise`	Specifies to use the rising edge of the reference clock as reference edge.
	Default: `-clock_rise`
delay	Specifies the input delay value. This is a floating number.
`-fall`	Specifies that the delay is measured with respect to the falling edge of the input signal.
`-group_path` *group*	Specifies the name of the group to which paths ending at the specified ports or pins must be added.
`-level_sensitive`	Specifies that the constraint comes from a level-sensitive latch.
	By default, the constraint comes from an edge-triggered flip-flop.
`-max`	Indicates that the specified delay applies for setup analysis.
`-min`	Indicates that the specified delay applies for hold analysis.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-name` *name*	Associates a name with the specified timing constraint.
`-network_latency_included`	
	Specifies that the input delay includes the clock network latency values. This implies that the tool does not need to consider the clock network latency values when optimizing this path.
	Sets the `clock_network_latency_included` attribute to `true`.
{*port* | *pin* | *hpin* | *port_bus* | *hpin_bus*}	
	Specifies the input and inout pins, ports, and port buses to which the specified input delay value applies.
`-reference_pin` {*pin* | *port*}	
	Adds source and network latency of the specified reference pin to the input delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the `-clock` parameter.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

`-rise`	Specifies that the delay is measured with respect to the rising edge of the input signal.
`-source_latency_included`	
	Specifies that the input delay includes the clock source latency values. This implies that the tool does not need to consider the clock source latency values when optimizing this path.
	Sets the `clock_source_latency_included` attribute to `true`.

Examples

- For the first delay specification on the input port, it is not necessary to specify `-add_delay`. If a user puts an `-add_delay` to the first specification, the option would simply be ignored:

```
set_input_delay -clock CLK1 -min 3000 [get_ports ABC]

set_input_delay -clock CLK1 -max 4000 [get_ports ABC]
```

- The following example overrides both the previous specifications, with respect to CLK1:

```
set_input_delay -clock CLK2 3500 [get_ports ABC]
```

- The following example adds to the existing value, without removing any of the earlier existing constraints:

```
set_input_delay -clock CLK1 -add_delay 3000 [get_ports ABC]
```

- The following example applies an input delay relative to CLK3 on all input ports of the design:

```
set_input_delay 1000 -clock  CLK3 [all_inputs]
```

- The following example applies an input delay relative to the falling edge of CLK1:

```
set_input_delay -clock CLK1 -clock_fall 2.0 [get_ports ABC]
```

Related Information

Sets these attributes:	clock_network_latency_included
	clock_source_latency_included
	external_delays

set_input_transition

```
set_input_transition
{-min | -max} {-rise | -fall}
float port_list
```

Description

Sets a specific transition time (slew) on the specified input and inout ports. You can specify the minimum rise, minimum fall, maximum rise, or maximum fall slew.

The minimum values are not used for timing analysis, but they will be written out when you execute the `write_sdc` command.

Options and Arguments

`-fall`	Indicates that the specified value is a falling transition value.

float	Specifies the transition time for the specified port(s).
-max	Indicates that the specified value is a maximum transition value.
-min	Indicates that the specified value is a minimum transition value.
port_list	Specifies the port(s) to which the transition time applies.
-rise	Indicates that the specified value is a rising transition value.

set_lib_pins

```
set_lib_pins
[-max_capacitance float] [-min_capacitance float]
[-max_fanout integer] [-min_fanout integer]
[-max_transition float] [-min_transition float]
lib_pin_list
```

Description

Sets the specified attributes on the specified lib_pin(s).

This command is also aliased to set_lib_pin.

Options and Arguments

-max_capacitance *float*	
	Specifies the maximum capacitive load that can be driven by the specified lib_cell pins.
-max_fanout *integer*	
	Specifies the maximum fanout that can be driven by the specified lib_cell pins.
-max_transition *float*	
	Specifies the maximum transition time on the specified lib_cell pins
-min_capacitance *float*	
	Specifies the minimum capacitive load that can be driven by the specified lib_cell pins.
-min_fanout *integer*	
	Specifies the minimum fanout that can be driven by the specified lib_cell pins.
-min_transition *float*	
	Specifies the minimum transition time on the specified lib_cell pins
lib_pin_list	Specifies the lib_cell pins to which the specified constraints apply.

Related Information

Sets these attributes:	max_capacitance
	max_fanout
	max_transition
	min_capacitance
	min_fanout
	min_transition

set_load

```
set_load
{ [-pin_load] [-subtract_pin_load] [-wire_load]
{port...| hnet...| port_bus ...}
[-min] [-max] capacitance
```

Description

Sets the specified capacitance on the specified ports or nets of the top cell. The capacitance that you specify using this command overrides the capacitance from wire load models and SPEF.

If you omit both the `-min` and `-max` options, the capacitance applies for both setup and hold analysis.

For ports, the asserted capacitance is either the external pin capacitance when the -`pin_load` option is specified, or the external wire capacitance when the -`wire_load` option is specified, or the total external capacitance when neither option is specified.

Options and Arguments

capacitance	Specifies the capacitance. This is a floating number.
	The capacitance is in units from the target technology library.
`-max`	Specifies the maximum capacitance. The maximum capacitance is used to compute worst-case delays in BcWc analysis mode and late path delays in OCV mode. It is also used to check the maximum capacitance design rule.
`-min`	Specifies the minimum capacitance. The minimum capacitance is used to compute best-case delays in BcWc analysis mode and early path delays in OCV analysis mode.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-pin_load`	Specifies the load of pins connected to this port. Sets the `external_pin_cap` attribute on the port. This option applies only to ports.
	This is the default option if neither the -`pin_load` option nor the -`wire_load` option is not specified.
{*port* | *hnet* | *port_bus*}	

	Specifies the port, net, or port_bus objects to which the specified capacitance applies.
`-subtract_pin_load`	
	Subtracts the capacitance of the pins connected to the net of the port from the specified capacitance value before it applies it to the ports. This option applies only to ports.
	Can be used if nets are specified in the object list.
`-wire_load`	Interprets the specified capacitance as the external wire capacitance of the port. Use this option only for ports. Use this option with the `port_net_list` argument if the list contains only ports. Sets the `external_wire_cap` attribute on the port.

Examples

- The following command sets an external minimum pin capacitance of 1.5 units on the in1in2 ports:

  ```
  set_load 1.5 -min -pin_load [get_ports {in1 in2}]
  ```

- The following command sets an external minimum wire capacitance of 1.2 units on the in1in2 ports:

  ```
  set_load 1.2 -min -wire_load [get_ports {in1 in2}]
  ```

- The following command sets an external maximum pin capacitance of 2.6 units on the in1in2 ports:

  ```
  set_load 2.6 -max -pin_load [get_ports {in1 in2}]
  ```

- The following command sets an external maximum wire capacitance of 3.0 units on the in1in2 ports:

  ```
  set_load 3.0 -max -wire_load [get_ports {in1 in2}]
  ```

Related Information

Sets these attributes:	external_pin_cap
	external_wire_cap
Related commands:	all_outputs
	report_port
	set_drive
	set_resistance

set_load_unit

```
set_load_unit
{-picofarads | -femtofarads } [float]
```

Description

Specifies the load unit used inside the SDC file or as part of the sdc command. By default, the time and load units use the read_sdc command's units whereas when using SDC commands interactively, the units of the first technology library are used.

> Although supported by the `read_sdc` command, it is *not* an official SDC command.

Options and Arguments

float	Indicates the number that any load value in subsequent SDC commands should be multiplied with.
-femtofarads	Indicates that the load values are specified in femto farads.
-picofarads	Indicates that the load values are specified in pico farads.

Example

Interactively you can override the load units using the following command. This command will overwrite the units read-in from the SDC file.

```
set_load_unit -femtofarads 1.0
```

```
set_load_unit -picofarads 1.0
```

set_logic_dc

```
set_logic_dc
{pin | hpin | port} ...
```

Description

If the specified object is an output port or an input pin, connects the object to constant 0.

If the specified object is an input port or an output pin, connects the loads of these objects to constant 0.

> This command does not work on pins or ports with multi-driver nets.

Options and Arguments

*pin	hpin*	Specifies the name of the pin(s) to which the command applies.
port	Specifies the name of the port(s) to which the command applies.

set_logic_one

```
set_logic_one
{pin | hpin | port} ...
```

Description

Sets ports or pins to logic 1 in the design.

If the specified object is an output port or an input pin, connects the object to constant 1.

If the specified object is an input port or an output pin, connects the loads of these objects to constant 1.

> This command does not work on pins or ports with multi-driver nets.

Options and Arguments

*pin	hpin	port*	Specifies the name of the pin(s) or port(s) that are to be set to logic 1.

Example

The following command sets the specified port to logic 1:

```
set_logic_one [get_ports {din[1]}]
```

Related Information

Related command:	report_case_analysis

set_logic_zero

```
set_logic_zero
{pin | hpin | port} ...
```

Description

Sets ports or pins to logic 0 in the design.

If the specified object is an output port or an input pin, connects the object to constant 0.

If the specified object is an input port or an output pin, connects the loads of these objects to constant 0.

> This command does not work on pins or ports with multi-driver nets.

Options and Arguments

*pin	hpin	port*	Specifies the name of the pin(s) or port(s) to which the command applies.

Example

The following command sets the specified port to logic 0:

```
set_logic_zero [get_ports {din[0]}]
```

Related Information

Related command:	report_case_analysis

set_max_capacitance

```
set_max_capacitance
[-rise] [-fall]
[-clock_path] [-data_path]
float {design | port | pin | hpin | clock | lib_pin}...
```

Description

Specifies the maximum capacitance that can be driven by all ports in the specified design(s) or that can be driven by the specified ports.

Sets the `max_capacitance` attribute on the specified design(s) or port(s).

Options and Arguments

clock	Specifies the name of the clock to which the maximum capacitance design rule constraint applies.
-clock_path	Specifies to apply this design rule constraint to all pins that are in the network of the specific clocks in the object list.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
-data_path	Specifies to apply this design rule constraint to all pins that are in the data paths launched by the specific clocks in the object list.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
design	Specifies the name of the design to which the maximum capacitance design rule constraint applies.
-fall	Indicates that the specified value applies to the falling capacitance.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
float	Specifies the maximum capacitance value.
lib_pin	Specifies the name of the library cell pin to which the maximum capacitance design rule constraint applies.
pin|hpin	Specifies the name of the pin to which the maximum capacitance design rule constraint applies.
port	Specifies the names of the ports to which the maximum capacitance design rule constraint applies.

`-rise`	Indicates that the specified value applies to the rising capacitance.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

Example

The following command constrains the maximum capacitance that can be driven by port `in_port_1` to `0.5`.

```
set_max_capacitance 0.5 [get_ports in_port_1]
```

set_max_delay

```
set_max_delay
{{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-rise_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-fall_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-rise] [-fall] [-ignore_clock_latency]
[-exception_name name] [-comment string]
[-combinational_from_to] [-reset_path] delay_value
```

Description

Specifies a maximum delay target for a timing path. The `set_max_delay` command takes precedence over other path exceptions specified for the same path, except for the set_false_path command.

Use either the `-from`, `-to`, or `-through` option with the `set_max_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the `clk` pin. This means the latency and delay from `clk` to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding /Q after the cell name. In this case, path from `clk` to Q is not considered in the calculation.

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Options and Arguments

`-combinational_from_to`	
	Defines a path defined by `-from` and `-to` pins as combinational.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
delay_value	Specifies the maximum delay value.
`-exception_name` *name*	
	Specifies the name of the timing exception you want to create.
	Default: `del_n`
`-fall`	Apply the assertion at the falling edge of the signal on the path endpoints.
`-fall_from	-rise_from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Specifies a list of timing paths start points. The argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries.
	The argument can be a collection.
	The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell • Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells.
	A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with set_input_delay
	`-rise_from`: Applies the maximum delay constraint at the rising edge of the signal on the from pins. `-fall_from`: Applies the maximum delay constraint at the falling edge of the signal on the from pins.
`-from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Applies the maximum delay constraint to all paths originating from the from argument list. If a clock is specified as the `-from` object, all primary inputs and registers clocked by that clock are used as start points.
	By default the `-from` option applies the constraint to both the rising and the falling edges.
`-fall_to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a falling edge at the specified end points.The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-ignore_clock_latency`	
	Specifies to ignore clock latency.

`-reset_path`	Resets any other exceptions (`false_path` and `multicycle`) on the path specified using the `-from`, `-through`, `-to` options and applies a `set_max_delay` exception for these paths.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-rise_from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-rise_to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge on the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-through | -rise_through | -fall_through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Specifies a list of timing paths through points.
	The argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The argument can be a collection.
	When you use this parameter without using the -`from` or -`to` options, all paths that go through the specified pins have the specified delay.
	When you use it with both the -`from` and -`to` options, any path starting at any pin on the -`from` list and going through any point on the -`through` pin list and going to any point on the -`to` list is affected by the constraint.
	By default the -`through` option applies the constraint to both the rising and the falling edges. -`rise_through`: Applies the assertion at the rising edge of the signal on the through pins. -`fall_through`: Applies the assertion at the falling edge of the signal on the through pins.
`-to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.

Examples

- The following command specifies a maximum delay of 10 time units to the path that starts from `inst1/CK` and ends at `inst2/D`.

```
set_max_delay  -from inst1/CK  -to inst2/D 10 -comment "my max delay"
```

- The following command specifies a maximum delay of 0.3 time units to the output port that has `output_delay` value also and the timing report along with the exception is shown below:

```
set_max_delay 0.3 -to port_pad_data_out[9]
exception:dtmf_recvr_core/del_1

report_timing -to [filter_collection [get_db ports *] {direction==out}] -encounter

Path 1: VIOLATED Late External Delay Assertion
Endpoint:      port_pad_data_out[9]                                (v)
Beginpoint:    TDSP_CORE_INST/DATA_BUS_MACH_INST_data_out_reg[9]/Q    (v)

Other End Arrival Time          0
- External Delay                300
+ Path Delay                    300
= Required Time                 0
- Arrival Time                  899
= Slack Time                   -899
```

Related Information

Related attribute:	comment

set_max_dynamic_power

```
set_max_dynamic_power
float unit [design]
```

Description

Specifies the maximum dynamic power constraint for the design and sets the `max_dynamic_power` design-level attribute. When a maximum dynamic constraint is set, Genus performs timing and dynamic power optimization simultaneously at all optimization stages like global mapping, refine mapping and incremental mapping. During incremental optimization, the value of the constraint is considered, while the constraint value is not taken into account during mapping. During global and refine mapping, logic on the non-critical paths is mapped to gates with less dynamic power dissipation. But during incremental mapping, the Genus-Low Power engine performs additional transformations including gate up-sizing or down-sizing, buffer removal and logic restructuring to further reduce the dynamic power dissipation of the design.

When both `max_leakage` and `max_dynamic` attributes are set, you must to set a weight factor using `lp_power_optimization_weight` design-attribute, so that Genus weighs the optimization cost function accordingly. If no weight factor is set, Genus optimizes only for the leakage power.

Options and Arguments

design	Specifies the name of the design to which the constraint applies.
float	Specifies the maximum dynamic power value.
unit	Specifies the power units. Following units can be specified: `pW`, `nW`, and `uW`.

Example

The following command sets the maximum dynamic power constraint to 0.5 pW.

```
set_max_dynamic_power 0.5pW
```

Related Information

Related command:	set_max_leakage_power

set_max_fanout

```
set_max_fanout
float {design | port | lib_pin}...
```

Description

Sets the maximum fanout load limit constraints on the specified instance pin, ports of the top cell, the specified modules, and/or clock waveforms, or lib pins. This constraint can be specified in SDC. When the net name is not specified in the constraint, then the constraints will be applied to all the nets in the design.

Sets the `max_fanout` attribute on the specified design(s) or port(s).

When set on a port, the fanout load of the net connected to the port must be less than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. Libraries and library cell pins may also have the maximum fanout load limit. In this case the most constraining limit is applied to the nets.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. You cannot apply the constraint on input pins and output ports.

Options and Arguments

design	Specifies the name of the design to which the maximum fanout design rule constraint applies.
float	Specifies the value of the maximum fanout load limit constraint. This is a float number and it must be greater than or equal to zero.
lib_pin	Specifies the name of the library cell pinto which the maximum fanout design rule constraint applies.
port	Specifies the names of the input ports to which the maximum fanout design rule constraint applies.

Examples

- The following command constrains the maximum fanout that can be driven by port `in_port_1` to `2.0`.

  ```
  set_max_fanout 2.0 [get_ports in_port_1]
  ```

- The following example sets a maximum fanout load limit of 5.0 units on all the ports of the top design instance or top module:

  ```
  set_max_fanout 5.0 [get_ports *]
  ```

Related Information

Related commands:	report_port
	set_fanout_load
	set_max_capacitance
	set_max_transition

set_max_leakage_power

```
set_max_leakage_power
float unit [design]
```

Description

Specifies the maximum leakage power constraint for the design and sets the `max_leakage_power` design-attribute. When the leakage power constraint is set, Genus performs timing, area, and leakage power optimizations simultaneously during mapping without taking the constraint value into account. During global incremental and incremental optimization, the value of the constraint is considered.

When both `max_leakage` and `max_dynamic` attributes are set, you have to set a weight factor using `lp_power_optimization_weight` design-attribute so that Genus weighs the optimization cost function accordingly. If no weight factor is set, Genus optimizes only for the leakage power.

Options and Arguments

design	Specifies the name of the design to which the constraint applies.
float	Specifies the maximum leakage power value.
unit	Specifies the power units. Following units can be specified: `pW`, `nW`, and `uW`.

Related Information

Related command:	set_max_dynamic_power

set_max_time_borrow

```
set_max_time_borrow
    -interface
    float
    [clock |inst | pin|...
```

Description

Specifies the maximum time that can be borrowed by one stage from the next logic stage following a latch to meet timing constraints. Time borrowing is sometimes called cycle stealing. Time borrowing is performed automatically when performing timing analysis.

Sets the `latch_max_borrow` attribute.

> If the `latch_borrow` attribute has already been set on the specified object then the setting from this command is ignored.

Use the `set_max_time_borrow` assertion on pins or waveforms. If the assertion is placed on a clock waveform, all latches triggered by this clock signal get the specified maximum time borrow limit. If both the clock waveform arriving at the clock pin of a latch and the clock pin itself have time borrow limit constraints, the maximum borrow limit on the latch is decided by the assertion on the latch clock pin.

Options and Arguments

-interface	Specifies the I/O paths (bool, optional)
{*clock* | *inst* | *pin* }	
	Specifies the object(s) to which the borrow value applies.
	You can set the borrow value for latch instances, clocks, latch enable pins, data pins.
	If the command is set on multiple objects that overlap each other, for example a latch instance and its data pin, the minimum value will be taken.
float	Specifies the borrow value to be applied.

Example

```
set_max_time_borrow 2.5 [get_clocks CLK1]

or

set_max_time_borrow 0 [get_pins LOCKUP/ENA]
```

Related Information

Sets this attribute:	latch_max_borrow

set_max_transition

```
set_max_transition
[-clock_path] [-data_path]
[-rise] [-fall]
float [design | clock | port | pin | hpin | lib_pin]...
```

Description

Specifies the maximum transition design rule for the specified design or port list.

Options and Arguments

`-clock_path`	Applies the max transition constraint only to pins in the clock network of the clocks defined in the object list.
clock	Specifies the name of the clock to which the maximum transition design rule constraint applies.
`-data_path`	Applies the max transition constraint only to data path pins, where the path is launched by the clocks defined in the object list.
design	Specifies the name of the design to which the maximum transition design rule constraint applies.
`-fall`	Applies the max transition constraint only to falling transitions.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
float	Specifies the maximum transition.
	The units are either the units set with the `set_time_unit` command or the units of the first technology library read in.
lib_pin	Specifies the name of the library cell pin to which the maximum transition design rule constraint applies.
pin|hpin	Specifies the name of the pin to which the maximum transition design rule constraint applies.
`-rise`	Applies the max transition constraint only to rising transitions.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
port	Specifies the name of the port to which the maximum transition design rule constraint applies.

Example

The following command sets the maximum transition on the design to 4.

```
genus:root:> set_max_transition 4
```

```
genus:root:> get_db design:buzz max_transition4000.0
```

In this example, the time unit that applied when the command was given was nanoseconds, while in the tool the value is stored in picoseconds.

Related Information

Sets this attribute:	max_transition

set_min_capacitance

```
set_min_capacitance
[-rise] [-fall]
[-clock_path] [-data_path]
float [<string>]
```

Description

Specifies the minimum capacitance that can be driven by all ports in the specified design(s) or that can be driven by the specified ports.

> Genus ignores this command for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the command out for consumption by downstream tools.

Options and Arguments

clock	Specifies the name of the clock to which this design rule constraint applies.
-clock_path	Specifies to apply this design rule constraint to all pins that are in the network of the specific clocks in the object list.
-data_path	Specifies to apply this design rule constraint to all pins that are in the data paths launched by the specific clocks in the object list.
string	Specifies the name of the design, port, pin lib pin or instance to which this design rule constraint applies.
-fall	Indicates that the specified value applies to the falling capacitance.
float	Specifies the minimum capacitance value.
-rise	Indicates that the specified value applies to the rising capacitance.

Example

The following command constrains the minimum capacitance that can be driven by port `in_port_1` to `0.5`.

```
set_min_capacitance 0.5 [get_ports in_port_1]
```

set_min_delay

```
set_min_delay
{{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-rise_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-fall_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-rise] [-fall] [-ignore_clock_latency]
[-exception_name name] [-comment string]
[-combinational_from_to] [-reset_path] float
```

Description

Constrains the specified timing paths by the given delay value for hold analysis. This value is not used for timing analysis by Genus. But it will be written out using `write_sdc`.

Options and Arguments

`-combinational_from_to`	
	Defines a path defined by `-from` and `-to` pins as combinational.
`-comment` *`string`*	Specifies a comment to be tagged to this command.
`-exception_name`	Specifies the name of the timing exception you want to create.
	Default: `del_n`
`-fall`	Limits the path selection to those paths who have a falling edge at the end points.
`-fall_from` {*`clock`*|*`port`*|*`inst`*|*`hinst`*|*`pin`*|*`hpin`*|*`port_bus`*|*`hpin_bus`*}	
	Selects all paths whose signals have a falling edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-fall_through` {*`port`*|*`inst`*|*`hinst`*|*`pin`*|*`hpin`*|*`hnet`*|*`port_bus`*|*`hpin_bus`*}	
	Selects all paths whose signals have a falling edge on the specified through points. The through points can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-fall_to` {*`clock`*|*`port`*|*`inst`*|*`hinst`*|*`pin`*|*`hpin`*|*`port_bus`*|*`hpin_bus`*}	
	Selects all paths whose signals have a falling edge at the specified end points.The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`float`	Specifies the delay constraint value, in picoseconds, by which the path has to be constrained for hold analysis.
	If the endpoint has an output external delay specified, that delay is added into the path delay.
`-from` {*`clock`*|*`port`*|*`inst`*|*`hinst`*|*`pin`*|*`hpin`*|*`port_bus`*|*`hpin_bus`*}	
	Selects all paths that start from the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-ignore_clock_latency`	
	Specifies to ignore clock latency.
`-reset_path`	Resets any other exceptions (`false_path` and `multicycle`) on the path specified using the `-from`, `-through`, `-to` options and applies a `set_max_delay` exception for these paths.
`-rise`	Limits the path selection to those paths who have a rising edge at the path end points.
`-rise_from` {*`clock`*|*`port`*|*`inst`*|*`hinst`*|*`pin`*|*`hpin`*|*`port_bus`*|*`hpin_bus`*}	
	Selects all paths whose signals have a rising edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-rise_through` {*`port`*|*`inst`*|*`hinst`*|*`pin`*|*`hpin`*|*`hnet`*|*`port_bus`*|*`hpin_bus`*}	

	Selects all paths whose signals have a rising edge on the specified through points. The through points can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-rise_to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge on the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list of end points.
`-through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths that traverse through the specified points. The points to traverse can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
`-to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list of end points.

Related Information

Related command:	set_max_delay
Related attribute:	comment

set_min_fanout

```
set_min_fanout
float {design | port | lib_pin}...
```

Description

Specifies the minimum fanout that can be driven by all input ports in the specified design(s) or that can be driven by the specified input ports.

> Genus ignores this command for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the command out for consumption by downstream tools.

Options and Arguments

design	Specifies the name of the design to which the minimum fanout design rule constraint applies.
float	Specifies the minimum fanout value.
lib_pin	Specifies the name of the library cell pin to which the minimum fanout design rule constraint applies.
port	Specifies the names of the input ports to which the minimum fanout design rule constraint applies.

Example

The following command constrains the minimum fanout that can be driven by port `in_port_1` to `2.0`.

```
set_min_fanout 2.0 [get_ports in_port_1]
```

set_min_pulse_width

```
set_min_pulse_width
 [-low] [-high] {design| pin | hpin | port | clock} ... float
```

Description

Selects the most restrictive value when multiple pulse width checks are set.

> This constraint is not used for timing analysis by Genus. The command will be read with `read_sdc` and will be written out with `write_sdc`.

Options and Arguments

float	Specifies the non negative floating minimum pulse width value. No default value is assumed for minimum pulse width checks.
-high	Applies the maximum pulse width check to high signal levels only.
-low	Applies the minimum pulse width check to low signal levels only.
	> If you specify neither the `-low` or the `-high` option, the constraint applies to both the low and high pulses of the clock signals
{*pin	design	hpin	port	clock*}	
	Specifies the list of pins, ports, design or clock for which the minimum pulse width constraint must be checked.

Examples

- The following command sets a minimum pulse width, both low and high, check of 1.0 to the clock network of the current design:

  ```
  set_min_pulse_width 1.0
  ```

- The following command sets a minimum pulse width low check of 2.0 to all the pins clocked by the CLK1 clock:

  ```
  set_min_pulse_width -low 2.0 [get_clocks CLK1]
  ```

Related Information

Related commands:	report_min_pulse_width
	report_timing

set_min_transition

```
set_min_transition
[-clock_path] [-data_path]
[-rise] [-fall] float [design | clock | port | lib_pin]...
```

Description

Specifies the minimum transition design rule for the specified design or port list.

> Genus ignores this command for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the command out for consumption by downstream tools.

Options and Arguments

`-clock_path`	Applies the minimum transition constraint only to pins in the clock network of the clocks defined in the object list.
clock	Specifies the name of the clock to which the minimum transition design rule constraint applies.
`-data_path`	Applies the minimum transition constraint only to data path pins, where the path is launched by the clocks defined in the object list.
design	Specifies the name of the design to which the minimum transition design rule constraint applies.
`-fall`	Applies the minimum transition constraint only to falling transitions.
float	Specifies the minimum transition.
	The units are either the units set with the `set_time_unit` command or the units of the first technology library read in.
lib_pin	Specifies the name of the library cell pin to which the minimum transition design rule constraint applies.
`-rise`	Applies the minimum transition constraint only to rising transitions.
port	Specifies the names of the ports to which the minimum transition design rule constraint applies.

set_mode

```
set_mode
[-type cell]
list_of_modes
object_list
```

Description

Specifies the Liberty timing library modes to be active for a specific instance.

A library mode is a subset of active timing arcs that is defined in the timing library. Library modes are similar to conditional timing arcs, but do not require the application of constants to be set. Instead of changing a cell's mode by applying constants to its input pins, you can use this command to refer to sets of active arcs by name (such as READ or WRITE, for a memory cell).

> By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive.

To reset the library modes for an instance, use the `reset_mode` command. Specify new value for the specified object attribute elements.

> Setting the library mode at the instance level is the only form of `set_mode` that is currently supported. This parameter exists to support constraints coming from external sources.

Options and Arguments

list_of_modes	Specifies the name of the library mode value defined in the timing library. You can specify one or more mode values, but only one mode value per defined mode group in the Liberty library.
object_list	Specifies the instance list to which to apply the library mode.
`-type` *cell*	Specifies the Liberty cell type of the instance.

Example

The following command specifies that only the timing arcs consistent with the `tst_mode_disable` library mode are active for instances of cell type `i_ram`. The `tst_mode_enable` library mode is made inactive.

```
set_mode tst_mode_disable [get_cells i_ram]
```

Related Information

Related command:	reset_mode

set_module_model

```
set_module_model [-type string] [-tag string] [-default_dir string] [-default_rc_corner string] [-cell string] [-add_on string]
```

Description

Specifies settings for the module_model to read and write.

Use this command before module model read/write to specify settings for module model with corresponding options.

Options and Arguments

`-type` *string*	Specifies the type of the module model.

`-tag string`	Specifies the tag name of module model.
`-default_dir string`	Specifies the default directory of the module model.
`-default_rc_corner string`	
	Specifies the name of the corner.
`-add_on string`	Specifies the add-on collateral to load.
`-cell string`	Specifies the name of the module.

Related Information

Related commands:	commit_module_model
	read_module_model
	write_module_model

set_multicycle_path

```
set_multicycle_path
{{-from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_from {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-rise_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...
 |-fall_to {clock|port|inst|hinst|pin|hpin|port_bus|hpin_bus}...}
|{-through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-rise_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...
 |-fall_through {port|inst|hinst|pin|hpin|hnet|port_bus|hpin_bus}...}...
[-rise] [-fall] [-setup] [-hold] [-start] [-end]
[-exception_name name] float
[-comment string] [-reset_path]
```

Description

Creates a timing exception object that overrides the default clock edge relationship for paths that meet the path selection criteria. Multicycle paths are timing paths that require more than a single cycle for execution. By default, all paths are considered as single cycle paths for setup analysis, that is, data launched at any edge of the clock should be captured by the next flop at the next rising edge of the clock on to the destination flop.

Options and Arguments

`-comment string`	Specifies a comment to be tagged to this command. The comment is retained by the tool and the same comment string is written out as part of the `set_multicycle_path` constraint when the constraints are dumped out using the `write_sdc` command.
`-end`	Specifies whether the multi-cycle path constraint must be applied relative to the ending clock edge for the path.
	By default, the setup constraint is applied relative to the ending clock edge for the path.

`-exception_name` *name*	
	Specifies the name of the timing exception you want to create.
	Default: `mc_n`
`-fall`	Limits the path selection to those paths who have a falling edge at the end points.
`-fall_from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a falling edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-fall_through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a falling edge on the specified through points. The through points can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
	-`fall_through` can be specified more than once in a single command.
`-fall_to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a falling edge at the specified end points.The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
float	Specifies the number of cycles allowed for the path for setup or hold with respect to start point or end point clock. -`setup` value is used for setup path calculations. -`hold` value is used for hold path calculations. If -`setup` or -`hold` is not specified, this value is used for `setup` and 0 is used for `hold`.
`-from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that start from the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list. For example: If a clock is specified, all primary inputs and flops related to that clock will be used as path startpoints.
`-hold`	Applies the multi-cycle path constraint for hold analysis.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	Default: setup
`-reset_path`	Resets any other exceptions (`false_path` and `path_delay`) on the path specified using the `-from`, and `-to` options and applies a `set_multicycle_path` exception for these paths.
	• -`to` option will reset all paths pointing to the specified end points. • -`from` option will reset all paths leading from the specified start points. • If -`from` and -`to` are used together, only paths between these points will be reset.
`-rise`	Limits the path selection to those paths who have a rising edge at the path end points.
`-rise_from` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge at the specified start points. The start points can be input ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list.
`-rise_through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	

	Selects all paths whose signals have a rising edge on the specified through points. The through points can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list.
	`-rise_through` can be specified more than once in a single command.
`-rise_to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths whose signals have a rising edge on the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list of end points.
`-setup`	Applies the multi-cycle path constraint for setup analysis. *Default*: setup
`-start`	Specifies whether the multi-cycle path constraint must be applied relative to the starting clock edge for the path.
	By default, the hold constraint is applied relative to the starting clock edge for the path.
`-through` {*port*|*inst*|*hinst*|*pin*|*hpin*|*hnet*|*port_bus*|*hpin_bus*}	
	Selects all paths that traverse through the specified points. The points to traverse can be ports of your design, hierarchical pins, pins on a sequential/mapped combinational cells, or sequential/mapped combinational instances or nets. Specify a Tcl list of start points. The multicycle values apply only to paths that pass through one of the points in the `through_list`. If `-through` option is specified multiple times, the multicycle values apply to paths that pass through a member of each `through_list` in the order in which the list was provided.
`-to` {*clock*|*port*|*inst*|*hinst*|*pin*|*hpin*|*port_bus*|*hpin_bus*}	
	Selects all paths that end at the specified end points. The end points can be output ports of your design, clock pins of flip-flops, clock objects, instances, or a combination of these. Specify a Tcl list of end points.

Examples

- The following command specifies that all paths from instance `ff1` to instance `ff2` require 2 cycles for execution for setup analysis.

  ```
  set_multicycle_path -setup 2 -from inst:top/ff1 -to isnt:top/ff2 -comment "my multicycle path"
  ```

- The setup multicycle constraint specifies that the path from FF0/CK to FF1/D can take up to three clock cycles to complete for a setup check. This implies that the design utilizes the required data from FF1/Q only every third cycle instead of every cycle.

  ```
  set_multicycle_path 3 -setup -from [get_pins FF0/Q] -to [get_pins FF1/D]
  ```

- The number of cycles denoted on a multicycle hold specifies how many clock cycles to move back from its default hold check edge (which is one active edge prior to the setup capture edge).

  ```
  set_multicycle_path 2 -hold -from [get_pins FF0/Q] -to [get_pins FF1/D]
  ```

- Since no -`hold` option is specified, the default option (-`setup`) is assumed. This implies that the setup multiplier is 2 and the hold multiplier is 0.

  ```
  set_multicycle_path 2 -from [get_pins FF0/CK] -to [get_pins FF3/D]
  ```

- The -`end` option specifies that the multicycle of 4 refers to the end point or the capture clock.

  ```
  set_multicycle_path 4 -setup -from [get_clocks CLKA] -to [get_clocks CLKB] -end
  ```

  ```
  set_multicycle_path 3 -hold -from [get_clocks CLKA] -to [get_clocks CLKB] -end
  ```

- The -`start` option refers to the launch clock and is the default for a multicycle hold..

  ```
  set_multicycle_path 2 -setup \
  ```

  ```
  -from [get_clocks CLKA] -to [get_clocks CLKB] -start
  ```

  ```
  set_multicycle_path 1 -hold \
  ```

  ```
  -from [get_clocks CLKA] -to [get_clocks CLKB] -start
  ```

- All paths which first pass through M1 or M2 and then passes through M3 will be affected by the given constraint. Path leading in a rise transition at an endpoint is constrained to two cycles, falling paths are constrained to one cycle.

  ```
  set_multicycle_path 2 -rise -through {M1 M2} -through {M3}
  ```

  ```
  set_multicycle_path 1 -fall -through {M1 M2} -through {M3}
  ```

Related Information

Related attribute:	comment

set_operating_conditions

```
set_operating_conditions
[-analysis_type string] [-object_list string]
{ -max_library string [-min_library string ]
| -library string }
{-min string -max string |operating_condition}
```

Description

Specifies the operating conditions to be used for timing. Sets the `operating_conditions` attribute.

Options and Arguments

`-analysis_type` *string*	
	The only accepted value is `single`. This indicates that only one operating condition is to be used.
`-library` *string*	Specifies the library containing operating condition definitions.
`-max` *string*	Specifies the operating condition for setup analysis.
`-max_library` *string*	
	Specifies the library under which the operating condition is present. This operating condition is used for setup analysis.
`-min` *string*	Specifies the operating condition for hold analysis.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-min_library` *string*	
	Specifies the library under which the operating condition is present for hold analysis.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-object_list`	
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
operating_condition	
	Specifies the operating conditions to be used for timing analysis.

set_opposite

`set_opposite`
port port

Description

Declares the two specified ports to be logically opposite.

The command does not work on output ports and ports with multidriven nets.

set_output_delay

```
set_output_delay
[-clock clock]
[-clock_fall] [-clock_rise]
[-level_sensitive]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin|port ...]
[-rise] [-fall] [-max] [-min]
[-add_delay] [-name name] [-group_path group]delay {port|pin|hpin|port_bus|hpin_bus}...
```

Description

Constrains output and inout ports, port busses, and pins (that are valid end points) within the design relative to a clock edge.

If you omit both the `-min` and `-max` options, the delay is assumed to apply for both setup and hold analysis.

If you omit both the `-rise` and `-fall` options, the delay applies for both the rising and falling edges of the output signal.

> This command also sets the `external_delays` attribute on the specified pins or ports.

Options and Arguments

`-add_delay`	Specifies to add the specified output delay information for the specified pins or ports.
	Use this option to capture the delay information if multiple paths lead to an output pin or port that are relative to different clocks or clock edges.
	If you omit this option, the specified delay information overwrites all existing output delays for the specified pins or ports.
`-clock clock`	Specifies the reference clock. The output delay is defined relative to this clock.
	If no clocks are specified, the timing paths to the specified pins and ports will be unconstrained as captured in the `report_timing` report.
`-clock_fall`	Specifies to use the falling edge of the reference clock as reference edge.
	Default: `-clock_rise`
`-clock_rise`	Specifies to use the rising edge of the reference clock as reference edge.
	Default: `-clock_rise`
delay	Specifies the output delay value. This is a floating number.
`-fall`	Specifies that the delay is measured with respect to the falling edge of the output signal.
`-group_path group`	Specifies the name of the group to which paths starting at the specified ports or pins must be added.
`-level_sensitive`	Specifies that the constraint goes to a level-sensitive latch.
	By default, the constraint goes to an edge-triggered flip-flop
`-max`	Indicates that the specified delay applies for setup analysis.

`-min`	Indicates that the specified delay applies for hold analysis.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-name` *name*	Associates a name with the specified timing constraint.
`-network_latency_included`	
	Specifies that the output delay includes the clock network latency values. This implies that the tool does not need to consider the clock network latency values when optimizing this path.
	Sets the `clock_network_latency_included` attribute to `true`.
{*port* | *pin* | *hpin* | *port_bus* | *hpin_bus*}	
	Specifies the output and inout pins, ports, and port buses to which the specified output delay value applies.
`-reference_pin` {*pin* | *port*}	
	Adds source and network latency of the specified reference pin to the output delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the -clock parameter.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-rise`	Specifies that the delay is measured with respect to the rising edge of the output signal.
`-source_latency_included`	
	Specifies that the output delay includes the clock source latency values. This implies that the tool does not need to consider the clock source latency values when optimizing this path.
	Sets the `clock_source_latency_included` attribute to `true`.

Related Information

Sets these attributes:	clock_network_latency_included
	clock_source_latency_included
	external_delays

set_path_group_options

```
set_path_group_options_fast
[-early]
[-late]
string
[-effort_level {low|high}]
[-weight integer]
[-skewing_slack_constraint float]
[-slack_adjustment float]
[-slack_adjustment_priority integer]
[-target_slack float]
```

Description

Specifies options to guide timing optimization and timing analysis when path groups are used. To reset parameters to their default value, use `reset_path_group_options`.

Options and Arguments

`-early`	Specifies early mode. **Note:** • If you do not specify either `-early` or `-late`, then `-late` is implied. • If you specify `-early`, then `-early` is used. • If you specify `-late`, then `-late` is used. • If you specify both `-early` and `-late`, then both `-early` and `-late` are used, which means the same value of `-early` and `-late` are set at the same time.
-late	Specifies late mode. **Note:** • If you do not specify either `-early` or `-late`, then `-late` is implied. • If you specify `-early`, then `-early` is used. • If you specify `-late`, then `-late` is used. • If you specify both `-early` and `-late`, then both `-early` and `-late` are used, which means the same value of `-early` and `-late` are set at the same time.
string	
-effort_level {low|high}	

	Specifies the effort applied to close timing on a specific path group. This parameter is honored by timing optimization (`opt_design`). Select one of the following options: `low`: Specifies that this path group is optimized together with other path groups for best overall slack. `low` is the default effort value for all user-created path groups. Cadence recommends you specify low effort for all path groups except the ones that the tool should focus on most strongly. `high`: Specifies that this path group is optimized separately from the low-effort path groups in order to achieve the best possible WNS for this path group. Default: `low` Data_type: enum, optional
-weight *integer*	Defines the optimization priority.
-skewing_slack_constraint *float*	
	When set, applies the specified skewing slack constraint to the path group. The value specified is in nanoseconds. If not set, no skewing slack constraint is applied to the path group.
-slack_adjustment *float*	
	Specifies a positive or negative value used to modify path constraints on all paths that are part of a specific path group. To relax the path constraints computed by the timing engine, specify a positive value. To tighten the timing constraints, specify a negative value.
-slack_adjustment_priority *integer*	
	Provides priority to a path group for the slack adjustment feature. The paths that are a part of overlapping path groups get slack adjustment from the path group that has a higher priority value.
-target_slack *float*	
	Specifies a target value for setup slack, for all paths that are part of a specific path group. The value can be positive or negative. This parameter is honored by timing optimization (`opt_design`). Using this parameter can impact the run time during timing optimization.

Examples

- ```
  set_path_group_options reg2reg -weight 10 -view analysis_view:Top/setup_analysis_view
  ```
- ```
  set_path_group_options reg2reg -early -skewing_slack_constraint 20
  ```

set_port_fanout_number

```
set_port_fanout_number integer port_list
```

Description

Sets the number of external fanout pins for ports in the current design.

The number of pins is used (along with wire load models) to calculate capacitance and resistance of nets.

The number of pins minus one is equal to the fanout of this port outside the design. This value is set on the `external_wireload_fanout` attribute.

Options and Arguments

integer	Specifies the number of external fanout pins. Since the value is the number of pins in the fanout of the port, it is expected to be an integer.
port_list	Specifies the ports for which the number is specified.

Example

The following command sets maximum fanout of a port.

```
set_port_fanout_number 10 [get_ports GPIO*]
```

Related Information

Sets this attribute:	external_wireload_fanout

set_resistance

```
set_resistance
{net | hnet} ...
[-min] [-max] resistance
```

Description

Sets the wire resistance to the specified value on the specified nets and hnets in the design. If you omit both the `-min` and `-max` options, the resistance is used as the minimum and maximum resistance.

> Genus ignores this command for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the command out for consumption by downstream tools.

Options and Arguments

`-max`	Specifies the maximum resistance.
`-min`	Specifies the minimum resistance.
{*net*|*hnet*}	Specifies a list of nets on which the wire resistance must be set.
resistance	Specifies the value for the wire resistance. This value is in units found in the target technology library.

Related Information

Sets this attribute:	resistance

set_sense

```
set_sense
[-clocks clock_list]
[-positive|-negative]
[-stop_propagation]
[-type string...]
[-clock_leaf]
{arc|pin|hpin|port}...
```

Description

Specifies clock phase or polarity with respect to the source clock from the specified pin in the forward direction only. This command selects which phase of the clock to filter at the specified point.

Options and Arguments

`-negative`	Specifies that the negative phase filtered clocks pass through pins in the `pin_list`.	
`-clocks clock_list`		
	Specifies the list of clocks passing through the specified point to which the constraint applies.	
`-positive`	Specifies that the positive phase filtered clocks pass through pins in the `pin_list`.	
`-stop_propagation`	Stops the propagation of specified clocks and clock source data paths passing through the specified pins.	
`-type string`	Specifies the type of stop (`all`, `clock`, `data`, `clock_source_data`) applied.	
	`all`	Stops all phases from propagating. This is equivalent to specify both the clock, `clock_source_data` and `data` options.
	`clock`	Stops clock phases launched by clock(s) specified in the clock_list from propagating.
	`data`	Stops all data phases launched by clock(s) specified in `clock_list`, starting from the pins or ports specified in the `pin_or_port_list`. The objects in the `pin_or_port_list` can be selected anywhere in the design - clock or data network.
	`clock_source_data`	Stops only clock source data phases from propagating. The combination of `clock_source_data` and clock is equivalent to the existing `-stop_propagation` option.
	`default_data`	
-clock_leaf	Adds the clock on leaf and data-paths from the specified pins.	
{arc|pin|hpin|port}		
	Specifies the list or collection of cell output pins. The unateness specified by the `set_sense` command is applied to clocks propagating through the specified point.	

	Genus supports this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

Examples

- The following command stops the propagation of clock CLK1 through the pin CLKBUF1/Z:

  ```
  genus:root:> set_sense -type clock -clocks CLK1 CLKBUF1/Z -stop_propagation
  ```

- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:

  ```
  genus:root:> set_sense -positive -clocks CLK1 CLKBUF1/Z
  ```

- The following command stops data phases based only on CK1 and CK2 starting from pin1:

  ```
  genus:root:> set_sense pin1 -type {data} -clocks {CK1 CK2} -stop_propagation
  ```

- The following command stops only the clock source data phases starting from pin1:

  ```
  genus:root:> set_sense pin1 -type {clock_source_data} -stop_propagation
  ```

Related Information

Related command:	reset_sense

set_size_only

```
set_size_only
{module|inst|hinst|hnet|design|lib_cell}...
```

Description

Sets the `preserve` attribute for an object to `map_size_ok` if the cell is an unmapped sequential cell (unmapped combinational cells are ignored), and sets the `preserve` attribute to `size_ok` if the cell is mapped.

The command `set_size_only` sets the attribute preserve to `map_size_ok` for a mapped instance only when the instance is totally unpreserved, that is when `preserve` = `FALSE`. Otherwise, it doesn't change the preserve attribute.

Options and Arguments

`{module|inst|hinst|hnet|design|lib_cell}...`	
	Specifies the object(s) to be considered for sizing.

Examples

- The following example shows that when the instance is totally unpreserved, that is, when attribute `preserve` is `false`, the `set_size_only` sets the attribute `preserve` to `map_size_ok` for a mapped instance only:

```
genus:root:> get_db inst:dtmf_recvr_core/ARB_INST/g667__4296 .preserve
false

genus:root:> set_size_only inst:dtmf_recvr_core/ARB_INST/g667__4296

genus:root:> get_db inst:dtmf_recvr_core/ARB_INST/g667__4296 .preserve
size_ok
```

- The following example shows that when the attribute `preserve` is `true` on any instance then the command `set_size_only` does not change the attribute `preserve` to `map_size_ok` for a mapped instance only:

```
genus:root:> get_db inst:dtmf_recvr_core/ARB_INST/g666__8780 .preserve
true

genus:root:> set_size_only inst:dtmf_recvr_core/ARB_INST/g666__8780

genus:root:> get_db inst:dtmf_recvr_core/ARB_INST/g666__8780 .preserve
true
```

set_time_unit

```
set_time_unit
{-picoseconds | -nanoseconds } [float]
```

Description

Specifies the time unit used inside the SDC file or as part of the `dc::` command

> Although supported by the `read_sdc` command, it is *not* an official SDC command.

Options and Arguments

float	Indicates the number that any timing value in subsequent SDC commands should be multiplied with.
-nanoseconds	Indicates that the timing values are specified in nanoseconds.
-picoseconds	Indicates that the timing values are specified in picoseconds.

set_timing_derate

```
set_timing_derate_c
[-cell_delay]
[-cell_check]
[-net_delay]
[-early]
[-late]
[-rise]
[-fall]
[-dynamic]
[-static]
[-clock]
[-data]
[-delay_corner delay_corner]
[-mean]
[-sigma]
[-power_domain power_domain]
[-transition]
[-add]
[-multiply]
[-incremental_adjust]
[-input_switching]
[-pulse_width_check]
[-clock_period_check]
float
string {lib_cell | lwlibcell | inst | hinst | design | hnet}...
```

Description

This command sets delay scaling or derating factors for early and late paths in the current design. You use this command to perform scaled on-chip variation around best-case and worst-case corners. The timing scaling factors affect the delay values generated in the timing reports.

You can set scaling factors for data paths, clock paths, minimum and maximum operating conditions.

> Omitting both the -cell_check and -cell_delay options results in the derating factor being applied to both delay and constraint arcs.

Genus only supports some of the option combinations, but does store all information and writes it out with the write_sdc command.

If you do not specify the -clock or -data parameter, the software applies the scaling factors to both clock and data paths. If you specify only one of the two parameters, -data or -clock, the software uses the last defined value for the other.

If you do not specify the -min or -max parameter, the software applies the scaling factor to both minimum and maximum operating conditions. You should only use the -min or -max parameter for best-case worst-case analysis. If you specify them in on-chip variation (OCV) mode, the software will use the -min -early derating factor for early path scaling, and the -max -late derating factor for late path adjustments.

When there are separate max and min libraries specified, only those set_timing_derate constraints are honored that are specified with max library. The constraints specified with min library are ignored.

In MMMC mode, for delay corners having different late and early library sets, only those set_timing_derate constraints are honored that are specified with late library set of the delay corner. The constraints specified with early library set are ignored.

Options and Arguments

-add	Specifies to add derate to existing derate value.
-cell_check	Specifies scaling on timing checks. Use the -cell_check option with the -early option to specify scaling on hold arcs and use with the -late option to specify scaling on setup arcs.

	When you specify this parameter, derating factors are also applied to user constraints such as `set_ouput_delay`, `set_data_check`, and `set_clock_gating_check`.
	The -`cell_check` parameter by default does not apply check derate to output external delay (e.g., set_ouput_delay). To enable this feature, you can set the `timing_apply_check_derate_to_external_output_delay` attribute to `true`. When set to `true`, this attribute applies cell derate to the top boundary port.
`-cell_delay`	Specifies scaling on cell delays.
	If neither the -`cell_delay` nor the -`net_delay` option is specified, the specified scaling factor applies to both types of delays
`-clock`	Applies the derating factors to clock paths only.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-data`	Applies the derating factor to data paths only.
`-delay_corner` *`delay_corner`*	
	Applies the derating factor to the specified delay calculation corner or a list of delay corners.
`design`	Specifies the name of the top-level design to which the derating factor applies.
`-dynamic`	Applies the specified derating factor to the delta portion of net delays only.
	You can specify -`net_delay` or -`cell_delay` parameter with the -`dynamic` parameter.
	You cannot specify -`dynamic` with -`mean` or -`sigma` parameters.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-early`	Specifies that the derating factor applies to early paths. Early paths are launching clock during hold analysis, receiving clock during setup analysis and datapath during hold analysis.
	Genus only supports this option in combination with the `-data` and `-cell_delay` options.
`-fall`	Applies the value only to the fall delays.
`float`	Specifies the derating value to be used.
	Specify a positive derating value. An error will be given if you specify a negative derating value.
`-incremental_adjust`	Specifies to incrementally apply the derate value.
`-input_switching`	Specifies the input switching derate value.
`-pulse_width_check`	Specifies the pulse width check derate value.
`-clock_period_check`	Specifies the clock period check derate value.
`inst|hinst|hnet`	Specifies the instance(s) to which the derating factor applies.
	Genus does *not* support this option in combination with the `-clock` and `-cell_check` options.

`-late`	Specifies that the derating factor applies to late paths. Late paths are receiving clock during hold analysis, launching clock during setup analysis and datapath during setup analysis.
	Genus does *not* support this option in combination with the `-clock` option.
lib_cell	Specifies the library cells to which the derating factor applies.
`-mean`	Specifies to apply separate derating value to the mean components of the delay.
`-multiply`	Specifies to multiply derate to existing derate value.
`-net_delay`	Specifies scaling on net delays.
	This option is only available in the physical flows.
`-power_domain` *power_domain*	
	Allows you to specify a specific power domain for the timing derate factors to be applied.
`-rise`	Applies the value only to the rise delays.
`-sigma`	Specifies to apply separate derating value to the sigma components of the delay.
`-static`	Applies the specified derating factor to the non-delta portion of the net delays only.
	You can specify this option with the `-cell_delay` and `-net_delay` options.
	You cannot specify -`static` with -`mean` or -`sigma` parameters.
`-transition`	Specifies the transition derate value.

Support Matrix

All option combinations are stored and written out during `write_sdc`.

Option combination			Combination supported?
`-early`	`-clock`	`-cell_delay`	Not supported.
`-early`	`-clock`	`-cell_check`	Not supported.
`-early`	`-data`	`-cell_delay`	Supported on instances and lib_cells for data-to-data checks.
`-early`	`-data`	`-cell_check`	Not supported.
`-late`	`-clock`	`-cell_delay`	Not supported.
`-late`	`-clock`	`-cell_check`	Not supported.
`-late`	`-data`	`-cell_delay`	Supported on instances and lib_cells.
`-late`	`-data`	`-cell_check`	Supported on lib_cells

- The following example uses two `set_timing_derate` commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:
  Command (1) adds pessimism to the setup analysis by making the latching clock path 20 percent faster.
  Command (2) adds pessimism to the hold analysis by making the latching clock path 10 percent slower.

  ```
  set_timing_derate -max -early 0.8 -late 1.0

  set_timing_derate -min -early 1.0 -late 1.1
  ```

- The following command sets the early path scaling factor of 0.85 for instance MULT_9 cell delays:

  ```
  set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
  ```

- The following command sets the early path scaling factor of 0.75 for the net named MUX2/Z:

  ```
  set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
  ```

- The following command specifies power domain PD1 for derating instances:

  ```
  set_timing_derate -power_domain PD1 -cell_delay 1.1
  ```

- In the following example, a design having max library as 'slow' and min library as 'fast', only the first two `set_timing_derate` constraints are honored:

  ```
  set_timing_derate -late 1.2 [get_lib_cells {slow/BUFX1}] -cell_delay

  set_timing_derate -early 0.8 [get_lib_cells {slow/BUFX1}] -cell_delay

  set_timing_derate -late 1.1 [get_lib_cells {fast/BUFX1}] -cell_delay

  set_timing_derate -early 0.9 [get_lib_cells {fast/BUFX1}] -cell_delay
  ```

- Consider the following library sets, in MMMC mode:

  ```
  create_library_set -name libset1 -timing slow.lib

  create_library_set -name libset2 -timing fast.lib

  create_delay_corner -name corner1 -late_library_set libset1 -early_library_set libset2
  ```

  From the following `set_timing_derate` constraints only the first two are honored.

  ```
  set_timing_derate -late 1.2 -cell_delay [get_lib_cells slow/*] -delay_corner corner1

  set_timing_derate -early 2.3 -cell_delay [get_lib_cells slow/*] -delay_corner corner1

  set_timing_derate -late 3.4 -cell_delay [get_lib_cells fast/*] -delay_corner corner1

  set_timing_derate -early 4.5 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
  ```

Related Information

Related command:	reset_timing_derate
Sets these attributes:	cell_delay_multiplier
	cell_min_delay_multiplier

set_unconnected

```
set_unconnected
port_list
```

Description

Edits the netlist by disconnecting the output port.

> Although supported by the `read_sdc` command, it is *not* an official SDC command.

Options and Arguments

port_list	Specifies the names of the output ports that must be disconnected.

Related Information

Related command:	check_design
	get_fanin
	get_fanout

set_units

```
set_units
[-capacitance string] [-resistance string] [-voltage string] [-current string] [-time string] [-power string]
```

Description

Specifies the unit of capacitance, resistance, voltage, current, timing and power in SDC. This command only affects the units in SDC, but not the units used in Genus calculations. For example, after `set_unit -time 1ns`, the timing unit used in Genus calculation will still be `ps`. By default, the units used in SDC file are read from the first library.

> This command was added in SDC 1.7.

Options and Arguments

`-capacitance` *string*	
	Specifies the capacitance unit. String can have the following format:
	`[`*float*`] {fF | pF | nF | uF | mF | kF | MF}`
`-current` *string*	Specifies the current unit.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-power` *string*	Specifies the power unit.

	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-resistance` *string*	Specifies the resistance unit.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-time` *string*	Specifies the time unit. String can have the following format:
	`[`*float*`] {fs | ps | ns | us | ms | ks | Ms}`
`-voltage` *string*	Specifies the voltage unit.
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.

Example

The following command sets units of time and capacitance

```
set_units -time ns -capacitance pf
```

set_wire_load_mode

```
set_wire_load_mode {top | enclosed| segmented}
```

Description

Sets the `wireload_mode` root attribute with the specified value. The `enclosed` mode is the default mode and if the `default_wire_load_mode` is specified in the first technology library, then, that value is used.

Options and Arguments

`enclosed`	Uses the wire-load model of the smallest block that fully encloses the net to compute the load of the net. Hierarchical boundary pins are not counted as fanouts.

`segmented`	Divides nets that cross hierarchical boundaries into segments with one segment for each level of hierarchy. Separate load values are computed for each segment (counting the hierarchical boundary pins as individual fanouts) and the load values are added together.
`top`	Uses the wire-load model of the top-level design for all nets in all modules. Hierarchical boundary pins are not counted as fanouts.

Related Information

Sets this attribute:	wireload_mode

set_wire_load_model

```
set_wire_load_model
-name {model_name | auto_select | none | inherit}
[-library library] [-min] [-max]
{design | module | inst | hinst | port}...
```

Description

Forces Genus to use a specific wire load model for the given design, modules, hierarchical instance or port.

Sets the `force_wireload` attribute if the specified object is a design, module or a hierarchical instance. If the specified object is a hierarchical instance, the attribute on the corresponding module is set.

Sets the `external_wireload_model` attribute if the specified object is a port.

Options and Arguments

design | *module* | *inst* | *hinst* | *port*	
	Specifies the design, module, hierarchical instance, or port for which you want to specify the wire load model.
	If no object is specified, the wire load model is applied to the top-level design.
`-library` *library*	
	Specifies the technology library to which the wire load model belongs.
`-max`	
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-min`	
	Genus ignores this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
`-name {`*model_name* | `auto_select` | `none` | `inherit}`	
	Specifies the name of the wire load model to use.

	`auto_select`	Automatically selects wire-load models according to the wire-load selection table or default wire-load model in the technology library.
		This value can only be applied for the design, modules, or hierarchical instances.
	`model_name`	Forces Genus to use the specified custom wire-load model. Specify the hierarchical path to the wire-load model to be used.
	`inherit`	Causes the same behavior as `auto_select`.
		This value can only be applied for the design, modules, or hierarchical instances.
	`none`	Prevents the use of any wire-load models.
		This value can only be applied for the design, modules, or hierarchical instances.

Examples

- The following command sets the `COM3_LV_WLM_0.0` wireload model from the `COM3_LV_W` library on the current module for all PVT corners:

  ```
  set_wire_load_model -library COM3_LV_W -name COM3_LV_WLM_0.0
  ```

- The following command sets the `B1X1` wireload from the `lca500kv` library on the `clk1` port for the minimum PVT corner:

  ```
  set_wire_load_model -library lca500kv -min -name B1X1 clk1
  ```

- The following command sets the `3M_TO_4500K` wireload from the `c13tsmc_slow2s108v125c` library on the current module for the minimum PVT corner:

  ```
  set_wire_load_model -library c13tsmc_slow2s108v125c -min -name 3M_TO_4500K
  ```

Related Information

Sets these attributes:	force_wireload
	external_wireload_model
Related attributes:	wireload_mode
	wireload_selection

set_wire_load_selection_group

```
set_wire_load_selection_group
[-library library] [-min] [-max]
[-object_list string] name
```

Description

Specifies a wire-load selection group. Sets the `wireload_selection` root attribute with the specified value.

Overrides the wireload selection table in the library specified by the `library_name` argument.

Options and Arguments

`-library` *library*	Specifies the name of the technology library to search for the wireload selection table. If not, search all libraries.
	If this option is omitted, the command searches for the specified wire-load selection table in the list of the libraries loaded. If the same table name appears in multiple libraries, the selection table from the earlier loaded library is used.
`-max`	Controls the choice of a wireload selection table only for the PVT max corner. Default: Selects the table for all PVT corners if neither the -min option nor the -`max` option is specified.
	Genus does not support this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	Genus issues SDC-201 error for this option.
`-min`	Controls the choice of a wireload selection table only for the minimum PVT corner. `Default`: Selects the table for all PVT corners if neither the -`min` option nor the -max option is specified.
	Genus does not support this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	Genus issues SDC-201 error for this option.
name	Specifies the wire-load selection table to use. Specify the hierarchical path to the wire-load selection table to be used, if multiple selection tables with same name exist.
`-object_list`	Specifies the object list on which the selection table applies.
	Genus does not support this option for optimization and reporting purposes. Physical flows will honor it as part of Innovus initial placement. The `write_sdc` command will write the option out for consumption by downstream tools.
	Genus issues SDC-201 error for this option.

Examples

- The following command selects `TLM` as the selection table used by the library:

```
genus:root:> set_wire_load_selection_group TLM
```

- The following command selects ASPEC1 as the selection table used by the sm333s library; only the minimum PVT corner is affected:

```
set_wire_load_selection_group -min -library sm333s ASPEC1
```

Related Information

Sets this attribute:	wireload_selection
Related attributes:	force_wireload
	wireload_mode

sizeof_collection

`sizeof_collection` *collection*

Description

Returns the total number of objects contained in the specified collection. If the collection contains different types of objects, `sizeof_collection` returns the sum of all the objects of different types in the collection.

The `sizeof_collection` command is an efficient mechanism for determining the number of objects in a collection.

Options and Arguments

collection	Specifies a list whose length needs to be returned.

Examples

- Following is a simple usage of the command:

```
genus:root:> sizeof_collection [get_cells *]
4
```

- The following example shows a simple way to find out how many objects matched a particular pattern and filter in a get_cells command.

```
genus:root:> puts "Number of hierarchical cells: [sizeof_collection [get_cells * -hier -filter            "is_hierarchical
== true"]]
Number of hierarchical cells: 71
```

- The following example shows what happens when the argument to sizeof_collection results in an empty collection:

```
genus:root:> set list [get_cells *]

genus:root:> set list1 [filter_collection $list "area < 0"]

genus:root:> puts "Cells with area < 0: [sizeof_collection $list1]"
Cells with area < 0: 0
```

sort_collection

```
sort_collection
{clock|design|inst|hinst|library|lib_arc|
lib_cell|lib_pin|hnet|pin|hpin|port}...
property [property]...
[-dictionary] [-descending] [-quiet]
```

Description

Sorts the collection based on the specified properties. You can specify any property associated with the object type.

Options and Arguments

{*clock*|*design*|*inst*|*hinst*|*library*|*lib_arc*|*lib_cell*|*lib_pin*|*hnet*|*pin*|*hpin*|*port*}	
	Defines the collection to be sorted. The collection can only contain one type of objects.
`-descending`	Specifies to sort the objects in a descending order.
	By default, the objects are listed in ascending order.
`-dictionary`	Specifies to sort the objects by dictionary order.
property	Specifies the property to use to order the objects.
	If you specify more than one property, the command sorts the collection in the order by which you specified the properties.
	To get a list of properties, refer to the filter options of the corresponding get command. For example, for a list of properties of instances, refer to the `get_cells` command.
`-quiet`	Suppresses all the generated messages.

Examples

- The following command sorts the list of cells based on area and orders the cell based on increasing area.

```
genus:root:> set list_cells [get_cells]
genus:root:> sort_collection $list_cells area

inst:m1/g127   inst:m1/g132   inst:m1/o_m1_clk1_0_reg_0
inst:m1/o_m1_clk1_0_reg_1   inst:m1/o_m1_clk1_0_reg_2   inst:m1/m2
```

- The following command sorts the list of cells based on name and area.

```
genus:root:> sort_collection $list_cells {name area}

inst:m1/g127   inst:m1/g132   hinst:m1/m2   inst:m1/o_m1_clk1_0_reg_0
inst:m1/o_m1_clk1_0_reg_1   inst:m1/o_m1_clk1_0_reg_2
```

- The following command sorts the list of pins in descending order.

```
genus:root:> set list [get_pins -filter " direction == out "]

genus:root:> sort_collection $list -descending inst
{pin:top/inst_q_reg[32]/Q}  {pin:top/inst_q_reg[31]/Q}
{pin:top/inst_q_reg[30]/Q}  {pin :top/inst_q_reg[29]/Q}
{pin:top/inst_q_reg[28]/Q}  {pin:top/inst_q_reg[27]/Q}
{pin:top/inst_q_reg[26]/Q}  {pin:top/inst_q_reg[25]/Q}
```

23

Unified Safety Format

Midas Unified Safety Format (USF) Physical is loaded at startup. To enable USF in Genus, you must set the `safety_midas_enable` attribute to `physical`.

```
set_db safety_midas_enable physical
```

You will see a message like this:

```
Cadence Midas USF - Physical
Copyright 2022 Cadence Design Systems, Inc. All rights reserved worldwide.
Cadence and the Cadence logo are registered trademarks of Cadence Design Systems Inc.
in the United States and other countries.
Midas USF - Physical Version: 25.03.001
1 physical
```

Once USF Physical is loaded, it is not possible to disable or reload it in the current session.

- USF File Commands
- USF Support Commands

USF File Commands

- apply_safety_mechanism
- create_esubpart
- create_failure_mode
- create_safety_mechanism
- create_subpart
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_ser
- set_safety_mechanism_rules_tmr

apply_safety_mechanism

```
apply_safety_mechanism
[-help]
safety_mechanism
[-generated]
-to failure_mode
```

Description

Maps a safety mechanism to one or more failure modes.

This command should be applied only via a USF file that is read in using `read_usf`.

Options and Arguments

`-help`	Prints a brief description of the `apply_safety_mechanism` command. For a detailed description of the command, use the man command: man `apply_safety_mechanism`.
`safety_mechanism`	Specifies the safety mechanism identifier. Data_type: string, required.
`-generated`	Specifies that the additional logic required to create this safety mechanism has already been inserted into the design. Data_type: bool, optional.
`-to` *`failure_mode`*	Specifies the failure mode identifier to which the specified safety mechanism should be associated. Note: One safety mechanism can be mapped to multiple failure modes. Data_type: string, required.

Example

The following command applies the `SM_TMR` safety mechanism to the `FM_22` failure mode:

```
apply_safety_mechanism SM_TMR -to FM_22
```

Related Information

Related commands:

- check_safety_mechanism
- create_safety_mechanism
- create_esubpart
- create_failure_mode
- read_usf
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_tmr

Related attributes:

safety Attributes

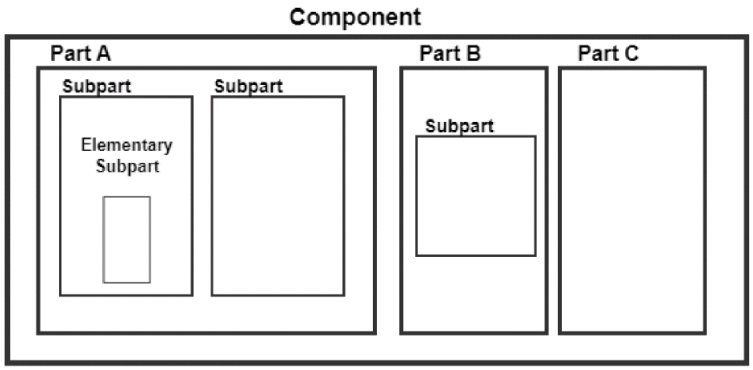
create_esubpart

```
create_esubpart
[-help]
esp
[-description description]
-inst string
-register_endpoint string
```

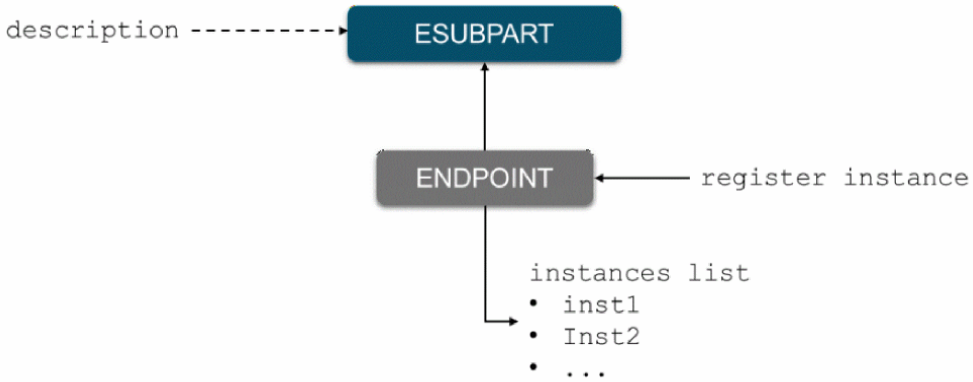
Description

Creates an elementary subpart, which is the smallest hardware subpart considered in safety analysis.

The diagram below represents a component with its parts, subparts, and elementary subparts.



An elementary subpart is a group of instances identified with a descriptor. The instances are associated to reference an endpoint. In the current implementation, the endpoint can only be a register.



The `create_esubpart` command should be applied only via a USF file that is read in using `read_usf`.

Options and Arguments

`-help`	Prints a brief description of the `create_esubpart` command. For a detailed description of the command, use the man command: man `create_esubpart`.
esp	Specifies the identifier to be associated with the elementary subpart being created. Data_type: string, required.
`-description` *description*	
	Specifies a textual description for the elementary subpart being created. Data_type: string, optional.
`-inst` *string*	Specifies the instances to be mapped to the elementary subpart. Data_type: string, required.
`-register_endpoint` *string*	
	Specifies a reference instance for the endpoint. Data_type: string, required.

Example

The following command creates an elementary subpart named with the specified description, endpoint, and instances:

```
create_esbupart ESP_VOT_0 -description "automatically generated elementary subpart"
    -register_endpoint {       xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\] }  -inst {
xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\
[5\]_tv_an1xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_an2
xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_or1
xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_or2
       }
```

Related Information

Related commands:

check_safety_mechanism
create_safety_mechanism
apply_safety_mechanism
create_failure_mode
read_usf

set_safety_mechanism_rules_dcls
set_safety_mechanism_rules_tmr

Related attributes:

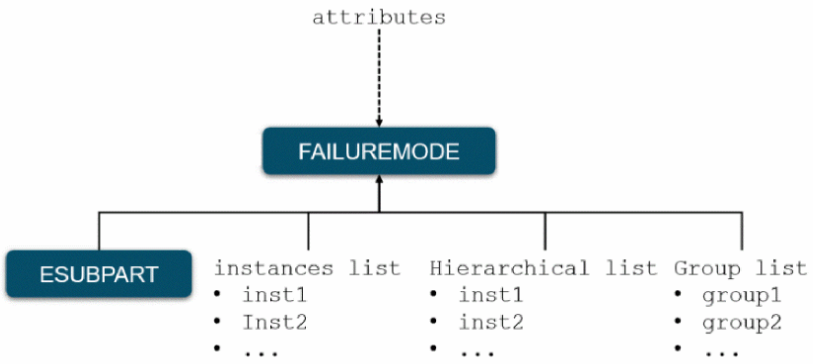
safety Attributes

create_failure_mode

```
create_failure_mode
[-help]
failuremode [-attributes string]
[-description description]
{-hinst string | -inst string | -esubpart string | -group string}
```

Description

Defines a failure mode. The failure mode groups a given set of logic that is relevant from the safety point of view. For example, it could be the logic that needs to be protected by one or more safety mechanisms or the logic that is describing a safety mechanism that has already been implemented.



The `create_failure_mode` command should be applied only via a USF file that is read in using `read_usf`.

Options and Arguments

`-help`	Prints a brief description of the `create_failure_mode` command. For a detailed description of the command, use the man command: man `create_failure_mode`.
`failuremode`	Specifies the identifier to be associated to the failure mode being created. Data_type: string, required.
`-attributes` *`string`*	
	Specifies the optional attributes attached to the failure mode. Data_type: string, optional.
`-description` *`description`*	
	Specifies a textual description for the failure mode being created. Data_type: string, optional.
`-esubpart` *`string`*	Specifies a list of elementary subparts. Data_type: string, required.
`-group` *`string`*	Specifies the groups to be mapped to the failure mode. Data_type: string, required.
`-hinst` *`string`*	Specifies the hierarchical instance(s) to be mapped to the failure mode. Data_type: string, required.
`-inst` *`string`*	Specifies the instances to be mapped to the failure mode. Data_type: string, required.

Example

The following command creates a failure mode named `FM_22` with the specified description and hierarchical instance:

```
create_failure_mode FM_22 -description "FPU Adder produces incorrect data due to fault into the FPU prenorm
addsubtracter"


-hinst xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/fpu_arith/fpu_prenorm_addsub
```

Related Information

Related commands:

check_safety_mechanism
create_safety_mechanism
apply_safety_mechanism
create_esubpart

read_usf
set_safety_mechanism_rules_dcls
set_safety_mechanism_rules_tmr

Related attributes:

safety Attributes

create_safety_mechanism

```
create_safety_mechanism
[-help]
safety_mechanism
[-class {aou hw sw aou_hw aou_sw}]
[-description description]
[-type {tmr dcls}]
```

Description

Defines a new safety mechanism. A safety mechanism is extra hardware inserted into your design to bypass, detect, or correct faults.

> The `create_safety_mechanism` command should be applied only via a USF file that is read in using `read_usf`.

Options and Arguments

`-help`	Prints a brief description of the `create_safety_mechanism` command. For a detailed description of the command, use the man command: man `create_safety_mechanism`.
`safety_mechanism`	Specifies a name for the safety mechanism being defined. Data_type: string, required.
`-class {aou hw sw aou_hw aou_sw}`	
	Specifies the supported safety mechanism classes. The possible values are: `aou`: assumption of use `hw`: hardware `sw`: software `aou_hw`: assumption of use hardware `aou_sw`: assumption of use software Default: `hw` Data_type: enum, optional
`-description` *`description`*	
	Specifies a description for the safety mechanism being defined. Data_type: string, optional
`-type {tmr dcls}`	Specifies the type of the safety mechanism being defined. The options are:`tmr` - triple modular redundancy `dcls` - dual-core lockstep Default: tmr Data_type: enum, optional

Example

The following command creates a safety mechanism named, SM_TMR, of the specified class and type:

```
create_safety_mechanism SM_TMR -description "Triple Modular Redundant Flip Flops" -type tmr -class hw
```

Related Information

Related commands:

check_safety_mechanism
create_failure_mode
apply_safety_mechanism
create_esubpart
read_usf

set_safety_mechanism_rules_dcls
set_safety_mechanism_rules_tmr

Related attributes:

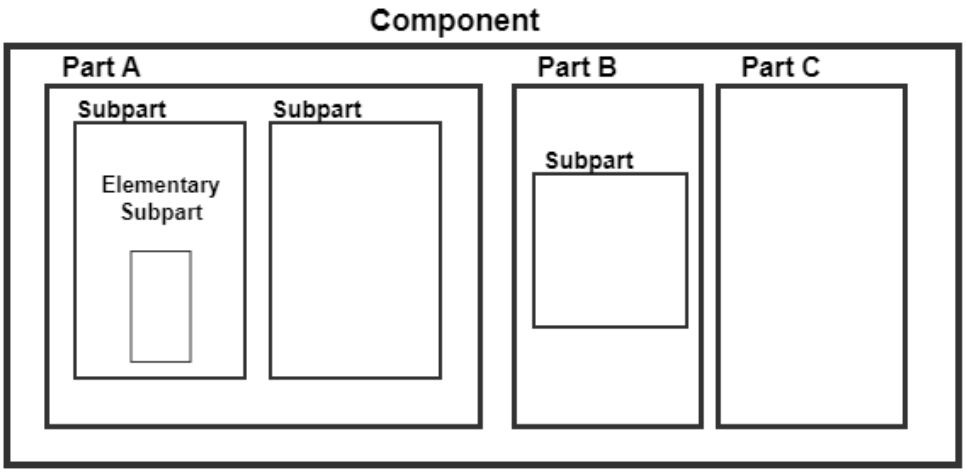
safety Attributes

create_subpart

```
create_subpart
[-help]
[-description string]
[sp]
[-hinst string]
[-physical_group]
```

Creates a subpart, which is used to model a Dual Core Lockstep (DCLS) group in safety analysis.

The diagram below represents a component with its parts, subparts, and elementary subparts.



The `create_subpart` command should be applied only via a USF file that is read in using `read_usf`.

Given below is an excerpt from a USF file with commands for creating subparts named `primary`, `shadow`, and `compare` and then defining failure modes for those subparts. DCLS rules can then be then be defined for these subparts:

```
##############################

## USF.
```

```
##############################
create_subpart primary   -hinst {hinstA1 hinstA2} -physical_group ;
create_subpart shadow   -hinst {hinstB1 hinstB2} -physical_group ;
create_subpart compare -hinst {hinstC1 hinstC2} -physical_group ;
create_failure_mode FM_mission -subpart {primary} ;
create_failure_mode FM_passive -subpart {secondary compare} ;
create_safety_mechanism SM -desc "Dual Core Lock Step" -type dcls -class hw ;
apply_safety_mechanism  SM -to {FM_mission FM_passive} -generated ;
set_safety_mechanism_rules_dcls \
  -exclusive_groups { {primary shadow compare} } \
  -spacing 10 \
  -isolate_clock { {primary split_network} {shadow split_network} {compare split_network} }
```

Parameters

`-help`	Prints a brief description of the `create_subpart` command. For a detailed description of the command, use the `man` command: `man create_subpart`.
`-description`	Specifies the subpart description. *Data_type*: string, optional
sp	Specifies the identifier or name to be associated with the subpart being created. *Data_type*: string, required
`-hinst` *string*	Specifies the hierarchical instances (hinst) to be mapped to the subpart. *Data_type*: string, required
`-physical_group`	Creates a physical group associated with the subpart, provided that a group with the same name as the subpart does not exist in the design. If a group with the same name as the subpart exists already, the tool maintains the existing group and boundary and checks whether the existing group has the same members as those specified in the USF file. If not, it update the group members to those defined in the USF and displays the following warning: `WARNING: the members of group "sp" are different to those defined for subpart "sp" in the USF. Updating the group members to match the USF.` *Data_type*: bool, optional

Example

- The following command creates a subpart named `primary` and a physical group associated with it. The specified hinsts are mapped to this subpart:
  ```
  create_subpart primary -hinst {hinstA1 hinstA2} -physical_group
  ```

Related Information

Related commands:

check_safety_mechanism
create_safety_mechanism

create_esubpart
create_failure_mode
read_usf
set_safety_mechanism_rules_dcls
set_safety_mechanism_rules_tmr

Related attributes:

safety Attributes

set_safety_mechanism_rules_dcls

```
set_safety_mechanism_rules_dcls
[-help]{[-exclusive_groups {{group_A1 {group_A2 group_A3 ...}} {group_B1 group_B2 group_B3 ...} ... }][-spacing
d][-isolate_clock {* | group_A1 group_A2 ...}][-isolate_reset string][-isolate_scan_enable string][-isolate_pin
string][-isolate_inputs string][-isolate_outputs string][-isolate_use_halo object][-isolate_halo_type enum {hard
soft}][-route_types enum {internal interface common top}][-isolate_type enum {hard soft}]
```

Description

Defines the separation rules to be applied to the dual core lockstep (DCLS) groups in the design.

Each time the `set_safety_mechansim_rules_dcls` command is issued, the tool clears any previously set DCLS-associated attributes across all groups prior to setting them in the way the latest command dictates. Therefore, this command cannot be used multiple times to accumulate a set of rules.

This `set_safety_mechansim_rules_dcls` command may be interactively applied at the command line. That is, it is not restricted to be read in as part of a USF file using `read_usf`. However, if the command is not in a USF file that is being read, it must be applied only after DCLS-related USF information has been read because it depends on this information to function correctly.

Attempting to use this command before a design is loaded or before the groups listed in arguments are defined will cause an error to be issued. In addition, any groups mentioned in this command must already be defined as DCLS groups using a combination of the `create_failure_mode`, `create_safety_mechanism` (with type `dcls`), and `apply_safety_mechanism` commands.

Options and Arguments

`-help`	Prints a brief description of the `set_safety_mechanism_rules_dcls` command. For a detailed description of the command, use the man command: man `set_safety_mechanism_rules_dcls`.

`-exclusive_groups {{group_A1 {group_A2 group_A3 ...}} {group_B1 group_B2 group_B3 ...} ... }`

Specifies a list of mutually exclusive DCLS group lists. This parameter is mandatory if the `-spacing` parameter is being used. Wildcards are not allowed in the `-exclusive_groups` arguments. If a group is referenced more than once in the `-exclusive_groups` arguments, an error will be issued.

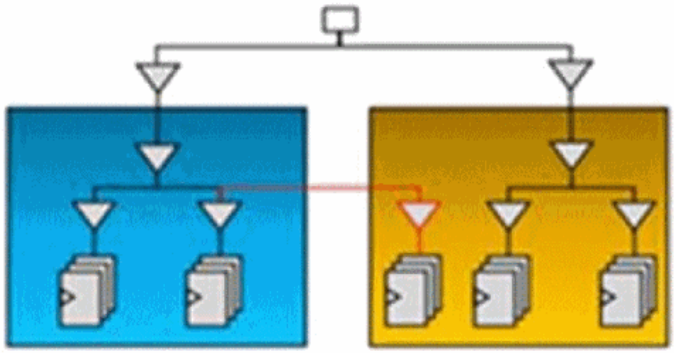
In the argument example above, {*group_A1* {*group_A2 group_A3 ...*} is a master/checker arrangement, where *group_A1* is the "master" group and *group_A2* and *group_A3* are the "checker" groups. This means that the checker groups must be separated from the master group by a distance of at least d as specified with the -spacing parameter. However, a checker group may overlap other checker groups. When a master/checker arrangement is specified, there can be only one master and there must be at least two checkers.

In the {*group_B1 group_B2 group_B3 ...*} part of the argument, *group_B1*, *group_B2*, and *group_B3* are all mutually exclusive. This means that they must all be separated from one another by a distance of at least d specified with the -spacing parameter. Each list of mutually exclusive groups must have at least two members or else an error will be issued.

`-isolate_clock {* | group_A1 group_A2 ...}`

Specifies a list of groups where clock isolation is required. This argument is optional. Clock isolation is specific to each listed group. If specified for a group, it isolates that group from the rest of the clock network in the design.

Clock isolation is not just about separating two groups. For example, in the diagram below, the path shown in red will not occur for the groups with isolated clocking.



- If * is specified, clock isolation is enacted for all the DCLS groups in the design.

`-spacing d`

	Defines in microns the minimum spacing between exclusive DCLS groups. This parameter is mandatory if the `-exclusive_groups` parameter is used. The distance `d` here is expected to be a positive rational number. The specified distance is rounded to the nearest 0.001 value. If the distance specified is a negative number, a warning is issued and the setting is changed to 0.
`-isolate_reset` *string*	
	Specifies a list of groups where reset isolation is required (string +, required).
`isolate_scan_enable` *string*	
	Specifies a list of groups where `scan_enable` isolation is required (string +, required).
`isolate_pin` *string*	
	Specifies a list of hpins where isolation is required (string +, required).
`isolate_inputs` *string*	
	Specifies a list of groups where isolation of inputs is required. (string +, required)
`isolate_outputs` *string*	
	Specifies a list of groups where isolation of outputs is required. (string +, required)
`isolate_use_halo` *object*	
	Specifies a list of groups where isolation halo buffers are required. (object group +, required).
`isolate_halo_type` *enum* `{hard soft}`	
	Specifies the isolation halo type for dcls groups. hard: Specifies that the isolation cells on a group's input/output nets should always be placed inside the group's isolation halo boundary. soft: Same as hard, but the isolation cells can be placed inside the group boundary if timing needs to be improved. (enum hard soft), optional)
`route_types enum {internal interface common top}`	
	Specifies the type of nets on which the routing rules for dcls groups are enforced. (enum internal interface common top +), optional)
`isolate_type` *enum* `{hard soft}`	

	Specifies the isolation type. A setting of hard forces isolated networks do avoid exclusive groups. (enum hard soft), optional)

Examples

- The following command specifies that groups A and B are mutually exclusive to one another. The group boundaries must be at least 5 microns apart:

  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {A B} } -spacing 5
  ```

- The following command specifies that groups A, B, and C are mutually exclusive. These groups must be at least 10 microns apart:

  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {A B C} } -spacing 10
  ```

- The following command specifies that group W must be at least 5 microns from groups X, Y, and Z. However, X, Y, and Z may overlap one another:

  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {W {X Y Z}} } -spacing 5
  ```

- The following command specifies that groups A, B, and C must be at least 5 microns apart from one another and group W must be at least 5 microns from groups X, Y, and Z. However, X, Y, and Z may overlap one another. In addition, groups A, B,C, W, X, Y, and Z are to have isolated clocks:

  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {A B C} {W {X Y Z}} } -spacing 5 -isolate_clock
  { * }
  ```

Related Information

Related commands:

check_safety_mechanism
create_safety_mechanism
create_esubpart
create_failure_mode
read_usf
set_safety_mechanism_rules_tmr
apply_safety_mechanism

Related Attributes:

safety Attributes

set_safety_mechanism_rules_ser

```
set_safety_mechanism_rules_ser
[-help]
-cells string
[-type {hard soft}]
```

Defines the Soft Error Resilience (SER) safety-mechanism rules that need to be enforced during implementation. The SER flow allows the user to control the placing of cells in Genus so that the resilient flip-flops are selected where possible.

As resilient flip-flops generally have poorer power, performance and area (PPA) aspects than normal standard cells, Genus uses them only when directed via the SER flow.

The `set_safety_mechanism_rules_ser` command can change the `safety_ser_cells` attribute at root level or power_domain level. It may also change the `dont_use` or `dont_touch` attributes of some library or base cells. Each time the `set_safety_mechanism_rules_ser` command is issued, it clears any previous setting of the `safety_ser_cells` attribute. Therefore, this command cannot be used multiple times to accumulate a set of rules.

The `set_safety_mechanism_rules_ser` command must be used in a USF file that is read in using `read_usf`. The `set_safety_mechanism_rules_ser` command is not intended for interactive use. An error will occur if the `set_safety_mechanism_rules_ser` command is executed before the design libraries have been read.

Parameters

`-help`	Prints a brief description of the `set_safety_mechanism_rules_ser` command. For a detailed description of the command, use the `man` command: `man set_safety_mechanism_rules_ser`.
`-cells string`	Specifies the list of base_cells allowed for the insts with an SER safety mechanism. Supported values are: • {*cells*}: A single list of base_cell names. This format may be used for designs without power domains. It may also be used where the same set of cells is used across all power_domains in the design. It is permissible to use the "*" wildcard when listing the cells. **Example:** `set_safety_mechanism_rules_ser -cells { *lnw6_auto* }` • { *{PD1 {cells}} {{PD2 PD3} {cells}}* }: A list of cells for each power_domain. To be used for designs with multiple power domains.
`-type {hard soft}`	
	Specifies the SER type. Supported values are: • `hard`: Always uses ser cells for the insts with an SER safety mechanism. • `soft`: Allows a trade-off between timing and SER cell selection. Uses ser cells for the insts with an SER safety mechanism, except when using a non-ser cell for such insts can improve timing. *Default*: `hard`

Example

- The following command specifies that the given list of cells should be allowed for the insts with the SER safety mechanism:

```
set_safety_mechanism_rules_ser -cells { m0hd_lnw6_auto_sdfarq1t01p00 m0hd_lnw6_auto_sdfarqhs1t02p00
m0hd_lnw6_auto_sdfarsvq1t01p00 m0hd_lnw6_auto_sdfasq1t01p00 m0hd_lnw6_auto_sdfq1t01p00
m0hd_lnw6_auto_sdfqhs1t02p00 m0hd_lnw6_auto_sdfqhsc1t02p00 m0hd_lnw6_auto_sdfqlss1t01p00
m0hd_lnw6_auto_sdfqx1t01p00 t0hd_lnw6_auto_sdf3t01p00 t0hd_lnw6_auto_sdf4t01p00
t0hd_lnw6_auto_sdfarhs4t02p00 t0hd_lnw6_auto_sdfarlcp2t01p00 t0hd_lnw6_auto_sdfarlcp8t01p00
t0hd_lnw6_auto_sdfhs2t02p00 t0hd_lnw6_auto_sdflcp2t01p00 t0hd_lnw6_auto_sdflcp8t01p00 }
```

- The following command uses the * wildcard to specify the cells for the insts with the SER safety mechanism:

```
set_safety_mechanism_rules_ser -cells { *lnw6_auto* }
```

Related Information

Related commands:	check_safety_mechanism
	create_safety_mechanism
	create_esubpart
	create_failure_mode
	read_usf
	set_safety_mechanism_rules_tmr
	apply_safety_mechanism
	set_safety_mechanism_rules_dcls
Related attributes:	safety Attributes

set_safety_mechanism_rules_tmr

```
set_safety_mechanism_rules_tmr
[-help]
{[-spacing double] [-spacing_x dx] [-spacing_y dy] [-isolate_clock {none unique_driver}] }
[-spacing double | -spacing_x dx]
[-spacing double | -spacing_y dy]
```

Description

Defines the triple modular redundancy (TMR) safety-mechanism rules that need to be enforced during implementation.

The rules concern the physical spacing between TMR flip-flops and clock isolation of TMR flip-flops. The `set_safety_mechanism_rules_tmr` command may be applied interactively at the command line. That is, it is not restricted to be read in as part of a USF file using `read_usf`.

Each time the `set_safety_mechansim_rules_tmr` command is issued, the tool first clears any previously set TMR rules. This means that the `set_safety_mechansim_rules_tmr` command cannot be used multiple times to accumulate a set of rules.

One of the following parameters must be specified with the command:

- `-spacing`
- `-spacing_x` and/or `-spacing_y`
- `-isolate_clock`

Additionally, the `-spacing` parameter is mutually exclusive to the `-spacing_x` and `-spacing_y` parameters.

Options and Arguments

`-help`	Prints a brief description of the `set_safety_mechanism_rules_tmr` command. For a detailed description of the command, use the man command: man `set_safety_mechanism_rules_tmr`.
`-isolate_clock {none unique_driver}`	
	Controls whether or not the three flops in a TMR group may share the same leaf clock buffer. Supported values are: none - Specifies that the three flops in a TMR group may also share the same leaf clock buffer. unique_driver - Specifies that the clock pin of every TMR flop should be directly driven by its own unique clock cell with fanout equal to 1. Default: none Data_type: enum, required.
`-spacing double`	Specifies the minimum radial spacing in µm between a group of TMR flip-flops sharing the same parent. This parameter is mutually exclusive to the `-spacing_x` and `-spacing_y` parameters. The spacing is expected to be a positive rational number. The specified value is rounded to the nearest 0.001. If a negative number is given, a warning is issued and the spacing value is set as 0. Data_type: double, required
`-spacing_x dx`	Specifies the minimum spacing in the X dimension in µm between a group of TMR flip-flops sharing the same parent. This parameter is mutually exclusive to the -spacing parameter. The specified value is rounded to the nearest 0.001. If a negative number is given, a warning is issued and the `-spacing_x` value is set as 0. Data_type: double, required
`-spacing_y dy`	Specifies the minimum spacing in the Y dimension in µm between a group of TMR flip-flops sharing the same parent. This parameter is mutually exclusive to the -spacing parameter. The specified value is rounded to the nearest 0.001. If a negative number is given, a warning is issued and the `-spacing_y` value is set as 0. Data_type: double, required

Example

The following command specifies a 5µm radial spacing between TMR flip-flops and requires isolation of clock pins.

```
set_safety_mechansim_rules_tmr -spacing 5 -isolate_clock unique_driver
```

Related Information

Related commands:

check_safety_mechanism
create_safety_mechanism
create_esubpart
create_failure_mode
read_usf
set_safety_mechanism_rules_dcls
apply_safety_mechanism

Related Attributes:

safety Attributes

USF Support Commands

- check_safety_mechanism
- read_safety_tmr
- read_usf
- reset_usf
- usf_version
- write_usf

check_safety_mechanism

```
check_safety_mechanism
[-help]
[-out_file file_name][-detailed][-type {tmr dcls ser parity dmr}][-exclude_rules file_ex][-flow_phase {prects cts
route}]
```

Description

Checks all the safety mechanisms and prints a report, optionally writing it to the specified file as well.

The report contains a list of:

- All the safety mechanisms
- The safety mechanism violations found for each safety mechanism, grouped by type

See Report Content for details.

Options and Arguments

`-help`	Prints a brief description of the `check_safety_mechanism` command. For a detailed description of the command, use the man command: man `check_safety_mechanism`.
`-out_file` *`file_name`*	
	Writes the report to the specified file. If omitted, no report file is generated. Data_type: string, optional.
`-detailed`	Triggers detailed reporting of the instances in the report file.
`-type {tmr dcls ser parity dmr}`	Specifies the safety mechanism type (enum tmr dcls ser parity dmr).
`-exclude_rules` *`file_ex`*	Specifies to read the list of checks to exclude from the specified file.
`-flow_phase {prects cts route}`	Executes related checks based on the stage specified by `flow_phase`.

Report Content

The report contains one section per safety mechanism, listing all violations (if any) of that safety mechanism, grouped by type.

Safety mechanisms of type `dcls` (dual-core lockstep) have the following violation types:

- `dcls` group overlap violations: Two exclusive `dcls` groups overlap.
- `dcls` group spacing violations: Two exclusive `dcls` groups are closer than permitted.
- `dcls` inst placement violations: An inst belonging to a `dcls` group is placed outside of that group's boundary.
- `dcls` internal net routing violations: A net connected only to the `insts` inside a single `dcls` group is routed outside that group's boundary.
- `dcls` interface net routing violations: A net connected to `insts` both inside and outside a `dcls` group is routed inside the boundary of an exclusive group.
- `dcls` clock isolation violations: A clock net connected to a `dcls` group is driving loads outside that group.

Safety mechanisms of type `tmr` (triple modular redundancy) have the following violation types:

- `tmr` spacing violations: Two `tmr insts` from the same safety mechanism are placed closer than permitted.
- `tmr` clock isolation violations: A clock net driving a `tmr` inst is also driving other `insts`.

Example

- The following command checks all the safety mechanisms and writes the report to the safety_report.rpt file:

  ```
  check_safety_mechanism -out_file safety_report.rpt
  ```

Related Information

Related commands:

read_safety_tmr
read_usf
reset_usf
usf_version
write_usf

Related Attributes:

safety Attributes

read_safety_tmr

```
read_safety_tmr

[-help]
tmr_info_file -failure_mode failure_mode_name -safety_mechanism safety_mechanism_name
```

Description

Reads the triple modular redundancy (TMR) information file. For designs with TMR logic already inserted, either in the register-transfer level (RTL) or with third party tools, this command identifies the existing TMR logic and assimilates it into the Unified Safety Format (USF).

You can issue the `read_safety_tmr` command multiple times to build up information in cases where there are multiple different safety mechanism names or failure mode names. However, this may be done only for new safety mechanism and failure mode names. You cannot accumulate TMR information by reusing a safety mechanism or failure mode name that has been already read in with the `read_safety_tmr` command.

Options and Arguments

`-help`	Prints a brief description of the `read_safety_tmr` command. For a detailed description of the command, use the man command: man `read_safety_tmr`.
`tmr_info_file`	Specifies the name of the TMR information file. This is a text file listing each flip-flop and its accompanying cloned flip-flops where TMR has already been applied. Optionally, the file may also list the names of the voter cells that are used. The TMR information file has the following format: <parent_FF1> <clone_FF1A> <clone_FF1B> <voter1A> <voter1B> ... <parent_FF2> <clone_FF2A> <clone_FF2B> <voter2A> <voter2B> ... ... The instance names may be supplied with the full path including the design name at the root. If the design name is not given at the root of the instance path for a given instance, that instance is assumed to be in the current design. The instance names are separated by white space. Comments are allowed in this file and must be preceded by #. Blank lines are also allowed. Data_type: string, required
`-failure_mode` *`failure_mode_name`*	
	Specifies the name of the failure mode for parent flops. The failure mode for clone flops will be <*`failure_mode_name`*>`_clones`. The failure mode for voter cells will be <*`failure_mode_name`*>`_voters`. Data_type: string, required
`-safety_mechanism` *`safety_mechanism_name`*	
	Specifies the name of the safety mechanism for the TMR logic. Data_type: string, required.

Related Information

Related commands:

- check_safety_mechanism
- read_usf
- reset_usf
- usf_version
- write_usf

Related Attributes:

safety Attributes

read_usf

```
read_usf
[-help]
USF_file
```

Description

Reads in a Unified Safety Format (USF) file specifying the safety intent of your design.

Options and Arguments

`-help`	Prints a brief description of the `read_usf` command. For a detailed description of the command, use the man command: man `read_usf`.
`USF_file`	Specifies the name of the USF file to be read. Data_type: string, required.

Example

- The following command reads in the USF file named myFile.usf:

  ```
  read_usf myFile.usf
  ```

Related Information

Related commands:

check_safety_mechanism
read_safety_tmr
reset_usf
usf_version
write_usf

Related Attributes:

safety Attributes

reset_usf

```
reset_usf
[-help]
```

Description

Resets the Unified Safety Format (USF) Physical safety information and related database attributes completely.

Options and Arguments

`-help`	Prints a brief description of the `reset_usf` command. For a detailed description of the command, use the man command: man `reset_usf`.

Related Information

Related commands:

- check_safety_mechanism
- read_safety_tmr
- read_usf
- usf_version
- write_usf

Related Attributes:

safety Attributes

usf_version

```
usf_version
[-help]
```

Description

Retrieves the version of the USF Physical being used to interpret the USF commands.

Options and Arguments

`-help`	Prints a brief description of the `usf_version` command. For a detailed description of the command, use the man command: man `usf_version`.

Related Information

Related commands:

- check_safety_mechanism
- read_safety_tmr
- read_usf
- write_usf
- reset_usf

Related Attributes:

safety Attributes

write_usf

```
write_usf
[-help]
USF_file
```

Description

Writes the safety intent of your design to the specified Unified Safety Format (USF) file.

Options and Arguments

`-help`	Prints a brief description of the `write_usf` command. For a detailed description of the command, use the man command: man `write_usf`.
`USF_file`	Specifies the name of the USF file to be saved. Data_type: string, required

Example

- The following command saves the USF information in the `myFile.usf` file:

```
write_usf myFile.usf
```

Related Information

Related commands:

check_safety_mechanism
read_safety_tmr
read_usf
usf_version
reset_usf

Related attributes:

safety Attributes

24

Command Name Mapping

This topic lists the Genus legacy UI equivalent commands for the stylus common UI commands.

Stylus Common UI	Legacy UI
add_assign_buffer_options	set_remove_assign_options
add_clock_gates_obs	clock_gating insert_obs
add_core_wrapper_cell	insert_dft wrapper_cell
add_hard_repair	insert_dft hard_repair
add_jtag_boundary_scan	insert_dft boundary_scan
add_jtag_macro	insert_dft jtag_macro
add_lbist	insert_dft logic_bist
add_lockup_element	insert_dft lockup_element
add_opcg	insert_dft opcg
add_pmbist	insert_dft pmbist
add_test_compression	insert_dft compression_logic
analyze_atpg_testability	analyze_testability
check_power_structure	verify_power_structure
connect_serial_scan_chains	concat_scan_chains
convert_to_opcg_scan	replace_opcg_scan
convert_to_scan	replace_scan
create_clock	define_clock

create_component	hdl_create component
create_component_parameter	hdl_create parameter
create_derived_design	derive_environment
create_floorplan	specify_floorplan
create_place_blockage	create_placement_blockage
create_place_halo	create_placement_halo_blockage
create_route_blockage	create_routing_blockage
create_route_halo	create_routing_halo_blockage
define_dft_cfg_mode	define_dft dft_configuration_mode
define_jtag_boundary_scan_segment	define_dft boundary_scan_segment
define_jtag_instruction	define_dft jtag_instruction
define_jtag_instruction_register	define_dft jtag_instruction_register
define_jtag_macro	define_dft jtag_macro
define_jtag_tap_port	define_dft tap_port
define_mbist_clock	define_dft mbist_clock
define_opcg_domain	define_dft opcg_domain
define_opcg_domain_macro_parameters	define_dft domain_macro_parameters
define_opcg_mode	define_dft opcg_mode
define_opcg_osc_source	define_dft osc_source
define_opcg_trigger	define_dft opcg_trigger
define_pmbist_direct_access	define_dft pmbist_direct_access
define_scan_abstract_segment	define_dft abstract_segment
define_scan_chain	define_dft scan_chain
define_scan_fixed_segment	define_dft fixed_segment

define_scan_floating_segment	define_dft floating_segment
define_scan_preserved_segment	define_dft preserved_segment
define_shift_register_segment	define_dft shift_register_segment
define_test_bus_port	define_dft test_bus_port
define_test_clock	define_dft test_clock
fix_pad_cfg	configure_pad_dft
identify_shift_register_scan_segments	dft_auto_identify_shift_register
read_dft_jtag_boundary_file	read_io_speclist
read_metric	read_metric_config
read_pmbist_memory_view	read_memory_view
remove_assigns_without_opt	remove_assigns_without_optimization
report_area	report area
report_cell_delay_calculation	report cell_delay_calculation
report_clocks	report clocks
report_design_rules	report design_rules
report_dft_violations	report dft_violations
report_dp	report datapath
report_gates	report gates
report_power	report power
report_opcg_clock_domain_info	report dft_clock_domain_info
report_qor	report qor
report_scan_chains	report dft_chains
report_scan_registers	report dft_registers
report_scan_setup	report dft_setup

report_summary	summary_table
report_timing	report timing
run_embedded_script	exec_embedded_script
set_path_specification	specify_paths
update_names	change_names
write_clp_script	write_do_clp
write_dft_atpg	write_et_atpg
write_dft_atpg_other_vendor_files	write_atpg
write_dft_bsdl	write_bsdl
write_dft_lbist_testbench	write_et_lbist
write_reports	generate_reports